

London Mathematical Society
Lecture Note Series 498

Polynomial Functors

A Mathematical Theory of Interaction

Nelson Niu and David L. Spivak



LONDON
MATHEMATICAL
SOCIETY
founded 1865

CAMBRIDGE

LONDON MATHEMATICAL SOCIETY LECTURE NOTE SERIES

Managing Editor: Professor Endre Süli, Mathematical Institute, University of Oxford,
Woodstock Road, Oxford OX2 6GG, United Kingdom

The titles below are available from booksellers, or from Cambridge University Press at
www.cambridge.org/mathematics

- 388 Groups St Andrews 2009 in Bath II, C.M. CAMPBELL *et al* (eds)
- 389 Random fields on the sphere, D. MARINUCCI & G. PECCATI
- 390 Localization in periodic potentials, D.E. PELINOVSKY
- 391 Fusion systems in algebra and topology, M. ASCHBACHER, R. KESSAR & B. OLIVER
- 392 Surveys in combinatorics 2011, R. CHAPMAN (ed)
- 393 Non-abelian fundamental groups and Iwasawa theory, J. COATES *et al* (eds)
- 394 Variational problems in differential geometry, R. BIELAWSKI, K. HOUSTON & M. SPEIGHT (eds)
- 395 How groups grow, A. MANN
- 396 Arithmetic differential operators over the p -adic integers, C.C. RALPH & S.R. SIMANCA
- 397 Hyperbolic geometry and applications in quantum chaos and cosmology, J. BOLTE & F. STEINER (eds)
- 398 Mathematical models in contact mechanics, M. SOFONEA & A. MATEI
- 399 Circuit double cover of graphs, C.-Q. ZHANG
- 400 Dense sphere packings: a blueprint for formal proofs, T. HALES
- 401 A double Hall algebra approach to affine quantum Schur–Weyl theory, B. DENG, J. DU & Q. FU
- 402 Mathematical aspects of fluid mechanics, J.C. ROBINSON, J.L. RODRIGO & W. SADOWSKI (eds)
- 403 Foundations of computational mathematics, Budapest 2011, F. CUCKER, T. KRICK, A. PINKUS
& A. SZANTO (eds)
- 404 Operator methods for boundary value problems, S. HASSI, H.S.V. DE SNOO & F.H. SZAFRANIEC (eds)
- 405 Torsors, étale homotopy and applications to rational points, A.N. SKOROBOGATOV (ed)
- 406 Appalachian set theory, J. CUMMINGS & E. SCHIMMERLING (eds)
- 407 The maximal subgroups of the low-dimensional finite classical groups, J.N. BRAY, D.F. HOLT
& C.M. RONEY-DOUGAL
- 408 Complexity science: the Warwick master's course, R. BALL, V. KOLOKOLTSOV & R.S. MACKAY (eds)
- 409 Surveys in combinatorics 2013, S.R. BLACKBURN, S. GERKE & M. WILDON (eds)
- 410 Representation theory and harmonic analysis of wreath products of finite groups, T.
CECCHERINI-SILBERSTEIN, F. SCARABOTTI & F. TOLLI
- 411 Moduli spaces, L. BRAMBILA-PAZ, O. GARCÍA-PRADA, P. NEWSTEAD & R.P. THOMAS (eds)
- 412 Automorphisms and equivalence relations in topological dynamics, D.B. ELLIS & R. ELLIS
- 413 Optimal transportation, Y. OLLIVIER, H. PAJOT & C. VILLANI (eds)
- 414 Automorphic forms and Galois representations I, F. DIAMOND, P.L. KASSAEI & M. KIM (eds)
- 415 Automorphic forms and Galois representations II, F. DIAMOND, P.L. KASSAEI & M. KIM (eds)
- 416 Reversibility in dynamics and group theory, A.G. O'FARRELL & I. SHORT
- 417 Recent advances in algebraic geometry, C.D. HACON, M. MUSTĂŢĂ & M. POPA (eds)
- 418 The Bloch–Kato conjecture for the Riemann zeta function, J. COATES, A. RAGHURAM, A. SAIKIA &
R. SUJATHA (eds)
- 419 The Cauchy problem for non-Lipschitz semi-linear parabolic partial differential equations, J.C. MEYER &
D.J. NEEDHAM
- 420 Arithmetic and geometry, L. DIEULEFAIT *et al* (eds)
- 421 O-minimality and Diophantine geometry, G.O. JONES & A.J. WILKIE (eds)
- 422 Groups St Andrews 2013, C.M. CAMPBELL *et al* (eds)
- 423 Inequalities for graph eigenvalues, Z. STANIĆ
- 424 Surveys in combinatorics 2015, A. CZUMAJ *et al* (eds)
- 425 Geometry, topology and dynamics in negative curvature, C.S. ARAVINDA, F.T. FARRELL & J.-F.
LAFONT (eds)
- 426 Lectures on the theory of water waves, T. BRIDGES, M. GROVES & D. NICHOLLS (eds)
- 427 Recent advances in Hodge theory, M. KERR & G. PEARLSTEIN (eds)
- 428 Geometry in a Fréchet context, C.T.J. DODSON, G. GALANIS & E. VASSILIOU
- 429 Sheaves and functions modulo p , L. Taelman
- 430 Recent progress in the theory of the Euler and Navier–Stokes equations, J.C. ROBINSON, J.L. RODRIGO,
W. SADOWSKI & A. VIDAL-LÓPEZ (eds)
- 431 Harmonic and subharmonic function theory on the real hyperbolic ball, M. STOLL
- 432 Topics in graph automorphisms and reconstruction (2nd Edition), J. LAURI & R. SCAPELLATO
- 433 Regular and irregular holonomic D-modules, M. KASHIWARA & P. SCHAPIRA
- 434 Analytic semigroups and semilinear initial boundary value problems (2nd Edition), K. TAIRA
- 435 Graded rings and graded Grothendieck groups, R. HAZRAT
- 436 Groups, graphs and random walks, T. CECCHERINI-SILBERSTEIN, M. SALVATORI & E. SAVA-HUSS (eds)
- 437 Dynamics and analytic number theory, D. BADZIAHIN, A. GORODNIK & N. PEYERIMHOFF (eds)

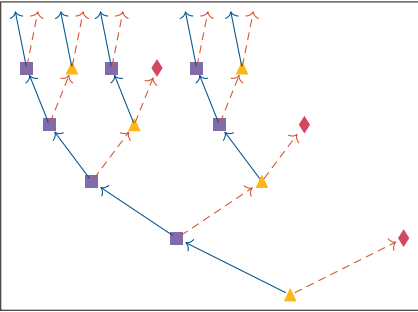
438 Random walks and heat kernels on graphs, M.T. BARLOW
 439 Evolution equations, K. AMMARI & S. GERBI (eds)
 440 Surveys in combinatorics 2017, A. CLAEISSON *et al* (eds)
 441 Polynomials and the mod 2 Steenrod algebra I, G. WALKER & R.M.W. WOOD
 442 Polynomials and the mod 2 Steenrod algebra II, G. WALKER & R.M.W. WOOD
 443 Asymptotic analysis in general relativity, T. DAUDÉ, D. HÄFNER & J.-P. NICOLAS (eds)
 444 Geometric and cohomological group theory, P.H. KROPHOLLER, I.J. LEARY, C. MARTÍNEZ-PÉREZ & B.E.A. NUCINKIS (eds)
 445 Introduction to hidden semi-Markov models, J. VAN DER HOEK & R.J. ELLIOTT
 446 Advances in two-dimensional homotopy and combinatorial group theory, W. METZLER & S. ROSEBROCK (eds)
 447 New directions in locally compact groups, P.-E. CAPRACE & N. MONOD (eds)
 448 Synthetic differential topology, M.C. BUNGE, F. GAGO & A.M. SAN LUIS
 449 Permutation groups and cartesian decompositions, C.E. PRAEGER & C. SCHNEIDER
 450 Partial differential equations arising from physics and geometry, M. BEN AYED *et al* (eds)
 451 Topological methods in group theory, N. BROADDUS, M. DAVIS, J.-F. LAFONT & I. ORTIZ (eds)
 452 Partial differential equations in fluid mechanics, C.L. FEFFERMAN, J.C. ROBINSON & J.L. RODRIGO (eds)
 453 Stochastic stability of differential equations in abstract spaces, K. LIU
 454 Beyond hyperbolicity, M. HAGEN, R. WEBB & H. WILTON (eds)
 455 Groups St Andrews 2017 in Birmingham, C.M. CAMPBELL *et al* (eds)
 456 Surveys in combinatorics 2019, A. LO, R. MYCROFT, G. PERARNAU & A. TREGLOWN (eds)
 457 Shimura varieties, T. HAINES & M. HARRIS (eds)
 458 Integrable systems and algebraic geometry I, R. DONAGI & T. SHASKA (eds)
 459 Integrable systems and algebraic geometry II, R. DONAGI & T. SHASKA (eds)
 460 Wigner-type theorems for Hilbert Grassmannians, M. PANKOV
 461 Analysis and geometry on graphs and manifolds, M. KELLER, D. LENZ & R.K. WOJCIECHOWSKI
 462 Zeta and L -functions of varieties and motives, B. KAHN
 463 Differential geometry in the large, O. DEARRICOTT *et al* (eds)
 464 Lectures on orthogonal polynomials and special functions, H.S. COHL & M.E.H. ISMAIL (eds)
 465 Constrained Willmore surfaces, Á.C. QUINTINO
 466 Invariance of modules under automorphisms of their envelopes and covers, A.K. SRIVASTAVA, A. TUGANBAEV & P.A. GUIL ASENSIO
 467 The genesis of the Langlands program, J. MUELLER & F. SHAHIDI
 468 (Co)end calculus, F. LOREGIAN
 469 Computational cryptography, J.W. BOS & M. STAM (eds)
 470 Surveys in combinatorics 2021, K.K. DABROWSKI *et al* (eds)
 471 Matrix analysis and entrywise positivity preservers, A. KHARE
 472 Facets of algebraic geometry I, P. ALUFFI *et al* (eds)
 473 Facets of algebraic geometry II, P. ALUFFI *et al* (eds)
 474 Equivariant topology and derived algebra, S. BALCHIN, D. BARNES, M. KĘDZIOREK & M. SZYMIK (eds)
 475 Effective results and methods for Diophantine equations over finitely generated domains, J.-H. EVERTSE & K. GYÓRY
 476 An indefinite excursion in operator theory, A. GHEONDEA
 477 Elliptic regularity theory by approximation methods, E.A. PIMENTEL
 478 Recent developments in algebraic geometry, H. ABBAN, G. BROWN, A. KASPRZYK & S. MORI (eds)
 479 Bounded cohomology and simplicial volume, C. CAMPAGNOLO, F. FOURNIER-FACIO, N. HEUER & M. MORASCHINI (eds)
 480 Stacks Project Expository Collection (SPEC), P. BELMANS, W. HO & A.J. DE JONG (eds)
 481 Surveys in combinatorics 2022, A. NIXON & S. PRENDIVILLE (eds)
 482 The logical approach to automatic sequences, J. SHALLIT
 483 Rectifiability: a survey, P. MATTILA
 484 Discrete quantum walks on graphs and digraphs, C. GODSIL & H. ZHAN
 485 The Calabi problem for Fano threefolds, C. ARAUJO *et al*
 486 Modern trends in algebra and representation theory, D. JORDAN, N. MAZZA & S. SCHROLL (eds)
 487 Algebraic combinatorics and the Monster group, A.A. IVANOV (ed)
 488 Maurer–Cartan methods in deformation theory, V. DOTSENKO, S. SHADRIN & B. VALLETTE
 489 Higher dimensional algebraic geometry, C. HACON & C. XU (eds)
 490 C^∞ -algebraic geometry with corners, K. FRANCIS-STAIITE & D. JOYCE
 491 Groups and graphs, designs and dynamics, R.A. BAILEY, P.J. CAMERON & Y. WU (eds)
 492 Homotopy theory of enriched Mackey functors, N. JOHNSON & D. YAU
 493 Surveys in combinatorics 2024, F. FISCHER & R. JOHNSON (eds)
 494 K -theory and representation theory, R. PLYMEN & M.H. ŞENGÜN (eds)
 495 Polygraphs: from rewriting to higher categories, D. ARA *et al*
 496 Groups St Andrews 2022 in Newcastle, C.M. CAMPBELL *et al* (eds)
 497 Proof complexity generators, J. KRAJÍČEK

“Crafted with evident care for the subject and the reader, Niu and Spivak invite us into the mathematically abundant world of polynomial functors. Their practical and pedagogical approach plants the seeds for a long, fruitful interaction between ‘Poly’ and those making sense of our dynamic and interconnected world.”

— Brendan Fong, *Topos Institute*

“This book by Niu and Spivak is a new perspective on automata and dynamical systems. It contains new kinds of mathematics, but is fun and easy to read. It is all about polynomials, but of a new kind. It is all about lenses, but what is a lens? Do you know that a Moore machine is a special kind of lens? That a polynomial comonoid is the same thing as a category? Do you know what a retrofunctor is? The book answers all these questions and more. It offers plenty of solved exercises.”

— André Joyal, *Université du Québec à Montréal* (Emeritus)



Polynomial Functors

A Mathematical Theory of Interaction

NELSON NIU
University of Washington

DAVID I. SPIVAK
Topos Institute



CAMBRIDGE
UNIVERSITY PRESS



CAMBRIDGE
UNIVERSITY PRESS

Shaftesbury Road, Cambridge CB2 8EA, United Kingdom

One Liberty Plaza, 20th Floor, New York, NY 10006, USA

477 Williamstown Road, Port Melbourne, VIC 3207, Australia

314–321, 3rd Floor, Plot 3, Splendor Forum, Jasola District Centre,
New Delhi – 110025, India

103 Penang Road, #05–06/07, Visioncrest Commercial, Singapore 238467

Cambridge University Press is part of Cambridge University Press & Assessment,
a department of the University of Cambridge.

We share the University's mission to contribute to society through the pursuit of
education, learning and research at the highest international levels of excellence.

www.cambridge.org

Information on this title: www.cambridge.org/9781009576710

DOI: [10.1017/9781009576734](https://doi.org/10.1017/9781009576734)

© Nelson Niu and David I. Spivak 2025

This publication is in copyright. Subject to statutory exception and to the provisions
of relevant collective licensing agreements, no reproduction of any part may take place
without the written permission of Cambridge University Press & Assessment.

When citing this work, please include a reference to the DOI [10.1017/9781009576734](https://doi.org/10.1017/9781009576734)

First published 2025

Printed in the United Kingdom by CPI Group Ltd, Croydon CR0 4YY

A catalogue record for this publication is available from the British Library

*A Cataloging-in-Publication data record for this book is available from the
Library of Congress*

ISBN 978-1-009-57671-0 Paperback

Cambridge University Press & Assessment has no responsibility for the persistence
or accuracy of URLs for external or third-party internet websites referred to in this
publication and does not guarantee that any content on such websites is, or will
remain, accurate or appropriate.

For EU product safety concerns, contact us at Calle de José Abascal, 56, 1º, 28003 Madrid,
Spain, or email eugpsr@cambridge.org

To André Joyal

—D.S.

To my graduate cohort at UW

—N.N.

Contents

Preface	xiii
I The category of polynomial functors	1
1 Representable functors from the category of sets	3
1.1 Representable functors and the Yoneda lemma	3
1.2 Sums and products of sets	7
1.3 Expanding products of sums	14
1.4 Sums and products of functors $\mathbf{Set} \rightarrow \mathbf{Set}$	18
1.5 Summary and further reading	22
1.6 Exercise solutions	22
2 Polynomial functors	29
2.1 Introducing polynomial functors	29
2.2 Special classes of polynomial functors	33
2.3 Interpreting positions and directions	35
2.4 Corolla forests	37
2.5 Polyboxes	41
2.6 Summary and further reading	44
2.7 Exercise solutions	47
3 The category of polynomial functors	51
3.1 Dependent lenses between polynomial functors	51
3.2 Dependent lenses as interaction protocols	55
3.3 Corolla forest pictures of dependent lenses	57
3.4 Polybox pictures of dependent lenses	59
3.5 Computations with dependent lenses	61
3.6 Dependent lenses between special polynomials	66
3.7 Translating between natural transformations and lenses	73
3.8 Identity lenses and lens composition	76

3.9	Polybox pictures of lens composition	79
3.10	Symmetric monoidal products of polynomial functors	82
3.11	Summary and further reading	90
3.12	Exercise solutions	91
4	Dynamical systems as dependent lenses	105
4.1	Moore machines	105
4.2	Dependent dynamical systems	114
4.3	New dynamical systems from old	122
4.4	General interaction	136
4.5	Closure of $\otimes$	160
4.6	Summary and further reading	165
4.7	Exercise solutions	166
5	More categorical properties of polynomials	185
5.1	Special polynomials and adjunctions	185
5.2	Epi-mono factorization of lenses	189
5.3	Cartesian closure	194
5.4	Limits and colimits of polynomials	196
5.5	Vertical-cartesian factorization of lenses	203
5.6	$*$ -Bifibration over Set	209
5.7	Cartesian lenses are exponentiable	214
5.8	Summary and further reading	219
5.9	Exercise solutions	219
II	A different category of categories	231
6	The composition product	233
6.1	Interpreting the composition product	233
6.2	Lenses to composites	252
6.3	Categorical properties of the composition product	265
6.4	Summary and further reading	280
6.5	Exercise solutions	281
7	Polynomial comonoids and retrofunctors	295
7.1	State systems, categorically	295
7.2	Polynomial comonoids are categories	314

7.3	Morphisms of polynomial comonoids are retrofunctors	331
7.4	Summary and further reading	360
7.5	Exercise solutions	361
8	Categorical properties of polynomial comonoids	377
8.1	Cofree comonoids	377
8.2	More categorical properties of $\mathbf{Cat}^\sharp$	418
8.3	Comodules over polynomial comonoids	426
8.4	Summary and further reading	439
8.5	Exercise solutions	440
	References	453
	Index	461

Preface

And that is the way to get the greatest possible variety, but with all the order there could be; i.e. it is the way to get as much perfection as there could be.

—Gottfried Wilhelm Leibniz
Monadology, 58

During the Fifth International Conference on Applied Category Theory in 2022, at least twelve of the fifty-nine presentations and two of the ten posters referenced the category of polynomial functors and dependent lenses or its close cousins (categories of optics and Dialectica categories) and the way they model diverse forms of interactive behavior. At the same time, all that is needed to grasp the construction of this category—called **Poly** for short—is an understanding of mathematical sets and functions. Our goal, then, is to present a widely accessible introduction to the theory of polynomial functors and their applications to interaction so that the versatility of **Poly** may be appreciated and leveraged by a diverse audience of practitioners.

Informally, a polynomial functor is a collection of elements we call *positions* and, for each position, a collection of elements we call *directions*. There is then a natural notion of a morphism between polynomial functors that sends positions forward and directions backward, modeling two-way communication. From these basic components, category theory allows us to construct an immense array of mathematical gadgets that model a diverse range of interactive processes. In this book, we will establish the theory of polynomial functors and categorical constructions on them while exploring how they model interaction.

Purpose and prerequisites

A categorical theory of general interaction must be interdisciplinary by its very nature. Already, drafts of this text have been read by every-

one from algebraic geometers to neuroscientists and AI developers. We hope to extend our reach ever further, to bring together thinkers and tinkerers from a diverse array of backgrounds under a common language by which to study interactive systems categorically. In short—we know about **Poly**; you know about other things; but only our collective knowledge can reveal how **Poly** could be applied to those other things.

As such, we have strived to write a friendly and accessible expository text that can serve as a stepping stone toward further investigations into polynomial functors. We include exercises and, crucially, solutions to guide the learning process; we draw extensive analogies to provide motivation and develop intuition; we pose examples whenever necessary. Proofs may bear far more detail than you would find in a research paper, but not so much detail that it would clutter the key ideas. To further aid intuition, a few critical proofs are argued through a pictorial alternative to more standard notation that we will introduce.

At the same time, there is some deep mathematical substance to the work we will discuss, drawing from the well-established theory of categories. Although you will find, for example, a complete proof of the Yoneda lemma within these pages, we don't intend to build up everything from scratch. There are plenty of excellent resources for learning category theory out there, catering to a variety of needs, without adding our own to the mix when our primary goal is to introduce **Poly**. So for the sake of contributing only what is genuinely helpful, we assume a certain level of mathematical background. You are ready to read this book if you can define the following fundamental concepts from category theory, and give examples of each:

- categories,
- functors,
- natural transformations,
- (co)limits,
- adjunctions, and
- (symmetric) monoidal categories.

We will additionally assume a passing familiarity with the language of graph-theoretic trees (e.g. vertices, roots, leaves, paths).

That said, with a little investment on your part, you could very well use this book as a way to teach yourself some category theory.

If you have ever tried to learn category theory, only to become lost in abstraction or otherwise overwhelmed by seemingly endless lists of examples from foreign fields, perhaps you will benefit from a focused case study of one particularly fruitful category. If you encounter terms or ideas that you would like to learn more about, we encourage you to look them up elsewhere, and you may find yourself spending a pleasant afternoon doing a deep dive into a new definition or theorem. Then come back when you're ready—we'll be here.

Outline

This book is designed to be read linearly. Part I introduces the category **Poly** and illustrates how it models interaction protocols; while Part II highlights a crucial operation on **Poly**, the composition product, which upgrades the theory so that it properly captures the time evolution of a dynamical system.

Part I (The category of polynomial functors) consists of:

- Chapter 1 (Representable functors from the category of sets), in which we review category-theoretic constructions on sets and the Yoneda lemma;
- Chapter 2 (Polynomial functors), in which we introduce our objects of study and present several perspectives from which to view them;
- Chapter 3 (The category of polynomial functors), in which we define **Poly** by introducing its morphisms and demonstrate many ways to work with them, including how they model interaction protocols;
- Chapter 4 (Dynamical systems as dependent lenses), in which we use a specific class of morphisms in **Poly** to model discrete-time dynamical systems; and
- Chapter 5 (More categorical properties of polynomials), in which we describe a smorgasbord of additional category-theoretic structures on **Poly**.

Part II (A different category of categories) consists of:

- Chapter 6 (The composition product), in which we examine a monoidal structure on **Poly** given by substituting one polynomial into another;
- Chapter 7 (Polynomial comonoids and retrofunctors), in which we show that the category of comonoids in **Poly** with respect to the composition product is equivalent to a category of small categories we call **Cat[#]** whose morphisms are not functors;
- Chapter 8 (Categorical properties of polynomial comonoids), in which we study the structure and utility of **Cat[#]**; and

Choices and conventions

Throughout this book, we have chosen to focus on polynomial functors of a single variable on the category of sets. The motivation for this seemingly narrow scope is twofold: to keep matters as concrete and intuitive as possible, with easy access to elements that we can work with directly; and to demonstrate the immense versatility of even this small corner of the theory. Furthermore, the subject of multi-variate polynomials arises by considering what are called comonoids and comodules in **Poly** [SGF25].

While this book focuses on single-variable polynomial functors over the category of sets, it's important to acknowledge that the theory extends into a large variety of advanced areas that are not covered here. In type theory, polynomial functors are instrumental in modeling inductive types, providing a categorical framework for understanding data structures and recursion. For example, the fact that the free monad on a polynomial endofunctor is again carried by a polynomial is important in the study of computational effects and recursion. In dependent type theory, Hancock and Setzer have shown that a “world” can be modeled as a polynomial functor, with the free monad on that functor encapsulating IO actions [HS00]. We also will not cover the fact [SGF25] that polynomial comonads, together with their homomorphisms and bicomodules, assemble into a double category. In homotopy theory, particularly in the work of Michael Batanin, polynomial monads have been studied in relation to the Grothendieck construction, with ap-

plications to model structures and delooping techniques for mapping spaces [BL20]. These perspectives, while beyond the scope of this text, showcase the depth and versatility of the theory of polynomial functors across various mathematical disciplines.

Below is a list of conventions we adopt; while it is not comprehensive, any unusual choices are justified within the text, often as a footnote.

The natural numbers include 0, so $\mathbb{N} := \{0, 1, 2, \dots\}$. Throughout this book, when referring to finite sets, we will adopt the following convention: $0 := \{\} = \emptyset$, $1 := \{1\}$, $2 := \{1, 2\}$, $3 := \{1, 2, 3\}$, and so on, with $n := \{1, \dots, n\}$, an n -element set, for each natural number n . For example, in standard font, 5 represents the usual natural number, while in sans serif font, 5 represents the 5-element set $\{1, 2, 3, 4, 5\}$. When the same variable name appears in both italicized and sans serif fonts, the italicized variable denotes a natural number and the sans serif variable denotes the corresponding set; for example, if we state that $m \in \mathbb{N}$, then we also understand $\mathfrak{m}$ to mean the set $\{1, \dots, m\}$.

The names of categories will be capitalized. We will mostly ignore size issues, but roughly speaking small categories will be written in script (e.g. $\mathcal{C}$, $\mathcal{D}$), while large categories (usually, but not always, named) will be written in bold (e.g. **Poly**, **C**). We use **Set** to denote the category of (small) sets and functions and **Cat** to denote the category of (small) categories and functors. We use exponential notation $\mathcal{D}^{\mathcal{C}}$ to denote the category of functors $\mathcal{C} \rightarrow \mathcal{D}$ and natural transformations.

We write either $c \in \text{Ob } \mathcal{C}$ or $c \in \mathcal{C}$ to denote an object c of a category $\mathcal{C}$. We use $\sum$ rather than $\coprod$ to denote coproducts. We denote the collection of morphisms $f: c \rightarrow d$ in a category $\mathcal{C}$ by using the name of the category itself, followed by the ordered pair of objects: $\mathcal{C}(c, d)$. We denote the domain of f by $\text{dom } f$ and the codomain of f by $\text{cod } f$. We use $:=$ for definitions and temporary assignments, as opposed to $=$ for identifications that can be observed and proven. We use $\cong$ to indicate an isomorphism of objects and $=$ to indicate an equality of objects, although the choice of the former does not preclude the possibility of the latter, nor does the latter necessarily imply any significance beyond an arbitrary selection that has been made. We will freely use the definite article “the” to refer to objects that are unique only up to unique isomorphism.

We list nullary operations before binary ones: for example, we de-

note a monoidal category $\mathcal{C}$ with monoidal unit I and monoidal product $\odot$ by $(\mathcal{C}, I, \odot)$, or say that $(I, \odot)$ is a monoidal structure on $\mathcal{C}$.

A history of the text

As we will share, most of the ideas in this book have existed in one form or another for a long time, but the two key mathematical results that the rest of this text is organized around were proven by Daniel Ahman and Tarmo Uustalu in 2016 [AU16] and by Richard Garner in 2019 [Gar19].

We then began to write this book in 2020, originally as part of a joint text with David Jaz Myers on using categories to model dynamical systems. It soon became clear, however, that our writing and his—while intimately related—would be better off as separate volumes. His book is nonetheless an excellent companion to ours: see [Mye22].

In the summer of 2021, we taught a course on a draft of this book that was livestreamed from the Topos Institute. Lecture recordings are freely available at <https://topos.institute/events/poly-course/>. A follow-up workshop with additional recorded lectures and write-ups was held in early 2024; see <https://topos.institute/events/poly-at-work/>.

The theory and application of polynomial functors comprise an active area of research. We explain a selection of the foundations here, but work is still ongoing. Even while writing this, many new results and uses for polynomial functors have been discovered, which only goes to show how bountiful **Poly** can be in its rewards—but of course, we had to cut things off somewhere. We hope this book will serve as a stepping stone for readers to continue their study of polynomial functors.

Acknowledgments

Special thanks to David Jaz Myers: a brilliant colleague, a wonderful conversation partner, a congenial housemate, a superb chef, and an all-around good guy.

Thanks go to John Baez, Eric Bond, Spencer Breiner, Kris Brown, Matteo Capucci, Valeria de Paiva, Joseph Dorta, Brendan Fong, Richard Garner, Bruno Gavranović, Neil Ghani, Ben Goertzel, Tim Hosgood,

Samantha Jarvis, Shaowei Lin, Owen Lynch, Joachim Kock, Jérémie Koenig, Sophie Libkind, Joshua Meyers, Dominic Orchard, Nathaniel Osgood, Evan Patterson, Brandon Shapiro, Juliet Szatko, Tish Tanski, Todd Trimble, Adam Vandervorst, Jonathan Weinberger, and Christian Williams.

This material is based upon work supported by the AFOSR under award numbers FA9550-20-1-0348 and FA9550-23-1-0376.

Part I

The category of polynomial functors

1. Representable functors from the category of sets

In this chapter, we lay the categorical groundwork needed to define our category of interest, the category of polynomial functors. We begin by examining a special kind of polynomial functor that you may already be familiar with—representable functors from the category **Set** of sets and functions. We highlight the role these representable functors play in what is arguably the fundamental theorem of category theory, the Yoneda lemma. We will also discuss sums and products of sets and of set-valued functors, which we will need to construct our polynomial functors.

1.1 Representable functors and the Yoneda lemma

Representable functors—special functors to the category of sets—provide the foundation for the category **Poly**. While much of the following theory applies to representable functors from any category, we need only focus on representable functors **Set** $\rightarrow$ **Set**.

Definition 1.1 (Representable functor). For a set S , we denote by $y^S: \mathbf{Set} \rightarrow \mathbf{Set}$ the functor that sends each set X to the set $X^S = \mathbf{Set}(S, X)$ and each function $h: X \rightarrow Y$ to the function $h^S: X^S \rightarrow Y^S$ that sends $g: S \rightarrow X$ to $g \circ h: S \rightarrow Y$.¹

We call a functor (isomorphic to one) of this form a *representable functor*, or a *representable*. In particular, we call y^S the functor *represented by* S , and we call S the *representing set of* y^S . As y^S denotes raising a variable to the power of S , we will also refer to representables as *pure powers*.

¹Given morphisms $f: A \rightarrow B$ and $g: B \rightarrow C$ in a category, we will denote their composite morphism $A \rightarrow C$ interchangeably as $f \circ g$ or gf (or sparingly $g \circ f$), whichever

The symbol y stands for Yoneda, for reasons we will explain in Lemma 1.10 and Exercise 1.12 #5.

Throughout this book, we will use the notation $0 := \{\} = \emptyset$, $1 := \{1\}$, $2 := \{1, 2\}$, $3 := \{1, 2, 3\}$, and so on, with $n := \{1, \dots, n\}$.

Example 1.2. The functor that sends each set X to $X \times X$ and each function $h: X \rightarrow Y$ to $(h \times h): (X \times X) \rightarrow (Y \times Y)$ is representable. After all, $X \times X \cong X^2$, so this functor is the pure power y^2 .

Exercise 1.3. For each of the following functors $\mathbf{Set} \rightarrow \mathbf{Set}$, say if it is representable or not; if it is, give the set that represents it.

1. The identity functor $X \mapsto X$, which sends each function to itself.
2. The constant functor $X \mapsto 2$, which sends every function to the identity on 2.
3. The constant functor $X \mapsto 1$, which sends every function to the identity on 1.
4. The constant functor $X \mapsto 0$, which sends every function to the identity on 0.
5. A functor $X \mapsto X^{\mathbb{N}}$. If it could be representable, where should it send each function?
6. A functor $X \mapsto 2^X$. If it could be representable, where should it send each function?

Now that we have introduced representable functors, we study the maps between them. As representables are functors, the maps between them are natural transformations.

Proposition 1.4. For any function $f: R \rightarrow S$, there is an induced natural transformation $y^f: y^S \rightarrow y^R$; on any set X its X -component $X^f: X^S \rightarrow X^R$ is given by sending $g: S \rightarrow X$ to $f \circ g: R \rightarrow X$.

Proof. See Exercise 1.5. □

is more convenient.

Exercise 1.5. To prove Proposition 1.4, show that for any function $f: R \rightarrow S$, the given construction $y^f: y^S \rightarrow y^R$ really is a natural transformation. That is, for any function $h: X \rightarrow Y$, show that the following naturality square commutes:

$$\begin{array}{ccc} X^S & \xrightarrow{h^S} & Y^S \\ X^f \downarrow & ? & \downarrow Y^f \\ X^R & \xrightarrow{h^R} & Y^R \end{array} \quad (1.6)$$

Exercise 1.7. Let X be an arbitrary set. For each of the following sets R, S and functions $f: R \rightarrow S$, describe the X -component $X^f: X^S \rightarrow X^R$ of the natural transformation $y^f: y^S \rightarrow y^R$.

1. $R = 5, S = 5, f = \text{id}_5$. (You should describe the function $X^{\text{id}_5}: X^5 \rightarrow X^5$.)
2. $R = 2, S = 1, f$ is the unique function.
3. $R = 1, S = 2, f(1) = 1$.
4. $R = 1, S = 2, f(1) = 2$.
5. $R = 0, S = 5, f$ is the unique function.
6. $R = \mathbb{N}, S = \mathbb{N}, f(n) = n + 1$.

These representable functors and natural transformations live in the larger category $\mathbf{Set}^{\mathbf{Set}}$, whose objects are functors $\mathbf{Set} \rightarrow \mathbf{Set}$ and whose morphisms are the natural transformations between them.

Exercise 1.8. Show that the construction $f \mapsto y^f$ from Proposition 1.4 defines a functor

$$y^-: \mathbf{Set}^{\text{op}} \rightarrow \mathbf{Set}^{\mathbf{Set}} \quad (1.9)$$

by verifying functoriality, as follows.

1. Show that for any set S , the natural transformation $y^{\text{id}_S}: y^S \rightarrow y^S$ is the identity.

2. Show that for functions $f: R \rightarrow S$ and $g: S \rightarrow T$, we have $y^g \circ y^f = y^{f \circ g}$.

We now have all the ingredients we need to state and prove the Yoneda lemma on the category of sets.

Lemma 1.10 (Yoneda lemma). Given a functor $F: \mathbf{Set} \rightarrow \mathbf{Set}$ and a set S , there is an isomorphism

$$F(S) \cong \mathbf{Set}^{\mathbf{Set}}(y^S, F) \quad (1.11)$$

where the right-hand side is the set of natural transformations $y^S \rightarrow F$. Moreover, (1.11) is natural in both S and F .

Proof. Let $m: y^S \rightarrow F$ be a natural transformation, and consider its S -component $m_S: S^S \rightarrow F(S)$. Applying this function to $\text{id}_S \in S^S$ yields an element $m_S(\text{id}_S) \in F(S)$.

Conversely, given an element $a \in F(S)$, there is a natural transformation we denote by $m^a: y^S \rightarrow F$ whose X -component is the function $X^S \rightarrow F(X)$ that sends $g: S \rightarrow X$ to $F(g)(a)$. In Exercise 1.12 we ask you to show that this is indeed natural in X ; that these two constructions, $m \mapsto m_S(\text{id}_S)$ and $a \mapsto m^a$, are mutually inverse; and that the resulting isomorphism is natural. $\square$

Exercise 1.12. In this exercise, we fill in the details of the preceding proof.

1. Show that for any $a \in F(S)$, the maps $X^S \rightarrow F(X)$ defined in the proof of Lemma 1.10 are natural in X .
2. Show that the two mappings given in the proof of Lemma 1.10 are mutually inverse, thus defining the isomorphism (1.11).
3. Show that (1.11) as defined is natural in F .
4. Show that (1.11) as defined is natural in S .
5. Use Lemma 1.10 to show that the functor $y^-: \mathbf{Set}^{\text{op}} \rightarrow \mathbf{Set}^{\mathbf{Set}}$ from (1.9) is fully faithful—in particular, there is an isomorphism $S^T \cong \mathbf{Set}^{\mathbf{Set}}(y^S, y^T)$ for sets S, T . For this reason, we call y^- the *Yoneda embedding*.

How will we go from these representable functors to polynomial ones? Recall that, in algebra, a polynomial is just a sum of pure powers. So we will define a *polynomial functor* $\mathbf{Set} \rightarrow \mathbf{Set}$ to be a sum of pure power functors—that is, the representable functors y^A for each set A we just introduced.²

All of our polynomials will be in one variable, y . Every other letter or number that shows up in our notation for a polynomial will denote a set. For example, in the polynomial

$$\mathbb{R}y^{\mathbb{Z}} + 3y^3 + y^A + \sum_{i \in I} y^{R_i + Q_i^2}, \quad (1.13)$$

$\mathbb{R}$ denotes the set of real numbers, $\mathbb{Z}$ denotes the set of integers, 2 and 3 respectively denote the sets $\{1, 2\}$ and $\{1, 2, 3\}$, and A , I , Q_i , and R_i denote arbitrary sets.

To make sense of these polynomials, we need to define functor addition, both in the binary case (i.e. what is $y^A + y^B$?) and more generally over arbitrary sets (i.e. what is $\sum_{i \in I} y^{A_i}$?). This will allow us to interpret polynomials like (1.13). In particular, just as $3y^3 = y^3 + y^3 + y^3$ in algebra, the summand $3y^3$ of (1.13) denotes the sum of representables $y^3 + y^3 + y^3$, while the summand $\mathbb{R}y^{\mathbb{Z}}$ denotes the sum over $\mathbb{R}$ of copies of $y^{\mathbb{Z}}$.

While polynomial functors will be defined as sums, products of polynomials will turn out to be polynomials as well, again mimicking polynomials in algebra. To make sense of these products, we will also define functor multiplication. The construction of sums and products of functors $\mathbf{Set} \rightarrow \mathbf{Set}$ will rely on the construction of sets and products of sets themselves.

1.2 Sums and products of sets

Let I be a set, and let X_i be a set for each $i \in I$. Classically, we may denote this *I-indexed family of sets* by $(X_i)_{i \in I}$. Categorically, we may view this data as a specific kind of functor: if we identify the set I with the *discrete category* on I , whose objects are the elements of I and whose morphisms

²This analogy isn't perfect: in algebra, polynomials are generally finite sums of pure powers, whereas our polynomial functors may be infinite sums of representables. However, we are not the first to use the term "polynomial" this way, and the name stuck.

are all identities, then $(X_i)_{i \in I}$ can be identified with a functor $X: I \rightarrow \mathbf{Set}$ with $X(i) := X_i$. To compromise, we will denote an indexed family of sets by $X: I \rightarrow \mathbf{Set}$ for a set I viewed as a discrete category (although we will occasionally use the classical notation when convenient), but denote the set obtained by evaluating X at each $i \in I$ by X_i rather than $X(i)$.

To pick out an element of one of the sets in the indexed family $X: I \rightarrow \mathbf{Set}$, we need to specify both an index $i \in I$ and an element $x \in X_i$. We call the set of such pairs (i, x) the *sum* of this indexed family, as below.

Definition 1.14 (Sum of sets). Let I be a set and $X: I \rightarrow \mathbf{Set}$ be an I -indexed family of sets. The *sum* $\sum_{i \in I} X_i$ of the indexed family X is the set

$$\sum_{i \in I} X_i := \{(i, x) \mid i \in I \text{ and } x \in X_i\}.$$

When $I := \{i_1, \dots, i_n\}$ is finite, we may alternatively denote this sum as $X_{i_1} + \dots + X_{i_n}$.

Say instead we pick an element from *every* set in the indexed family: that is, we construct an assignment $i \mapsto x_i$, where each $x_i \in X_i$. If every X_i were the same set X , then this would just be a function $I \rightarrow X$. More generally, this assignment is what we call a *dependent function*: its codomain X_i *depends* on its input i . We write the signature of such a dependent function as

$$f: (i \in I) \rightarrow X_i.$$

Note that the indexed family of sets $X: I \rightarrow \mathbf{Set}$ completely determines this signature. The set of all dependent functions whose signature is determined by a given indexed family of sets is the *product* of that indexed family, as below.

Definition 1.15 (Product of sets). Let I be a set and $X: I \rightarrow \mathbf{Set}$ be an I -indexed family of sets. The *product* $\prod_{i \in I} X_i$ of the indexed family

X is the set of dependent functions

$$\prod_{i \in I} X_i := \{f: (i \in I) \rightarrow X_i\}.$$

When $I := \{i_1, \dots, i_n\}$ is finite, we may alternatively denote this product as $X_{i_1} \times \dots \times X_{i_n}$ or $X_{i_1} \cdots X_{i_n}$.

For a dependent function $f: (i \in I) \rightarrow X_i$, we may denote the element of X_i that f assigns to $i \in I$ by $f(i)$, fi , or f_i . When $I := \{i_1, \dots, i_n\}$ is finite, we may identify f with the n -tuple $(f(i_1), \dots, f(i_n))$; similarly, when $I := \mathbb{N}$, we may identify f with the infinite sequence $(f_0, f_1, f_2, \dots)$.

Example 1.16. If $I := 2 = \{1, 2\}$, then an I -indexed family $X: I \rightarrow \mathbf{Set}$ consists of two sets—say $X_1 := \{a, b, c\}$ and $X_2 := \{c, d\}$. Their sum is then the disjoint union

$$\sum_{i \in 2} X_i = X_1 + X_2 = \{(1, a), (1, b), (1, c), (2, c), (2, d)\}.$$

The cardinality³ of $X_1 + X_2$ will always be the sum of the cardinalities of X_1 and X_2 , justifying the use of the word “sum.”

Meanwhile, their product is the usual cartesian product

$$\prod_{i \in 2} X_i \cong X_1 \times X_2 = \{(a, c), (a, d), (b, c), (b, d), (c, c), (c, d)\}.$$

The cardinality of $X_1 \times X_2$ will always be the product of the cardinalities of X_1 and X_2 , justifying the use of the word “product.”

Exercise 1.17. Let I be a set.

1. Show that there is an isomorphism of sets $I \cong \sum_{i \in I} 1$.
2. Show that there is an isomorphism of sets $1 \cong \prod_{i \in I} 1$.

As a special case, suppose that $I := 0 = \emptyset$ and that $X: \emptyset \rightarrow \mathbf{Set}$ is the unique empty indexed family of sets.

³The *cardinality* of a set is the number of elements it contains, at least when the set is finite; with care the notion can be extended to infinite sets as well.

3. Is it true that $X_i = 1$ for each $i \in I$?
4. Justify the statement “the empty sum is 0” by showing that there is an isomorphism of sets $\sum_{i \in \emptyset} X_i \cong 0$.
5. Justify the statement “the empty product is 1” by showing that there is an isomorphism of sets $\prod_{i \in \emptyset} X_i \cong 1$.

The following standard fact describes the constructions from Definitions 1.14 and 1.15 categorically and further justifies why we call them sums and products.

Proposition 1.18. Let I be a set and $X: I \rightarrow \mathbf{Set}$ be an I -indexed family of sets. Then the sum $\sum_{i \in I} X_i$ is the categorical coproduct of these sets in $\mathbf{Set}$ (i.e. the colimit of the functor $X: I \rightarrow \mathbf{Set}$, viewed as a diagram), and the product $\prod_{i \in I} X_i$ is the categorical product of these sets in $\mathbf{Set}$ (i.e. the limit of the functor $X: I \rightarrow \mathbf{Set}$, viewed as a diagram).

Proof. The sum $\sum_{i \in I} X_i$ comes equipped with an inclusion $\iota_j: X_j \rightarrow \sum_{i \in I} X_i$ for each $j \in I$ given by $x \mapsto (j, x)$. The product $\prod_{i \in I} X_i$ comes equipped with a projection $\pi_j: \prod_{i \in I} X_i \rightarrow X_j$ for each $j \in I$ sending each $f: (i \in I) \rightarrow X_i$ to $f(j)$. These satisfy the universal properties for categorical coproducts and products, respectively; see Exercise 1.19. $\square$

Exercise 1.19.

1. Show that $\sum_{i \in I} X_i$ along with the inclusions $\iota_j: X_j \rightarrow \sum_{i \in I} X_i$ described in the proof of Proposition 1.18 satisfy the universal property of a categorical coproduct: for any set Y with functions $g_j: X_j \rightarrow Y$ for each $j \in I$, there exists a unique function $h: \sum_{i \in I} X_i \rightarrow Y$ for which $\iota_j \circ h = g_j$ for all $j \in I$.
2. Show that $\prod_{i \in I} X_i$ along with the projections $\pi_j: \prod_{i \in I} X_i \rightarrow X_j$ described in the proof of Proposition 1.18 satisfy the universal property of a categorical product: for any set Y with functions $g_j: Y \rightarrow X_j$ for each $j \in I$, there exists a unique function $h: Y \rightarrow \prod_{i \in I} X_i$ for which $h \circ \pi_j = g_j$ for all $j \in I$.

Though we proved above explicitly only that **Set** has all small products and coproducts, from here on out, we will assume the standard categorical fact that **Set** is complete (has all small limits) and cocomplete (has all small colimits): see for instance [Rie17, Theorem 3.2.6 and Proposition 3.5.1] for a proof.

We have constructed categorical sums and products of sets, but we can also construct categorical sums and products of the maps between them: functions.

Definition 1.20 (Categorical sum and product of functions). Let I be a set and $X, Y: I \rightarrow \mathbf{Set}$ be I -indexed families of sets. Given a natural transformation $f: X \rightarrow Y$, i.e. an I -indexed family of functions $(f_i: X_i \rightarrow Y_i)_{i \in I}$, its *categorical sum* (or *coproduct*) is the function

$$\sum_{i \in I} f_i: \sum_{i \in I} X_i \rightarrow \sum_{i \in I} Y_i$$

that, given $i \in I$ and $x \in X_i$, sends $(i, x) \mapsto (i, f_i(x))$; while its *categorical product* is the function

$$\prod_{i \in I} f_i: \prod_{i \in I} X_i \rightarrow \prod_{i \in I} Y_i$$

that sends each $g: (i \in I) \rightarrow X_i$ to the *composite dependent function* $(i \in I) \rightarrow Y_i$, denoted $g \circ f$ or fg , which sends $i \in I$ to $f_i(g(i))$.

When $I := \{i_1, \dots, i_n\}$ is finite, we may alternatively denote this categorical sum and product of functions respectively as⁴

$$f_{i_1} + \dots + f_{i_n} \quad \text{and} \quad f_{i_1} \times \dots \times f_{i_n}.$$

Exercise 1.21.

1. Show that the categorical sum of functions is the one induced by the universal property of the categorical sum of sets. That is, given a set I , two I -indexed families of sets $X, Y: I \rightarrow \mathbf{Set}$, and a natural transformation $f: X \rightarrow Y$, the function

⁴We will take care to highlight when this notation may clash with a sum (resp. product) of functions with common domain and codomain whose codomain has an additive (resp. multiplicative) structure.

$\sum_{i \in I} f_i: \sum_{i \in I} X_i \rightarrow \sum_{i \in I} Y_i$ that we called the categorical sum is induced by the following composite maps for $j \in I$:

$$X_j \xrightarrow{f_j} Y_j \xrightarrow{\iota'_j} \sum_{i \in I} Y_i,$$

where ι'_j is the inclusion. It then follows by a standard categorical argument that the sum is functorial, i.e. that the sum of identities is an identity and that the sum of composites is the composite of sums.

2. Similarly, show that the categorical product of functions is the one induced by the universal property of the categorical product of sets. That is, given the same setup as the previous part, the function $\prod_{i \in I} f_i: \prod_{i \in I} X_i \rightarrow \prod_{i \in I} Y_i$ that we called the categorical product is induced by the following composite maps for $j \in J$:

$$\prod_{i \in I} X_i \xrightarrow{\pi_j} X_j \xrightarrow{f_j} Y_j.$$

Again, this implies that the product is functorial.

We now highlight some tools and techniques to help us work with sum and product sets.

Exercise 1.22. Let I be a set and $X: I \rightarrow \mathbf{Set}$ be an indexed family. There is a projection function $\pi_1: \sum_{i \in I} X_i \rightarrow I$ defined by $\pi_1(i, x) := i$.

1. What is the signature of the second projection $\pi_2(i, x) := x$? (Hint: it is a dependent function.)
2. A *section* of a function $r: A \rightarrow B$ is a function $s: B \rightarrow A$ such that $s \circ r = \text{id}_B$. Show that the product of the indexed family is isomorphic to the set of sections of π_1 :

$$\prod_{i \in I} X_i \cong \left\{ s: I \rightarrow \sum_{i \in I} X_i \mid s \circ \pi_1 = \text{id}_I \right\}.$$

A helpful way to think about sum or product sets is to consider what choices must be made to specify an element of such a set. In the

following examples, say that we have a set I and an I -indexed family $X: I \rightarrow \mathbf{Set}$.

Below, we give the instructions for choosing an element of $\sum_{i \in I} X_i$.

To choose an element of $\sum_{i \in I} X_i$:

1. choose an element $i \in I$;
2. choose an element of X_i .

Then the projection π_1 from Exercise 1.22 sends each element of $\sum_{i \in I} X_i$ to the element of $i \in I$ chosen in step 1, while the projection π_2 sends each element of $\sum_{i \in I} X_i$ to the element of X_i chosen in step 2.

Next, we give the instructions for choosing an element of $\prod_{i \in I} X_i$.

To choose an element of $\prod_{i \in I} X_i$:

1. for each element $i \in I$:
 - 1.1. choose an element of X_i .

Armed with these interpretations, we can tackle more complicated expressions, including those with nested $\sum$'s and $\prod$'s such as

$$A := \sum_{i \in I} \prod_{j \in J(i)} \sum_{k \in K(i, j)} X(i, j, k). \quad (1.23)$$

The instructions for choosing an element of A form a nested list, as follows.

To choose an element of A :

1. choose an element $i \in I$;
2. for each element $j \in J(i)$:
 - 2.1. choose an element $k \in K(i, j)$;
 - 2.2. choose an element of $X(i, j, k)$.

Here the choice of $k \in K(i, j)$ may depend on i and j : different values of i and j may lead to different sets $K(i, j)$.

By describing A like this, we see that each $a \in A$ can be projected to an element $i := \pi_1(a) \in I$, chosen in step 1, and a dependent function $\pi_2(a)$, chosen in step 2. This dependent function in turn sends each $j \in J(i)$ to a pair that can be projected to an element $k := \pi_1(\pi_2(a)(j)) \in K(i, j)$ chosen in step 2.1 and an element $\pi_2(\pi_2(a)(j)) \in X(i, j, k)$ chosen in step 2.2.

Example 1.24. Let $I := \{1, 2\}$; let $J(1) := \{j\}$ and $J(2) := \{j, j'\}$; let $K(1, j) := \{k_1, k_2\}$, $K(2, j) := \{k_1\}$, and $K(2, j') := \{k'\}$; and let $X(i, j, k) := \{x, y\}$ for all i, j, k . Now the formula

$$\sum_{i \in I} \prod_{j \in J(i)} \sum_{k \in K(i, j)} X(i, j, k)$$

from (1.23) specifies a fixed set:

$$\left\{ \begin{array}{l} (1, j \mapsto (k_1, x)), (1, j \mapsto (k_1, y)), (1, j \mapsto (k_2, x)), (1, j \mapsto (k_2, y)), \\ (2, j \mapsto (k_1, x), j' \mapsto (k', x)), (2, j \mapsto (k_1, x), j' \mapsto (k', y)), \\ (2, j \mapsto (k_1, y), j' \mapsto (k', x)), (2, j \mapsto (k_1, y), j' \mapsto (k', y)) \end{array} \right\}.$$

In each case, we first chose an element $i \in I$, either 1 or 2. Then for each $j \in J(i)$ we chose an element $k \in K(i, j)$ and an element of $X(i, j, k)$.

Exercise 1.25. Consider the set

$$B := \prod_{i \in I} \sum_{j \in J(i)} \prod_{k \in K(i, j)} X(i, j, k). \quad (1.26)$$

1. Give the instructions for choosing an element of B as a nested list, like we did for A just below (1.23).
2. With I, J, K , and X as in Example 1.24, how many elements are in B ?
3. Write out three of these elements in the style of Example 1.24.

1.3 Expanding products of sums

We will often encounter sums of sets nested within products, as in (1.23) and (1.26). The following proposition helps us work with these; it is sometimes called the *type-theoretic axiom of choice*, but it is perhaps more familiar as a set-theoretic analogue of the *distributive property* of multiplication over addition. While the identity may look foreign, it captures for sets the same process that you would use to multiply multi-

digit numbers from grade school arithmetic or polynomials from high school algebra.

Proposition 1.27 (Pushing $\prod$ past $\sum$). For any sets $I, (J(i))_{i \in I}$, and $(X(i, j))_{i \in I, j \in J(i)}$, we have a natural isomorphism

$$\prod_{i \in I} \sum_{j \in J(i)} X(i, j) \cong \sum_{\bar{j} \in \prod_{i \in I} J(i)} \prod_{i \in I} X(i, \bar{j}(i)).^5 \quad (1.28)$$

Proof. First, we construct a map from the left-hand set to the right. An element of the set on the left is a dependent function $f: (i \in I) \rightarrow \sum_{j \in J(i)} X(i, j)$, which we can compose with projections from its codomain to yield $\pi_1(f(i)) \in J(i)$ and $\pi_2(f(i)) \in X(i, \pi_1(f(i)))$ for every $i \in I$. We can then form the following pair:⁶

$$(i \mapsto \pi_1 f i, i \mapsto \pi_2 f i).$$

This is an element of the right-hand set, because $i \mapsto \pi_1 f i$ is a dependent function in $\prod_{i \in I} J(i)$ and $i \mapsto \pi_2 f i$ is a dependent function in $\prod_{i \in I} X(i, \pi_1 f i)$.

Now we go from right to left. An element of the right-hand set is a pair of dependent functions, $\bar{j}: (i \in I) \rightarrow J(i)$ and $g: (i \in I) \rightarrow X(i, \bar{j}i)$. We map this pair to the following element of the left-hand set, a dependent function $(i \in I) \rightarrow \sum_{j \in J(i)} X(i, j)$:

$$i \mapsto (\bar{j}i, gi).$$

Finally, we verify that the maps are mutually inverse. An element $(\bar{j}, g)$ of the right-hand set is sent by one map and then the other to the pair

$$(i \mapsto \pi_1(\bar{j}i, gi), i \mapsto \pi_2(\bar{j}i, gi)) = (i \mapsto \bar{j}i, i \mapsto gi) = (\bar{j}, g).$$

⁵We draw a bar over j in $\bar{j}$ to remind ourselves that $\bar{j}$ is no longer just an index but a (dependent) function.

⁶We omit parentheses for function application here and throughout the text for compactness whenever the meaning is clear.

Meanwhile, an element f of the left-hand set is sent by one map and then the other to the dependent function

$$i \mapsto (\pi_1 f i, \pi_2 f i).$$

As $f i$ is a pair whose components are $\pi_1 f i$ and $\pi_2 f i$, the dependent function above is precisely f . $\square$

When $J(i) = J$ does not depend on $i \in I$, we can simplify the formula in (1.28).

Corollary 1.29. For any sets I, J , and $(X(i, j))_{i \in I, j \in J}$, we have a natural isomorphism

$$\prod_{i \in I} \sum_{j \in J} X(i, j) \cong \sum_{\bar{j}: I \rightarrow J} \prod_{i \in I} X(i, \bar{j} i), \quad (1.30)$$

where $\bar{j}$ ranges over all (standard, non-dependent) functions $I \rightarrow J$.

Proof. Take $J(i) := J$ for all $i \in I$ in (1.28). Then the set $\prod_{i \in I} J(i)$ becomes $\prod_{i \in I} J$ (which we may denote in exponential form by J^I); its elements, dependent functions $\bar{j}: (i \in I) \rightarrow J(i) = J$, become standard functions $\bar{j}: I \rightarrow J$. $\square$

It turns out that being able to push $\prod$ past $\sum$ as in (1.28) is not a property that is unique to sets. In general, we refer to a category having this property as follows.

Definition 1.31 (Completely distributive category). A category $\mathbf{C}$ with all small products and coproducts is *completely distributive*⁷ if products distribute over coproducts as in (1.28); that is, for any set I , sets $(J(i))_{i \in I}$, and objects $(X(i, j))_{i \in I, j \in J(i)}$ from $\mathbf{C}$, we have a natural isomorphism

$$\prod_{i \in I} \sum_{j \in J(i)} X(i, j) \cong \sum_{\bar{j} \in \prod_{i \in I} J(i)} \prod_{i \in I} X(i, \bar{j} i). \quad (1.32)$$

So Proposition 1.27 states that **Set** is completely distributive. Once we define the category of polynomial functors, we will see that it, too, is completely distributive.

Corollary 1.29 generalizes to all completely distributive categories as well; we state this formally in Corollary 1.33.

Corollary 1.33. Let $\mathbf{C}$ be a completely distributive category. For any sets I and J and objects $(X(i, j))_{i \in I, j \in J}$ in $\mathbf{C}$, we have a natural isomorphism

$$\prod_{i \in I} \sum_{j \in J} X(i, j) \cong \sum_{\bar{j}: I \rightarrow J} \prod_{i \in I} X(i, \bar{j}i). \quad (1.34)$$

Proof. Again, take $J(i) := J$ for all $i \in I$ in (1.32). □

Exercise 1.35. Let $\mathbf{C}$ be a completely distributive category. How is the usual distributive law

$$X \times (Y + Z) \cong X \times Y + X \times Z$$

for $X, Y, Z \in \mathcal{C}$ a special case of (1.32)?

Throughout this book, such as in Exercise 1.36, you will see expressions consisting of alternating products and sums. Using (1.32), you can always rewrite such an expression as a sum of products, in which every $\sum$ appears before every $\prod$.⁸ This is analogous to how products of sums in high school algebra can always be expanded into sums of products via the distributive property.

Exercise 1.36. Let $I, (J(i))_{i \in I}$, and $(K(i, j))_{(i, j) \in IJ}$ be sets, and for each $(i, j, k) \in IJK$, let $X(i, j, k)$ be an object in a completely distributive

⁷While our terminology generalizes that of a completely distributive lattice, which has the additional requirement that the category be a poset, it is unfortunately not standard: a completely distributive category refers to a different concept in some categorical literature. We will not use this other concept, so there is no ambiguity.

⁸When an expression is written so that every $\sum$ appears before every $\prod$, it is said to be in *disjunctive normal form*.

category.

1. Rewrite so that every $\sum$ appears before every $\prod$:

$$\sum_{i \in I} \prod_{j \in J(i)} \sum_{k \in K(i, j)} X(i, j, k).$$

2. Rewrite so that every $\sum$ appears before every $\prod$:

$$\prod_{i \in I} \sum_{j \in J(i)} \prod_{k \in K(i, j)} X(i, j, k).$$

3. Rewrite so that every $\sum$ appears before every $\prod$:

$$\prod_{i \in I} \prod_{j \in J(i)} \sum_{k \in K(i, j)} X(i, j, k).$$

Now that we understand sums and products of sets, we are ready to explore sums and products of set-valued functors.

1.4 Sums and products of functors $\mathbf{Set} \rightarrow \mathbf{Set}$

Recall that our goal is to define polynomial functors such as $y^2 + 2y + 1$ and the maps between them. We have defined representable functors such as y^2 , y , and 1 ; we just need to interpret sums of functors $\mathbf{Set} \rightarrow \mathbf{Set}$. We will interpret products of such functors as well, which will also be useful. Both these concepts generalize to limits and colimits in $\mathbf{Set}^{\mathbf{Set}}$.

Proposition 1.37. The category $\mathbf{Set}^{\mathbf{Set}}$ has all small limits and colimits, and they are computed pointwise. In particular, on objects, given a small category $\mathcal{J}$ and a functor $F: \mathcal{J} \rightarrow \mathbf{Set}^{\mathbf{Set}}$, for all $X \in \mathbf{Set}$, the limit and colimit of F satisfy the following isomorphisms natural in X :

$$\left(\lim_{j \in \mathcal{J}} F(j) \right)(X) \cong \lim_{j \in \mathcal{J}} (F(j)(X))$$

and

$$\left(\operatorname{colim}_{j \in \mathcal{J}} F(j) \right)(X) \cong \operatorname{colim}_{j \in \mathcal{J}} (F(j)(X)).$$

Proof. This is a special case of a more general fact when $\mathbf{Set}^{\mathbf{Set}}$ is replaced by an arbitrary functor category $\mathbf{D}^{\mathbf{C}}$, where $\mathbf{D}$ is a category that (like $\mathbf{Set}$) has all small limits and colimits; see [MM92, pages 22–23, displays (24) and (25)]. $\square$

Focusing on the case of coproducts and products, the following corollary is immediate.

Corollary 1.38 (Sums and products of functors $\mathbf{Set} \rightarrow \mathbf{Set}$). Given functors $F, G: \mathbf{Set} \rightarrow \mathbf{Set}$, their categorical coproduct or sum in $\mathbf{Set}^{\mathbf{Set}}$, denoted $F+G$, is the functor $\mathbf{Set} \rightarrow \mathbf{Set}$ defined for $X, Y \in \mathbf{Set}$ and $f: X \rightarrow Y$ by

$$(F+G)(X) := F(X) + G(X) \quad \text{and} \quad (F+G)(f) := F(f) + G(f);$$

while their categorical product in $\mathbf{Set}^{\mathbf{Set}}$, denoted $F \times G$ or FG , is the functor $\mathbf{Set} \rightarrow \mathbf{Set}$ defined for $X, Y \in \mathbf{Set}$ and $f: X \rightarrow Y$ by

$$(F \times G)(X) := F(X) \times G(X) \quad \text{and} \quad (F \times G)(f) := F(f) \times G(f).$$

More generally, given functors $(F_i)_{i \in I}$ indexed over a set I , their categorical coproduct or sum and categorical product in $\mathbf{Set}^{\mathbf{Set}}$, respectively denoted

$$\sum_{i \in I} F_i: \mathbf{Set} \rightarrow \mathbf{Set} \quad \text{and} \quad \prod_{i \in I} F_i: \mathbf{Set} \rightarrow \mathbf{Set},$$

are respectively defined for $X \in \mathbf{Set}$ by

$$\left(\sum_{i \in I} F_i \right)(X) := \sum_{i \in I} F_i(X) \quad \text{and} \quad \left(\prod_{i \in I} F_i \right)(X) := \prod_{i \in I} F_i(X)$$

and for functions $f: X \rightarrow Y$ by

$$\left(\sum_{i \in I} F_i \right)(f) := \sum_{i \in I} F_i(f) \quad \text{and} \quad \left(\prod_{i \in I} F_i \right)(f) := \prod_{i \in I} F_i(f).$$

We also note the special case of initial and terminal objects. Given a set $I \in \mathbf{Set}$, we will also use I to denote the constant functor $\mathbf{Set} \rightarrow \mathbf{Set}$

that sends every set to I .

Corollary 1.39 (Initial and terminal functors $\mathbf{Set} \rightarrow \mathbf{Set}$). The constant functor $0: \mathbf{Set} \rightarrow \mathbf{Set}$ is initial in $\mathbf{Set}^{\mathbf{Set}}$, while the constant functor $1: \mathbf{Set} \rightarrow \mathbf{Set}$ is terminal in $\mathbf{Set}^{\mathbf{Set}}$.

Proof. As the set 0 is initial in $\mathbf{Set}$ (for every set X there is a unique map $0 \rightarrow X$), Proposition 1.37 implies that the constant functor 0 is initial in $\mathbf{Set}^{\mathbf{Set}}$. Similarly, as the set 1 is terminal in $\mathbf{Set}$ (for every set X there is a unique map $X \rightarrow 1$), Proposition 1.37 implies that the constant functor 1 is terminal in $\mathbf{Set}^{\mathbf{Set}}$. $\square$

Finally, we note that $\mathbf{Set}^{\mathbf{Set}}$ inherits the distributivity of $\mathbf{Set}$.

Proposition 1.40. The category $\mathbf{Set}^{\mathbf{Set}}$ is completely distributive.

Proof. This follows directly from the fact that $\mathbf{Set}$ itself is completely distributive (Proposition 1.27) and the fact that sums and products in $\mathbf{Set}^{\mathbf{Set}}$ are computed pointwise (Corollary 1.38). $\square$

The following exercises justify some notational shortcuts we will use when denoting polynomial functors. First, for any set A and functor $F: \mathbf{Set} \rightarrow \mathbf{Set}$, we may write an A -indexed sum of copies of F as AF , the product of F and the constant functor A ; for instance, $y + y \cong 2y$.

Exercise 1.41. Show that for a set $A \in \mathbf{Set}$ and a functor $F: \mathbf{Set} \rightarrow \mathbf{Set}$, an A -indexed sum of copies of F is isomorphic to the product of the constant functor A and F :

$$\sum_{a \in A} F \cong AF.$$

(This is analogous to the fact that adding up n copies of number is equal to multiplying that same number by n .)

Similarly, we may wish to write an A -indexed product of copies of F in exponential form as F^A . But since we have already introduced exponential notation for representable functors, this yields two possible

interpretations for the functor **Set** → **Set** denoted by y^A : as the functor represented by A , or as the A -indexed product of copies of the identity functor $y: \mathbf{Set} \rightarrow \mathbf{Set}$. In fact, the following exercise shows that there is no ambiguity, as the two interpretations are isomorphic.

Exercise 1.42.

1. Show that for a set $I \in \mathbf{Set}$, an I -indexed product of copies of the identity functor $y: \mathbf{Set} \rightarrow \mathbf{Set}$ is isomorphic to the functor $y^I: \mathbf{Set} \rightarrow \mathbf{Set}$ represented by I :

$$\prod_{i \in I} y \cong y^I.$$

(This is analogous to the fact that multiplying n copies of a number together is equal to raising that same number to the power of n .)

2. Show that the I -indexed product of copies of a representable functor $y^A: \mathbf{Set} \rightarrow \mathbf{Set}$ for some $A \in \mathbf{Set}$ is isomorphic to the functor $y^{IA}: \mathbf{Set} \rightarrow \mathbf{Set}$ represented by the product set IA :

$$\prod_{i \in I} y^A \cong y^{IA}.$$

(Hint: you may use the fact that the natural isomorphism

$$\{f: I \times A \rightarrow X\} \cong \{g: I \rightarrow X^A\}$$

holds between sets of functions. The process of converting a function f in the left-hand set to the corresponding function $i \mapsto (a \mapsto f(i, a))$ in the right is known as *currying*.)

Henceforth, given $A \in \mathbf{Set}$ and a functor $F: \mathbf{Set} \rightarrow \mathbf{Set}$, we define

$$F^A := \prod_{a \in A} F.$$

Exercise 1.42 shows that this notation does not conflict with the way we write representable functors as powers of the identity functor y . The exercise also shows how a power of a representable functor can be simplified to a single representable functor.

With these ingredients, we are finally ready to define what a polynomial functor is. We will begin with this definition in the next chapter.

1.5 Summary and further reading

In this chapter, we reviewed the definition of a *representable functor* $y^S: \mathbf{Set} \rightarrow \mathbf{Set}$ for $S \in \mathbf{Set}$ sending $X \mapsto X^S$. We stated and proved the *Yoneda lemma*, a foundational result characterizing maps out of these representables: for an arbitrary functor $F: \mathbf{Set} \rightarrow \mathbf{Set}$, natural transformations $y^S \rightarrow F$ are in natural correspondence with elements of $F(S)$.

We then reviewed other categorical constructions in $\mathbf{Set}$, many of which carry over to the polynomial functors we introduce in the next chapter. For a set I , we can view it as a discrete category and consider a functor $X: I \rightarrow \mathbf{Set}$ as an *I -indexed family of sets* comprised of a set X_i for each $i \in I$. An I -indexed family of sets X has a *sum* (or *coproduct*), the set of pairs (i, x) with $i \in I$ and $x \in X_i$; and a *product*, the set of *dependent functions* $f: (i \in I) \rightarrow X_i$. Such a dependent function sends each $i \in I$ to an element of X_i . These constructions satisfy the universal properties of coproducts and products; moreover, products distribute over coproducts, making $\mathbf{Set}$ a completely distributive category. All these constructions and properties are inherited by $\mathbf{Set}^{\mathbf{Set}}$, whose limits (including products) and colimits (including coproducts) are computed pointwise: on one object at a time according to limits and colimits in $\mathbf{Set}$.

For other introductions to the Yoneda lemma, the category of sets, or both, take your pick of [Pie91; Bor94; Mac98; Lei14; Rie17; FS19; Che22].

1.6 Exercise solutions

Solution to Exercise 1.3.

1. The identity functor $X \mapsto X$ is represented by the set 1 : a function $1 \rightarrow X$ can be identified with an element of X , so $\mathbf{Set}(1, X) \cong X$. Alternatively, note that $X^1 \cong X$.
2. The constant functor $X \mapsto 2$ is not representable: it sends 1 to 2 , but $1^S \cong 1 \not\cong 2$ for any set S .
3. The constant functor $X \mapsto 1$ is represented by $S = 0$: there is exactly one function $0 \rightarrow X$, so $\mathbf{Set}(0, X) \cong 1$. Alternatively, note that $X^0 \cong 1$.
4. The constant functor $X \mapsto 0$ is not representable for the same reason as in #2.

5. The functor $y^{\mathbb{N}}$ that sends $X \mapsto X^{\mathbb{N}}$ is represented by $\mathbb{N}$, by definition. It should send each function $h: X \rightarrow Y$ to the function $h^{\mathbb{N}}: X^{\mathbb{N}} \rightarrow Y^{\mathbb{N}}$ that sends each $g: \mathbb{N} \rightarrow X$ to $g \circ h: \mathbb{N} \rightarrow Y$.
6. No $\mathbf{Set} \rightarrow \mathbf{Set}$ functor $X \mapsto 2^X$ is representable, for the same reason as in #2. (There *is*, however, a functor $\mathbf{Set}^{\text{op}} \rightarrow \mathbf{Set}$ sending $X \mapsto 2^X$ that is understood to be representable in a more general sense.)

Solution to Exercise 1.5.

By the construction of the components of y^f in the statement of Proposition 1.4, both vertical maps in (1.6) compose functions from S with $f: R \rightarrow S$ on the left; and by Definition 1.1, both horizontal maps compose functions to X with $h: X \rightarrow Y$ on the right. So by the associativity of composition, the diagram commutes: $(f \circ g) \circ h = f \circ (g \circ h)$ for all $g: S \rightarrow X$.

Solution to Exercise 1.7.

In each case, given $f: R \rightarrow S$, we can find the X -component $X^f: X^S \rightarrow X^R$ of the natural transformation $y^f: y^S \rightarrow y^R$ by applying Proposition 1.4, which says that X^f sends each $g: S \rightarrow X$ to $f \circ g: R \rightarrow X$.

1. Here $X^{\text{id}_S}: X^S \rightarrow X^S$ is the identity function.
2. If $f: 2 \rightarrow 1$ is the unique function, then $X^f: X^1 \rightarrow X^2$ sends each $g \in X$ (i.e. function $g: 1 \rightarrow X$) to the function that maps both elements of 2 to g . We can think of X^f as the diagonal $X \rightarrow X \times X$.
3. If $f: 1 \rightarrow 2$ sends $1 \mapsto 1$, then $X^f: X^2 \rightarrow X^1$ sends each $g: 2 \rightarrow X$ to $g(1)$, viewed as a function $1 \rightarrow X$. We can think of X^f as the left projection $X \times X \rightarrow X$.
4. If $f: 1 \rightarrow 2$ sends $1 \mapsto 2$, then $X^f: X^2 \rightarrow X^1$ sends each $g: 2 \rightarrow X$ to $g(2)$, viewed as a function $1 \rightarrow X$. We can think of X^f as the right projection $X \times X \rightarrow X$.
5. Here $X^f: X^5 \rightarrow X^0 \cong 1$ is the unique function.
6. If $f: \mathbb{N} \rightarrow \mathbb{N}$ sends $n \mapsto n+1$, then $X^f: X^{\mathbb{N}} \rightarrow X^{\mathbb{N}}$ sends each $g: \mathbb{N} \rightarrow X$ to the function $h: \mathbb{N} \rightarrow X$ defined by $h(n) := g(n+1)$ for all $n \in \mathbb{N}$. We can think of X^f as removing the first term of an infinite sequence of elements $(g(0), g(1), g(2), \dots)$ of X to obtain a new sequence $(g(1), g(2), g(3), \dots)$.

Solution to Exercise 1.8.

1. This is just a generalization of Exercise 1.7 #1. For any set X , the X -component $X^{\text{id}_S}: X^S \rightarrow X^S$ of y^{id_S} sends each $h: S \rightarrow X$ to $\text{id}_S \circ h = h$, so X^{id_S} is the identity natural transformation on X^S . Hence y^{id_S} is the identity on y^S .
2. It suffices to show componentwise that $X^g \circ X^f = X^{f \circ g}$ for every set X . Indeed, X^g sends each $h: T \rightarrow X$ to $g \circ h$; then X^f sends $g \circ h$ to $f \circ (g \circ h) = X^{f \circ g}(h)$.

Solution to Exercise 1.12.

1. It suffices to verify that the naturality square

$$\begin{array}{ccc}
 X^S & \xrightarrow{h^S} & Y^S \\
 F(-)(a) \downarrow & & \downarrow F(-)(a) \\
 F(X) & \xrightarrow{F(h)} & F(Y)
 \end{array}$$

commutes for all $h: X \rightarrow Y$. The top map h^S sends any $g: S \rightarrow X$ to $g \circ h$ (Definition 1.1), which is then sent to $F(g \circ h)(a)$ by the right map. Meanwhile, the left map sends g to $F(g)(a)$, which is then sent to $F(h)(F(g)(a))$ by the bottom map. So by the functoriality of F , the square commutes.

2. We show that the maps $m \mapsto m_S(\text{id}_S)$ and $a \mapsto m^a$ defined in the proof of Lemma 1.10 are mutually inverse. First, we show that for any natural transformation $m: y^S \rightarrow F$, we have $m^{m_S(\text{id}_S)} = m$. Given a set X , the X -component of $m^{m_S(\text{id}_S)}$ sends each $g: S \rightarrow X$ to $F(g)(m_S(\text{id}_S))$; it suffices to show that this is also where the X -component of m sends g . Indeed, by the naturality of m , the square

$$\begin{array}{ccc} S^S & \xrightarrow{g^S} & X^S \\ m_S \downarrow & & \downarrow m_X \\ F(S) & \xrightarrow{F(g)} & F(X) \end{array}$$

commutes, so in particular, following $\text{id}_S \in S^S$ around this diagram, we have

$$F(g)(m_S(\text{id}_S)) = m_X(g^S(\text{id}_S)) = m_X(\text{id}_S \circ g) = m_X(g). \quad (1.43)$$

In the other direction, we show that for any $a \in F(S)$, we have $m_S^a(\text{id}_S) = a$: by construction, $m_S^a: S^S \rightarrow F(S)$ sends id_S to $F(\text{id}_S)(a) = a$.

3. It suffices to show that given functors $F, G: \mathbf{Set} \rightarrow \mathbf{Set}$ and a natural transformation $\alpha: F \rightarrow G$, the naturality square

$$\begin{array}{ccc} \mathbf{Set}^{\mathbf{Set}}(y^S, F) & \xrightarrow{\sim} & F(S) \\ \downarrow \alpha_S & & \downarrow \alpha_S \\ \mathbf{Set}^{\mathbf{Set}}(y^S, G) & \xrightarrow{\sim} & G(S) \end{array}$$

commutes. The top map sends any $m: y^S \rightarrow F$ to $m_S(\text{id}_S)$, which in turn is sent by the right map to $\alpha_S(m_S(\text{id}_S)) = (m \circ \alpha)_S(\text{id}_S)$. This is also where the bottom map sends $m \circ \alpha$, so the square commutes.

4. It suffices to show that given a function $g: S \rightarrow X$, the naturality square

$$\begin{array}{ccc} \mathbf{Set}^{\mathbf{Set}}(y^S, F) & \xrightarrow{\sim} & F(S) \\ \downarrow y^g \circ - & & \downarrow F(g) \\ \mathbf{Set}^{\mathbf{Set}}(y^X, F) & \xrightarrow{\sim} & F(X) \end{array}$$

commutes. The left map sends any $m: y^S \rightarrow F$ to $y^g \circ m$, which is sent by the bottom map to $(y^g \circ m)_X(\text{id}_X) = m_X(X^g(\text{id}_X)) = m_X(g \circ \text{id}_X) = m_X(g)$. Meanwhile, the top map sends m to $m_S(\text{id}_S)$, which is sent by the right map to $F(g)(m_S(\text{id}_S))$. So the square commutes by (1.43).

5. Take $F := y^T$ in Lemma 1.10 so that $F(S) = S^T$; the desired isomorphism follows.

Solution to Exercise 1.17.

1. Observe that $x \in 1 = \{1\}$ if and only if $x = 1$, so $\sum_{i \in I} 1 = \{(i, 1) \mid i \in I\}$. Then the function $I \rightarrow \sum_{i \in I} 1$ that sends each $i \in I$ to $(i, 1)$ is an isomorphism.

2. It suffices to show there is a unique dependent function $f: (i \in I) \rightarrow 1$. As $1 = \{1\}$, such a function f must always send $i \in I$ to 1. This uniquely characterizes f , so there is indeed only one such dependent function.
3. Yes: since I is empty, there are no $i \in I$. So it is true that $X_i = 1$ holds whenever $i \in I$ holds, because $i \in I$ never holds. We say that this sort of statement is *vacuously true*.
4. As $I = 0 = \emptyset$, we have $\sum_{i \in \emptyset} X_i = \sum_{i \in I} 1 \cong I = 0$, where the equation on the left follows from #3 and the isomorphism in the middle follows from #1.
5. As $I = \emptyset$, we have $\prod_{i \in \emptyset} X_i = \prod_{i \in I} 1 \cong 1$, where the equation on the left follows from #3 and the isomorphism on the right follows from #2.

Solution to Exercise 1.19.

1. Any function $h: \sum_{i \in I} X_i \rightarrow Y$ for which $\iota_j \circ h = g_j$ for all $j \in I$ must satisfy $h(j, x) = h(\iota_j(x)) = g_j(x)$ for all $j \in I$ and $x \in X_j$. This uniquely characterizes h , so if we define $h(j, x) := g_j(x)$ we are done.
2. Any function $h: Y \rightarrow \prod_{i \in I} X_i$ for which $h \circ \pi_j = g_j$ for all $j \in I$ must satisfy $h(y)_j = \pi_j(h(y)) = g_j(y)$ for all $y \in Y$ and $j \in I$. This uniquely characterizes $h(y)$ and thus h , so if we define $h(y): (i \in I) \rightarrow X_i$ to be the dependent function given by $i \mapsto g_i(y)$ we are done.

Solution to Exercise 1.21.

1. It suffices to show that the following square, where the vertical maps are the inclusions, commutes for all $j \in I$:

$$\begin{array}{ccc} X_j & \xrightarrow{f_j} & Y_j \\ \downarrow \iota_j & & \downarrow \iota'_j \\ \sum_{i \in I} X_i & \xrightarrow{\sum_{i \in I} f_i} & \sum_{i \in I} Y_i \end{array}$$

Given $x \in X_j$, the left inclusion map sends x to (j, x) , which the bottom sum of maps sends to $(j, f_j(x))$. Meanwhile, the top map sends x to $f_j(x)$, which the right inclusion map again sends to $(j, f_j(x))$.

2. It suffices to show that the following square, where the vertical maps are the projections, commutes for all $j \in I$:

$$\begin{array}{ccc} \prod_{i \in I} X_i & \xrightarrow{\prod_{i \in I} f_i} & \prod_{i \in I} Y_i \\ \downarrow \pi_j & & \downarrow \pi'_j \\ X_j & \xrightarrow{f_j} & Y_j \end{array}$$

Given $g: (i \in I) \rightarrow X_i$ in $\prod_{i \in I} X_i$, the top product of maps sends g to fg , which the right projection map sends to $f_j(g(j))$. Meanwhile, the left projection map sends g to $g(j)$, which the bottom map again sends to $f_j(g(j))$.

Solution to Exercise 1.22.

1. The second projection $\pi_2(i, x) = x$ sends each pair $p := (i, x) \in \sum_{i \in I} X_i$ to x , an element of X_i . Note that we can write i in terms of p as $\pi_1(p)$. This allows us to write the signature of π_2 as $\pi_2: (p \in \sum_{i \in I} X_i) \rightarrow X_{\pi_1(p)}$.

2. Let $S := \{s: I \rightarrow \sum_{i \in I} X_i \mid s \circ \pi_1 = \text{id}_I\}$ be the set of sections of π_1 . To show that $\prod_{i \in I} X_i \cong S$, we will exhibit maps in either direction and show that they are mutually inverse. For each $f: (i \in I) \rightarrow X_i$ in $\prod_{i \in I} X_i$, we have $f(i) \in X_i$ for $i \in I$, so we can define a function $s_f: I \rightarrow \sum_{i \in I} X_i$ that sends $i \mapsto (i, f(i))$. Then $\pi_1(s_f(i)) = \pi_1(i, f(i)) = i$, so s_f is a section of π_1 . Hence $f \mapsto s_f$ is a map $\prod_{i \in I} X_i \rightarrow S$.
- In the other direction, for each section $s: I \rightarrow \sum_{i \in I} X_i$ we have $\pi_1(s(i)) = i$ for $i \in I$, so we can write $s(i)$ as an ordered pair $(i, \pi_2(s(i)))$ with $\pi_2(s(i)) \in X_i$. Hence we can define a dependent function $f_s: (i \in I) \rightarrow X_i$ sending $i \mapsto \pi_2(s(i))$. Then $s \mapsto f_s$ is a map $S \rightarrow \prod_{i \in I} X_i$. By construction $s_{f_s}(i) = (i, f_s(i)) = (\pi_1(s(i)), \pi_2(s(i))) = s(i)$ and $f_{s_f}(i) = \pi_2(s_f(i)) = \pi_2(i, f(i)) = f(i)$, so these maps are mutually inverse.

Solution to Exercise 1.25.

1. Here are the instructions for choosing an element of B .
To choose an element of B :
 1. for each element $i \in I$:
 - 1.1. choose an element $j \in J(i)$;
 - 1.2. for each element $k \in K(i, j)$:
 - 1.2.1. choose an element of $X(i, j, k)$.
2. Given $I := \{1, 2\}$, $J(1) := \{j\}$, $J(2) := \{j, j'\}$, $K(1, j) := \{k_1, k_2\}$, $K(2, j) := \{k_1\}$, $K(2, j') := \{k'\}$, and $X(i, j, k) := \{x, y\}$ for all i, j, k , our goal is to count the number of elements in B . To compute the cardinality of B , we can use the fact that the cardinality of a sum (resp. product) is the sum (resp. product) of the cardinalities of the summands (resp. factors). So

$$\begin{aligned}
 |B| &= \prod_{i \in I} \sum_{j \in J(i)} \prod_{k \in K(i, j)} |X(i, j, k)| \\
 &= \prod_{i \in \{1, 2\}} \sum_{j \in J(i)} \prod_{k \in K(i, j)} 2 \\
 &= \left(\sum_{j \in J(1)} 2^{|K(1, j)|} \right) \left(\sum_{j \in J(2)} 2^{|K(2, j)|} \right) \\
 &= (2^2)(2^1 + 2^1) = 16.
 \end{aligned}$$

3. Here are three of the elements of B (you may have written down others):
 - $(1 \mapsto (j, k_1 \mapsto x, k_2 \mapsto y), 2 \mapsto (j', k' \mapsto x))$
 - $(1 \mapsto (j, k_1 \mapsto y, k_2 \mapsto y), 2 \mapsto (j, k_1 \mapsto y))$
 - $(1 \mapsto (j, k_1 \mapsto y, k_2 \mapsto x), 2 \mapsto (j', k' \mapsto y))$

Solution to Exercise 1.35.

On the left-hand side, we have a 2-fold product: a single object times a 2-fold sum. So we should let $I := 2$ and let $J(1) := 1$, with $X(1, 1) := X$; and $J(2) := 2$, with $X(2, 1) := Y$ and $X(2, 2) := Z$. Then

$$X \times (Y + Z) \cong \prod_{i \in 2} \sum_{j \in J(i)} X(i, j) \cong \sum_{\substack{\bar{j} \in \prod_{i \in 2} J(i)}} \prod_{i \in 2} X(i, \bar{j}(i)) \cong \sum_{\substack{\bar{j} \in \prod_{i \in 2} J(i)}} X(1, \bar{j}(1)) \times X(2, \bar{j}(2)),$$

where the middle isomorphism follows from (1.32). The set $\prod_{i \in 2} J(i)$ contains two functions: $(1 \mapsto 1, 2 \mapsto 1)$ and $(1 \mapsto 1, 2 \mapsto 2)$. So we can rewrite the right-hand side as

$$X(1, 1) \times X(2, 1) + X(1, 1) \times X(2, 2) \cong X \times Y + X \times Z.$$

Solution to Exercise 1.36.

1. By applying (1.32), we can rewrite

$$\sum_{i \in I} \prod_{j \in J(i)} \sum_{k \in K(i, j)} X(i, j, k)$$

as

$$\sum_{i \in I} \sum_{\bar{k} \in \prod_{j \in J} K(i, j)} \prod_{j \in J(i)} X(i, j, \bar{k}j).$$

2. By applying (1.32), we can rewrite

$$\prod_{i \in I} \sum_{j \in J(i)} \prod_{k \in K(i, j)} X(i, j, k)$$

as

$$\sum_{\bar{j} \in \prod_{i \in I} J(i)} \prod_{i \in I} X(i, \bar{j}i) \prod_{k \in K(i, \bar{j}i)} X(i, \bar{j}i, k).$$

3. By applying (1.32), we can rewrite

$$\prod_{i \in I} \prod_{j \in J(i)} \sum_{k \in K(i, j)} X(i, j, k)$$

once as

$$\prod_{i \in I} \sum_{\bar{k} \in \prod_{j \in J} K(i, j)} \prod_{j \in J(i)} X(i, j, \bar{k}j)$$

and then again as

$$\sum_{\bar{k} \in \prod_{i \in I} \prod_{j \in J} K(i, j)} \prod_{i \in I} \prod_{j \in J(i)} X(i, j, \bar{k}(i, j)).$$

Solution to Exercise 1.41.

It suffices to show that for all $X \in \mathbf{Set}$, there is an isomorphism

$$\sum_{a \in A} F(X) \cong (AF)(X)$$

natural in X . The left-hand side is the set $\{(a, s) \mid a \in A \text{ and } s \in F(X)\} \cong A \times F(X)$, while the right-hand side is also naturally isomorphic to the set $A(X) \times F(X) \cong A \times F(X)$. Alternatively, since $\mathbf{Set}^{\mathbf{Set}}$ is completely distributive by Proposition 1.40, the result also follows from (1.32), with $I := 2, J(1) := A, J(2) := 1, X(1, a) := 1$ (the constant functor) for $a \in A$, and $X(2, 1) := F$:

$$AF \cong \left(\sum_{a \in A} 1 \right) F \cong \prod_{i \in 2} \sum_{j \in J(i)} X(i, j) \cong \sum_{\bar{j} \in \prod_{i \in 2} J(i)} \prod_{i \in 2} X(i, \bar{j}i) \cong \sum_{a \in A} 1 \times F \cong \sum_{a \in A} F.$$

Here we used the fact that $A \cong \sum_{a \in A} 1$ from Exercise 1.17 #1 (there we proved the statement for sets, but the same statement for the corresponding constant set-valued functors follows immediately).

Solution to Exercise 1.42.

1. It suffices to show that for all $X \in \mathbf{Set}$, there is an isomorphism

$$\prod_{i \in I} y(X) \cong y^I(X)$$

natural in X . We have that $y(X) \cong X$ and that $y^I(X) \cong X^I$. So both sides are naturally isomorphic to the set of functions $I \rightarrow X$.

2. It suffices to show that for all $X \in \mathbf{Set}$, there is an isomorphism

$$\prod_{i \in I} y^A(X) \cong y^{IA}(X)$$

natural in X . We have that $y^A(X) \cong X^A$, so $\prod_{i \in I} y^A(X) \cong (X^A)^I$, and that $y^{IA}(X) \cong X^{IA}$. By currying, both sides are naturally isomorphic to the set of functions $I \times A \rightarrow X$.

2. Polynomial functors

In this chapter, we will formally introduce our objects of study: polynomial functors. In addition to the set-theoretic perspective, we will present several more concrete ways to think about polynomials to aid intuition, which we will use throughout the rest of this book. We keep the mathematical content of this chapter fairly light, preferring to solidify our conceptual understanding of polynomials, before advancing to deeper categorical content.

2.1 Introducing polynomial functors

Definition 2.1 (Polynomial functor). A *polynomial functor* (or simply *polynomial*) is a functor $p: \mathbf{Set} \rightarrow \mathbf{Set}$ such that there exists a set I , an I -indexed family of sets $(p[i])_{i \in I}$, and an isomorphism

$$p \cong \sum_{i \in I} y^{p[i]}$$

to the corresponding I -indexed sum of representables.

So, up to isomorphism, a polynomial functor is just a sum of representables.

Remark 2.2. Given sets $I, A \in \mathbf{Set}$, it follows from Exercise 1.41 that we have an isomorphism of polynomials

$$\sum_{i \in I} y^A \cong I y^A.$$

So when we write down a polynomial, we will often combine identical representable summands y^A by writing them in the form $I y^A$. In particular, the constant functor 1 is a representable functor ($1 \cong y^0$), so every constant functor I is a polynomial functor: $I \cong \sum_{i \in I} 1$.

Exercise 2.3. Consider the polynomial $q := y^8 + 4y$.

1. Does the polynomial q have a representable summand y^2 ?
2. Does the polynomial q have a representable summand y ?
3. Does the polynomial q have a representable summand $4y$?

Example 2.4. Consider the polynomial $p := y^2 + 2y + 1$. It denotes a functor $\mathbf{Set} \rightarrow \mathbf{Set}$; where does this functor send the set $X := \{a, b\}$? To be precise, we will rather verbosely say that $I := 4$ and

$$p[1] := 2, \quad p[2] := 1, \quad p[3] := 1, \quad \text{and} \quad p[4] := 0$$

so that $p \cong \sum_{i \in I} y^{p[i]}$. Now we have

$$\begin{aligned} p(X) &\cong \sum_{i \in 4} \{a, b\}^{p[i]} \\ &= \{a, b\}^2 + \{a, b\}^1 + \{a, b\}^1 + \{a, b\}^0 \\ &\cong \{(1, (a, a)), (1, (a, b)), (1, (b, a)), (1, (b, b)), (2, (a)), (2, (b)), \\ &\quad (3, (a)), (3, (b)), (4, ())\}. \end{aligned}$$

Above, we denote each function $f: p[i] \rightarrow \{a, b\}$ in the set $\{a, b\}^{p[i]}$ by the n -tuple $(f(1), \dots, f(n))$ whenever $p[i] := n$. For ease of reading, we may drop the parentheses around these n -tuples to obtain the equivalent set

$$p(X) \cong \{(1, a, a), (1, a, b), (1, b, a), (1, b, b), (2, a), (2, b), (3, a), (3, b), (4)\}.$$

As we might expect, the set $p(X)$ contains $2^2 + 2 + 2 + 1 = 9$ elements, equal to the value obtained when we plug $|X| = 2$ into the original polynomial p when we interpret its coefficients and exponents as numbers instead of sets.

In general, a polynomial $p := \sum_{i \in I} y^{p[i]}$ applied to a set X expands to

$$\sum_{i \in I} X^{p[i]}$$

and can be thought of as the set of all pairs comprised of an element of I and a function $p[i] \rightarrow X$ or, equivalently, a $p[i]$ -tuple of elements of X .

Exercise 2.5. In the verbose style of Example 2.4, write out all the elements of $p(X)$ for p and X as follows (if there are infinitely many, denote the set $p(X)$ some other way):

1. $p := y^3$ and $X := \{4, 9\}$.
2. $p := 3y^2 + 1$ and $X := \{a\}$.
3. $p := 0$ and $X := \mathbb{N}$.
4. $p := 4$ and $X := \mathbb{N}$.
5. $p := y$ and $X := \mathbb{N}$.

The following proposition shows how the polynomial functor p itself determines the set I over which we sum up representables to obtain p .

Proposition 2.6. Let $p := \sum_{i \in I} y^{p[i]}$ be an arbitrary polynomial functor. Then $I \cong p(1)$, so there is an isomorphism of functors

$$p \cong \sum_{i \in p(1)} y^{p[i]}. \quad (2.7)$$

Proof. We need to show that $I \cong p(1)$; the latter claim follows directly. In Exercise 1.17 #1, we showed that $I \cong \sum_{i \in I} 1$, so it suffices to show that $(y^{p[i]})(1) \cong 1$ for all $i \in I$. Indeed, $1^{p[i]} \cong 1$ because there is a unique function $p[i] \rightarrow 1$ for each $p[i]$. $\square$

We can draw an analogy between Proposition 2.6 and evaluating $p(1)$ for a polynomial p from high school algebra, which yields the sum of the coefficients of p . The notation in (2.7) will be how we denote arbitrary polynomials from now on, and we will use the following terms to denote the sets $p(1)$ and $p[i]$ for $i \in p(1)$ on which a polynomial p depends.

Definition 2.8 (Position and direction). Given a polynomial functor

$$p \cong \sum_{i \in p(1)} y^{p[i]},$$

we call an element $i \in p(1)$ a *position of p* or a *p -position*, and we call an element $a \in p[i]$ a *direction of p at i* or a *$p[i]$ -direction*. We call $p(1)$ the *position-set of p* and $p[i]$ the *direction-set of p at i* .

Note that the position-set $p(1)$ along with the $p(1)$ -indexed family of direction-sets $p[-]: p(1) \rightarrow \mathbf{Set}$ uniquely characterize a polynomial p up to isomorphism. Throughout this book, we will often specify a polynomial by giving its positions and its directions at each position.

Exercise 2.9. We saw in Proposition 2.6 how to interpret the position-set $p(1)$ of a polynomial p , e.g. $p := y^3 + 3y^2 + 4$, as the sum of the coefficients of p : here $p(1) \cong 1 + 3 + 4 \cong 8$. How might you interpret $p(0)$?

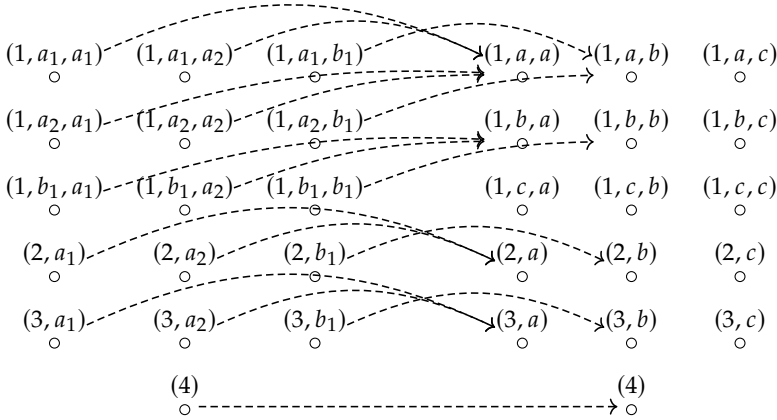
As a functor $\mathbf{Set} \rightarrow \mathbf{Set}$, a polynomial should act on functions as well as on sets. Below, we explain how.

Proposition 2.10. Let p be an arbitrary polynomial functor, which our notation lets us write as $p \cong \sum_{i \in p(1)} y^{p[i]}$, and let $f: X \rightarrow Y$ be an arbitrary function. Then $p(f): p(X) \rightarrow p(Y)$ sends each $(i, g) \in p(X)$, with $i \in p(1)$ and $g: p[i] \rightarrow X$, to $(i, g \circ f)$ in $p(Y)$.

Proof. For each $i \in p(1)$, by Definition 1.1, the functor $y^{p[i]}$ sends f to the function $X^{p[i]} \rightarrow Y^{p[i]}$ mapping each $g: p[i] \rightarrow X$ to $g \circ f: p[i] \rightarrow Y$. So the sum of these functors over $i \in p(1)$ sends each $(i, g) \in p(X)$ to $(i, g \circ f) \in p(Y)$. $\square$

Example 2.11. Suppose $p := y^2 + 2y + 1$. Let $X := \{a_1, a_2, b_1\}$ and $Y := \{a, b, c\}$, and let $f: X \rightarrow Y$ be the function sending $a_1, a_2 \mapsto a$ and $b_1 \mapsto b$. The induced function $p(f): p(X) \rightarrow p(Y)$, according

to Proposition 2.10, is shown below:



Exercise 2.12. Let $p := y^2 + y$. Choose a function $f: 1 \rightarrow 2$ and write out the induced function $p(f): p(1) \rightarrow p(2)$.

2.2 Special classes of polynomial functors

Here we describe several special classes of polynomials. We have already defined two such special classes: *representables* and *constants*. A *representable polynomial* (or simply a *representable*) is a representable functor, i.e. a polynomial functor isomorphic to y^A for some set A . Meanwhile, a *constant polynomial* (or simply a *constant*) is a constant functor, i.e. a polynomial functor isomorphic to I , interpreted as a functor, for some set I .

Exercise 2.13.

1. Characterize when a polynomial p is *representable* in terms of its positions and/or its directions.
2. Characterize when a polynomial p is *constant* in terms of its positions and/or its directions.

Like constants, the other two special classes of polynomials we define here will share their names with their algebraic analogues. Throughout, let $p \cong \sum_{i \in p(1)} y^{p[i]}$ be a polynomial functor.

Definition 2.14 (Linear). We say that p is *linear*¹ if $p \cong Iy$ for some set I .

Definition 2.15 (Monomial). We say that p is a *monomial* if $p \cong Iy^A$ for sets I and A .

Example 2.16. Every constant polynomial $I \cong Iy^0$ is a monomial, as is every linear polynomial $Iy \cong Iy^1$ and every representable $y^A \cong 1y^A$. On the other hand, there are monomials that are neither constant, linear, nor representable, such as $2y^2$ or $\mathbb{N}y^{\mathbb{R}}$. Moreover, there are polynomials that are not monomials, such as $y^4 + 3$ or $\sum_{n \in \mathbb{N}} y^n$.

There is only one polynomial that is both constant and linear, namely $0 \cong 0y$. Similarly, there is only one polynomial (up to isomorphism) that is both constant and representable, namely $1 \cong y^0$. Finally, there is only one polynomial (up to isomorphism) that is both linear and representable, namely the identity functor $1y \cong y \cong y^1$.

In general, every set S has a corresponding constant polynomial S , linear polynomial Sy , and representable polynomial y^S ; and as long as $|S| \geq 2$, these are all distinct.

Exercise 2.17.

1. Characterize when a polynomial p is *linear* in terms of its positions and/or its directions.
2. Characterize when a polynomial p is a *monomial* in terms of

¹Unlike linear polynomials from high school algebra (which are really *affine linear functions* rather than necessarily *linear functions*), our linear polynomial functors have no (nonzero) constant terms: they always send 0 to 0. A polynomial is *affine* if it is of the form $Ay + B$, though we will not use this concept much in the book.

its positions and/or its directions.

Later on in Section 5.1, we will see how all four of these special classes of polynomials arise from various adjunctions.

2.3 Interpreting positions and directions

Let us make an informal digression on how we will think about positions and directions of polynomials in this book. While this section has little mathematical content, the intuition we build here will guide us as we delve into the deeper theory of polynomials and their applications to modeling interaction.

The main idea is that a *position* is some status that may be held, while the *directions* at each position are the options available when holding that status. While these positions and directions may be imagined abstractly, here we give some concrete examples.

Example 2.18 (Directions as menu options). Consider a representable and thus polynomial functor y^A for a set A . It has 1 position and the elements of A as its directions. We may think of A as a menu of options to choose from.

The menu may consist of dinner options available at a wedding; then the corresponding representable functor could be

$$y^{\{\text{chicken, beef, vegetarian}\}};$$

or it may be the menu of a text editor, in which case the representable could be

$$y^{\{\text{cut, copy, paste}\}}.$$

In both these cases, there are exactly 3 directions, so there is an isomorphism of representable functors

$$y^{\{\text{chicken, beef, vegetarian}\}} \cong y^{\{\text{cut, copy, paste}\}}.$$

Similarly, we may interpret the representable y^2 as a 2-option menu. Such menus are ubiquitous in life: yes or no, true or false,

heads or tails, 0 or 1. A 1-option menu, represented by $y^1 \cong y$, is also familiar as an unavoidable choice, the only option: “sorry, you just have to go through it.” Having no options, represented by $y^0 \cong 1$, is when you actually do not get through it: an impossible decision, a “dead end.”

In contrast, we may interpret the representable $y^{[0,1]}$ as a menu with an infinite range of options: a slider with one end labeled 0 and the other labeled 1, able to take on any value in between.

For consistency, we will favor the term “direction” over “option” when referring to the elements of A for a summand y^A of a polynomial. Nevertheless, when we think of a polynomial’s directions, we will often think of them as options to choose from.

Example 2.18 shows how we may interpret the directions of a single representable summand as options in a menu. By having multiple representable summands—one for each position—a polynomial may capture more general scenarios with a range of possible menus.

Example 2.19 (Modeling with a polynomial). Consider a coin jar with a slot that may be open or closed. When the slot is open, the jar may accept a penny, a nickel, a dime, or a quarter—there are 4 options to choose from. When the slot is closed, the jar may not accept any coins at all—there are 0 options. We may model this scenario with the polynomial

$$y^{\{\text{penny, nickel, dime, quarter}\}} + y^0 \cong y^4 + 1.$$

This polynomial has 2 positions, corresponding to the two statuses the slot could take: open or closed. To delineate these positions, we could take advantage of the fact that every singleton set is isomorphic to 1 and that $1y^A \cong y^A$ to rewrite the above polynomial as

$$\{\text{open}\}y^{\{\text{penny, nickel, dime, quarter}\}} + \{\text{closed}\}y^0 \cong y^4 + 1.$$

Exercise 2.20. Give another example of a real-world scenario that may be modeled by a polynomial with more than 1 position.

2.4 Corolla forests

We would like to have graphical depictions of our polynomials to make them easy to visualize. These will take the form of special graphs known as *corolla forests*. We build up to defining them as follows.

Our first definition will be familiar to students of graph theory, although we will add some technical details suited to our purposes.

Definition 2.21 (Rooted tree). A *rooted tree* is a directed acyclic graph with a distinguished vertex called the *root* such that there exists a unique directed path from the root to each vertex.

We allow infinitely and even uncountably many vertices and infinitely and even uncountably many edges incident to each vertex; on the other hand, each pair of vertices is connected by a (necessarily unique) path of finitely many adjacent edges.

Since all our trees will be rooted, we may refer to them simply as *trees*—roots are implied. We will draw our trees with roots at the bottom and other vertices “growing” upward.

The following terminology will be handy when working with our trees; these terms should be familiar, or at the very least they should match your intuition.

Definition 2.22 (Rooted path; height). A *rooted path* is a (directed) path in a rooted tree from its root to any vertex.

Given a vertex of a rooted tree, its *height* is the length of (i.e. number of edges in) the rooted path to that vertex.

In any rooted tree, the root has height 0, the length of the empty rooted path to the root itself; every neighbor of the root has height 1; every

neighbor of a vertex of height 1 either is the root or has height 2; and so forth.

Now we can define a special kind of tree that we will use to depict representable functors.

Definition 2.23 (Corolla). A *corolla* is a rooted tree in which every vertex aside from the root has height 1. We call these vertices the *leaves* of the corolla.

The *corolla associated to a representable functor* y^A for $A \in \mathbf{Set}$ is the corolla whose leaves are in bijection with A .

Example 2.24. Here are the corollas associated to various representables:



In Example 2.24, the roots are indicated by dots ($\bullet$), and the leaves are indicated by arrows ($\uparrow$). Because the direction-set of the representable is in bijection with the leaves of the associated corolla, we can think of each leaf as a direction, so it makes sense to draw the leaves as arrows pointing in different directions. Thinking of directions as menu options like in the previous section, we may view these corollas as mini-decision trees, indicating all the possible options we could select.

Example 2.25. The corolla associated to $y^0 \cong 1$ has *no* leaves: it is the rooted tree consisting of one vertex—the root—and no edges.

$$y^0 \cong 1.$$

By definition, the root itself is *not* a leaf, so the corolla above does in fact have 0 leaves. With no arrows pointing out, it is the corolla associated to a representable with no directions.

As each representable functor has an associated corolla, each polynomial functor will have an associated disjoint union of corollas that we call a *corolla forest*.

Definition 2.26 (Corolla forest). A *corolla forest* is a disjoint union of corollas.

The *corolla forest associated to a polynomial functor* $p \cong \sum_{i \in p(1)} y^{p[i]}$ is the disjoint union of the corollas associated to each representable summand $y^{p[i]}$ of p . When we draw the corolla forest associated to p , we may say that we are *drawing p as a (corolla) forest*. We call the corollas in this forest corresponding to p -positions *p -corollas* and the leaves corresponding to $p[i]$ -directions *$p[i]$ -leaves*.

Example 2.27. We may draw $p := y^2 + 2y + 1$ as a forest like so:

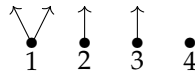


Each of the 4 corollas in (2.28) corresponds to one of the 4 representable summands of p . The 4 roots in (2.28) correspond to the 4 positions of p , and the leaves connected to each root correspond to the directions at each position. Note that p has 1 position with 2 directions, 2 positions with 1 direction each, and 1 position with 0 directions. Hence (2.28) is the disjoint union of 1 corolla with 2 leaves, 2 corollas with 1 leaf each, and 1 corolla with 0 leaves.

Since $p(1) \cong 4$, we could label the positions of p with the elements of $4 = \{1, 2, 3, 4\}$ so that

$$p[1] = 2, \quad p[2] = 1, \quad p[3] = 1, \quad p[4] = 0.$$

Then we could give these same labels to the roots in (2.28):



Similarly, we could label the directions and their corresponding leaves, but we will reserve leaf labels for another purpose.

Exercise 2.29. Consider the polynomial $p := 2y^3 + 2y + 1$.

1. Draw p as a corolla forest.
2. How many roots does this forest have?
3. How many positions of p do these roots represent?
4. For each p -corolla, say how many leaves it has.
5. For each p -position, say how many directions it has.

The position-set or any of the direction-sets of a polynomial may be infinite. This makes their associated corolla forests impossible to draw precisely, but they may be approximated. We sketch the polynomial $y^3 + \mathbb{N}y^{[0,1]}$ as a forest below.



Exercise 2.30. If you were a suitor choosing the corolla forest you love, aesthetically speaking, which would strike your interest?

Answer by selecting the associated polynomial:

1. $y^2 + y + 1$
2. $y^3 + 3y^2 + 3y + 1$
3. y^2
4. $y + 1$
5. $(\mathbb{N}y)^{\mathbb{N}}$
6. Sy^5 for some set S
7. $y^{100} + y^2 + 3y$
8. $y + 2y^4 + 3y^9 + 4y^{16} + \dots$
9. Your polynomial's name p here.

Any reason for your choice? Draw a sketch of your forest.

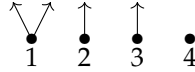
Corolla forests help us visualize the positions and directions of polynomials, and they will especially come in handy in the next chapter, when we describe the morphisms between our polynomials and how they interact with positions and directions. They may also depict the elements of a polynomial functor applied to a given set, as follows. We

have seen that for a polynomial p and a set X ,

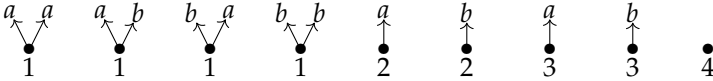
$$p(X) \cong \sum_{i \in p(1)} X^{p[i]} \cong \{(i, f) \mid i \in p(1), f: p[i] \rightarrow X\}.$$

So an element of $p(X)$ is a p -position i along with a function f that maps each direction at i to an element of X . Equivalently, it is a p -corolla along with a function that maps each of its leaves to an element of X . Then to draw an element $(i, f) \in p(X)$, we simply need to draw the p -corolla corresponding to i and label its leaves with elements of X according to f .

Example 2.31. In Example 2.27, we drew $p := y^2 + 2y + 1$ as a corolla forest like so:



Previously, in Example 2.4, we wrote out all 9 elements of p applied to the set $X := \{a, b\}$ as tuples. We could draw them out instead—an element of $p(X)$ may be depicted as one of the four corollas above with each of its leaves labeled with an element of X :



Throughout this book, we will generally use corolla forests to depict polynomials with relatively small numbers of positions or directions, where drawing out entire corolla forests is manageable. Later, we will study how building larger rooted trees out of these corollas corresponds to conducting various categorical operations on our polynomials.

2.5 Polyboxes

Before we conclude this chapter, we introduce one more tool for visualizing polynomials whose full power will not be evident until later.

Throughout this book, we may depict a polynomial p as a pair of boxes stacked on top of each other, like so:

$$\begin{array}{c} p[-] \\ p(1) \end{array} \begin{array}{|c|} \hline \\ \hline \\ \hline \end{array}$$

p

We call this picture the *polyboxes* for p . Think of these boxes as cells in a spreadsheet. The bottom cell, or the *position box*, is restricted to values in the set $p(1)$ (as indicated by the label to its left)—it must be filled with a p -position, say $i \in p(1)$:

$$\begin{array}{c} p[-] \\ p(1) \end{array} \begin{array}{|c|} \hline \\ \hline i \\ \hline \end{array}$$

p

The top cell, or the *direction box*, cannot be filled until the position box below it is. Once the position box contains a p -position i , the direction box must be filled with a $p[i]$ -direction, say $a \in p[i]$:

$$\begin{array}{c} p[-] \\ p(1) \end{array} \begin{array}{|c|} \hline a \\ \hline i \\ \hline \end{array}$$

p

The $p[-]$ label to the left of the direction box reminds us that the a within it is an element of $p[i]$, where i is the entry in the position box. Once we are accustomed to polyboxes, we will often drop these reminder labels, so that

$$\begin{array}{|c|} \hline a \\ \hline i \\ \hline \end{array}$$

p

serves as a graphical shorthand for the statement “consider a polynomial functor p with position $i \in p(1)$ and direction $a \in p[i]$.”

Viewing polynomials as these restricted two-cell spreadsheets reinforces the idea that directions are like menu options: imagine a drop-down menu for the direction box above a filled position box that lists all the directions to choose from at the given position. Polyboxes also help us conceptualize the possible pairs of positions and directions of a

polynomial whose corolla forest is impractical to draw, as suggested by the following example.

Example 2.32. Consider the polynomial

$$p := \sum_{r \in \mathbb{R}} y^{[-|r|, |r|]},$$

whose positions are the real numbers and whose directions at position r are the real numbers with magnitude at most $|r|$. There is no clear way to draw p as a corolla forest, but we could draw its polyboxes

$$\begin{array}{c} p[-] \begin{array}{|c|} \hline s \\ \hline \end{array} \\ p(1) \begin{array}{|c|} \hline r \\ \hline \end{array} \\ p \end{array}$$

with the condition that r and s are real numbers satisfying $|s| \leq |r|$.

We may also use polyboxes to highlight our special classes of polynomials. When a position box may only be filled with one possible entry, we shade it in like so:

$$\begin{array}{c} A \begin{array}{|c|} \hline \text{ } \\ \hline \end{array} \\ p(1) \cong 1 \begin{array}{|c|} \hline \text{ } \\ \hline \end{array} \\ p \cong y^A \end{array}$$

The idea is that if there is only one entry that could fill a given box, then it should come pre-filled—no further choice needs to be made to fill it. Here $p(1) \cong 1$, so p is *representable*; indeed, $p \cong y^A$, where A is the set of possible entries for the unfilled direction box.

Similarly, the polyboxes for a *linear* polynomial Iy , whose direction-set at each position is a singleton, can be drawn like so:

$$\begin{array}{c} \begin{array}{|c|} \hline \text{ } \\ \hline \end{array} \\ I \begin{array}{|c|} \hline \text{ } \\ \hline \end{array} \\ Iy \end{array}$$

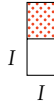
No matter what fills the position box, there is exactly one entry that could fill the direction box, so it comes pre-filled. The identity poly-

mial functor y , which is both representable and linear, therefore has the following polyboxes:



It has exactly one position and exactly one direction, so both its boxes come pre-filled.

Finally, a *constant* polynomial I for some set I has empty direction-sets. We indicate this by filling its direction box with red dots:



Because every direction-set is empty, there is nothing that may be written in the direction box. The red dots suggest an error—the direction box cannot be filled. The polynomial functor 1 , which is both representable and constant, therefore has the following polyboxes:



In the next chapter, we will introduce the morphisms between polynomial functors and see how their behavior may be depicted using polyboxes.

2.6 Summary and further reading

In this chapter, we introduced the main objects of study in this book: *polynomial functors*, which are sums of representable functors $\mathbf{Set} \rightarrow \mathbf{Set}$. We write a polynomial p as $\sum_{i \in p(1)} y^{p[i]}$, calling the elements of $p(1)$ the *positions* of p and the elements of $p[i]$ the *directions* of p at position i . As a polynomial is determined up to isomorphism by its position-set and direction-sets, we can think of the data of a polynomial as an indexed family of sets $(p[i])_{i \in p(1)}$.

A categorical perspective on polynomial-like functors (those functors that can be written as sums and products of the identity functor) in program semantics was explored by Manes and Arbib in their book [MA12], where they study algebraic approaches to computation, automata, and categorical semantics.

We highlighted four special classes of polynomials (here I and A are sets):

- constants I , whose direction-sets are all empty;
- linear polynomials Iy , whose direction-sets are all singletons;
- representables y^A , whose position-sets are singletons;
- monomials Iy^A , whose direction-sets all have the same cardinality.

Throughout this book, we will use polynomials to model decision-making agents that hold positions and take directions from those positions. We can draw a polynomial p graphically as a *corolla forest*, with a *corolla* (a rooted tree whose non-root vertices are all leaves) for every p -position i that has a leaf for every $p[i]$ -direction. We can also depict a polynomial as a *polybox picture*, resembling two stacked cells in a spreadsheet, to be filled in with an element of i below and an element of $p[i]$ above.

This book deals far more with the coalgebraic aspects of polynomial functors than algebraic aspects. For the latter, including a categorical treatment of wellfounded trees and their connection to polynomial functors, see the work of Moerdijk and Palmgren [MP00], where they construct initial algebras for polynomial (and other) endofunctors, as well as that of Gambino and Hyland [GH03] who studied dependent polynomial functors, with applications to dependent type theory.

There are many fine sources on polynomial functors. Some of the computer science literature is more relaxed about what a polynomial is. For example, the “coalgebra community” often defines a polynomial to include finite power set functors (see e.g. [Jac17]). Girard [Gir88] discussed multivariate polynomials (which are bicomodules between discrete categories; see Definition 8.96), calling them *normal functors*; these have influenced the category-theoretic underpinnings of type theory and functional programming. Carboni and Johnstone [CJ95] explored *familiarily representable* functors, defined as the connected limit preserving functors from a presheaf category to the category of sets; these are

precisely right comodules, a concept we will introduce in Definition 8.86.

Other computer science communities use the same definition of polynomial as ours, but refer to it as a *container* and use different words for its positions (they call them “shapes”) and directions (they call them “positions,” a name clash for which we apologize). See e.g. [Abb03; AAG05], which provided a categorical framework for modeling data types in functional programming.

Still other domains define *polynomial functors* to mean different notions of functors entirely; we mention these here both to alleviate confusion and to point the intrepid reader to fascinating foundational work in other areas. In algebraic topology and homological algebra, Eilenberg and MacLane introduced polynomial functors (under the name *functors of degree k*) between categories of abelian groups to study group homology in [EM54]; these were later generalized by Dwyer to study homological stability in [Dwy80], by Goodwillie to *k -excisive functors* from spaces to spaces or spectra for his eponymous calculus in [Goo90], and by Lurie to functors between certain ∞ -categories in [Lur09]. In representation theory, Schur introduced in [Sch27] what Macdonald later recognized as polynomial functors between categories of vector spaces, generalizing them in [Mac80] (in fact, as of the writing of this text, this is the definition that appears on the Wikipedia page for “Polynomial functor” [Wik24]). The common thread between all these concepts is that they generalize a kind of functor that is *linear* to a kind of functor that is *polynomial*.

A good categorical introduction to a class of polynomial functors more general than those in this text, along with an extensive bibliography of references, can be found in the work of Gambino and Kock [GK12]. More extensive notes are in [Koc]; in particular the Historical remarks work section on page 5 provides a nice survey. Kock credits André Joyal for his guidance on the subject, as well as with discovering the presentation of polynomial functors via set maps and defining the closely-related notion of combinatorial species [Joy81].

A reader may also be interested in the Workshops on Polynomial Functors organized by the Topos Institute: <https://topos.institute/events/p-func-workshop/>.

2.7 Exercise solutions

Solution to Exercise 2.3.

1. No, q does not have y^2 as a representable summand.
2. Yes, q does have y as a representable summand.
3. No, q does not have $4y$ as a representable summand, because $4y$ is not a representable functor! But to make amends, we could say that $4y$ is a *summand*; this means that there is some q' such that $q \cong q' + 4y$, namely $q' := y^8$. So $3y$ is also a summand, but y^2 and $5y$ are not.

Solution to Exercise 2.5.

1. Let $I := 1$ and $p[1] := 3$ so that $p := y^3 \cong \sum_{i \in I} y^{p[i]}$. Then

$$p(X) \cong \{(1, 4, 4, 4), (1, 4, 4, 9), (1, 4, 9, 4), (1, 4, 9, 9), \\ (1, 9, 4, 4), (1, 9, 4, 9), (1, 9, 9, 4), (1, 9, 9, 9)\}.$$

2. Let $I := 4$, $p[1] := p[2] := p[3] := 2$, and $p[4] := 1$, so that $p := 3y^2 + 1 \cong \sum_{i \in I} y^{p[i]}$. Then $p(X) \cong \{(1, a, a), (2, a, a), (3, a, a), (4)\}$.
3. Let $I := 0$ so that $p := 0 \cong \sum_{i \in I} y^{p[i]}$. Then $p(X) \cong 0$. Alternatively, note that 0 is the constant functor that sends every set to 0 .
4. Let $I := 4$ and $p[i] := 0$ for every $i \in I$ so that $p := 4 \cong \sum_{i \in I} y^{p[i]}$. Then $p(X) \cong \{(1), (2), (3), (4)\} \cong 4$. Alternatively, note that 4 is the constant functor that sends every set to 4 .
5. Let $I := 1$ and $p[1] := 1$ so that $p := y \cong \sum_{i \in I} y^{p[i]}$. So $p(X) \cong \{(1, n) \mid n \in \mathbb{N}\} \cong \mathbb{N}$. Alternatively, note that y is the identity functor, so it sends $\mathbb{N}$ to itself.

Solution to Exercise 2.9.

We consider $p(0)$ for arbitrary polynomials p . A representable functor y^S for $S \in \mathbf{Set}$ sends $0 \mapsto 0$ if $S \neq 0$ (as there are then no functions $S \rightarrow 0$), but sends $0 \mapsto 1$ if $S = 0$ (as there is a unique function $0 \rightarrow 0$). So

$$p(0) \cong \sum_{i \in p(1)} \left(y^{p[i]} \right) (0) \cong \sum_{\substack{i \in p(1), \\ p[i] \neq 0}} 0 + \sum_{\substack{i \in p(1), \\ p[i] = 0}} 1 \cong \{i \in p(1) \mid p[i] = 0\}.$$

That is, $p(0)$ is the set of *constant* positions of p , the positions of p that have no directions. For example, if $p := y^3 + 3y^2 + 4$, then $p(0) = 4$. In the language of high school algebra, we might call $p(0)$ the *constant term* of p .

Solution to Exercise 2.12.

We have

$$p(1) \cong \{(1, 1, 1), (2, 1)\} \quad \text{and} \quad p(2) \cong \{(1, 1, 1), (1, 1, 2), (1, 2, 1), (1, 2, 2), (2, 1), (2, 2)\}.$$

Say we choose the function $f: 1 \rightarrow 2$ that sends $1 \mapsto 1$. Then $p(f)$ would send $(1, 1, 1) \mapsto (1, 1, 1)$ and $(2, 1) \mapsto (2, 1)$. If we had instead picked $1 \mapsto 2$ as our function f , then $p(f)$ would send $(1, 1, 1) \mapsto (1, 2, 2)$ and $(2, 1) \mapsto (2, 2)$.

Solution to Exercise 2.13.

1. A polynomial p is representable when $p \cong y^A$ for some set A , and y^A has exactly 1 position. Conversely, if a polynomial p has exactly 1 position, then $p(1) \cong 1$, so we may write p (up to isomorphism) as $p \cong \sum_{1 \in 1} y^{p[1]} \cong y^{p[1]}$, which is representable. So a polynomial p is representable if and only if it has exactly 1 position.
2. A polynomial p is constant when $p \cong I$ for some set I , and I has no directions at any of its positions. Conversely, if every direction-set of a polynomial p is empty, then

$$p \cong \sum_{i \in p(1)} y^{p[i]} \cong \sum_{i \in p(1)} y^0 \cong \sum_{i \in p(1)} 1 \cong p(1),$$

i.e. the set $p(1)$ viewed as a constant functor. So a polynomial p is constant if and only if it has exactly 0 directions at each position.

Solution to Exercise 2.17.

1. A polynomial p is linear when $p \cong Iy$ for some set I , and Iy has exactly 1 direction at each position. (Note that this is even true when $p \cong 0y \cong 0$, for then it is true vacuously.) Conversely, if a polynomial p has exactly 1 direction at each position, then $p[i] \cong 1$ for all $i \in p(1)$, so

$$p \cong \sum_{i \in p(1)} y^{p[i]} \cong \sum_{i \in p(1)} y^1 \cong \sum_{i \in p(1)} y \cong p(1)y,$$

which is linear. So a polynomial p is linear if and only if it has exactly 1 direction at each position.

2. A polynomial p is a monomial when $p \cong Iy^A$ for sets I and A , implying that there is an isomorphism of direction-sets $p[i] \cong A \cong p[j]$ for all p -positions i and j (i.e. all the direction-sets of p have the same cardinality). Conversely, if all the direction-sets of a polynomial p are isomorphic to each other, then they are all isomorphic to some set A , so we have

$$p \cong \sum_{i \in p(1)} y^{p[i]} \cong \sum_{i \in p(1)} y^A \cong p(1)y^A,$$

which is a monomial. So a polynomial p is a monomial if and only if all of its direction-sets have the same cardinality.

Solution to Exercise 2.20.

The stopwatch app on my phone has three positions: a **zero** position, from which I may tap a single **start** button; a **running** position, from which I may tap either a **lap** button or a **stop** button; and a **stopped** position, from which I may tap either a **start** button or a **reset** button. Thinking of the buttons available to press as the directions at each position, the corresponding polynomial is

$$\{\mathbf{zero}\}_y^{\{\mathbf{start}\}} + \{\mathbf{running}\}_y^{\{\mathbf{lap}, \mathbf{stop}\}} + \{\mathbf{stopped}\}_y^{\{\mathbf{start}, \mathbf{reset}\}} \cong y + 2y^2.$$

Solution to Exercise 2.29.

1. Here is the corolla forest associated to $p := 2y^3 + 2y + 1$ (note that the order in which the corollas are drawn does not matter):



2. The forest has 5 roots.
3. The roots represent the 5 positions, one position per root.
4. The first and second corollas have 3 leaves each, the third and fourth corollas have 1 leaf each, and the fifth corolla has 0 leaves.
5. The directions at each position correspond to the leaves in each corolla, so just copy the answer from #4, replacing “corolla” with “position” and “leaf” with “direction”: the first and second positions have 3 directions each, the third and fourth positions have 1 direction each, and the fifth position has 0 directions.

Solution to Exercise 2.30.

Aesthetically speaking, here is a polynomial that may be drawn as a beautiful corolla forest:

$$p := y^0 + y^1 + y^2 + y^3 + \dots$$

It is reminiscent (and formally related) to the notion of lists: if A is any set, then $p(A) \cong A^0 + A^1 + A^2 + \dots$ is the set $\text{List}(A)$ of lists (i.e. finite ordered sequences) with entries in A . Here is a picture of the lovely forest associated to p :



3. The category of polynomial functors

In this chapter, we will define **Poly**, our main category of interest, so that we have a firm foundation from which to speak about interactive systems. The objects of **Poly** are the polynomial functors that we defined in the previous chapter. Here we will examine the morphisms of **Poly**: natural transformations between polynomial functors. Along the way, we will present some of **Poly**'s most versatile categorical properties.

3.1 Dependent lenses between polynomial functors

Before we define the category **Poly** of polynomial functors, we note that polynomial functors live inside a category already: the category $\mathbf{Set}^{\mathbf{Set}}$ of functors $\mathbf{Set} \rightarrow \mathbf{Set}$, whose morphisms are natural transformations. This leads to a natural definition of morphisms between polynomial functors, from which we can derive a category of polynomial functors for free. We call such a morphism a *dependent lens*, or a *lens* for short. If you are familiar with lenses from functional programming, we'll see in Example 3.41 how our notion of a dependent lens is related.

Definition 3.1 (Dependent lens, **Poly**). Given polynomial functors p and q , a *dependent lens* (or simply *lens*) from p to q is a natural transformation $p \rightarrow q$. Then **Poly** is the category whose objects are polynomial functors and whose morphisms are dependent lenses.

In other words, **Poly** is the full subcategory of $\mathbf{Set}^{\mathbf{Set}}$ spanned by the polynomial functors: we take the category $\mathbf{Set}^{\mathbf{Set}}$, throw out all the objects that are not (isomorphic to) polynomials, but keep all the same morphisms between the objects that remain.

Unraveling the familiar definition of a natural transformation, a dependent lens between polynomial functors $p \rightarrow q$ thus consists of a

function $p(X) \rightarrow q(X)$ for every set X such that naturality squares commute. That is a lot of data to keep track of! Fortunately, there is a much simpler way to think about these lenses, which we will discover using the Yoneda lemma.

Exercise 3.2. Given a set S and a polynomial q , show that a lens $y^S \rightarrow q$ can be naturally identified with an element of the set $q(S)$. That is, show that there is an isomorphism natural in both S and q given by

$$\mathbf{Poly}(y^S, q) \cong q(S).$$

(Hint: use the Yoneda lemma from Lemma 1.10.)

Exercise 3.2 gives us an alternative characterization for lenses out of representable functors. But before we can characterize lenses out of polynomial functors in general, we need to describe how coproducts work in **Poly**. Fortunately, since polynomial functors are defined as coproducts of representables, coproducts in **Poly** are easy to understand.

Proposition 3.3. The category **Poly** has all small coproducts, coinciding with coproducts in $\mathbf{Set}^{\mathbf{Set}}$ given by the operation $\sum_{i \in I}$ for each set I .

Proof. By Corollary 1.38, the category $\mathbf{Set}^{\mathbf{Set}}$ has all small coproducts given by $\sum_{i \in I}$. The full subcategory inclusion $\mathbf{Poly} \rightarrow \mathbf{Set}^{\mathbf{Set}}$ reflects these coproducts, and by definition **Poly** is closed under the operation $\sum_{i \in I}$. $\square$

Explicitly, given an I -indexed family of polynomials $(p_i)_{i \in I}$, its coproduct is

$$\sum_{i \in I} p_i \cong \sum_{i \in I} \sum_{j \in p_i(1)} y^{p_i[j]} \cong \sum_{(i, j) \in \sum_{i \in I} p_i(1)} y^{p_i[j]} \quad (3.4)$$

by Corollary 1.38. This coincides with our notion of polynomial addition from high school algebra: just add all the terms together, combining like terms to simplify. Binary coproducts are given by binary sums of functors, appropriately denoted by $+$, while the initial object of **Poly** is the constant polynomial 0.

In particular, (3.4) implies that for any polynomials p and q , their coproduct $p + q$ is given as follows. The position-set of $p + q$ is the coproduct of sets $p(1) + q(1)$. At position $(1, i) \in p(1) + q(1)$ with $i \in p(1)$, the directions of $p + q$ are just the $p[i]$ -directions; at position $(2, j) \in p(1) + q(1)$ with $j \in q(1)$, the directions of $p + q$ are just the $q[j]$ -directions.

Crucially, we have the following corollary.

Corollary 3.5. In the category **Poly**, every polynomial p is the coproduct of its representable summands $(y^{p[i]})_{i \in p(1)}$.

In other words, writing p as the sum $\sum_{i \in p(1)} y^{p[i]}$ is not just a coproduct in **Set^{Set}**; it is also a coproduct in **Poly** itself.

We are now ready to give our alternative characterization of dependent lenses. Recall that a polynomial $p \cong \sum_{i \in p(1)} y^{p[i]}$ can be uniquely identified with an indexed family $p[-]: p(1) \rightarrow \mathbf{Set}$, a functor from the set $p(1)$ viewed as a discrete category.

Proposition 3.6. Given polynomials p and q , there is an isomorphism

$$\mathbf{Poly}(p, q) \cong \prod_{i \in p(1)} \sum_{j \in q(1)} p[i]^{q[j]} \quad (3.7)$$

natural in p and q . In particular, a lens $f: p \rightarrow q$ can be identified with a pair $(f_1, f^\#)$

$$\begin{array}{ccc} p(1) & \xrightarrow{f_1} & q(1) \\ & \searrow f^\# \swarrow & \\ & p[-] & \leftarrow q[-] \\ & \mathbf{Set} & \end{array} \quad (3.8)$$

where $f_1: p(1) \rightarrow q(1)$ is a function (equivalently, a functor between discrete categories) and $f^\#: q[f_1(-)] \rightarrow p[-]$ is a natural transformation: a function $f_i^\#: q[f_1 i] \rightarrow p[i]$ for each $i \in p(1)$.

Proof. We have $p \cong \sum_{i \in p(1)} y^{p[i]}$. Then by Corollary 3.5 and the universal

property of the coproduct, we have a natural isomorphism

$$\mathbf{Poly}\left(\sum_{i \in p(1)} y^{p[i]}, q\right) \cong \prod_{i \in p(1)} \mathbf{Poly}(y^{p[i]}, q).$$

Applying Exercise 3.2 (i.e. the Yoneda lemma) and the fact that $q \cong \sum_{j \in q(1)} y^{q[j]}$ yields the natural isomorphism

$$\prod_{i \in p(1)} \mathbf{Poly}(y^{p[i]}, q) \cong \prod_{i \in p(1)} q(p[i]) \cong \prod_{i \in p(1)} \sum_{j \in q(1)} p[i]^{q[j]},$$

so (3.7) follows.

The right-hand side of (3.7) is the set of dependent functions $f: (i \in p(1)) \rightarrow \sum_{j \in q(1)} p[i]^{q[j]}$. Each such dependent function is uniquely determined by its two projections $\pi_1 f: (i \in p(1)) \rightarrow q(1)$ and $\pi_2 f: (i \in p(1)) \rightarrow p[i]^{q[\pi_1 f i]}$. These can be identified respectively with a (non-dependent) function $f_1 := \pi_1 f$ with signature $p(1) \rightarrow q(1)$ and a natural transformation $f^\sharp: q[f_1(-)] \rightarrow p[-]$ whose i -component for $i \in p(1)$ is $f_i^\sharp := \pi_2 f i \in p[i]^{q[f_1 i]}$. $\square$

We have now greatly simplified our characterization of a dependent lens $f: p \rightarrow q$: rather than infinitely many functions satisfying infinitely many naturality conditions, f may simply be specified by a function $f_1: p(1) \rightarrow q(1)$ and, for each $i \in p(1)$, a function $f_i^\sharp: q[f_1 i] \rightarrow p[i]$, without any additional restrictions. This characterization can be expressed entirely in the language of positions and directions: f_1 is a function from p -positions to q -positions, while $f_i^\sharp$ for a p -position i is a function from $q[f_1 i]$ -directions to $p[i]$ -directions. This leads to the following definition.

Definition 3.9 (On-positions function, on-directions map and function). Given a lens $f: p \rightarrow q$, let $(f_1, f^\sharp)$ denote the pair identified with f via Proposition 3.6. Then we call the function $f_1: p(1) \rightarrow q(1)$ the *(forward) on-positions function* of f , while we call the natural transformation $f^\sharp: q[f_1(-)] \rightarrow p[-]$ the *(backward) on-directions map* of f . For $i \in p(1)$, we call the i -component $f_i^\sharp: q[f_1 i] \rightarrow p[i]$ of $f^\sharp$ the *(backward) on-directions function* of f at i .

Definition 3.9 highlights the bidirectional nature of a lens $f: p \rightarrow q$: it consists of a function going *forward* on positions, following the direction of f from p to q , as well as functions going *backward* on directions, opposing the direction of f from q to p . This forward-backward interaction is what drives the applications of **Poly** we will study.

We prefer to call a morphism between polynomial functors a “lens” rather than a “natural transformation” because we wish to emphasize this concrete on-positions and on-directions perspective. Whenever we do need to view a morphism in **Poly** as a natural transformation, we will refer to them as such.

In the next several sections, we will give some examples of lenses and intuition for thinking about them in terms of interaction protocols, corolla forests, and polyboxes.

3.2 Dependent lenses as interaction protocols

Here is our first example of a dependent lens and a real-world interaction it might model.

Example 3.10 (Modeling an interaction protocol with a lens). Recall our coin jar polynomial from Example 2.19:

$$q := \{\text{open}\}y^{\{\text{penny, nickel, dime, quarter}\}} + \{\text{closed}\}y^0.$$

It has 2 positions: its *open* position has 4 directions representing the 4 denominations of coins it may take, while its *closed* position has 0 directions to indicate that it cannot take anything.

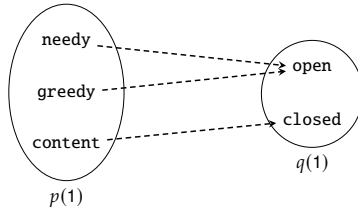
Now imagine that we model the owner of this coin jar with the following polynomial:

$$\begin{aligned} p := & \{\text{needy}\}y^{\{\text{save, spend}\}} \\ & + \{\text{greedy}\}y^{\{\text{accept, reject, ask for more}\}} \\ & + \{\text{content}\}y^{\{\text{count, rest}\}}. \end{aligned}$$

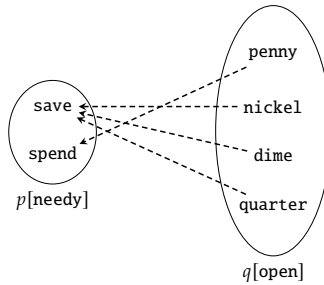
Each of its 3 positions represents a possible mood of the owner, and the directions at each position represent the options available

to an owner in the corresponding mood. We will construct a lens $f: p \rightarrow q$ to model the interaction between the owner and their coin jar.

Say that a *needy* or *greedy* owner will keep their coin jar *open*, while a *content* owner will keep their coin jar *closed*. We can express this with an on-positions function f_1 from the set of p -positions (on the left) to the set of q -positions (on the right), as follows (the dashed arrows indicate the function assignments):

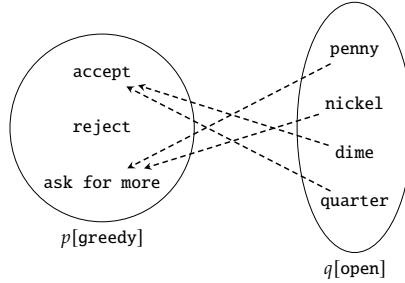


From there, say that a *needy* owner whose coin jar receives a *nickel* or higher will choose to *save* it, but one whose coin jar receives a *penny* will choose to *spend* it. Meanwhile, a *greedy* owner whose coin jar receives a *penny* or *nickel* will ask for more, but one whose coin jar receives a *dime* or *quarter* will accept it. We can express this behavior with an on-directions map $f^\#: q[f_1(-)] \rightarrow p[-]$. Its *needy*-component is the on-directions function $f_{\text{needy}}^\#: q[f_1(\text{needy})] \rightarrow p[\text{needy}]$ drawn as follows:



Notice that, by keeping the positions and directions of p on the left and those of q on the right, the on-positions function is drawn from left to right, while the on-directions functions must be drawn right

to left. The greedy-component of $f^\#$ is the on-directions function $f_{\text{greedy}}^\# : q[f_1(\text{greedy})] \rightarrow p[\text{greedy}]$ drawn like so:



Finally, as $q[f_1(\text{content})] = q[\text{closed}] = 0$, the content-component of $f^\#$ is the vacuously defined on-directions function $f_{\text{content}}^\# : 0 \rightarrow p[\text{content}]$. Together, the on-positions function f_1 and the on-directions map $f^\#$ defined above completely characterize a lens $f : p \rightarrow q$ depicting the interaction between the coin jar and its owner.

More generally, a lens depicts what we call an *interaction protocol*, a kind of dialogue between two agents regarding their positions and directions. Say that one agent is represented by a polynomial p and another by a polynomial q . Then a lens $f : p \rightarrow q$ is an interaction protocol that prescribes how the positions of p influence the positions of q and how the directions of q influence the directions of p . Each p -position $i \in p(1)$ is passed forward via the on-positions function of f to a q -position $f_1 i \in q(1)$. Then each $q[f_1 i]$ -direction b is passed backward via the on-directions function of f at i to a $p[i]$ -direction $f_i^\# b$.

To visualize these lenses, we may use either our corolla forests or our polyboxes.

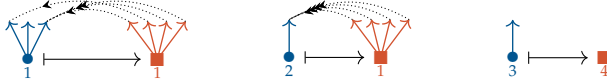
3.3 Corolla forest pictures of dependent lenses

The corolla forest associated to a polynomial concretely depicts its positions and directions, making it easy to extend our corolla forest pictures to depict the dependencies between the positions and directions of two polynomials that a lens between them prescribes.

Example 3.11. Let $p := y^3 + 2y$ and $q := y^4 + y^2 + 2$. We draw them as corolla forests with their positions labeled (we use different shapes to distinguish the positions of different polynomials):



To give a lens $p \rightarrow q$, we must send each p -position $i \in p(1)$ to a q -position $j \in q(1)$, then send each direction in $q[j]$ back to one in $p[i]$. We can draw such a lens as follows.



This represents one possible lens $f: p \rightarrow q$. The horizontal solid arrows pointing rightward in the picture above tell us that the on-positions function $f_1: p(1) \rightarrow q(1)$ is given by

$$f_1(1) := 1, \quad f_1(2) := 1, \quad \text{and} \quad f_1(3) := 4.$$

Then the curved dashed arrows pointing leftward in the picture above describe the on-directions map $f^\sharp: q[f_1(-)] \rightarrow p[-]$. On the left, the arrows depict a possible on-directions function $f_1^\sharp: q[1] \rightarrow p[1]$ from the 4 directions in $q[1]$ to the 3 directions in $p[1]$. In the middle, the arrows depict the only possible on-directions function $f_2^\sharp: q[1] \rightarrow p[2]$ because $|p[2]| = 1$. Finally, on the right, there are no curved dashed arrows, depicting the only possible on-directions function $f_3^\sharp: q[4] \rightarrow p[3]$ because $|q[4]| = 0$.

Exercise 3.12.

1. Draw the corolla forests associated to $p := y^3 + y + 1$, $q := y^2 + y^2 + 2$, and $r := y^3$.
2. Pick an example of a dependent lens $p \rightarrow q$ and draw it as we

did in Example 3.11.

3. Explain the behavior of your lens as an interaction protocol in terms of positions and directions.
4. Explain in those terms why there can't be any lenses $p \rightarrow r$.

3.4 Polybox pictures of dependent lenses

Another way to visualize a dependent lens $f: p \rightarrow q$ is to draw the polyboxes for p and q .

$$\begin{array}{ccc}
 & f: p \rightarrow q & \\
 \begin{array}{c} p[-] \\ p(1) \end{array} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \end{array} & \begin{array}{c} \longleftarrow \\ \longrightarrow \end{array} & \begin{array}{c} q[-] \\ q(1) \end{array} \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \end{array} \\
 p & & q
 \end{array} \tag{3.13}$$

Thinking of the polyboxes as cells in a spreadsheet, the lens prescribes how the values of the cells are computed. The boxes colored blue accept user input, while the other boxes are computed from that input according to the spreadsheet's rules.

The arrows track the flow of information, starting from the lower left. When the user fills the blue position box of p with some $i \in p(1)$, the arrow pointing right indicates that the lens fills the position box of q with some $j \in q(1)$ based on i . The corresponding assignment $i \mapsto j$ is the on-position function f_1 of the lens.

Then when the user fills the blue direction box of q with some $b \in q[j]$, the arrow pointing left indicates that the lens fills the direction box of p with some $a \in p[i]$ based on i and b . Fixing $i \in p(1)$, the assignment $b \mapsto a$ is an on-directions function $f_i^\#$ of the lens.

Once all the boxes are filled, we obtain the following:

$$\begin{array}{ccc}
 \begin{array}{|c|} \hline a \\ \hline i \\ \hline \end{array} p & \begin{array}{c} \xleftarrow{f_i^\#} \\ \xrightarrow{f_1} \end{array} & \begin{array}{|c|} \hline b \\ \hline j \\ \hline \end{array} q
 \end{array}$$

Here $j := f_1 i$ and $a := f_i^\# b$. So a lens is any protocol that will fill the remaining boxes once the user fills the blue boxes, following the

directions of the arrows drawn. Be careful: although the arrow $f^\sharp$ is drawn from the codomain's direction box, it also takes into account what is entered into the domain's position box previously. After all, the on-directions map of a lens is dependent on both a position of the domain and a direction of the codomain.

Here is an example of a lens depicted with polyboxes that would be difficult to draw with corolla forests.

Example 3.14 (Modeling with a lens in polyboxes). Caroline asks each of her parents for 20 dollars. Each parent gives Caroline a positive amount of money not exceeding 20 dollars. Caroline spends some of the money she receives before returning the remainder to each parent proportionally according to the amount she received from each.

To model this interaction as a lens $f : p \rightarrow q$, we first define the polynomials p and q . Let

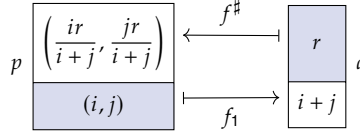
$$p := \sum_{(i,j) \in (0,20] \times (0,20]} y^{[0,i] \times [0,j]}$$

be the polynomial that models the parents: its position $(i, j) \in (0, 20] \times (0, 20]$ consists of the quantities of money that each parent gives to Caroline, and its direction $(i', j') \in [0, i] \times [0, j]$ at that position consists of the quantities of money that Caroline returns to each parent. Then let

$$q := \sum_{k \in (0, \infty)} y^{[0,k]}$$

be the polynomial that models Caroline: its position $k \in (0, \infty)$ is the total money that Caroline receives (perhaps Caroline is prepared to receive more money than she is asking for, even if her parents are not prepared to give it), while its direction $r \in [0, k]$ is the money Caroline has remaining after spending some of it. Then

we draw the lens f that models their interaction in polyboxes as



We interpret this as saying that the on-positions function f_1 from $p(1) = (0, 20] \times (0, 20]$ to $q(1) = (0, \infty)$ is defined on $(i, j) \in (0, 20] \times (0, 20]$ to be

$$f_1(i, j) := i + j,$$

while the on-directions function $f_{(i,j)}^\#$ from $q[i + j] = [0, i + j]$ to $p[(i, j)] = [0, i] \times [0, j]$ is defined on $r \in [0, i + j]$ to be

$$f_{(i,j)}^\#(r) := \left(\frac{ir}{i+j}, \frac{jr}{i+j} \right);$$

the polybox picture expresses this more compactly. Notice that the position box of q depends only on the position box of p , while the direction box of p depends on the position box of p as well as the direction box of q .

Example 3.14 illustrates how we can use polyboxes to specify a particular lens—or, equivalently, how we can use polyboxes to *define* a lens, the same way we might define a function by writing it as a formula in a dummy variable. Later on we will see how polyboxes help depict how lenses compose.

3.5 Computations with dependent lenses

Our concrete characterization of dependent lenses allows us to enumerate them.

Example 3.15 (Enumerating lenses). Let $p := y^3 + 2y$ and $q := y^4 + y^2 + 2$, as in Example 3.11. Again, we draw them as corolla forests

with their positions labeled:



How many lenses are there from p to q ? The first p -position must be sent to any q -position: 1, 2, 3, or 4. Sending it to 1 would require choosing an on-directions function $q[1] \rightarrow p[1]$, or $4 \rightarrow 3$; there are 3^4 of these. Similarly, there are 3^2 possible on-directions functions $q[2] \rightarrow p[1]$, as well as 3^0 on-directions functions $q[3] \rightarrow p[1]$ and 3^0 on-directions functions $q[4] \rightarrow p[1]$. Hence there are a total of $3^4 + 3^2 + 3^0 + 3^0 = 92$ ways to choose $f_1(1)$ and $f_1^\#$.

The second p -position must also be sent to 1, 2, 3, or 4 before selecting $f_2^\#$; there are $1^4 + 1^2 + 1^0 + 1^0 = 4$ ways to do this. Identically, there are 4 ways to choose $f_1(3)$ and $f_3^\#$.

In total, there are $92 \cdot 4 \cdot 4 = 1472$ lenses $p \rightarrow q$. This coincides with what we obtain by taking the cardinality of both sides of (3.7) and plugging in our values for p and q :

$$\begin{aligned}
 |\mathbf{Poly}(p, q)| &= \prod_{i \in p(1)} |q(p[i])| \\
 &= \prod_{i \in 3} (|p[i]|^4 + |p[i]|^2 + 2) \\
 &= (3^4 + 3^2 + 2)(1^4 + 1^2 + 2)^2 \\
 &= 92 \cdot 4^2 = 1472.
 \end{aligned}$$

Exercise 3.16. For any polynomial p and set A , e.g. $A := 2$, the Yoneda lemma gives an isomorphism $\mathbf{Poly}(y^A, p) \cong p(A)$, so the number of lenses $y^A \rightarrow p$ should be equal to the cardinality of $p(A)$.

1. Choose a polynomial p with finitely many positions and directions and draw both y^2 and p as corolla forests.
2. Count all the lenses $y^2 \rightarrow p$. How many are there?

3. Compute the cardinality of $p(2)$. Is this the same as the previous answer?

Exercise 3.17. For each of the following polynomials p, q , compute the number of lenses $p \rightarrow q$.

1. $p := y^3, \quad q := y^4$.
2. $p := y^3 + 1, \quad q := y^4$.
3. $p := y^3 + 1, \quad q := y^4 + 1$.
4. $p := 4y^3 + 3y^2 + y, \quad q := y$.
5. $p := 4y^3, \quad q := 3y$.

The following exercises provide alternative formulas for the set of lenses between two polynomials.

Exercise 3.18.

1. Show that the following are isomorphic:

$$\mathbf{Poly}(p, q) \cong \prod_{i \in p(1)} \sum_{j \in q(1)} \prod_{b \in q[j]} \sum_{a \in p[i]} 1. \quad (3.19)$$

2. Show that the following are isomorphic:

$$\mathbf{Poly}(p, q) \cong \sum_{f_1: p(1) \rightarrow q(1)} \prod_{j \in q(1)} \mathbf{Set} \left(q[j], \prod_{\substack{i \in p(1), \\ f_1 i = j}} p[i] \right). \quad (3.20)$$

3. Using the language of positions and directions, describe how an element of the right-hand side of (3.20) corresponds to a lens $p \rightarrow q$.

In (3.4), we gave an explicit formula for coproducts in **Poly** inherited from coproducts in **Set^{Set}**, as justified by Proposition 3.3. We can now use (3.7) to directly verify that the expression on the right-hand side of (3.4) satisfies the universal property for the coproduct of polynomials $(p_i)_{i \in I}$ in **Poly**.

Exercise 3.21. Use (3.7) to verify that

$$\mathbf{Poly}\left(\sum_{(i,j) \in \sum_{i \in I} p_i(1)} y^{p_i[j]}, q\right) \cong \prod_{i \in I} \mathbf{Poly}(p_i, q)$$

for all polynomials $(p_i)_{i \in I}$ and q .

For the remaining exercises in this section, we introduce the concept of the *derivative* $\dot{p}$ of a polynomial p .

Example 3.22 (Derivatives). The *derivative* of a polynomial p , denoted $\dot{p}$, is defined as

$$\dot{p} := \sum_{i \in p(1)} \sum_{a \in p[i]} y^{p[i] - \{a\}}.$$

For example, if $p := y^{\{U,V,W\}} + \{A\}y^{\{X\}} + \{B\}y^{\{X\}}$ then

$$\dot{p} = \{U\}y^{\{V,W\}} + \{V\}y^{\{U,W\}} + \{W\}y^{\{U,V\}} + \{(A, X)\}y^0 + \{(B, X)\}y^0.$$

Up to isomorphism $p \cong y^3 + 2y$ and $\dot{p} \cong 3y^2 + 2$. Indeed, this coincides with the familiar notion of derivatives of polynomials from calculus.

Since

$$\dot{p}y \cong \sum_{i \in p(1)} \sum_{a \in p[i]} y^{p[i] - \{a\}} y \cong \sum_{i \in p(1)} \sum_{a \in p[i]} y^{p[i] - \{a\} + 1} \cong \sum_{i \in p(1)} \sum_{a \in p[i]} y^{p[i]},$$

there exists a canonical lens $\dot{p}y \rightarrow p$; you will define this lens in Exercise 3.23. The lens arises in computer science in the context of “plugging in to one-hole contexts”; we will not explore that here, but see [McB01] and [Abb+03] for details.

A lens $f': p \rightarrow \dot{q}$ is similar to a lens $f: p \rightarrow q$, except that each p -position explicitly selects a direction of q to remain unassigned. More precisely, while the on-positions function of f sends each p -position to a q -position, the on-positions function of f' sends each p -position i to some $(j, a) \in \sum_{j \in q(1)} q[j]$, picking out not only

a q -position j , but also a $q[j]$ -direction a . Then the on-directions function of f' at i sends every $q[j]$ -direction *other than* a back to a $p[i]$ -direction.

Exercise 3.23. The derivative is not very well-behaved categorically, but it is nevertheless intriguing. Take $p, q \in \mathbf{Poly}$.

1. Give an explicit construction for the canonical lens $\dot{p}y \rightarrow p$ from Example 3.22.
2. Is there always a lens $p \rightarrow \dot{p}$? If so, prove it; if not, give a counterexample.
3. Is there always a lens $\dot{p} \rightarrow p$? If so, prove it; if not, give a counterexample.
4. Given a lens $p \rightarrow q$, is there always a lens $\dot{p} \rightarrow \dot{q}$? If so, prove it; if not, give a counterexample.
5. We will define the binary operations $\otimes$ and $[-, -]$ on $\mathbf{Poly}$ later on in (3.66) and (4.75); and in Exercise 4.80, you will be able to use Exercise 4.78 to deduce that

$$[p, y] \otimes p \cong \sum_{f \in \prod_{i \in p(1)} p[i]} \sum_{i \in p(1)} y^{p(1) \times p[i]}. \quad (3.24)$$

Construct a canonical lens $[p, y] \otimes p \rightarrow \dot{p}$.

6. In Example 3.22, we described a lens $p \rightarrow \dot{q}$ in terms of “unassigned” directions. Describe a lens $py \rightarrow q$ in terms of “unassigned” directions as well.

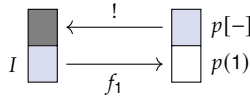
We will not use derivatives very much in the rest of this text, except as shorthand to denote the set of all directions of a polynomial: given a polynomial p , its directions comprise the set $\dot{p}(1)$.

Exercise 3.25. Show that $\dot{p}(1)$ is isomorphic to the set of all directions of p (i.e. the sum of all direction-sets of p).

3.6 Dependent lenses between special polynomials

In Section 2.2, we considered four special classes of polynomials (here I and A are sets): constant polynomials I , linear polynomials Iy , representable polynomials y^A , and monomials Iy^A . Of special note are the constant and linear polynomial 0 , the constant and representable polynomial 1 , and the linear and representable polynomial y . We now consider lenses with these special polynomials as domains or codomains, highlighting some important examples using polyboxes and leaving most of the rest as exercises. Let p be a polynomial throughout.

Example 3.26 (Lenses from linear polynomials). A lens $f: Iy \rightarrow p$ can be drawn in polyboxes as follows:



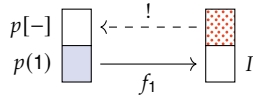
Recall that we shade in the direction box of a linear polynomial to indicate that it can only be filled with one entry. Hence the on-directions map of f is uniquely determined, so f is completely characterized by its on-positions function f_1 . We conclude that lenses $Iy \rightarrow p$ can be identified with functions $I \rightarrow p(1)$.

Exercise 3.27 (Lenses from 0 and y).

1. Use Example 3.26 to verify that 0 is the initial object of **Poly**.
2. Use Example 3.26 to show that lenses $y \rightarrow p$ can be identified with p -positions.

Exercise 3.28 (Lenses to linear polynomials). Characterize lenses $p \rightarrow Iy$ in terms of I and the positions and directions of p .

Example 3.29 (Lenses to constants). A lens $f: p \rightarrow I$ can be drawn in polyboxes as follows:



Recall that we fill the direction box of a constant polynomial with red dots to indicate that it cannot be filled with any entry. Hence the on-directions map of f is again uniquely determined, so f is completely characterized by its on-positions function f_1 . (While the on-directions map of f does exist—it is vacuous—it can never produce an element to fill the direction box of p , so we draw it with a dashed line.) We conclude that lenses $p \rightarrow I$ can be identified with functions $p(1) \rightarrow I$.

Exercise 3.30 (Lenses to 1 and 0).

1. Use Example 3.29 to show that 1 is the terminal object of **Poly**.
2. Show that there is exactly one lens whose codomain is 0. What is its domain?

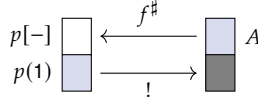
Exercise 3.31 (Lenses from constants such as 1).

1. Characterize lenses $I \rightarrow p$ in terms of I and the positions and directions of p . You may find it helpful to refer to $p(0)$; see Exercise 2.9.
2. Use the previous part to characterize lenses $1 \rightarrow p$.

We already know from the Yoneda lemma (see Exercise 3.2) that lenses $y^A \rightarrow p$ correspond to elements of $p(A)$, so we understand lenses from representables. Thus we turn our attention to lenses $p \rightarrow y^A$.

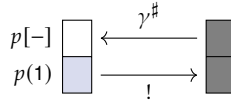
Example 3.32 (Lenses to representables). A lens $f: p \rightarrow y^A$ can be

drawn in polyboxes as follows:

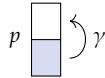


Recall that we shade in the position box of a representable to indicate that it can only be filled with one entry. Hence the on-positions function of f is uniquely determined, so f is completely characterized by its on-directions map $f^\#$, which takes a p -position i and a direction $a \in A$ and sends them to a direction $b \in p[i]$. We conclude that lenses $p \rightarrow y^A$ can be identified with dependent functions $((i, a) \in p(1) \times A) \rightarrow p[i]$.

Example 3.33 (Lenses to y). As a special case of the previous example, a lens $\gamma: p \rightarrow y$ can be drawn in polyboxes as follows:



Such lenses can be identified with dependent functions $(i \in p(1)) \rightarrow p[i]$, which, abusing notation, we also denote by γ . For each p -position i in the blue position box, γ picks out a $p[i]$ -direction to fill the unshaded direction box. (Remember that the arrow labeled $\gamma^\#$ depends not only on the direction box to its right, but also on the position box of p .) So it makes sense to abbreviate the polybox picture of γ like so:



(3.34)

The correspondence between lenses $p \rightarrow y$ and dependent functions $(i \in p(1)) \rightarrow p[i]$ exhibited in the previous example also follows directly

from (3.7): taking $q := y$, we have

$$\mathbf{Poly}(p, y) \cong \prod_{i \in p(1)} \sum_{j \in 1} p[i]^1 \cong \prod_{i \in p(1)} p[i], \quad (3.35)$$

where the right-hand side is precisely the set of dependent functions $(i \in p(1)) \rightarrow p[i]$. By Exercise 1.22, such functions may be identified with the *sections* of the projection function from $\dot{p}(1) \cong \sum_{i \in p(1)} p[i]$, the set of all directions of p (see Exercise 3.25), to $p(1)$, the set of all positions of p , sending each $(i, a) \in \dot{p}(1)$ to $i \in p(1)$. The fact that this projection determines p (up to isomorphism) motivates the following definition.

Definition 3.36 (Section; bundle). For $p \in \mathbf{Poly}$, a *section* of p is a lens $p \rightarrow y$. We denote the set of all sections of p by $\Gamma(p)$; that is,

$$\Gamma(p) := \mathbf{Poly}(p, y). \quad (3.37)$$

The *bundle* of p , denoted π_p , is the projection function

$$\dot{p}(1) \cong \sum_{i \in p(1)} p[i] \rightarrow p(1)$$

sending $(i, a) \mapsto i$.

With this terminology, we can say that p is determined (up to isomorphism) by its bundle, and that the sections of p can be identified with the sections of its bundle.

To visualize the bundle of p , simply draw it as a corolla forest: a *bundle* of arrows. The bundle projects each leaf down to its root. To visualize a section of p , picture its corollas piled atop each other; a section $\gamma: p \rightarrow y$ is then a *cross-section* of this pile of p -corollas, picking out an arrow from each one—a direction at each position.

Alternatively, you could think of the arrow curving back to the polyboxes for p in our picture (3.34) of a section $\gamma: p \rightarrow y$ as *sectioning* off the polyboxes for p from any polyboxes that may otherwise appear to its right. We clarify this intuition by returning to a previous example of a polynomial and considering its sections.

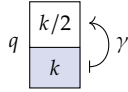
Example 3.38 (Modeling with sections). Recall from Example 3.14 the polynomial

$$q := \sum_{k \in (0, \infty)} y^{[0, k]}$$

whose positions $k \in (0, \infty)$ are the possible quantities of money that Caroline receives from her parents and whose directions $r \in [0, k]$ are the possible quantities of money that Caroline has remaining after spending some of it.

One component that was missing from our model was how Caroline spends her money. A section for q fills in this gap by closing the loop from the money Caroline receives to the money she has remaining. Explicitly, a section $\gamma: q \rightarrow y$ corresponds to a dependent function $(k \in (0, \infty)) \rightarrow [0, k]$ that uses the amount of money that Caroline receives to determine the amount of money that she will have remaining.

For instance, if Caroline always spends half the money she receives, then the polyboxes for the section $\gamma: q \rightarrow y$ that models this behavior can be drawn as follows:



Without γ , we do not know how much money Caroline will decide to spend; having γ makes her decision deterministic and sections this decision off from unknown variables. Of course, the position box of q , the amount of money Caroline receives, is still open to outside influence, as determined by a lens to q such as the one from Example 3.14.

The definition of Γ given in (3.37) makes Γ a functor $\mathbf{Poly} \rightarrow \mathbf{Set}^{\text{op}}$ satisfying the following.

Proposition 3.39. The sections functor $\Gamma: \mathbf{Poly} \rightarrow \mathbf{Set}^{\text{op}}$ sends

$(0, +)$ to $(1, \times)$:

$$\Gamma(0) \cong 1 \quad \text{and} \quad \Gamma(p + q) \cong \Gamma(p) \times \Gamma(q).$$

Exercise 3.40. Prove Proposition 3.39.

We conclude this section by discussing lenses between monomials, which arise in functional programming.

Example 3.41 (Lenses between monomials are bimorphic lenses). Lenses whose domains and codomains are both monomials are especially simple to write down, because they can be characterized as a pair of (standard, not dependent) functions that are independent of each other, as follows.

Given $I, J, A, B \in \mathbf{Set}$, a lens $f: Iy^A \rightarrow Jy^B$ is determined by an on-positions function $f_1: I \rightarrow J$ and an on-directions map: for each $i \in I$, an on-directions function $f_i^\sharp: B \rightarrow A$. But the data of such an on-directions map may be repackaged as a single function $f^\sharp: I \times B \rightarrow A$. We can do this because every position in I has the same direction-set A , and every position in J has the same direction-set B .

In functional programming, such a pair of functions is called a *bimorphic lens*, or a *lens* for short. In categorical terms, we may say that the monomials in **Poly** span a full subcategory of **Poly** equivalent to *the category of bimorphic lenses*, defined in [Hed18a] (here the category is named after its morphisms rather than its objects). When such a lens arises in functional programming, the two functions that comprise it are given special names:

$$\begin{aligned} \text{get} &:= f_1: I \rightarrow J; \\ \text{put} &:= f^\sharp: I \times B \rightarrow A. \end{aligned} \tag{3.42}$$

Each position $i \in I$ gets a position $f_1 i \in J$ and puts each direction $b \in B$ back to a direction $f^\sharp(i, b) \in A$.

So a natural transformation between two monomial functors is a bimorphic lens. Then a natural transformation between two polynomial functors is a more general kind of lens: a *dependent* lens, where the direction-sets depend on the positions. Favoring the dependent version, we call these natural transformations *lenses*.

Example 3.43 (Very well-behaved lenses). Consider the monomial Sy^S . Its position-set is S , and its direction-set at each position $s \in S$ is again just S . We could think of each direction as pointing to the ‘next’ position to move to. We will start to formalize this idea in Example 4.43 and continue this work throughout the following chapters.

Then here is one way we can think of a lens $f: Sy^S \rightarrow Ty^T$. Say that Otto takes positions in S , while Tim takes positions in T . Tim will act as Otto’s proxy as follows. Tim will model Otto’s position via the on-positions function $S \rightarrow T$ of f : if Otto is at position $s \in S$, then Tim will be at position $f_1 s \in T$. On the other hand, Otto will take his directions from Tim via the on-directions map $S \times T \rightarrow S$ of f : if Tim follows the direction $t' \in T$, then Otto will head from his current position $s \in S$ in the direction $f^\sharp(s, t') \in S$. We interpret these directions as new positions for Otto and Tim to move to. So as Otto moves through the positions in S , he is both modeled and directed by Tim moving through the positions in T .

With this setup, there are three conditions that we might expect the lens $f: Sy^S \rightarrow Ty^T$ to satisfy:

1. With Otto at $s \in S$, if Tim stays put at $f_1 s$ (i.e. the direction he selects at $f_1 s$ is still $f_1 s$), then Otto should stay put at s (i.e. the direction he selects at s is still s):

$$f^\sharp(s, f_1 s) = s.$$

2. Once Tim moves to $t \in T$ and Otto moves from $s \in S$ accordingly, Tim’s new position should model Otto’s new position:

$$f_1(f^\sharp(s, t)) = t.$$

3. If Tim moves to t , then to t' , Otto should end up at the same position as where he would have ended up if Tim had moved directly to t' in the first place:

$$f^\#(f^\#(s, t), t') = f^\#(s, t').$$

Such a lens is known to functional programmers as a *very well-behaved lens*; the three conditions above are its *lens laws*. We will see these conditions emerge from more general theory in Example 7.85.

3.7 Translating between natural transformations and lenses

We now know that we can specify a morphism $p \rightarrow q$ in **Poly** in one of two ways:

- in the language of functors, by specifying a natural transformation $p \rightarrow q$, i.e. for each $X \in \mathbf{Set}$, a function $p(X) \rightarrow q(X)$ such that naturality squares commute; or
- in the language of positions and directions, by specifying a lens $p \rightarrow q$, i.e. a function $f_1: p(1) \rightarrow q(1)$ and, for each $i \in p(1)$, a function $f_i^\#: q[f_1 i] \rightarrow p[i]$.

But how are these two formulations related? Given the data of a lens and that of a natural transformation between polynomials, how can we tell if they correspond to the same morphism? We want to be able to translate between these two languages.

Our Rosetta Stone turns out to be the proof of the Yoneda lemma. The lemma itself is the crux of the proof of Proposition 3.6, which states that these two formulations of morphisms between polynomials are equivalent; so unraveling these proofs reveals the translation we seek.

Proposition 3.44. Given $p, q \in \mathbf{Poly}$, let $f_1: p(1) \rightarrow q(1)$ be a function between their position-sets (like an on-positions function) and $f^\#: q[f_1(-)] \rightarrow p[-]$ be a natural transformation whose components are functions between their direction-sets (like an on-directions map). Then the isomorphism in (3.7) identifies $(f_1, f^\#)$ with the natural transformation $f: p \rightarrow q$ whose X -component

$f_X: p(X) \rightarrow q(X)$ for $X \in \mathbf{Set}$ sends each

$$(i, g) \in \sum_{i \in p(1)} X^{p[i]} \cong p(X)$$

with $i \in p(1)$ and $g: p[i] \rightarrow X$ to

$$(f_1 i, f_i^\# \circ g) \in \sum_{j \in q(1)} X^{q[j]} \cong q(X).$$

Proof. As an element of the product over I on the right-hand side of (3.7), the pair $(f_1, f^\#)$ is equivalently an I -indexed family of pairs $((f_1 i, f_i^\#))_{i \in I}$, where each pair $(f_1 i, f_i^\#)$ is an element of

$$\sum_{j \in q(1)} p[i]^{q[j]} \cong q(p[i]).$$

By the Yoneda lemma (Lemma 1.10), we have an isomorphism $q(p[i]) \cong \mathbf{Poly}(y^{p[i]}, q)$; and by the proof of the Yoneda lemma, this isomorphism sends $(f_1 i, f_i^\#)$ to the natural transformation $f^i: y^{p[i]} \rightarrow q$ whose X -component is the function $f_X^i: X^{p[i]} \rightarrow q(X)$ given by sending $g: p[i] \rightarrow X$ to

$$q(g)(f_1 i, f_i^\#) = \left(\sum_{j \in q(1)} g^{q[j]} \right) (f_1 i, f_i^\#) \quad (\text{Corollary 1.38})$$

$$= (f_1 i, g^{q[f_1 i]}(f_i^\#)) \quad (\text{Definition 1.20 and Exercise 1.21})$$

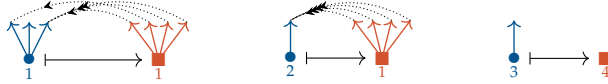
$$= (f_1 i, f_i^\# \circ g). \quad (\text{Definition 1.1})$$

Then the $p(1)$ -indexed family of natural transformations $(f^i)_{i \in p(1)}$ is an element of

$$\prod_{i \in p(1)} \mathbf{Poly}(y^{p[i]}, q) \cong \mathbf{Poly}\left(\sum_{i \in p(1)} y^{p[i]}, q\right),$$

where the isomorphism follows from the universal property of coproducts, as in the proof of Proposition 3.6. Unwinding this isomorphism, we find that $(f^i)_{i \in I}$ corresponds to the natural transformation f from $\sum_{i \in p(1)} y^{p[i]} \cong p$ to q that we desire. $\square$

Example 3.45. Let us return once more to the polynomials $p := y^3 + 2y$ and $q := y^4 + y^2 + 2$ from Example 3.11 and the lens $f: p \rightarrow q$ depicted below:



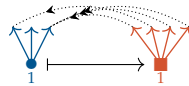
Fix a set $X := \{a, b, c, d, e\}$. When viewed as a natural transformation, f has as its X -component a function $f_X: p(X) \rightarrow q(X)$. In other words, for each element of $p(X)$, the lens f should tell us how to obtain an element of $q(X)$.

We saw in Example 2.31 that each $(i, g) \in p(X)$ may be drawn as a p -corolla (corresponding to i) whose leaves are labeled with elements of X (according to g). For example, here we draw $(1, g) \in p(X)$, where $g: p[1] \rightarrow X$ is given by $1 \mapsto c$, $2 \mapsto e$, and $3 \mapsto a$:



Similarly, each element of $q(X)$ can be drawn as a q -corolla whose leaves are labeled with elements of X . So what element of $q(X)$ is $f_X(1, g)$?

Proposition 3.44 tells us that $f_X(1, g) = (f_1(1), f_1^\# \circ g)$, so we need only focus on the behavior of f at p -position 1:



To draw $(f_1(1), f_1^\# \circ g)$, we first draw the q -corolla corresponding to $f_1(1)$, the corolla on the right-hand side above. Then we label each leaf of that corolla by following the arrow from that leaf to a $p[1]$ -leaf, and use the label there from (3.46) (as prescribed by g). So $f_X(1, g)$ looks like



Proposition 3.44 lets us translate from lenses to natural transformations. The following corollary tells us how to go in the other direction. In particular, it justifies the notation f_1 for the on-positions function of f : it is the 1-component of f as a natural transformation.

Corollary 3.47. Let p and q be polynomial functors, and let $f: p \rightarrow q$ be a natural transformation between them. Then the isomorphism in (3.7) sends f to the lens whose on-positions function $f_1: p(1) \rightarrow q(1)$ is the 1-component of f and whose on-directions map $f^\sharp: q[f_1(-)] \rightarrow p[-]$ satisfies, for all $i \in p(1)$,

$$(f_1 i, f_i^\sharp) = f_{p[i]}(i, \text{id}_{p[i]}).$$

Proof. By Proposition 3.44, the 1-component of f is a function $p(1) \rightarrow q(1)$ sending every $i \in p(1)$ to $f_1 i \in q(1)$, so the on-positions function f_1 is indeed equal to the 1-component of f . Moreover, for each $i \in p(1)$, the $p[i]$ -component $f_{p[i]}: p(p[i]) \rightarrow q(p[i])$ of f sends $(i, \text{id}_{p[i]}) \in p(p[i])$ to $(f_1 i, f_i^\sharp \circ \text{id}_{p[i]}) = (f_1 i, f_i^\sharp)$. $\square$

3.8 Identity lenses and lens composition

Thus far, we have seen how the category **Poly** of polynomial functors and natural transformations can be identified with the category of indexed families of sets and lenses. But in order to actually discuss the latter category, we need to be able to give identity lenses and describe how lenses compose. We can do so by translating between lenses and natural transformations.

For instance, given a polynomial p , its identity lens should correspond to the identity natural transformation of p as a functor.

Exercise 3.48 (Identity lenses). For $p \in \mathbf{Poly}$, let $\text{id}_p: p \rightarrow p$ be its identity natural transformation: its X -component $(\text{id}_p)_X: p(X) \rightarrow p(X)$ for $X \in \mathbf{Set}$ is the identity function on $p(X)$; that is, $(\text{id}_p)_X = \text{id}_{p(X)}$.

Use Corollary 3.47 to demonstrate that the on-positions function

$(\text{id}_p)_1: p(1) \rightarrow p(1)$ as well as, for each $i \in p(1)$, the on-directions function $(\text{id}_p)_i^\sharp: p[(\text{id}_p)_1 i] \rightarrow p[i]$ are all identity functions.

Similarly, we may infer how two lenses compose by translating them to natural transformations, composing those, then translating back to lenses.

Exercise 3.49 (Composing lenses). For $p, q, r \in \mathbf{Poly}$, let $f: p \rightarrow q$ and $g: q \rightarrow r$ be natural transformations, and let $h := f \circ g$ be their composite, whose X -component $h_X: p(X) \rightarrow r(X)$ for $X \in \mathbf{Set}$ is the composite of the X -components of f and g ; that is, $h_X = f_X \circ g_X$.

Use Proposition 3.44 and Corollary 3.47 to show that the on-positions function $h_1: p(1) \rightarrow r(1)$ of h is given by $h_1 = f_1 \circ g_1$, while the on-directions function $h_i^\sharp$ of h for $i \in p(1)$ is given by $h_i^\sharp = g_{f_1 i}^\sharp \circ f_i^\sharp$.

The following proposition, a restatement of the previous exercise, allows us to interpret commutative diagrams of polynomials in $\mathbf{Poly}$ in terms of commutative diagrams of their position- and direction-sets in $\mathbf{Set}$.

Proposition 3.50. Given $p, q, r \in \mathbf{Poly}$ and lenses $f: p \rightarrow q, g: q \rightarrow r$, and $h: p \rightarrow r$, the diagram

$$\begin{array}{ccc} p & \xrightarrow{f} & q \\ & \searrow h & \downarrow g \\ & & r \end{array}$$

commutes in $\mathbf{Poly}$ if and only if the forward on-positions diagram

$$\begin{array}{ccc} p(1) & \xrightarrow{f_1} & q(1) \\ & \searrow h_1 & \downarrow g_1 \\ & & r(1) \end{array}$$

commutes in $\mathbf{Set}$ and, for each $i \in p(1)$, the backward on-directions

diagram

$$\begin{array}{ccc}
 p[i] & \xleftarrow{f_i^\sharp} & q[f_1 i] \\
 & \nwarrow h_i^\sharp & \uparrow g_{f_1 i}^\sharp \\
 & & r[h_1 i]
 \end{array}$$

commutes in **Set**.

We can use this fact to determine whether a given diagram in **Poly** commutes, as in the following exercise.

Exercise 3.51. Using Proposition 3.50, verify explicitly that, for $p, q \in \mathbf{Poly}$, the polynomial $p + q$ given by the binary sum of p and q satisfies the universal property of the coproduct of p and q . That is, provide lenses $\iota: p \rightarrow p + q$ and $\kappa: q \rightarrow p + q$, then show that for any other polynomial r equipped with lenses $f: p \rightarrow r$ and $g: q \rightarrow r$, there exists a unique lens $h: p + q \rightarrow r$ (shown dashed) making the following diagram commute:

$$\begin{array}{ccccc}
 p & \xrightarrow{\iota} & p + q & \xleftarrow{\kappa} & q \\
 & \searrow f & \downarrow h & \swarrow g & \\
 & & r & &
 \end{array} \tag{3.52}$$

Now that we know how lens composition works in **Poly**, we have a better handle on how it behaves categorically. For instance, we can verify functoriality in **Poly**, as in the following exercise.

Exercise 3.53 (A functor $\mathbf{Top} \rightarrow \mathbf{Poly}$). This exercise is for those who know what topological spaces and continuous maps are. It will not be used again in this book.

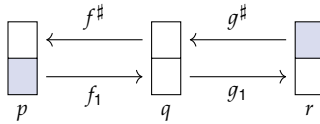
1. Given a topological space X , define a polynomial p_X whose positions are the points in X and whose directions at $x \in X$ are the open neighborhoods of x . That is,

$$p_X := \sum_{x \in X} y^{\{U \subseteq X \mid x \in U, U \text{ open}\}}.$$

- Given a continuous map $f: X \rightarrow Y$, define a lens $p_X \rightarrow p_Y$ either by writing down its formula or drawing it in polyboxes.
2. Show that the assignment above defines a functor **Top** $\rightarrow$ **Poly**.
 3. Is this functor full? Is it faithful?

3.9 Polybox pictures of lens composition

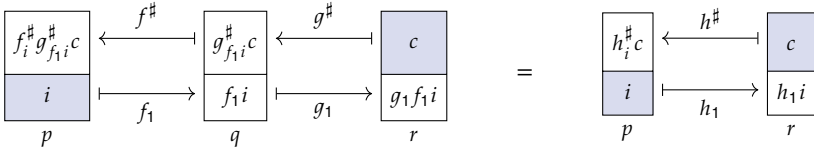
Given lenses $f: p \rightarrow q$ and $g: q \rightarrow r$, we can piece their polyboxes together to form polyboxes for their composite, $f \circ g: p \rightarrow r$:



The position box for q , which would be blue as part of the polyboxes for $g: q \rightarrow r$ alone, is instead filled in via f_1 ; similarly, the direction box for q , which would be blue as part of the polyboxes for just $f: p \rightarrow q$, is filled in via $g^\#$. This forms a spreadsheet-filling protocol that acts as the polyboxes for $f \circ g$.

As we follow the arrows from left to right and up and left again, take care to note that the arrow $g^\#$ depends not only on the direction box of r , but also the position box of q that came before it. Similarly, $f^\#$ depends on both the position box of p and the direction box of q . On the other hand, the arrow g_1 depends only on the position box of q , and not the position box of p that came before it: g_1 is the on-positions function for a lens $q \rightarrow r$ and therefore depends only on its domain q . (Of course, changing the position box of p may change the position box of q via f_1 , thus indirectly affecting what g_1 enters in the position box for r ; we mean that if the position box of p changes but the position box of q does not, g_1 will not change the position box of r .) Similarly, $g^\#$ does not depend on the position box of p , and $f^\#$ does not depend on either box of r . The key is to let each arrow depend on exactly the boxes that come before it in the domain and codomain of the lens that the arrow is a part of.

If we have another lens $h: p \rightarrow r$, we can interpret the equation $f \circ g = h$ by filling in their polyboxes and comparing them:

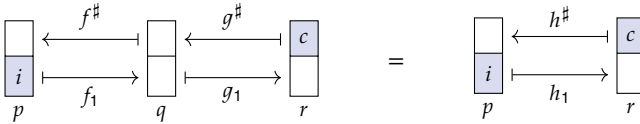


Here we have filled the blue boxes on either side with the same entries. Then if we match up the uncolored boxes in the domain and codomain on either side, we can read off the equations

$$g_1 f_1 i = h_1 i \quad \text{and} \quad f_i^\# g_{f_1 i}^\# c = h_i^\# c$$

for every p -position i and $r[h_1 i]$ -direction c , which agrees with Exercise 3.49 and Proposition 3.50. Throughout this book, we will often read off equalities of positions and directions from polybox pictures of lens equations in this way.

Note that there is redundancy in the above polybox picture: we have filled in all the boxes for clarity, but their entries are determined by the entries in the blue boxes and the labels on the arrows. So we may omit the entries in the uncolored boxes without losing information, leaving the reader to fill in the blanks:



Remark 3.54. The reader may be concerned that when working with polyboxes, we refer to “spreadsheets” and “protocols” without being rigorous about what they are or what it means to set them equal. We choose to elide this issue to highlight the graphical intuition rather than grinding through the details. This is not to say our work with polyboxes will lack rigor moving forward—if you are particularly worried, you should think of polyboxes as an alternate way to present information about indexed families of sets, dependent functions, and sum

and product sets that can be systematically translated—via elementary steps, though perhaps with some laborious bookkeeping—into the more standard $\in$ and $\sum$ and $\prod$ notation we have been using thus far.

For example, given lenses $f: p \rightarrow q$ and $g: q \rightarrow r$, the polyboxes on the left-hand side of the equation above should be interpreted as the element of the set

$$\prod_{i \in p(1)} \sum_{k \in r(1)} p[i]^{r[k]} \cong \mathbf{Poly}(p, r)$$

corresponding to the lens $p \rightarrow r$ whose on-positions function $p(1) \rightarrow r(1)$ is the composite of the on-positions functions f_1 and g_1 and whose on-directions function $r[g_1 f_1 i] \rightarrow p[i]$ at $i \in p(1)$ is equal to the composite of the on-directions functions $g_{f_1 i}^\#$ and $f_i^\#$. In other words, the polyboxes represent the composite lens $f \circ g$. But the polyboxes show how lenses pass positions and directions back and forth far more legibly than the last two sentences can. Throughout the rest of this book, we will see how this polybox notation provides immediate, reader-friendly computations and justifications; but all these results can be translated back into more grounded mathematical language as desired.

Example 3.55 (Modeling with a composite lens in polyboxes). By composing the lens $f: p \rightarrow q$ from Example 3.14 that models the exchange of money between Caroline (modeled by q) and her parents (modeled by p) with the lens $\gamma: q \rightarrow y$ from Example 3.38 that models how Caroline spends her money, we obtain a lens $f \circ \gamma: p \rightarrow y$ that models how Caroline's parents spend their money through Caroline. The polybox picture of the composite lens $f \circ \gamma$ is given by merging the polybox pictures of f and γ :

$$\begin{array}{c}
 \boxed{\begin{array}{c} \left(\frac{i}{i+j} \cdot \frac{i+j}{2}, \frac{j}{i+j} \cdot \frac{i+j}{2} \right) \\ (i, j) \end{array}} \xleftarrow{f^\#} \boxed{\begin{array}{c} \frac{i+j}{2} \\ i+j \end{array}} \xleftarrow{\gamma} \\
 \xrightarrow{f_1} \boxed{\begin{array}{c} \frac{i+j}{2} \\ i+j \end{array}} \xrightarrow{\gamma} \boxed{\begin{array}{c} (i/2, j/2) \\ (i, j) \end{array}} \xleftarrow{f \circ \gamma} \\
 p \qquad \qquad q \qquad \qquad p
 \end{array} =$$

Here $(i, j) \in p(1) = (0, 20] \times (0, 20]$. The right-hand side summarizes what happens to the parents: if the first parent gives away i dollars and the second parent gives away j dollars, eventually the first parent will receive $i/2$ dollars and the second parent will receive $j/2$ dollars. The factored left-hand side describes how this happens: the parents give i and j dollars respectively to Caroline, who takes the $i + j$ dollars total and spends half of it. She then returns the remaining half to her parents, splitting the money proportionately according to the amount each parent contributed.

3.10 Symmetric monoidal products of polynomial functors

One of the reasons **Poly** is so versatile is that there is an abundance of monoidal structures on it. Monoidal structures are the key ingredient to many applications of categories to real-world settings, and **Poly** is no different in that regard. As a bonus, if you know how to add and multiply polynomials from high school algebra, then you already know how to compute two of the monoidal products on **Poly**.

We have already seen one of these monoidal structures on **Poly**: the cocartesian monoidal structure, which gives **Poly** its finite coproducts. In fact, we know from Proposition 3.3 that **Poly** has all coproducts: they are given by an operation that looks just like addition. It turns out **Poly** has all products as well, giving it a cartesian monoidal structure that looks just like multiplication.

Proposition 3.56. The category **Poly** has arbitrary products, coinciding with products in $\mathbf{Set}^{\mathbf{Set}}$ given by the operation $\prod_{i \in I}$.

Proof. Unsurprisingly, the proof is very similar to that of Proposition 3.3.

By Corollary 1.38, the category $\mathbf{Set}^{\mathbf{Set}}$ has arbitrary products given by $\prod_{i \in I}$. The full subcategory inclusion $\mathbf{Poly} \rightarrow \mathbf{Set}^{\mathbf{Set}}$ reflects these products. It remains to show that **Poly** is closed under the operation $\prod_{i \in I}$.

By Proposition 1.40, $\mathbf{Set}^{\mathbf{Set}}$ is completely distributive. Hence, given polynomials $(p_i)_{i \in I}$, we can use (1.32) to write their product in $\mathbf{Set}^{\mathbf{Set}}$ as

$$\prod_{i \in I} p_i \cong \prod_{i \in I} \sum_{j \in p_i(1)} y^{p_i[j]} \cong \sum_{\bar{j} \in \prod_{i \in I} p_i(1)} \prod_{i \in I} y^{p_i[\bar{j}i]} \cong \sum_{\bar{j} \in \prod_{i \in I} p_i(1)} y^{\sum_{i \in I} p_i[\bar{j}i]}, \quad (3.57)$$

which, as a coproduct of representables, is in **Poly**. $\square$

Corollary 3.58. The category **Poly** is completely distributive.

Proof. This is a direct consequence of the fact that **Poly** has arbitrary (co)products coinciding with (co)products in $\mathbf{Set}^{\mathbf{Set}}$ (Propositions 3.3 and 3.56) and the fact that $\mathbf{Set}^{\mathbf{Set}}$ itself is completely distributive (Proposition 1.40). $\square$

The result above will allow us to apply (1.32), or sometimes specifically (1.34), to push $\prod$'s past $\sum$'s of polynomials whenever we so desire.

Exercise 3.59. Use (3.7) to verify that

$$\mathbf{Poly}\left(q, \prod_{i \in I} p_i\right) \cong \prod_{i \in I} \mathbf{Poly}(q, p_i)$$

for all polynomials $(p_i)_{i \in I}$ and q , as one would expect from the universal property of products.

Exercise 3.60. Let $p_1 := y + 1$, $p_2 := y + 2$, and $p_3 := y^2$. What is $\prod_{i \in 3} p_i$ according to (3.57)? Is the answer what you would expect?

It follows from (3.57) that the terminal object of **Poly** is 1, and that binary products are given by

$$p \times q \cong \sum_{i \in p(1)} \sum_{j \in q(1)} y^{p[i] + q[j]}. \quad (3.61)$$

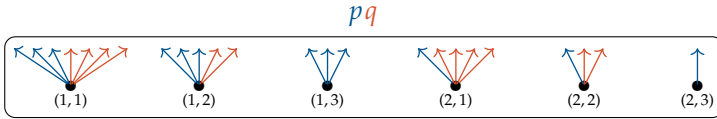
We will sometimes write pq rather than $p \times q$:

$$pq := p \times q.$$

Example 3.62. We can draw the product of two polynomials in terms of their associated forests. Let $p := y^3 + y$ and $q := y^4 + y^2 + 1$.



Then $pq \cong y^7 + 2y^5 + 2y^3 + y$. We take all pairs of positions, and for each pair we take the disjoint union of the directions.



In practice, we can multiply polynomial functors the same way we would multiply two polynomials in high school algebra.

Exercise 3.63.

1. Show that for sets A_1, B_1, A_2, B_2 , we have

$$B_1 y^{A_1} \times B_2 y^{A_2} \cong B_1 B_2 y^{A_1 + A_2}.$$

2. Show that for sets $(A_i)_{i \in I}, (A_j)_{j \in J}, (B_i)_{i \in I}$, and $(B_j)_{j \in J}$, we have

$$\left(\sum_{i \in I} B_i y^{A_i} \right) \times \left(\sum_{j \in J} B_j y^{A_j} \right) \cong \sum_{i \in I} \sum_{j \in J} B_i B_j y^{A_i + A_j}.$$

As lenses, the canonical projections $\pi: pq \rightarrow p$ and $\varphi: pq \rightarrow q$ behave quite naturally: on positions, they are the projections from $(pq)(1) \cong p(1) \times q(1)$ to $p(1)$ and $q(1)$, respectively; on directions, they are the inclusions $p[i] \rightarrow p[i] + q[j]$ and $q[j] \rightarrow p[i] + q[j]$ for each position (i, j) of pq .

Exercise 3.64. Verify that, for $p, q \in \mathbf{Poly}$, the polynomial pq given by (3.61) along with the lenses $\pi: pq \rightarrow p$ and $\varphi: pq \rightarrow q$ described

above satisfy the universal property of the product of p and q .

Much of Part II will focus on the features of another monoidal structure, an asymmetric one, whose definition we will postpone—we will save its surprises for when we can better savor them. But here we introduce a third symmetric monoidal structure, given by an operation you were not allowed to do to polynomials back in high school.

Definition 3.65 (Parallel product of polynomials). Let p and q be polynomials. Their *parallel product* (also called *Dirichlet product*), denoted $p \otimes q$, is given by the formula

$$p \otimes q := \sum_{i \in p(1)} \sum_{j \in q(1)} y^{p[i] \times q[j]}. \quad (3.66)$$

One should compare this with the formula for the product of polynomials shown in (3.61). The difference is that the parallel product multiplies exponents where the categorical product adds them.

Exercise 3.67.

1. Show that for sets A_1, B_1, A_2, B_2 , we have

$$B_1 y^{A_1} \otimes B_2 y^{A_2} \cong B_1 B_2 y^{A_1 A_2}.$$

2. Show that for sets $(A_i)_{i \in I}, (A_j)_{j \in J}, (B_i)_{i \in I}$, and $(B_j)_{j \in J}$, we have

$$\left(\sum_{i \in I} B_i y^{A_i} \right) \otimes \left(\sum_{j \in J} B_j y^{A_j} \right) \cong \sum_{i \in I} \sum_{j \in J} B_i B_j y^{A_i A_j}.$$

Exercise 3.68.

1. If $p := A$ and $q := B$ are constant polynomials, what is $p \otimes q$?
2. If $p := Ay$ and $q := By$ are linear polynomials, what is $p \otimes q$?
3. For arbitrary $p, q \in \mathbf{Poly}$, show that the sets $(p \otimes q)(1)$ and $p(1) \times q(1)$ are isomorphic.

Exercise 3.69. Consider the polynomials $p := 2y^2 + 3y$ and $q := y^4 + 3y^3$.

1. What is $p \times q$?
2. What is $p \otimes q$?
3. Expand the following expression in the variable y according to the ordinary laws of arithmetic:

$$(2 \cdot 2^y + 3 \cdot 1^y) \cdot (1 \cdot 4^y + 3 \cdot 3^y).$$

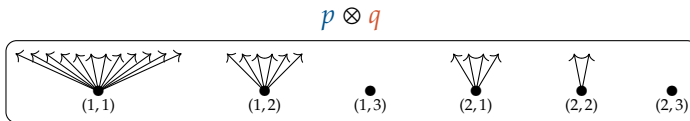
The factors of the above product are called *Dirichlet series*.

4. Describe the connection between the last two parts. (This is why the parallel product $\otimes$ is also known as the *Dirichlet product*.)

Example 3.70. We can draw the parallel product of two polynomials in terms of their associated forests. Let $p := y^3 + y$ and $q := y^4 + y^2 + 1$.



Then $p \otimes q \cong y^{12} + y^6 + y^4 + y^2 + 2$. We take all pairs of positions, and for each pair we take the product of the directions.



Exercise 3.71. Let $p := y^2 + y$ and $q := 2y^4$.

1. Draw p and q as corolla forests.
2. Draw $pq = p \times q$ as a corolla forest.
3. Draw $p \otimes q$ as a corolla forest.

Exercise 3.72. Let $p, q, r \in \mathbf{Poly}$ be any polynomials.

1. Show that there is an isomorphism $p \otimes y \cong p$.
2. Show that there is an isomorphism $(p \otimes q) \otimes r \cong p \otimes (q \otimes r)$.
3. Show that there is an isomorphism $p \otimes q \cong q \otimes p$.

In Exercise 3.72, we have gone most of the way to proving that $(\mathbf{Poly}, y, \otimes)$ is a symmetric monoidal category. We sketch the rest of the proof as follows.

Proposition 3.73. The category **Poly** has a symmetric monoidal structure $(y, \otimes)$ where $\otimes$ is the parallel product from Definition 3.65.

Sketch of proof. Given two lenses $f: p \rightarrow p'$ and $g: q \rightarrow q'$, we need to define a lens $(f \otimes g): (p \otimes q) \rightarrow (p' \otimes q')$. This is easiest to define using polyboxes, keeping in mind that the positions and directions of a parallel product are pairs of positions and directions of its constituent factors:

$$\begin{array}{ccc}
 p \otimes q & & p' \otimes q' \\
 \begin{array}{|c|} \hline (f_i^\# a, g_j^\# b) \\ \hline (i, j) \\ \hline \end{array} & \begin{array}{c} \xleftarrow{(f \otimes g)^\#} \\ \xrightarrow{(f \otimes g)_1} \end{array} & \begin{array}{|c|} \hline (a, b) \\ \hline (f_1 i, g_1 j) \\ \hline \end{array}
 \end{array}$$

Here $i \in p(1)$, $j \in q(1)$, $a \in p'[f_1 i]$, and $b \in q'[g_1 j]$.

Then Exercise 3.72 gives us the unitors, associator, and braiding. We have not proven the functoriality of $\otimes$, the naturality of the isomorphisms from Exercise 3.72, or all the coherences between these isomorphisms, but we ask the reader to take them on trust or to check them for themselves. Alternatively, we may invoke the Day convolution to obtain the monoidal structure $(y, \otimes)$ directly: see Proposition 3.79. $\square$

Exercise 3.74.

1. What is $(3y^5 + 6y^2) \otimes 4$? (Hint: $4 = 4y^0$.)
2. Is the class of constant polynomials a $\otimes$ -ideal; that is, is the parallel product of a polynomial and a constant polynomial always a constant?

Exercise 3.75. Which of the following special classes of polynomials are closed under $\otimes$? Note also whether they contain y .

1. The class $\{Ay^0 \mid A \in \mathbf{Set}\}$ of constant polynomials.
2. The class $\{Ay \mid A \in \mathbf{Set}\}$ of linear polynomials.
3. The class $\{Ay + B \mid A, B \in \mathbf{Set}\}$ of affine polynomials.
4. The class $\{Ay^2 + By + C \mid A, B, C \in \mathbf{Set}\}$ of quadratic polynomials.
5. The class $\{Ay^B \mid A, B \in \mathbf{Set}\}$ of monomials.
6. The class $\{Sy^S \mid S \in \mathbf{Set}\}$.
7. The class $\{p \in \mathbf{Poly} \mid p(1) \text{ is finite}\}$.

Exercise 3.76. What is the smallest class of polynomials that is closed under $\otimes$ and contains y ?

Exercise 3.77. Show that for any $p_1, p_2, q \in \mathbf{Poly}$ there is an isomorphism

$$(p_1 + p_2) \otimes q \cong (p_1 \otimes q) + (p_2 \otimes q).$$

Remark 3.78. Monoids in $\mathbf{Poly}$ with respect to the parallel product $\otimes$ are particularly interesting—they have a kind of collective semantics, letting agents aggregate their contributions and distribute returns on those contributions in a coherent way. We leave discussion of them to other work, so as not to distract us from our main story: see [NS22].

There is a more general way to obtain monoidal structures on $\mathbf{Poly}$ like $\times$ and $\otimes$ using a construction known as the *Day convolution*, defined by a special kind of colimit known as a *coend*. If you have not seen the Day convolution or coends before, do not fret: we will not use them elsewhere in the book, and rest assured that the fact about coends known as the *co-Yoneda lemma* employed in the following proof is a standard and purely formal result.

Proposition 3.79. For any monoidal structure $(I, \star)$ on $\mathbf{Set}$, there is a corresponding monoidal structure $(y^I, \odot)$ on $\mathbf{Poly}$, where $\odot$ is the

Day convolution. Moreover, $\odot$ distributes over coproducts.

In the case of $(0, +)$ and $(1, \times)$, this procedure returns the $(1, \times)$ and $(y, \otimes)$ monoidal structures respectively.

Proof. Any monoidal structure $(I, \star)$ on **Set** induces a monoidal structure on $\mathbf{Set}^{\mathbf{Set}}$ with the Day convolution $\odot$ as the tensor product and y^I as the unit. To prove that this monoidal structure restricts to **Poly**, it suffices to show that **Poly** is closed under the Day convolution.

Given polynomials p and q , their Day convolution in $\mathbf{Set}^{\mathbf{Set}}$ is given by the coend

$$p \odot q \cong \int^{(A,B) \in \mathbf{Set}^2} y^{A \star B} \times p(A) \times q(B). \quad (3.80)$$

We can rewrite the product $p(A) \times q(B)$ as

$$p(A) \times q(B) \cong \left(\sum_{i \in p(1)} A^{p[i]} \right) \times \left(\sum_{j \in q(1)} B^{q[j]} \right) \cong \sum_{(i,j) \in p(1) \times q(1)} A^{p[i]} \times B^{q[j]}.$$

So because products distribute over coproducts in $\mathbf{Set}^{\mathbf{Set}}$ and coends always commute with coproducts (as they are both colimits), we can rewrite (3.80) as

$$\begin{aligned} p \odot q &\cong \sum_{(i,j) \in p(1) \times q(1)} \int^{(A,B) \in \mathbf{Set}^2} y^{A \star B} \times A^{p[i]} \times B^{q[j]} \\ &\cong \sum_{(i,j) \in p(1) \times q(1)} \int^{(A,B) \in \mathbf{Set}^2} y^{A \star B} \times \mathbf{Set}^2((p[i], q[j]), (A, B)), \end{aligned}$$

which, by the co-Yoneda lemma, can be rewritten as

$$p \odot q \cong \sum_{(i,j) \in p(1) \times q(1)} y^{p[i] \star q[j]}, \quad (3.81)$$

which is in **Poly**. That the Day convolution distributes over coproducts also follows from the fact that products distribute over coproducts in $\mathbf{Set}^{\mathbf{Set}}$ and that coends commute with coproducts; or, alternatively, directly from (3.81).

We observe that (3.81) gives $(y^I, \odot) = (1, \times)$ when $(I, \star) := (0, +)$ and $(y^I, \odot) = (y, \otimes)$ when $(I, \star) := (1, \times)$. $\square$

Exercise 3.82. There is a monoidal structure on **Set** whose unit is 0 and whose product is given by $(A, B) \mapsto A + AB + B$.

1. Verify that the operation $(A, B) \mapsto A + AB + B$ on **Set** is associative.
2. Verify that 0 is the unit for the above operation.
3. Let $(1, \odot)$ denote the corresponding monoidal structure on **Poly** obtained via Proposition 3.79. Compute the monoidal product $(y^3 + y) \odot (2y^2 + 2)$.

3.11 Summary and further reading

In this chapter, we introduced the category **Poly**, whose objects are polynomial functors and whose morphisms are the natural transformations between them. We call these natural transformations *dependent lenses*, or *lenses* for short. We also proved our first categorical property of **Poly**: that it has all small coproducts.

The main result of this chapter was a concrete characterization of our dependent lenses between polynomial functors. A dependent lens $f: p \rightarrow q$ is characterized by its

- *on-positions function*, a function that we denote by $f_1: p(1) \rightarrow q(1)$ sending p -positions forward to q -positions; and its
- *on-directions functions*, one for each p -position i that we denote by $f_i^\# : q[f_1 i] \rightarrow p[i]$ sending $q[f_1 i]$ -directions backward to $p[i]$ -directions.

This forward-backward relationship is what makes dependent lenses so well-suited for modeling *interaction protocols*. Given two agents with positions and directions, a dependent lens between them defines an interaction protocol that describes how the position of the first agent determines the position of the second agent and how the direction of the second agent determines the direction of the first. This perspective is exhibited by our corolla and polybox pictures for lenses. We studied examples of lenses between special polynomials: in particular, lenses between monomials are known as *bimorphic lenses* in functional programming literature.

We then unwound our interpretation of natural transformations between polynomials as dependent lenses with on-positions and on-directions functions to describe what happens to these functions when lenses compose. This gave us an accessible way to interpret commutative diagrams in **Poly** that is particularly convenient to express using polyboxes.

Finally, we considered various categorical structures on **Poly**, e.g. that it has all products and coproducts, and that these distribute. We also discussed how one can take any monoidal product $\star$ from **Set** and lift it to a monoidal product $\cdot$ on **Poly**:

$$p_1 \odot p_2 := \sum_{(i_1, i_2) \in p_1(1) \times p_2(1)} y^{p_1[i_1] \cdot p_2[i_2]}.$$

A special case of this is the product structure $\times$ on **Poly**, which emerges from the coproduct structure $+$ on **Set**. The other case of interest is the parallel (or Dirichlet) product structure $\otimes$ on **Poly**, which emerges from the product structure $\times$ on **Set**:

$$p_1 \otimes p_2 := \sum_{(i_1, i_2) \in p_1(1) \times p_2(1)} y^{p_1[i_1] \times p_2[i_2]}.$$

Variants of lenses are studied in compositional game theory [Hed+16; Hed17; Hed18a; Hed18b], in categorical database theory [JRW12], in functional programming and programming language theory [BPV06; OCo11; Abo+16], and in more generalized categorical settings [GJ12; Spi19].

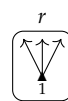
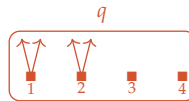
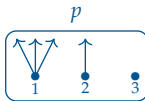
3.12 Exercise solutions

Solution to Exercise 3.2.

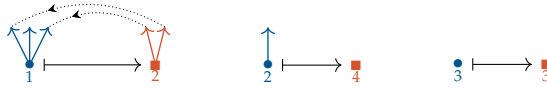
We know that **Poly** is the full subcategory of $\mathbf{Set}^{\mathbf{Set}}$ spanned by polynomial functors, including y^S and q . So $\mathbf{Poly}(y^S, q) = \mathbf{Set}^{\mathbf{Set}}(y^S, q)$. Hence the natural isomorphism $\mathbf{Poly}(y^S, q) \cong q(S)$ follows directly from the Yoneda lemma (Lemma 1.10) with $F := q$.

Solution to Exercise 3.12.

- Here are the corolla forests associated to $p := y^3 + y + 1$, $q := y^2 + y^2 + 2$, and $r := y^3$ (with each root labeled for convenience).



2. Here is one possible lens $p \rightarrow q$ (you may have drawn others).



3. As depicted, our lens assigns to the first position of p the second position of q , whose first and second directions are passed back to the third and first directions, respectively, of the first position of p . Then the second position of p is assigned the fourth position of q , which has no directions; effectively, the choice of direction of the second position of p has been canceled. Finally, the third position of p is assigned the third position of q ; here neither position has any directions.
4. There cannot be a lens $p \rightarrow r$ for the following reason: if we send the third position of p , which has no directions, to the sole position of r , which has 3 directions, then there is no way to pass a choice of one of those 3 directions back to any of the options on the third menu of p , as there are no such options.

Solution to Exercise 3.16.

1. We let $p := y^3 + 1$ (you could have selected others) and draw both y^2 and p as corolla forests, labeling each root for convenience.



2. When constructing a lens $y^2 \rightarrow p$, the unique position of y^2 can be sent to either p -position. If it is sent to the first p -position, then each of the 3 directions in $p[1]$ must be sent to one of the 2 directions of y^2 , for a total of $2^3 = 8$ lenses. Otherwise, the unique position of y^2 is sent to the second p -position, at which there are no directions; so there is exactly 1 lens like this. Hence there are $8 + 1 = 9$ lenses $y^2 \rightarrow p$.
3. The cardinality of $p(2)$ is $|2^3 + 1| = 9$, which agrees with the previous answer, as predicted by the Yoneda lemma.

Solution to Exercise 3.17.

By (3.7), we have for all $p, q \in \mathbf{Poly}$ that

$$|\mathbf{Poly}(p, q)| = \prod_{i \in p(1)} |q(p[i])|.$$

1. If $p := y^3$ and $q := y^4$, then

$$|\mathbf{Poly}(p, q)| = \prod_{i \in 1} |p[i]|^4 = 3^4 = 81.$$

2. If $p := y^3 + 1$ and $q := y^4$, then

$$|\mathbf{Poly}(p, q)| = \prod_{i \in 2} |p[i]|^4 = 3^4 \cdot 0^4 = 0.$$

3. If $p := y^3 + 1$ and $q := y^4 + 1$, then

$$|\mathbf{Poly}(p, q)| = \prod_{i \in 2} \left(|p[i]|^4 + 1 \right) = (3^4 + 1)(0^4 + 1) = 82.$$

4. If $p := 4y^3 + 3y^2 + y$ and $q := y$, then

$$|\mathbf{Poly}(p, q)| = \prod_{i \in 8} |p[i]| = 3^4 \cdot 2^3 \cdot 1 = 648.$$

5. If $p := 4y^3$ and $q := 3y$, then

$$|\mathbf{Poly}(p, q)| = \prod_{i \in 4} 3|p[i]| = (3 \cdot 3)^4 = 6561.$$

Solution to Exercise 3.18.

1. By (3.7), it suffices to show that for all $i \in p(1)$ and $j \in q(1)$, we have

$$p[i]^{q[j]} \cong \prod_{b \in q[j]} \sum_{a \in p[i]} 1.$$

Indeed, by (1.30) and Exercise 1.17, we have

$$\prod_{b \in q[j]} \sum_{a \in p[i]} 1 \cong \sum_{\bar{a}: q[j] \rightarrow p[i]} \prod_{b \in q[j]} 1 \quad (1.30)$$

$$\cong \sum_{\bar{a}: q[j] \rightarrow p[i]} 1 \quad (\text{Exercise 1.17 \#2})$$

$$\cong \mathbf{Set}(q[j], p[i]) \quad (\text{Exercise 1.17 \#1})$$

$$\cong p[i]^{q[j]}.$$

2. By (3.7) and (1.30), we have

$$\mathbf{Poly}(p, q) \cong \prod_{i \in p(1)} \sum_{j \in q(1)} p[i]^{q[j]} \quad (3.7)$$

$$\cong \sum_{f_1: p(1) \rightarrow q(1)} \prod_{i \in p(1)} p[i]^{q[f_1 i]} \quad (1.30)$$

$$\cong \sum_{f_1: p(1) \rightarrow q(1)} \prod_{j \in q(1)} \prod_{\substack{i \in p(1), \\ f_1 i = j}} p[i]^{q[j]} \quad (*)$$

$$\cong \sum_{f_1: p(1) \rightarrow q(1)} \prod_{j \in q(1)} \mathbf{Set} \left(q[j], \prod_{\substack{i \in p(1), \\ f_1 i = j}} p[i] \right),$$

(Universal property of products)

where (*) follows from the fact that for any function $f_1: p(1) \rightarrow q(1)$, its domain $p(1)$ can be written as the disjoint union of preimages $f_1^{-1}(j) = \{i \in p(1) \mid f_1 i = j\}$ for each $j \in q(1)$.

3. To explain how an element of the set

$$D_{p,q} := \sum_{f_1: p(1) \rightarrow q(1)} \prod_{j \in q(1)} \mathbf{Set} \left(q[j], \prod_{\substack{i \in p(1), \\ f_1 i = j}} p[i] \right)$$

corresponds to a lens $p \rightarrow q$, we first give the instructions for choosing an element of $D_{p,q}$ as a nested list.

To choose an element of $D_{p,q}$:

1. choose a function $f_1: p(1) \rightarrow q(1)$;
2. for each element $j \in q(1)$:
 1. for each element of $q[j]$:
 1. for each element $i \in p(1)$ satisfying $f_1 i = j$:
 1. choose an element of $p[i]$.

So f_1 sends each p -position to a q -position, as an on-positions function should. Then for each q -position j , each $q[j]$ -direction b , and each p -position i that f_1 sends to j , we choose an element of $p[i]$ that $f_i^\# : q[j] \rightarrow p[i]$ assigns to b . As every p -position i is sent to some q -position j by f_1 , this completely characterizes $f_i^\#$ for every p -position i .

Solution to Exercise 3.21.

We have

$$\begin{aligned}
 \text{Poly} \left(\sum_{(i,j) \in \sum_{i \in I} p_i(1)} y^{p_i[j]}, q \right) &\cong \prod_{(i,j) \in \sum_{i \in I} p_i(1)} q(p_i[j]) \\
 &\cong \prod_{i \in I} \prod_{j \in p_i(1)} q(p_i[j]) \\
 &\cong \prod_{i \in I} \text{Poly}(p_i, q).
 \end{aligned} \tag{3.7}$$

Solution to Exercise 3.23.

1. We construct a lens $\dot{p}y \rightarrow p$, or a lens

$$\sum_{i \in p(1)} \sum_{a \in p[i]} y^{p[i]} \rightarrow \sum_{i \in p(1)} y^{p[i]},$$

as follows. It sends each position $(i, a) \in \sum_{i \in p(1)} p[i]$ of $\dot{p}y$ to its first projection $i \in p(1)$, and it is the identity on directions.

2. There is not always a lens $p \rightarrow \dot{p}$: if $p := 1$, then $\dot{p} := 0$, and there is no lens $1 \rightarrow 0$.
3. There is not always a lens $\dot{p} \rightarrow p$: take $p := y$, so $\dot{p} := 1$. A lens $1 \rightarrow y$ must have an on-directions function $1 \rightarrow 0$, but there is no such function.
4. We show that even when there is a lens $p \rightarrow q$, there is not necessarily a lens $\dot{p} \rightarrow \dot{q}$. Take $p := y$ and $q := 1$. Then there is a lens $p \rightarrow q$ that sends the unique position of y to the unique position of 1 and is the empty function on directions. But $\dot{p} = 1$ and $\dot{q} = 0$, and there is no lens $1 \rightarrow 0$.
5. We construct a lens $g: [p, y] \otimes p \rightarrow \dot{p}$, where $[p, y] \otimes p$ is given by (3.24), as follows. The on-positions function g_1 takes $f \in \prod_{i \in p(1)} p[i]$ and $i \in p(1)$ and sends the pair of them to the $\dot{p}$ -position corresponding to $i \in p(1)$ and $f i \in p[i]$. Then $\dot{p}[(i, f i)] = p[i] - \{f i\}$ and $([p, y] \otimes p)[(f, i)] \cong p(1) \times p[i]$, so the on-directions function $g_{(f, i)}^\#$ can send each $a \in p[i] - \{f i\}$ to $(i, a) \in p(1) \times p[i]$.

6. We describe a lens $py \rightarrow q$ in terms of “unassigned” directions. Observe that py has the same positions as p but has one more direction than p does at each position. Given a position $i \in p(1)$, we denote this extra $(py)[i]$ -direction by $*_i$, identifying $(py)[i]$ with $p[i] + \{*_i\}$. So a lens $f: py \rightarrow q$ sends each p -position i to a q -position j , but every $q[j]$ -direction could be sent back to either an original $p[i]$ -direction or the extra $(py)[i]$ -direction $*_i$. So a lens $py \rightarrow q$ is like a lens $p \rightarrow q$, except that some of the directions of q may remain “unassigned” to any direction of p , which we signify by assigning them to $*_i$ instead. In other words, a lens $py \rightarrow q$ could be interpreted as a lens $p \rightarrow q$ whose on-directions functions may only be partially defined.

Solution to Exercise 3.25.

We have

$$\dot{p}(1) \cong \sum_{i \in p(1)} \sum_{a \in p[i]} 1^{p[i] - \{a\}} \cong \sum_{i \in p(1)} p[i],$$

which is precisely the set of all directions of p .

Solution to Exercise 3.27.

1. Since $0 \cong 0y$ is a linear polynomial, Example 3.26 tells us that lenses $0 \rightarrow p$ can be identified with functions $0 \rightarrow p(1)$. There is exactly one function $0 \rightarrow p(1)$, so there is exactly one lens $0 \rightarrow p$.
2. Since $y \cong 1y$ is a linear polynomial, Example 3.26 tells us that lenses $y \rightarrow p$ can be identified with functions $1 \rightarrow p(1)$, which in turn can be identified with elements of $p(1)$.

Solution to Exercise 3.28.

A lens $f: p \rightarrow Iy$ consists of an on-positions function $f_1: p(1) \rightarrow I$ and, for each $j \in p(1)$, an on-directions function $f_j^\# : 1 \rightarrow p[j]$. Equivalently, this is a function $p(1) \rightarrow I$ and a choice of direction at every position of p , i.e. a dependent function $(j \in p(1)) \rightarrow p[j]$.

Solution to Exercise 3.30.

1. Since 1 is a constant, Example 3.29 tells us that lenses $p \rightarrow 1$ can be identified with functions $p(1) \rightarrow 1$. There is exactly one function $p(1) \rightarrow 1$, so there is exactly one lens $p \rightarrow 1$.
2. Since 0 is a constant, Example 3.29 tells us that lenses $p \rightarrow 0$ can be identified with functions $p(1) \rightarrow 0$, which only exist when $p(1) = 0$. The only polynomial with an empty position-set is 0 itself, and there is a unique function from the set 0 to itself, so there is a unique lens from the constant polynomial 0 to itself as well. If p is not the constant 0, then there are no lenses $p \rightarrow 0$.

Solution to Exercise 3.31.

1. A lens $f: I \rightarrow p$ consists of an on-positions function $f_1: I \rightarrow p(1)$ and, for each $i \in I$, an on-directions function $f_i^\# : p[i] \rightarrow 0$. There is exactly one such on-directions function when $p[i] = 0$ and no such on-directions function otherwise. It follows that a lens $f: I \rightarrow p$ can be identified with a function $f_1: I \rightarrow p(1)$ whose image

is contained in the set of p -positions with no directions. By Exercise 2.9, this set of p -positions can be identified with the set $p(0)$ (the *constant* term of p); so a lens $I \rightarrow p$ is equivalent to a function $I \rightarrow p(0)$.

2. From the previous part, a lens $1 \rightarrow p$ may be identified with a function $1 \rightarrow p(0)$ and thus an element of $p(0)$.

Solution to Exercise 3.40.

The functor Γ is defined as the hom-functor $\mathbf{Poly}(-, y): \mathbf{Poly} \rightarrow \mathbf{Set}^{\text{op}}$, which exhibits the universal property of colimits by sending colimits in $\mathbf{Poly}$ to limits in $\mathbf{Set}$. Hence Proposition 3.39 follows from Proposition 3.3. More explicitly, $\Gamma(0) = \mathbf{Poly}(0, y) \cong 1$ since 0 is initial in $\mathbf{Poly}$, and $\Gamma(p + q) = \mathbf{Poly}(p + q, y) \cong \mathbf{Poly}(p + q, y) = \Gamma(p) \times \Gamma(q)$ since $+$ gives coproducts in $\mathbf{Poly}$.

Solution to Exercise 3.48.

Fix $i \in p(1)$. Since the $p[i]$ -component $(\text{id}_p)_{p[i]}$ of id_p is the identity function on $p(p[i])$, by Corollary 3.47,

$$((\text{id}_p)_1 i, (\text{id}_p)_i^\sharp) = (\text{id}_p)_{p[i]}(i, \text{id}_{p[i]}) = (i, \text{id}_{p[i]}).$$

Hence the on-positions function $(\text{id}_p)_1: p(1) \rightarrow p(1)$ maps every $i \in p(1)$ to itself, so it is an identity function; and each on-directions function $(\text{id}_p)_i^\sharp: p[i] \rightarrow p[i]$ is equal to $\text{id}_{p[i]}$.

Solution to Exercise 3.49.

Fix $i \in p(1)$. By Proposition 3.44 and Corollary 3.47,

$$\begin{aligned} (h_1 i, h_i^\sharp) &= h_{p[i]}(i, \text{id}_{p[i]}) && \text{(Corollary 3.47)} \\ &= g_{p[i]}(f_{p[i]}(i, \text{id}_{p[i]})) && (h = f \circ g) \\ &= g_{p[i]}(f_1 i, f_i^\sharp) && \text{(Corollary 3.47)} \\ &= (g_1 f_1 i, g_{f_1 i}^\sharp \circ f_i^\sharp). && \text{(Proposition 3.44)} \end{aligned}$$

Solution to Exercise 3.51.

We provide lenses $\iota: p \rightarrow p + q$ and $\kappa: q \rightarrow p + q$ as follows. On positions, they are the canonical inclusions $\iota_1: p(1) \rightarrow p(1) + q(1)$ and $\kappa_1: q(1) \rightarrow p(1) + q(1)$; on directions, they are identities. To show that $p + q$ equipped with ι and κ satisfies the universal property of the coproduct, we apply Proposition 3.50. In order for (3.52) to commute, it must commute on positions—that is, the following diagram of sets must commute:

$$\begin{array}{ccccc} p(1) & \xrightarrow{\iota_1} & p(1) + q(1) & \xleftarrow{\kappa_1} & q(1) \\ & \searrow f_1 & \downarrow \text{id}_{h_1} & \swarrow g_1 & \\ & & r(1) & & \end{array} \quad (3.83)$$

But since $p(1) + q(1)$ along with the inclusions ι_1 and κ_1 form the coproduct of $p(1)$ and $q(1)$ in $\mathbf{Set}$, there exists a unique h_1 for which (3.83) commutes. Hence h is uniquely

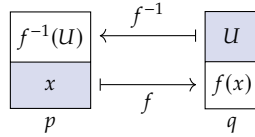
characterized on positions. In particular, it must send each $(1, i) \in p(1) + q(1)$ with $i \in p(1)$ to $f_1 i$ and each $(2, j) \in p(1) + q(1)$ with $j \in q(1)$ to $g_1 j$. Moreover, if (3.52) is to commute on directions, then for every $i \in p(1)$ and $j \in q(1)$, the following diagrams of sets must commute:

$$\begin{array}{ccc}
 p[i] & \xleftarrow{\iota_i^\#} & (p+q)[(1, i)] \\
 & \nwarrow f_i^\# & \uparrow h_{(1, i)}^\# \\
 & & r[f_1 i]
 \end{array}
 \qquad
 \begin{array}{ccc}
 (p+q)[(2, j)] & \xrightarrow{\kappa_j^\#} & q[j] \\
 \uparrow h_{(2, j)}^\# & \nearrow g_j^\# & \\
 r[g_1 j] & &
 \end{array}
 \tag{3.84}$$

But $(p+q)[(1, i)] \cong p[i]$ and $\iota_i^\#$ is the identity, so we must have $h_{(1, i)}^\# = f_i^\#$. Similarly, $(p+q)[(2, j)] \cong q[j]$ and $\kappa_j^\#$ is the identity, so we must have $h_{(2, j)}^\# = g_j^\#$. Hence h is also uniquely characterized on directions, so it is unique overall. Moreover, we have shown that we can define h on positions so that (3.83) commutes, and that we can define h on directions such that the diagrams in (3.84) commute. As the commutativity of the diagrams in (3.83) and (3.84) together imply the commutativity of (3.52), it follows that there exists h for which (3.52) commutes.

Solution to Exercise 3.53.

- Given a continuous map $f: X \rightarrow Y$, we define a lens $p_f: p_X \rightarrow p_Y$ as follows. The on-positions function is just f ; then for each p_X -position $x \in X$, the on-directions function $(p_f)_x^\#: p_Y[f(x)] \rightarrow p_X[x]$ sends each open neighborhood U of $f(x)$ to $f^{-1}(U)$, which we know is an open neighborhood of x because f is continuous. The polybox picture for p_f is as follows:



- To show that p_X is functorial in X , it suffices to show that sending continuous maps $f: X \rightarrow Y$ to their induced lenses $p_f: p_X \rightarrow p_Y$ preserves identities and composition. First, we show for $X \in \mathbf{Top}$ that the lens p_{id_X} is an identity. By #1, the on-positions function of p_{id_X} is id_X , and for each $x \in X$ the on-directions function $(p_f)_x^\#: p_X[x] \rightarrow p_X[x]$ sends $U \in p_X[x]$ to $(\text{id}_X)^{-1}(U) = U$. Hence p_{id_X} is the identity on both positions and directions; it follows from Exercise 3.48 that p_{id_X} is the identity lens.

We now show for $X, Y, Z \in \mathbf{Top}$ and continuous maps $f: X \rightarrow Y$ and $g: Y \rightarrow Z$ that $p_f \circ p_g = p_{f \circ g}$. By #1 and Exercise 3.49, the on-positions functions of both $p_f \circ p_g$ and $p_{f \circ g}$ are equal to $f \circ g$, so it suffices to show for each $x \in X$ that

$$(p_{f \circ g})_x^\# = (p_g)_{f(x)}^\# \circ (p_f)_x^\#.$$

By #1, the left-hand side sends each $U \in p_Z[g(f(x))]$ to $(f \circ g)^{-1}(U)$, while the right-hand side sends U to $f^{-1}(g^{-1}(U))$; by elementary set theory, these sets are equal.

3. The functor is not full. Consider the spaces $X := 2$ with the indiscrete topology (i.e. the only open sets are the empty set and X) and $Y := 2$ with the discrete topology (i.e. all subsets are open). Then $p_X \cong 2y$ (each point in X has exactly one open neighborhood: the entire space X) and $p_Y \cong 2y^2$ (each point in Y has exactly two open neighborhoods: a singleton set and Y itself), so our functor induces a map from the set of continuous functions $X \rightarrow Y$ to the set of lenses $2y \rightarrow 2y^2$. We claim this map is not surjective: in particular, consider the lens $h: 2y \rightarrow 2y^2$ that is the identity on positions (and uniquely defined on directions). Then a continuous function $f: X \rightarrow Y$ that our functor sends to h must also be the identity on the underlying sets of X and Y . But such a function cannot be continuous: a singleton subset of Y is open, but its preimage under f is a singleton subset of X and therefore not open. So our functor sends no continuous function $X \rightarrow Y$ to h and therefore is not full. The functor is, however, faithful: given spaces X and Y and continuous function $f: X \rightarrow Y$, we can uniquely recover f from p_f by taking its on-positions function $(p_f)_1 = f$.

Solution to Exercise 3.59.

Given $q \in \mathbf{Poly}$ and $p_i \in \mathbf{Poly}$ for each $i \in I$ for some set I , we use (3.7) to verify that

$$\begin{aligned}
 \mathbf{Poly}\left(q, \prod_{i \in I} p_i\right) &\cong \prod_{k \in q(1)} \left(\prod_{i \in I} p_i \right) (q[k]) \\
 &\cong \prod_{k \in q(1)} \prod_{i \in I} p_i(q[k]) \\
 &\cong \prod_{i \in I} \prod_{k \in q(1)} p_i(q[k]) \\
 &\cong \prod_{i \in I} \mathbf{Poly}(q, p_i).
 \end{aligned} \tag{3.7}$$

Solution to Exercise 3.60.

Given $p_1 := y + 1$, $p_2 := y + 2$, and $p_3 := y^2$, we compute $\prod_{i \in 3} p_i$ via (3.57) as follows:

$$\begin{aligned}
 \prod_{i \in 3} p_i &\cong \sum_{\tilde{j} \in \prod_{i \in 3} p_i(1)} y^{\sum_{i \in 3} p_i[\tilde{j}(i)]} \\
 &\cong \sum_{\tilde{j}: (i \in 3) \rightarrow p_i(1)} y^{p_1[\tilde{j}(1)] + p_2[\tilde{j}(2)] + p_3[\tilde{j}(3)]} \\
 &\cong y^{p_1[1] + p_2[1] + p_3[1]} + y^{p_1[1] + p_2[2] + p_3[1]} + y^{p_1[1] + p_2[3] + p_3[1]} \\
 &\quad + y^{p_1[2] + p_2[1] + p_3[1]} + y^{p_1[2] + p_2[2] + p_3[1]} + y^{p_1[2] + p_2[3] + p_3[1]} \\
 &\cong y^{1+1+2} + y^{1+0+2} + y^{1+0+2} \\
 &\quad + y^{0+1+2} + y^{0+0+2} + y^{0+0+2} \\
 &\cong y^4 + 3y^3 + 2y^2,
 \end{aligned} \tag{3.57}$$

as we might expect from standard polynomial multiplication.

Solution to Exercise 3.63.

1. We compute the product using (3.61):

$$\begin{aligned} B_1 y^{A_1} \times B_2 y^{A_2} &\cong \left(\sum_{i \in B_1} y^{A_1} \right) \times \left(\sum_{j \in B_2} y^{A_2} \right) \\ &\cong \sum_{i \in B_1} \sum_{j \in B_2} y^{A_1 + A_2} \\ &\cong B_1 B_2 y^{A_1 + A_2}. \end{aligned}$$

2. We expand the product by applying (1.28), with $I_1 := I$ and $I_2 := J$:

$$\begin{aligned} \left(\sum_{i \in I} B_i y^{A_i} \right) \times \left(\sum_{j \in J} B_j y^{A_j} \right) &\cong \prod_{k \in 2} \sum_{i \in I_k} B_i y^{A_i} \\ &\cong \sum_{\bar{i} \in \prod_{k \in 2} I_k} \prod_{k \in 2} B_{\bar{i}(k)} y^{A_{\bar{i}(k)}} \\ &\cong \sum_{(i, j) \in IJ} B_i y^{A_i} \times B_j y^{A_j} \\ &\cong \sum_{i \in I} \sum_{j \in J} B_i B_j y^{A_i + A_j} \end{aligned}$$

where the last isomorphism follows from #1.

Solution to Exercise 3.64.

We wish to show that, for $p, q \in \mathbf{Poly}$, the polynomial pq along with the lenses $\pi: pq \rightarrow p$ and $\varphi: pq \rightarrow q$ as described in the text satisfy the universal property of the product. That is, we must show that for any $r \in \mathbf{Poly}$ and lenses $f: r \rightarrow p$ and $g: r \rightarrow q$, there exists a unique lens $h: r \rightarrow pq$ for which the following diagram commutes:

$$\begin{array}{ccc} r & \xrightarrow{g} & q \\ f \downarrow & \searrow h & \uparrow \varphi \\ p & \xleftarrow{\pi} & pq. \end{array} \quad (3.85)$$

We apply Proposition 3.50. In order for (3.85) to commute, it must commute on positions—that is, the following diagram of sets must commute:

$$\begin{array}{ccc} r(1) & \xrightarrow{g_1} & q(1) \\ f_1 \downarrow & \searrow h_1 & \uparrow \varphi_1 \\ p(1) & \xleftarrow{\pi_1} & (pq)(1). \end{array} \quad (3.86)$$

But since $(pq)(1) \cong p(1) \times q(1)$ along with the projections π_1 and φ_1 form the product of $p(1)$ and $q(1)$ in **Set**, there exists a unique h_1 for which (3.86) commutes. Hence h is uniquely characterized on positions. In particular, it must send each $k \in r(1)$ to the pair $(f_1 k, g_1 k) \in (pq)(1)$.

Moreover, if (3.52) is to commute on directions, then for every $k \in r(1)$, the following diagram of sets must commute:

$$\begin{array}{ccc}
 r[k] & \xleftarrow{g_k^\#} & q[g_1 k] \\
 \uparrow f_k^\# & \nwarrow h_k^\# & \downarrow \phi_{(f_1 k, g_1 k)}^\# \\
 p[f_1 k] & \xrightarrow{\pi_{(f_1 k, g_1 k)}^\#} & (pq)[(f_1 k, g_1 k)].
 \end{array} \tag{3.87}$$

As $(pq)[(f_1 k, g_1 k)] \cong p[f_1 k] + q[g_1 k]$ along with the inclusions $\pi_{(f_1 k, g_1 k)}^\#$ and $\phi_{(f_1 k, g_1 k)}^\#$ form the coproduct of $p[f_1 k]$ and $q[g_1 k]$ in **Set**, there exists a unique $h_k^\#$ for which (3.87) commutes. Hence h is also uniquely characterized on directions, so it is unique overall. Moreover, we have shown that we can define h on positions so that (3.86) commutes, and that we can define h on directions such that (3.87) commutes. As the commutativity of (3.86) and (3.87) together imply the commutativity of (3.85), it follows that there exists h for which (3.85) commutes.

Solution to Exercise 3.67.

1. We compute the parallel product using (3.66):

$$\begin{aligned}
 B_1 y^{A_1} \otimes B_2 y^{A_2} &\cong \left(\sum_{i \in B_1} y^{A_1} \right) \otimes \left(\sum_{j \in B_2} y^{A_2} \right) \\
 &\cong \sum_{i \in B_1} \sum_{j \in B_2} y^{A_1 \times A_2} \\
 &\cong B_1 B_2 y^{A_1 A_2}.
 \end{aligned}$$

2. We expand the parallel product as follows:

$$\begin{aligned}
 \left(\sum_{i \in I} B_i y^{A_i} \right) \otimes \left(\sum_{j \in J} B_j y^{A_j} \right) &\cong \left(\sum_{i \in I} \sum_{i' \in B_i} y^{A_i} \right) \otimes \left(\sum_{j \in J} \sum_{j' \in B_j} y^{A_j} \right) \\
 &\cong \sum_{i \in I} \sum_{i' \in B_i} \sum_{j \in J} \sum_{j' \in B_j} y^{A_i \times A_j} \\
 &\cong \sum_{i \in I} \sum_{j \in J} \sum_{i' \in B_i} \sum_{j' \in B_j} y^{A_i A_j} \\
 &\cong \sum_{i \in I} \sum_{j \in J} B_i B_j y^{A_i A_j}.
 \end{aligned}$$

Solution to Exercise 3.68.

1. By Exercise 3.67 #1, we have $A \otimes B \cong Ay^0 \otimes By^0 \cong AB y^0 \cong AB$.
2. By Exercise 3.67 #1, we have $Ay \otimes By \cong Ay^1 \otimes By^1 \cong AB y^1 \cong AB y$.
3. By (3.66),

$$(p \otimes q)(1) \cong \sum_{i \in p(1)} \sum_{j \in q(1)} 1^{p[i] \times q[j]} \cong p(1) \times q(1).$$

Solution to Exercise 3.69.

1. We compute $p \times q$ using Exercise 3.63 #2:

$$\begin{aligned} p \times q &\cong 2y^{2+4} + (2 \times 3)y^{2+3} + 3y^{1+4} + (3 \times 3)y^{1+3} \\ &\cong 2y^6 + 6y^5 + 3y^5 + 9y^4 \\ &\cong 2y^6 + 9y^5 + 9y^4. \end{aligned}$$

2. We compute $p \otimes q$ using Exercise 3.67 #2:

$$\begin{aligned} p \otimes q &\cong 2y^{2 \times 4} + (2 \times 3)y^{2 \times 3} + 3y^4 + (3 \times 3)y^3 \\ &\cong 2y^8 + 6y^6 + 3y^4 + 9y^3. \end{aligned}$$

3. We evaluate $(2 \cdot 2^y + 3 \cdot 1^y + 1) \cdot (1 \cdot 4^y + 3 \cdot 3^y + 2)$ using ordinary laws of arithmetic:

$$\begin{aligned} (2 \cdot 2^y + 3 \cdot 1^y) \cdot (1 \cdot 4^y + 3 \cdot 3^y) &= (2 \cdot 2^y + 3)(4^y + 3 \cdot 3^y) \\ &= 2 \cdot 2^y \cdot 4^y + 2 \cdot 3 \cdot 2^y \cdot 3^y + 3 \cdot 4^y + 3 \cdot 3 \cdot 3^y \\ &= 2 \cdot 8^y + 6 \cdot 6^y + 3 \cdot 4^y + 9 \cdot 3^y. \end{aligned}$$

4. We describe the connection between the last two parts as follows. Given a polynomial p , we let $d(p)$ denote the Dirichlet series $\sum_{i \in p(1)} |p[i]|^y$. Then by (3.66),

$$\begin{aligned} d(p \otimes q) &= \sum_{i \in p(1)} \sum_{j \in q(1)} |p[i] \times q[j]|^y \\ &= \sum_{i \in p(1)} |p[i]|^y \sum_{j \in q(1)} |q[j]|^y \\ &= d(p) \cdot d(q). \end{aligned}$$

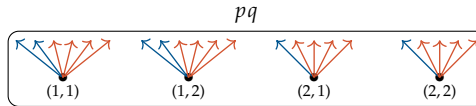
The last two parts are simply an example of this identity for a specific choice of p and q .

Solution to Exercise 3.71.

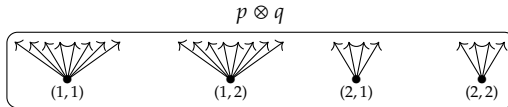
1. Here are p and q drawn as corolla forests:



2. Here is pq drawn as a corolla forest:



3. Here is $p \otimes q$ drawn as a corolla forest:



Solution to Exercise 3.72.

1. We show that $p \otimes y \cong p$:

$$\begin{aligned} p \otimes y &\cong \sum_{i \in p(1)} \sum_{j \in 1} y^{p[i] \times 1} \\ &\cong \sum_{i \in p(1)} y^{p[i]} \cong p. \end{aligned} \quad (3.66)$$

2. We show that $(p \otimes q) \otimes r \cong p \otimes (q \otimes r)$:

$$(p \otimes q) \otimes r \cong \left(\sum_{i \in p(1)} \sum_{j \in q(1)} y^{p[i] \times q[j]} \right) \otimes r \quad (3.66)$$

$$\cong \sum_{i \in p(1)} \sum_{j \in q(1)} \left(\sum_{k \in r(1)} y^{(p[i] \times q[j]) \times r[k]} \right) \quad (3.66)$$

$$\cong \sum_{i \in p(1)} \left(\sum_{j \in q(1)} \sum_{k \in r(1)} y^{p[i] \times (q[j] \times r[k])} \right) \quad (\text{Associativity of } \sum \text{ and } \times)$$

$$\cong p \otimes \left(\sum_{j \in q(1)} \sum_{k \in r(1)} y^{q[j] \times r[k]} \right) \quad (3.66)$$

$$\cong p \otimes (q \otimes r). \quad (3.66)$$

3. We show that $(p \otimes q) \cong (q \otimes p)$:

$$p \otimes q \cong \sum_{i \in p(1)} \sum_{j \in q(1)} y^{p[i] \times q[j]} \quad (3.66)$$

$$\cong \sum_{j \in q(1)} \sum_{i \in p(1)} y^{q[j] \times p[i]} \quad (\text{Commutativity of } \sum \text{ and } \times)$$

$$\cong q \otimes p. \quad (3.66)$$

Solution to Exercise 3.74.

1. We compute $(3y^5 + 6y^2) \otimes 4$ using Exercise 3.67 #2 and the fact that $4 = 4y^0$:

$$\begin{aligned} (3y^5 + 6y^2) \otimes 4y^0 &\cong (3 \times 4)y^{5 \times 0} + (6 \times 4)y^{2 \times 0} \\ &\cong 12y^0 + 24y^0 \\ &\cong 36. \end{aligned}$$

2. Given a polynomial p and a set J viewed as a constant polynomial, we have

$$\begin{aligned} p \otimes J &\cong \left(\sum_{i \in p(1)} y^{p[i]} \right) \otimes \left(\sum_{j \in J} y^0 \right) \\ &\cong \sum_{i \in p(1)} \sum_{j \in J} y^{p[i] \times 0} \end{aligned}$$

$$\begin{aligned} &\cong p(1)Jy^0 \\ &\cong p(1)J, \end{aligned}$$

itself a constant polynomial; so the class of constant polynomials is indeed a $\otimes$ -ideal.

Solution to Exercise 3.75.

For each of the following classes of polynomials, we determine whether they are closed under $\otimes$ and whether they contain y .

1. The set $\{Ay^0 \mid A \in \mathbf{Set}\}$ of constant polynomials is closed under $\otimes$ by the solution to Exercise 3.68 #1. But the set does not contain y , as y is not a constant polynomial.
2. The set $\{Ay \mid A \in \mathbf{Set}\}$ of linear polynomials is closed under $\otimes$ by the solution to Exercise 3.68 #2 and does contain y , as $y \cong 1y$.
3. The set $\{Ay + B \mid A, B \in \mathbf{Set}\}$ of affine polynomials is closed under $\otimes$, for Exercise 3.67 #2 yields

$$(Ay + B) \otimes (A'y + B') \cong AA'y + AB' + BA' + BB'.$$

The set contains y , as $y \cong 1y + 0$.

4. The set $\{Ay^2 + By + C \mid A, B, C \in \mathbf{Set}\}$ of quadratic polynomials is not closed under $\otimes$, for even though $y^2 \cong 1y^2 + 0y + 0$ is a quadratic polynomial, Exercise 3.67 #1 implies that

$$y^2 \otimes y^2 \cong y^4,$$

which is not quadratic. The set contains y , as $y \cong 0y^2 + 1y + 0$.

5. The set $\{Ay^B \mid A, B \in \mathbf{Set}\}$ of monomials is closed under $\otimes$ by Exercise 3.67 #1 and does contain y , as $y \cong 1y^1$.
6. The set $\{Sy^S \mid S \in \mathbf{Set}\}$ is closed under $\otimes$, for Exercise 3.67 #1 returns

$$Sy^S \otimes Ty^T \cong STy^{ST}.$$

The set contains y , as $y \cong 1y^1$.

7. The set $\{p \in \mathbf{Poly} \mid p(1) \text{ is finite}\}$ is closed under $\otimes$ by the solution to Exercise 3.68 #3 and the fact that the product of two finite sets is itself finite. The set contains y , as $y(1) \cong 1$ is finite.

Solution to Exercise 3.76.

The smallest class of polynomials that is closed under $\otimes$ and contains y is just $\{y\}$. This is because by Exercise 3.67 #1, we have $y \otimes y \cong y$.

Solution to Exercise 3.77.

We show that $(p_1 + p_2) \otimes q \cong (p_1 \otimes q) + (p_2 \otimes q)$ using (3.66):

$$\begin{aligned} (p_1 + p_2) \otimes q &\cong \sum_{k \in 2} \sum_{i \in p_k(1)} \sum_{j \in q(1)} y^{p_k[i] \times q[j]} \\ &\cong \sum_{i \in p_1(1)} \sum_{j \in q(1)} y^{p_1[i] \times q[j]} + \sum_{i \in p_2(1)} \sum_{j \in q(1)} y^{p_2[i] \times q[j]} \\ &\cong (p_1 \otimes q) + (p_2 \otimes q). \end{aligned}$$

Solution to Exercise 3.82.

1. To show that the operation $(A, B) \mapsto A + AB + B$ on **Set** is associative, observe that

$$\begin{aligned} (A + AB + B) + (A + AB + B)C + C &\cong A + AB + B + AC + ABC + BC + C \\ &\cong A + AB + ABC + AC + B + BC + C \\ &\cong A + A(B + BC + C) + (B + BC + C). \end{aligned}$$

2. To show that 0 is the unit for this operation, observe that

$$(A, 0) \mapsto A + A0 + 0 \cong A$$

and

$$(0, B) \mapsto 0 + 0B + B \cong B.$$

3. Taking $A \star B := A + AB + B$ in Proposition 3.79 to obtain a monoidal product $\odot$ on **Poly**, we can use (3.81) to compute that

$$\begin{aligned} (y^3 + y) \odot (2y^2 + 2) &\cong (y^3 + y^1) \odot (y^2 + y^2 + y^0 + y^0) \\ &\cong y^{3\star 2} + y^{3\star 2} + y^{3\star 0} + y^{3\star 0} + y^{1\star 2} + y^{1\star 2} + y^{1\star 0} + y^{1\star 0} \\ &\cong 2y^{11} + 2y^3 + 2y^5 + 2y^1. \end{aligned}$$

4. Dynamical systems as dependent lenses

One of the main goals of this book is to use dependent lenses in **Poly** to model dynamical systems and automata. In this chapter, we will begin to see how to do this through an array of examples.

4.1 Moore machines

We start with our simplest example of a dynamical system: a deterministic state machine with a fixed range of states, inputs, and outputs. At any point in time, this machine will inhabit one of its possible states and return output according to that current state. It can also update its current state according to the input it receives.

Definition 4.1 (Moore machine). A *Moore machine* consists of the following data: three sets,

- a set S , called the *state-set*, whose elements are *states*;
- a set I , called the *position-set* (or *output-set*), whose elements are *positions* (or *outputs*);
- a set A , called the *direction-set* (or *input-set*), whose elements are *directions* (or *inputs*);

and two functions,

- return: $S \rightarrow I$;
- update: $S \times A \rightarrow S$.

To emphasize the role that the three sets play, we can specify that this is an (A, I) -*Moore machine with states* S .

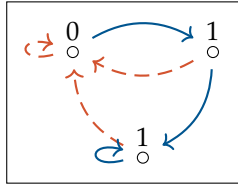
The input and output terminology is standard, while the position and direction terminology is our own: we will soon see how the positions and directions of a Moore machine relate to that of a polynomial.

We should interpret an (A, I) -Moore machine as follows. At any time, the machine inhabits one of the states in its state-set S . Say its current state is $s \in S$. We can ask the machine to perform one of the following two tasks.

- We can ask the machine to *return its position*: it should then produce the position $\text{return}(s) \in I$.
- We can feed the machine one of its *directions* $a \in A$ and ask it to *update its state*: it should then replace its current state with the new state $\text{update}(s, a) \in S$. Note that the new state depends not only on the direction the machine receives but also on the state the machine inhabits when it receives that direction.

We may visualize a Moore machine with a *transition diagram* as follows.

Example 4.2 (A Moore machine's transition diagram). Given $A := \{\text{red}, \text{blue}\}$ and $I := \{0, 1\}$, we can draw a transition diagram for an (A, I) -Moore machine with $S := 3$ states as follows:



(4.3)

Each state is labeled by the position it returns according to the machine's return function. Additionally, each state has two outgoing arrows, one **red** (dashed) and one **blue** (solid), corresponding to the two possible directions. The targets of the arrows indicate the updated state according to the machine's update function.

Say the machine starts at the bottom state. By feeding it a sequence of directions—say (**red**, **red**, **blue**, **red**, ...)—we can send the machine through its states via its update function and return the position at each state:

1. Starting at the bottom state, the machine returns the position labeled 1.

2. Following the **red** arrow from the bottom state, the machine updates its state to the left state.
3. At the left state, the machine returns the position 0.
4. Following the **red** arrow from the left state, the machine updates its state to—once again—the left state.
5. At the left state, the machine returns the position 0.
6. Following the **blue** arrow from the left state, the machine updates its state to the right state.
7. At the right state, the machine returns the position 1.
8. Following the **red** arrow from the right state, the machine updates its state to the left state.
9. At the left state, the machine returns the position 0.
- ...

In summary, starting from the bottom state, this Moore machine sends the sequence (**red**, **red**, **blue**, **red**, ...) of directions in A to the sequence (1, 0, 0, 1, 0, ...) of positions in I .

In general, given an initial state $s_0 \in S$, an (A, I) -Moore machine with states S sends every sequence $(a_1, a_2, a_3, \dots)$ of directions in A to a sequence $(i_0, i_1, i_2, i_3, \dots)$ of positions in I , defined inductively as follows, via an intermediary sequence $(s_0, s_1, s_2, s_3, \dots)$ of states in S :

$$i_k := \text{return}(s_k) \quad \text{and} \quad s_{k+1} := \text{update}(s_k, a_{k+1})$$

for all $k \in \mathbb{N}$. We will see that **Poly** gives us a more concise way to express this in Example 8.52.

Comparing Definition 4.1 with Example 3.41, we find that an (A, I) -Moore machine with states S is precisely a lens between monomials $\varphi: Sy^S \rightarrow Iy^A$ with on-positions function $\varphi_1 := \text{return}: S \rightarrow I$ and on-directions map $\varphi^\# := S \times A \rightarrow S$. The positions and directions of the Moore machine are the positions and directions of the codomain of the corresponding lens, while the domain of the lens has the states of the Moore machine as both its positions and its directions. So we can repackage Definition 4.1 as follows.

Definition 4.4 (Moore machine, version 2). For $S, I, A \in \mathbf{Set}$, an

(A, I) -Moore machine with states S is a lens

$$\varphi: Sy^S \rightarrow Iy^A$$

in **Poly**. We call

- the domain monomial Sy^S the machine's *state system*: its position-set (equivalently, its direction-set) is the machine's *state-set*, and its positions (equivalently, its directions) are the machine's *states*;
- the codomain monomial Iy^A the machine's *interface*: its position-set and direction-set are the machine's *position-set* and *direction-set*, and its positions and directions are the machine's *positions* and *directions*;
- the on-positions function $\varphi_1: S \rightarrow I$ the machine's *return function*;
- the on-directions map $\varphi^\# : S \times A \rightarrow S$ the machine's *update function*.

We call the codomain of a Moore machine its *interface* because it encodes how an outsider interacts with the machine: an outsider observes the positions of the interface that the machine returns and feeds the directions of the interface to the machine to update it. Rather than directly observing and altering the machine's states, an outsider must interact with the machine via its interface.

Exercise 4.5. In this exercise, we will write the Moore machine from Example 4.2 as a lens φ between monomials.

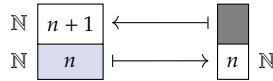
1. What is the machine's state system, the domain of φ ?
2. What is the machine's interface, the codomain of φ ?

Call the left state L , the right state R , and the bottom state B .

3. What is the machine's return function, the on-positions function of φ ?
4. What is the machine's update function, the on-directions map of φ ?
5. Draw the first two steps listed in Example 4.2 of the machine's operation (starting at the bottom state and receiving the direction **red**) using polyboxes.

Here are some more examples of Moore machines.

Example 4.6 (Counter). There is a $(1, \mathbb{N})$ -Moore machine with states $\mathbb{N}$ that, with initial state $0 \in \mathbb{N}$, returns the sequence of natural numbers $(0, 1, 2, 3, \dots)$. The machine is given by the lens $\mathbb{N}y^{\mathbb{N}} \rightarrow \mathbb{N}y$ whose on-positions function is the identity $\mathbb{N} \rightarrow \mathbb{N}$ and whose on-directions map $\mathbb{N} \times 1 \cong \mathbb{N} \rightarrow \mathbb{N}$ sends $n \mapsto n + 1$. Here it is in polyboxes (recall that the shaded direction box indicates that the direction-set is a singleton, i.e. there is no choice to be made in filling it in):

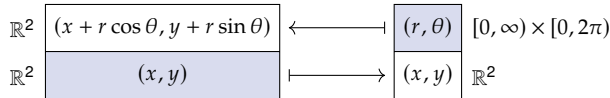


The picture tells us that if the current state (the left position box) is $n \in \mathbb{N}$, the next state (the left direction box) is $n + 1 \in \mathbb{N}$. Since the machine just returns its current state as a position, the sequence of positions returned will always be an increasing sequence of consecutive natural numbers starting at the initial state.

Example 4.7 (Moving in the plane). Let us construct a Moore machine with positions in $\mathbb{R}^2$, which we may think of as locations in the coordinate plane, and directions in $[0, \infty) \times [0, 2\pi)$, which we may think of as commands to move a certain distance $r \in [0, \infty)$ at a certain angle $\theta \in [0, 2\pi)$. We will let the machine's state-set be $\mathbb{R}^2$ as well, so the machine is a lens

$$\mathbb{R}^2 y^{\mathbb{R}^2} \rightarrow \mathbb{R}^2 y^{[0, \infty) \times [0, 2\pi)}.$$

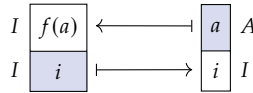
We can define such a lens using polyboxes:



Exercise 4.8. Explain in words what the Moore machine in Example 4.7 does.

Example 4.9 (Functions as memoryless Moore machines). Given a function $f: A \rightarrow I$, there is a corresponding (A, I) -Moore machine with states I that takes in an element of A and returns the element of I obtained by applying f .

It is given by the lens $Iy^I \rightarrow Iy^A$ defined as follows:



That is, this lens is the identity on positions, returning the state directly as its position, and on directions it is the function $I \times A \xrightarrow{\pi_2} A \xrightarrow{f} I$, which ignores the current state and applies f to the direction received to compute the new state.

If the machine starts in state i_0 and is given a sequence of directions $(a_1, a_2, \dots)$ from A , the machine will return the positions $(i_0, f(a_1), f(a_2), \dots)$. We say this machine is *memoryless*, because at no point does the state of the machine actually depend on any previous states; instead, its state depends only on the last direction it received.

Exercise 4.10. Suppose we have a function $f: A \times I \rightarrow I$.

1. Find a corresponding (A, I) -Moore machine $Iy^I \rightarrow Iy^A$. You may draw it out in polyboxes.
2. Would you say the machine is memoryless?

Exercise 4.11. Find $A, I \in \mathbf{Set}$ such that the following can be identified with a lens $Sy^S \rightarrow Iy^A$, and explain in words what the corresponding (A, I) -Moore machine does (there may be multiple possible solutions):

1. a *discrete dynamical system*, i.e. a set of states S and a transition function $S \rightarrow S$ that describes how to transfer from state to state;
2. a *magma*, i.e. a set S and a function $S \times S \rightarrow S$;
3. a set S and a subset $S' \subseteq S$.

The previous examples of Moore machines mostly had identities as return functions. In the following exercises, we will build examples of Moore machines that do not return their entire states as positions.

Exercise 4.12 (Robot with health). Think of the Moore machine in Example 4.7 as a robot and modify it as follows.

Add to its state a “health meter,” which takes a real value between 0 and 1 representing the robot’s health. Have the robot lose half its health each time it moves to a location whose x -coordinate is negative. Do not return the robot’s health; instead, use its health h as a multiplier, allowing it to move a distance of hr given an input of r .

Exercise 4.13 (Tape of a Turing machine). A Turing machine has a tape consisting of a cell for each integer. Each cell bears a value $v \in V := \{0, 1, -\}$, and one of the cells $c \in \mathbb{Z}$ is distinguished as the “current” cell. So the set of states of the tape is $V^{\mathbb{Z}} \times \mathbb{Z}$.

The Turing machine interacts with the tape by asking for the value of the current cell, an element of V ; and by changing the value of the current cell before moving left (i.e. replacing the current cell $c \in \mathbb{Z}$ with the new cell $c - 1$) or right (i.e. replacing c with $c + 1$). Hence the tape’s position-set is V and its direction-set is $V \times \{\text{left}, \text{right}\}$.

1. If we model the tape as a Moore machine $t: Sy^S \rightarrow Iy^A$, what are S, I , and A ?
2. Write down the specific t that makes it act like a tape as specified above.

Exercise 4.14 (File-reader). Say that a *file* of length n is a function $f: n \rightarrow \text{ascii}$, where $\text{ascii} := 256$. We refer to elements of $n = \{1, \dots, n\}$ as *entries* in the file and, for each entry $i \in n$, the value $f(i) \in \text{ascii}$ as the *character* at entry i .

Given a file f , design a file-reading Moore machine whose position-set is $\text{ascii} + \{\text{done}\}$ and whose direction-set is

$$\{(s, t) \mid 1 \leq s \leq t \leq n\} + \{\text{continue}\}.$$

Given a direction (s, t) , the file-reader should go to entry s in the file and return the character at that entry. If the given direction is instead *continue*, the file-reader should move to the next entry (i.e. from s to $s + 1$) and read that character—unless the new entry would be greater than t , in which case the file-reader should return *done* until it receives another (s, t) pair.

While Exercise 4.14 gives us a functioning file-reader, it is rather awkward that we are still able to give the direction *continue* even when the position is *done*, or provide a new range of entries before the file-reader has finished reading from the previous range. In Section 4.2, we will introduce a generalization of Moore machines to handle cases like these, where the array of directions the machine can receive changes depending on its current position. In particular, we will be able to let the file-reader “close its port,” so that it cannot receive signals while it is busy reading, but open its port once it is done; see Example 4.28.

4.1.1 Deterministic state automata

The diagram in Example 4.2 may look familiar to those who have studied automata theory; in fact, a deterministic state automaton can be expressed as a Moore machine with a distinguished initial state.

Definition 4.15 (Deterministic state automaton, language). A *deterministic state automaton* consists of

- a set S of *states*;
- a set A of *symbols*;

- an *update function* $u : S \times A \rightarrow S$;
- an *initial state* $s_0 \in S$;
- a subset $F \subseteq S$ of *accept states*.

Let

$$\text{List}(A) = \sum_{n \in \mathbb{N}} A^n$$

denote the set of finite sequences $(a_1, \dots, a_n)$ of symbols in A ; we call such a sequence a *word*. We say that the automaton *accepts* the word $(a_1, \dots, a_n)$ if starting at the initial state and following the symbols in the word leads us to an accept state—or, more formally, if the sequence $(s_0, s_1, \dots, s_n)$ defined inductively by

$$s_{k+1} := u(s_k, a_{k+1})$$

is such that s_n is an accept state: $s_n \in F$.

We call a subset of $\text{List}(A)$ a *language*, and we say that the set of all words in $\text{List}(A)$ that the automaton accepts is the language *recognized* by the automaton.

Remark 4.16. When we study a deterministic state automaton, we are usually interested in which words the automaton accepts and, more generally, what language the automaton recognizes. While intuitive, the condition we provided for when an automaton accepts a word can be cumbersome to work with. In Example 8.51, we will give a more compact way of describing whether an automaton accepts a word and specifying the language the automaton recognizes. Better yet, we will find that this alternative formulation arises naturally from the theory of **Poly**.

Proposition 4.17. A deterministic state automaton with a set of states S and a set of symbols A can be identified with a pair of lenses

$$y \rightarrow Sy^S \rightarrow 2y^A.$$

Proof. By Exercise 3.27, a lens $y \rightarrow Sy^S$ can be identified with an initial state $s_0 \in S$. Then a lens $Sy^S \rightarrow 2y^A$ consists of a return function

$f: S \rightarrow 2$, which can be identified with a subset of accept states $F \subseteq S$, together with an update function $u: S \times A \rightarrow S$. $\square$

In other words, we can think of a deterministic state automaton as a Moore machine with position set 2 along with a distinguished initial state; the Moore machine has the same states and update function as the automaton and the automaton's symbols as its directions.

Now imagine if we wanted to construct a version of this automaton that stops reading symbols (i.e. directions) whenever the machine enters an accept state (i.e. returns one position instead of the other). To do this would require a machine whose set of possible directions is dependent on its current position. Instead of an update function $u: S \times A \rightarrow S$, we would need an update function that takes a direction $a \in A$ if the state $s \in S$ is *not* an accept state (say, if $f(s) = 1$) but takes a direction in 0 (i.e. no direction) if the state s is an accept state (if $f(s) = 2$). So there would be one update function $u_s: A \rightarrow S$ if $f(s) = 1$ and a different update function $u_s: 0 \rightarrow S$ if $f(s) = 2$. But these are exactly the on-directions functions of a lens $Sy^S \rightarrow y^A + 1$! Indeed, replacing our interface monomial with a general polynomial is exactly how we will obtain our generalized dependent Moore machines.

4.2 Dependent dynamical systems

Each of our Moore machines above has a monomial Iy^A as an interface. Every representable summand of such an interface has the same representing set A , so the set of directions that can be fed into the machine is always A . But by replacing Iy^A with an arbitrary polynomial p , which may have a different direction-set at each position, we can model a broader class of machines.

Definition 4.18 (Dependent dynamical system). A *dependent dynamical system* (or a *dependent Moore machine*, or simply a *dynamical system*) is a lens

$$\varphi: Sy^S \rightarrow p$$

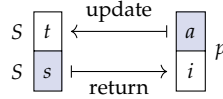
for some $S \in \mathbf{Set}$ and $p \in \mathbf{Poly}$. We call

- the domain Sy^S the machine's *state system*: its position-set

(equivalently, its direction-set) is the machine's *state-set*, and its positions (equivalently, its directions) are the machine's *states*;

- the codomain p the machine's *interface*: its position-set and direction-sets are the same as the machine's *position-set* and *direction-sets*, and its positions and directions are the machine's *positions* and *directions*;
- the on-positions function $\varphi_1: S \rightarrow p(1)$ the machine's *return function*;
- the on-directions map $\varphi^\sharp: p[\varphi_1(-)] \rightarrow S$ the machine's *update map*, and the on-directions function $\varphi_s^\sharp: p[\varphi_1 s] \rightarrow S$ at $s \in S$ the machine's *update function* at s .

Example 4.19 (Dynamical systems as polyboxes). We can express a dynamical system $\varphi: Sy^S \rightarrow p$ in polyboxes as



We can visualize φ as a channel between the internal state system on the left and the external interface on the right. The state system enters its current state $s \in S$ into the left position box, and the return function converts this state to a position $i \in p(1)$ of the interface. At i , the interface has a direction-set $p[i]$; an interacting agent selects one of these directions $a \in p[i]$ to enter into the right direction box. Finally, the update map uses the current state s and the position i to fill the left direction box with the new state $t \in S$. Then the process repeats with t in place of s .

Remark 4.20. It may seem limiting that the set of possible directions a dependent dynamical system can receive should depend on the current *position* rather than the current *state*; but this makes sense philosophically if we accept that the system's interface should capture *everything* about how it interacts with the outside world. In particular, the system's

position should capture everything an external observer could possibly perceive about the system, while the direction-set should capture all the ways in which an external agent can choose to interact with the system. But if the set of directions available to an external agent *changes*, the external agent should be able to detect this fact—the system’s position must have changed as well! On the other hand, if the internal state changes, but the external position remains the same, the agent would see no difference—they would not know to interact with the system any differently, so the directions available to them would have to stay the same, too.

Here are some examples of dependent dynamical systems. We begin by finishing the example at the end of the last section.

Example 4.21 (Halting deterministic state automata). Recall deterministic state automata from Definition 4.15. Say we want such an automaton to halt after reaching an accept state and read no more symbols. Then rather than a lens $Sy^S \rightarrow 2y^A$, we could use a lens

$$\varphi: Sy^S \rightarrow y^A + 1 \cong \{\text{reject}\}y^A + \{\text{accept}\}.$$

To give such a lens, we first need to provide a return function $\varphi_1: S \rightarrow \{\text{reject}, \text{accept}\}$. We let φ send the accept states to `accept` and every other state to `reject`.

If we reach an accept state, we want the machine to halt. So at the position `accept`, corresponding to the summand 1, there are no directions available. This makes the update function $\varphi_s^\#$ vacuous when $\varphi_1 s = \text{accept}$.

On the other hand, when $\varphi_1 s = \text{reject}$, the update functions $\varphi_s^\#: A \rightarrow S$ specify how the machine updates its state for each direction in A if the current state is s . This corresponds to the automaton’s update function.

When equipped with an initial state $s_0 \in S$ specified by a lens $y \rightarrow Sy^S$, we call these dependent dynamical systems *halting deterministic state automata*. Given a word $(a_1, \dots, a_n) \in \text{List}(A)$, we say that the automaton *accepts* this word if starting at the initial state and following the elements in the sequence leads us to an accept

state, without reaching an accept state any earlier—or, more formally, if the sequence $(s_0, s_1, \dots, s_n)$ defined inductively by

$$s_{k+1} := \varphi_{s_k}^\# a_{k+1}$$

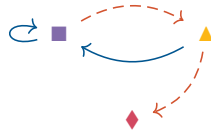
is such that s_n is the sequence's first accept state:

$$\varphi_1 s_k = \begin{cases} \text{reject} & \text{if } k < n; \\ \text{accept} & \text{if } k = n. \end{cases}$$

We call the set of all words accepted by the automaton the language *recognized* by the automaton.

Remark 4.22. Again, the conditions for when such an automaton accepts a word are rather awkward to formally state. We will see in Example 8.50 an alternative way of saying whether a word is accepted by a halting deterministic state automaton.

Exercise 4.23. Consider the halting deterministic state automaton shown below:



(4.24)

Let the left state $\blacksquare$ be 1, the right state $\blacktriangle$ be 2, and the bottom state $\blacklozenge$ be 3. We designate $\blacksquare$, state 1, as the initial state. We can also call the dashed red arrows **red** and the solid blue arrows **blue**. Answer the following questions, in keeping with the notation from Example 4.21.

1. What is S ?
2. What is A ?
3. Based on the labeled transition diagram, which states are accept states, and which are not?
4. Specify the corresponding lens $Sy^S \rightarrow y^A + 1$.
5. Name a word that is accepted by this automaton.
6. Name a word that is not accepted by this automaton. Why

not? Can you find another word that is not accepted by this automaton for a different reason?

For further examples, every graph gives rise to a dynamical system; but to ensure that we are discussing the same concept, let us fix the definition of a graph.

Definition 4.25 (Graph). A graph $G := (E \rightrightarrows V)$ consists of

- a set E of *edges*;
- a set V of *vertices*;
- a *source function* $s: E \rightarrow V$ that assigns each edge a source vertex;
- a *target function* $t: E \rightarrow V$ that assigns each edge a target vertex.

So when we say “graph,” we mean a *directed* graph, and we allow multiple edges between the same pair of vertices as well as self-loops.

Example 4.26 (Graphs as dynamical systems). Given a graph $G := (E \rightrightarrows V)$ with source and target functions $s, t: E \rightarrow V$, there is an associated polynomial

$$g := \sum_{v \in V} y^{s^{-1}(v)}.$$

Its positions are the vertices of the graph, and its directions at $v \in V$ are the edges coming out of v . We call this the *emanation polynomial* of G .

The graph itself induces a dynamical system $\varphi: Vy^V \rightarrow g$, where $\varphi_1 = \text{id}_V$ and $\varphi_v^\# e = t(e)$. So its states as well as its positions are the vertices of the graph, and a direction at a vertex $v \in V$ is an edge $e \in E$ coming out of v that takes us from $v = s(e)$ along the edge e to its target vertex $\varphi_v^\# e = t(e)$.

Exercise 4.27. Pick your favorite graph G , and consider the as-

sociated dynamical system as in Example 4.26. Draw its labeled transition diagram as in (4.3) or (4.24).

Example 4.28. In Exercise 4.14, we built a file-reader as a Moore machine, where a file is a function $f: n \rightarrow \text{ascii}$ from entries to characters. Now we turn that file-reader into a dependent dynamical system $\varphi: Sy^S \rightarrow p$ with only one direction while reading.

We let $S := \{(s, t) \mid 1 \leq s \leq t \leq n\}$, so that each state consists of a current entry s and a terminal entry t . Meanwhile, our interface p will have two labeled copies of ascii as positions:

$$p(1) := \{\text{ready}, \text{busy}\} \times \text{ascii}.$$

So each p -position is a pair (m, c) , where $c \in \text{ascii}$ and m is one of two modes: **ready** or **busy**. Then we define the direction-sets of p for each $c \in \text{ascii}$ as follows:

$$p[(\text{ready}, c)] := S \quad \text{and} \quad p[(\text{busy}, c)] := \{\text{advance}\} \cong 1.$$

That way, our file-reader can receive as its direction any pair of entries in S when it is **ready** but can only be told to **advance** when it is **busy**.

We want our file-reader to be **ready** if its current entry is the terminal entry; otherwise, it will be **busy**. In either case, it will return the character at the current entry. So we define the return function φ_1 such that, for all $(s, t) \in S$,

$$\varphi_1(s, t) = \begin{cases} (\text{ready}, f(s)) & \text{if } s = t; \\ (\text{busy}, f(s)) & \text{otherwise.} \end{cases}$$

While the file-reader is **ready**, we want to set its new current and terminal entries to equal the given direction. So for each $(s, s) \in S$, define the update function $\varphi_{(s, s)}^\# : S \rightarrow S$ to be the identity on S .

On the other hand, while the file-reader is **busy**, we want it to step forward through the file each time it receives an input. So for each $(s, t) \in S$ for which $s < t$, we let the update function

$\varphi_{(s,t)}^\# : 1 \rightarrow S$ specify the element $(s+1, t) \in S$, thus shifting its current entry up by 1.

Exercise 4.29. Say instead of a file-reader, we wanted a file-searcher, which acts just like the file-reader from Example 4.28 except that it only returns $c \in \text{ascii}$ in its position when c is a specific character; say $c = 100$. Otherwise, it returns the placeholder character $_$. Give the lens for this file-searcher by explicitly defining its return (on-positions) and update (on-directions) functions. (Hint: you should be able to use the same state system.)

In the previous exercise, we manually constructed a file-searcher that acted very much like a file-reader. In Exercise 4.40, we will see a simpler way to construct a file-searcher by leveraging the file-reader we have already defined. Moreover, this construction will highlight precisely how our file-searcher is related to our file-reader. This will be possible using *wrapper interfaces*, which we will introduce in Section 4.3.3.

Example 4.30. Choose $n \in \mathbb{N}$, a *grid size*, and for each $i \in n$, let D_i be the set

$$D_i := \begin{cases} \{0, +1\} & \text{if } i = 1; \\ \{-1, 0, +1\} & \text{if } 1 < i < n; \\ \{-1, 0\} & \text{if } i = n. \end{cases}$$

We can think of D_i as the set of ways a robot could move from location i . If $1 < i < n$, a robot may shift its location by -1 (move left/down), 0 (remain still), or $+1$ (move right/up). But a robot already at $i = 1$ cannot shift its location by -1 ; likewise, a robot already at $i = n$ cannot shift its location by $+1$.

Then we can model a robot told to move in an $n \times n$ grid as a dependent dynamical system

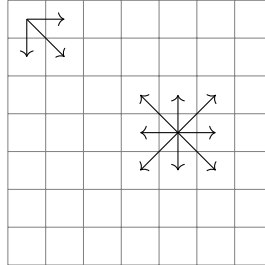
$$\varphi : (n \times n)y^{n \times n} \rightarrow \sum_{(i,j) \in n \times n} y^{D_i \times D_j}.$$

The robot's state is a location $(i, j) \in n \times n$ in the grid. We let $\varphi_1 := \text{id}_{n \times n}$ so that the dynamical system returns its state as its position: the robot expresses its position by moving to that location in the grid.

For each $(i, j) \in n \times n$, we let $\varphi_{(i, j)}^\#$ send each pair $(d, e) \in D_i \times D_j$ to the grid location $(i + d, j + e) \in (n, n)$. Concretely, this says that if a robot located at (i, j) receives the pair (d, e) as its direction, its new position will be $(i + d, j + e)$. As polyboxes, the dynamical system is given by

$$\begin{array}{ccc} n \times n & \boxed{(i + d, j + e)} & \longleftarrow \boxed{(d, e)} \quad D_i \times D_j \\ n \times n & \boxed{(i, j)} & \longrightarrow \boxed{(i, j)} \quad n \times n \end{array}$$

Our definition of D_i for each $i \in n$ guarantees that this position is still inside the grid; with this setup, the robot has fewer ways to move on the sides or corners of the grid than anywhere else:



In this picture, $n := 7$, and the 4 directions at position $(1, 7)$ and 9 directions at position $(5, 4)$ are shown (recall that remaining still is an option in either case).

Note that in this example, the positions are literally the positions in the grid where the robot could be, and the directions at each position are literally the directions in which the robot can move!

Exercise 4.31. Modify the dynamical system from Example 4.30 as follows.

1. Replace the interface with a new polynomial p so that at each grid location, the robot can receive not only the direction it should move in but also a “reward value” $r \in \mathbb{R}$.
2. Replace the state-set with a new set S so that an element $s \in S$ may include both the robot’s position and a list of all reward values so far.
3. With your new p and S , define a new dynamical system $\varphi': Sy^S \rightarrow p$ that preserves the behavior of the dynamical system from Example 4.30 while updating the robot’s reward list without returning it externally.

In the previous exercise, we added a reward system to the robot on the grid by manually redefining the associated lens. But there is a simpler way to think about the new system: it is the juxtaposition of two systems, a robot system and a reward system, in parallel. We will see how to express this in terms of lenses in Exercise 4.37, once we explain how to juxtapose systems like this in general in Section 4.3.2. In fact, we will see in Exercise 4.38 that the robot-on-a-grid system itself can be viewed as the juxtaposition of two systems, and this perspective will provide a structured way to generalize Example 4.30 to more than two dimensions.

4.3 New dynamical systems from old

We have seen how dependent dynamical systems can be modeled as lenses in **Poly** of the form $Sy^S \rightarrow p$. But we have yet to take full advantage of the categorical structure that **Poly** provides. In particular, based only on what we know of **Poly** so far from Chapter 3, we have three rather different ways of constructing new dynamical systems from old ones:

1. Given dynamical systems $Sy^S \rightarrow p$ and $Sy^S \rightarrow q$, we can use the universal property of the *categorical product* to obtain a dynamical system $Sy^S \rightarrow p \times q$; see Section 4.3.1.
2. Given dynamical systems $\varphi: Sy^S \rightarrow p$ and $\psi: Ty^T \rightarrow q$, we can take their parallel product to obtain a dynamical system $\varphi \otimes \psi: STy^{ST} \rightarrow p \otimes q$; see Section 4.3.2.

3. Given a dynamical system $\varphi: Sy^S \rightarrow p$ and a lens $f: p \rightarrow q$, we can compose them to obtain a dynamical system $\varphi \circ f: Sy^S \rightarrow q$; see Section 4.3.3.

Each of these operations has a concrete interpretation in terms of the systems' behavior. In this section, we will review each of them in turn.

4.3.1 Categorical products: multiple interfaces operating on the same states

Let I be a set, and say that we have an I -indexed family of dependent dynamical systems $(\varphi_i: Sy^S \rightarrow p_i)_{i \in I}$ that all share the same state-set S . Then since **Poly** has all small products, the universal property of products induces a lens

$$\varphi: Sy^S \rightarrow \prod_{i \in I} p_i,$$

which is itself a dynamical system with state-set S . By (3.57), the interface of φ (i.e. the product that is its codomain) has position-set

$$\left(\prod_{i \in I} p_i \right) (1) \cong \prod_{i \in I} (p_i(1))$$

and, at each position $\bar{j}: (i \in I) \rightarrow p_i(1)$, direction-set

$$\sum_{i \in I} p_i[\bar{j}i].$$

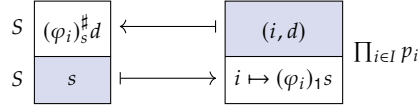
We then characterize the dynamics of φ in terms of each φ_i by leveraging the universal property of products in **Poly**, detailed in the binary case in the solution to Exercise 3.64, as follows. The return function

$$\varphi_1: S \rightarrow \prod_{i \in I} (p_i(1))$$

sends each state $s \in S$ to the dependent function $(i \in I) \rightarrow p_i(1)$ sending $i \in I$ to the position $(\varphi_i)_1 s$ returned by the corresponding dynamical system φ_i at the state s . Then the update function

$$\varphi_s^\#: \sum_{i \in I} p_i[(\varphi_i)_1 s] \rightarrow S$$

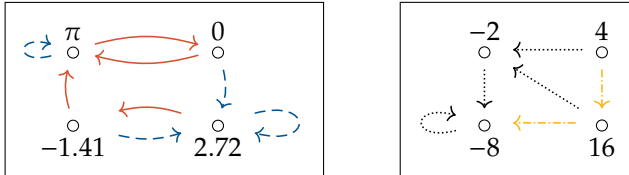
at $s \in S$ sends each pair (i, d) in its domain, with $i \in I$ and $d \in p_i[(\varphi_i)_1 s]$, to where the update function of φ_i at s sends the direction d : namely $(\varphi_i)_s^\# d$. We can write φ using polyboxes as follows:



In other words, if there are multiple interfaces that drive the same set of states, we may view them as a single product interface that drives those states. This single dynamical system returns positions in all of the original systems at once; then it can receive a direction from any one of the original systems' direction-sets and update its state accordingly. It is as though all of the dynamical systems can detect the current state of the combined system, but only one of them can change the state at a time. So products give us a universal way to combine multiple polynomial interfaces into one.

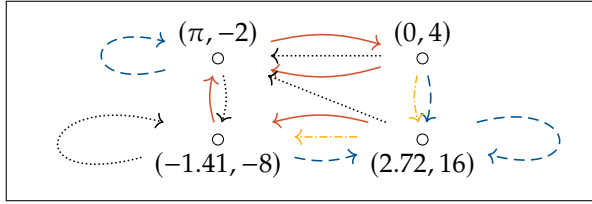
Exercise 4.32. Given a set I , suppose we have an (A_i, B_i) -Moore machine with state-set S for each $i \in I$. Show that there is an induced $(\sum_{i \in I} A_i, \prod_{i \in I} B_i)$ -Moore machine, again with state-set S .

Example 4.33. Consider two four-state dependent dynamical systems $\varphi: 4y^4 \rightarrow \mathbb{R}y^{\{\text{red}, \text{blue}\}}$ and $\psi: 4y^4 \rightarrow \mathbb{Z}_{\geq 0}y^{\{\text{black}, \text{yellow}\}} + \mathbb{Z}_{< 0}y^{\{\text{black}\}}$, drawn below as labeled transition diagrams, with **red** solid arrows and **blue** dashed arrows on the left and **black** dotted arrows and **yellow** dashed-and-dotted arrows:



The universal property of products provides a unique way to put these systems together to obtain a dynamical system $4y^4 \rightarrow$

$\mathbb{R}Z_{\geq 0}y^{\{\text{red}, \text{blue}, \text{black}, \text{yellow}\}} + \mathbb{R}Z_{< 0}y^{\{\text{red}, \text{blue}, \text{black}\}}$ that looks like this:



Each state now returns two positions: one according to the return function of φ , and another according to the return function of ψ . As for the possible directions, we can now choose either a direction of φ (either **red** or **blue**), in which case the dynamical system will update its state according to the update map of φ ; or a direction of ψ (either **black** or sometimes **yellow**), in which case the dynamical system will update its state according to the update map of ψ .

Exercise 4.34 (Toward event-based systems). Let $\varphi: Sy^S \rightarrow p$ be a dynamical system. We can think of it as requiring a direction at each time step to update its state.

Suppose we want to change φ into an *event-based system*: one that does not always receive a direction and changes state only when it does. That is, we want every position of p to have an extra direction that, when selected, never changes the state. We want a new system $\varphi': Sy^S \rightarrow p'$ that has this behavior; what should p' and φ' be?

4.3.2 Parallel products: Juxtaposing dynamical systems

Another way to combine two polynomials—and indeed two lenses—is by taking their parallel product, as in Definition 3.65. In particular, the parallel product of two state systems is still a state system. So parallel products give us another way to create new dynamical systems from old ones. The procedure is straightforward: take the product of the state-sets as the new state-set, the product of the position-sets as the new

position-set, and the product of the direction-sets at each of position in a tuple of positions as the new direction-set at that tuple.

For $n \in \mathbb{N}$, say that we have n dynamical systems: a lens $\varphi_i: S_i y^{S_i} \rightarrow p_i$ for each $i \in n$. Then we can take the parallel product of all of them to obtain a lens

$$\varphi: \bigotimes_{i \in n} (S_i y^{S_i}) \rightarrow \bigotimes_{i \in n} p_i.$$

By inductively applying Exercise 3.67, we find that the domain of φ is

$$\bigotimes_{i \in n} (S_i y^{S_i}) \cong \left(\prod_{i \in n} S_i \right) y^{\prod_{i \in n} S_i},$$

so φ is itself a dependent dynamical system with state-set $\prod_{i \in n} S_i$. Meanwhile, inductively applying (3.66) yields

$$\left(\bigotimes_{i \in n} p_i \right) (1) \cong \prod_{i \in n} (p_i(1))$$

as the position-set and, at each position $(j_i)_{i \in n}$,

$$\prod_{i \in n} p_i[j_i]$$

as the direction-set of the interface of φ .

We can characterize the dynamics of φ in terms of each constituent dynamical system φ_i as follows. By our proof sketch of Proposition 3.73, the return function

$$\varphi_1: \prod_{i \in n} S_i \rightarrow \prod_{i \in n} (p_i(1))$$

sends each n -tuple of states $(s_i)_{i \in n}$ in its domain, with each $s_i \in S_i$, to the n -tuple of positions $((\varphi_i)_1 s_i)_{i \in n}$ returned by each of the constituent dynamical systems at each state. Then at the n -tuple of states $(s_i)_{i \in n} \in \prod_{i \in n} S_i$, the update function

$$\varphi_{(s_i)_{i \in n}}^\sharp: \prod_{i \in n} p_i[(\varphi_i)_1 s_i] \rightarrow \prod_{i \in n} S_i$$

sends each n -tuple of directions $(d_i)_{i \in n}$ in its domain, with each $d_i \in p_i[(\varphi_i)_1 s_i]$, to the n -tuple $((\varphi_i)_{s_i}^\sharp d_i)_{i \in n}$ consisting of states to which the

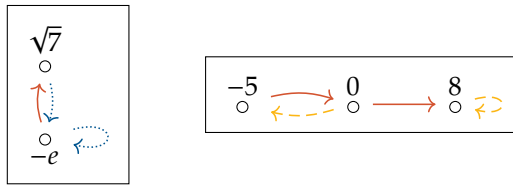
update function of each φ_i at s_i sends d_i . We can write φ using polyboxes as follows:

$$\begin{array}{ccc} \prod_{i \in n} S_i & \begin{array}{|c|} \hline ((\varphi_i)_{s_i}^\# d_i)_{i \in n} \\ \hline \end{array} & \longleftarrow \begin{array}{|c|} \hline (d_i)_{i \in n} \\ \hline \end{array} \\ \prod_{i \in n} S_i & \begin{array}{|c|} \hline (s_i)_{i \in n} \\ \hline \end{array} & \longrightarrow \begin{array}{|c|} \hline ((\varphi_i)_1 s_i)_{i \in n} \\ \hline \end{array} \end{array} \otimes_{i \in n} p_i$$

In other words, multiple dynamical systems running in parallel can be thought of as a single dynamical system. This system stores the states of all the constituent systems at once and returns positions from all of them together; then it can receive directions from all of the constituent systems' direction-sets at those positions at once and update each constituent state accordingly. So parallel products give us a way to juxtapose multiple dynamical systems in parallel to form a single system.

Exercise 4.35. Given $n \in \mathbb{N}$, suppose we have an (A_i, B_i) -Moore machine with state-set S_i for every $i \in n$. Show that there is an induced $(\prod_{i \in n} A_i, \prod_{i \in n} B_i)$ -Moore machine with state-set $\prod_{i \in n} S_i$.

Example 4.36. Consider dependent dynamical systems $\varphi: 2y^2 \rightarrow \mathbb{R}_{<0}y^{\{b, r\}} + \mathbb{R}_{\geq 0}y^{\{b\}}$ and $\psi: 3y^3 \rightarrow \mathbb{Z}_{<0}y^{\{r\}} + \{0\}y^{\{r, y\}} + \mathbb{Z}_{>0}y^{\{y\}}$, drawn below as labeled transition diagrams (we think of **b** as blue and dotted, **r** as red and solid, and **y** as yellow and dashed):

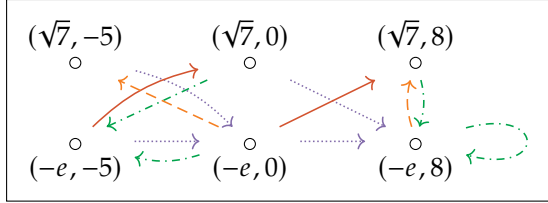


Taking their parallel product, we obtain a dynamical system with state system $6y^6$ and interface

$$\begin{aligned} & \mathbb{R}_{<0}\mathbb{Z}_{<0}y^{\{(b, r), (r, r)\}} + \mathbb{R}_{<0}\{0\}y^{\{(b, r), (b, y), (r, r), (r, y)\}} \\ & + \mathbb{R}_{<0}\mathbb{Z}_{>0}y^{\{(b, y), (r, y)\}} + \mathbb{R}_{\geq 0}\mathbb{Z}_{<0}y^{\{(b, r)\}} \end{aligned}$$

$$+ \mathbb{R}_{\geq 0}\{0\}y^{\{(b, r), (b, y)\}} + \mathbb{R}_{\geq 0}\mathbb{Z}_{> 0}y^{\{(b, y)\}}$$

that looks like this (we use **purple** and dotted arrows for (b, r) ; **red** and solid arrows for (r, r) ; **green**, dotted, and dashed arrows for (b, y) ; and **orange** and dashed arrows for (r, y)):



Each state—actually a pair of states from the constituent state-sets—returns two positions, one according to the return function of φ and another according to the return function of ψ . Then every direction must be a pair of directions from the constituent interfaces at those positions, with the update function updating each state in the pair according to each direction in the pair via the constituent update functions of φ and ψ .

Exercise 4.37. Explain how the dynamical system $\varphi': S'y^{S'} \rightarrow p'$ you built in Exercise 4.31 can be expressed as the parallel product of the robot-on-a-grid dynamical system $\varphi: Sy^S \rightarrow p$ from Example 4.30 with another dynamical system, $\psi: Ty^T \rightarrow q$. Be sure to specify T, q , and ψ .

Exercise 4.38.

1. Explain how the robot-on-a-grid dynamical system $\varphi: Sy^S \rightarrow p$ from Example 4.30 can be written as the parallel product of some dynamical system with itself.
2. Use k -fold parallel products to generalize Example 4.30 to robots on k -dimensional grids.

Intuitively, the parallel product takes two dynamical systems and puts them in the same room together so that they can be run at the same

time. But it does not allow for any interaction *between* the two systems. For that, we will need to use what we call a wrapper interface. We will introduce wrapper interfaces in the next section before describing how they can be used in conjunction with parallel products to model general interaction in Section 4.4.

4.3.3 Composing lenses: Wrapper interfaces

Given a dynamical system $\varphi: Sy^S \rightarrow p$, say that we want to interact with its state system using a new interface q rather than p . We can do this whenever we have a lens $f: p \rightarrow q$, which we could compose with our original dynamical system to obtain a new system $Sy^S \xrightarrow{\varphi} p \xrightarrow{f} q$. We call the lens f the *wrapper* and its codomain q the *wrapper interface*, which we *wrap* around φ (or sometimes just p , if a dynamical system φ has yet to be specified) using f .

How does this new composite system $\varphi \circ f$ relate to the original dynamical system φ ? The lens f converts a position i from p to a position $f_1 i$ from q ; at the same time, it allows the choice of direction from $p[i]$ to depend on a choice of direction from $q[f_1 i]$, converting directions of the wrapper interface q to directions of the original interface p . Pre-composing f with a dynamical system yields a new dynamical system that lets an agent interact with the original system using only this new interface wrapped around it.

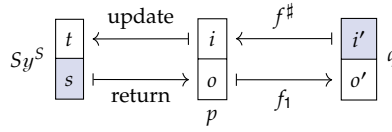
Example 4.39. Consider a dependent dynamical system $\varphi: 6y^6 \rightarrow p$ with

$$p := \{1\}y^{\{\text{blue}, \text{yellow}, \text{red}\}} + \{2\}y^{\{\text{blue}, \text{red}\}} + \{3\}y^{\{\text{blue}\}} + \{4\}y^{\{\text{red}\}},$$

drawn below as a labeled transition diagram (with **blue** dashed

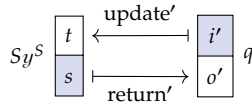
Exercise 4.40. In Exercise 4.29, we built a file-searcher $\psi: Sy^S \rightarrow q$ by taking the file-reader $\varphi: Sy^S \rightarrow p$ from Example 4.28 and replacing its interface p with a new interface q while keeping its state system Sy^S the same. Express this construction as wrapping q around φ by giving a lens $f: p \rightarrow q$ for which composing φ with f yields ψ .

Example 4.41 (Polybox pictures of wrapper interfaces). In polyboxes, composing a dynamical system $\varphi: Sy^S \rightarrow p$ with a wrapper $f: p \rightarrow q$ looks like this:



The position o displayed by the intermediary interface p is instead exposed as a position $f_1(o) = o'$ of the wrapper interface q in the rightmost position box. Moreover, the direction box of p is no longer blue: an agent who wishes to interact with the middle interface p can only do so via the rightmost interface q . The on-directions function of the wrapper at o converts a direction $i' \in q[o']$ from the rightmost direction box into a direction $i \in p[o]$.

Picture the agent standing to the right of all the polyboxes (i.e. “outside” of the system) with their attention directed leftward (i.e. “inward”), receiving positions from the white position box and feeding directions into the blue direction box. To an agent who is unaware of its inner workings, the composite dynamical system $\varphi \circ f$ might as well look like this:



In the next section, we describe a special kind of wrapper.

4.3.4 Sections as wrappers

Say we wanted to model a dynamical system $\varphi: Sy^S \rightarrow p$ within a closed system, for which an external agent can perceive no change in position and effect no change in direction. We can think of this as wrapping y , the interface with one position and one direction, around φ . To do so, we must specify a wrapper $\gamma: p \rightarrow y$. In the language of Definition 3.36, this is precisely a *section* of p . As we noted then, this name is appropriate, since γ a way of sectioning off the interface p from the outside world.

Recall that a section $\gamma: p \rightarrow y$ can be identified with a dependent function of the form $(i \in p(1)) \rightarrow p[i]$ that sends each p -position i to a $p[i]$ -direction, fixing a direction at every position of p . So a section for an interface dictates the direction it receives given any position it inhabits; there is no need for any further outside interference.

Exercise 4.42. Let $\varphi: Sy^S \rightarrow By^A$ be an (A, B) -Moore machine.

1. Is it true that a section $\gamma: By^A \rightarrow y$ can be identified with a function $A \rightarrow B$?
2. Describe how to interpret a section $\gamma: By^A \rightarrow y$ as a wrapper around an interface By^A .
3. Given a section γ , describe the dynamics of the composite Moore machine

$$Sy^S \xrightarrow{\varphi} By^A \xrightarrow{\gamma} y$$

obtained by wrapping y around φ using γ .

Example 4.43 (The do-nothing section). There is something rather off-putting about the way we model dynamical systems as lenses $\varphi: Sy^S \rightarrow p$. We know that φ sends states-as-positions to positions of the interface and, at each state-as-position, sends directions of the interface to states-as-directions. But we rely only on the labels of elements in S to tell us which positions and directions refer to the same states!

Nothing inherent in the language of **Poly** makes these associations between states-as-positions and states-as-directions for us;

we have to rely on the position-set and direction-sets of the state system being the same set for the machine to work properly. Put another way, the monomials $\{4, 6\}y^{\{4,6\}}$, $\{4, 6\}y^{\{4,8\}}$, and $\{3, 5\}y^{\{6,7\}}$ are all isomorphic in **Poly**, but the first can be a state system while the other two cannot!

To address this issue, we need a way to connect the positions of a polynomial to its own directions in the language of **Poly**. This is where sections can help: a lens $Sy^S \rightarrow y$ is just a way of assigning to each position in S a direction in S . So we can define $\epsilon: Sy^S \rightarrow y$ to be the section that sends each position $s \in S$ to the direction s at s with the same name, corresponding to the same state. Note that ϵ can be identified with the identity function on S (see Exercise 4.42). Now **Poly** knows for each position which direction is associated with the same state the position is.

In this way, we can generalize our notion of state systems to monomials $Sy^{S'}$ equipped with a bijection $S \rightarrow S'$, which we can then translate to a section $\epsilon: Sy^{S'} \rightarrow y$. But for convenience of notation, we will continue to identify the position-set of a state system with each of its direction-sets.

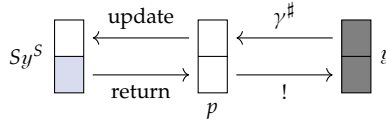
More concretely, the section $\epsilon: Sy^S \rightarrow y$ acts as a very special (if rather unexciting) dynamical system: it is the *do-nothing section*, with only one possible position and one possible direction that always keeps the current state the same. While the system literally does nothing, we do know one key fact about it: given any state system, regardless of the state-set, we can always define a do-nothing section on it.¹

Yet this is not the whole story. The do-nothing section knows, at each position, the direction that keeps the system at the same state; but it does not know which of the other directions at that position correspond to which of the other states of the system. We are still relying on the labels of direction-sets being the same for that: for instance, the polynomials $\{1', 2', 3'\}y^{\{1,2,3\}}$ and $\{1'\}y^{\{0,1,4\}} + \{2'\}y^{\{2,5,6\}} + \{3'\}y^{\{-8,-1,3\}}$ are isomorphic, but even though each has a do-nothing section matching $1' \mapsto 1, 2' \mapsto 2, 3' \mapsto 3$ that makes the first one into a state system, we do not have a way to tell

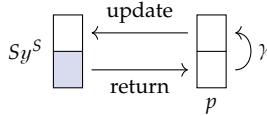
Poly how to make the second one a state system yet.

From another perspective, $\epsilon: Sy^S \rightarrow y$ does nothing, while $\varphi: Sy^S \rightarrow p$ does “one thing”: it steps through the system once, producing the current state’s position with the return function and taking in directions with the update function. It is ready to take another step, but how does **Poly** know which state to visit next? Is there a lens that does “two things,” “ n things,” or “arbitrarily many things”? Can we actually *run* a dynamical system in **Poly**? We will develop the machinery to answer these questions over the course of Part II, starting in Section 6.1.4.

Example 4.44 (Polybox pictures of sections as wrappers). In polyboxes, composing a system $Sy^S \rightarrow p$ with a section γ of p can be depicted as



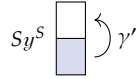
or, equivalently, as



Remember: in a polybox depiction of a dynamical system, the world outside the system exists to the right of all the boxes. So the first picture represents y as a gray wall, cutting off any interaction between the system to its left and the world to its right. Meanwhile, the second picture illustrates how a sectioned-off sys-

¹It may have bothered you that we call Sy^S , which is a single polynomial, a state *system*, when we also use the word “system” to refer to dependent dynamical systems, which are lenses $Sy^S \rightarrow p$. The existence of the do-nothing section explains why our terminology does not clash: every polynomial Sy^S comes equipped with a dependent dynamical system $\epsilon: Sy^S \rightarrow y$, so it really is a state *system*.

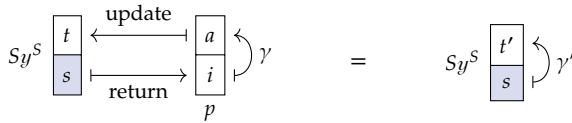
tem independently selects directions of the intermediary interface p via γ according to the p -positions that the inner (leftward) system $Sy^S \rightarrow p$ returns. While the second picture shows us why the closed system neither seeks nor requires external directions, the first picture helps remind us that any returned p -positions never reach the outside world either. The composite system is therefore equivalent to the section drawn as follows:



In lens parlance, $\gamma': Sy^S \rightarrow y$ is the original system $Sy^S \rightarrow p$ composed with $\gamma: p \rightarrow y$; in the language of dependent functions, $\gamma': S \rightarrow S$ is given by

$$\gamma'(s) = \text{update}(s, \gamma(\text{return}(s))) \quad \text{for all } s \in S,$$

where we interpret γ as a dependent function $(i \in p(1)) \rightarrow p[i]$. We can deduce this equation by equating the previous two polybox pictures, knowing they represent the same lens:

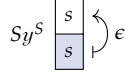


Equating the directions boxes of the domain on either side, we have that $t = t'$, so

$$\gamma'(s) = \text{update}(s, a) = \text{update}(s, \gamma(i)) = \text{update}(s, \gamma(\text{return}(s))).$$

Later on we will read more intricate equations from polyboxes in this manner, although we will not spell out the procedure in so much detail; we encourage you to trace through the arrows on your own.

Example 4.45 (The do-nothing section in polyboxes). In Example 4.43, we saw that every state system Sy^S can be equipped with a section $\epsilon: Sy^S \rightarrow y$ called the do-nothing section, which assigns each state-as-position to its corresponding state-as-direction, thus leaving the state unchanged. That is, it is the section whose polyboxes can be drawn as follows:



4.4 General interaction

We are now ready to use **Poly** to model interactions between dependent dynamical systems that can change their interfaces and interaction patterns.

4.4.1 Wrapping juxtaposed dynamical systems together

When wrapper interfaces are used in conjunction with parallel products, they may encode multiple interacting dynamical systems as a single system. Explicitly, given $n \in \mathbb{N}$ and a dynamical system $\varphi_i: S_i y^{S_i} \rightarrow p_i$ for each $i \in n$, we can first juxtapose them to form a single dynamical system $\varphi \circ f$:

$$\varphi: \left(\prod_{i \in n} S_i \right) y^{\prod_{i \in n} S_i} \rightarrow \bigotimes_{i \in n} p_i$$

by taking their parallel product. Then we can wrap an interface q around φ using a wrapper $f: \bigotimes_{i \in n} p_i \rightarrow q$, yielding a new composite dynamical system

$$\left(\prod_{i \in n} S_i \right) y^{\prod_{i \in n} S_i} \xrightarrow{\varphi} \bigotimes_{i \in n} p_i \xrightarrow{f} q.$$

On positions, f gives a way of combining all the positions of the constituent interfaces into a single position of the wrapper interface. On directions, f takes into account the current positions of each constituent

interface along with a direction for the wrapper interface to specify a direction for each of the constituent interfaces. In particular, a judiciously chosen on-directions function could feed positions of some interfaces as directions to others. When f is a wrapper around a parallel product of interfaces, we call f the *interaction pattern* between those interfaces.

Example 4.46 (Repeater). Suppose we have a dynamical system $\varphi: Sy^S \rightarrow Ay + y$ that takes unchanging directions and sometimes returns elements of A while other times returning only silence (the position associated with the right-hand summand y). What if we wanted to construct a system ψ that operates just like φ , but *always* returns elements of A as output? Where φ would have returned silence, we want ψ to instead *repeat* the last element of A that was returned. (We allow ψ to repeat an arbitrary element of A if φ returns silence before it has returned any elements of A yet.)

What we need is a way to store an element of A and retrieve it as needed. So whenever φ returns an element of A , we store it; then when φ returns silence, we retrieve the last element of A we stored and return that instead.

What should this storage-retrieval dynamical system look like? It needs to take elements of A as directions, return elements of A as positions, and store elements of A as states. In fact, the identity lens $\iota: Ay^A \rightarrow Ay^A$ works perfectly: it returns the element of A currently stored as its position and updates its state to the direction it receives.

Now we can juxtapose our original system φ with the storage-retrieval system ι by taking their parallel product, yielding a dynamical system

$$\varphi \otimes \iota: Sy^S \otimes Ay^A \cong SAy^{SA} \rightarrow (Ay + y) \otimes Ay^A$$

that runs both systems in parallel: simultaneously and independently. But what we want is for φ and ι to interact with each other, and for the resulting system to only return elements of A . To do so, we need to wrap an interface Ay around $\varphi \otimes \iota$ by composing it

with some lens

$$f: (Ay + y) \otimes Ay^A \rightarrow Ay,$$

the interaction pattern between the interfaces $Ay + y$ and Ay^A that we must define. Then we will be able to define $\psi := (\varphi \otimes \iota) \circ f$.

Since $\otimes$ distributes over $+$, by the universal property of the coproduct, it suffices to give lenses

$$g: (A \times A)y^A \rightarrow Ay \quad \text{and} \quad h: Ay^A \rightarrow Ay.$$

The former corresponds to the case where φ returns an element of A , while the latter corresponds to the case where φ is silent.

When φ returns an element of A , we want the composite system ψ to return that same element, but we also want to give that position as a direction to ι so that it can be stored. We do not need to do anything with the position returned by ι ; we can simply discard it. So g should map $(a, a') \mapsto a$ on positions, yielding the position returned by φ and discarding the position returned by ι ; and the on-directions function $g_{(a, a')}^\# : 1 \rightarrow A$ should pick out the direction $a \in A$, feeding the position returned by φ as a direction for ι .

On the other hand, when φ returns silence, we want ψ to return the position of ι instead. We also need to feed this position back to ι as a direction so that it can continue to be stored. So h should be the identity on positions as well as the identity on directions.

Example 4.47 (Paddling). We build a Moore machine with interface $\mathbb{N}y$, interpreting the natural number position it returns as the machine's current location. Suppose we want to control how far the machine can move and what can make it move.

To model this, we introduce two intermediary systems, which we call the *paddler* and the *tracker*:²

$$\text{paddler}: Sy^S \rightarrow 2y \quad \text{and} \quad \text{tracker}: Ty^T \rightarrow \mathbb{N}y^2.$$

The paddler has interface $2y$ because it is blind (i.e. takes no directions) and can only move its paddle (i.e. return a position) left

or right: its position-set is $2 \cong \{\text{left}, \text{right}\}$. The tracker has interface $\mathbb{N}y^2$ because it will return the location of the machine (as an element $n \in \mathbb{N}$) as its position and take in the position of the paddler (as an element of 2) as its direction. We can wrap an interface $\mathbb{N}y$ around them both using an interaction pattern

$$2y \otimes \mathbb{N}y^2 \cong 2\mathbb{N}y^2 \rightarrow \mathbb{N}y$$

whose on-positions function is the canonical projection $2\mathbb{N} \rightarrow \mathbb{N}$, returning the location returned by the tracker, and whose on-directions map is the projection $2\mathbb{N} \rightarrow 2$, passing the position of the paddler as a direction to the tracker.

We focus on the dynamics of the tracker. We want it to watch for when the paddle switches from left to right or from right to left, whereupon it should push the machine forward one unit. Thus the states of the tracker are given by $T := 2\mathbb{N}$, storing which side the paddler is on and the machine's current location. The on-positions function of the tracker is the canonical projection $2\mathbb{N} \rightarrow \mathbb{N}$ that returns the current location; then at each $(d, i) \in 2\mathbb{N}$, the on-directions function of the tracker $2 \rightarrow 2\mathbb{N}$ sends

$$d' \mapsto \begin{cases} (d', i) & \text{if } d = d'; \\ (d', i + 1) & \text{if } d \neq d'; \end{cases}$$

storing the new position of the paddler as well as moving the machine forward one unit if the paddle switches while keeping the machine still if the paddle stays still.

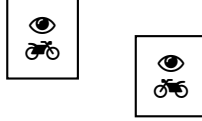
Exercise 4.48. Change the dynamics and state system of the tracker in Example 4.47 so that it exhibits the following behavior.

When the paddle switches once and stops, the tracker increases the location by one unit and stops, as before in Example 4.47. But when the paddle switches twice in a row, the tracker increases the

²Perhaps one could refer to the tracker as the *demiurge*; it is responsible for maintaining the material universe.

location by two units on the second switch. So if the paddler is stable for a while, then switches three times in a row, the tracker will increase the location by one, then two, then two again.

Example 4.49. Suppose you have two systems with the same interface $p := q := \mathbb{R}^2 y^{\mathbb{R}^2 - \{(0,0)\}}$.



The ordered pair comprising the position of each interface indicates the location of the corresponding system, while the range of possible directions indicates the locations that the system could observe, relative to the location of the system itself. Taking all pairs of reals except $(0,0)$ corresponds to the fact that the eye cannot see anything at the same location as the eye itself.

Let us have the two systems approach each other accelerating at a rate equal to the reciprocal of the squared distance between them, modeled with discrete time units. If they finally collide, let us have both systems halt. To do this, we want the wrapper interface to be $\{\text{go}\}y + \{\text{stop}\}$, so that if the system returns *go*, it can still advance to the next state; but if it returns *stop*, it halts. The wrapper $\mathbb{R}^2 y^{\mathbb{R}^2 - \{(0,0)\}} \otimes \mathbb{R}^2 y^{\mathbb{R}^2 - \{(0,0)\}} \rightarrow \{\text{go}\}y + \{\text{stop}\}$ is given on positions by

$$((x_1, y_1), (x_2, y_2)) \mapsto \begin{cases} \text{stop} & \text{if } x_1 = x_2 \text{ and } y_1 = y_2; \\ \text{go} & \text{otherwise.} \end{cases}$$

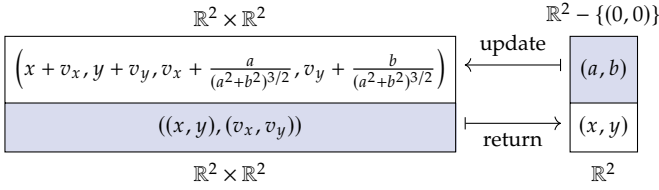
On directions, we use the function

$$((x_1, y_1), (x_2, y_2)) \mapsto ((x_2 - x_1, y_2 - y_1), (x_1 - x_2, y_1 - y_2)),$$

so that each system is able to see the location of the other system relative to its own, i.e. the vector pointing from itself to the other

system (unless that vector is zero, in which case the system should have returned the stop position and halted).

We can use these vectors to define the internal dynamics of each system so that they move the way we want them to. Each system will hold as its internal state its current location and velocity as vectors, i.e. $S := \mathbb{R}^2 \times \mathbb{R}^2$. To define a lens $Sy^S \rightarrow \mathbb{R}^2 y^{\mathbb{R}^2 - \{(0,0)\}}$ we simply return the current location, update the current location by adding the current velocity vector, and update the current velocity vector by adding an acceleration vector with appropriate magnitude pointing to the other system:



Exercise 4.50. Suppose (X, d) is a *metric space*, i.e. X is a set of *points* and $d: X \times X \rightarrow \mathbb{R}_{\geq 0}$ is a function called the *distance function* for which $d(x, y) = d(y, x)$, $d(x, y) = 0$ if and only if $x = y$, and $d(x, y) + d(y, z) \geq d(x, z)$ for all $x, y, z \in X$. Let us have robots interact in this space.

Let A, A' be sets, each thought of as a set of signals, and let $a_0 \in A$ and $a'_0 \in A'$ be elements, each thought of as a default value. Let $p := AXy^{A'X}$ and $p' := A'Xy^{AX}$, and imagine there are two robots, one with interface p , returning a signal as an element of A and its location as a point in X , and one with interface p' , returning a signal as an element of A' and also its location as a point in X .

1. Write down an interaction pattern $p \otimes p' \rightarrow y$ such that each robot receives the other's location but only receives the other's signal when their locations x, x' are sufficiently close, namely when $d(x, x') < 1$. Otherwise, it receives the default signal.
2. Modify the previous interaction pattern to specify a new interaction pattern $p \otimes p' \rightarrow y^{[0,5]}$ where the value $s \in [0, 5]$ is a

- scalar that changes the distance threshold for the signal to s .
3. Suppose that each robot has a set S, S' of possible private states in addition to their locations. What functions are involved in providing a dynamical system $\varphi: SXy^{SX} \rightarrow AXy^{A'X}$, if the location state $x \in X$ is directly returned without modification?
 4. Change the setup in any way so that each robot only extends a port to hear the other's signal when the distance between them is less than s . Otherwise, they can only detect the position (element of X) that the other currently inhabits. (Don't worry too much about timing—one missed signal when the robots first get close or one extra signal when the robots first get far is okay.)

4.4.2 Sectioning off juxtaposed dynamical systems

We saw in Section 4.3.4 that a section (i.e. lens to y) for the interface of a dynamical system sections that dynamical system off as a closed system. So it should not come as a surprise that a section for a parallel product of interfaces yields an interaction pattern between the interfaces that only allows the interfaces to interact with each other, cutting off any other interaction with the outside world.

Example 4.51 (Picking up the chalk). Imagine that you see some chalk and you pinch it between your thumb and forefinger. An amazing thing about reality is that you will then have the chalk, in the sense that you can move it around. How might we model this in **Poly**? We will construct a closed dynamical system—one with interface y —consisting of only you and the chalk. To do so, we will provide an interface for you, and interface for the chalk, and a section for your juxtaposition.

Say that your hand can be at one of two heights, down or up, and that your fingers can either be pressed (with pressure between your thumb and forefinger) or unpressed. Say too that you take in information about the chalk's height, which can be down or up as

well. Here are the two sets we will be using:

$$H := \{\text{down}, \text{up}\} \quad \text{and} \quad P := \{\text{pressed}, \text{unpressed}\}.$$

Your interface is HPy^H : your position is your own height and pressure, and your possible directions are the chalk's possible heights. As for the chalk, it is either in your possession or out of it as well as either down or up. The direction the chalk receives includes whether it is pressed or unpressed. When it's out of your possession, that is the entire direction, but when it is in your possession, its direction also comprises your hand's height. In summary, here are the two interfaces:

$$\text{You} := HPy^H \quad \text{and} \quad \text{Chalk} := \{\text{out}\}Hy^P + \{\text{in}\}Hy^{HP}.$$

Now we want to give the interaction pattern between you and the chalk. As we said before, you see the chalk's height. If your hand is not at the height of the chalk, the chalk remains unpressed. Otherwise, your hand is at the height of the chalk, so the chalk receives your pressure (or lack thereof). Furthermore, if the chalk is in your possession, it also receives your hand's height.

To provide a lens $\gamma: \text{You} \otimes \text{Chalk} \rightarrow y$, we use the fact that Chalk is a coproduct and that $\otimes$ distributes over coproduct to write $\text{You} \otimes \text{Chalk}$ as a coproduct itself:

$$\text{You} \otimes \text{Chalk} \cong HPy^H \otimes \{\text{out}\}Hy^P + HPy^H \otimes \{\text{in}\}Hy^{HP}.$$

Then by the universal property of coproducts, to define γ , it suffices to define two lenses

$$\alpha: HPy^H \otimes \{\text{out}\}Hy^P \rightarrow y \quad \text{and} \quad \beta: HPy^H \otimes \{\text{in}\}Hy^{HP} \rightarrow y.$$

The lens β , corresponding to when the chalk is in your possession, is easy to describe: it can be identified with a function $HPH \rightarrow HHP$, and we take it to be the obvious function sending your height and pressure to the chalk and the chalk's height to you; see Exercise 4.53. But α , corresponding to when the chalk is out of your possession, is

more semantically interesting: it can be identified with a function $HPH \rightarrow HP$ given by

$$(h_{\text{You}}, p_{\text{You}}, h_{\text{Chalk}}) \mapsto \begin{cases} (h_{\text{Chalk}}, \text{unpressed}) & \text{if } h_{\text{You}} \neq h_{\text{Chalk}}; \\ (h_{\text{Chalk}}, p_{\text{You}}) & \text{if } h_{\text{You}} = h_{\text{Chalk}}. \end{cases}$$

In words, this says that if you and the chalk are at different heights, then regardless of your pressure, the chalk remains unpressed; but if you are at the same height as the chalk, the chalk receives your pressure.

Now that you and the chalk are sectioned off together by γ , we are ready to add some dynamics. Your dynamics can be whatever you want, so let us focus on giving dynamics to the chalk (you will be able to give yourself dynamics in Exercise 4.53). The chalk's state is comprised of its height and whether or not it is in your possession, so we give a dynamical system with state-set $C := \{\text{out}, \text{in}\} \times H$ and interface Chalk: that is, a lens

$$\{\text{out}, \text{in}\} \times Hy^{\{\text{out}, \text{in}\} \times H} \rightarrow \{\text{out}\}Hy^P + \{\text{in}\}Hy^{HP}. \quad (4.52)$$

On positions, the chalk returns its height and whether it is in your possession directly: the on-positions function is the identity. On directions, we have two cases. If the chalk is out of your possession, it falls down unless you catch it, making it pressed so that it becomes in your possession and retains its current height. So we can express the on-directions function of (4.52) at $(\text{out}, h_{\text{Chalk}})$ as

$$\begin{aligned} \text{unpressed} &\mapsto (\text{out}, \text{down}); \\ \text{pressed} &\mapsto (\text{in}, h_{\text{Chalk}}). \end{aligned}$$

On the other hand, if the chalk is in your possession, it takes whatever height it receives from you, remaining in your possession if pressed but coming out of your possession if unpressed. So the on-directions function of (4.52) at $(\text{in}, h_{\text{Chalk}})$ is given by

$$(h_{\text{You}}, \text{unpressed}) \mapsto (\text{out}, h_{\text{You}});$$

$$(h_{\text{You}}, \text{pressed}) \mapsto (\text{in}, h_{\text{You}}).$$

We have thus defined an interaction pattern that allows one system to engage with or disengage from another system and control the behavior of the other system only when the two are engaged.

Exercise 4.53.

1. In Example 4.51, we said that $\beta: HPy^H \otimes \{\text{in}\}Hy^{HP} \rightarrow y$ was easy to describe and given by a function $HPH \rightarrow HHP$. Explain what is being said, and provide the function.
2. Provide dynamics to the You interface (i.e. specify a dynamical system with interface $\text{You} = HPy^H$) so that you repeatedly reach down and grab the chalk, lift it with your hand, and drop it.

Given $n \in \mathbb{N}$ and polynomials $p_1, \dots, p_n$ as interfaces, a section $p_1 \otimes \dots \otimes p_n \rightarrow y$ sections off these n interfaces together. The following proposition provides an alternative perspective on such sections.

Proposition 4.54. Given polynomials $p, q \in \mathbf{Poly}$, there is a bijection

$$\Gamma(p \otimes q) \cong \mathbf{Set}(q(1), \Gamma(p)) \times \mathbf{Set}(p(1), \Gamma(q)). \quad (4.55)$$

The idea is that specifying a section for the interfaces p and q together is equivalent to specifying a section for p for every output q might return and specifying a section for q for every output p might return.

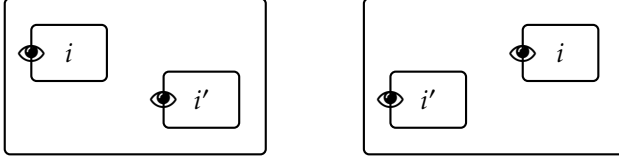
Proof of Proposition 4.54. This is a direct calculation:

$$\begin{aligned} \Gamma(p \otimes q) &\cong \prod_{i \in p(1)} \prod_{j \in q(1)} (p[i] \times q[j]) \\ &\cong \left(\prod_{j \in q(1)} \prod_{i \in p(1)} p[i] \right) \times \left(\prod_{i \in p(1)} \prod_{j \in q(1)} q[j] \right) \\ &\cong \mathbf{Set}(q(1), \Gamma(p)) \times \mathbf{Set}(p(1), \Gamma(q)). \end{aligned}$$

□

Example 4.56. A section $f: Iy^A \otimes I'y^{A'} \rightarrow y$ corresponds to a function $I \times I' \rightarrow A \times A'$. In other words, for every pair of positions $(i, i') \in I \times I'$, the section f specifies a pair of directions $(a, a') \in A \times A'$.

Let us think of the positions in I and I' as locations that two machines may occupy.



Then given a pair of locations $(i, i') \in I \times I'$, the interaction pattern f as a function $I \times I' \rightarrow A \times A'$ tells us the directions the machines observe, i.e. the ordered pair $(a, a') \in A \times A'$ comprised of what each machine receives. Equivalently, (4.55) says that the interaction pattern tells us what direction the first machine observes at each location when the second machine's location is fixed at i' , along with the direction the second machine observes at each location when the first machine's location is fixed at i .

Here we see that (4.55) provides two ways to interpret the interaction pattern between two interfaces in a closed system: either as a section around each interface parametrized by the other's position, or as a single section around them both.

Exercise 4.57. Let $p := q := \mathbb{N}y^{\mathbb{N}}$. We wish to specify a section around their juxtaposition.

1. Say we wanted to feed the position of q as a direction for p . What function $f: q(1) \rightarrow \Gamma(p)$ captures this behavior?
2. Say we wanted to feed the sum of the positions of p and q as a direction for q . What function $g: p(1) \rightarrow \Gamma(q)$ captures this behavior?
3. What section $\gamma: p \otimes q \rightarrow y$ does the pair of functions (f, g) correspond to via (4.55)?
4. Let dynamical systems $\varphi: \mathbb{N}y^{\mathbb{N}} \rightarrow p$ and $\psi: \mathbb{N}y^{\mathbb{N}} \rightarrow q$ both

be the identity on $\mathbb{N}_y^{\mathbb{N}}$. Suppose φ starts in the state $0 \in \mathbb{N}$ and ψ starts in the state $1 \in \mathbb{N}$. Describe the behavior of the system obtained by sectioning φ and ψ off together with γ , i.e. the system $(\varphi \otimes \psi) \circ \gamma$.

Exercise 4.58. We will use (4.55) to consider the interaction pattern γ between You and Chalk from Example 4.51 as a pair of functions $\text{You}(1) \rightarrow \Gamma(\text{Chalk})$ and $\text{Chalk}(1) \rightarrow \Gamma(\text{You})$.

1. How does the chalk's position specify a section for you? That is, describe the function $\text{Chalk}(1) \rightarrow \Gamma(\text{You})$.
2. How does your position specify a section for the chalk? That is, describe the function $\text{You}(1) \rightarrow \Gamma(\text{Chalk})$.

Exercise 4.59.

1. State and prove a generalization of (4.55) from Proposition 4.54 for n -many polynomials $p_1, \dots, p_n \in \mathbf{Poly}$.
2. Generalize the “idea” statement between Proposition 4.54 and its proof.

4.4.3 Wiring diagrams as interaction patterns

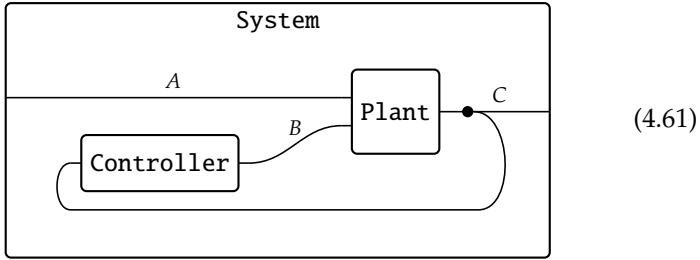
A *wiring diagram* is a graphical depiction of interactions between systems. Wiring diagrams depict systems as boxes, showing how they send signals to each other through the wires between them, as well as how multiple systems can combine to form a larger system whenever smaller boxes are nested within a larger box.

Formally, and more precisely, we can think of each box in a wiring diagram as an interface given by some monomial. The box itself is not a dynamical system, but it becomes a dynamical system once we equip it with a lens from a state system to the interface the box represents. Then the entire wiring diagram—specifying how these boxes nest within a larger box—is just an interaction pattern between the corresponding interfaces, with the larger box playing the role of the wrapper interface.

Once every nested box is equipped with a lens from a state system, we obtain a dynamical system whose interface is the larger box.

In the examples to come, we follow the convention that the signals emitted by a box, i.e. positions returned by the corresponding interface, travel along wires out of the right side of that box; while the signals received, i.e. directions observed by the corresponding interface, by a box travel along wires into the left side of that box. A wire may optionally be labeled by the name of the set of elements that may travel as signals along that wire.

Example 4.60. Here is a simple wiring diagram.



The Plant is receiving information from the world outside the System along the wire labeled A as well as from the Controller along the wire labeled B . It is also producing information for the outside world along the wire labeled C which is also being monitored by the Controller.

There are three boxes shown in (4.61): the Controller, the Plant, and the System. Each has a fixed set of positions corresponding to the wire(s) connected to its right and a fixed set of directions corresponding to the wire(s) connected to its left, so we can consider each box as a monomial interface, as follows:

$$\text{Controller} := By^C \quad \text{Plant} := Cy^{AB} \quad \text{System} := Cy^A. \quad (4.62)$$

Note that in the case of the Plant, two wires labeled A and B enter the box from the left, so we take their cartesian product to be the direction-set of the Plant.

The wiring diagram itself is a wrapper

$$w: \text{Controller} \otimes \text{Plant} \rightarrow \text{System},$$

specifying an interaction pattern between the **Controller** and the **Plant** with the **System** as the wrapper interface. Concretely, w is a lens $BCy^{CAB} \rightarrow Cy^A$ that prescribes how wires feed positions to directions. As a lens between monomials, w consists of an on-positions function $BC \rightarrow C$ and an on-directions map $BCA \rightarrow CAB$.

The wiring diagram is a picture that tells us what on-positions function and on-directions map to use. In particular, the on-positions function sends positions of the inner interfaces to positions of the outer interface, so it is depicted by how the wires coming from the right sides of the inner boxes connect to the right side of the outer box. Given inner boxes that return positions in B and C , the outer box must return a position in C . Here the wire labeled B is not connected to the outer box, but the wire labeled C is, so the on-positions function $BC \rightarrow C$ sends $(b, c) \mapsto c$.

Meanwhile, the on-directions function sends positions of the inner interfaces and directions of the outer interface to directions of the inner interfaces, so it is depicted by how the wires coming from both the right sides of the inner boxes and the left side of the outer box connect to the left sides of the inner boxes. Given inner boxes that return positions in B and C and an outer box that receives directions in A , the inner boxes must receive directions in C and B . Again, we can read the on-directions map $BCA \rightarrow CAB$ off the wiring diagram: it sends $(b, c, a) \mapsto (c, a, b)$.

Note that neither the wiring diagram nor any of the boxes within it represent dynamical systems on their own. Rather, each box is a monomial that could be the interface of a dynamical system. When we assign to a box a dynamical system having that box as its interface, we say that we *give dynamics* to the box. So the entire wiring diagram is a wrapper that tells us how, once we give

dynamics for each inner box,

$$\varphi: Sy^S \rightarrow \text{Controller} \quad \text{and} \quad \psi: Ty^T \rightarrow \text{Plant},$$

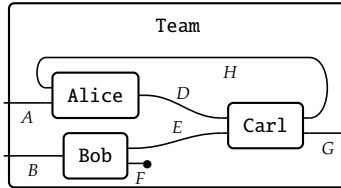
we have given dynamics for the entire outer box:

$$STy^{ST} \xrightarrow{\varphi \otimes \psi} \text{Controller} \otimes \text{Plant} \xrightarrow{w} \text{System}.$$

Exercise 4.63.

1. Make a new wiring diagram like (4.61) except where the controller also receives information from the outside world as an element of a set A' .
2. What are the monomials represented by the boxes in your diagram (replacing (4.62))?
3. What is the interaction pattern represented by this wiring diagram? Give the corresponding lens, including its on-positions and on-directions functions.

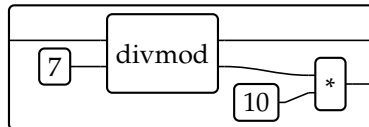
Exercise 4.64. Consider the following wiring diagram.



1. Write out the monomials for Alice, Bob, and Carl.
2. Write out the monomial for the outer box, Team.
3. The wiring diagram constitutes a lens f in **Poly**; what is its domain and codomain?
4. What lens is f ?
5. Say we have dynamical systems $\alpha: Ay^A \rightarrow \text{Alice}$, $\beta: By^B \rightarrow \text{Bob}$, and $\gamma: Cy^C \rightarrow \text{Carl}$. What is the induced dynamical system with interface Team?

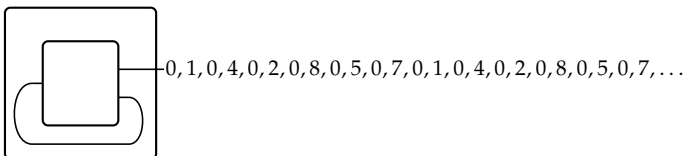
Exercise 4.65 (Long division).

1. Let $\text{divmod}: \mathbb{N} \times \mathbb{N}_{\geq 1} \rightarrow \mathbb{N} \times \mathbb{N}$ send $(a, b) \mapsto (a \text{ div } b, a \text{ mod } b)$; for example, it sends $(10, 7) \mapsto (1, 3)$ and $(30, 7) \mapsto (4, 2)$. Use Example 4.9 to turn it into a dynamical system.
2. In the following wiring diagram, we have already given dynamics to each box, as follows.



The dynamical system corresponding to the box divmod , with the box as its interface, is the dynamical system from the previous part (the upper wires correspond to the left-hand factors of the domain and codomain of the divmod function, while the lower wires correspond to the right-hand factors). Similarly, the box labeled $*$ corresponds to the dynamical system arising from the multiplication function $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ sending $(m, n) \mapsto mn$ following Example 4.9. Meanwhile the boxes labeled 7 and 10 correspond to dynamical systems with 1 state each that always return $7 \in \mathbb{N}_{\geq 1}$ and $10 \in \mathbb{N}$. Describe the behavior of the dynamical system corresponding to the entire outer box.

3. Using the outer box from the wiring diagram above as the inner box of the wiring diagram below, pick an initial state so that the resulting dynamical system alternates between returning 0's and the base-10 digits of $1/7$ after the decimal point, like so:



We will see in Section 7.1.5 how to make a dynamical system run twice as fast, then apply this to the above system in Example 7.13 so that it skips the 0's.

Example 4.66 (Graphs as wiring diagrams and cellular automata). Suppose we have a graph $G = (E \rightrightarrows V)$ as in Definition 4.25 and a set $\tau(v)$ associated with each vertex $v \in V$:

$$E \begin{array}{c} \xrightarrow{s} \\ \xleftarrow{t} \end{array} V \xrightarrow{\tau} \mathbf{Set}$$

We can think of G as an alternative representation of a specific kind of wiring diagram, one in which each inner box has exactly one position wire coming out and the outer box is closed (i.e. no wires in or out, representing the interface y with exactly 1 position and 1 direction). The vertices $v \in V$ are the inner boxes, the set $\tau(v)$ is the position-set associated with the wire coming out of v , and each edge e is a wire connecting the position wire of its target $t(e)$ to a direction wire of its source $s(e)$. An edge from a vertex v_0 to a vertex v_1 indicates that the directions received by v_0 depend on the positions returned by v_1 .³

In other words, we can associate each vertex $v \in V$ with the monomial

$$p_v := \tau(v) y^{\prod_{e \in E_v} \tau(t(e))}$$

specifying its positions and directions, where $E_v := s^{-1}(v) \subseteq E$ denotes the set of edges emanating from v . The graph then determines a section

$$\gamma: \bigotimes_{v \in V} p_v \rightarrow y$$

given by a function

$$\prod_{v \in V} \tau(v) \longrightarrow \prod_{e \in E} \tau(t(e))$$

that sends each dependent function $i: (v \in V) \rightarrow \tau(v)$ to the dependent function $(e \in E) \rightarrow \tau(t(e))$ sending $e \mapsto i(t(e))$. In other

words, given the p_v -position $i(v) \in \tau(v)$ returned by each vertex $v \in V$, we know for each edge $e \in E$ that the direction that its source vertex $s(e)$ receives is the position $i(t(e))$ returned by its target vertex $t(e)$.

So once we give dynamics to each p_v , namely by specifying a dynamical system $S_v y^{S_v} \rightarrow p_v$ with positions in $\tau(v)$ and directions in $\prod_{e \in E_v} \tau(t(e))$, we will obtain a closed dynamical system that updates the state of each vertex according to the information that they observe from each other along their edges.

Effectively, by interpreting a graph as a wiring diagram and giving each vertex dynamics, we have created what is known as a *cellular automaton*—a network of vertices (or *cells*), each with an internal state, in which each vertex $v \in V$ broadcasts a signal (i.e. returns a position) according to its current state, then updates its state according to the signals broadcasted by its *neighbors* in $t(E_v)$ (i.e. positions returned by its neighbors it receives as directions).

For example, many cellular automata have their cells on a 2-dimensional integer lattice. The corresponding graph has vertices $V := \mathbb{Z} \times \mathbb{Z}$ and edges given by

$$E := (\{-1, 0, 1\} \times \{-1, 0, 1\} - \{(0, 0)\}) \times V,$$

with $s((i, j), (m, n)) = (m, n)$ and $t((i, j), (m, n)) = (m + i, n + j)$, so that the neighbors of each vertex are the eight vertices that surround it.

Exercise 4.67 (Conway's Game of Life). Conway's Game of Life is a cellular automaton taking place on a 2-dimensional integer lattice as follows. Each lattice point is either *live* or *dead*, and each point observes its eight *neighbors* to which it is horizontally, vertically, or diagonally adjacent. The following occurs at every time step:

- Any live point with 2 or 3 live neighbors remains live.
- Any dead point with 3 live neighbors becomes live.

³We could swap the roles of the sources and targets, so that edges point in the direction of data flow rather than data dependencies; the choice is arbitrary.

- All other points either become or remain dead.

We can use Example 4.66 to model Conway's Game of Life as a closed dynamical system.

1. What is the appropriate graph $E \rightrightarrows V$?
2. What is the appropriate assignment of sets $\tau: V \rightarrow \mathbf{Set}$?
3. What are the monomials p_v from Example 4.66?
4. What is the appropriate state-set S_v for each interface p_v ?
5. What is the appropriate dynamical system lens $S_v y^{S_v} \rightarrow p_v$?

4.4.4 More examples of general interaction

While wiring diagrams are a handy visualization tool for certain simple interaction patterns, there are more general interaction patterns that cannot be captured by such a static diagram. For example, here we generalize our previous cellular automata example.

Example 4.68 (Generalized cellular automata: voting on who your neighbors are). Recall from Example 4.66 how we constructed a cellular automaton on a graph $G = (E \rightrightarrows V)$. For each $v \in V$, the graph specifies the set $N(v) := t(E_v)$ of target vertices of edges emanating from v . These vertices are the *neighbors* of v , or the vertices that v can “observe.” We call the function $N: V \rightarrow 2^V$ from each vertex to the set of its neighbors the *neighbor function*. For simplicity, we let each vertex store and return one of two states, so $S_v := \tau(v) := 2$.

Now consider only the vertices of our graph and forget the edges. Suppose we are then given a function $n: V \rightarrow \mathbb{N}$ that we can think of as specifying the number $n(v)$ of neighbors each $v \in V$ could potentially have. Let $\mathfrak{n}(v) := \{1, 2, \dots, n(v)\}$. Then the monomial each vertex represents is

$$p_v \cong 2y^{2^{n(v)}},$$

with its own possible states as positions and its potential neighbors' possible states as directions.

Say that a neighbor function $N: V \rightarrow 2^V$ respects n if we have

an isomorphism $N(v) \cong n(v)$ for each $v \in V$. Now suppose we have a function $N'_-: 2^V \rightarrow (2^V)^V$ that sends each set of vertices $S \in 2^V$ to a neighbor function $N'_S: V \rightarrow 2^V$ that respects n . In other words, each possible state configuration S of all the vertices in V determines a neighbor function N'_S . In the case of Example 4.66, when we had a graph, it told us what the neighbor function should always be. Now we could imagine all the vertices returning their states to vote, via N' , on what neighbor function to use to determine which vertices are observing which others.

We can put this all together by providing a section for all the vertices,

$$\bigotimes_{v \in V} p_v \cong 2^V y^{2^{\sum_{v \in V} n(v)}} \longrightarrow y. \quad (4.69)$$

Such a section is equivalent to a function $g: 2^V \rightarrow 2^{\sum_{v \in V} n(v)}$ that sends each possible state configuration $S \in 2^V$ of all the vertices in V to a function $g(S): \sum_{v \in V} n(v) \rightarrow 2$ specifying the states every vertex observes. But we already have a neighbor function assigned to S that respects n , namely N'_S : we have $N'_S(v) \cong n(v)$ for all $v \in V$. So we can think of $g(S)$ equivalently as a function $g(S): \sum_{v \in V} N'_S(v) \rightarrow 2$ that says for each $v \in V$ what signal in 2 it should receive from its neighbor $w \in N'_S(v)$. We will let it receive the current state of that neighbor, as given by S :

$$g(S)(v, w) := S(w).$$

We have accomplished our goal: the vertices “vote” on how they should be connected in that their states together determine the neighbor function. We do not mean to imply that this vote needs to be democratic or fair in any way: it is an arbitrary function $N'_-: 2^V \rightarrow (2^V)^V$. For instance, the state of a given vertex $v_0 \in V$ may completely determine the neighbor function $V \rightarrow 2^V$; this would be expressed by saying that N'_- factors as $2^V \rightarrow 2^{\{v_0\}} \cong 2 \rightarrow (2^V)^V$, where the left function is the evaluation map at $v_0 \in V$.

Here are some more examples of dependent dynamical systems with interaction patterns in which the way the constituent components are wired together may change.

Example 4.70. In the picture below, forces are applied to the connected boxes on the left; we would like to model how too much force could cause the connection between the boxes to sever, as depicted on the right.



We will imagine the dependent dynamical systems $\varphi_1: Sy^S \rightarrow p_1$ and $\varphi_2: Sy^S \rightarrow p_2$ as initially connected in space. They experience forces from the outside world, and—for as long as they are connected—they experience forces from each other. More precisely, their interfaces are given by

$$p_1 := p_2 := Fy^{F \times F} + \{\text{snapped}\}y^F,$$

where F is our set of *forces*. We need to be able to add and compare forces, so we need F to be an ordered monoid; let us say $F := \mathbb{N}$ for simplicity. Here each interface has two kinds of positions it can return: either a force $f \in F$ that will be applied to the other interface (i.e. sent to the other interface as a direction) or *snapped*, indicating that the interfaces are no longer connected. The interface always receives a force from the outside world as part of its direction, but when the position of the interface is not *snapped*, it receives a force from the other interface as part of its direction as well. So its direction set is $F \times F$ at positions in F , when the two interfaces are connected; and just F at the position *snapped*, when the two interfaces are not. We define the wrapper interface to be

$$p := y^{F \times F};$$

it can return only 1 position, while its directions are ordered pairs of forces (f_L, f_R) indicating the two external forces acting on the composite system.

Though the systems φ_1 and φ_2 may be initially connected, if the forces on either one surpass a threshold, that system stops communicating with the other system. The connection is broken

and neither system ever receives forces from the other again. To implement this explicitly, we need to define an interaction pattern $\kappa: p_1 \otimes p_2 \rightarrow p$ that wraps p around φ_1 and φ_2 . That is, we need to give a lens

$$\kappa: (Fy^{F \times F} + \{\text{snapped}\}y^F) \otimes (Fy^{F \times F} + \{\text{snapped}\}y^F) \rightarrow y^{F \times F}.$$

Distributing and leveraging coproducts as usual, we find that it suffices to give four lenses:

$$\begin{aligned} \kappa_{11}: F \times Fy^{(F \times F)(F \times F)} &\rightarrow y^{F \times F} \\ \kappa_{12}: F\{\text{snapped}\}y^{(F \times F)F} &\rightarrow y^{F \times F} \\ \kappa_{21}: \{\text{snapped}\}Fy^{F(F \times F)} &\rightarrow y^{F \times F} \\ \kappa_{22}: \{\text{snapped}\}\{\text{snapped}\}y^{F \times F} &\rightarrow y^{F \times F} \end{aligned} \quad (4.71)$$

The lenses κ_{12} and κ_{21} will not actually occur in our dynamics (when one interface returns `snapped`, both should), so we take them to be arbitrary. We take the lens κ_{22} to be the obvious isomorphism, passing the external forces to the two internal interfaces. Finally, the lens κ_{11} is equivalent to a function $(F \times F)(F \times F) \rightarrow (F \times F)(F \times F)$ which, taking care to remember what each F refers to, we find should send $((f_1, f_2), (f_L, f_R)) \mapsto ((f_L, f_2), (f_1, f_R))$. While the multiple F 's may be a little hard to keep track of, what this map says is that if φ_1 returns the force f_1 on φ_2 as output and φ_2 returns the force f_2 on φ_1 as output, then φ_1 receives the force f_2 from the right as input and φ_2 receives the force f_1 from the left as input; and in the meantime the left external force f_L is given to φ_1 on the left, while the right external force is given to φ_2 on the right.

Now that we have the interfaces wrapped together, it remains to specify each dynamical system. The state-sets for the two systems will be the same, namely $S := F + \{\text{snapped}\}$: each system is either applying a force to the other system or not. The dynamical systems themselves will be the same as well, up to a symmetry swapping left and right; we will define only the left system. It is given by a lens

$$\varphi_1: (F + \{\text{snapped}\})y^{F + \{\text{snapped}\}} \rightarrow Fy^{F \times F} + \{\text{snapped}\}y^F$$

which we write as the sum of two lenses

$$Fy^{F+\{\text{snapped}\}} \rightarrow Fy^{F \times F} \text{ and } \{\text{snapped}\}y^{F+\{\text{snapped}\}} \rightarrow \{\text{snapped}\}y^F.$$

Both lenses are identities on positions, directly returning their current states. The second lens corresponds to when the connection is broken, after which the connection should remain broken: so its on-directions function is constant, sending any direction to `snapped`. Meanwhile, the first lens corresponds to the case where the systems are still connected; in this state, the system can receive a pair of forces as its direction and must update its state—either the force it applies or `snapped`—accordingly. We let the on-directions function $F(F \times F) \rightarrow F + \{\text{snapped}\}$ send

$$(f_1, (f_L, f_2)) \mapsto \begin{cases} \text{snapped} & \text{if } f_1 + f_2 \geq 100; \\ f_L & \text{otherwise.} \end{cases}$$

Thus, when the sum of forces is above a certain threshold (arbitrarily chosen here to be 100), the internal state is updated to the `snapped` state; otherwise, the internal state is set to the external force applied to the system, which it is now ready to transfer to the other system.

Example 4.72. Consider the case of a company that may change its supplier based on its internal state. The company returns two possible positions, corresponding to whether it wants to receive gizmos in G from the first supplier or widgets in W from the second:



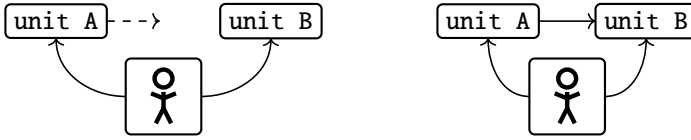
So the company has interface $\{1\}y^G + \{2\}y^W$, the first supplier has interface Gy , and the second supplier has interface Wy . Then a

section for the company and the suppliers is a lens

$$\left(\{1\}y^G + \{2\}y^W \right) \otimes Gy \otimes Wy \rightarrow y,$$

corresponding to a pair of functions $\{1\} \times GW \cong GW \rightarrow G$ and $\{2\} \times GW \cong GW \rightarrow W$ given by canonical projections. In other words, the company's position determines its supplier and what it receives.

Example 4.73. When someone assembles a machine, their own positions dictate the interaction pattern of the machine's components.



Define $S := \{\text{attach}, \text{separate}\}$. We say that unit A has interface

$$(\{\text{attached}\} \times X + \{\text{separated}\})y^S.$$

It can always receive either the direction to attach or to separate from S , while its position is either separated or attached and returning some element of a fixed set X . Meanwhile, unit B has interface

$$\{\text{attached}\}y^{X \times S} + \{\text{separated}\}y^S.$$

It can also always receive either direction from S , but when it is attached it can further receive an element of X . Finally, the role of the person is simply to return whether the units should attach or separate, so we give it the interface Sy .

Then a section for the person and the units is a lens

$$\begin{aligned} & (\{\text{attached}\} \times X + \{\text{separated}\})y^S \\ & \otimes \left(\{\text{attached}\}y^{X \times S} + \{\text{separated}\}y^S \right) \otimes Sy \rightarrow y. \end{aligned}$$

Such a lens corresponds to four functions, two of which can be arbitrary because our dynamics should never return them (either both units are attached or both are separated). The other two functions consist of one function

$$\{(\text{attached}, \text{attached})\} \times X \times S \cong X \times S \rightarrow S \times X \times S$$

that sends $(x, s) \mapsto (s, x, s)$ and another function

$$\{(\text{separated}, \text{separated})\} \times S \cong S \rightarrow S \times S$$

that sends $s \mapsto (s, s)$.

In words: the person's position tells the units whether they should attach or separate. If, and only if, the units are attached, one unit sends elements of X to the other.

We can easily generalize Example 4.73. Indeed, we will see in the next section that there is an interface $[q_1 \otimes \cdots \otimes q_k, r]$ that represents all the interaction patterns between $q_1, \dots, q_k$ with wrapper interface r , and that wrapping it around p can be interpreted as a larger interaction pattern with wrapper interface r :

$$\mathbf{Poly}(p, [q_1 \otimes \cdots \otimes q_k, r]) \cong \mathbf{Poly}(p \otimes q_1 \otimes \cdots \otimes q_k, r).$$

In other words, if the positions p returns is deciding the interaction pattern between $q_1, \dots, q_k$ with wrapper interface r , and the directions p receives is from the subsequent behavior of that interaction pattern itself, then this is equivalent to an interaction pattern with wrapper interface r that p is part of alongside $q_1, \dots, q_k$.

What it also means is that a dynamical system with interface $[q_1 \otimes \cdots \otimes q_k, r]$ is simply selecting interaction patterns $q_1 \otimes \cdots \otimes q_k \rightarrow r$. Let us see how this works.

4.5 Closure of $\otimes$

The parallel monoidal product is closed—we have a closed monoidal structure on $\mathbf{Poly}$ —meaning that there is a closure operation, which we denote $[-, -]: \mathbf{Poly}^{\text{op}} \times \mathbf{Poly} \rightarrow \mathbf{Poly}$, such that there is an isomorphism

$$\mathbf{Poly}(p \otimes q, r) \cong \mathbf{Poly}(p, [q, r]) \quad (4.74)$$

natural in p, q, r . The closure operation is defined on q, r as follows:

$$[q, r] := \prod_{j \in q(1)} r \circ (q[j]y). \quad (4.75)$$

Here $\circ$ denotes standard functor composition; informally, $r \circ (q[j]y)$ is the polynomial obtained by replacing each appearance of y in r by $q[j]y$. Composition, together with the unit y , is in fact yet another monoidal structure, as we will cover in greater depth in Part II.

Before we prove that the isomorphism (4.74) holds naturally, let us investigate the properties of the closure operation, starting with some simple examples.

Exercise 4.76. Calculate $[q, r]$ for $q, r \in \mathbf{Poly}$ given as follows.

1. $q := 0$ and r arbitrary.
2. $q := 1$ and r arbitrary.
3. $q := y$ and r arbitrary.
4. $q := A$ for $A \in \mathbf{Set}$ (constant) and r arbitrary.
5. $q := Ay$ for $A \in \mathbf{Set}$ (linear) and r arbitrary.
6. $q := y^2 + 2y$ and $r := 2y^3 + 3$.

Exercise 4.77. Show that for any polynomials p_1, p_2, q , we have an isomorphism

$$[p_1 + p_2, q] \cong [p_1, q] \times [p_2, q].$$

Exercise 4.78. Show that there is an isomorphism

$$[q, r] \cong \sum_{f: q \rightarrow r} y^{\sum_{j \in q(1)} r[f_1 j]} \quad (4.79)$$

where the sum is indexed over $f \in \mathbf{Poly}(q, r)$.

Exercise 4.80. Verify that (3.24) holds.

Example 4.81. By (4.75), for any $A \in \mathbf{Set}$ we have

$$[y^A, y] \cong Ay \quad \text{and} \quad [Ay, y] \cong y^A. \quad (4.82)$$

More generally, for any polynomial $p \in \mathbf{Poly}$ we have

$$[p, y] \cong \Gamma(p)y^{p(1)}. \quad (4.83)$$

Exercise 4.84. Verify the isomorphisms in (4.82) and (4.83).

Exercise 4.85. Show that for any $p \in \mathbf{Poly}$, if there is an isomorphism $[[p, y], y] \cong p$, then p is either linear Ay or representable y^A for some A . (Hint: first show that p must be a monomial.)

Proposition 4.86. With $[-, -]$ as defined in (4.75), there is a natural isomorphism

$$\mathbf{Poly}(p \otimes q, r) \cong \mathbf{Poly}(p, [q, r]). \quad (4.87)$$

Proof. We have the following chain of natural isomorphisms:

$$\begin{aligned} \mathbf{Poly}(p \otimes q, r) &\cong \mathbf{Poly}\left(\sum_{i \in p(1)} \sum_{j \in q(1)} y^{p[i]q[j]}, r\right) \\ &\cong \prod_{i \in p(1)} \prod_{j \in q(1)} \mathbf{Poly}(y^{p[i]q[j]}, r) \\ &\quad \text{(Universal property of coproducts)} \\ &\cong \prod_{i \in p(1)} \prod_{j \in q(1)} r(p[i]q[j]) \quad \text{(Yoneda lemma)} \\ &\cong \prod_{i \in p(1)} \prod_{j \in q(1)} \mathbf{Poly}(y^{p[i]}, r \circ (q[j]y)) \quad \text{(Yoneda lemma)} \\ &\cong \mathbf{Poly}\left(\sum_{i \in p(1)} y^{p[i]}, \prod_{j \in q(1)} r \circ (q[j]y)\right) \\ &\quad \text{(Universal property of (co)products)} \\ &\cong \mathbf{Poly}(p, [q, r]). \end{aligned}$$

□

Exercise 4.88. Show that for any p, q we have an isomorphism of sets

$$\mathbf{Poly}(p, q) \cong [p, q](1).$$

(Hint: you can either use the formula (4.75), or just use (4.87) with the Yoneda lemma and the fact that $y \otimes p \cong p$.)

The closure of $\otimes$ implies that for any $q, r \in \mathbf{Poly}$, there is a canonical *evaluation* lens

$$\text{eval}: [q, r] \otimes q \rightarrow r \quad (4.89)$$

given by sending the identity lens on $[q, r]$ leftward through the natural isomorphism

$$\mathbf{Poly}([q, r] \otimes q, r) \cong \mathbf{Poly}([q, r], [q, r]).$$

As in any closed monoidal category, such an evaluation lens has the universal property that for any $p \in \mathbf{Poly}$ and lens $f: p \otimes q \rightarrow r$, there is a unique lens $f': p \rightarrow [q, r]$ such that the following diagram commutes:

$$\begin{array}{ccc} p \otimes q & \xrightarrow{f' \otimes q} & [q, r] \otimes q \xrightarrow{\text{eval}} r \\ & \searrow f & \nearrow \end{array}$$

Exercise 4.90. Describe the behavior on positions and on directions of the evaluation lens $\text{eval}: [p, q] \otimes p \rightarrow q$ from (4.89).

Exercise 4.91.

1. For any set S , obtain the do-nothing section $Sy^S \rightarrow y$ from Example 4.43 whose on-directions is the identity on S using eval and Example 4.81.
2. Show that four lenses in (4.71) from Example 4.70, written

equivalently as

$$\begin{aligned}
 \kappa_{11}: Fy^{FF} \otimes Fy^{FF} &\rightarrow y^F \otimes y^F; \\
 \kappa_{12}: Fy^{FF} \otimes y^F &\rightarrow y^F \otimes y^F; \\
 \kappa_{21}: y^F \otimes Fy^{FF} &\rightarrow y^F \otimes y^F; \\
 \kappa_{22}: y^F \otimes y^F &\rightarrow y^F \otimes y^F,
 \end{aligned} \tag{4.92}$$

can be obtained by taking the parallel product of identity lenses and evaluation lenses.

Example 4.93 (Modeling your environment without knowing what it is). Imagine a robot whose interface is an arbitrary polynomial q that is part of an interaction pattern $f: p \otimes q \rightarrow r$ alongside its environment with interface p . Then f induces a lens $f': p \rightarrow [q, r]$ such that the following diagram commutes:

$$\begin{array}{ccc}
 p \otimes q & \xrightarrow{f' \otimes q} & [q, r] \otimes q \xrightarrow{\text{eval}} r \\
 & \searrow f & \nearrow
 \end{array}$$

In other words, $[q, r]$ holds within it all of the possible ways q can interact with other systems when they are wrapped in r together. For example, in the case of $r := y$, note that $[q, y] \cong \prod_{i \in q(1)} q[i]y$. That is, a position of $[q, y]$ sends each q -position to a direction at that position, which is exactly what we need to specify to put q in a closed system.

Now suppose we were to give dynamics to q by specifying a lens $Sy^S \rightarrow q$. One could aim to choose a set S along with an interesting map $g: S \rightarrow \mathbf{Poly}(q, r)$. Then each state s would include a guess $g(s)$ about the state of its environment. This is not the real environment p , but just the environment as it affects q , namely $[q, r]$. The robot's states model its environmental conditions.

Example 4.94 (Chu &). Say we have $p_1, p_2, q_1, q_2, r \in \mathbf{Poly}$ and lenses

$$\varphi_1: p_1 \otimes q_1 \rightarrow r \quad \text{and} \quad \varphi_2: p_2 \otimes q_2 \rightarrow r.$$

One might call these “ r -Chu spaces.” One operation you can do with these as Chu spaces is to return something denoted $\varphi_1 \& \varphi_2$, or “ φ_1 with φ_2 ,” of the following type:

$$\varphi_1 \& \varphi_2: (p_1 \times p_2) \otimes (q_1 + q_2) \rightarrow r.$$

Suppose we are given a position in p_1 and a position in p_2 . Then given a position in either q_1 or q_2 , one evaluates either φ_1 or φ_2 respectively to get a position in r ; given a direction there, one returns the corresponding direction in q_1 or q_2 respectively, as well as a direction in $p_1 \times p_2$ which is either a direction in p_1 or in p_2 .

This sounds complicated, but it can be constructed formally via the monoidal closure. We use the closure to rewrite φ_1 and φ_2 :

$$\psi_1: p_1 \rightarrow [q_1, r] \quad \text{and} \quad \psi_2: p_2 \rightarrow [q_2, r].$$

Now we take their categorical product to obtain $\psi_1 \times \psi_2: p_1 \times p_2 \rightarrow [q_1, r] \times [q_2, r]$. Then we apply Exercise 4.77 to find that $[q_1, r] \times [q_2, r] \cong [q_1 + q_2, r]$, and finally leverage the monoidal closure again to obtain $(p_1 \times p_2) \otimes (q_1 + q_2) \rightarrow r$ as desired.

4.6 Summary and further reading

In this chapter we explained how discrete dynamical systems can be expressed as certain lenses between polynomial functors. For example, a Moore machine has an input set A , an output set B , a set of states S , a return function $S \rightarrow B$, and an update function $A \times S \rightarrow S$. All this is captured in a dependent lens

$$Sy^S \rightarrow By^A.$$

We discussed a generalization $\varphi: Sy^S \rightarrow p$, where the output is an arbitrary polynomial $p \in \mathbf{Poly}$. This is similar to the notion of *interaction system* defined by Hyvénat in [Hyv09]. We also talked about how to

wire machines in parallel by using the parallel product $\otimes$ and how to add wrapper interfaces by composing with lenses $p \rightarrow q$.

Throughout the chapter we gave quite a few different examples. For example, we discussed how every function $A \rightarrow B$ counts as a memoryless dynamical system. In fact, it was shown in [BPS19] that every dynamical system can be obtained by wiring together memoryless ones. We discussed examples such as file-readers, moving robots, colliding particles, companies that change their suppliers, materials that break when too much force is applied, etc.

For further reading on the mathematics of Moore machines, see [Con12]. For more on mode-dependent interaction, see [ST17]. For a similar and complementary categorical approach to dynamical systems, we recommend David Jaz Myers' *Categorical Systems Theory* book, found at <http://davidjaz.com/Papers/DynamicalBook.pdf>.

4.7 Exercise solutions

Solution to Exercise 4.5.

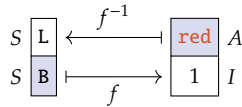
1. As $S = 3$, the state system is $Sy^S = 3y^3$.
2. As $I = \{0, 1\}$ and $A = \{\text{red}, \text{blue}\}$, the interface is $Iy^A = \{0, 1\}y^{\{\text{red}, \text{blue}\}}$.
3. The return function $S \rightarrow I$ sends $L \mapsto 0$, $R \mapsto 1$, and $B \mapsto 1$.
4. The update function $S \times A \rightarrow S$ sends

$$(L, \text{red}) \mapsto L, \quad (L, \text{blue}) \mapsto R,$$

$$(R, \text{red}) \mapsto L, \quad (R, \text{blue}) \mapsto B,$$

$$(B, \text{red}) \mapsto L, \quad (B, \text{blue}) \mapsto B.$$

5. In the first two steps, the machine sends its bottom state B to its position 1 via its return function, then sends its bottom state B and its direction red to its left state L via its update function. We can interpret this in terms of the lens φ and depict the steps in polyboxes as



Solution to Exercise 4.8.

At any time, the Moore machine in Example 4.7 is located at a point on the coordinate plane, say $(x, y) \in \mathbb{R}^2$. This location is its current state. When we ask the machine to return its position, it will tell us those coordinates, since the return function is the identity. Then if we give the machine a direction (r, θ) for some distance $r \in [0, 1]$ and

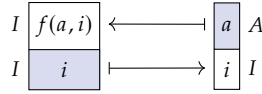
angle $\theta \in [0, 2\pi)$, the machine will move by that distance, at that angle counterclockwise from the positive x -axis, from (x, y) to

$$(x + r \cos \theta, y + r \sin \theta) = (x, y) + r(\cos \theta, \sin \theta)$$

(here we treat $\mathbb{R}^2$ as a vector space, so that $r(\cos \theta, \sin \theta)$ is a vector of length r at the angle θ).

Solution to Exercise 4.10.

1. We seek an (A, I) -Moore machine $Iy^I \rightarrow Iy^A$ corresponding to the function $f: A \times I \rightarrow I$. We know that an (A, I) -Moore machine $Iy^I \rightarrow Iy^A$ consists of a return function $I \rightarrow I$ and an update function $I \times A \rightarrow I$. So we can simply let the return function be the identity on I and the update function be $I \times A \cong A \times I \xrightarrow{f} I$, i.e. the function f with its inputs swapped. In polyboxes, the machine looks like



2. Generally, such a machine is not memoryless. Unlike in Example 4.9, the update function $I \times A \cong A \times I \xrightarrow{f} I$ does appear to depend on its first input, namely the previous state, which f takes as its second input. We can see this from our polybox picture above: the left direction box, which contains the new state $f(a, i)$, depends on the current state $i \in I$ in the left position box. However, if f factors through the projection $\pi_1: A \times I \rightarrow A$, i.e. if f can be written as a composite $A \times I \xrightarrow{\pi_1} A \xrightarrow{f'} I$ for some $f': A \rightarrow I$, then the resulting machine is memoryless: it is the memoryless Moore machine from Example 4.9 corresponding to f' .

Solution to Exercise 4.11.

For each of the following constructs, we find $A, I \in \mathbf{Set}$ such that the construct can be identified with a lens $\varphi: Sy^S \rightarrow Iy^A$, i.e. a return function $\varphi_1: S \rightarrow A$ and an update function $\varphi^\sharp: S \times A \rightarrow S$.

1. Given a discrete dynamical system with states S and transition function $n: S \rightarrow S$, we can set $A := I := 1$. Then $\varphi_1: S \rightarrow 1$ is unique, while $\varphi^\sharp: S \times 1 \rightarrow S$ is given by $S \times 1 \cong S \xrightarrow{n} S$. The corresponding Moore machine can only be fed one direction (you could think of that direction as a button that simply says “advance to the next state”) and can only return one position (which tells us no information). So it is just a set of states and a deterministic way to move from state to state. We could have also set $A := 0$ and $I := S$, so that $\varphi_1 := n$ and $\varphi^\sharp: S \times 0 \rightarrow S$ is unique, but this formulation is somewhat less satisfying: this is a Moore machine that never moves between its states, effectively functioning as a lookup table between whatever state the machine happens to be in and its position, which also happens to refer to some state.
2. Given a magma consisting of a set S and a function $m: S \times S \rightarrow S$, we can set $A := S$ and $I := 1$. Then $\varphi_1: S \rightarrow 1$ is unique, while $\varphi^\sharp: S \times S \rightarrow S$ is equal to m .

The corresponding Moore machine always returns the same position. It uses the binary operation m to combine the current state with a given direction—which also refers to a state—to obtain the new state.

Alternatively, we could have set the update function to be m with its inputs swapped. The difference here is that the new state is given by applying m with the direction on the left and the current state on the right, rather than the other way around. If m is noncommutative, this would yield a different Moore machine.

We could have also set $A := 0$ and $I := S^S$, so that $\varphi^\sharp: S \times 0 \rightarrow S$ is unique, while currying m gives φ_1 , so that $\varphi_1 s$ is the function $S \rightarrow S$ given by $s' \mapsto m(s, s')$. Alternatively, $\varphi_1 s$ could be the function $s' \mapsto m(s', s)$. Either way, this is again a Moore machine that never moves between its states, functioning as a lookup table between the machine's current state and the function m partially applied to that state on one side or the other.

3. Given a set S and a subset $S' \subseteq S$, we can set $A := 0$ and $I := 2$. Then $\varphi^\sharp: S \times 0 \rightarrow S$ is unique, while we define $\varphi_1: S \rightarrow 2$ by

$$\varphi_1 s = \begin{cases} 1 & \text{if } s \in S'; \\ 2 & \text{if } s \notin S'. \end{cases}$$

Then S' can be recovered from φ_1 as its fiber over 1. The corresponding Moore machine never moves between its states, but returns one of two positions indicating whether or not the current state is in the subset S' .

Solution to Exercise 4.12.

The original Moore machine had states $\mathbb{R}^2$, so to add a health meter that takes values in $[0, 1]$, we take the cartesian product to obtain a new set of states $\mathbb{R}^2 \times [0, 1]$. The position-set and direction-set are unchanged, so the Moore machine is a lens

$$\mathbb{R}^2 \times [0, 1] y^{\mathbb{R}^2 \times [0, 1]} \rightarrow \mathbb{R}^2 y^{[0, \infty) \times [0, 2\pi]}.$$

Its return function $\mathbb{R}^2 \times [0, 1] \rightarrow \mathbb{R}^2$ is the canonical projection, as the machine returns only its location in $\mathbb{R}^2$ and not its health; while its update function

$$\mathbb{R}^2 \times [0, 1] \times [0, \infty) \times [0, 2\pi) \rightarrow \mathbb{R}^2 \times [0, 1]$$

sends (x, y, h, r, θ) to

$$(x + hr \cos \theta, y + hr \sin \theta, h'),$$

where $h' = h/2$ if the machine's new x -coordinate $x + hr \cos \theta < 0$ and $h' = h$ otherwise.

As polyboxes, the lens is

$$\begin{array}{ccc} \mathbb{R}^2 \times [0, 1] & \boxed{(x + hr \cos \theta, y + hr \sin \theta, h')} & \longleftarrow \boxed{(r, \theta)} [0, \infty) \times [0, 2\pi) \\ \mathbb{R}^2 \times [0, 1] & \boxed{(x, y, h)} & \longrightarrow \boxed{(x, y)} \mathbb{R}^2 \end{array}$$

Solution to Exercise 4.13.

1. The tape has states $S := V^{\mathbb{Z}} \times \mathbb{Z}$, positions $I := V$, and directions $A := V \times \{\text{left}, \text{right}\}$; as a Moore machine, it is a lens

$$t: (V^{\mathbb{Z}} \times \mathbb{Z}) y^{V^{\mathbb{Z}} \times \mathbb{Z}} \rightarrow V y^{V \times \{\text{left}, \text{right}\}}.$$

2. The return function of t should give the value in the current cell of the tape. So $t_1: V^{\mathbb{Z}} \times \mathbb{Z} \rightarrow V$ is the evaluation map: it sends (f, c) with $f: \mathbb{Z} \rightarrow V$ and $c \in \mathbb{Z}$ to $f(c) \in V$. Then on a given direction $(v, d) \in V \times \{\text{left}, \text{right}\}$, the update function of t writes v in the tape's current cell before shifting the current cell number up or down by one according to whether d is right or left. More precisely,

$$t^{\sharp}: (V^{\mathbb{Z}} \times \mathbb{Z}) \times (V \times \{\text{left}, \text{right}\}) \rightarrow V^{\mathbb{Z}} \times \mathbb{Z}$$

sends current tape $f: \mathbb{Z} \rightarrow V$, current cell number $c \in \mathbb{Z}$, new value $v \in V$, and $d \in \{\text{left}, \text{right}\}$ to the new tape $f': \mathbb{Z} \rightarrow V$ defined by

$$f'(n) := \begin{cases} v & \text{if } n = c; \\ f(n) & \text{if } n \neq c; \end{cases}$$

and the new cell number $c - 1$ if $d = \text{left}$ and $c + 1$ if $d = \text{right}$.

Solution to Exercise 4.14.

There are many options for the machine's state-set; we choose to use pairs of entries $(i, t) \in n^2$, where i is the entry where the file-reader is currently located and t is the entry where the file-reader should stop. We will also include a stopped state for when the file-reader has already stopped. So our Moore machine is a lens

$$(n^2 + \{\text{stopped}\})y^{n^2 + \{\text{stopped}\}} \rightarrow (\text{ascii} + \{\text{done}\})y^{\{(s,t) \mid 1 \leq s \leq t \leq n\} + \{\text{continue}\}}.$$

If the file-reader's current state is **stopped**, then the file-reader should return the position "done." Otherwise, the file-reader should return the character at the entry where the file-reader is currently located. So its return function $n^2 + \{\text{stopped}\} \rightarrow \text{ascii} + \{\text{done}\}$ sends (i, t) to $f(i)$ and **stopped** to **done**. Meanwhile, the update function

$$(n^2 + \{\text{stopped}\}) \times (\{(s, t) \mid 1 \leq s \leq t \leq n\} + \{\text{continue}\}) \rightarrow n^2 + \{\text{stopped}\}$$

behaves as follows on the current state and given direction: regardless of the current state, if the given direction is a pair (s, t) , the new state will also be (s, t) . On the other hand, if the given direction is **continue** and the current state is a pair (i, t) , the new state should be the pair $(i + 1, t)$ if $i + 1 \leq t$ and **done** otherwise. Finally, if the given direction is **continue** and the current state is **stopped**, the new state should still be **stopped**.

Solution to Exercise 4.23.

1. The set of states is $S := \{1, 2, 3\} = 3$ (or equivalently $S := \{\blacksquare, \blacktriangle, \blacklozenge\} \cong 3$).
2. The set of input symbols is $A := \{\text{red}, \text{blue}\}$.
3. The automaton should halt at the accept states, so the accept states are exactly the states that have no arrows coming out of them—in this case, only state 3. States 1 and 2 are not accept states.
4. Let the corresponding lens be $\varphi: Sy^S \rightarrow y^A + 1$, or $\varphi: 3y^3 \rightarrow y^{\{\text{red}, \text{blue}\}} + 1$. According to the previous part, φ has a return function $\varphi_1: S \rightarrow 2$ sending states 1 and 2, as non-accept states, to 1; and sending state 3, as an accept state, to 2. Then $\varphi_3^{\sharp}$ is vacuous, while the other two update functions are given by the targets of the arrows in (4.24) as follows:

$$\varphi_1^{\sharp}(\text{red}) := 2, \varphi_1^{\sharp}(\text{blue}) := 1;$$

$$\varphi_2^\#(\text{red}) := 3, \varphi_2^\#(\text{blue}) := 1.$$

5. Some words accepted by this automaton include the word (**red**, **red**), the word (**red**, **blue**, **red**, **red**), and the word (**blue**, **red**, **blue**, **blue**, **blue**, **red**, **red**).
6. Some words are not accepted by the automaton because they lead you to a non-accept state (1 or 2); others are not accepted by the automaton because they lead you to an accept state (3) too early. Some examples of the former possibility include the words (**blue**, **blue**) and (**red**, **blue**, **red**, **blue**), while some examples of the latter possibility include the words (**blue**, **red**, **red**, **blue**) and (**red**, **red**, **red**, **red**, **red**, **red**, **red**).

Solution to Exercise 4.27.

No matter what graph you chose, Example 4.26 tells us that if you were to draw the labeled transition diagram of its associated dynamical system, you would just end up with a picture of your graph! The vertices of your graph are the states, and the edges of your graph are the possible transitions between them.

Solution to Exercise 4.29.

We give a file-searcher $\psi: Sy^S \rightarrow q$ according to the specification as follows. Its possible positions should form the set

$$q(1) := \{\text{ready}, \text{busy}\} \times \{100, _ \}.$$

The direction-sets of q can be defined in the same way we defined the direction-sets of p : for each $c \in \{100, _ \}$, we have

$$q[(\text{ready}, c)] := S \quad \text{and} \quad q[(\text{busy}, c)] := 1.$$

Then we set the return function ψ_1 to behave like φ_1 , but with characters not equal to 100 replaced with $_$: so for all $(s, t) \in S$,

$$\psi_1(s, t) = \begin{cases} (\text{ready}, 100) & \text{if } s = t \text{ and } f(s) = 100; \\ (\text{ready}, _) & \text{if } s = t \text{ and } f(s) \neq 100; \\ (\text{busy}, 100) & \text{if } s \neq t \text{ and } f(s) = 100; \\ (\text{busy}, _) & \text{otherwise.} \end{cases}$$

Then the update functions of ψ behave just like those of φ . For each $(s, t) \in S$ for which $s = t$, we define the update function $\psi_{(s, t)}^\#: S \rightarrow S$ to be the identity on S . On the other hand, for each $(s, t) \in S$ for which $s \neq t$, we let the update function $\psi_{(s, t)}^\#: 1 \rightarrow S$ specify the element $(s + 1, t) \in S$, thus shifting its current entry up by 1.

Solution to Exercise 4.31.

We modify the dynamical system from Example 4.30.

1. Previously, the direction-set at position $(i, j) \in n \times n$ of our interface was $D_i \times D_j$. But now we also want to give the robot a "reward value" $r \in \mathbb{R}$. So our new direction-set should be $D_i \times D_j \times \mathbb{R}$:

$$p := \sum_{(i, j) \in n \times n} y^{D_i \times D_j \times \mathbb{R}}.$$

2. Previously, a state was just a location in the grid: an element of $n \times n$. But now we want to be able to record a list of reward values as well. Since each reward value is a real number, we define the state-set to be $S := n \times n \times \text{List}(\mathbb{R})$.
3. The former return function φ_1 was the identity on $n \times n$. The new return function φ'_1 should still just return the robot's current grid position; but since it is now a function from $S = n \times n \times \text{List}(\mathbb{R})$, it should instead be the canonical projection $\varphi'_1: n \times n \times \text{List}(\mathbb{R}) \rightarrow n \times n$.

For each former state $(i, j) \in n \times n$, the former update function $\phi_{(i,j)}^\sharp: D_i \times D_j \rightarrow n \times n$ sent $(d, e) \mapsto (i + d, j + e)$. With an extra component $(r_1, \dots, r_k) \in \text{List}(\mathbb{R})$ of the state, the new update function $(\varphi')_{(i,j,(r_1,\dots,r_k))}^\sharp: D_i \times D_j \times \mathbb{R} \rightarrow n \times n \times \text{List}(\mathbb{R})$ sends $(d, e, r) \mapsto (i + d, j + e, (r_1, \dots, r_k, r))$, also updating the list of rewards. As polyboxes, the new dynamical system is given by

$$\begin{array}{ccc}
 n \times n \times \text{List}(\mathbb{R}) & \begin{array}{|c|} \hline (i + d, j + e, (r_1, \dots, r_k, r)) \\ \hline \end{array} & \longleftarrow \begin{array}{|c|} \hline (d, e, r) \\ \hline \end{array} D_i \times D_j \times \mathbb{R} \\
 n \times n \times \text{List}(\mathbb{R}) & \begin{array}{|c|} \hline (i, j, (r_1, \dots, r_k)) \\ \hline \end{array} & \longrightarrow \begin{array}{|c|} \hline (i, j) \\ \hline \end{array} n \times n
 \end{array}$$

Solution to Exercise 4.32.

We are given a lens $Sy^S \rightarrow B_i y^{A_i}$ for each $i \in I$. The universal property of products in **Poly** then induces a lens

$$Sy^S \rightarrow \prod_{i \in I} B_i y^{A_i}.$$

By (3.57), its codomain is the product of monomials

$$\prod_{i \in I} B_i y^{A_i} \cong \left(\prod_{i \in I} B_i \right) y^{\sum_{i \in I} A_i},$$

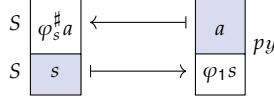
which, in particular, is still a monomial. Hence the induced lens is a $(\sum_{i \in I} A_i, \prod_{i \in I} B_i)$ -Moore machine with state-set S .

Solution to Exercise 4.34.

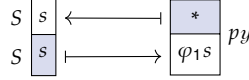
Given a dynamical system $\varphi: Sy^S \rightarrow p$, we seek a new dynamical system $\varphi': Sy^S \rightarrow p'$ with an additional direction that does not change the state. We can think of this as having two different interfaces acting on the same system: the original interface p of φ and a new interface with only one possible direction that does not change the state. This latter interface also need not distinguish between its positions; it should have a single position that provides no additional information. So the second interface we want acting on Sy^S is y .

If y were the only interface acting on the state system, we would have a Moore machine $\epsilon: Sy^S \rightarrow y$ whose return function is the unique function $S \rightarrow 1$ and whose update function should be the identity function on S , since the direction never changes the state. Then p' is the product of the two interfaces p and y , while $\varphi': Sy^S \rightarrow p'$ is the unique lens induced by $\varphi: Sy^S \rightarrow p$ and $\epsilon: Sy^S \rightarrow y$. In particular, $p' \cong py \cong \sum_{i \in p(1)} y^{p[i]+1}$, and if we let $*$ denote the additional direction not in p at each position of py , we can

write φ' in polyboxes as



when the chosen direction is from the original p (i.e. $a \in p[\varphi_1 s]$ above), coinciding with the behavior of the original system φ ; or as



when the chosen direction is the additional direction $*$ that does not change the state. It turns out this construction is universal; it is known as *copointing*.

Solution to Exercise 4.35.

We are given a lens $S_i y^{S_i} \rightarrow B_i y^{A_i}$ for each $i \in n$. By inductively applying Exercise 3.67, we find that their parallel product in **Poly** is a lens

$$\left(\prod_{i \in n} S_i \right) y^{\prod_{i \in n} S_i} \cong \bigotimes_{i \in n} S_i y^{S_i} \rightarrow \bigotimes_{i \in n} B_i y^{A_i} \cong \left(\prod_{i \in n} B_i \right) y^{\prod_{i \in n} A_i},$$

which is a $(\prod_{i \in n} A_i, \prod_{i \in n} B_i)$ -Moore machine with state-set $\prod_{i \in n} S_i$. This works because the parallel product of monomials is still a monomial.

Solution to Exercise 4.37.

We will show that taking the parallel product of the robot-on-a-grid dynamical system $\varphi: S y^S \rightarrow p$ from Example 4.30 and a reward-tracking dynamical system $\psi: T y^T \rightarrow q$ we will define shortly yields the dynamical system $\varphi': S' y^{S'} \rightarrow p'$ from Exercise 4.31. The reward-tracking dynamical system should have states in $\text{List}(\mathbb{R})$ to record a list of reward values, unchanging position, and directions in $\mathbb{R}$ to give new reward values. So it is the lens $\text{List}(\mathbb{R}) y^{\text{List}(\mathbb{R})} \rightarrow y^{\mathbb{R}}$ that has a uniquely defined return function, while its update map sends each state $(r_1, \dots, r_k) \in \text{List}(\mathbb{R})$ and each direction $r \in \mathbb{R}$ to the new state $(r_1, \dots, r_k, r)$.

Then the dynamical system from Exercise 4.31 is the parallel product of the robot-on-a-grid dynamical system from Example 4.30 with the reward-tracking dynamical system $\text{List}(\mathbb{R}) y^{\text{List}(\mathbb{R})} \rightarrow y^{\mathbb{R}}$, as can be seen in the solution to Exercise 4.31.

Solution to Exercise 4.38.

1. The robot-on-a-grid dynamical system from Example 4.30 can be written as the parallel product of two robot-on-a-line dynamical systems of the form $\lambda: n y^n \rightarrow \sum_{i \in n} y^{D_i}$, where $\lambda_1 := \text{id}_n$ and $\lambda_i^\#$ for each $i \in n$ sends each direction $d \in D_i$ to the position on the line given by $i + d$. This yields a robot that can move along a single axis, and the parallel product of this robot with itself yields a robot that can move along two different axes at once, which is precisely our robot-on-a-grid dynamical system.

2. To create a dynamical system consisting of a robot moving in a k -dimensional grid of size n along every dimension, we just take the k -fold parallel product of the dynamical system $\lambda: ny^n \rightarrow \sum_{i \in n} y^{D_i}$ we just defined to obtain a dynamical system

$$\lambda^{\otimes k}: n^k y^{n^k} \rightarrow \sum_{(i_1, \dots, i_k) \in n^k} y^{\prod_{j \in k} D_{i_j}}.$$

In fact, we could have used a different n_j for each $j \in k$ instead of n to obtain a robot moving in an arbitrary k -dimensional grid of size $n_1 \times \dots \times n_k$ as a k -fold parallel product.

Solution to Exercise 4.40.

We give a lens $f: p \rightarrow q$ for which composing the file-reader $\varphi: Sy^S \rightarrow p$ from Example 4.28 with f yields the file-searcher $\psi: Sy^S \rightarrow q$ from Exercise 4.29. The file-searcher returns the same position as the file-reader when the second coordinate of that position is 100, but replaces the second coordinate with a blank $_$ otherwise. So the on-positions function of f should send each $(m, c) \in p(1)$ to

$$f_1(m, c) := \begin{cases} (m, c) & \text{if } c = 100; \\ (m, _) & \text{otherwise.} \end{cases}$$

Then the file-searcher acts just like the file-reader does on inputs, so every on-directions function of f should be an identity function.

Solution to Exercise 4.42.

1. No, it represents a function $B \rightarrow A!$ A section sends each position $b \in B$ to a direction $a \in A$.
2. As a wrapper around an interface By^A , a section $\gamma: By^A \rightarrow y$ corresponds to a function $g: B \rightarrow A$ that feeds the direction $g(b) \in A$ into the system whenever it returns the position $b \in B$.
3. Composing our original Moore machine $Sy^S \rightarrow By^A$ with a section γ yields a Moore machine $Sy^S \xrightarrow{\varphi} By^A \xrightarrow{\gamma} y$ that returns unchanging output and receives unchanging input. If we identify the Moore machine with its return function $S \rightarrow B$ and its update function $S \times A \rightarrow S$, and if we identify the section γ with a function $g: B \rightarrow A$, then the composite Moore machine $Sy^S \xrightarrow{\varphi} By^A \xrightarrow{\gamma} y$ can be identified with a function $S \rightarrow S$, equal to the composite

$$S \xrightarrow{\Delta} S \times S \xrightarrow{\text{id}_S \times \text{return}} S \times B \xrightarrow{\text{id}_S \times g} S \times A \xrightarrow{\text{update}} S,$$

where Δ is the diagonal map $s \mapsto (s, s)$. This composite map $S \rightarrow S$ sends every state to the next according to the position the original state returns, the direction that the section gives in response to that position, and the update function that sends the original state and the selected direction to the new state.

Solution to Exercise 4.48.

We define a new tracker $T'y^{T'} \rightarrow \mathbb{N}y^2$ based on the one from Example 4.47 to watch for when the paddle switches sides once, at which point the tracker should increase its

location by one, and watch for when the paddle switches sides twice in a row, at which point the tracker should increase its location by two. To do this, we need the tracker to remember not just the current side the paddle is on, but the previous side the paddle was on as well. The tracker should still remember the current location. Thus the states of the tracker are given by $T := 2 \times 2 \times \mathbb{N}$, storing the previous side the paddle was on, the current side the paddle is on, and the current location. The on-positions function of the tracker is the canonical projection $2 \times 2 \times \mathbb{N} \rightarrow \mathbb{N}$ that returns the current location; then at each $(d, d', i) \in 2 \times 2 \times \mathbb{N}$, the on-directions function of the tracker $2 \rightarrow 2 \times 2 \times \mathbb{N}$ sends

$$d'' \mapsto \begin{cases} (d', d'', i) & \text{if } d' = d''; \\ (d', d'', i + 1) & \text{if } d' \neq d'' \text{ and } d = d'; \\ (d', d'', i + 2) & \text{if } d' \neq d'' \text{ and } d \neq d'; \end{cases}$$

storing both the last side the paddle was on and the new side the paddle is on as well as moving the machine forward one unit if the paddle switches after not switching and two units if the paddle switches after just switching.

Solution to Exercise 4.50.

1. An interaction pattern $p \otimes p' = AXy^{A'X} \otimes A'Xy^{AX} \rightarrow y$ consists of a trivial on-positions function $AX \times A'X \rightarrow 1$ and an on-directions map $AX \times A'X \rightarrow A'X \times AX$ indicating what directions the robots should receive according to the positions they return. To model the robots receiving each others' locations but only receiving each others' signals when the distance between their locations is less than 1, this on-directions function should send

$$((a, x), (a', x')) \mapsto \begin{cases} ((a', x'), (a, x)) & \text{if } d(x, x') < 1; \\ ((a'_0, x'), (a_0, x)) & \text{otherwise.} \end{cases}$$

2. An interaction pattern $p \otimes p' \rightarrow y^{[0,5]}$ that changes the distance threshold for the signal to $s \in [0, 5]$ consists of a still trivial on-positions function and an on-directions map $AX \times A'X \times [0, 5] \rightarrow A'X \times AX$ indicating what directions the robots should receive according to the external direction $s \in [0, 5]$ and the positions they return. To model the fact that the robots receive each others' locations, but only receive each others' signals when the distance between their locations is less than s , this on-directions function should send

$$((a, x), (a', x'), s) \mapsto \begin{cases} ((a', x'), (a, x)) & \text{if } d(x, x') < s; \\ ((a'_0, x'), (a_0, x)) & \text{otherwise.} \end{cases}$$

3. To provide a dynamical system $\varphi: SXy^{SX} \rightarrow AXy^{A'X}$ under the condition that the on-positions function preserves the second coordinate $x \in X$, we must provide the first projection $SX \rightarrow A$ of an on-positions function that turns the robot's private state and current location into the signal it returns, as well as an on-directions function $SX \times A'X \rightarrow SX$ that provides a new private state and location for the robot given its old private state, old location, and the signal and location it receives from the other robot.
4. To have the robots listen for each others' signals only when they are sufficiently close, we must move away from monomial interfaces and Moore machines to

leverage dependency. There are several ways of doing this; we give just one method below. With $D := \{\text{'close'}, \text{'far'}\}$, let the robots' new interfaces be

$$\begin{aligned} p &:= \{\text{'close'}\}AXy^{DA'X} + \{\text{'far'}\}AXy^{DX} \quad \text{and} \\ p' &:= \{\text{'close'}\}A'Xy^{DAX} + \{\text{'far'}\}A'Xy^{DX}, \end{aligned}$$

so that they may receive input telling them whether they are close or far, but cannot receive signals in A or A' when they are 'far'.

Then by the distributivity of $\otimes$ over $+$, their new interaction pattern $p \otimes p' \rightarrow y^{[0,5]}$ can be specified by four lenses, all trivial on positions: the lens

$$\{\text{'close'}\}AXy^{DA'X} \otimes \{\text{'close'}\}A'Xy^{DAX} \rightarrow y^{[0,5]},$$

given by the on-directions function

$$\begin{aligned} &((\text{'close'}, a, x), (\text{'close'}, a', x'), s) \\ &\mapsto \begin{cases} ((\text{'close'}, a', x'), (\text{'close'}, a, x)) & \text{if } d(x, x') < s; \\ ((\text{'far'}, a'_0, x'), (\text{'far'}, a_0, x)) & \text{otherwise;} \end{cases} \end{aligned}$$

the lens

$$\{\text{'far'}\}AXy^X \otimes \{\text{'far'}\}A'Xy^X \rightarrow y^{[0,5]},$$

given by the on-directions function

$$((\text{'far'}, a, x), (\text{'far'}, a', x'), s) \mapsto \begin{cases} ((\text{'close'}, x'), (\text{'close'}, x)) & \text{if } d(x, x') < s; \\ ((\text{'far'}, x'), (\text{'far'}, x)) & \text{otherwise;} \end{cases}$$

and two other lenses that can be defined arbitrarily, as they should never come up in practice.

Finally, in order for each robot to properly remember whether the other is close or far, we record an element of D in its state that is returned and updated: one robot is a lens

$$p: DSXy^{DSX} \rightarrow \{\text{'close'}\}AXy^{DA'X} + \{\text{'far'}\}AXy^{DX}$$

whose on-positions function preserves not just the third coordinate $x \in X$ but also the first coordinate $d \in D$, while the on-directions function also preserves the first coordinate $d \in D$; and the other robot is constructed similarly.

Solution to Exercise 4.53.

1. A lens $HPy^H \otimes \{\text{in}\}Hy^{HP} \rightarrow y$ is a section of $HPy^H \otimes \{\text{in}\}Hy^{HP} \cong HPHy^{HHP}$ and can thus be identified with a function from its position-set to its direction-set: $HPH \rightarrow HHP$. Here the first factor of the domain refers to your height, the second factor to your pressure, and the third factor to the chalk's height; while in the codomain, the first factor refers to the chalk's height that you receive, the second factor refers to your height that the chalk receives, and the third factor refers to your pressure that the chalk receives. We should therefore define the function $HPH \rightarrow HHP$ to be the isomorphism that sends $(h_{\text{You}}, p_{\text{You}}, h_{\text{Chalk}}) \mapsto (h_{\text{Chalk}}, h_{\text{You}}, p_{\text{You}})$.

2. If you always cycle through three possible actions—reaching down and grabbing the chalk so that it is pressed, moving your hand up while keeping it pressed to lift the chalk, and dropping the chalk by leaving it unpressed while your hand is still up—you only need 3 possible states. So we can provide your dynamics using a lens $3y^3 \rightarrow \text{You} = HPy^H$. The return function $3 \rightarrow HP$ indicates what happens at each state, sending $1 \mapsto (\text{down}, \text{pressed})$, $2 \mapsto (\text{up}, \text{pressed})$, and $3 \mapsto (\text{up}, \text{unpressed})$. Meanwhile the update function $3H \rightarrow 3$ always changes the state to the next one, regardless of direction: it ignores the H coordinate and sends $1 \mapsto 2$, $2 \mapsto 3$, and $3 \mapsto 1$.

Solution to Exercise 4.57.

1. The function $f: q(1) \rightarrow \Gamma(p)$ should send each q -position b to the section of p corresponding to the constant function $\mathbb{N} \rightarrow \mathbb{N}$ that sends every p -position to b itself. That is, f is the function $b \mapsto (_ \mapsto b)$.
2. The function $g: p(1) \rightarrow \Gamma(q)$ should send each position p -position a to the section of q corresponding to the function $\mathbb{N} \rightarrow \mathbb{N}$ that sends each q -position b to the sum $a + b$. That is, g is the function $a \mapsto (b \mapsto a + b)$.
3. Together, f and g form a function $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N} \times \mathbb{N}$ mapping $(a, b) \mapsto ((fb)a, (ga)b) = (b, a + b)$. Then $\gamma: p \otimes q \rightarrow y$ is the section with this function as its on-directions function.
4. As φ and ψ are both the identity, their parallel product is the identity as well, so $(\varphi \otimes \psi) \circ \gamma = \gamma$. From the previous part, if the current state of this system is (a, b) , its next state will be $(b, a + b)$. So if its initial state is $(0, 1)$, its following states will be $(1, 1), (1, 2), (2, 3), (3, 5), (5, 8), (8, 13), \dots$, forming the familiar Fibonacci sequence.

Solution to Exercise 4.58.

1. Fix a position $(s_{\text{Chalk}}, h_{\text{Chalk}}) \in \text{Chalk}(1) = \{\text{out}, \text{in}\}H$ of the chalk. If $s_{\text{Chalk}} = \text{out}$, then the corresponding section $\text{You} \cong HPy^H \rightarrow y$ given by f via (4.55) can be thought of as the function $HP \rightarrow H$ sending

$$(h_{\text{You}}, p_{\text{You}}) \mapsto h_{\text{Chalk}},$$

according to the behavior of $\alpha: HPy^H \otimes Hy^P \rightarrow y$ when we fix the position of the domain's right factor to be $h_{\text{Chalk}} \in H$ and focus on the direction in H of the domain's left factor. Meanwhile, if $s_{\text{Chalk}} = \text{in}$, then the corresponding section $\text{You} \cong HPy^H \rightarrow y$ can also be thought of as the function $HP \rightarrow H$ sending

$$(h_{\text{You}}, p_{\text{You}}) \mapsto h_{\text{Chalk}},$$

according to the behavior of $\beta: HPy^H \otimes Hy^{HP} \rightarrow y$ when we again fix the position of the domain's right factor to be $h_{\text{Chalk}} \in H$ and focus on the direction in H of the domain's left factor. So overall, the desired function $\text{Chalk}(1) \rightarrow \Gamma(\text{You})$ is given by

$$(_, h_{\text{Chalk}}) \mapsto ((_, _) \mapsto h_{\text{Chalk}}).$$

2. Fix a position $(h_{\text{You}}, p_{\text{You}}) \in \text{You}(1) = HP$ of the chalk. Then the corresponding section $\text{Chalk} \cong \{\text{out}\}Hy^P + \{\text{in}\}Hy^{HP} \rightarrow y$ can be thought of as a pair of

functions: one $\{\text{out}\}H \rightarrow P$ sending

$$(\text{out}, h_{\text{Chalk}}) \mapsto \begin{cases} \text{unpressed} & \text{if } h_{\text{You}} \neq h_{\text{Chalk}}; \\ p_{\text{You}} & \text{if } h_{\text{You}} = h_{\text{Chalk}}; \end{cases}$$

according to the behavior of α : $HPy^H \otimes Hy^P \rightarrow y$ when we fix the position of the domain's left factor to be $(h_{\text{You}}, p_{\text{You}}) \in HP$ and focus on the direction in P of the domain's right factor; and another $\{\text{in}\}H \rightarrow HP$ sending

$$(\text{in}, h_{\text{Chalk}}) \mapsto (h_{\text{You}}, p_{\text{You}})$$

according to the behavior of β : $HPy^H \otimes Hy^{HP} \rightarrow y$ when we again fix the position of the domain's left factor to be $(h_{\text{You}}, p_{\text{You}}) \in HP$ and focus on the direction in HP of the domain's right factor. So overall, the desired function $\text{You}(1) \rightarrow \Gamma(\text{Chalk})$ is given by

$$(h_{\text{You}}, p_{\text{You}}) \mapsto \left((\text{out}, h_{\text{Chalk}}) \mapsto \begin{cases} \text{unpressed} & \text{if } h_{\text{You}} \neq h_{\text{Chalk}} \\ p_{\text{You}} & \text{if } h_{\text{You}} = h_{\text{Chalk}} \end{cases}, (\text{in}, h_{\text{Chalk}}) \mapsto (h_{\text{You}}, p_{\text{You}}) \right).$$

Solution to Exercise 4.59.

1. We generalize (4.55) for n polynomials as follows. Given polynomials $p_1, \dots, p_n \in \mathbf{Poly}$, we claim there is a bijection

$$\Gamma \left(\bigotimes_{i=1}^n p_i \right) \cong \prod_{i=1}^n \mathbf{Set} \left(\prod_{\substack{1 \leq j \leq n, \\ j \neq i}} p_j(1), \Gamma(p_i) \right).$$

The $n = 1$ case is tautological, and the $n = 2$ case is given by (4.55). Then by induction on n , we have

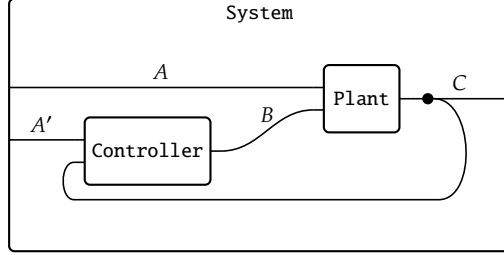
$$\begin{aligned} \Gamma \left(\bigotimes_{i=1}^n p_i \right) &\cong \mathbf{Set} \left(p_n(1), \Gamma \left(\bigotimes_{i=1}^{n-1} p_i \right) \right) \times \mathbf{Set} \left(\prod_{i=1}^{n-1} p_i(1), \Gamma(p_n) \right) \\ &\cong \mathbf{Set} \left(p_n(1), \prod_{i=1}^{n-1} \mathbf{Set} \left(\prod_{\substack{1 \leq j \leq n-1, \\ j \neq i}} p_j(1), \Gamma(p_i) \right) \right) \times \mathbf{Set} \left(\prod_{i=1}^{n-1} p_i(1), \Gamma(p_n) \right) \\ &\quad \text{(Inductive hypothesis)} \\ &\cong \prod_{i=1}^{n-1} \left(\mathbf{Set} \left(\prod_{\substack{1 \leq j \leq n, \\ j \neq i}} p_j(1), \Gamma(p_i) \right) \right) \times \mathbf{Set} \left(\prod_{i=1}^{n-1} p_i(1), \Gamma(p_n) \right), \\ &\quad \text{(Universal properties of products and internal homs)} \end{aligned} \tag{4.55}$$

and the result follows.

- The general idea is that specifying a section for interfaces $p_1, \dots, p_n$ together is equivalent to specifying a section for p_i for every combination of positions that all the other interfaces might return together, for each $i \in n$.

Solution to Exercise 4.63.

- Here is the wiring diagram (4.61) modified so that the controller also receives information from the outside world as an element of A' .



- The monomials represented by the boxes in this diagram are the same, except that the Controller and the System both have extra A' factors in their exponent:

$$\text{Controller} := By^{A'C} \quad \text{Plant} := Cy^{AB} \quad \text{System} := Cy^{AA'}.$$
- The interaction pattern represented by this wiring diagram is the lens

$$w': \text{Controller} \otimes \text{Plant} \rightarrow \text{System}$$

consisting of an on-positions function $BC \rightarrow C$ given by $(b, c) \mapsto c$ and an on-directions function $BCAA' \rightarrow A'CAB$ given by $(b, c, a, a') \mapsto (a', c, a, b)$.

Solution to Exercise 4.64.

- According to the wiring diagram, we have that $\text{Alice} := Dy^{HA}$, that $\text{Bob} := EFy^B$, and that $\text{Carl} := HGy^{DE}$.
- According to the wiring diagram, we have that $\text{Team} := Gy^{AB}$.
- The wiring diagram constitutes a wrapper

$$f: \text{Alice} \otimes \text{Bob} \otimes \text{Carl} \rightarrow \text{Team}.$$

Its domain is $\text{Alice} \otimes \text{Bob} \otimes \text{Carl} \cong DEFHGy^{HABDE}$, while its codomain is $\text{Team} = Gy^{AB}$.

- On positions, the lens f is a function $DEFHG \rightarrow G$ that sends $(d, e, f, h, g) \mapsto g$, while on directions, f is a function $DEFHGAB \rightarrow HABDE$ that sends $(d, e, f, h, g, a, b) \mapsto (h, a, b, d, e)$.
- Given dynamical systems $\alpha: Ay^A \rightarrow \text{Alice}$, $\beta: By^B \rightarrow \text{Bob}$, and $\gamma: Cy^C \rightarrow \text{Carl}$, the dynamical system induced by the wiring diagram is given by the composite lens

$$ABCy^{ABC} \xrightarrow{\alpha \otimes \beta \otimes \gamma} \text{Alice} \otimes \text{Bob} \otimes \text{Carl} \xrightarrow{f} \text{Team}.$$

Solution to Exercise 4.65.

- Using Example 4.9, we can turn `divmod` into the dynamical system $\text{divmod}: \mathbb{N} \times \mathbb{N}y^{\mathbb{N} \times \mathbb{N}} \rightarrow \mathbb{N} \times \mathbb{N}y^{\mathbb{N} \times \mathbb{N}_{\geq 1}}$ whose return function is the identity on $\mathbb{N} \times \mathbb{N}$ and whose update map $\mathbb{N} \times \mathbb{N} \times \mathbb{N} \times \mathbb{N}_{\geq 1} \rightarrow \mathbb{N} \times \mathbb{N}$ sends $(_, _, a, b) \mapsto (a \text{ div } b, a \text{ mod } b)$.

2. From left to right, the inner boxes represent monomial interfaces $\mathbb{N}_{\geq 1}y, \mathbb{N} \times \mathbb{N}y^{\mathbb{N} \times \mathbb{N}_{\geq 1}}, \mathbb{N}y$, and $\mathbb{N}y^{\mathbb{N} \times \mathbb{N}}$. The box labeled 7 is given dynamics $7: y \rightarrow \mathbb{N}_{\geq 1}y$ so that it always returns the position 7; similarly, the box labeled 10 is given dynamics $10: y \rightarrow \mathbb{N}y$ so that it always returns the position 10. Meanwhile, the box labeled divmod is given dynamics $\text{divmod}: \mathbb{N} \times \mathbb{N}y^{\mathbb{N} \times \mathbb{N}} \rightarrow \mathbb{N} \times \mathbb{N}y^{\mathbb{N} \times \mathbb{N}_{\geq 1}}$ from the previous part; and applying Exercise 4.10 to the multiplication function $*$: $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ yields the dynamics for the box labeled $*$: a dynamical system $*$: $\mathbb{N}y^{\mathbb{N}} \rightarrow \mathbb{N}y^{\mathbb{N} \times \mathbb{N}}$ whose return function is the identity on $\mathbb{N}$ and whose update map $\mathbb{N} \times \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ sends $(_, m, n) \mapsto mn$.

Then the outer box is the monomial interface $\mathbb{N} \times \mathbb{N}y^{\mathbb{N}}$, and the wiring diagram is the interaction pattern

$$w: \mathbb{N}_{\geq 1}y \otimes (\mathbb{N} \times \mathbb{N}y^{\mathbb{N} \times \mathbb{N}_{\geq 1}}) \otimes \mathbb{N}y \otimes \mathbb{N}y^{\mathbb{N} \times \mathbb{N}} \rightarrow \mathbb{N} \times \mathbb{N}y^{\mathbb{N}}$$

whose on-positions function sends $(s, q, r, t, p) \mapsto (q, p)$ and whose on-directions map sends $(s, q, r, t, p, a) \mapsto (a, s, r, t)$. So the dynamical system induced by the wiring diagram is the composite lens φ given by

$$\begin{array}{c} y \otimes (\mathbb{N} \times \mathbb{N}y^{\mathbb{N} \times \mathbb{N}}) \otimes y \otimes \mathbb{N}y^{\mathbb{N}} \\ \downarrow 7 \otimes \text{divmod} \otimes 10 \otimes * \\ \mathbb{N}_{\geq 1}y \otimes (\mathbb{N} \times \mathbb{N}y^{\mathbb{N} \times \mathbb{N}_{\geq 1}}) \otimes \mathbb{N}y \otimes \mathbb{N}y^{\mathbb{N} \times \mathbb{N}} \xrightarrow{w} \mathbb{N} \times \mathbb{N}y^{\mathbb{N}}, \end{array}$$

whose return function is given by the composite map $(q, r, p) \mapsto (7, q, r, 10, p) \mapsto (q, p)$ and whose update function at state (q, r, p) is given by the composite map $a \mapsto (a, 7, r, 10) \mapsto (a \text{ div } 7, a \text{ mod } 7, 10r)$.

In other words, the dynamical system φ behaves as follows: its state consists of a quotient q , a remainder r , and a product p , of which it returns the quotient and the product. Then it is fed a dividend a and evaluates $a \text{ div } 7$ to obtain the new quotient and $a \text{ mod } 7$ to obtain the new remainder. Meanwhile, the new product is given by the previous remainder multiplied by 10.

3. This second wiring diagram specifies an interaction pattern

$$w': \mathbb{N} \times \mathbb{N}y^{\mathbb{N}} \rightarrow \mathbb{N}y$$

with on-positions function $(q, p) \mapsto q$ and on-directions function $(q, p) \mapsto p$. So the dynamical system induced by nesting the first wiring diagram within the inner box of the second wiring diagram is the composite lens

$$(\mathbb{N} \times \mathbb{N}y^{\mathbb{N} \times \mathbb{N}}) \otimes \mathbb{N}y^{\mathbb{N}} \xrightarrow{\varphi} \mathbb{N} \times \mathbb{N}y^{\mathbb{N}} \xrightarrow{w'} \mathbb{N}y$$

whose return function is given by the composite map $(q, r, p) \mapsto (q, p) \mapsto q$ and whose update function at state (q, r, p) specifies the new state $(p \text{ div } 7, p \text{ mod } 7, 10r)$.

In other words, the dynamical system φ behaves as follows: its state consists of a quotient q , a remainder r , and a product p , of which it returns just the quotient. Then it advances to a new state by evaluating $p \text{ div } 7$ to obtain the new quotient and $p \text{ mod } 7$ to obtain the new remainder. Meanwhile, the new product is given by the previous remainder multiplied by 10.

If we set the initial state to be $(q, r, p) := (0, 0, 10)$, then the subsequent states will be as follows, with the values of q in the left column giving us the positions returned:

$q \ (p \text{ div } 7)$	$r \ (p \text{ mod } 7)$	$p \ (10r)$
0	0	10
1	3	0
0	0	30
4	2	0
0	0	20
2	6	0
0	0	60
8	4	0
0	0	40
5	5	0
0	0	50
7	1	0
0	0	10
$\vdots$	$\vdots$	$\vdots$

Solution to Exercise 4.67.

1. Following the suggestion from the end of Example 4.66, we can use a graph with $V := \mathbb{Z} \times \mathbb{Z}$ and $E := (\{-1, 0, 1\} \times \{-1, 0, 1\} - \{(0, 0)\}) \times V$ with $s((i, j), (m, n)) = (m, n)$ and $t((i, j), (m, n)) = (m+i, n+j)$ to model cellular automata like Conway's Game of Life on a 2-dimensional integer lattice in which each point observes only its eight immediate neighbors.
2. Each vertex only needs to return whether it is live or dead, so we assign $\tau(v) := \{\text{live}, \text{dead}\}$ for every $v \in V$.
3. For each $v \in V$, the monomial represented by v from Example 4.66 can be written as

$$p_v \cong \{\text{live}, \text{dead}\} y^{\text{Set}(\{-1, 0, 1\} \times \{-1, 0, 1\} - \{(0, 0)\}, \{\text{live}, \text{dead}\})}.$$

Every vertex returns either live or dead as its position and receives as its direction whether each of its eight neighbors is live or dead.

4. Each vertex $v \in V$ only needs to record whether it is live or dead, so $S_v := \{\text{live}, \text{dead}\}$.
5. The appropriate dynamical system lens $S_v y^{S_v} \rightarrow p_v$ for each vertex $v \in V$ should have the identity function on $\{\text{live}, \text{dead}\}$ as its return function, while its update map should be a function

$$\{\text{live}, \text{dead}\} \times \text{Set}(\{-1, 0, 1\} \times \{-1, 0, 1\} - \{(0, 0)\}, \{\text{live}, \text{dead}\}) \rightarrow \{\text{live}, \text{dead}\}$$

that takes whether v is live or dead as its first coordinate and a function from $\{-1, 0, 1\} \times \{-1, 0, 1\} - \{(0, 0)\}$ to $\{\text{live}, \text{dead}\}$ that says whether each of its eight neighbors is live or dead as a second coordinate, then executes the rules from Conway's Game of Life to determine whether it should be live or dead in the next time step.

Solution to Exercise 4.76.

We compute $[q, r]$ for various values of $q, r \in \mathbf{Poly}$ using (4.75).

1. If $q := 0$, then $q(1) \cong 0$, so $[q, r]$ is an empty product. Hence $[q, r] \cong 1$.
2. If $q := 1$, then $q(1) \cong 1$ and $q[1] \cong 0$, so $[q, r] \cong r \circ (0y) \cong r(0)$.
3. If $q := y$, then $q(1) \cong 1$ and $q[1] \cong 1$, so $[q, r] \cong r \circ (1y) \cong r$.
4. If $q := A$ for $A \in \mathbf{Set}$, then $q(1) \cong A$ and $q[j] \cong 0$ for every $j \in A$, so $[q, r] \cong \prod_{j \in A} (r \circ (0y)) \cong r(0)^A$.
5. If $q := Ay$ for $A \in \mathbf{Set}$, then $q(1) \cong A$ and $q[j] \cong 1$ for every $j \in A$, so $[q, r] \cong \prod_{j \in A} (r \circ (1y)) \cong r^A$.
6. If $q := y^2 + 2y$ and $r := 2y^3 + 3$, then

$$\begin{aligned} [q, r] &\cong (r \circ (2y))(r \circ (1y))^2 \\ &\cong (2(2y)^3 + 3)(2y^3 + 3)^2 \\ &\cong 64y^9 + 204y^6 + 180y^3 + 27. \end{aligned}$$

Solution to Exercise 4.77.

We wish to show that for all $p_1, p_2, q \in \mathbf{Poly}$, we have $[p_1 + p_2, q] \cong [p_1, q] \times [p_2, q]$. By (4.75),

$$[p_1 + p_2, q] \cong \left(\prod_{i \in p_1(1)} q \circ (p_1[i]y) \right) \left(\prod_{i \in p_2(1)} q \circ (p_2[i]y) \right) \cong [p_1, q] \times [p_2, q].$$

Solution to Exercise 4.78.

We may compute

$$[q, r] \cong \prod_{j \in q(1)} r \circ (q[j]y) \tag{4.75}$$

$$\cong \prod_{j \in q(1)} \sum_{k \in r(1)} (q[j]y)^{r[k]} \quad (\text{Replacing each } y \text{ in } r \text{ by } q[j]y)$$

$$\cong \sum_{f_1: q(1) \rightarrow r(1)} \prod_{j \in q(1)} (q[j]y)^{r[f_1(j)]} \tag{1.30}$$

$$\cong \sum_{f_1: q(1) \rightarrow r(1)} \left(\prod_{j \in q(1)} q[j]^{r[f_1(j)]} \right) \left(\prod_{j \in q(1)} y^{r[f_1(j)]} \right)$$

$$\begin{aligned} &\cong \sum_{f_1: q(1) \rightarrow r(1)} \sum_{f^\# \in \prod_{j \in q(1)} q[j]^{r[f_1(j)]}} y^{\sum_{j \in q(1)} r[f_1(j)]} \\ &\cong \sum_{f: q \rightarrow r} y^{\sum_{j \in q(1)} r[f_1(j)]}. \end{aligned} \tag{3.7}$$

Solution to Exercise 4.80.

We verify (3.24) as follows:

$$[p, y] \otimes p \cong \left(\sum_{f: p \rightarrow y} y^{\sum_{i \in p(1)} y[f_i i]} \right) \otimes p \tag{4.79}$$

$$\begin{aligned}
&\cong \sum_{f \in \Gamma(p)} y^{p(1)} \otimes \sum_{i \in p(1)} y^{p[i]} \\
&\cong \sum_{f \in \Gamma(p)} \sum_{i \in p(1)} y^{p(1) \times p[i]} \quad (3.66)
\end{aligned}$$

$$\cong \sum_{f \in \prod_{i \in p(1)} p[i]} \sum_{i \in p(1)} y^{p(1) \times p[i]}. \quad (3.35)$$

Solution to Exercise 4.84.

We have that

$$[y^A, y] \cong \prod_{j \in y^A(1)} y \circ (y^A[j]y) \cong \prod_{j \in 1} Ay \cong Ay,$$

that

$$[Ay, y] \cong \prod_{j \in Ay(1)} y \circ ((Ay)[j]y) \cong \prod_{j \in A} y \cong y^A,$$

and that

$$\begin{aligned}
[p, y] &\cong \sum_{f: p \rightarrow y} y^{\sum_{i \in p(1)} y[f_1 i]} \quad (4.79) \\
&\cong \sum_{f \in \Gamma(p)} y^{\sum_{i \in p(1)} 1} \\
&\cong \Gamma(p) y^{p(1)}.
\end{aligned}$$

Solution to Exercise 4.85.

Given $p \in \mathbf{Poly}$ and an isomorphism $[[p, y], y] \cong p$, we wish to show that p is either linear or representable. Applying (4.83) twice, we have that

$$[[p, y], y] \cong \Gamma \left(\Gamma(p) y^{p(1)} \right) y^{\Gamma(p)}.$$

By (3.35),

$$\Gamma \left(\Gamma(p) y^{p(1)} \right) \cong \prod_{\gamma \in \Gamma(p)} p(1) \cong p(1)^{\Gamma(p)}.$$

Hence taking $[[p, y], y] \cong p$ and rewriting the left-hand side using the isomorphisms above yields

$$p(1)^{\Gamma(p)} y^{\Gamma(p)} \cong p. \quad (4.95)$$

In particular, p is a monomial, so we can write $p := Iy^A$ for some $I, A \in \mathbf{Set}$. Then $p(1) \cong I$ and (3.35) tells us that $\Gamma(p) \cong A^I$. Equating the direction-sets on either side of (4.95) yields $A^I \cong A$; then equating position-sets gives $I^A \cong I^{A^I} \cong I$.

We conclude with some elementary set theory. If either one of I or A were (isomorphic to) 1, then p would be either linear or representable, and we would be done. Meanwhile, if either one of I or A were 0, then the other would be 1, and we would again be done. Otherwise, $|A|, |B| \geq 2$. But by Cantor's theorem,

$$|I| < |2^I| \leq |A^I| = |A| \quad \text{and} \quad |A| < |2^A| \leq |I^A| = |I|,$$

a contradiction.

Solution to Exercise 4.88.

The isomorphism $\mathbf{Poly}(p, q) \cong [p, q](1)$ follows directly from Exercise 4.78 when both sides are applied to 1. Alternatively, we can apply (4.87). Since $p \cong y \otimes p$, we have that

$$\begin{aligned} \mathbf{Poly}(p, q) &\cong \mathbf{Poly}(y \otimes p, q) \\ &\cong \mathbf{Poly}(y, [p, q]) \\ &\cong [p, q](1). \end{aligned} \tag{4.87} \quad (\text{Yoneda lemma})$$

Solution to Exercise 4.90.

To obtain the evaluation lens $\text{eval}: [q, r] \otimes q \rightarrow r$, we need to send the identity lens on $[q, r]$ leftward through the natural isomorphism

$$\mathbf{Poly}([q, r] \otimes q, r) \cong \mathbf{Poly}([q, r], [q, r]).$$

To do so, we can start from the identity lens on $[q, r]$ and work our way along a chain of natural isomorphisms from $\mathbf{Poly}([q, r], [q, r])$ until we get to $\mathbf{Poly}([q, r] \otimes q, r)$. To start, Exercise 4.78 implies that

$$\begin{aligned} \mathbf{Poly}([q, r], [q, r]) &\cong \mathbf{Poly}\left(\sum_{f: q \rightarrow r} \prod_{i' \in q(1)} y^{r[f_1 i']}, \prod_{i \in q(1)} \sum_{j \in r(1)} (q[i]y)^{r[j]}\right) \\ &\cong \prod_{f: q \rightarrow r} \prod_{i \in q(1)} \mathbf{Poly}\left(\prod_{i' \in q(1)} y^{r[f_1 i']}, \sum_{j \in r(1)} (q[i]y)^{r[j]}\right), \end{aligned}$$

where the second isomorphism follows from the universal properties of products and coproducts. In particular, under this isomorphism, the identity lens on $[q, r]$ corresponds to a collection of lenses, namely for each $f: q \rightarrow r$ and each $i \in q(1)$ the composite

$$\prod_{i' \in q(1)} y^{r[f_1 i']} \rightarrow y^{r[f_1 i]} \rightarrow \sum_{g: r[f_1 i] \rightarrow q[i]} y^{r[f_1 i]} \cong (q[i]y)^{r[f_1 i]} \rightarrow \sum_{j \in r(1)} (q[i]y)^{r[j]}$$

of the canonical projection with index $i' := i$, the canonical inclusion with index $g := f_i^\sharp$, and the canonical inclusion with index $j := f_1 i$. On positions, this lens picks out the position of $\sum_{j \in r(1)} (q[i]y)^{r[j]}$ corresponding to $j = f_1 i \in r(1)$ and $f_i^\sharp: r[f_1 i] \rightarrow q[i]$; on directions, the lens is the canonical inclusion $r[f_1 i] \rightarrow \sum_{i' \in q(1)} r[f_1 i']$ with index $i' = i$. By the Yoneda lemma, we can reinterpret each of these lenses as a lens

$$y^{q[i] \times \sum_{i' \in q(1)} r[f_1 i']} \rightarrow \sum_{j \in r(1)} y^{r[j]} \cong r$$

that, on positions, picks out the position $f_1 i \in r(1)$ of r and, on directions, is the map $r[f_1 i] \rightarrow q[i] \times \sum_{i' \in q(1)} r[f_1 i']$ induced by the universal property of products applied to the map $f_i^\sharp: r[f_1 i] \rightarrow q[i]$ and the inclusion $r[f_1 i] \rightarrow \sum_{i' \in q(1)} r[f_1 i']$. Then by the universal property of coproducts, this collection of lenses induces a single lens $\text{eval}: [q, r] \otimes q \rightarrow r$ that sends each position $f: q \rightarrow r$ of $[q, r]$ and position $i \in q(1)$ of q to the position $f_1 i$ of r , with the same behavior on directions as the corresponding lens described previously.

Solution to Exercise 4.91.

1. Given a set S , Example 4.81 shows that

$$[Sy, y] \otimes (Sy) \cong y^S \otimes (Sy) \cong Sy^S,$$

so by setting $q := Sy$ and $r := y$ in (4.89), we obtain an evaluation lens $\text{eval}: Sy^S \rightarrow y$. By the solution to Exercise 4.90, given a position $s \in S$ of Sy^S , the evaluation lens on directions is the map $1 \rightarrow S$ that picks out s . In other words, it is indeed the identity on directions.

2. We wish to write the four lenses in (4.92) from Example 4.70 as the parallel product of identity lenses and evaluation lenses. By the solution to Exercise 4.90, the evaluation lens $[Fy, y^F] \otimes Fy \rightarrow y^F$ is a lens from

$$[Fy, y^F] \otimes Fy \cong F \left(\sum_{f: Fy \rightarrow y^F} \prod_{i \in F} y^F \right) \cong Fy^{FF}$$

to y^F that is uniquely determined on positions and has the on-directions map $FF \rightarrow FF$ given by the identity. Then we can verify that κ_{11} is equivalent to the parallel product of this evaluation lens with itself. We can define κ_{12} and κ_{21} to be the parallel product of this evaluation lens with the identity on y^F , while κ_{22} is the parallel product of the identity on y^F with itself.

5. More categorical properties of polynomials

The category **Poly** has very useful formal properties, including many adjunctions with **Set**, factorization systems, cartesian closure, completion under colimits and limits, and so on. We detail some of these properties here. Most of the following material is not necessary for the development of our main story, but we collect it here for reference. The reader may skip directly to Part II if so inclined and check back here when needed; or they may skim the chapter to see how well-behaved and versatile **Poly** is.

5.1 Special polynomials and adjunctions

In Section 2.2, we identified four special classes of polynomials:

- a) constant polynomials I for $I \in \mathbf{Set}$;
- b) linear polynomials Iy for $I \in \mathbf{Set}$;
- c) representable polynomials y^A for $A \in \mathbf{Set}$; and
- d) monomials Iy^A for $I, A \in \mathbf{Set}$.

The first two classes, constant and linear polynomials, are interesting because they may each be viewed as a copy of **Set** inside **Poly**, as we will see in Propositions 5.2 and 5.3. The third class may be viewed as a copy of $\mathbf{Set}^{\text{op}}$ inside **Poly** via the Yoneda embedding, as we saw in Exercise 1.12. Finally, the fourth may be viewed as a copy of the category of bimorphic lenses in **Poly**, as we saw in Example 3.41.

Exercise 5.1. Which of the four classes above are closed under

1. coproducts?
2. cartesian products?
3. parallel products?

4. composition? Recall that the composite of $p, q \in \mathbf{Poly}$ is $p \circ q$, given by replacing each appearance of y in p by q .

Proposition 5.2. There is a fully faithful functor $\mathbf{Set} \rightarrow \mathbf{Poly}$ sending sets to constants: $I \mapsto Iy^0 = I$.

Proof. A lens $f: Iy^0 \rightarrow Jy^0$ consists of a function $f_1: I \rightarrow J$ and, for each $i \in I$, a function $0 \rightarrow 0$. There is exactly one function $0 \rightarrow 0$, so f can be identified with just a function between sets $I \rightarrow J$; in particular $\mathbf{Set}(I, J) \cong \mathbf{Poly}(Iy^0, Jy^0)$. This identification respects identities and composition, so it defines a fully faithful functor. $\square$

Proposition 5.3. There is a fully faithful functor $\mathbf{Set} \rightarrow \mathbf{Poly}$ sending $I \mapsto Iy$.

Proof. A lens $f: Iy \rightarrow Jy$ consists of a function $f_1: I \rightarrow J$ and, for each $i \in I$, a function $1 \rightarrow 1$. There is exactly one function $1 \rightarrow 1$, so f can be identified with just a function between sets $I \rightarrow J$; in particular $\mathbf{Set}(I, J) \cong \mathbf{Poly}(Iy, Jy)$. This identification respects identities and composition, so it defines a fully faithful functor. $\square$

Theorem 5.4. **Poly** has an adjoint quadruple with $\mathbf{Set}$:¹

$$\begin{array}{ccc}
 & \xleftarrow{q(0)} & \\
 & \Rightarrow & \\
 \mathbf{Set} & \xrightarrow{I} & \mathbf{Poly} \\
 & \Leftarrow & \\
 & \xleftarrow{q(1)} & \\
 & \Rightarrow & \\
 & \xrightarrow{Iy} &
 \end{array} \tag{5.5}$$

where the functors have been labeled by where they send $I \in \mathbf{Set}$ and $q \in \mathbf{Poly}$.

¹We denote an adjunction with a double arrow $\Rightarrow$ pointing in the direction of the left adjoint.

Proof. For each $A \in \mathbf{Set}$, there is a representable functor $\mathbf{Poly} \rightarrow \mathbf{Set}$ sending $q \mapsto \mathbf{Poly}(y^A, q)$. By the Yoneda lemma, this functor is isomorphic to the functor sending $q \mapsto q(A)$. This, along with Propositions 5.2 and 5.3, gives us the four functors in the quadruple. It remains to provide the following three natural isomorphisms:

$$\mathbf{Poly}(I, q) \cong \mathbf{Set}(I, q(0)), \quad \mathbf{Poly}(q, I) \cong \mathbf{Set}(q(1), I),$$

$$\text{and } \mathbf{Poly}(Iy, q) \cong \mathbf{Set}(I, q(1)).$$

All three come from our formula (3.7) for computing hom-sets in $\mathbf{Poly}$; we leave the details to the reader in Exercise 5.6. $\square$

Exercise 5.6. Here we prove the remainder of Theorem 5.4 using (3.7):

1. Exhibit a natural isomorphism $\mathbf{Poly}(I, q) \cong \mathbf{Set}(I, q(0))$.
2. Exhibit a natural isomorphism $\mathbf{Poly}(q, I) \cong \mathbf{Set}(q(1), I)$.
3. Exhibit a natural isomorphism $\mathbf{Poly}(Iy, q) \cong \mathbf{Set}(I, q(1))$.

Exercise 5.7. Show that for any polynomial q , its set $q(1)$ of positions is in bijection with the set of functions $y \rightarrow q$.

In Theorem 5.4 we see that $q \mapsto q(0)$ and $q \mapsto q(1)$ have left adjoints. It turns out this is true more generally for any set A in place of 0 and 1, as we will show in Corollary 5.10. However, the fact that $q \mapsto q(1)$ is itself the left adjoint of the left adjoint of $q \mapsto q(0)$ —and hence that we have the *quadruple* of adjunctions in (5.5)—is special to $A = 0, 1$.

We also have a copower-hom-power two-variable adjunction between $\mathbf{Poly}$, $\mathbf{Set}$, and $\mathbf{Poly}$.

Proposition 5.8. There is a two-variable adjunction between $\mathbf{Poly}$, $\mathbf{Set}$, and $\mathbf{Poly}$:

$$\mathbf{Poly}(Ip, q) \cong \mathbf{Set}(I, \mathbf{Poly}(p, q)) \cong \mathbf{Poly}(p, q^I) \quad (5.9)$$

for $I \in \mathbf{Set}$ and $p, q \in \mathbf{Poly}$.

Proof. Since I_p is the I -fold coproduct of p and q^I is the I -fold product of q , the universal properties of coproducts and products give natural isomorphisms

$$\mathbf{Poly}(I_p, q) \cong \prod_{i \in I} \mathbf{Poly}(p, q) \cong \mathbf{Poly}(p, q^I).$$

The middle set is then naturally isomorphic to $\mathbf{Set}(A, \mathbf{Poly}(p, q))$. $\square$

Taking $p := y^A$ in (5.9) yields the following via the Yoneda lemma.

Corollary 5.10. For any set A there is an adjunction

$$\mathbf{Set} \begin{array}{c} \xrightarrow{Iy^A} \\ \Rightarrow \\ \xleftarrow{q(A)} \end{array} \mathbf{Poly}$$

where the functors are labeled by where they send $I \in \mathbf{Set}$ and $q \in \mathbf{Poly}$.

Exercise 5.11. Prove Corollary 5.10 from Proposition 5.8.

Proposition 5.12. The Yoneda embedding $A \mapsto y^A$ has a left adjoint

$$\mathbf{Set}^{\text{op}} \begin{array}{c} \xrightarrow{y^-} \\ \Leftarrow \\ \xleftarrow{\Gamma} \end{array} \mathbf{Poly}$$

where $\Gamma(p) := \mathbf{Poly}(p, y) \cong \prod_{i \in p(1)} p[i]$, as in (3.35) and (3.37). That is, there is a natural isomorphism

$$\mathbf{Poly}(p, y^A) \cong \mathbf{Set}(A, \Gamma(p)). \quad (5.13)$$

Proof. By (3.7), we have the natural isomorphism

$$\mathbf{Poly}(p, y^A) \cong \prod_{i \in p(1)} p[i]^A \cong \left(\prod_{i \in p(1)} p[i] \right)^A,$$

which in turn is naturally isomorphic to $\mathbf{Set}(A, \Gamma(p))$ by (3.35). $\square$

Exercise 5.14. Give an alternate proof for Proposition 5.12 using Proposition 5.8.

Corollary 5.15 (Principal monomial). There is an adjunction

$$\mathbf{Poly} \begin{array}{c} \xrightarrow{(p(1), \Gamma(p))} \\ \Rightarrow \\ \xleftarrow{Iy^A} \end{array} \mathbf{Set} \times \mathbf{Set}^{\text{op}}$$

where the functors are labeled by where they send $p \in \mathbf{Poly}$ and $(I, A) \in \mathbf{Set} \times \mathbf{Set}^{\text{op}}$. That is, there is a natural isomorphism

$$\mathbf{Poly}(p, Iy^A) \cong \mathbf{Set}(p(1), I) \times \mathbf{Set}(A, \Gamma(p)). \quad (5.16)$$

Proof. By the universal property of the product of I and y^A , we have a natural isomorphism

$$\mathbf{Poly}(p, Iy^A) \cong \mathbf{Poly}(p, I) \times \mathbf{Poly}(p, y^A).$$

Then the desired natural isomorphism follows from Exercise 5.6 #2 and (5.13). $\square$

Exercise 5.17. Use (5.16) together with (4.83) and (4.87) to find an alternate proof for Proposition 4.54, i.e. that there is an isomorphism

$$\Gamma(p \otimes q) \cong \mathbf{Set}(q(1), \Gamma(p)) \times \mathbf{Set}(p(1), \Gamma(q))$$

for any $p, q \in \mathbf{Poly}$.

5.2 Epi-mono factorization of lenses

We begin by characterizing monic and epic lenses in $\mathbf{Poly}$.

Proposition 5.18. Let $f: p \rightarrow q$ be a lens in $\mathbf{Poly}$. It is a monomorphism if and only if the on-positions function $f_1: p(1) \rightarrow q(1)$ is a monomorphism in $\mathbf{Set}$ and, for each $i \in p(1)$, the on-directions

function $f_i^\sharp: q[f_1 i] \rightarrow p[i]$ is an epimorphism in **Set**.

Proof. To prove the forward direction, suppose that f is monic. Since $p \mapsto p(1)$ is a right adjoint (Theorem 5.4), it preserves monomorphisms, so the on-positions function f_1 is also monic.

We now need to show that for any $i \in p(1)$, the on-directions function $f_i^\sharp: q[f_1 i] \rightarrow p[i]$ is epic. Suppose we are given a set A and a pair of functions $g^\sharp, h^\sharp: p[i] \rightrightarrows A$ with $f_i^\sharp \circ g^\sharp = f_i^\sharp \circ h^\sharp$. Then there exist lenses $g, h: y^A \rightrightarrows p$ whose on-positions functions both pick out i and whose on-directions functions are $g^\sharp$ and $h^\sharp$, so that $g \circ f = h \circ f$. As f is monic, $g = h$; in particular, their on-directions functions $g^\sharp$ and $h^\sharp$ are equal, as desired.

Conversely, suppose that f_1 is monic and that, for each $i \in p(1)$, the function $f_i^\sharp$ is epic. Let r be a polynomial and $g, h: r \rightrightarrows p$ be two lenses such that $g \circ f = h \circ f$. Then $g_1 \circ f_1 = h_1 \circ f_1$, which implies $g_1 = h_1$. We also have that $f_{g_1 k}^\sharp \circ g_k^\sharp = f_{g_1 k}^\sharp \circ h_k^\sharp$ for any $k \in r(1)$. But $f_{g_1 k}^\sharp$ is epic, so in fact $g_k^\sharp = h_k^\sharp$, as desired. $\square$

Example 5.19. Choose a finite nonempty set k for $1 \leq k \in \mathbb{N}$, e.g. $k := 12$. There is a monomorphism

$$f: ky^k \rightarrow \mathbb{N}y^{\mathbb{N}}$$

defined as follows. On positions, we have $f_1 i := i$ for all $i \in k$. On directions, for any $i \in k$, we have $f_i^\sharp n := n \bmod k$ for all $n \in \mathbb{N}$.

Exercise 5.20. In Example 5.19, we gave a lens $12y^{12} \rightarrow \mathbb{N}y^{\mathbb{N}}$. This allows us to turn any dynamical system with state-set $\mathbb{N}$ into a dynamical system with 12 states, while keeping the same interface—say, p —by composing with this lens.

Explain how the behavior of the new system $12y^{12} \rightarrow p$ relates to the behavior of the old system $\mathbb{N}y^{\mathbb{N}} \rightarrow p$.

Proposition 5.21. Let $f: p \rightarrow q$ be a lens in **Poly**. It is an epimorphism if and only if the function $f_1: p(1) \rightarrow q(1)$ is an epimorphism in **Set** and, for each $j \in q(1)$, the induced function

$$f_j^b: q[j] \rightarrow \prod_{\substack{i \in p(1), \\ f_1 i = j}} p[i]$$

from (3.20) is a monomorphism.

Proof. To prove the forward direction, suppose that f is epic. Since $p \mapsto p(1)$ is a left adjoint (Theorem 5.4), it preserves epimorphisms, so the on-positions function f_1 is also epic.

We now need to show that for any $j \in q(1)$, the induced function f_j^b is monic. Suppose we are given a set A and a pair of functions $g', h': A \rightrightarrows q[j]$ with $g' \circ f_j^b = h' \circ f_j^b$. Then we can construct lenses $g, h: q \rightrightarrows \{0\}y^A + \{1\}$ that send $j \mapsto 0$ and every other position to 1 on positions; on directions, $g_j^\# := g'$ and $h_j^\# := h'$, while every other on-direction function is uniquely determined. Then $f \circ g = f \circ h$. As f is epic, $g = h$; in particular, their on-directions functions are equal, so $g' = h'$, as desired.

Conversely, suppose that f_1 is epic and that, for each $j \in q(1)$, the function f_j^b is monic. Let r be a polynomial and $g, h: q \rightrightarrows r$ be two lenses such that $f \circ g = f \circ h$. Then $f_1 \circ g_1 = f_1 \circ h_1$, which implies $g_1 = h_1$. We also have that $g_{f_1 i}^\# \circ f_i^\# = h_{f_1 i}^\# \circ f_i^\#$ for any $i \in p(1)$. It follows that, for any $j \in q(1)$, the two composites

$$r[g_1 j] \xrightarrow[h_j^\#]{g_j^\#} q[j] \xrightarrow{f_j^b} \prod_{\substack{i \in p(1), \\ f_1 i = j}} p[i]$$

are equal, which implies that $g_j^\# = h_j^\#$ as desired. $\square$

Exercise 5.22. In Proposition 5.18, we characterized monomorphisms in **Poly** as lenses whose on-positions functions are monic

and whose on-directions functions are epic. On the other hand, in Proposition 5.21, we did *not* characterize epimorphisms in **Poly** as lenses whose on-positions functions are epic and whose on-directions functions are monic.

1. Give an example of an epimorphism in **Poly** that has at least one on-directions function that is *not* monic, or show that no such example exists.
2. Give an example of a lens that is *not* epic in **Poly**, but whose on-positions function is epic and whose on-directions functions are monic; or show that no such example exists.

Exercise 5.23. Show that the only way for a lens $p \rightarrow y$ to *not* be an epimorphism is when $p = 0$.

Exercise 5.24. Given sets A and B , find an epimorphism $y^A + y^B \twoheadrightarrow y^{AB}$.

Exercise 5.25. Suppose a lens $f: p \rightarrow q$ is both a monomorphism and an epimorphism; is it then an isomorphism? That is, is **Poly** *balanced*?

(Hint: you may use the following facts.)

1. A function that is both a monomorphism and an epimorphism in **Set** is an isomorphism.
2. A lens is an isomorphism if and only if its on-positions function is an isomorphism and its every on-directions function is an isomorphism.

We are often interested in whether epimorphisms and monomorphisms form what is called a *factorization system* in a given category, which we define here.

Definition 5.26 (Factorization system). Given a category **C** and two classes of morphisms E and M in **C**, we say that (E, M) is a

factorization system of $\mathbf{C}$ if:

1. every morphism f in $\mathbf{C}$ factors uniquely (up to unique isomorphism) as a morphism $e \in E$ composed with a morphism $m \in M$, so that $f = e \circ m$;
2. E and M each contain every isomorphism; and
3. E and M are each closed under composition.

If E is the class of epimorphisms and M is the class of monomorphisms (in which case conditions 2 and 3 are automatically satisfied), we say that $\mathbf{C}$ has *epi-mono factorization*.

Example 5.27 (Epi-mono factorization in **Set**). The category **Set** has epi-mono factorization: a function $f: X \rightarrow Y$ can be uniquely factored into an epimorphism (surjection) e followed by a monomorphism (injection) i , as follows. The epimorphism $e: X \rightarrow f(X)$ is given by restricting the codomain of f to its image (also known as *corestricting* f), so e sends $x \mapsto f(x)$ for all $x \in X$. The monomorphism $i: f(X) \rightarrow Y$ is then given by including the image into the codomain, so i sends $y \mapsto y$ for all $y \in f(X) \subseteq Y$.

Proposition 5.28. **Poly** has epi-mono factorization.

Proof. Given a lens $\varphi: p \rightarrow q$, it suffices to show that there exists a unique polynomial r equipped with an epimorphism $\epsilon: p \rightarrow r$ and a monomorphism $\mu: r \rightarrow q$ such that $\varphi = \epsilon \circ \mu$.

On positions, we must have $\varphi_1 = \epsilon_1 \circ \mu_1$, with μ_1 a monomorphism and ϵ_1 an epimorphism per Propositions 5.18 and 5.21. By Example 5.27, since **Set** has epi-mono factorization, the set $r(1)$ and the functions ϵ_1 and μ_1 uniquely exist. In particular, we must have that $r(1) \cong \varphi_1(p(1))$, that $\epsilon_1: p(1) \rightarrow \varphi_1(p(1))$ is the corestriction of φ_1 sending $i \mapsto \varphi_1 i$ for each p -position i , and that $\mu_1: \varphi_1(p(1)) \rightarrow q(1)$ is the inclusion sending $j \mapsto j$ for each r -position j .

Then on directions, for any $i \in p(1)$, we must have that

$$\begin{array}{ccc} q[\varphi_1 i] & \xrightarrow{\mu_{\varphi_1 i}^\#} & r[\varphi_1 i] \\ & \searrow \varphi_i^\# & \downarrow \epsilon_i^\# \\ & & p[i] \end{array}$$

commutes—or, equivalently, for every $j \in r(1) \cong \varphi_1(p(1))$,

$$\begin{array}{ccc} q[j] & \xrightarrow{\mu_j^\#} & r[j] \\ & \searrow \varphi_j^\flat & \downarrow \epsilon_j^\flat \\ & & \prod_{\substack{i \in p(1), \\ \varphi_1 i = j}} p[i] \end{array}$$

commutes (here $\varphi_j^\flat$ and $\epsilon_j^\flat$ are the induced functions from (3.20)), with $\mu_j^\#$ epic and $\epsilon_j^\flat$ monic per Propositions 5.18 and 5.21. So again since **Set** has epi-mono factorization, the set $r[j]$ and the functions $\mu_j^\#$ and $\epsilon_j^\flat$ uniquely exist. Hence such $p \xrightarrow{\epsilon} r \xrightarrow{\mu} q$ uniquely exists overall. $\square$

5.3 Cartesian closure

We have already seen in Section 4.5 the closure operation $[-, -]$ for one monoidal structure on **Poly**, namely $(y, \otimes)$. But this is not the only closed monoidal structure on **Poly**: we will show that **Poly** is cartesian closed as well.

For any two polynomials q, r , define $r^q \in \mathbf{Poly}$ by the formula

$$r^q := \prod_{j \in q(1)} r \circ (y + q[j]) \quad (5.29)$$

where $\circ$ denotes composition.

Before proving that this really is an exponential in **Poly**, which we do in Theorem 5.32, let us first see it in action.

Example 5.30. Let A be a set. We have been writing the polynomial Ay^0 as A , so we need there to be an isomorphism

$$y^A \cong y^{Ay^0},$$

where the left-hand side is the representable and the right-hand side is the cartesian closure, in order for our exponentiation notation to be consistent. Fortunately, this is true: by (5.29), we have

$$y^{Ay^0} = \prod_{a \in A} y \circ (y + 0) \cong y^A.$$

Exercise 5.31. Compute the following exponentials in **Poly** using (5.29):

1. p^0 for an arbitrary $p \in \mathbf{Poly}$.
2. p^1 for an arbitrary $p \in \mathbf{Poly}$.
3. 1^p for an arbitrary $p \in \mathbf{Poly}$.
4. A^p for an arbitrary $p \in \mathbf{Poly}$ and $A \in \mathbf{Set}$.
5. y^y .
6. y^{Ay} .
7. $(y^A)^{y^B}$ for arbitrary sets $A, B \in \mathbf{Set}$.

Theorem 5.32. The category **Poly** is cartesian closed. That is, we have a natural isomorphism

$$\mathbf{Poly}(p, r^q) \cong \mathbf{Poly}(p \times q, r),$$

where r^q is the polynomial defined in (5.29).

Proof. We have the following chain of natural isomorphisms:

$$\begin{aligned} \mathbf{Poly}(p, r^q) &\cong \mathbf{Poly}\left(p, \prod_{j \in q(1)} r \circ (y + q[j])\right) & (5.29) \\ &\cong \prod_{i \in p(1)} \prod_{j \in q(1)} \mathbf{Poly}(y^{p[i]}, r \circ (y + q[j])) \\ &\quad \text{(Universal property of (co)products)} \end{aligned}$$

$$\begin{aligned}
&\cong \prod_{i \in p(1)} \prod_{j \in q(1)} r \circ (p[i] + q[j]) && \text{(Yoneda lemma)} \\
&\cong \prod_{i \in p(1)} \prod_{j \in q(1)} \sum_{k \in r(1)} (p[i] + q[j])^{r[k]} \\
&\cong \prod_{(i,j) \in (p \times q)(1)} \sum_{k \in r(1)} (p \times q)[(i,j)]^{r[k]} && (3.61) \\
&\cong \mathbf{Poly}(p \times q, r). && (3.7)
\end{aligned}$$

□

Exercise 5.33. Use Theorem 5.32 to show that for any polynomials p, q , there is a canonical evaluation lens

$$p^q \times q \rightarrow p.$$

5.4 Limits and colimits of polynomials

We have already seen that **Poly** has all coproducts (Proposition 3.3) and products (Proposition 3.56). We will now see that **Poly** has all small limits and colimits.

Theorem 5.34. The category **Poly** has all small limits.

Proof. A category has all small limits if and only if it has products and equalizers, so by Proposition 3.56, it suffices to show that **Poly** has equalizers.

We claim that equalizers in **Poly** are equalizers on positions and coequalizers on directions. More precisely, given lenses $f, g: p \rightrightarrows q$, we construct their equalizer $p' \in \mathbf{Poly}$ as follows.² We define its position-set $p'(1)$ to be the equalizer of $f_1, g_1: p(1) \rightrightarrows q(1)$ in **Set**; that is,

$$p'(1) := \{i \in p(1) \mid f_1 i = g_1 i\}.$$

²Technically, a “(co)equalizer” is an object equipped with a morphism, but we will use the term to refer to either just the object or just the morphism when the context is clear.

Then for each $i \in p'(1)$, we define the direction-set $p'[i]$ to be the co-equalizer of $f_i^\sharp, g_i^\sharp: q[f_i i] \rightrightarrows p[i]$. In this way, we obtain a polynomial p' that comes equipped with a lens $e: p' \rightarrow p$. One can check that p' together with e satisfies the universal property of the equalizer of f and g ; see Exercise 5.35. $\square$

Exercise 5.35. Complete the proof of Theorem 5.34 as follows:

1. We said that p' comes equipped with a lens $e: p' \rightarrow p$; what is it?
2. Show that $e \circ f = e \circ g$.
3. Show that e is the equalizer of the pair f, g .

Example 5.36 (Computing general limits in **Poly**). The proof of Theorem 5.34 justifies the following mnemonic for limits in **Poly**:

*The positions of the limit are the limit of the positions.
The directions of the limit are the colimit of the directions.*

We can make this precise as follows: the limit of a functor $p_-: \mathcal{G} \rightarrow \mathbf{Poly}$ is the polynomial whose position-set is

$$\left(\lim_{j \in \mathcal{G}} p_j \right)(1) \cong \lim_{j \in \mathcal{G}} p_j(1), \quad (5.37)$$

equipped with a canonical projection π_j to each $p_j(1)$, and whose direction-set for each position i is

$$\left(\lim_{j \in \mathcal{G}} p_j \right)[i] \cong \operatorname{colim}_{j \in \mathcal{G}^{\text{op}}} p_j[\pi_j(i)]. \quad (5.38)$$

This notation obscures what is happening with the lenses, but in particular, each lens $\varphi: p_j \rightarrow p_{j'}$ in the diagram p_- induces an on-positions function $\varphi_1: p_j(1) \rightarrow p_{j'}(1)$ in the diagram whose limit we take in (5.37) and, for every position i of the limit, an on-directions function $\varphi_{\pi_j i}^\sharp: p_{j'}[\pi_{j'} i] \rightarrow p_j[\pi_j i]$ in the diagram whose colimit we take in (5.38) (note that, by the definition of a limit, $\varphi_1 \pi_j i = \pi_{j'} i$).

We have seen (5.37) and (5.38) to be true for products: the position-set of the product is just the product of the original position-sets, while the direction-set at a tuple of the original positions is just the coproduct of the direction-sets at every position in the tuple. We have also just shown (5.37) and (5.38) to be true for equalizers in the proof of Theorem 5.34. It follows from the construction of any limit as an equalizer of products that it is true for arbitrary limits.

*Example 5.39 (Pullbacks in **Poly**).* Given $q, q', r \in \mathbf{Poly}$ and lenses $q \xrightarrow{f} r \xleftarrow{f'} q'$, the pullback

$$\begin{array}{ccc} p & \xrightarrow{g'} & q' \\ g \downarrow & \lrcorner & \downarrow f' \\ q & \xrightarrow{f} & r \end{array}$$

is given as follows. The position-set of p is the pullback of the position-sets of q and q' over that of r in **Set**. Then at each position $(i, i') \in p(1) \subseteq q(1) \times q'(1)$ with $f_1 i = f'_1 i'$, we take the direction-set $p[(i, i')]$ to be the pushout of the direction-sets $q[i]$ and $q'[i']$ over $r[f_1 i] = r[f'_1 i']$ in **Set**. These pullback and pushout squares also give the lenses g and g' on positions and on directions:

$$\begin{array}{ccc} p(1) & \xrightarrow{g'_1} & q'(1) \\ g_1 \downarrow & \lrcorner & \downarrow f'_1 \\ q(1) & \xrightarrow{f_1} & r(1) \end{array} \quad \text{and} \quad \begin{array}{ccc} p[(i, i')] & \xleftarrow{(g')^\#_{(i, i')}} & q'[i'] \\ g^\#_{(i, i')} \uparrow & \lrcorner & \uparrow (f')^\#_{i'} \\ q[i] & \xleftarrow{f^\#_i} & r[f_1 i] \end{array} \quad (5.40)$$

Exercise 5.41. Let p be any polynomial.

1. There is a canonical choice of lens $\eta: p \rightarrow p(1)$; what is it?
2. Given an element $i \in p(1)$, i.e. a function (or lens between

constant polynomials) $i: 1 \rightarrow p(1)$, let p_i be the pullback

$$\begin{array}{ccc} p_i & \xrightarrow{g} & p \\ f \downarrow & \lrcorner & \downarrow \eta \\ 1 & \xrightarrow{i} & p(1) \end{array}$$

What is p_i ? What are the lenses $f: p_i \rightarrow 1$ and $g: p_i \rightarrow p$?

Exercise 5.42. Let $q := y^2 + y$, $q' := 2y^3 + y^2$, and $r := y + 1$.

1. Choose lenses $f: q \rightarrow r$ and $f': q' \rightarrow r$ and write them down.
2. Find the pullback of $q \xrightarrow{f} r \xleftarrow{f'} q'$.

Exercise 5.43. An alternative way to prove Theorem 5.34 would have been to show that the equalizer of two natural transformations between polynomial functors in $\mathbf{Set}^{\mathbf{Set}}$ is still a polynomial functor—since the full subcategory inclusion $\mathbf{Poly} \rightarrow \mathbf{Set}^{\mathbf{Set}}$ reflects these equalizers, it would follow that $\mathbf{Poly}$ has equalizers. But we already know what polynomial the equalizer should be from the proof of Theorem 5.34. So in this exercise, we will show that the equalizer of polynomials we found in $\mathbf{Poly}$ is also the equalizer of those same functors in $\mathbf{Set}^{\mathbf{Set}}$.

Let $f, g: p \rightrightarrows q$ be a pair of natural transformations $f, g: p \rightrightarrows q$ between polynomial functors p and q , and let $e: p' \rightarrow p$ be their equalizer in $\mathbf{Poly}$ that we computed in the proof of Theorem 5.34.

1. Given a set X , show that $e_X: p'(X) \rightarrow p(X)$ is the equalizer of the X -components $f_X, g_X: p(X) \rightrightarrows q(X)$ in $\mathbf{Set}$.
2. Deduce that equalizers in $\mathbf{Poly}$ coincide with equalizers in $\mathbf{Set}^{\mathbf{Set}}$.
3. Conclude that limits in $\mathbf{Poly}$ coincide with limits in $\mathbf{Set}^{\mathbf{Set}}$.

Theorem 5.44. The category $\mathbf{Poly}$ has all small colimits.

Proof. A category has all small colimits if and only if it has coproducts and coequalizers, so by Proposition 3.3, it suffices to show that **Poly** has coequalizers.

Given lenses $s, t: p \rightrightarrows q$, we construct the coequalizer q' of s and t as follows. The pair of functions $s_1, t_1: p(1) \rightrightarrows q(1)$ define a directed graph $G: \boxed{\bullet \rightrightarrows \bullet} \rightarrow \mathbf{Set}$ with vertices in $q(1)$, edges in $p(1)$, source vertices of edges indicated by s_1 , and target vertices of edges indicated by t_1 . Then the set C of connected components of G is given by the coequalizer $g_1: q(1) \rightarrow C$ of s_1 and t_1 . We define the position-set of q' to be C . Each direction-set of q' will be a limit of a diagram of direction-sets of p and q , as follows.

For each connected component $c \in C$, we have a connected subgraph $G_c \subseteq G$ with vertices $V_c := g_1^{-1}(c)$ and edges $E_c := s_1^{-1}(g_1^{-1}(c)) = t_1^{-1}(g_1^{-1}(c))$. Note that $E_c \subseteq p(1)$ and $V_c \subseteq q(1)$, so to each $e \in E_c$ (resp. to each $v \in V_c$) we have an associated direction-set $p[e]$ (resp. $q[v]$).

The category of elements $\int G_c$ has objects $E_c + V_c$ and two kinds of (non-identity) morphisms, $e \rightarrow s_1(e)$ and $e \rightarrow t_1(e)$, associated to each $e \in E_c$, all pointing from an object in E_c to an object in V_c . There is a functor $F: (\int G_c)^{\text{op}} \rightarrow \mathbf{Set}$ sending every $v \mapsto q[v]$, every $e \mapsto p[e]$, and every morphism to a function between them, namely either $s_e^\# : q[s_1 e] \rightarrow p[e]$ or $t_e^\# : q[t_1 e] \rightarrow p[e]$. So we can define $q'[c]$ to be the limit of F in **Set**.

We claim that $q' := \sum_{c \in C} y^{q'[c]}$ is the coequalizer of s and t . We leave the complete proof to the interested reader in Exercise 5.45. $\square$

Exercise 5.45. Complete the proof of Theorem 5.44 as follows:

1. Provide a lens $g: q \rightarrow q'$.
2. Show that $s \circ g = t \circ g$.
3. Show that g is a coequalizer of the pair s, t .

Example 5.46. Given a diagram in **Poly**, one could either take its (co)limit as a diagram of *polynomial* functors (i.e. its (co)limit in **Poly**) or its (co)limit simply as a diagram of functors (i.e. its (co)limit in **Set^{Set}**). We saw in Exercise 5.43 that in the case of limits, these

yield the same result; so, too, in the case of coproducts, per Proposition 3.3.

But in the case of general colimits, there are diagrams that yield different results: after all, by the co-Yoneda lemma, *every* functor $\mathbf{Set} \rightarrow \mathbf{Set}$ —even those that are not polynomials—can be written as the colimit of representable functors in $\mathbf{Set}^{\mathbf{Set}}$, yet the colimit of the same representables in $\mathbf{Poly}$ can only be another polynomial.

For a concrete example, consider the two distinct projections $y^2 \rightarrow y$, which form the diagram

$$y^2 \rightrightarrows y. \quad (5.47)$$

According to Theorem 5.44, the colimit of (5.47) in $\mathbf{Poly}$ has the coequalizer of $1 \rightrightarrows 1$, namely 1 , as its position-set, and the limit of the diagram $1 \rightrightarrows 2$ consisting of the two inclusions as its sole direction-set. But this latter limit is just 0 , so in fact the colimit of (5.47) in $\mathbf{Poly}$ is the constant functor $1y^0 \cong 1$.

But as functors, by Proposition 1.37, the colimit of (5.47) can be computed pointwise: it is the (nonconstant!) functor

$$X \mapsto \begin{cases} 0 & \text{if } X = 0; \\ 1 & \text{if } X \neq 0. \end{cases}$$

Exercise 5.48. By Theorem 5.4, for any polynomial p , there are canonical lenses involving positions and global sections:

$$\epsilon: p(1)y \rightarrow p \quad \text{and} \quad \eta: p \rightarrow y^{\Gamma(p)}.$$

1. Characterize the behavior of the canonical lens $\epsilon: p(1)y \rightarrow p$.
2. Characterize the behavior of the canonical lens $\eta: p \rightarrow y^{\Gamma(p)}$.

3. Show that the following is a pushout in **Poly**:

$$\begin{array}{ccc}
 p(1)y & \xrightarrow{!} & y \\
 \epsilon \downarrow & \ulcorner & \downarrow ! \\
 p & \xrightarrow{\eta} & y^{\Gamma(p)}
 \end{array} \quad (5.49)$$

Proposition 5.50. For polynomials p, q , the following is a pushout:

$$\begin{array}{ccc}
 p(1)y \otimes q(1)y & \longrightarrow & p(1)y \otimes q \\
 \downarrow & \ulcorner & \downarrow \\
 p \otimes q(1)y & \longrightarrow & p \otimes q
 \end{array}$$

Proof. All the lenses shown are identities on positions, so the displayed diagram is the coproduct over all $(i, j) \in p(1) \times q(1)$ of the diagram on the left below:

$$\begin{array}{ccc}
 y & \longrightarrow & y^{q[j]} \\
 \downarrow & \ulcorner & \downarrow \\
 y^{p[i]} & \longrightarrow & y^{p[i] \times q[j]}
 \end{array}
 \qquad
 \begin{array}{ccc}
 1 & \longleftarrow & q[j] \\
 \uparrow & \ulcorner & \uparrow \\
 p[i] & \longleftarrow & p[i] \times q[j]
 \end{array}$$

where we use $(p \otimes q)[(i, j)] \cong p[i] \times q[j]$. This is the image under the Yoneda embedding of the diagram of sets shown right, which is a pullback. The result then follows by Proposition 5.12. $\square$

So to give a lens $\varphi: p \otimes q \rightarrow r$, it suffices to give two lenses $\varphi_p: p \otimes q(1)y \rightarrow r$ and $\varphi_q: p(1)y \otimes q \rightarrow r$ that agree on positions. Intuitively, the lens φ_p describes how information about q 's position is sent to p , and the lens φ_q says how information about p 's position is sent to q . The next corollary follows by induction.

Corollary 5.51. Suppose we have polynomials $p_1, \dots, p_n \in \mathbf{Poly}$.

Then $p_1 \otimes \cdots \otimes p_n$ is isomorphic to the wide pushout of

$$\begin{array}{ccc}
 & \bigotimes_{i=1}^n (p_i(1)y) & \\
 \swarrow & & \searrow \\
 p_1 \otimes p_2(1)y \otimes \cdots \otimes p_n(1)y & \cdots & p_1(1)y \otimes \cdots \otimes p_{n-1}(1)y \otimes p_n
 \end{array}$$

5.5 Vertical-cartesian factorization of lenses

Aside from epi-mono factorization, there is another useful factorization system on **Poly**.

Definition 5.52 (Vertical and cartesian lenses). Let $f: p \rightarrow q$ be a lens. It is called *vertical* if the on-positions function $f_1: p(1) \rightarrow q(1)$ is an isomorphism. It is called *cartesian* if, for each $i \in p(1)$, the on-directions function $f_i^\sharp: q[f_1 i] \rightarrow p[i]$ is an isomorphism.

Proposition 5.53. Vertical and cartesian lenses form a factorization system of **Poly**.

Proof. It follows from the definitions that isomorphisms are both vertical and cartesian and that vertical and cartesian lenses are each closed under composition. It remains to show that every lens $f: p \rightarrow q$ in **Poly** can be factored uniquely (up to unique isomorphism) as a vertical lens composed with a cartesian lens.

If $v: p \rightarrow r$ is a vertical lens and $c: r \rightarrow q$ is cartesian, then $v \circ c = f$ if and only if $v_1 \circ c_1 = f_1$ and $c_{v_1 i}^\sharp \circ v_i^\sharp = f_i^\sharp$ for all $i \in p(1)$. As v_1 and $c_{v_1 i}^\sharp$ are isomorphisms, these equations uniquely determine c_1 and $v_i^\sharp$ for each $i \in p(1)$, so this factorization is necessarily unique up to unique isomorphism. In particular, we can take $r := \sum_{i \in p(1)} y^{q[f_1 i]}$, with $v_1 := \text{id}_{p(1)}$ and $v_i^\sharp := f_i^\sharp$ for each $i \in p(1)$ as well as $c_1 := f_1$ and $c_i^\sharp := \text{id}_{q[f_1 i]}$ for each $i \in p(1)$. $\square$

We can view this factorization diagrammatically as follows. Recall from (3.8) that a lens $f: p \rightarrow q$ can be written as a functor f_1 between

discrete categories and a natural transformation $f^\sharp$, as on the left; we can rewrite this as on the right, where the triangle on the right commutes:

$$\begin{array}{ccc}
 p(1) & \xrightarrow{f_1} & q(1) \\
 \searrow p[-] & \begin{array}{c} f^\sharp \\ \Leftarrow \end{array} & \swarrow q[-] \\
 & \mathbf{Set} &
 \end{array}
 \qquad
 \begin{array}{ccc}
 p(1) & \xlongequal{\quad} p(1) & \xrightarrow{f_1} q(1) \\
 \searrow p[-] & \begin{array}{c} f^\sharp \\ \Downarrow q[f_1(-)] \end{array} & \swarrow q[-] \\
 & \mathbf{Set} &
 \end{array}$$

Then the left triangle on the right corresponds to a vertical lens $p \rightarrow \sum_{i \in p(1)} y^{q[f_1 i]}$ whose on-directions function at $i \in p(1)$ is $f^\sharp$; the right triangle corresponds to a cartesian lens $\sum_{i \in p(1)} y^{q[f_1 i]} \rightarrow q$ whose on-positions function is f^1 ; and their composite is the original lens f .

Proposition 5.54. Vertical lenses satisfy the 2-out-of-3 condition: given $p \xrightarrow{f} q \xrightarrow{g} r$ with $h = f \circ g$, if any two of f, g, h are vertical, then so is the third.

If g is cartesian, then h is cartesian if and only if f is cartesian.

Proof. Given $h = f \circ g$, we have that $h_1 = f_1 \circ g_1$. Since isomorphisms satisfy 2-out-of-3, it follows that vertical lenses satisfy 2-out-of-3 as well.

Now assume g is cartesian. On directions, $h = f \circ g$ implies that for every $i \in p(1)$, we have $h_i^\sharp = g_{f_1 i}^\sharp \circ f_i^\sharp$. Since $g_{f_1 i}^\sharp$ is an isomorphism, it follows that every $h_i^\sharp$ is an isomorphism if and only if every $f_i^\sharp$ is an isomorphism, so h is cartesian if and only if f is cartesian. $\square$

Exercise 5.55. Give an example of polynomials p, q, r and lenses $p \xrightarrow{f} q \xrightarrow{g} r$ such that f and $f \circ g$ are cartesian but g is not.

Here is an alternative characterization of a cartesian lens in **Poly**. Recall from Definition 3.36 that the bundle of a polynomial p is the function $\pi_p: \dot{p}(1) \rightarrow p(1)$ whose fiber over $i \in p(1)$ is $p[i]$. Then a lens $f: p \rightarrow q$ consists of a function $f_1: p(1) \rightarrow q(1)$ along with a function

$f^\#$ that makes the following diagram in **Set** commute:

$$\begin{array}{ccccc}
 \dot{p}(1) & \xleftarrow{f^\#} & \bullet & \xrightarrow{\quad} & \dot{q}(1) \\
 \pi_p \downarrow & & \downarrow & \lrcorner & \downarrow \pi_q \\
 p(1) & \equiv & p(1) & \xrightarrow{f_1} & q(1)
 \end{array} \tag{5.56}$$

Here, the pullback denoted by the dot $\bullet$ is the set of pairs comprised of a p -position i and a $q[f_1 i]$ -direction e . The function $f^\#$ sends each such pair to a direction $f_i^\# e$ of p , and the commutativity of the left square implies that $f_i^\# e$ is a $p[i]$ -direction. So $f_i^\#$ is indeed our familiar on-directions function $q[f_1 i] \rightarrow p[i]$, and $f^\#$ is just the sum of all these on-directions functions over $i \in p(1)$.

Exercise 5.57. Show that a lens $f: p \rightarrow q$ in **Poly** is cartesian if and only if the square on the left-hand side of (5.56) is also a pullback:

$$\begin{array}{ccccc}
 \dot{p}(1) & \xleftarrow{f^\#} & \bullet & \xrightarrow{\quad} & \dot{q}(1) \\
 \pi_p \downarrow & & \downarrow & \lrcorner & \downarrow \pi_q \\
 p(1) & \equiv & p(1) & \xrightarrow{f_1} & q(1)
 \end{array}$$

Proposition 5.58. The monoidal structures $+$, $\times$, and $\otimes$ preserve both vertical and cartesian lenses.

Proof. Suppose that $f: p \rightarrow p'$ and $g: q \rightarrow q'$ are vertical, so that the on-positions functions f_1 and g_1 are isomorphisms.

We can obtain the on-positions function of a lens by passing it through the functor $\mathbf{Poly} \xrightarrow{p(1)} \mathbf{Set}$ from Theorem 5.4. As this functor is both a left adjoint and a right adjoint, it preserves both sums and products, so $(f + g)_1 = f_1 + g_1$ and $(f \times g)_1 = f_1 \times g_1$. Hence $f + g$ and $f \times g$ are both vertical. On-positions, the behavior of $\otimes$ is identical to the behavior of $\times$, so $f \otimes g$ must be vertical as well.

Now suppose that $f: p \rightarrow p'$ and $g: q \rightarrow q'$ are cartesian.

A position of $p + q$ is a position $i \in p(1)$ or a position $j \in q(1)$, and the map $(f + g)^\sharp$ at that position is either $f_i^\sharp$ or $g_j^\sharp$; either way, it is an isomorphism, so $f + g$ is cartesian.

A position of $p \times q$ (resp. of $p \otimes q$) is a pair $(i, j) \in p(1) \times q(1)$. The lens $(f \times g)_{(i,j)}^\sharp$ (resp. $(f \otimes g)_{(i,j)}^\sharp$) is $f_i^\sharp + g_j^\sharp$ (resp. $f_i^\sharp \times g_j^\sharp$) which is again an isomorphism if $f_i^\sharp$ and $g_j^\sharp$ are. Hence $f \times g$ (resp. $f \otimes g$) is cartesian, completing the proof. $\square$

Proposition 5.59. Pullbacks preserve vertical (resp. cartesian) lenses. In other words, if $f: p \rightarrow q$ is a lens and $g: q' \rightarrow q$ a vertical (resp. cartesian) lens, then the pullback g' of g along p

$$\begin{array}{ccc} p \times_q q' & \longrightarrow & q' \\ g' \downarrow & \lrcorner & \downarrow g \\ p & \xrightarrow{f} & q \end{array}$$

is vertical (resp. cartesian).

Proof. This follows from Example 5.39, since the pullback (resp. pushout) of an isomorphism is an isomorphism. $\square$

Exercise 5.60. Is the pushout of a cartesian lens always cartesian?

Why do we use the word *cartesian* to describe cartesian lenses? It turns out that, as natural transformations, cartesian morphisms are precisely what are known as cartesian natural transformations.

Definition 5.61 (Cartesian natural transformation). A *cartesian natural transformation* is a natural transformation whose naturality squares are all pullbacks. That is, given categories $\mathcal{C}, \mathcal{D}$, functors $F, G: \mathcal{C} \rightarrow \mathcal{D}$, and a natural transformation $\alpha: F \rightarrow G$, we say that α is *cartesian* if for all morphisms $h: c' \rightarrow c$ in $\mathcal{C}$, the induced

naturality square

$$\begin{array}{ccc} Fc' & \xrightarrow{Fh} & Fc \\ \alpha_{c'} \downarrow & \lrcorner & \downarrow \alpha_c \\ Gc' & \xrightarrow{Gh} & Gc \end{array}$$

is a pullback square in $\mathcal{D}$.

There is a simpler characterization of cartesian natural transformations when $\mathcal{C}$ has a terminal object 1 by the pasting lemma, a standard result about pullbacks that we review.

Lemma 5.62 (Pasting lemma). Given a commutative diagram

$$\begin{array}{ccccc} \bullet & \longrightarrow & \bullet & \longrightarrow & \bullet \\ \downarrow & & \downarrow & \lrcorner & \downarrow \\ \bullet & \longrightarrow & \bullet & \longrightarrow & \bullet \end{array}$$

in which the right square is a pullback, the left square is a pullback if and only if the composite rectangle is a pullback.

Lemma 5.63. Given categories $\mathcal{C}, \mathcal{D}$, functors $F, G: \mathcal{C} \rightrightarrows \mathcal{D}$, and a natural transformation $\alpha: F \rightarrow G$, if $\mathcal{C}$ has a terminal object 1 , then α is cartesian if and only if for all $c \in \mathcal{C}$, the naturality square induced by the unique morphism $!: c \rightarrow 1$ is a pullback in $\mathcal{D}$:

$$\begin{array}{ccc} Fc & \xrightarrow{F!} & F1 \\ \alpha_c \downarrow & \lrcorner & \downarrow \alpha_1 \\ Gc & \xrightarrow{G!} & G1. \end{array}$$

Proof. This follows from Lemma 5.62; see the following exercise. $\square$

Exercise 5.64. Show Lemma 5.63 using Lemma 5.62.

Proposition 5.65. Let $f: p \rightarrow q$ be a morphism in **Poly**. The following are equivalent:

- (1) viewed as a lens, f is cartesian in the sense of Definition 5.52: for each $i \in p(1)$, the on-directions function $f_i^\#$ is a bijection;
- (2) the square on the left-hand side of (5.56) is also a pullback:

$$\begin{array}{ccccc}
 \dot{p}(1) & \xleftarrow{f^\#} & \bullet & \xrightarrow{\quad} & \dot{q}(1) \\
 \pi_p \downarrow & & \downarrow & \lrcorner & \downarrow \pi_q \\
 p(1) & \xlongequal{\quad} & p(1) & \xrightarrow{f_1} & q(1)
 \end{array}$$

- (3) viewed as a natural transformation, f is cartesian in the sense of Definition 5.61: for any sets A, B and function $h: A \rightarrow B$, the naturality square

$$\begin{array}{ccc}
 p(A) & \xrightarrow{p(h)} & p(B) \\
 f_A \downarrow & \lrcorner & \downarrow f_B \\
 q(A) & \xrightarrow{q(h)} & q(B)
 \end{array} \tag{5.66}$$

is a pullback.

Proof. We already showed that (1) $\Leftrightarrow$ (2) in Exercise 5.57. By Lemma 5.63 and the fact that 1 is terminal in **Set**, (3) is equivalent to the condition that for any set A , the naturality square

$$\begin{array}{ccc}
 p(A) = \sum_{i \in p(1)} A^{p[i]} & \xrightarrow{p(!)} & p(1) \\
 f_A \downarrow & \lrcorner & \downarrow f_1 \\
 q(A) = \sum_{j \in q(1)} A^{q[j]} & \xrightarrow{q(!)} & q(1)
 \end{array}$$

induced by the unique function $!: A \rightarrow 1$ is a pullback. Proposition 2.10 tells us that $p(!)$ sends $(i, k) \in p(A)$, with $i \in p(1)$ and $k: p[i] \rightarrow A$, to just i ; similarly, $q(!)$ sends $(j, \ell) \in q(A)$, with $j \in q(1)$ and $\ell: q[j] \rightarrow A$, to j . So to say that this square is a pullback is to say that for every $i \in p(1)$, the function f_A restricted to the fiber $A^{p[i]}$ of $p(!)$ over i is a bijection onto the fiber $A^{q[f_1 i]}$ of $q(!)$ over $f_1 i$. As Proposition 3.44 says that f_A sends

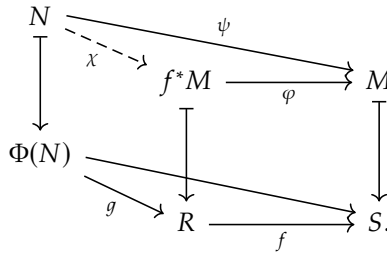
$(i, k) \mapsto (f_1 i, f_i^\sharp \circ k)$, we could restate this as saying that the function $A^{p[i]} \rightarrow A^{q[f_i]}$ sending $k \mapsto f_i^\sharp \circ k$ is a bijection for all $i \in p(1)$. But this is true if and only if $f_i^\sharp$ is a bijection for all $i \in p(1)$. So $(3) \Leftrightarrow (1)$. $\square$

There is yet another categorical notion of cartesian morphisms that our cartesian lenses satisfy, which we will explore in the next section.

5.6 *-Bifibration over **Set**

We will now show that the functor $\mathbf{Poly} \rightarrow \mathbf{Set}$ sending $p \mapsto p(1)$ is what [Shu08] refers to as a **-bifibration*. Roughly speaking, this means that **Set** parametrizes **Poly** in a coherent way. First we need the following definitions, including yet another characterization of cartesian morphisms.

Definition 5.67 ((Op)cartesian morphisms, (op)fibrations, and bifibrations). Let $\Phi: \mathbf{A} \rightarrow \mathbf{B}$ be a functor, let $f: R \rightarrow S$ be a morphism in $\mathbf{B}$, and let $M \in \mathbf{A}$ be such that $\Phi(M) = S$. A morphism $\varphi: f^*M \rightarrow M$ in $\mathbf{A}$ is *cartesian over f* if $\Phi(\varphi) = f$ (in particular $\Phi(f^*M) = R$) and, for all $\psi: N \rightarrow M$ in $\mathbf{A}$ and $g: \Phi(N) \rightarrow R$ in $\mathbf{B}$ with $\Phi(\psi) = g \circ f$, there exists a unique $\chi: N \rightarrow f^*M$ for which $\Phi(\chi) = g$ and $\psi = \chi \circ \varphi$:



We say that Φ is a *fibration* if for all $f: R \rightarrow S$ and $M \in \mathbf{A}$ with $\Phi(M) = S$, there exists a cartesian morphism over f . If we instead consider $\Phi^{\text{op}}: \mathbf{A}^{\text{op}} \rightarrow \mathbf{B}^{\text{op}}$, cartesian morphisms in $\mathbf{A}^{\text{op}}$ are *opcartesian* in $\mathbf{A}$, and Φ is an *opfibration* if Φ^{op} is a fibration. A functor that is both a fibration and an opfibration is a *bifibration*.

Proposition 5.68. Consider the functor **Poly** $\rightarrow$ **Set** that sends $p \mapsto p(1)$. A lens $\varphi: p \rightarrow q$ is a cartesian lens in the sense of Definition 5.52 if and only if it is a cartesian morphism over its on-positions function $\varphi_1: p(1) \rightarrow q(1)$ in the sense of Definition 5.67.

Proof. In the forward direction, say $\varphi: p \rightarrow q$ is a cartesian lens in the sense of Definition 5.52. Given a lens $\psi: r \rightarrow q$ and a function $g: r(1) \rightarrow p(1)$ with $\psi_1 = g \circ \varphi_1$, we wish to show there is a unique $\chi: r \rightarrow p$ for which $\chi_1 = g$ and $\psi = \chi \circ \varphi$. The on-positions function of such a lens χ is necessarily g ; then for each $k \in r(1)$, the on-directions function $\chi_k^\sharp: p[gk] \rightarrow r[k]$ must satisfy $\psi_k^\sharp = \varphi_{gk}^\sharp \circ \chi_k^\sharp$. But φ is cartesian, so $\varphi_{gk}^\sharp$ is invertible, and $\chi_k^\sharp$ is uniquely determined.

Conversely, say $\varphi: p \rightarrow q$ is cartesian over its on-positions function in the sense of Definition 5.67. By the proof of Proposition 5.53, we can factor $\varphi = \nu \circ \psi$ with $\nu: p \rightarrow \sum_{i \in p(1)} y^{q[\varphi_1 i]}$ vertical (with $\nu_1 = \text{id}_{p(1)}$) and $\psi: \sum_{i \in p(1)} y^{q[\varphi_1 i]} \rightarrow q$ cartesian in the sense of Definition 5.52 (with every on-directions function an identity), so that $\psi_1 = \varphi_1 = \text{id}_{p(1)} \circ \varphi_1$. Then there exists a unique $\chi: \sum_{i \in p(1)} y^{q[\varphi_1 i]} \rightarrow p$ for which $\chi_1 = \text{id}_1$ and $\psi = \chi \circ \varphi$; equivalently, for each $i \in p(1)$, there is a unique choice of $\chi_i^\sharp$ such that $\varphi_i^\sharp \circ \chi_i^\sharp = \psi_i^\sharp = \text{id}_{p[i]}$. The existence of such $\chi_i^\sharp$ implies that $\varphi_i^\sharp$ is injective, while uniqueness implies that $\varphi_i^\sharp$ is surjective. $\square$

Proposition 5.69. The functor **Poly** $\rightarrow$ **Set** sending $p \mapsto p(1)$ is a fibration.

Proof. Given a function $f: I \rightarrow J$ and a polynomial q with $q(1) = J$, by Proposition 5.68, it suffices to show that there exists a polynomial f^*q with position-set I and a cartesian lens $f^*q \rightarrow q$ with on-positions function f . Viewing I and J as constant polynomials, we can view $f: I \rightarrow J$ as a cartesian lens between them. Then we can take f^*q to be the pullback

$$\begin{array}{ccc} f^*q & \longrightarrow & q \\ \downarrow & \lrcorner & \downarrow \\ I & \xrightarrow{f} & J, \end{array}$$

where the arrow on the right is the vertical lens $q \rightarrow J$ that is the identity on positions. By Proposition 5.59, the top arrow is cartesian and the left arrow is vertical; indeed, we can ensure the on-positions function of the left arrow is the identity so that $(f^*q)(1) = I$ and for each $i \in I$,

$$(f^*q)[i] \cong q[fi]. \quad (5.70)$$

□

Remark 5.71. This also justifies our usage of the term *vertical* in Definition 5.52: vertical morphisms are precisely those that fibrations send to isomorphisms.

A classical result due to Grothendieck establishes an equivalence between fibrations over a category **B** and what are called *pseudofunctors* from $\mathbf{B}^{\text{op}}$ to the category of small categories **Cat**. The full details are beyond the scope of this text, but the key takeaways are as follows.

Given a fibration $\Phi: \mathbf{A} \rightarrow \mathbf{B}$, each object $R \in \mathbf{B}$ gives rise to a subcategory $\mathbf{A}_R$ of $\mathbf{A}$ consisting of all objects that Φ sends to R and all morphisms that Φ sends to id_R . Then using our notation from Definition 5.67, each morphism $f: R \rightarrow S$ in $\mathbf{B}$ gives rise to a *base change* functor $f^*: \mathbf{A}_S \rightarrow \mathbf{A}_R$ as follows.

On objects, f^* sends $M \in \mathbf{A}_S$ to f^*M . Then on morphisms, given $h: M' \rightarrow M$ in $\mathbf{A}_S$, we have cartesian morphisms $\varphi: f^*M \rightarrow M$ and $\varphi': f^*M' \rightarrow M'$ over f , as in the following commutative square in $\mathbf{A}$ and its image via Φ in $\mathbf{B}$. As φ is cartesian over f , we have a unique morphism $f^*h: f^*M' \rightarrow f^*M$ making the top square commute (functoriality follows from uniqueness):

$$\begin{array}{ccccc}
 f^*M' & \xrightarrow{\varphi'} & M' & & \\
 \downarrow & \nearrow f^*h & \downarrow & \searrow h & \\
 f^*M & \xrightarrow{\varphi} & M & & \\
 \downarrow & & \downarrow & & \\
 R & \xrightarrow{f} & S & \xrightarrow{\text{id}_S} & S \\
 \parallel \text{id}_R & & \downarrow & & \downarrow \\
 R & \xrightarrow{f} & S & &
 \end{array} \quad (5.72)$$

Applying this to our fibration $\mathbf{Poly} \rightarrow \mathbf{Set}$ yields the following constructions.

Definition 5.73. For $I \in \mathbf{Set}$, let $\mathbf{Poly}_I$ denote the category whose objects are vertical lenses $p \rightarrow I$ whose on-positions functions are identities (equivalently, $p \in \mathbf{Poly}$ with $p(1) = I$) and whose morphisms are commutative triangles in $\mathbf{Poly}$ (equivalently, lenses whose on-positions functions are identities).

So $\mathbf{Poly}_I$ is the subcategory of $\mathbf{Poly}$ that our fibration $\mathbf{Poly} \rightarrow \mathbf{Set}$ mapping $p \mapsto p(1)$ sends to $I \in \mathbf{Set}$. Note that given $p, p' \in \mathbf{Poly}_I$, so that $p(1) = p'(1) = I$, a lens $p \rightarrow p'$ in $\mathbf{Poly}_I$ necessarily has id_I as its on-positions function and is determined by independent choices of on-directions functions $p'[i] \rightarrow p[i]$ for each $i \in I$. Hence

$$\mathbf{Poly}_I(p, p') \cong \prod_{i \in I} \mathbf{Set}(p'[i], p[i]). \quad (5.74)$$

Proposition 5.75 (Base change). For any function $f: I \rightarrow J$, pull-back along f induces a functor $f^*: \mathbf{Poly}_J \rightarrow \mathbf{Poly}_I$.

Proof. Viewing f as a lens, this follows from Proposition 5.59. □

Applying Propositions 5.68 and 5.69, we see this coincides with our notation in and above (5.72) for our fibration $\mathbf{Poly} \rightarrow \mathbf{Set}$ mapping $p \mapsto p(1)$. For a function $f: I \rightarrow J$ and a lens $h: q' \rightarrow q$ in $\mathbf{Poly}_J$ (so $q'(1) = q(1) = J$), we see that (5.72) becomes the commutative cube

$$\begin{array}{ccccc}
 f^*q' & \xrightarrow{\quad} & q' & & \\
 \downarrow & \nearrow f^*h & \downarrow & \searrow h & \\
 I & \xrightarrow{f} & J & \xrightarrow{\text{id}_J} & J \\
 \parallel \text{id}_I & & \downarrow & & \downarrow \\
 I & \xrightarrow{f} & J & &
 \end{array}$$

where the vertical arrows are all vertical lenses that are identities on positions and the front and back faces are both pullback squares. Explicitly, f^*h is given by id_I on positions and, for each $i \in I$,

$$(f^*q')[i] \cong q'[fi] \xrightarrow{h_{fi}^\#} q[fi] \cong (f^*q)[i]$$

on directions (the isomorphisms come from (5.70)).

Another result about fibrations we will invoke without proof is that a fibration is also an opfibration if and only if all of its base change functors f^* have left adjoints $f_!$, which are then the base change functors for the opposite fibration. When the base change functors have right adjoints as well, we have what [Shu08] calls a **-bifibration*. It turns out that our fibration $\mathbf{Poly} \rightarrow \mathbf{Set}$ is such a *-bifibration.

Theorem 5.76. Given a function $f: I \rightarrow J$, the base change functor from Proposition 5.75 has both a left and a right adjoint, making the functor $\mathbf{Poly} \rightarrow \mathbf{Set}$ sending $p \mapsto p(1)$ a *-bifibration:

$$\begin{array}{ccc} & \xrightarrow{f_!} & \\ & \Rightarrow & \\ \mathbf{Poly}_I & \xleftarrow{f^*} & \mathbf{Poly}_J \\ & \Leftarrow & \\ & \xrightarrow{f_*} & \end{array} \quad (5.77)$$

Proof. Given $p \in \mathbf{Poly}_I$ (so $p(1) = I$), we define $f_!p$ and f_*p in $\mathbf{Poly}_J$ by

$$f_!p := \sum_{j \in J} y^{\prod_{i \in f^{-1}(j)} p[i]} \quad \text{and} \quad f_*p := \sum_{j \in J} y^{\sum_{i \in f^{-1}(j)} p[i]}. \quad (5.78)$$

Then given $q \in \mathbf{Poly}_J$ (so $q(1) = J$), we have

$$\mathbf{Poly}_I(p, f^*q) \cong \prod_{i \in I} \mathbf{Set}((f^*q)[i], p[i]) \quad (5.74)$$

$$\cong \prod_{i \in I} \mathbf{Set}(q[f i], p[i]) \quad (5.70)$$

$$\cong \prod_{j \in J} \prod_{i \in f^{-1}(j)} \mathbf{Set}(q[j], p[i])$$

$$\begin{aligned} &\cong \prod_{j \in J} \mathbf{Set}\left(q[j], \prod_{i \in f^{-1}(j)} p[i]\right) \\ &\cong \mathbf{Poly}_J(f_!p, q) \end{aligned} \quad (5.74)$$

and

$$\mathbf{Poly}_I(f^*q, p) \cong \prod_{i \in I} \mathbf{Set}(p[i], (f^*q)[i]) \quad (5.74)$$

$$\cong \prod_{i \in I} \mathbf{Set}(p[i], q[f i]) \quad (5.70)$$

$$\begin{aligned} &\cong \prod_{j \in J} \prod_{i \in f^{-1}(j)} \mathbf{Set}(p[i], q[j]) \\ &\cong \prod_{j \in J} \mathbf{Set}\left(\sum_{i \in f^{-1}(j)} p[i], q[j]\right) \\ &\cong \mathbf{Poly}_J(q, f_* p). \end{aligned} \quad (5.74)$$

□

Given a function $f: I \rightarrow J$ and a polynomial p with $p(1) = I$, the corresponding opcartesian lens $p \rightarrow f!p$ is given by composing the adjunction unit $p \rightarrow f^* f!p$ with the cartesian lens $f^* f!p \rightarrow f!p$ over f . We have

$$f^* f!p \cong \sum_{i \in I} y^{i' \in f^{-1}(fi)} \prod_{p[i']};$$

the unit is id_I on positions and the canonical projection $\prod_{i' \in f^{-1}(fi)} p[i'] \rightarrow p[i]$ for each $i \in I$ on directions, while the cartesian lens is f on positions and identities on directions. It follows that the opcartesian lens $p \rightarrow f!p$ is given by f on positions and projections on directions.

5.7 Cartesian lenses are exponentiable

We conclude this chapter with a discussion of what are called *exponentiable* morphisms in **Poly**. To introduce these, we first need the following definition.

Definition 5.79 ((Co)slice category). Given an object c in a category $\mathbf{C}$, the *slice* (resp. *coslice*) *category* of $\mathbf{C}$ over (resp. *under*) c , denoted $\mathbf{C}/c$ (resp. $c/\mathbf{C}$), is the category whose objects are morphisms in $\mathbf{C}$ with codomain (resp. *domain*) c and whose morphisms are commutative triangles in $\mathbf{C}$. By convention, we often refer to an object in $\mathbf{C}/c$ (resp. $c/\mathbf{C}$) only by its domain (resp. codomain) in $\mathbf{C}$ when the morphism is implied.

Example 5.80. Fix a polynomial $p \in \mathbf{Poly}$ and consider two objects $f: q \rightarrow p$ and $g: r \rightarrow p$ in $\mathbf{Poly}/p$. Their hom-set $(\mathbf{Poly}/p)(q, r)$ consists of lenses $h: q \rightarrow r$ for which $f = h \circ g$; equivalently, $f_1 = h_1 \circ g_1$ and $f_j^\# = g_{h_1 j}^\# \circ h_j^\#$ for all $j \in q(1)$. On positions, h should send each $j \in q(1)$ to some $k \in g_1^{-1}(f_1 j)$; then for each $j \in q(1)$, the on-directions function $h_j^\#$ can be thought of as an element of the hom-set $(p[f_1 j]/\mathbf{Set})(r[k], q[j])$. Hence

$$(\mathbf{Poly}/p)(q, r) \cong \prod_{j \in q(1)} \sum_{k \in g_1^{-1}(f_1 j)} (p[f_1 j]/\mathbf{Set})(r[k], q[j]). \quad (5.81)$$

Example 5.82 (Slice category over terminal object). Let 1 be the terminal object of a category $\mathbf{C}$. The universal property of the terminal object implies that the slice category $\mathbf{C}/1$ is equivalent to the original category $\mathbf{C}$ itself.

Example 5.83 (Slice category over y). We characterize the slice category $\mathbf{Poly}/y$ as follows. Given $p \in \mathbf{Poly}$ and a lens $\varphi: p \rightarrow y$, each on-directions function of φ selects a $p[i]$ -direction $\varphi_i^\# 1$ at each p -position i ; in particular, each position has at least one direction. Then we can construct a polynomial

$$p' := \sum_{i \in p(1)} y^{p[i] - \{\varphi_i^\# 1\}}$$

so that $p'y \cong p$, and the lens $p'y \cong p \xrightarrow{\varphi} y$ is the canonical projection. Then given a second polynomial $q \in \mathbf{Poly}$ and a lens $\psi: q \rightarrow y$, a morphism from $q \rightarrow p$ in $\mathbf{Poly}/y$ is given by a lens $f: q \rightarrow p \cong p'y$ in $\mathbf{Poly}/y$ satisfying $f \circ \varphi = \psi$. But by the universal property of the product $p'y$, a morphism $q \rightarrow p'y$ is equivalent to the pair of morphisms $q \rightarrow p'$ and $q \rightarrow y$ obtained by composing with the canonical projections. Since the latter is already specified to be equal to ψ , we can conclude that a morphism $q \rightarrow p$ in $\mathbf{Poly}/y$ is

equivalent to a lens $q \rightarrow p'$ in **Poly**. That is, we have the following natural isomorphism:

$$(\mathbf{Poly}/y)(q, p'y) \cong \mathbf{Poly}(q, p'). \quad (5.84)$$

Definition 5.85 (Exponentiable morphism). Given a category **C** with objects c, d and morphism $f: c \rightarrow d$ such that all pullbacks along f exist in **C**, we say that f is *exponentiable* if the functor $f^*: \mathbf{C}/d \rightarrow \mathbf{C}/c$ given by pulling back along f is a left adjoint.

Theorem 5.86. Cartesian lenses in **Poly** are exponentiable. That is, if $f: p \rightarrow q$ is cartesian, then the functor $f^*: \mathbf{Poly}/q \rightarrow \mathbf{Poly}/p$ given by pulling back along f is a left adjoint:

$$\mathbf{Poly}/p \begin{array}{c} \xleftarrow{f^*} \\ \Leftarrow \\ \xrightarrow{f^*} \end{array} \mathbf{Poly}/q.$$

Proof. Fix $e: p' \rightarrow p$ and $g: q' \rightarrow q$. We need to define a functor $f_*: \mathbf{Poly}/p \rightarrow \mathbf{Poly}/q$ and prove the corresponding isomorphism establishing it as right adjoint to f^* . We do so as follows:

$$f_*p' := \sum_{j \in q(1)} \sum_{\substack{\bar{i}' \in \prod_{i \in f_1^{-1}(j)} e_1^{-1}(i)}} q[j] / \sum_{i \in f_1^{-1}(j)} p'[\bar{i}'i] \quad y. \quad (5.87)$$

Here $q[j] / \sum_{i \in f_1^{-1}(j)} p'[\bar{i}'i]$ is the coproduct of the objects $q[j] \rightarrow p'[\bar{i}'i]$ in $q[j]/\mathbf{Set}$, or equivalently their wide pushout in **Set**. We equip f_*p' with a lens to q that on positions sends $(j, \bar{i}') \mapsto j$ and on directions is the canonical map $q[j] \rightarrow q[j] / \sum_{i \in f_1^{-1}(j)} p'[\bar{i}'i]$ at $(j, \bar{i}')$.

By Example 5.39, a f^*q' -position is a pair $(j', i) \in q'(1) \times p(1)$ with $g_1j' = f_1i$; equivalently, it is some $j' \in q'(1)$ and some $i \in f_1^{-1}(g_1j')$. Then for each such pair (j', i) , the direction-set $(f^*q')[j', i]$ is the pushout $q'[j'] +_{q[f_1i]} p[i]$. So by (5.81), the hom-set $(\mathbf{Poly}/p)(f^*q', p')$ is given by

$$\prod_{j' \in q'(1)} \prod_{i \in f_1^{-1}(g_1j')} \sum_{i' \in e_1^{-1}(i)} (p[i]/\mathbf{Set})(p'[\bar{i}'], q'[j'] +_{q[f_1i]} p[i]).$$

As f is cartesian, $q[f_1 i] \cong p[i]$ for all $i \in p(1)$, so our hom-set above is naturally isomorphic to

$$\prod_{j' \in q'(1)} \prod_{i \in f_1^{-1}(g_1 j')} \sum_{i' \in e_1^{-1}(i)} (q[f_1 i] / \mathbf{Set})(p'[i'], q'[j']);$$

by the distributive property and the fact that $g_1 j' = f_1 i$ in the inner sum, we can rewrite this as

$$\prod_{j' \in q'(1)} \prod_{\bar{i}' \in \prod_{i \in f_1^{-1}(g_1 j')} e_1^{-1}(i)} \sum_{i \in f_1^{-1}(g_1 j')} (q[g_1 j'] / \mathbf{Set})(p'[\bar{i}' i], q'[j']).$$

Finally, by the universal property of coproducts, the above set is naturally isomorphic to

$$\prod_{j' \in q'(1)} \prod_{\bar{i}' \in \prod_{i \in f_1^{-1}(g_1 j')} e_1^{-1}(i)} \sum_{i \in f_1^{-1}(g_1 j')} (q[g_1 j'] / \mathbf{Set}) \left(q[g_1 j'] / \sum_{i \in f_1^{-1}(g_1 j')} p'[\bar{i}' i], q'[j'] \right),$$

which according to (5.81) as well as how we defined $f_* p'$ in (5.87) is precisely the hom-set $(\mathbf{Poly}/q)(q', f_* p')$. $\square$

Example 5.88. Let $p := 2y^2$ and $q := y^2 + y^0$, and let $f: p \rightarrow q$ be the cartesian lens between them whose on-directions functions are each the identity on 2. Then (5.87) provides the following description for the pushforward $f_* p'$ for any $e: p' \rightarrow p$.

Over the $j := 1$ position of q with $q[1] = 2$, we have $f_1^{-1}(1) = 2$ with $p[1] = p[2] = 2$, so

$$\prod_{i \in f_1^{-1}(1)} e_1^{-1}(i) \cong e_1^{-1}(1) \times e_1^{-1}(2).$$

For $(i'_1, i'_2) \in e_1^{-1}(1) \times e_1^{-1}(2)$, we have that $q[1] / \sum_{i \in f_1^{-1}(2)} p'[i'_i] \cong$

$X_{(i'_1, i'_2)}$ in the following pushout square:

$$\begin{array}{ccc} 2 & \xrightarrow{e_{i'_1}^\#} & p'[i'_1] \\ e_{i'_2}^\# \downarrow & \lrcorner & \downarrow \\ p'[i'_2] & \longrightarrow & X_{i'}. \end{array}$$

Over the $j := 2$ position with $q[2] = 0$, we have $f_1^{-1}(2) = 0$, so we have an empty product

$$\prod_{i \in f_1^{-1}(2)} e_1^{-1}(i) \cong 1$$

and an empty pushout $q[2]/\sum_{i \in f_1^{-1}(2)} p'[i'_i]$. Hence the corresponding summand of (5.87) is $y^0 \cong 1$.

Summing these up, we obtain

$$f_* p' \cong \left(\sum_{(i'_1, i'_2) \in e_1^{-1}(1) \times e_2^{-1}(2)} y^{X_{(i'_1, i'_2)}} \right) + 1.$$

Exercise 5.89. Using Example 5.83, prove that the unique lens $f: y \rightarrow 1$ is exponentiable.

Exercise 5.90. In this exercise, we will show that the unique lens $f: y \rightarrow y^2$ is *not* exponentiable. Assume for the sake of contradiction that it is exponentiable: that is, the pullback functor $f^*: \mathbf{Poly}/y^2 \rightarrow \mathbf{Poly}/y$ has a left adjoint $f_*: \mathbf{Poly}/y \rightarrow \mathbf{Poly}/y^2$. The following argument comes from [ALS10].

1. Let $\varphi \in \mathbf{Poly}/y$ be y^2 equipped with the lens to y that is uniquely determined on positions and picks out $1 \in 2$ on directions, and let $\psi \in \mathbf{Poly}/y^2$ be y^2 equipped with the identity lens. Show that

$$(\mathbf{Poly}/y^2)(\psi, f_* \varphi) \cong 1. \quad (5.91)$$

2. Let $\chi \in \mathbf{Poly}/y^2$ be y^3 equipped with the lens to y^2 that is uniquely determined on positions and sends $1 \mapsto 1$ and $2 \mapsto 3$ on directions. Show that

$$(\mathbf{Poly}/y^2)(\chi, f_*\varphi) \cong 2. \quad (5.92)$$

3. Use (5.91) and (5.92) to derive a contradiction. (Hint: start by characterizing what kind of polynomial equipped with a lens to y^2 the object $f_*\varphi$ must be for (5.91) to hold.)

5.8 Summary and further reading

In this chapter, we discussed several categorical properties of **Poly**: it has various adjunctions with **Set** and **Set**^{op}, including a $*$ -bifibration to **Set**; it is cartesian closed; it has limits and colimits; and it has both an epi-mono factorization system and a vertical-cartesian factorization system.

The principal monomial functor $p \mapsto p(1)y^{\Gamma p}$ discussed in Corollary 5.15 is in fact distributive monoidal and comes up in work on entropy [Spi22] and on noncooperative strategic games [Cap22].

5.9 Exercise solutions

Solution to Exercise 5.1.

Here $I, J, A, B \in \mathbf{Set}$.

1. We determine whether each class of polynomials is closed under coproducts.
 - a) Constants are closed under coproducts: given constants I, J , their coproduct $I + J$ is also a constant.
 - b) Linear polynomials are closed under coproducts: given linear polynomials Iy, Jy , their coproduct $Iy + Jy \cong (I + J)y$ is also linear.
 - c) Representables are *not* closed under coproducts: for example, y is a representable, but the coproduct of y with itself, $2y$, is not.
 - d) Monomials are *not* closed under addition: for example, y and $2y^3$ are monomials, but their coproduct $y + 2y^3$ is not.
2. We determine whether each class of polynomials is closed under products.
 - a) Constants are closed under products: given constants I, J , their product IJ is also a constant.
 - b) Linear polynomials are *not* closed under products: for example, y and $2y$ are linear polynomials, but their product $2y^2$ is not.
 - c) Representables are closed under products: given representables y^A, y^B , their product y^{A+B} is also a representable.

- d) Monomials are closed under products: given monomials Iy^A, Jy^B , their product IJy^{A+B} is also a monomial.
3. We determine whether each class of polynomials is closed under parallel products.
- a) Constants are closed under parallel products: given constants I, J , their parallel product IJ is also a constant.
 - b) Linear polynomials are closed under parallel products: given linear polynomials Iy, Jy , their parallel product IJy is also linear.
 - c) Representables are closed under parallel products: given representables y^A, y^B , their parallel product y^{A+B} is also a representable.
 - d) Monomials are closed under parallel products: given monomials Iy^A, Jy^B , their parallel product IJy^{A+B} is also a monomial.
4. We determine whether each class of polynomials is closed under composition.
- a) Constants are closed under composition: given constants I, J , their composite $I \circ J \cong I$ is also a constant.
 - b) Linear polynomials are closed under composition: given linear polynomials Iy, Jy , their composite $Iy \circ Jy \cong I(Jy) \cong IJy$ is also linear.
 - c) Representables are closed under composition: given representables y^A, y^B , their composite $y^A \circ y^B \cong (y^B)^A \cong y^{BA}$ is also a representable.
 - d) Monomials are closed under composition: given monomials Iy^A, Jy^B , their composite $Iy^A \circ Jy^B \cong I(Jy^B)^A \cong IJ^A y^{BA}$ is also a monomial.

Solution to Exercise 5.6.

1. By (3.7), we have the natural isomorphism

$$\begin{aligned} \mathbf{Poly}(I, q) &\cong \prod_{i \in I} \sum_{j \in q(1)} 0^{q[j]} \\ &\cong \prod_{i \in I} q(0) \\ &\cong \mathbf{Set}(I, q(0)). \end{aligned}$$

2. By (3.7), we have the natural isomorphism

$$\begin{aligned} \mathbf{Poly}(q, I) &\cong \prod_{j \in q(1)} \sum_{i \in I} q[j]^0 \\ &\cong \prod_{j \in q(1)} \sum_{i \in I} 1 \\ &\cong \prod_{j \in q(1)} I \\ &\cong \mathbf{Set}(q(1), I). \end{aligned}$$

3. By (3.7), we have the natural isomorphism

$$\begin{aligned} \mathbf{Poly}(Iy, q) &\cong \prod_{i \in I} \sum_{j \in q(1)} 1^{q[j]} \\ &\cong \prod_{i \in I} q(1) \\ &\cong \mathbf{Set}(I, q(1)). \end{aligned}$$

Solution to Exercise 5.7.

Given $q \in \mathbf{Poly}$, we wish to show that $q(1)$ is in bijection with the set of functions $y \rightarrow q$. In fact, this follows directly from the Yoneda lemma, but we can also invoke the isomorphism from Exercise 5.6 #3 with $I := 1$ to observe that

$$\mathbf{Poly}(y, q) \cong \mathbf{Set}(1, q(1)) \cong q(1).$$

Solution to Exercise 5.11.

To prove Corollary 5.10, it suffices to exhibit a natural isomorphism

$$\mathbf{Poly}(Iy^A, q) \cong \mathbf{Set}(I, q(A)).$$

Taking $p := y^A$ in (5.9) yields the natural isomorphism

$$\mathbf{Poly}(Iy^A, q) \cong \mathbf{Set}(I, \mathbf{Poly}(y^A, q)).$$

By the Yoneda lemma, $\mathbf{Poly}(y^A, q)$ is naturally isomorphic to $q(A)$, yielding the desired result.

Solution to Exercise 5.14.

Taking $I := A$ and $q := y$ in the second isomorphism in (5.9) yields the natural isomorphism

$$\mathbf{Set}(A, \mathbf{Poly}(p, y)) \cong \mathbf{Poly}(p, y^A).$$

As $\Gamma(p) = \mathbf{Poly}(p, y)$, this yields the desired result.

Solution to Exercise 5.17.

We have the following chain of natural isomorphisms involving global sections:

$$\Gamma(p \otimes q) = \mathbf{Poly}(p \otimes q, y) \tag{3.37}$$

$$\cong \mathbf{Poly}(p, [q, y]) \tag{4.87}$$

$$\cong \mathbf{Poly}(p, \Gamma(q)y^{q(1)}) \tag{4.83}$$

$$\cong \mathbf{Set}(p(1), \Gamma(q)) \times \mathbf{Set}(q(1), \Gamma(p)). \tag{5.16}$$

Solution to Exercise 5.20.

We are given a monomorphism $f: 12y^{12} \rightarrow \mathbb{N}y^{\mathbb{N}}$ from Example 5.19. Let $g: \mathbb{N}y^{\mathbb{N}} \rightarrow p$ be a dynamical system with return function $g_1: \mathbb{N} \rightarrow p(1)$ and update functions $g_n^\sharp: p[g_1 n] \rightarrow \mathbb{N}$ for each state $n \in \mathbb{N}$. Then the new composite dynamical system $h := f \circ g$ has a return function $h_1: 12 \rightarrow p(1)$ which sends each state $i \in 12$ to the position $h_1 i = g_1 f_1 i = g_1 i$, the same position that the original system returns in the state $i \in \mathbb{N}$. Meanwhile, the update function for each state $i \in 12$ is a function $h_i^\sharp: p[g_1 i] \rightarrow 12$ which, given a direction $a \in p[g_1 i]$, updates the state from i to $h_i^\sharp a = f_{g_1 i}^\sharp g_i^\sharp a = g_i^\sharp a \pmod{12}$, which is where the original system would have taken the same state, but reduced modulo 12. In other words, the new system behaves like the old system but with only the states in $12 \subseteq \mathbb{N}$ retained, and on any input that would have caused the old system to move to a state outside of 12, the new system moves to the equivalent state (modulo 12) within 12 instead.

Solution to Exercise 5.22.

1. We give an example lens $\{1\}y^{\{5\}} + \{2\}y^{\{6,7\}} \rightarrow y^{\{3,4\}}$ as follows. Its on-positions function is the unique function from 2 to a singleton; its on-directions function at 1 sends both 3 and 4 to 5, while its on-directions function at 2 sends $3 \mapsto 6$ and $4 \mapsto 7$. Then its on-positions function is epic, while its induced function

$$\{3, 4\} \rightarrow \{5\} \times \{6, 7\}$$

sends $3 \mapsto (5, 6)$ and $4 \mapsto (5, 7)$, so it is monic; thus, by Proposition 5.21, this lens is epic. But its on-directions function at 1 is not monic.

2. We show that if a lens $f: p \rightarrow q$ has an epic on-positions function and monic on-directions functions, then f is epic in **Poly**. By Proposition 5.21, it suffices to show that if $f_1: p(1) \rightarrow q(1)$ is epic and $f_i^\sharp: q[f_1 i] \rightarrow p[i]$ is monic for all $i \in p(1)$, then given $j \in q(1)$, the induced function

$$f_j^b: q[j] \rightarrow \prod_{\substack{i \in p(1), \\ f_1 i = j}} p[i]$$

is monic. Suppose we are given a set A and a pair of functions $g', h': A \rightrightarrows q[j]$ with

$$g' \circ f_j^b = h' \circ f_j^b. \quad (5.93)$$

As f_1 is epic, there exists $i \in p(1)$ for which $f_1 i = j$, and composing f_j^b with the canonical projection from its codomain to that $p[i]$ yields the on-directions function $f_i^\sharp: q[j] \rightarrow p[i]$. Then composing both sides of (5.93) with that same projection yields the equation $g' \circ f_i^\sharp = h' \circ f_i^\sharp$. As $f_i^\sharp$ is monic, it follows that $g' = h'$, so f_j^b is monic as well.

Solution to Exercise 5.23.

Given $p \in \mathbf{Poly}$ and a lens $f: p \rightarrow y$, we will use Proposition 5.21 to show that either f is epic or $p = 0$. First, note that $f_1: p(1) \rightarrow 1$ must be epic unless $p(1) \cong 0$, in which case $p = 0$. Next, note that the induced function

$$f^b: 1 \rightarrow \prod_{i \in p(1)} p[i]$$

must be monic. So it follows from Proposition 5.21 that either f is epic or $p = 0$.

Solution to Exercise 5.24.

By Proposition 5.21, a lens $f: y^A + y^B \rightarrow y^{AB}$ is epic if its on-positions function $f_1: 2 \rightarrow 1$ is epic (which must be true) and if the induced function

$$f^b: AB \rightarrow \prod_{i \in 2} (y^A + y^B)[i] \cong AB$$

is monic. If we take the on-directions functions $AB \rightarrow A$ and $AB \rightarrow B$ of f to be the canonical projections, then the induced function $f^b: AB \rightarrow AB$ would be the identity, which is indeed monic. So f would be epic.

Solution to Exercise 5.25.

Let $f: p \rightarrow q$ be a lens in **Poly** that is both monic and epic. We claim that f is an isomorphism. By Proposition 5.18 and Proposition 5.21, the on-positions function $f_1: p(1) \rightarrow q(1)$ is both monic and epic, so it is an isomorphism. Meanwhile, Proposition 5.21 says that, for each $j \in q(1)$, the induced function

$$f_j^b: q[j] \rightarrow \prod_{\substack{i \in p(1), \\ f_1 i = j}} p[i]$$

is monic. As f_1 is an isomorphism, it follows that for each $i \in p(1)$, the function

$$f_{f_1 i}^b: q[f_1 i] \rightarrow p[i]$$

is monic. But this is just the on-directions function $f_i^\#$ of f . From Proposition 5.18, we also know that $f_i^\#$ is epic. It follows that every on-directions function of f is an isomorphism. Hence f itself is an isomorphism.

Solution to Exercise 5.31.

We use (5.29) to compute various exponentials. Here $p \in \mathbf{Poly}$ and $A, B \in \mathbf{Set}$.

1. We have that p^0 is an empty product, so $p^0 \cong 1$ as expected.
2. We have that $p^1 \cong p \circ (y + 0) \cong p$, as expected.
3. We have that $1^p \cong \prod_{i \in p(1)} 1 \circ (y + p[i]) \cong 1$, as expected.
4. We have that $A^p \cong \prod_{i \in p(1)} A \circ (y + p[i]) \cong A^{p(1)}$.
5. We have that $y^y \cong y \circ (y + 1) \cong y + 1$.
6. We have that $y^{4y} \cong \prod_{j \in 4} y \circ (y + 1) \cong (y + 1)^4 \cong y^4 + 4y^3 + 6y^2 + 4y + 1$.
7. We have that $(y^A)^{y^B} \cong (y^A) \circ (y + B) \cong (y + B)^A \cong \sum_{f: A \rightarrow 2} B^{f^{-1}(1)} y^{f^{-1}(2)}$.

Solution to Exercise 5.33.

By Theorem 5.32, there is a natural isomorphism

$$\mathbf{Poly}(p^q, p^q) \cong \mathbf{Poly}(p^q \times q, p).$$

Under this isomorphism, there exists a lens $p^q \times q \rightarrow p$ corresponding to the identity lens on p^q that is the canonical evaluation lens.

Solution to Exercise 5.35.

1. The lens $e: p' \rightarrow p$ can be characterized as follows. The on-positions function $e_1: p'(1) \rightarrow p(1)$ is the equalizer of $f_1, g_1: p(1) \rightrightarrows q(1)$ in **Set**. In particular, e_1 is the canonical inclusion that sends each element of $p'(1)$ to the same element in $p(1)$. Then for each $i \in p'(1)$, the on-directions function $e_i^\#: p[i] \rightarrow p'[i]$ is the coequalizer of $f_i^\#, g_i^\#: q[f_1 i] \rightrightarrows p[i]$ in **Set**.
2. To show that $e \circ f = e \circ g$, it suffices to show that both sides are equal on positions and on directions. On positions, e_1 is defined to be the equalizer of f_1 and g_1 , so $e_1 \circ f_1 = e_1 \circ g_1$. Then for each $i \in p'(1)$, the on-directions function $e_i^\#$ is defined to be the coequalizer of $f_i^\#$ and $g_i^\#$, so $f_i^\# \circ e_i^\# = g_i^\# \circ e_i^\#$.

3. To show that e is the equalizer of f and g , it suffices to show that for any $r \in \mathbf{Poly}$ and lens $a: r \rightarrow p$ satisfying $a \circ f = a \circ g$, there exists a unique lens $h: r \rightarrow p'$ for which $a = h \circ e$, so that the following diagram commutes.

$$\begin{array}{ccc} p' & \xrightarrow{e} & p \\ \uparrow h & \nearrow a & \downarrow f \\ r & & p \end{array} \quad \begin{array}{c} \xrightarrow{f} \\ \xrightarrow{g} \end{array} q$$

In order for $a = h \circ e$ to hold, we must have $a_1 = h_1 \circ e_1$ on positions. But we have that $a_1 \circ f_1 = a_1 \circ g_1$, so by the universal property of $p'(1)$ and the map e_1 as the equalizer of f_1 and g_1 in **Set**, there exists a unique h_1 for which $a_1 = h_1 \circ e_1$. Hence h is uniquely characterized on positions. In particular, it must send each $k \in r(1)$ to $a_1(k) \in p'(1)$.

Then for $a = h \circ e$ to hold on directions, we must have that $a_k^\# = e_{a_1 k}^\# \circ h_k^\#$ for each $k \in r(1)$. But we have that $f_{a_1 k}^\# \circ a_{a_1 k}^\# = g_{a_1 k}^\# \circ a_{a_1 k}^\#$, so by the universal property of $p'[a_1 k]$ and the map $e_{a_1 k}^\#$ as the coequalizer of $f_{a_1 k}^\#$ and $g_{a_1 k}^\#$ in **Set**, there exists a unique $h_k^\#$ for which $a_k^\# = e_{a_1 k}^\# \circ h_k^\#$, so that the diagram below commutes.

$$\begin{array}{ccccc} p'[a_1 k] & \xleftarrow{e_{a_1 k}^\#} & p[a_1 k] & \xleftarrow[f_{a_1 k}^\#]{g_{a_1 k}^\#} & q[f_1 a_1 k] \\ \downarrow h_k^\# & & \swarrow a_k^\# & & \\ r[k] & & & & \end{array}$$

Hence h is also uniquely characterized on directions, so it is unique overall. Moreover, we have shown that we can define h on positions so that $a_1 = h_1 \circ e_1$, and that we can define h on directions such that $a_k^\# = e_{a_1 k}^\# \circ h_k^\#$ for all $k \in r(1)$. It follows that there exists h for which $a = h \circ e$.

Solution to Exercise 5.41.

Here $p \in \mathbf{Poly}$.

1. The canonical lens $\eta: p \rightarrow p(1)$ is the identity $\eta_1: p(1) \rightarrow p(1)$ on positions and the empty function on directions.
2. On positions, we have that $p_i(1)$ along with f_i and g_1 form the following pullback square in **Set**:

$$\begin{array}{ccc} p_i(1) & \xrightarrow{g_1} & p(1) \\ f_i \downarrow & \lrcorner & \parallel \\ 1 & \xrightarrow{i} & p(1) \end{array}$$

So $p_i(1) := \{(a, i') \in 1 \times p(1) \mid i = i'\} = \{(1, i)\}$, with f_i uniquely determined and g_1 picking out $i \in p(1)$. Then on directions, we have that $p_i[(1, i)]$ along with

$f_{(1,i)}^\#$ and $g_{(1,i)}^\#$ form the following pushout square in **Set**:

$$\begin{array}{ccc} p_i[(1,i)] & \xleftarrow{g_{(1,i)}^\#} & p[i] \\ f_{(1,i)}^\# \uparrow & \lrcorner & \uparrow ! \\ 0 & \xleftarrow{!} & 0 \end{array}$$

So $p_i[(1,i)] := p[i]$, with $f_{(1,i)}^\#$ uniquely determined and $g_{(1,i)}^\#$ as the identity. It follows that $p_i := \{(1,i)\}y^{p[i]} \cong y^{p[i]}$, where $f: p_i \rightarrow 1$ is uniquely determined and $g: p_i \rightarrow p$ picks out $i \in p(1)$ on positions and is the identity on $p[i]$ on directions.

Solution to Exercise 5.42.

- There are many possible answers, but one lens $f: q \rightarrow r$, on positions, sends $1 \in q(1)$ (corresponding to y^2) to $2 \in r(1)$ (corresponding to 1) and $2 \in q(1)$ (corresponding to y) to $1 \in r(1)$ (corresponding to y). Then the on-directions functions $f_1^\#: 0 \rightarrow 2$ and $f_2^\#: 1 \rightarrow 1$ are uniquely determined. Another morphism $f': q' \rightarrow r$, on positions, sends $1 \in q'(1)$ (corresponding to one of the y^3 terms) to $2 \in r(1)$ and both $2 \in q'(1)$ (corresponding to the other y^3 term) and $3 \in q'(1)$ (corresponding to the y^2 term) to $1 \in r(1)$. Then the on-directions function $(f')_1^\#: 0 \rightarrow 3$ is uniquely determined, while we can let $(f')_2^\#: 1 \rightarrow 3$ pick out 3 and $(f')_3^\#: 1 \rightarrow 2$ pick out 1.
- We compute the pullback p along with the lenses $g: p \rightarrow q$ and $g': p \rightarrow q'$ of $q \xrightarrow{f} r \xleftarrow{f'} q'$ by following Example 5.39. We can compute $p(1)$ by taking the pullback in **Set**:

$$p(1) := \{(i, i') \in 2 \times 3 \mid f_1 i = f'_1(i')\} = \{(1, 1), (2, 2), (2, 3)\}.$$

Moreover, the on-positions functions g_1 and g'_1 send each pair in $p(1)$ to its left component and its right component, respectively.

To compute the direction-set at each p -position, we must compute a pushout. At $(1, 1)$, we have $r[f_1(1)] = r[f'_1(1)] = r[2] = 0$, so the pushout $p[(1, 1)]$ is just the sum $q[1] + q'[1] = 2 + 3 \cong 5$. Moreover, the on-directions functions $g_{(1,1)}^\#$ and $(g')_{(1,1)}^\#$ are the canonical inclusions $2 \rightarrow 2 + 3$ and $3 \rightarrow 2 + 3$.

At $(2, 2)$, we have $r[f_1(2)] = r[f'_1(2)] = r[1] = 1$, with $f_2^\#$ picking out $1 \in 1 = q[2]$ and $(f')_2^\#$ picking out $3 \in 3 = q'[2]$. So the pushout $p[(2, 2)]$ is the set $1 + 3 = \{(1, 1), (2, 1), (2, 2), (2, 3)\}$ but with $(1, 1)$ identified with $(2, 3)$; we can think of it as the set of equivalence classes $p[(2, 2)] \cong \{\{(1, 1), (2, 3)\}, \{(2, 1)\}, \{(2, 2)\}\} \cong 3$.

Moreover, the on-directions function $g_{(2,2)}^\#$ maps $1 \mapsto \{(1, 1), (2, 3)\}$, while the on-directions function $(g')_{(2,2)}^\#$ maps $1 \mapsto \{(2, 1)\}$, $2 \mapsto \{(2, 2)\}$, and $3 \mapsto \{(1, 1), (2, 3)\}$.

Finally, at $(2, 3)$, we have $r[f_1(2)] = r[f'_1(3)] = r[1] = 1$, with $f_2^\#$ still picking out $1 \in 1 = q[2]$ and $(f')_3^\#$ picking out $1 \in 2 = q'[3]$. So the pushout $p[(2, 3)]$ is the

set $1 + 2 = \{(1, 1), (2, 1), (2, 2)\}$ but with $(1, 1)$ identified with $(2, 1)$; we can think of it as the set of equivalence classes $p[(2, 3)] \cong \{\{(1, 1), (2, 1)\}, \{(2, 2)\}\} \cong 2$. Moreover, the on-directions function $g_{(2,3)}^\#$ maps $1 \mapsto \{(1, 1), (2, 1)\}$, while the on-directions function $(g')_{(2,3)}^\#$ maps $1 \mapsto \{(1, 1), (2, 1)\}$ and $2 \mapsto \{(2, 2)\}$. It follows that $p \cong y^5 + y^3 + y^2$, with g and g' as described.

Solution to Exercise 5.43.

1. By Proposition 3.44, f_X (resp. g_X) sends each $(i, h) \in p(X)$ with $i \in p(1)$ and $h: p[i] \rightarrow X$ to $(f_1 i, f_i^\# \circ h)$ (resp. $(g_1 i, g_i^\# \circ h)$) in $q(X)$. So the equalizer of f_X and g_X is the set of all $(i, h) \in p(X)$ for which both $f_1 i = g_1 i$ and $f_i^\# \circ h = g_i^\# \circ h$. Indeed, by our construction of p' , the set $p'(X)$ consists of all pairs (i, h') with $i \in p(1)$ such that $f_1 i = g_1 i$ and $h': p'[i] \rightarrow X$, where $p'[i]$ is the coequalizer of $f_i^\#, g_i^\#: q[f_1 i] \rightrightarrows p[i]$. By the universal property of the coequalizer, functions $h': p'[i] \rightarrow X$ precisely correspond to functions $h: p[i] \rightarrow X$ for which $f_i^\# \circ h = g_i^\# \circ h$. So $p'(X)$ is indeed the equalizer of f_X and g_X .
The equalizer natural transformation $e': p' \rightarrow p$ has the inclusion $e'_X: p'(X) \rightarrow p(X)$ as its X -component, so by Corollary 3.47, it is the lens whose on-positions function is the canonical equalizer inclusion $e'_1: p'(1) \rightarrow p(1)$, while its on-directions function at $i \in p'(1)$ is the map $p[i] \rightarrow p'[i]$ corresponding to the identity on $p'[i]$ given by the universal property of the coequalizer—which is just the canonical coequalizer map $p[i] \rightarrow p'[i]$. But this is exactly the lens $e: p' \rightarrow p$ constructed in the proof of Proposition 3.56, as desired.
2. By Proposition 1.37, limits—including equalizers—in **Set**^{Set} are computed pointwise. So if $e_X: p'(X) \rightarrow p(X)$ is the equalizer of $f_X, g_X: p(X) \rightrightarrows q(X)$ for every $X \in \mathbf{Set}$, then $e: p' \rightarrow p$ is the equalizer of $f, g: p(X) \rightrightarrows q(X)$.
3. We have just shown that equalizers in **Poly** coincide with equalizers in **Set**^{Set}. We saw in the proof of Proposition 3.56 that products in **Poly** also coincide with products in **Set**^{Set}. Since every limit can be computed as an equalizer of products, we can conclude that limits in **Poly** coincide with limits in **Set**^{Set}.

Solution to Exercise 5.45.

1. We define a lens $g: q \rightarrow q'$ as follows. The on-positions function $g_1: q(1) \rightarrow q'(1)$ is the coequalizer of $s_1, t_1: p(1) \rightrightarrows q(1)$. In particular, g_1 sends each vertex in $q(1)$ to its corresponding connected component in $q'(1) = C$. Then for each $v \in q(1)$, if we let its corresponding connected component be $c := g_1(v)$, we can define the on-directions function $g_v^\#: q'[c] \rightarrow q[v]$ to be the projection from the limit $q'[c]$ to its component $q[v]$.
2. To show that $s \circ g = t \circ g$, we must show that both sides are equal on positions and on directions. The on-positions function g_1 is defined to be the coequalizer of s_1 and t_1 , so $s_1 \circ g_1 = t_1 \circ g_1$. So it suffices to show that for all $e \in p(1)$, if we let its corresponding connected component be $c := g_1(s_1(e)) = g_1(t_1(e))$, then the

following diagram of on-directions functions commutes:

$$\begin{array}{ccc}
 & q[s_1(e)] & \\
 s_e^\# \swarrow & & \nwarrow g_{s_1 e}^\# \\
 p[e] & & q'[c] \\
 t_e^\# \swarrow & & \nwarrow g_{t_1 e}^\# \\
 & q[t_1(e)] &
 \end{array}$$

But this is automatically true by the definition of $q'[c]$ as a limit—specifically the limit of a functor with $s_e^\#$ and $t_e^\#$ in its image—and the definitions of $g_{s_1 e}^\#$ and $g_{t_1 e}^\#$ as projections from this limit.

3. To show that g is the coequalizer of s and t , it suffices to show that for any $r \in \mathbf{Poly}$ and lens $f: q \rightarrow r$ satisfying $s \circ f = t \circ f$, there exists a unique lens $h: q' \rightarrow r$ for which $f = g \circ h$, so that the following diagram commutes.

$$\begin{array}{ccc}
 p & \xrightarrow[s]{s} q & \xrightarrow{g} q' \\
 & \searrow f & \downarrow h \\
 & & r
 \end{array}$$

In order for $f = g \circ h$ to hold, we must have $f_1 = g_1 \circ h_1$ on positions. But we have that $s_1 \circ f_1 = t_1 \circ f_1$, so by the universal property of $q'(1)$ and the map g_1 as the coequalizer of s_1 and t_1 in **Set**, there exists a unique h_1 for which $f_1 = g_1 \circ h_1$. Hence h is uniquely characterized on positions. In particular, it must send each connected component $c \in q'(1)$ to the element in $r(1)$ to which f_1 sends every vertex $v \in V_c = g_1^{-1}(c)$ that lies in the connected component c .

Then for $f = g \circ h$ to hold on directions, we must have that $f_v^\# = h_{g_1 v}^\# \circ g_v^\#$ for each $v \in q(1)$. Put another way, given $c \in q'(1)$, we must have that $f_v^\# = h_c^\# \circ g_v^\#$ for every $v \in V_c$. But $s \circ f = t \circ f$ implies that for each $e \in E_c = s_1^{-1}(g_1^{-1}(c)) = t_1^{-1}(g_1^{-1}(c)) \subseteq p(1)$, the following diagram of on-directions functions commutes:

$$\begin{array}{ccc}
 & q[s_1(e)] & \\
 s_e^\# \swarrow & & \nwarrow f_{s_1 e}^\# \\
 p[e] & & r[f_1(v)] \\
 t_e^\# \swarrow & & \nwarrow f_{t_1 e}^\# \\
 & q[t_1(e)] &
 \end{array}$$

It follows that $r[f_1(v)]$ together with the maps $(f_v^\#)_{v \in V_c}$ form a cone over the functor F . So by the universal property of the limit $q'[c]$ of F with projection maps $(g_v^\#)_{v \in V_c}$, there exists a unique $h_c^\#: r[f_1(v)] \rightarrow q'[c]$ for which $f_v^\# = h_c^\# \circ g_v^\#$ for every $v \in V_c$. Hence h is also uniquely characterized on directions, so it is unique overall. Moreover, we have shown that we can define h on positions so that $f_1 = g_1 \circ h_1$, and that we can define h on directions such that $f_v^\# = h_c^\# \circ g_v^\#$ for all $c \in q'(1)$ and $v \in V_c$. It follows that there exists h for which $f = g \circ h$.

Solution to Exercise 5.48.

1. We characterize the lens $\epsilon: p(1)y \rightarrow p$ as follows. On positions, it is the identity on $p(1)$. Then for each $i \in p(1)$, on directions, it is the unique map $p[i] \rightarrow 1$.

2. We characterize the lens $\eta: p \rightarrow y^{\Gamma(p)}$ as follows. On positions, it is the unique map $p(1) \rightarrow 1$. Then for each $i \in p(1)$, on directions, it is the canonical projection $\Gamma(p) \cong \prod_{i' \in p(1)} p[i'] \rightarrow p[i]$.
3. Showing that (5.49) is a pushout square is equivalent to showing that, in the diagram

$$\begin{array}{ccccc}
 & & y & & \\
 & \nearrow ! & \downarrow \iota & \nwarrow ! & \\
 p(1)y & \xrightarrow{s} & y+p & \xrightarrow{g} & y^{\Gamma(p)} \\
 & \searrow \epsilon & \uparrow \iota' & \nearrow \eta & \\
 & & p & &
 \end{array} \tag{5.94}$$

in which ι, ι' are the canonical inclusions and the four triangles commute, $y^{\Gamma(p)}$ equipped with the lens g is the coequalizer of s and t . To do so, we apply Theorem 5.44 to compute the coequalizer q' of s and t . The position-set of q' is the coequalizer of $s_1 = (! \circ \iota)_1$, which sends every $i \in p(1)$ to the position of $y + p$ corresponding to the summand y , and $t_1 = (\epsilon \circ \iota')_1$, which sends each $i \in p(1)$ to the corresponding position in the summand p of $y + p$. It follows that the coequalizer of s_1 and t_1 is 1, so $q'(1) \cong 1$.

Then the direction-set of q' at its sole position is the limit of the functor F whose image consists of lenses of the form $1 \rightarrow 1$ or $p[i] \rightarrow 1$ for every $i \in p(1)$. It follows that the limit of F is just a product, namely $\prod_{i \in p(1)} p[i] \cong \Gamma(p)$. Hence $q' \cong y^{\Gamma(p)}$, as desired.

It remains to check that the upper right and lower right triangles in (5.94) commute. The upper right triangle must commute by the uniqueness of morphisms $y \rightarrow y^{\Gamma(p)}$; and the lower right triangle must commute on positions. Moreover, the on-directions function of the coequalizer morphism g at each position $i \in p(1) \subseteq (y+p)(1)$ must be the canonical projection $\Gamma(p) \rightarrow p[i]$, which matches the behavior of the corresponding on-directions function of η ; hence the lower right triangle also commutes on directions.

Solution to Exercise 5.55.

Consider the lenses $y \xrightarrow{f} y^2 + y \xrightarrow{g} y$ where f is the canonical inclusion and g is uniquely determined on positions and picks out $1 \in 2$ and $1 \in 1$ on directions. Then the only on-directions function of f is a function $1 \rightarrow 1$, an isomorphism, so f is cartesian. Meanwhile, one of the on-directions functions of g is a function $1 \rightarrow 2$, which is not an isomorphism, so g is not cartesian. Finally, $f \circ g$ can only be the unique lens $y \rightarrow y$, namely the identity, which is cartesian.

Solution to Exercise 5.57.

We already know that the square on the left-hand side of (5.56) commutes, and since its bottom arrow is an identity, it is a pullback square if and only if $f^\#$ is an isomorphism. The right pullback square tells us that the $\bullet$ is $\sum_{i \in p(1)} q[f_1 i]$. So $f_i^\#: q[f_1 i] \rightarrow p[i]$ is an isomorphism for every $i \in p(1)$ if and only if their sum $f^\#: \sum_{i \in p(1)} q[f_1 i] \rightarrow$

$\sum_{i \in p(1)} p[i] \cong \dot{p}(1)$ is an isomorphism as well. Hence f is cartesian if and only if $f^\sharp$ is an isomorphism, as desired.

Solution to Exercise 5.60.

The pushout of a cartesian lens is *not* necessarily cartesian. Take the pushout square (5.49). The lens $! : p(1)y \rightarrow y$ has $1 \rightarrow 1$ as every on-directions function, so it is cartesian, but its pushout $\eta : p \rightarrow y^{\Gamma(p)}$ is not going to be cartesian as long as there is some $i \in p(1)$ for which $\Gamma(p) \not\cong p[i]$. For instance, when $p := y + 1$, we have that $\Gamma(p) \cong 0 \not\cong 1 \cong p[1]$, so η is not cartesian.

Solution to Exercise 5.64.

One direction is immediate, so assume that the naturality squares of α induced by unique morphisms to 1 are pullbacks; it suffices to show that all naturality squares of α are pullbacks. Given a morphism $h : c' \rightarrow c$ in $\mathcal{C}$, consider the commutative diagram consisting of the naturality squares for α induced by h and by the unique morphism $! : c \rightarrow 1$:

$$\begin{array}{ccccc} Fc' & \xrightarrow{Fh} & Fc & \xrightarrow{F!} & F1 \\ \downarrow \alpha_{c'} & & \downarrow \alpha_c & \lrcorner & \downarrow \alpha_1 \\ Gc' & \xrightarrow{Gh} & Gc & \xrightarrow{G!} & G1 \end{array}$$

we know that the square on the right is a pullback square. By functoriality, the pairs of arrows on the top and bottom compose to $F(h\sharp!)$ and $G(h\sharp!)$, respectively; then by the universal property of the terminal object, $h\sharp!$ is the unique arrow $c' \rightarrow 1$. It follows that the composite rectangle is the naturality square of α induced by the unique arrow $c' \rightarrow 1$, so it is a pullback as well. Hence the square on the left is a pullback by Lemma 5.62.

Solution to Exercise 5.89.

Since 1 is terminal in **Poly**, the slice category **Poly**/1 is equivalent to **Poly** itself. So we seek a functor $f_* : \mathbf{Poly}/y \rightarrow \mathbf{Poly}$ such that for all $q' \in \mathbf{Poly}/y$ and $p \in \mathbf{Poly}$, we have a natural isomorphism $(\mathbf{Poly}/y)(f^*p, q') \cong \mathbf{Poly}(p, f_*q')$. Again because 1 is terminal in **Poly**, the pullback of $p \rightarrow 1$ along $f : y \rightarrow 1$ is the product py equipped with its canonical projection to y , so $f^*p \cong py$. By Example 5.83, we can write $q' \cong qy$ for some $q \in \mathbf{Poly}$, so by (5.84),

$$\begin{aligned} (\mathbf{Poly}/y)(f^*p, q') &\cong (\mathbf{Poly}/y)(py, qy) \\ &\cong \mathbf{Poly}(py, q) \\ &\cong \mathbf{Poly}(p, q^y), \end{aligned}$$

where q^y is the cartesian closure from (5.29) and Theorem 5.32. We can therefore set $f_*q' := q^y$.

Solution to Exercise 5.90.

1. We have $(\mathbf{Poly}/y^2)(\psi, f_*\varphi) \cong (\mathbf{Poly}/y)(f^*\psi, \varphi)$, so it suffices to show that the latter set is a singleton. As ψ is an identity lens and the pullback of an identity is an identity, $f^*\psi$ is y equipped with the identity lens. Then by (5.84) we have

$$(\mathbf{Poly}/y)(f^*\psi, \varphi) \cong (\mathbf{Poly}/y)(y, y^2) \cong \mathbf{Poly}(y, y) \cong 1.$$

2. We have $(\mathbf{Poly}/y^2)(\chi, f_*\varphi) \cong (\mathbf{Poly}/y)(f^*\chi, \varphi)$, so it suffices to show that the latter set has exactly two elements. We use Example 5.39 to compute the pullback $f^*\chi$ given by

$$\begin{array}{ccc} f^*y^3 & \longrightarrow & y^3 \\ f^*\chi \downarrow & \lrcorner & \downarrow \chi \\ y & \xrightarrow{f} & y^2 \end{array}$$

On positions, $(f^*y^3)(1)$ is the pullback of the position-sets of the other three polynomials, which are all 1; so $(f^*y^3)(1) \cong 1$ and we can write $f^*y^3 = y^A$ for some set A . Then on directions, A is the pushout of direction-sets given by

$$\begin{array}{ccc} A & \longleftarrow & 3 \\ (f^*\chi)_1^\# \uparrow & \lrcorner & \uparrow \chi_1^\# \\ 1 & \longleftarrow & 2 \\ & f_1^\# & \end{array}$$

which takes the disjoint union of 1 and 3 and identifies $f_1^\#1 = f_1^\#2 = 1 \in 1$ with both $\chi_1^\#1 = 1$ and $\chi_1^\#2 = 3$, yielding $A \cong 2$; so $f^*\chi$ can be identified with y^2 equipped with a lens to y^2 . Then by (5.84) we have

$$(\mathbf{Poly}/y)(f^*\chi, \varphi) \cong (\mathbf{Poly}/y)(y^2, y^2) \cong \mathbf{Poly}(y^2, y) \cong 2.$$

3. We have that $f_*\varphi \in \mathbf{Poly}/y^2$ is a polynomial p equipped with a lens $\omega: p \rightarrow y^2$ that picks out a pair of directions $(\omega_i^\#1, \omega_i^\#2) \in p[i]^2$ for each $i \in p(1)$. By (5.91), there is a unique morphism $\psi \rightarrow f_*\varphi$ in $\mathbf{Poly}/y^2$. Such a morphism is a lens $g: y^2 \rightarrow p$ for which $g \circ \omega = \text{id}_{y^2}$, or equivalently $\omega_{g_1^\#1}^\# \circ g_1^\# = \text{id}_2$. For such $g_1^\#$ to exist, we must have $\omega_{g_1^\#1}^\#1 \neq \omega_{g_1^\#1}^\#2$; for $g_1^\#$ to be unique, $p[g_1^\#1]$ must not contain any elements outside of the image of $\omega_{g_1^\#1}^\#$. Moreover, if we had another position $i' \in p(1)$ for which $\omega_{i'}^\#1 \neq \omega_{i'}^\#2$, we could define another lens $g': y^2 \rightarrow p$ with $g'_1^\#1 = i'$ and $(g')_1^\#$ sending $\omega_{i'}^\#1 \mapsto 1$ and $\omega_{i'}^\#2 \mapsto 2$, yielding another morphism $\psi \rightarrow f_*\varphi$ in $\mathbf{Poly}/y^2$; uniqueness implies that $g = g'$, so $g_1^\#1 = i'$. It follows that there is exactly one position $i \in p(1)$ for which $\omega_i^\#1 \neq \omega_i^\#2$, and furthermore $p[i]$ contains only these 2 directions. Meanwhile, a morphism $\chi \rightarrow f_*\varphi$ in $\mathbf{Poly}/y^2$ is a lens $h: y^2 \rightarrow p$ for which $h \circ \omega = \chi$, or equivalently $\omega_{h_1^\#1}^\# \circ h_1^\# = \chi_1^\#$, sending $1 \mapsto 1$ and $2 \mapsto 3$. So in particular $\omega_{h_1^\#1}^\#1 \neq \omega_{h_1^\#1}^\#2$, which implies that $h_1^\#1 = i$; furthermore, $h_1^\#$: $p[i] \rightarrow 3$ sends $\omega_{h_1^\#1}^\#1 \mapsto 1$ and $\omega_{h_1^\#1}^\#2 \mapsto 3$. As $p[i]$ contains only these 2 directions, this completely characterizes h on directions. It follows that there is at most 1 morphism $\chi \rightarrow f_*\varphi$, contradicting (5.92).

Part II

A different category of categories

6. The composition product

We have seen that the category **Poly** of polynomial functors has plenty of interoperating mathematical structure. Further, it provides an expressive way to model dynamical systems that can change their interfaces and wiring patterns based on their internal states.

But we touched upon one aspect of the theory only briefly. That aspect is quite simple to state, yet it has profound consequences. Namely, polynomials can be composed:

$$y^2 \circ (y + 1) = (y + 1)^2 \cong y^2 + 2y + 1.$$

In other words, $(y + 1)$ is *substituted* for the variable y in y^2 .

It turns out that this operation, which we will soon see is a monoidal product, has much to do with the concept of *time*. There is a sense—made precise in Proposition 6.2—in which the polynomial $p \circ q$ represents “starting at a p -position i , choosing a $p[i]$ -direction, landing at a q -position j , and choosing a $q[j]$ -direction.” This is exactly what we need to run through multiple steps of a dynamical system: the very thing we did not know how to do in Example 4.43. We will continue that story in Section 6.1.4.

Throughout the rest of this book, we will see that the composition product is surprisingly versatile, with many interesting properties.¹

6.1 Interpreting the composition product

There are many ways to define and interpret the composition product in **Poly**, and we will introduce and explore them throughout this section.

¹Some authors refer to $\circ$ as the *substitution* product, rather than the composition product. We elect to use the composition product terminology because it provides a good noun form “the composite” for $p \circ q$, whereas “the substitute” is misleading.

6.1.1 Composite functors

We begin with the definition of the composition product in terms of polynomials as functors.

Definition 6.1 (Composition product). Given polynomial functors p, q , we let $p \circ q$ denote their *composition product*, or their composite as functors. That is, $p \circ q: \mathbf{Set} \rightarrow \mathbf{Set}$ sends each set X to the set $p(q(X))$.

Functor composition gives a monoidal structure on the category $\mathbf{Set}^{\mathbf{Set}}$ of functors $\mathbf{Set} \rightarrow \mathbf{Set}$, but to check that the full subcategory $\mathbf{Poly}$ of $\mathbf{Set}^{\mathbf{Set}}$ inherits this monoidal structure, we need to verify that the composite of two functors in $\mathbf{Poly}$ is still a functor in $\mathbf{Poly}$.

Proposition 6.2. Suppose $p, q \in \mathbf{Poly}$ are polynomial functors. Then their composite $p \circ q$ is again a polynomial functor, and we have the following isomorphism:

$$p \circ q \cong \sum_{i \in p(1)} \prod_{a \in p[i]} \sum_{j \in q(1)} \prod_{b \in q[j]} y. \quad (6.3)$$

Proof. We write p and q as

$$p \cong \sum_{i \in p(1)} \prod_{a \in p[i]} y \quad \text{and} \quad q \cong \sum_{j \in q(1)} \prod_{b \in q[j]} y,$$

so for any set X we have

$$(p \circ q)(X) = p(q(X)) \cong p\left(\sum_{j \in q(1)} \prod_{b \in q[j]} X\right) \cong \sum_{i \in p(1)} \prod_{a \in p[i]} \sum_{j \in q(1)} \prod_{b \in q[j]} X,$$

verifying (6.3). To see this is a polynomial, we apply (1.30), which says we can rewrite the $\prod \sum$ in (6.3) as a $\sum \prod$ to obtain

$$\begin{aligned} p \circ q &\cong \sum_{i \in p(1)} \sum_{\bar{j}: p[i] \rightarrow q(1)} \prod_{a \in p[i]} \prod_{b \in q[\bar{j}a]} y \\ &\cong \sum_{i \in p(1)} \sum_{\bar{j}: p[i] \rightarrow q(1)} y^{\sum_{a \in p[i]} q[\bar{j}a]}, \end{aligned}$$

which is a polynomial. $\square$

Corollary 6.4. The category **Poly** has a monoidal structure $(y, \circ)$, where y is the identity functor and $\circ$ is given by composition.

Instead of the symbol $\circ$, which may denote composition in any category, we use a special symbol for polynomial composition, namely

$$p \triangleleft q := p \circ q.$$

The symbol $\triangleleft$ resembles the composition symbol in that it is an open shape. But $\triangleleft$ highlights the asymmetry of composition, in contrast with the other monoidal structures on **Poly** we have encountered. Moreover, $\triangleleft$ evokes the trees that we will soon introduce as a way to depict polynomial composition.

For each $n \in \mathbb{N}$, we will also use $p^{\triangleleft n}$ to denote the n -fold composition product of p , i.e. n copies of p all composed with each other.² In particular, $p^{\triangleleft 0} = y$ and $p^{\triangleleft 1} = p$.

We repeat the important formulas from Proposition 6.2 and its proof in the new notation:

$$p \triangleleft q \cong \sum_{i \in p(1)} \prod_{a \in p[i]} \sum_{j \in q(1)} \prod_{b \in q[j]} y; \quad (6.5)$$

$$p \triangleleft q \cong \sum_{i \in p(1)} \sum_{\bar{j}: p[i] \rightarrow q(1)} y^{\sum_{a \in p[i]} q[\bar{j}a]}. \quad (6.6)$$

Exercise 6.7. Let us consider (6.6) piece by piece, with concrete polynomials $p := y^2 + y^1$ and $q := y^3 + 1$.

1. What is $y^2 \triangleleft q$?
2. What is $y^1 \triangleleft q$?
3. What is $(y^2 + y^1) \triangleleft q$? This is what $p \triangleleft q$ “should be.”

²When we say “the n -fold composition product of p ,” we mean n copies of p all composed with each other; but when we discuss an “ n -fold composition product” in general, we refer to an arbitrary composition product of n polynomials that may or may not all be equal to each other. This will apply to composition products of lenses as well, once we define those.

4. How many functions $\bar{j}_1: p[1] \rightarrow q(1)$ are there?
5. For each function $\bar{j}_1$ as above, what is $\sum_{a \in p[1]} q[\bar{j}_1 a]$?
6. How many functions $\bar{j}_2: p[2] \rightarrow q(1)$ are there?
7. For each function $\bar{j}_2$ as above, what is $\sum_{a \in p[2]} q[\bar{j}_2 a]$?
8. Write out

$$\sum_{i \in p(1)} \sum_{\bar{j}: p[i] \rightarrow q(1)} y^{\sum_{a \in p[i]} q[\bar{j}a]}.$$

Does the result agree with what $p \triangleleft q$ should be?

Exercise 6.8.

1. If p and q are representable, show that $p \triangleleft q$ is too.
2. If p and q are linear, show that $p \triangleleft q$ is too.
3. If p and q are constant, show that $p \triangleleft q$ is too.

Exercise 6.9. Recall the closure operation $[-, -]: \mathbf{Poly}^{\text{op}} \times \mathbf{Poly} \rightarrow \mathbf{Poly}$ for $\otimes$ from (4.75). Show that for all $A \in \mathbf{Set}$ and $q \in \mathbf{Poly}$, there is an isomorphism

$$y^A \triangleleft q \cong [Ay, q].$$

Now that we know how $\triangleleft$ acts on the objects of $\mathbf{Poly}$, we can examine its behavior on the morphisms. For any pair of natural transformations $f: p \rightarrow p'$ and $g: q \rightarrow q'$ between polynomial functors, their composite $f \triangleleft g: p \triangleleft q \rightarrow p' \triangleleft q'$ is given by *horizontal composition*.

Definition 6.10 (Horizontal composition of natural transformations). Let $f: p \rightarrow p'$ and $g: q \rightarrow q'$ be two natural transformations between (polynomial) functors $p, p', q, q': \mathbf{Set} \rightarrow \mathbf{Set}$. Then the *horizontal composite* of f and g , denoted $f \triangleleft g$, is the natural transformation $p \triangleleft q \rightarrow p' \triangleleft q'$ whose X -component for each $X \in \mathbf{Set}$ is the function

$$p(q(X)) \xrightarrow{f_{q(X)}} p'(q(X)) \xrightarrow{p'(g_X)} p'(q'(X)) \quad (6.11)$$

obtained by composing the $q(X)$ -component of f with the functor

p' applied to the X -component of g .

Exercise 6.12. Show that we could have replaced the composite function (6.11) in Definition 6.10 with the function

$$p(q(X)) \xrightarrow{p(g_X)} p(q'(X)) \xrightarrow{f_{q'(X)}} p'(q'(X)) \quad (6.13)$$

obtained by composing p applied to the X -component of g with the $q'(X)$ -component of f , without altering the definition.

Remark 6.14. So far, we have introduced two different notions of lens composition, so let us mitigate confusion by standardizing terminology. We reserve the term *composite lens* for lenses $h \circ j: r \rightarrow t$ obtained by composing a lens $h: r \rightarrow s$ with a lens $j: s \rightarrow t$, according to the composition rule of the category **Poly**. This corresponds to *vertical composition* of natural transformations. This is also the kind of composition we mean whenever we use the verb *compose*, if the objects of that verb are lenses.

Meanwhile, we use the term *composition product (of lenses)* for lenses $f \triangleleft g: p \triangleleft q \rightarrow p' \triangleleft q'$ obtained by applying the monoidal product functor $\triangleleft: \mathbf{Poly} \times \mathbf{Poly} \rightarrow \mathbf{Poly}$ on the lenses $f: p \rightarrow p'$ and $g: q \rightarrow q'$. This corresponds to *horizontal composition* of natural transformations. In this case, we use the verb phrase *taking the monoidal product*.

On the other hand, we use the terms *composite* and *composition product* interchangeably to refer to polynomials $p \triangleleft q$, obtained by composing $p, q \in \mathbf{Poly}$ as functors or, equivalently, applying the monoidal product functor $\triangleleft$ on them—there is no risk of confusion here.

This is another reason we avoid the symbol $\circ$, preferring to use $\circ$ or juxtaposition for vertical composition and $\triangleleft$ for horizontal composition. Of course, if you are ever confused, you can always check whether the codomain of the first lens matches up with the domain of the second: if they do not, we must be taking their monoidal product.

Composition products of polynomials and lenses will be crucial in what follows. But just as we prefer to think of polynomials in terms of their positions and directions rather than as functors, we seek a more

concrete interpretation of the composition product in terms of positions and directions as well.

6.1.2 Composite positions and directions

Let us interpret our formula (6.6) for the composition product of two polynomials in terms of positions and directions. The position-set of $p \triangleleft q$ is

$$(p \triangleleft q)(1) \cong \sum_{i \in p(1)} \sum_{\bar{j}: p[i] \rightarrow q(1)} 1 \cong \sum_{i \in p(1)} \mathbf{Set}(p[i], q(1)). \quad (6.15)$$

So we can identify a $(p \triangleleft q)$ -position with a p -position i along with a function $\bar{j}: p[i] \rightarrow q(1)$, i.e. a q -position $\bar{j}a$ for each $p[i]$ -direction a .

Given such a $(p \triangleleft q)$ -position $(i, \bar{j})$, the direction-set of $p \triangleleft q$ at $(i, \bar{j})$ is then

$$(p \triangleleft q)[(i, \bar{j})] \cong \sum_{a \in p[i]} q[\bar{j}a]. \quad (6.16)$$

So a $(p \triangleleft q)[(i, \bar{j})]$ -direction consists of a $p[i]$ -direction a and a $q[\bar{j}a]$ -direction.

While this description completely characterizes $p \triangleleft q$, it may appear rather opaque. Here is an alternate perspective that can provide more intuition for the composition product of polynomials.

Back in Section 1.2, we saw how to write the instructions for choosing an element of a sum or product of sets. For instance, given a polynomial p and a set X , the instructions for choosing an element of

$$p \triangleleft X = p(X) \cong \sum_{i \in p(1)} \prod_{a \in p[i]} X$$

would be written as follows.

To choose an element of $p(X)$:

1. choose an element $i \in p(1)$;
2. for each element $a \in p[i]$:
 - 2.1. choose an element of X .

But say we had not picked a set X yet and might replace X with an arbitrary polynomial instead. We will replace “an element of X ” with a

placeholder—the words “a future”—indicating that we do not yet know what will go there. Furthermore, to highlight that these instructions are associated with some polynomial p , we will use our familiar positions and directions terminology.

The instructions associated with a polynomial p are:

1. choose a p -position i ;
2. for each $p[i]$ -direction a :
 - 2.1. choose a future.

If we think of polynomials in terms of their instructions, then (6.5) tells us that the composition product simply nests one set of instructions within another, as follows.

The instructions associated with a polynomial $p \triangleleft q$ are:

1. choose a p -position i ;
2. for each $p[i]$ -direction a :
 - 2.1. choose a q -position j ;
 - 2.2. for each $q[j]$ -direction b :
 - 2.2.1. choose a future.

Similarly, we could write down the instructions associated with any n -fold composition product by nesting further. We might think of such instructions as specifying a length- n *strategy*, in the sense of game theory, for choosing positions given any directions chosen by an opponent.

When we rewrite (6.5) as (6.6), we collapse the associated instructions into the following, highlighting the positions and directions of $p \triangleleft q$.

The instructions associated with a polynomial $p \triangleleft q$ are:

1. choose a p -position i and, for each $p[i]$ -direction a , a q -position $\bar{j}a$;
2. for each $p[i]$ -direction a and each $q[\bar{j}a]$ -direction b :
 - 2.1. choose a future.

We will see in Section 6.1.3 that these instructions have a natural interpretation when we depict these polynomials as corolla forests.

Exercise 6.17.

1. Let p be an arbitrary polynomial. Write out the (uncollapsed) instructions associated with $p^{\triangleleft 3} = p \triangleleft p \triangleleft p$.
2. Write out the (uncollapsed) instructions for choosing an element of $p \triangleleft p \triangleleft 1$, but where you would normally write “choose an element of 1,” just write “done.”

It remains to examine how the composition product acts on lenses. Given lenses $f: p \rightarrow p'$ and $g: q \rightarrow q'$, we can translate them to natural transformations, take their horizontal composite, then translate this back to a lens. The following exercise guides us through this process.

Exercise 6.18 (The composition product of lenses). Fix lenses $f: p \rightarrow p'$ and $g: q \rightarrow q'$. We seek to characterize their composition product $f \triangleleft g: p \triangleleft q \rightarrow p' \triangleleft q'$.

1. Use Proposition 3.44 to compute the $q(X)$ -component of f as a natural transformation.
2. Use Propositions 2.10 and 3.44 to compute p' applied to the X -component of g as a natural transformation.
3. Combine #1 and #2 using Definition 6.10 to compute the horizontal composite $f \triangleleft g$ of f and g as natural transformations.
4. Use Corollary 3.47 to translate the natural transformation $f \triangleleft g$ obtained in #3 to a lens $p \triangleleft q \rightarrow p' \triangleleft q'$. Verify that for each $(i, \bar{j})$ in $(p \triangleleft q)(1)$ (see (6.15)), its on-positions function sends

$$(i, \bar{j}) \xrightarrow{(f \triangleleft g)_1} (f_1 i, f_i^\# \circ \bar{j} \circ g_1); \quad (6.19)$$

while for each (a', b') in $(p' \triangleleft q')[((f_1 i, f_i^\# \circ \bar{j} \circ g_1))]$ (see (6.16)), its on-directions function sends

$$(a', b') \xrightarrow{(f \triangleleft g)_{(i, \bar{j})}^\#} (f_i^\# a', g_{\bar{j} f_i^\# a'}^\# b'). \quad (6.20)$$

Let us summarize what Exercise 6.18 says about the behavior of $f \triangleleft g: p \triangleleft q \rightarrow p' \triangleleft q'$. By (6.19), on positions, $f \triangleleft g$ takes a p -position i and sends it to the p' -position $f_1 i$; then for each direction a' at this

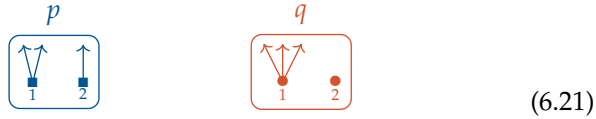
position, the associated q' -position is obtained by sending a' back to a $p[i]$ -direction via $f_i^\sharp$, checking what q -position is associated to that $p[i]$ -direction via some $\bar{j}$, then sending that q -position forward again to a q' -position via g_1 .

Then by (6.19), on directions, $f \triangleleft g$ sends a direction of p' back to a direction of p via an on-directions function of f , then sends a direction of q' back to a direction of q via an on-directions function of g . We will get a better sense of what is happening when we draw this out as corolla forests in Example 6.30.

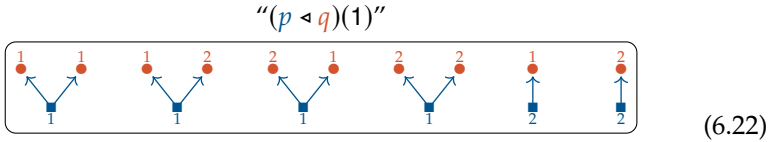
6.1.3 Composition product on corolla forests

It turns out that the forest of $p \triangleleft q$ is given by grafting q -corollas onto the leaves of p -corollas in every possible way. We will demonstrate this using an example.

Let $p := y^2 + y$ and $q := y^3 + 1$, whose corolla forests we draw as follows:

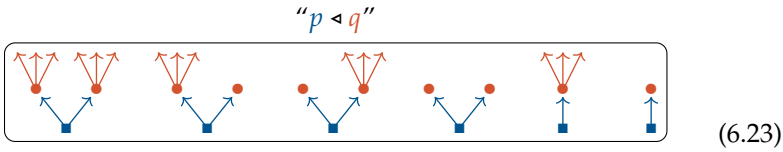


By (6.15), choosing a position of $p \triangleleft q$ amounts to first choosing a p -corolla i , then choosing a q -corolla for every $p[i]$ -leaf. For now, we can disregard the leaves of these q -corolla, as they do not play a part in the positions of $p \triangleleft q$. So we may depict $(p \triangleleft q)(1)$ by grafting roots from the corolla forest of q to leaves in the corolla forest of p in every possible way, as follows:



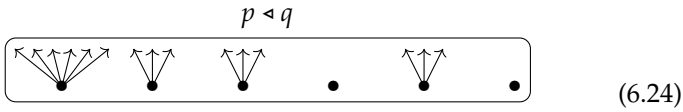
Now fix one of the positions of $p \triangleleft q$ drawn above: a p -corolla i and a q -corolla grafted to every $p[i]$ -leaf. By (6.16), a direction of $p \triangleleft q$ at that position consists of a $p[i]$ -leaf a and a second leaf emanating from the q -corolla that has been grafted on to a . In other words, in the following picture, where we have grafted not just roots but entire q -corollas onto

leaves in p , the directions of $p \triangleleft q$ at the position corresponding to each tree are the rooted paths³ of that tree of length 2 (we omit the labels):



Equivalently, we can think of the directions in the picture above as the leaves at the second level of each tree. So $p \triangleleft q$ has six positions: the first has six directions; the second, third, and fifth each have three directions; and the fourth and sixth have no directions. In total, we can read off that $p \triangleleft q$ is isomorphic to $y^6 + 3y^3 + 2$.

We put the $p \triangleleft q$ in scare quotes above (6.23) because, to be pedantic, the corolla forest of $p \triangleleft q$ should have leaves of height 1 instead of 2:



Usually, we will prefer the style of (6.23) rather than the more pedantic style of (6.24).

We have now seen how to draw a single polynomial as a corolla forest, with height-1 leaves as directions; as well as how to draw a 2-fold composite of polynomials as a forest of trees, with height-2 leaves as directions. We can verify the validity of these pictures by observing that drawing a corolla of p or a tree of $p \triangleleft q$ is just a graphical way of following the instructions associated with the polynomial p or $p \triangleleft q$ that we saw in Section 6.1.2, where the arrows—the top-level leaves—are where the “futures” would go. Similarly, we could depict any n -fold composite as a forest with height- n leaves as directions. You will have an opportunity to try this in the following exercise.

³Recall from Definition 2.22 that a *rooted path* is a path in a rooted tree from its root to any vertex, and its *length* is the number of edges it contains.

Exercise 6.25. Use $p := y^2 + y$ and $q := y^3 + 1$ as in (6.21) and $r := 2y + 1$ in the following.

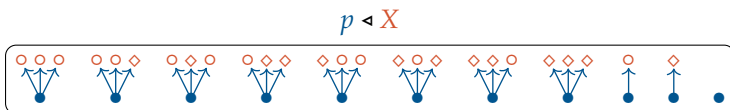
1. Draw $q \triangleleft p$.
2. Draw $p \triangleleft p$.
3. Draw $p \triangleleft p \triangleleft 1$.
4. Draw $r \triangleleft r$.
5. Draw $r \triangleleft r \triangleleft r$.

Example 6.26 (Composing polynomials with constants). For any set X and polynomial p , we have $p(X) \in \mathbf{Set}$; after all, $p: \mathbf{Set} \rightarrow \mathbf{Set}$ is a functor. For instance, we have written $p(1)$ many times. But we have also seen that X is itself a polynomial, namely a constant one. Is it then true that $p(X) \cong p \triangleleft X$?

Here is a picture of an example, where $p := y^3 + y + 1$ and $X := 2$.

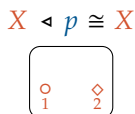


Let us then draw $(y^3 + y + 1) \triangleleft 2$.



There are no height-2 leaves in this picture, so $p \triangleleft X$ has no directions: it is a constant polynomial, or a set. There are 11 positions, which coincides with the fact that $p \triangleleft X \cong 2^3 + 2 + 1 \cong 11$.

We could also draw $X \triangleleft p$:



Each of the leaves in X , of which there are none, is given a p -corolla.

Exercise 6.27.

1. Choose a polynomial p and draw $p \triangleleft 1$ in the style of Example 6.26.
2. Show that if X is a set (considered as a constant polynomial) and p is any polynomial, then $X \triangleleft p \cong X$.
3. Show that if X is a set and p is a polynomial, then $p \triangleleft X \cong p(X)$, where $p(X)$ is the set given by applying p as a functor to X .

In particular, this means we could write the position-set of a polynomial p interchangeably as $p(1)$ or as $p \triangleleft 1$. We will generally write $p(1)$ when we want to emphasize the position-set as a set and $p \triangleleft 1$ when we want to emphasize the position-set as a constant polynomial.

Exercise 6.28. Let $\varphi: p \rightarrow q$ be a lens and X be a set viewed as a constant polynomial. Consider the lens $\varphi \triangleleft X: p \triangleleft X \rightarrow q \triangleleft X$, given by taking the composition product of φ with the identity lens on X . Show that $\varphi \triangleleft X$, when viewed as a function $p(X) \rightarrow q(X)$ between sets, is exactly the X -component of φ viewed as a natural transformation.

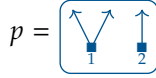
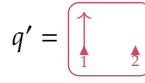
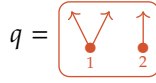
Exercise 6.29. For any $p \in \mathbf{Poly}$ there are natural isomorphisms $p \cong p \triangleleft y$ and $p \cong y \triangleleft p$.

1. Thinking of polynomials as functors $\mathbf{Set} \rightarrow \mathbf{Set}$, what functor does y represent?
2. Why are $p \triangleleft y$ and $y \triangleleft p$ isomorphic to p ?
3. Let $p := y^3 + y + 1$. In terms of tree pictures, draw $p \triangleleft y$ and $y \triangleleft p$, and explain pictorially how to see the isomorphisms $p \triangleleft y \cong p \cong y \triangleleft p$.

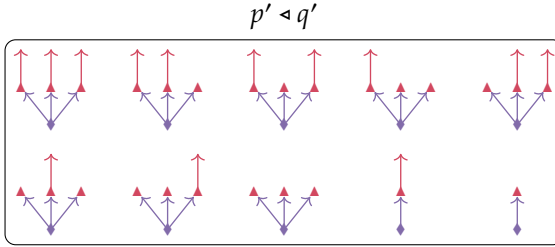
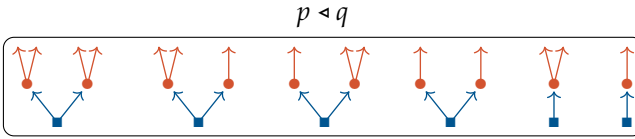
Let us now interpret the composition product of lenses in terms of our tree pictures. We can depict the results of Exercise 6.18 as follows.

Example 6.30. Take $p := y^2 + y$, $q := y^2 + y$, $p' := y^3 + y$, and

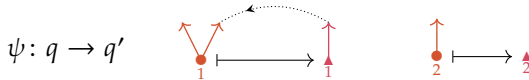
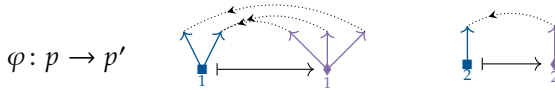
$$q' := y + 1.$$



For any pair of lenses $p \rightarrow p'$ and $q \rightarrow q'$, we have a lens $p \triangleleft q \rightarrow p' \triangleleft q'$. We draw $p \triangleleft q$ and $p' \triangleleft q'$ below.



We also pick a pair of lenses, $\varphi: p \rightarrow p'$ and $\psi: q \rightarrow q'$.

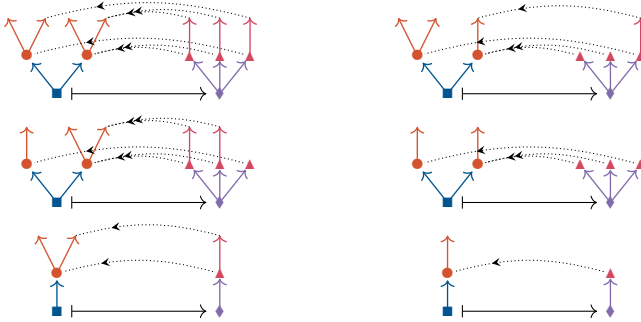


Then by Exercise 6.18, we can form the lens $\varphi \triangleleft \psi: p \triangleleft q \rightarrow p' \triangleleft q'$ as follows. On positions, we follow (6.19): for each tree t in the picture

of $p \triangleleft q$, we use φ_1 to send the p -corolla i that forms the bottom level of t to a p' -corolla i' . Then for each $p'[i']$ -leaf a' , to choose which q' -corolla gets grafted onto a' , we use $\varphi_i^\#$ to send a' back to a $p[i]$ -leaf a . Since t has the corolla i as its bottom level, a is just a height-1 vertex of the tree t . So we can take the q -corolla j that is grafted onto a in t , then use ψ_1 to send j forward to a q' -corolla j' . This is the corolla we graft onto the $p'[i']$ -leaf a' . All this specifies a tree t' in $p' \triangleleft q'$ to which t is sent by $(\varphi \triangleleft \psi)_1$.

On directions, we follow (6.20): picking a direction of t' consists of picking a height-1 vertex a' and a height-2 leaf b' emanating from a' . The on-directions function $\varphi_i^\#$ sends a' back to a height-1 vertex a of t , and as we saw, the on-positions function ψ_1 sends the q -corolla j grafted onto a in t forward to the q' -corolla grafted onto a' . Then b' is a leaf of that q' -corolla, and $\psi_j^\#$ sends b' back to a leaf b emanating from a . So the on-directions function $(\varphi \triangleleft \psi)_i^\#$ sends the height-2 leaf b' to the height-2 leaf b .

We draw the lens $\varphi \triangleleft \psi: p \triangleleft q \rightarrow p' \triangleleft q'$ below. To avoid clutter, we leave out the rightward arrows for ψ_1 that show how each of the magenta corollas with triangular roots are selected.



Exercise 6.31. With p, q, p', q' and φ, ψ as in Example 6.30, draw the lens $\psi \triangleleft \varphi: q \triangleleft p \rightarrow q' \triangleleft p'$ in terms of trees as in the example.

Exercise 6.32. Given polynomials p, q , and r and lenses $\varphi: q \rightarrow p \triangleleft q$ and $\psi: q \rightarrow q \triangleleft r$, does the following diagram necessarily commute?⁴

$$\begin{array}{ccc} q & \xrightarrow{\psi} & q \triangleleft r \\ \varphi \downarrow & ? & \downarrow \varphi \triangleleft r \\ p \triangleleft q & \xrightarrow{p \triangleleft \psi} & p \triangleleft q \triangleleft r \end{array}$$

That is, do we have $\varphi \circ (p \triangleleft \psi) =? \psi \circ (\varphi \triangleleft r)$?

6.1.4 Dynamical systems and the composition product

Back in Example 4.43, we asked how **Poly** could model running multiple steps of a dynamical system. The answer is that we can use the composition product.

Recall our definition of a (*dependent*) *dynamical system* $\varphi: Sy^S \rightarrow p$ from Definition 4.18. More generally, we saw in Example 4.43 that we could replace the state system Sy^S with a monomial $q := Sy^{S'}$, where S' is another set, as long as there is a bijection $e: S \rightarrow S'$, which we can view as a certain section $\epsilon: Sy^{S'} \rightarrow y$.

Then the lens $\varphi: q \rightarrow p$ models a dynamical system as follows. Every state $s \in q(1) = S$ returns a position $i := \varphi_1 s \in p(1)$, at which every direction $a \in p[i]$ yields an updated direction $s' := \varphi_s^\# a \in q[s] = S'$. Then to model a second step through the system, we identify the $q[s]$ -direction s' with a q -position $e^{-1}(s')$, plug this position back into φ_1 , and repeat the process.

But this is exactly what the composition product $\varphi \triangleleft \varphi: q \triangleleft q \rightarrow p \triangleleft p$ does: by (6.19), its on-positions function sends $(s_0, e^{-1}) \in (q \triangleleft q)(1)$, comprised of an initial state $s_0 \in q(1)$ and the function $e^{-1}: q[s_0] = S' \rightarrow S = q(1)$, to

$$\left(\varphi_1 s_0, \varphi_{s_0}^\# \circ e^{-1} \circ \varphi_1 \right) \in (p \triangleleft p)(1), \quad (6.33)$$

⁴When the name of an object is used in place of a morphism, we refer to the identity morphism on that object. So for instance, $\varphi \triangleleft r$ is the composition product of φ with the identity lens on r .

comprised of the initial position $i_0 := \varphi_1 s \in p(1)$ and a composite function

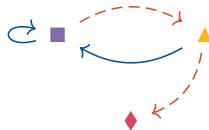
$$p[i_0] \xrightarrow{\varphi_{s_0}^\#} q[s_0] = S' \xrightarrow{e^{-1}} S = q(1) \xrightarrow{\varphi_1} p(1), \quad (6.34)$$

which uses the update function at s_0 and the return function to tell us what the next position i_1 will be for every possible direction a_1 we could select. Then by (6.20), the on-directions function of $\varphi \triangleleft \varphi$ sends each direction (a_1, a_2) at the position (6.33), comprised of an initial direction $a_1 \in p[i_0]$ and (setting i_1 to be the function (6.34) applied to a_1) a second direction $a_2 \in p[i_1]$, to the pair

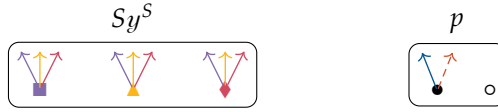
$$\left(\varphi_{s_0}^\# a_1, \varphi_{e^{-1}\varphi_{s_0}^\# a_1}^\# a_2 \right),$$

comprised of directions in S' that (under e^{-1}) correspond to the next state s_1 upon selecting direction a_1 at state s_0 and the successive state s_2 upon selecting direction a_2 at s_1 . In summary, at certain positions, $\varphi \triangleleft \varphi$ tells us how the dynamical system will behave when we step through it twice: starting from state s_0 , returning position i_0 , receiving direction a_1 , updating its state to s_1 , returning position i_1 , receiving direction a_2 , and preparing to update its state to s_2 . Adding another layer, $\varphi \triangleleft \varphi \triangleleft \varphi: q \triangleleft q \triangleleft q \rightarrow p \triangleleft p \triangleleft p$ will tell us how the system behaves when we step through it three times; and in general, $\varphi^{\triangleleft n}: q^{\triangleleft n} \rightarrow p^{\triangleleft n}$ will tell us how the system behaves when we step through it n times.

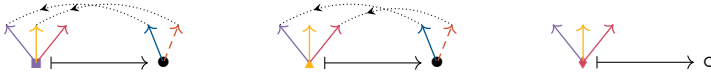
Example 6.35 (Composition products of dynamical systems as trees). Consider the dynamical system $\varphi: Sy^S \rightarrow p$ with $p := y^A + 1$, corresponding to the halting deterministic state automaton (4.24) from Exercise 4.23, depicted again here for convenience:



Below, we draw the corolla pictures for Sy^S and for p .

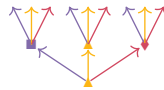


In the picture for Sy^S , the roots are the three states $\{\blacksquare, \blacktriangle, \blacklozenge\}$ appearing in the automaton, while the leaves of each corolla correspond to the three states as well, from left to right. In the picture for p , there is one corolla whose two leaves correspond to the two arrows coming out of every state except the halting state, which is sent to the corolla with no leaves instead. So the lens $\varphi: Sy^S \rightarrow p$ capturing the dynamics of the automaton can be drawn as follows:



The corolla picture tells us, for example, that from the $\blacktriangle$ state, we can take one of two directions: the solid blue direction, which leads us to the $\blacksquare$ state, or the dashed red direction, which leads us to the $\blacklozenge$ state. This describes the dynamics of the automaton one step away from the $\blacktriangle$ state.

But what if we want to understand the dynamics of φ *two* steps away from the $\blacktriangle$ state? Consider the following tree, corresponding to a position of the 2-fold composite $Sy^S \triangleleft Sy^S$, where each height-1 vertex has been selected to match the leaf it is grafted to:



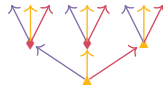
(6.36)

We can follow the steps from Example 6.30 to find that the composition product of lenses $\varphi \triangleleft \varphi: Sy^S \triangleleft Sy^S \rightarrow p \triangleleft p$ acts on this tree

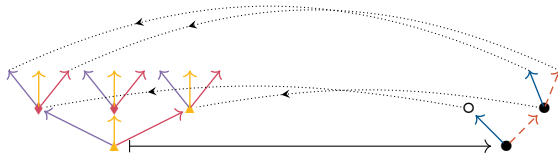
is a communication issue: as you probably sensed, our set-theoretic notation for $\varphi^{\triangleleft n}$ is rather cumbersome, and that was just for $n = 2$. We could depict the behavior of our composition products of dynamical systems more clearly using tree pictures (see Example 6.35), but even that becomes infeasible in greater generality. A concise visual representation of the back-and-forth interaction of lenses would help us reason about composition products more effectively.

The second issue is more technical: to ensure that $\varphi \triangleleft \varphi$ behaves the way we want, when we specify a position of its domain $q \triangleleft q$, we must provide not only an initial state s_0 but also the isomorphism $e^{-1}: S' \rightarrow S$ to let the lens know which state in S each $q[s_0]$ -direction in S' should lead to. But there are many other positions (s_0, f) of $q \triangleleft q$ that we could have specified, and each $f: S' \rightarrow S$ associates $q[s_0]$ -directions to q -positions in a different way: see the following example.

Example 6.37 (Composition products of dynamical systems can be misleading). In Example 6.35, instead of (6.36), we could have picked the following tree, corresponding to a different (yet entirely valid) position of $Sy^S \triangleleft Sy^S$:



The composition product $\varphi \triangleleft \varphi: Sy^S \triangleleft Sy^S \rightarrow p \triangleleft p$ acts on this tree like so:



Now the picture tells us that, starting from the $\blacktriangle$ state, it is the solid blue arrow that will lead to a halting state, whereas we should somehow be able to follow the dashed red arrow twice!

Of course, this is nonsense—stemming from the fact that we have grafted the “wrong” corollas to each leaf when forming the position of $Sy^S \triangleleft Sy^S$ on the left. It is important to note, however, that this is nevertheless part of the data of $\varphi \triangleleft \varphi$, and we do not yet know how to tell **Poly** to rule it out.

So $\varphi \triangleleft \varphi$ is carrying around extraneous—even misleading—data about how our dynamical system behaves when the state system moves in the right direction but to the wrong state. Our isomorphism e^{-1} is a temporary fix, but as we pointed out in Example 4.43, it relies on the set-theoretic equality of the direction-sets of q —there is nothing inherent to the categorical structure of q in **Poly** that encodes how directions map to states, at least not yet.

The key to resolving both these issues lies in the next section, where we study lenses whose codomains are composite polynomials and extend our polybox notation to depict them.

6.2 Lenses to composites

Lenses to composites—that is, lenses of the form $f: p \rightarrow q_1 \triangleleft \cdots \triangleleft q_n$ for some $n \in \mathbb{N}$ with composites as their codomains—will be ubiquitous in the remainder of our story. In particular, they have nice properties that make them convenient to work with, especially using polyboxes.

6.2.1 Lenses to composites as polyboxes

A lens $p \rightarrow q_1 \triangleleft q_2$ is an element of the set

$$\mathbf{Poly}(p, q_1 \triangleleft q_2) \cong \mathbf{Poly}\left(p, \sum_{j_1 \in q_1(1)} \prod_{b_1 \in q_1[j_1]} \sum_{j_2 \in q_2(1)} \prod_{b_2 \in q_2[j_2]} y\right) \quad (6.5)$$

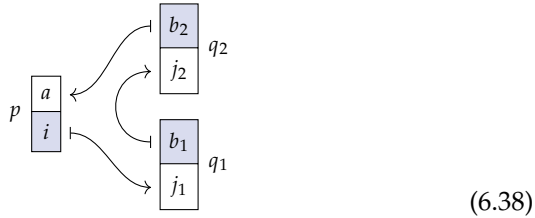
$$\cong \prod_{i \in p(1)} \sum_{j_1 \in q_1(1)} \prod_{b_1 \in q_1[j_1]} \sum_{j_2 \in q_2(1)} \prod_{b_2 \in q_2[j_2]} p[i]. \quad (3.7)$$

So we can write down the instructions for picking a lens $p \rightarrow q_1 \triangleleft q_2$ as follows.

To choose a lens $p \rightarrow q_1 \triangleleft q_2$:

1. for each p -position i :
 - 1.1. choose a q_1 -position j_1 ;
 - 1.2. for each $q_1[j_1]$ -direction b_1 :
 - 1.2.1. choose a q_2 -position j_2 ;
 - 1.2.2. for each $q_2[j_2]$ -direction b_2 :
 - 1.2.2.1. choose a $p[i]$ -direction a .

We could try to write out the dependent functions that these instructions correspond to. Alternatively, we could simply draw this protocol out using polyboxes, with every “for each” step corresponding to a blue box to be filled by the user and every “choose” step corresponding to a white box to be filled by the protocol:



Whenever we draw two pairs of polyboxes on top of each other, as we do with the polyboxes for q_1 and q_2 above on the right, we are indicating that the entire column of polyboxes depicts the composite of the polynomials depicted by each individual pair. So the column of polyboxes on the right represents the composite $q_1 \triangleleft q_2$. In particular, the entry $j_2 \in q_2(1)$ in the position box of the top pair is associated with the entry $b_1 \in q_1[j_1]$ in the direction box of the bottom pair, in accordance with the depicted position of the composite.

So a lens $p \rightarrow q_1 \triangleleft q_2$ is any protocol that will fill in the white boxes above as the user fills in the blue boxes in the direction of the arrows, starting as always from the bottom left and ending as always at the top left. We will see this in action in Example 6.39.

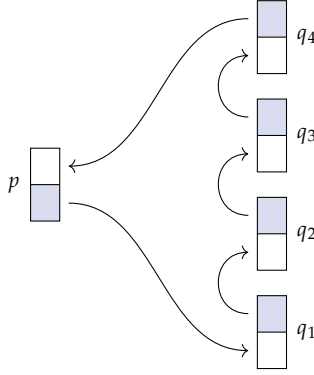
In fact, (3.34), (3.13), and (6.38) are the respective polybox depictions of the $n = 0$, $n = 1$, and $n = 2$ cases of lenses $p \rightarrow q_1 \triangleleft \cdots \triangleleft q_n$ to n -fold composites (we consider the monoidal unit y of $\triangleleft$ to be the 0-fold composite, and a 1-fold composite is just a polynomial on its own). In

general, for any $n \in \mathbb{N}$, we have

$$\mathbf{Poly}(p, q_1 \triangleleft \cdots \triangleleft q_n) \cong \mathbf{Poly}\left(p, \sum_{j_1 \in q_1(1)} \prod_{b_1 \in q_1[j_1]} \cdots \sum_{j_n \in q_n(1)} \prod_{b_n \in q_n[j_n]} y\right) \quad (6.5)$$

$$\cong \prod_{i \in p(1)} \sum_{j_1 \in q_1(1)} \prod_{b_1 \in q_1[j_1]} \cdots \sum_{j_n \in q_n(1)} \prod_{b_n \in q_n[j_n]} p[i], \quad (3.7)$$

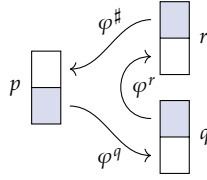
so the polybox depiction of $p \rightarrow q_1 \triangleleft \cdots \triangleleft q_n$ generalizes analogously. For example, here are the polyboxes for a lens to a 4-fold composite:



These lenses to n -fold composites lend themselves to a natural interpretation as interaction protocols. Each p -position is passed forward to a q_1 -position. For every direction at that q_1 -position, there is then also a q_2 -position. Then for every direction at that q_2 -position, there is a q_3 -position, and so on, all the way until a direction has been selected at a q_n -position. Together, all the directions chosen from $q_1, \dots, q_n$ then inform the direction that is selected at the original p -position. A lens $p \rightarrow q_1 \triangleleft \cdots \triangleleft q_n$ is therefore a multi-step protocol for p to make decisions by asking for decisions from q_1 , then q_2 , etc., all the way to q_n , then interpreting the results.

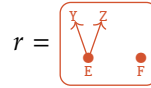
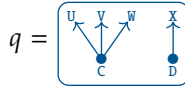
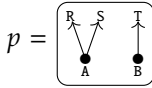
Example 6.39 (Lenses $p \rightarrow q \triangleleft r$). Consider a lens $\varphi: p \rightarrow q \triangleleft r$. Let

us label the three arrows in the lens's polybox depiction:



So the on-position function of φ can be split into two components: a function $\varphi^q: p(1) \rightarrow q(1)$ and, for each $i \in p(1)$, a function $\varphi_i^r: q[\varphi^q i] \rightarrow r(1)$. Meanwhile, the on-directions function $\varphi_i^\#: (q \triangleleft r)[\varphi_1 i] \rightarrow p[i]$ takes the direction of q and the direction of r in the two blue boxes on the right and sends them to a $p[i]$ -direction to fill the white box on the left.

For example, let $p := \{A\}y^{\{R,S\}} + By^{\{T\}}$, $q := \{C\}y^{\{U,V,W\}} + \{D\}y^{\{X\}}$, and $r := \{E\}y^{\{Y,Z\}} + \{F\}$.



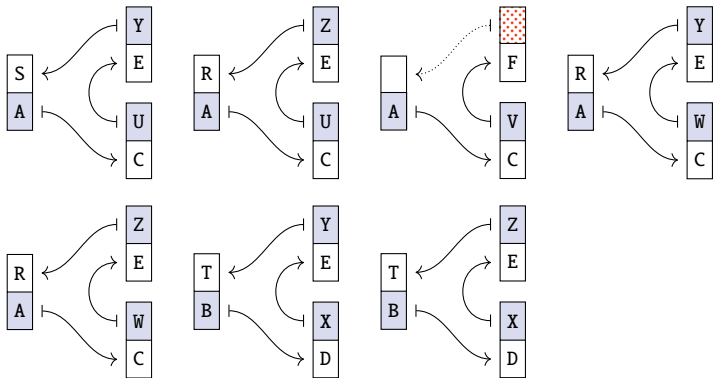
Here is a tree picture of a lens $\varphi: p \rightarrow q \triangleleft r$:



If we write φ as the corresponding triple $(\varphi^q, \varphi^r, \varphi^\#)$, then we have

$$\begin{aligned} \varphi^q A &= C, & \varphi^q B &= D; \\ \varphi_A^r U &= E, & \varphi_A^r V &= F, & \varphi_A^r W &= E; \\ \varphi_B^r X &= E; \\ \varphi_A^\#(U, Y) &= S, & \varphi_A^\#(U, Z) &= R, & \varphi_A^\#(W, Y) &= R, & \varphi_A^\#(W, Z) &= R; \\ \varphi_B^\#(X, Y) &= T, & \varphi_B^\#(X, Z) &= T. \end{aligned}$$

Polyboxes display the same data in a different format:



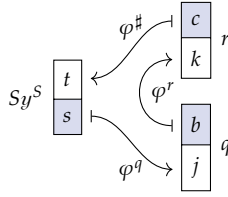
As always, keep in mind that each arrow of a lens depends not only on the box it emerges from, but also on every box that came before it in our reading order for polyboxes (lower left to lower right to upper right to upper left).

The third set of polyboxes, where the left blue box has been filled with A and the lower right blue box has been filled with V, is worth highlighting: as $\varphi_A^r V = F$ and $r[F] = \emptyset$, it is impossible to write a direction of r at F to fill the upper right box. To indicate this, we fill the upper right box with red dots and leave the arrow emerging from it dotted.

Example 6.40 (Dynamical systems with composite interfaces). We explored dynamical systems with *product* interfaces in Section 4.3.1 and *parallel product* interfaces in Section 4.3.2. Now have all the tools we need to characterize dynamical systems with *composite* interfaces.

By the previous example, a dynamical system $\varphi: Sy^S \rightarrow q \triangleleft r$

can be drawn as



We can interpret the behavior of this system as follows. Rather than a single interface $q \triangleleft r$, we view φ as having two interfaces that must be interacted with in succession, q followed by r .

Given the current state $s \in S$, the system feeds it into φ^q to return a position j of the first interface q . Upon receiving a direction $b \in q[j]$, it then uses φ^r to return another position k (dependent on the state s and the direction b), this time belonging to the second interface r . Once a second direction c is received, this time from $r[k]$, the system updates its state by feeding the current state s and the pair of directions (b, c) it received into $\varphi^\#$, yielding a new state $t \in S$.

By following the arrows in the polybox picture, we see that a dynamical system with a composite interface captures a very natural type of interaction. We could use φ to model a system displaying cascading menus, where selecting option b on the first menu j opens a second menu k . After the interacting agent selects option c from the second menu, both choices are sent back to the state system, which updates its state accordingly.

All this generalizes to n -fold composite interfaces exactly how you would expect: a dynamical system with interface $q_1 \triangleleft \dots \triangleleft q_n$ returns a position and receives a direction through interface q_1 , then accordingly returns a position and receives a direction through interface q_2 , and so on, until it returns a position (according to the current state and all previous directions) and receives a direction through q_n , whereupon it updates its state according to the n directions it received along with the current state.

6.2.2 The composition product of lenses as polyboxes

A special case of a lens whose codomain is a composite is a lens that is itself the composition product of lenses. If we draw such a lens using polyboxes by following the instructions from (6.19) and (6.20), we would just be stacking the polyboxes for the constituent lenses atop each other. For example, given lenses $\varphi: p \rightarrow q$ and $\varphi': p' \rightarrow q'$, here is $\varphi \triangleleft \varphi'$ drawn as polyboxes:

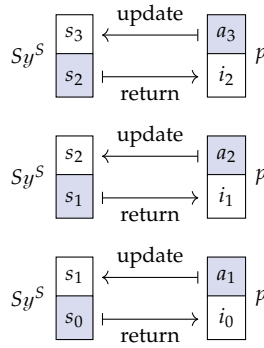
$$\begin{array}{ccc}
 \begin{array}{|c|} \hline \square \\ \hline \blacksquare \\ \hline \end{array} & \begin{array}{c} \xleftarrow{(\varphi')^\sharp} \\ \xrightarrow{\varphi'_1} \end{array} & \begin{array}{|c|} \hline \blacksquare \\ \hline \square \\ \hline \end{array} \\
 p' & & q' \\
 \\
 \begin{array}{|c|} \hline \square \\ \hline \blacksquare \\ \hline \end{array} & \begin{array}{c} \xleftarrow{\varphi^\sharp} \\ \xrightarrow{\varphi_1} \end{array} & \begin{array}{|c|} \hline \blacksquare \\ \hline \square \\ \hline \end{array} \\
 p & & q
 \end{array} \tag{6.41}$$

What distinguishes this from simply writing down the polyboxes for φ and the polyboxes for φ' is that we are explicitly associating the position in the lower box of p' with the direction in the upper box of p ; likewise, we are associating the position in the lower box of q' with the direction in the upper box of q . We should think of the user as filling out the lower set of boxes first and working their way up, so that, in particular, they may use the behavior of φ_1 and $\varphi^\sharp$ to decide what to put in the position box of p' . So this really does depict a lens $p \triangleleft p' \rightarrow q \triangleleft q'$.

Let us consider how (6.41) extends our polybox depiction of a lens to a composite from (6.38), but with the domain also replaced with a composite. A user who interacts with (6.41) can fill the lower set of polyboxes (the ones for φ) first, ignoring the upper set of polyboxes (the ones for φ') until the entire lower half is filled. Alternatively, after they fill the position box of p , but before they fill anything else, they can already decide what position to put in the lower box of p' for every possible direction that could fill the upper box of p . By (6.15), such a choice is equivalent to picking a position of the composite $p \triangleleft p'$. Then by (6.19), following the bottom arrow φ_1 leads to the corresponding position of q given by $\varphi \triangleleft \varphi'$, while filling in the direction box of q and following $\varphi^\sharp$, then φ'_1 leads to the position of q' that goes in the bottom box of q' . Finally, once the user fills in the direction box of q' , following

the top arrow $(\varphi')^\sharp$ completes the specification of a direction of $p \triangleleft p'$. In this way, (6.41) can be thought of as a special case of (6.38).

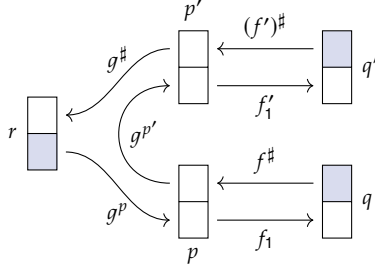
Example 6.42 (Dynamical systems and the composition product, revisited). In Section 6.1.4, we explained how the n -fold composition product $\varphi^{\triangleleft n}$ of a dynamical system $\varphi: Sy^S \rightarrow p$ models the behavior of running through the system n times, provided we choose the positions of $(Sy^S)^{\triangleleft n}$ appropriately. We can visualize this behavior using polyboxes—for example, here is what the $n = 3$ case looks like:



This polybox picture makes it visually clear what $\varphi^{\triangleleft 3}$ does, resolving the first issue we raised in Section 6.1.4, page 250: we now have a concise way of depicting the n -fold composite of a dynamical system. The second issue becomes evident when we look at which boxes are blue along the left: we would really like the position s_i to be entered above each direction s_i automatically, rather than having to specify the contents of those blue boxes manually—we should not have the option to fill those blue boxes in with anything else. We will see how to address this issue in Example 6.43.

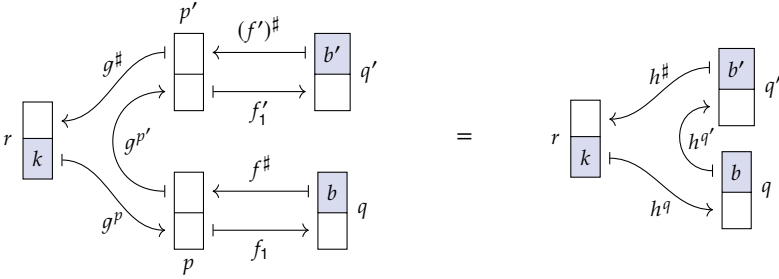
We can see the advantage of drawing polyboxes like (6.41) when we compose them with polyboxes that look like (6.38). Given a lens

$g: r \rightarrow p \triangleleft p'$, consider the polyboxes for $g \circ (f \triangleleft f')$:



To fill out these polyboxes, we start from the position box of r ; go right through g^p to the position box of p and through f_1 to the polyboxes of q ; loop back left through $f^\#$ to the direction box of p , up through $g^{p'}$ to the position box of p' , and right again through f_1' to the polyboxes for q' ; then left through $(f')^\#$ to the direction box of p' and through $g^\#$ to the direction box of r .

Say we knew that $g \circ (f \triangleleft f')$ were equal to some other lens $h: r \rightarrow q \triangleleft q'$:

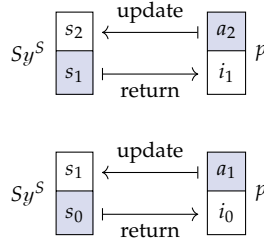


We filled in the corresponding blue boxes on either side of the equation with the same entries. Then for this equation to hold, each of the three white boxes in the domain and codomain on the left should end up with the same entry as the corresponding white box on the right (although the intermediary mechanics may differ). By following the arrows in order on either side, matching up the white boxes in the domain and codomain along the way, we can read off three equations:

$$g^p \circ f_1 = h^q, \quad f_{g^p k}^\# \circ g_k^{p'} \circ f_1' = h_k^{q'}, \quad \text{and} \quad (f')^\#_{g_k^{p'} f_{g^p k}^\# b} \circ g_k^\# = h_k^\#.$$

The converse holds as well: if the three equations above all hold, then $g \circ (f \triangleleft f') = h$.

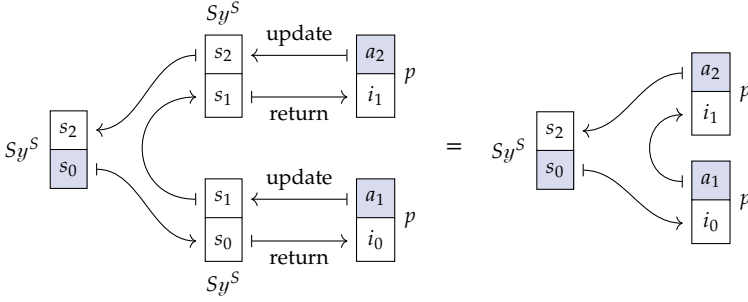
Example 6.43 (Transition lenses for state systems). In Example 6.42, we saw that the 2-fold composition product $\varphi^{\triangleleft 2}$ of a dynamical system $\varphi: Sy^S \rightarrow p$ can be drawn as follows:



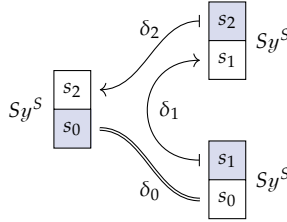
This *almost* models the behavior of running through the system twice, except that we should only have one blue box on the domain side—the one we fill with the initial state s_0 . The second blue box on the domain side, the one we fill with s_1 , should instead be filled automatically with the same state as the direction s_1 in the white box below it.

In fact, we would like to construct a lens $Sy^S \rightarrow p \triangleleft p$ that takes an initial state $s_0 \in S$ and runs the original system φ twice, returning two positions and receiving two directions before stopping at the new state s_2 . But we just learned how to take a composition product of lenses such as $\varphi^{\triangleleft 2}: Sy^S \triangleleft Sy^S \rightarrow p \triangleleft p$ and convert its domain to a new polynomial, say Sy^S , with only one blue box on the domain side—we can compose it with another lens $Sy^S \rightarrow Sy^S \triangleleft Sy^S$ like

so:



We will denote our new lens $Sy^S \rightarrow Sy^S \triangleleft Sy^S$ on the far left by δ . Let us take a closer look at how δ behaves on its own, labeling its arrows for ease of reference:



We see that δ arises naturally from the structure of Sy^S ; indeed, every state system can be equipped with such a lens, just as every state system can be equipped with a do-nothing section from Example 4.43. The bottom arrow $\delta_0: S \rightarrow S$ sending $s_0 \mapsto s_0$ is the identity on the position-set S : it sends each state to itself. We use a double arrow to denote this equality. While the middle arrow δ_1 also looks like an identity arrow, remember that we should think of it as depending on the left position box as well; so it is really a function $\delta_1: S \times S \rightarrow S$ sending (s_0, s_1) , a state-as-position s_0 and a state-as-direction s_1 at s_0 , to the new state-as-position s_1 . Similarly, $\delta_2: S \times S \times S \rightarrow S$ sends (s_0, s_1, s_2) , where the last coordinate is the state-as-direction s_2 at state-as-position s_1 , to the state-as-direction s_2 at the state-as-direction s_0 .

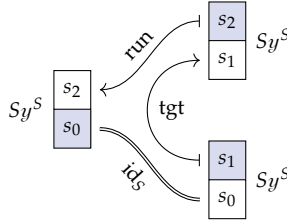
Note the crucial role δ_1 plays: it assigns every direction at a given state to the new state that the direction in question should point to, encoding how the state system transitions from one state to the next. We have been labeling each direction by the state that it points to, but we should really think of all the directions of Sy^S as pairs of states $(s, t) \in S \times S$, where (s, t) is a direction at state-as-position s that represents the transition from state s to state t . The structure of the polynomial Sy^S tells us the state that each direction transitions from, but it is δ_1 that tells us the new state $\delta_1(s, t) = t$ that (s, t) transitions to. For that reason, we call δ the *transition lens* of the state system Sy^S , and we call $\delta_1: S \times S \rightarrow S$ the *target function*, relabeling it *tgt* for short, indicating where each direction leads.

This is exactly what we wanted back in Example 4.43: a way to encode which directions of a state system point to which positions in the language of **Poly**. Expressions in that language are lenses such as δ , and polyboxes like the ones above are the way we write them down.

We highlighted δ_1 above, but the arrows δ_0 and δ_2 are no less important. As an identity function, $\delta_0 = \text{id}_S$ remembers the initial state s_0 as we shift from its original setting Sy^S , where we can only move one state away from s_0 , to a new setting $Sy^S \triangleleft Sy^S$, where we can think about all the ways to move two states away from s_0 . Meanwhile, δ_2 shifts us from the two-state setting back to the one-state setting while ensuring a sort of transitive coherence condition: the state where we end up after moving from s_0 through s_1 to s_2 is the same state we would end up at if we had moved from s_0 directly to s_2 . We will call it the *run function*, because it runs a sequence of two transitions together, and because it is what keeps the system running as it tells the original state system to actually move along one of its directions.

We draw the polyboxes picture of the transition lens δ of Sy^S

with our new arrow names below.



Example 6.43 resolves the second issue we raised in Section 6.1.4, page 250 for the case of $n = 2$, giving us a dynamical system $\delta \ddagger (\varphi \triangleleft \varphi): Sy^S \rightarrow p \triangleleft p$ that simulates stepping through the system φ twice. It remains to consider more general values of n .

The $n = 0$ case is modeled by the do-nothing section $\epsilon: Sy^S \rightarrow y$. But there is some overlap in how ϵ matches up state-positions with directions and how δ does. Put another way, if ϵ tells us how to do nothing, and δ tells us how to do two things, then we had better check that if one of the two things we do is nothing, then that is the same as doing just one thing. We must ensure that ϵ and δ agree on what they are saying about our state system.

Then for $n = 3$, we would like a dynamical system $Sy^S \rightarrow p \triangleleft p \triangleleft p$ that simulates stepping through φ three times. One way to do this would be to compose $\varphi^{\triangleleft 3}$ with a lens of the form $Sy^S \rightarrow (Sy^S)^{\triangleleft 3}$ that we obtain by extending δ from modeling two-step transitions to three-step transitions. But there are two ways to derive a lens with codomain $(Sy^S)^{\triangleleft 3}$ from δ : we could either take $\delta \ddagger (\delta \triangleleft \text{id}_{Sy^S})$, or we could take $\delta \ddagger (\text{id}_{Sy^S} \triangleleft \delta)$. For larger values of n , there are even more possibilities for what we could do, yet there should only be one dynamical system that models stepping through φ a fixed number of times. We need to guarantee that all these different ways of extending δ to n -step transitions end up expressing the same process.

In summary, we need a compatibility condition between ϵ and δ as well an associativity condition on δ to guarantee that it can be extended coherently. In fact, we already have all the tools we need to characterize these conditions: we will see exactly how to state the properties we want in the next chapter.

Exercise 6.44. Let $S := \mathbb{N}$ and $p := \mathbb{R}y^1$, and define $\varphi: Sy^S \rightarrow p$ to be the dynamical system with return function $\varphi_1 k := k$ and update function $\varphi_k^\# 1 := k + 1$.

1. Draw the polyboxes for φ and describe its dynamics: what does one run through the system look like?
2. Let $\delta: Sy^S \rightarrow Sy^S \triangleleft Sy^S$ be the transition lens of Sy^S , and draw the polyboxes for the new system $\delta \circ (\varphi \triangleleft \varphi): Sy^S \rightarrow p \triangleleft p$. Describe its dynamics: how does it model two runs through the system?

Exercise 6.45. As a lens whose domain is a state system, the transition lens $\delta: Sy^S \rightarrow Sy^S \triangleleft Sy^S$ of a state system Sy^S can be interpreted as a standalone dynamical system itself. Describe its dynamics.

6.3 Categorical properties of the composition product

We conclude this chapter by discussing several properties of the composition product, many of which we will leverage in the following chapters. We will focus on how $\triangleleft$ interacts with the other constructions on **Poly** that we have previously introduced.

6.3.1 Interaction with products and coproducts

The composition product behaves well with products and coproducts on the left.

Proposition 6.46 (Left distributivity of $\triangleleft$ over $+$ and $\times$). Given a polynomial r , the functor $(- \triangleleft r): \mathbf{Poly} \rightarrow \mathbf{Poly}$ that sends each $p \in \mathbf{Poly}$ to $p \triangleleft r$ commutes with coproducts and products (up to natural isomorphism). That is, for any $p, q \in \mathbf{Poly}$, we have the following natural isomorphisms:

$$(p + q) \triangleleft r \cong (p \triangleleft r) + (q \triangleleft r) \quad (6.47)$$

and

$$pq \triangleleft r \cong (p \triangleleft r)(q \triangleleft r). \quad (6.48)$$

More generally, given a set A and polynomials $(q_a)_{a \in A}$, we have the following natural isomorphisms:

$$\left(\sum_{a \in A} q_a \right) \triangleleft r \cong \sum_{a \in A} (q_a \triangleleft r) \quad (6.49)$$

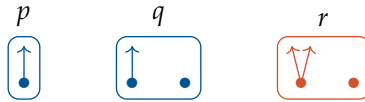
and

$$\left(\prod_{a \in A} q_a \right) \triangleleft r \cong \prod_{a \in A} (q_a \triangleleft r). \quad (6.50)$$

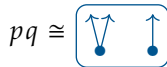
Proof. Formally, this follows from the fact that (co)products of functors $\mathbf{Set} \rightarrow \mathbf{Set}$ are computed pointwise (Proposition 1.37) and that (co)products in **Poly** coincide with (co)products in $\mathbf{Set}^{\mathbf{Set}}$ (Propositions 3.3 and 3.56). One could instead give an explicit proof using (6.5); this is done in Exercise 6.51. In fact, we will see yet another proof of (6.50) (and thus of (6.48)) in Exercise 6.69 #2. $\square$

Exercise 6.51. Prove Proposition 6.46 using the explicit formula for $\triangleleft$ given in (6.5) by manipulating sums and products.

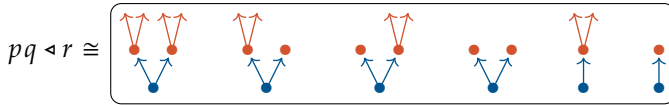
Example 6.52 (Picturing the left distributivity of $\triangleleft$ over $\times$). We can understand the left distributivity given by (6.48) intuitively using corolla forest pictures. Let $p := y$, $q := y + 1$, and $r := y^2 + 1$:



Then $pq \cong y^2 + y$ can be drawn as follows, with each corolla comprised of a p -corolla and a q -corolla with their roots glued together:

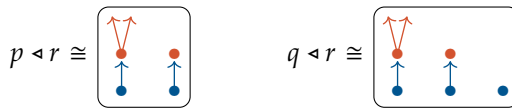


We can therefore draw $pq \triangleleft r$ by grafting r -corollas to leaves of pq in every way, as follows:

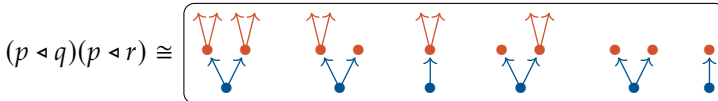


So each tree in $pq \triangleleft r$ is obtained by grafting together the roots of a p -corolla and a q -corolla, then attaching r -corollas to each leaf.

Alternatively, we can compute $p \triangleleft r$ and $q \triangleleft r$ separately, grafting r -corollas to leaves of p in every way, then to leaves of q in every way:



Their product is then obtained by taking each tree from $p \triangleleft r$ and pairing it with each tree from $q \triangleleft r$ by gluing their roots together:



So each tree in $(p \triangleleft r)(q \triangleleft r)$ is obtained by grafting r -corollas to each leaf of a p -corolla and a q -corolla before gluing their roots together.

But it does not matter if we graft r -corollas onto leaves first, or if we glue the roots of p - and q -corollas together first: the processes are equivalent. Hence the isomorphism $pq \triangleleft r \cong (p \triangleleft r)(q \triangleleft r)$ holds.

Exercise 6.53. Follow Example 6.52 with coproducts $(+)$ in place of products $(\times)$: use corolla forest pictures to explain the left distributivity given by (6.47).

Exercise 6.54. Show that for any set A and polynomials p, q , we have an isomorphism $A(p \triangleleft q) \cong (Ap) \triangleleft q$.

In Section 6.3.2, we will see how to generalize the left distributivity of $\triangleleft$ over products to arbitrary limits. But first, we observe that right distributivity does not hold.

Exercise 6.55. Show that the distributivities of Proposition 6.46 do not hold on the other side:

1. Find polynomials p, q, r such that $p \triangleleft (qr) \not\equiv (p \triangleleft q)(p \triangleleft r)$.
2. Find polynomials p, q, r such that $p \triangleleft (q + r) \not\equiv (p \triangleleft q) + (p \triangleleft r)$.

Nevertheless, there is something to be said about the relationship between $p \triangleleft q$, $p \triangleleft r$, and $p \triangleleft (qr)$. We will see this in action after we discuss how $\triangleleft$ preserves limits on the left.

6.3.2 Interaction with limits on the left

We saw in Theorem 5.34 that **Poly** has all limits, and we saw in Exercise 5.43 that these limits coincide with limits in **Set**^{Set}. Hence the argument in the proof of Proposition 6.46 by appealing to Proposition 1.37 can be generalized to arbitrary limits. It follows that $\triangleleft$ preserves all limits on the left. But we will present a proof of this fact from an alternative perspective: by appealing to the left coclosure of $\triangleleft$.

Proposition 6.56. The composition product is left coclosed. That is, there exists a left coclosure operation, which we denote $\left[\begin{smallmatrix} q \\ p \end{smallmatrix} \right] : \mathbf{Poly}^{\text{op}} \times \mathbf{Poly} \rightarrow \mathbf{Poly}$, such that there is a natural isomorphism

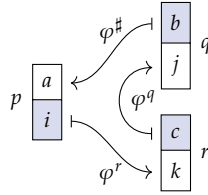
$$\mathbf{Poly}(p, r \triangleleft q) \cong \mathbf{Poly} \left(\left[\begin{smallmatrix} q \\ p \end{smallmatrix} \right], r \right). \quad (6.57)$$

In particular, the left coclosure operation sends $q, p \in \mathbf{Poly}$ to

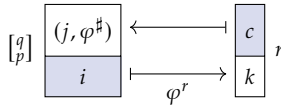
$$\left[\begin{smallmatrix} q \\ p \end{smallmatrix} \right] := \sum_{i \in p(1)} y^{q(p[i])}. \quad (6.58)$$

Proof. We present an argument using polyboxes; we leave the reader to write this proof in more standard mathematical notation in Exercise 6.59.

As in Example 6.39, a lens $\varphi: p \rightarrow r \triangleleft q$ can be written as follows:



But this is equivalent to the following protocol (to visualize this equivalence, imagine leaving the positions box for p , the arrow φ^r , and the polyboxes for r untouched, while dragging the polyboxes for q leftward to the directions box for p , merging all the data from q and the arrows φ^q and $\varphi^\#$ into a single on-directions arrow and directions box):



Here the on-directions function encodes the behaviors of both φ^q and $\varphi^\#$ by sending each $r[k]$ -direction c to both a q -position j , as φ^q does, and a function sending $q[j]$ -directions to $p[i]$ -directions, as $\varphi^\#$ does. So the polyboxes on the left represent a polynomial whose positions are the same as those of p , but whose directions at $i \in p(1)$ are pairs $(j, \varphi^\#)$ consisting of a q -position j and a function $\varphi^\#: q[j] \rightarrow p[i]$. Such pairs are precisely the elements of $q(p[i])$, so the polynomial represented by the polyboxes on the left is indeed the one defined as $\begin{bmatrix} q \\ p \end{bmatrix}$ in (6.58). It follows that there is a natural isomorphism between lenses $p \rightarrow r \triangleleft q$ and lenses $\begin{bmatrix} q \\ p \end{bmatrix} \rightarrow r$. $\square$

Exercise 6.59. Translate the polyboxes proof of Proposition 6.56 into standard mathematical notation, i.e. the $\sum$ and $\prod$ notation we have been using up till now.

Remark 6.60. The proof you gave in Exercise 6.59 may be more concise and more clearly rigorous than the one we presented in the main text. But polyboxes help us visualize exactly what this adjunction describes:

how data on the codomain-side of a lens $p \rightarrow r \triangleleft q$ can be repackaged and transferred to the domain-side of a new lens $\begin{bmatrix} q \\ p \end{bmatrix} \rightarrow r$.

Exercise 6.61. In stating Proposition 6.56, we implicitly assumed that $\begin{bmatrix} - \end{bmatrix}$ is a functor $\mathbf{Poly}^{\text{op}} \times \mathbf{Poly} \rightarrow \mathbf{Poly}$. Here we show that this is indeed the case.

1. Given a polynomial q and a lens $\varphi: p \rightarrow p'$, to what lens $\begin{bmatrix} q \\ p \end{bmatrix} \rightarrow \begin{bmatrix} q \\ p' \end{bmatrix}$ should the covariant functor $\begin{bmatrix} - \end{bmatrix}$ send φ ? Prove that your construction is functorial.
2. Given a polynomial p and a lens $\psi: q' \rightarrow q$, to what lens $\begin{bmatrix} q \\ p \end{bmatrix} \rightarrow \begin{bmatrix} q' \\ p \end{bmatrix}$ should the contravariant functor $\begin{bmatrix} - \end{bmatrix}$ send ψ ? Prove that your construction is functorial.

Exercise 6.62. In personal communication, David Jaz Myers noted the left coclosure is a left Kan extension

$$\begin{array}{ccc}
 \mathbf{Set} & \xrightarrow{p} & \mathbf{Set} \\
 & \searrow q \quad \Downarrow & \nearrow \begin{bmatrix} q \\ p \end{bmatrix} \\
 & \mathbf{Set}_* &
 \end{array}$$

so $\begin{bmatrix} q \\ p \end{bmatrix} \cong \text{Lan}_q p$. If you know what a left Kan extension is, verify this.

Exercise 6.63. Let A and B be sets, and let p and q be polynomials.

1. Prove that the following natural isomorphism holds:

$$\mathbf{Poly}(Ay^B, p) \cong \mathbf{Set}(A, p(B)). \quad (6.64)$$

2. Prove that the following natural isomorphism holds:

$$\mathbf{Poly}\left(Ay \triangleleft p \triangleleft y^B, q\right) \cong \mathbf{Poly}\left(p, y^A \triangleleft q \triangleleft By\right). \quad (6.65)$$

(Hint: break the isomorphism down into two parts. You may find (5.9) helpful.)

Example 6.66 (Dynamical systems as coalgebras). Taking $A := B := S \in \mathbf{Set}$ in (6.64), we find that there is a natural isomorphism between dynamical systems $Sy^S \rightarrow p$ and functions $S \rightarrow p(S)$. Such a function is known as a *coalgebra for the functor p* or a *p -coalgebra*.⁵

Coalgebras as models of dynamical systems have been studied extensively in the context of computer science, most notably by Jacobs in [Jac17]. Indeed, much of what we developed in Sections 4.1 and 4.2 stems from the theory of coalgebras. The coalgebraic perspective has the benefit of staying in the familiar category of sets; moreover, it can be generalized to functors $\mathbf{Set} \rightarrow \mathbf{Set}$ that are not polynomial, although many of the interesting examples are.

On the other hand, we have already seen that viewing dynamical systems as lenses $Sy^S \rightarrow p$ rather than as functions $S \rightarrow p(S)$ has the benefit of isolating the internal state system to the domain and the external interface to the codomain, aiding both intuition and functionality. Additionally, our adjunction lets us switch to the coalgebraic perspective whenever we see fit: **Poly** lets us talk about both.

Proposition 6.67 (Left preservation of limits). The operation $\triangleleft$ preserves limits on the left (up to natural isomorphism). That is, if $\mathcal{J}$ is a category, $p_- : \mathcal{J} \rightarrow \mathbf{Poly}$ is a functor, and $q \in \mathbf{Poly}$ is a polynomial,

⁵There are two versions of coalgebras we are interested in (and more that we are not) with distinct definitions: a *coalgebra for a functor*, which is the version used here, and a *coalgebra for a comonad*, which is a coalgebra for a functor with extra conditions that we will introduce later in Section 7.3.3. The version we are using will usually be clear from context—here, for example, we do not expect p to be a comonad—but we will try to be explicit with our terminology whenever the interpretation may be ambiguous.

then there is a natural isomorphism

$$\left(\lim_{j \in \mathcal{J}} p_j \right) \triangleleft q \cong \lim_{j \in \mathcal{J}} (p_j \triangleleft q). \quad (6.68)$$

Proof. By Proposition 6.56, the functor $(- \triangleleft q): \mathbf{Poly} \rightarrow \mathbf{Poly}$ is the right adjoint of the functor $[^q]: \mathbf{Poly} \rightarrow \mathbf{Poly}$, and right adjoints preserve limits. $\square$

Exercise 6.69.

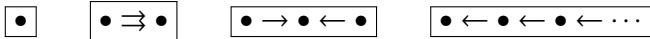
1. Complete Exercise 6.27 #3 using (6.68) and (6.49).
2. Deduce (6.50) using (6.68).

6.3.3 Interaction with limits on the right

We have seen that $\triangleleft$ preserves limits on the left; on the other hand, Exercise 6.55 shows that $\triangleleft$ does not even preserve products on the right, so it certainly does not preserve all limits. But it turns out that there is a special class of limits that $\triangleleft$ does preserve on the right.

Definition 6.70 (Connected limit). A *connected limit* is one whose indexing category $\mathcal{J}$ is nonempty and connected. That is, $\mathcal{J}$ has at least one object, and any two objects are connected by a finite zigzag of arrows.

Example 6.71. The following categories are connected:



In particular, equalizers, pullbacks, and directed limits are examples of connected limits.

The following categories are *not* connected:



In particular, terminal objects and products are *not* examples of connected limits.

Connected limits are intimately related to slice categories, which we defined back in Definition 5.79. For example, products in a slice category $\mathcal{C}/c$ are just pullbacks in $\mathcal{C}$, allowing us to view a non-connected limit as a connected one. By relating **Poly** to its slice categories via an adjunction, we can show that $\triangleleft$ preserves connected limits. (An alternative proof of this fact can be found in [GK12, Proposition 1.16].)

Recall that objects in a slice category $\mathcal{C}/c$ are just morphisms with codomain c . For ease of notation, we will often suppress the actual morphism and just write down the name of its domain when there is a canonical choice for the morphism, or when it is clear from context. So for example, on the left-hand side of (6.73) below, p represents the lens $f: p \rightarrow q \triangleleft 1$ and $q \triangleleft r$ represents the lens $q \triangleleft !: q \triangleleft r \rightarrow q \triangleleft 1$, both objects in the slice category **Poly**/ $q \triangleleft 1$.

Proposition 6.72. Given polynomials $p, q, r \in \mathbf{Poly}$ and a lens $f: p \rightarrow q \triangleleft 1$, there is a natural isomorphism

$$\mathbf{Poly}/(q \triangleleft 1)(p, q \triangleleft r) \cong \mathbf{Poly}\left(p \overset{f}{\frown} q, r\right), \quad (6.73)$$

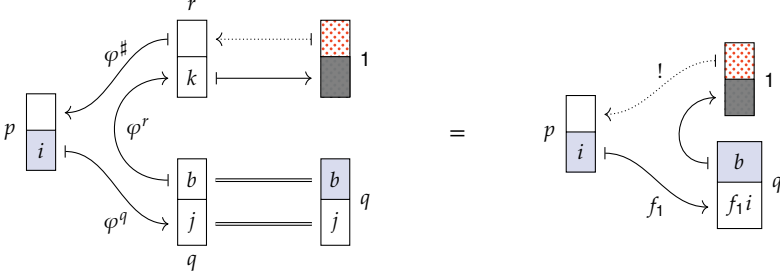
where

$$p \overset{f}{\frown} q := \sum_{i \in p(1)} q[f_1 i] y^{p[i]}. \quad (6.74)$$

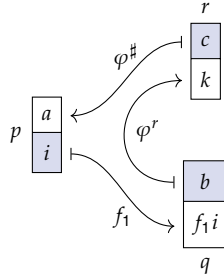
Proof. Again, we present our argument using polyboxes, leaving the reader to rewrite this proof in standard mathematical notation in Exercise 6.76.

To consider $q \triangleleft r$ as an object in **Poly**/ $(q \triangleleft 1)$, we implicitly use the lens $q \triangleleft !: q \triangleleft r \rightarrow q \triangleleft 1$. By definition, morphisms from f to $q \triangleleft !$ in **Poly**/ $(q \triangleleft 1)$ are lenses $\varphi: p \rightarrow q \triangleleft r$ for which $\varphi \circ (q \triangleleft !) = f$. We write

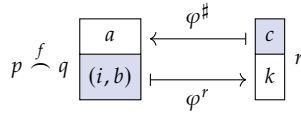
this equation using polyboxes:



We read off the picture that a lens $\varphi: p \rightarrow q \triangleleft r$ is a morphism from f to $q \triangleleft !$ in $\mathbf{Poly}/q \triangleleft 1$ if and only if $\varphi^q = f_1$. So morphisms from f to $q \triangleleft !$ are equivalent to protocols



with f_1 fixed. But this, in turn, is equivalent to the following protocol (to visualize this equivalence, imagine leaving the polyboxes for r , the arrow $\varphi^\#$, and the directions box for p untouched; while dragging the polyboxes for q leftward to the positions box for p , merging the data from q and the predetermined arrow f into a single positions box and adapting the arrow φ^r into an on-positions arrow):



Here the user can provide both the p -position i and the $q[f_1 i]$ -direction b right from the start, as they know what to expect from f ahead of

time. Then the on-positions function encodes the behavior of φ^r . So the polyboxes on the left represent a polynomial whose positions are pairs (i, b) with $i \in p(1)$ and $b \in q[f_1 i]$, and whose directions at (i, b) are the directions of the original polynomial p at i . This is precisely the polynomial we defined in (6.74), so the isomorphism holds. $\square$

Remark 6.75. As a lens $p \rightarrow q \triangleleft 1$ can be identified with its on-positions function $p(1) \rightarrow q(1)$, we will use the notation $p \overset{f}{\frown} q$ whether f is a lens $f: p \rightarrow q \triangleleft 1$ or a function $f: p(1) \rightarrow q(1)$.

Exercise 6.76.

1. Translate the polyboxes proof of Proposition 6.72 into standard mathematical notation.
2. Prove that the following natural isomorphism holds:

$$\mathbf{Poly}(p, q \triangleleft r) \cong \sum_{f: p(1) \rightarrow q(1)} \mathbf{Poly}\left(p \overset{f}{\frown} q, r\right). \quad (6.77)$$

Thus the functor $(q \triangleleft -): \mathbf{Poly} \rightarrow \mathbf{Poly}$ is said to have a *left multiadjoint*.

Exercise 6.78. In stating Proposition 6.72, we implicitly assumed that $p \overset{f}{\frown} q \in \mathbf{Poly}$ is functorial in each variable: covariantly on the left and contravariantly on the right. Here we show that this is indeed the case.

1. Given lenses $f: p \rightarrow q \triangleleft 1$ and $g: p' \rightarrow p$, construct the lens $p' \overset{g \circ f}{\frown} q \rightarrow p \overset{f}{\frown} q$ to which the covariant functor $- \overset{f}{\frown} q$ should send g . Prove that your construction is functorial.
2. Given lenses $f: p \rightarrow q \triangleleft 1$ and $h: q \rightarrow q'$, to construct the lens $p \overset{f \circ (h \triangleleft 1)}{\frown} q' \rightarrow p \overset{f}{\frown} q$ to which the contravariant functor $p \overset{f}{\frown} -$ should send h . Prove that your construction is functorial.

Theorem 6.79 (Preservation of connected limits). The operation $\triangleleft$ preserves connected limits on both sides. That is, if $\mathcal{G}$ is a connected category, $p: \mathcal{G} \rightarrow \mathbf{Poly}$ is a functor, and $q \in \mathbf{Poly}$ is a polynomial, then there are natural isomorphisms

$$\left(\lim_{j \in \mathcal{G}} p_j \right) \triangleleft q \cong \lim_{j \in \mathcal{G}} (p_j \triangleleft q) \quad \text{and} \quad q \triangleleft \left(\lim_{j \in \mathcal{G}} p_j \right) \cong \lim_{j \in \mathcal{G}} (q \triangleleft p_j).$$

Proof. The claim for the left side is just a special case of Proposition 6.67; it remains to prove the claim on the right.

By Theorem 5.34, $\mathbf{Poly}$ is complete, so by [nLa19, Theorem 4.3],⁶ it suffices to show that the functor $(q \triangleleft -): \mathbf{Poly} \rightarrow \mathbf{Poly}$ preserves wide pullbacks on the right. By Proposition 6.72, the functor $(q \triangleleft -): \mathbf{Poly} \rightarrow \mathbf{Poly}/q \triangleleft 1$ is a right adjoint, so it preserves limits, including wide pullbacks. Thus $(q \triangleleft -)$ sends a wide pullback over $r \in \mathbf{Poly}$ to a wide pullback over the canonical lens $q \triangleleft r \rightarrow q \triangleleft 1$ in $\mathbf{Poly}/q \triangleleft 1$, corresponding to a limit in $\mathbf{Poly}$ of a diagram consisting of arrows to $q \triangleleft r$ and arrows to $q \triangleleft 1$ factoring through $q \triangleleft r$. So up to isomorphism, this limit is a wide pullback in $\mathbf{Poly}$ over $q \triangleleft r$, namely $(q \triangleleft -): \mathbf{Poly} \rightarrow \mathbf{Poly}$ applied to the original wide pullback. So $\triangleleft$ preserves wide pullbacks on the right. $\square$

Exercise 6.80. Use Theorem 6.79 in the following.

1. Let p be a polynomial, thought of as a functor $p: \mathbf{Set} \rightarrow \mathbf{Set}$. Show that p preserves connected limits (of sets).
2. Show that for any polynomials p, q, r we have an isomorphism:

$$p \triangleleft (qr) \cong (p \triangleleft q) \times_{(p \triangleleft 1)} (p \triangleleft r). \quad (6.81)$$

3. Take the polynomials p, q, r from the counterexample you found in Exercise 6.55 #1 and check that (6.81) holds.

While we are here, it will be helpful to record the following.

⁶The idea is that every connected limit can be written in terms of wide pullbacks.

Proposition 6.82. For any polynomial $q \in \mathbf{Poly}$, tensoring with q (on either side) preserves connected limits. That is, if $\mathcal{G}$ is connected and $p: \mathcal{G} \rightarrow \mathbf{Poly}$ is a functor, then there is a natural isomorphism:

$$\left(\lim_{j \in \mathcal{G}} p_j \right) \otimes q \cong \lim_{j \in \mathcal{G}} (p_j \otimes q).$$

6.3.4 Interaction with parallel products

Before we explore how $\otimes$ interacts with $\triangleleft$, here is a warm-up exercise.

Exercise 6.83. Let A and B be arbitrary sets, and let p be an arbitrary polynomial. Which of the following isomorphisms always hold?

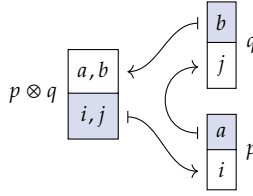
If the isomorphism does not always hold, is there still a canonical lens in one direction or the other?

1. $(Ay) \otimes (By) \cong? (Ay) \triangleleft (By).$
2. $y^A \otimes y^B \cong? y^A \triangleleft y^B.$
3. $A \otimes B \cong? A \triangleleft B.$
4. $By \otimes p \cong? By \triangleleft p.$
5. $y^A \otimes p \cong? y^A \triangleleft p.$
6. $p \otimes By \cong? p \triangleleft By.$
7. $p \otimes y^A \cong? p \triangleleft y^A.$

What do all of the lenses you found in this exercise have in common (whether or not they were isomorphisms)?

Example 6.84 (Lenses from $\otimes$ to $\triangleleft$). For any p and q , there is an interesting cartesian lens $o_{p,q}: p \otimes q \rightarrow p \triangleleft q$ that, stated informally, “orders” the operation, taking the symmetric monoidal product $\otimes$ and reinterprets it as a special case of the asymmetric monoidal product $\triangleleft$. Defining this lens in the usual way is tedious and unilluminating, but written in polyboxes, the lens looks like this (recall that positions of $p \otimes q$ are just pairs of positions of p and q , while directions at each such pair are pairs of directions of p and q , one at each position in the pair; we drop the parentheses around the

ordered pair for readability):



Usually, the positions box of q is allowed to depend on the directions box of p in the polyboxes for $p \triangleleft q$ on its own. But in the polyboxes above, j is not allowed to depend on a in $p \otimes q$ on the left, so the arrow from the positions box of q to the directions box of p on the right doesn't actually take a into account at all. So the lens $o_{p,q}$ is in some sense the inclusion of the order-independent positions of $p \triangleleft q$; when drawn as trees, the positions in its image are the ones whose upper-level corollas are all the same. And of course we can flip the order using the symmetry $q \otimes p \cong p \otimes q$. This is, we just as well have a lens $p \otimes q \rightarrow q \triangleleft p$.

Both $\otimes$ and $\triangleleft$ have the same monoidal unit, the identity functor y , whose identity is the unique lens $y \rightarrow y$. In fact the lenses $o_{p,q}$ constitute a lax monoidal functor $(\mathbf{Poly}, y, \otimes) \rightarrow (\mathbf{Poly}, y, \triangleleft)$. In particular, $o_{p,q}$ commutes with associators and unitors.

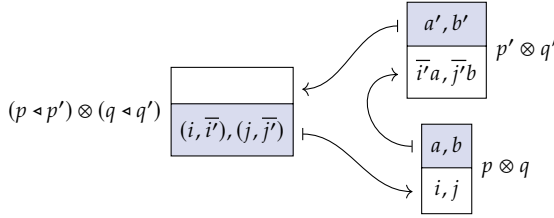
This can be used in the following way. Lenses $p \rightarrow q \triangleleft r$ into composites are fairly easy to understand (through polyboxes, for example), whereas lenses $q \triangleleft r \rightarrow p$ are not so easy to think about. However, given such a lens, one may always compose it with $o_{q,r}$ to obtain a lens $q \otimes r \rightarrow p$. This is quite a bit simpler to think about: they are our familiar interaction patterns from Section 4.4.

It turns out that the monoidal structures $\otimes$ and $\triangleleft$ together satisfy an interesting property known as *duoidality*. We will not give the full definition of what it means for two monoidal structures to be duoidal here—there are a few commutative diagrams to verify for technical reasons—but the key condition is that there is a natural lens

$$(p \triangleleft p') \otimes (q \triangleleft q') \rightarrow (p \otimes q) \triangleleft (p' \otimes q'). \quad (6.85)$$

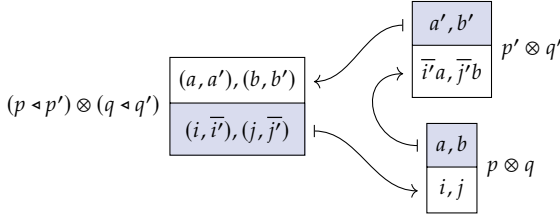
Proposition 6.86. The monoidal structures $(y, \otimes)$ and $(y, \triangleleft)$ together comprise a duoidal structure on **Poly**.

Idea of proof. The key is to give the natural lens from (6.85), as follows. A position of $p \triangleleft p'$ is a pair $(i, \bar{i}')$ with $i \in p(1)$ and $\bar{i}': p[i] \rightarrow p'(1)$; similarly, a position $q \triangleleft q'$ is a pair $(j, \bar{j}')$ with $j \in q(1)$ and $\bar{j}': q[j] \rightarrow q'(1)$. So we can define the first two parts of the lens using polyboxes, like so (again we drop parentheses around some ordered pairs for readability):



Here (a, b) is a direction of $p \otimes q$ at (i, j) , with $a \in p[i]$ and $b \in q[j]$.

Then to fill in the remaining empty box, we need a direction of $p \triangleleft p'$ at $(i, \bar{i}')$, which can be given by the $p[i]$ -direction a followed by a $p'[\bar{i}'(a)]$ -direction, namely a' . We also need a direction of $q \triangleleft q'$, which can be obtained analogously:



□

6.3.5 Interaction with vertical and cartesian lenses

We conclude this section by examining how $\triangleleft$ interacts with vertical and cartesian lenses, as defined in Definition 5.52.

Proposition 6.87 (Preservation of cartesian lenses). If $\varphi: p \rightarrow p'$ and $\psi: q \rightarrow q'$ are cartesian lenses, then so is $\varphi \triangleleft \psi: p \triangleleft q \rightarrow p' \triangleleft q'$.

Proof. We use the third characterization of cartesian lenses given in Proposition 5.65, as lenses whose naturality squares are pullbacks. For any sets A, B and function $h: A \rightarrow B$, consider the diagram

$$\begin{array}{ccccc}
 p \triangleleft q \triangleleft A & \rightarrow & p' \triangleleft q \triangleleft A & \rightarrow & p' \triangleleft q' \triangleleft A \\
 \downarrow & & \downarrow & & \downarrow \\
 p \triangleleft q \triangleleft B & \rightarrow & p' \triangleleft q \triangleleft B & \rightarrow & p' \triangleleft q' \triangleleft B.
 \end{array}$$

The square on the left is a pullback because $\varphi: p \rightarrow p'$ is cartesian. Meanwhile, the square on the right is a pullback because $\psi: q \rightarrow q'$ is cartesian and $\triangleleft$ preserves pullbacks by Theorem 6.79. Hence the outer rectangle is a pullback as well, implying that $\varphi \triangleleft \psi: p \triangleleft q \rightarrow p' \triangleleft q'$ is cartesian. $\square$

Exercise 6.88. Let $\varphi: p \rightarrow p'$ and $\psi: q \rightarrow q'$ be lenses.

1. Show that if φ is an isomorphism and ψ is vertical, then $\varphi \triangleleft \psi$ is vertical.
2. Find a vertical lens φ and a polynomial q for which the lens $\varphi \triangleleft q: p \triangleleft q \rightarrow p' \triangleleft q$ is not vertical.

6.4 Summary and further reading

In this chapter we introduced the composition (sometimes called “substitution”) product $\triangleleft$. Given polynomials p, q thought of as functors, their composite is again polynomial and is given by $p \triangleleft q$. We explained how it looks in terms of algebra, e.g. how to compute $y^2 \triangleleft (y + 1)$; in terms of sets, e.g. as a sum-product-sum-product $\sum \prod \sum \prod$, which can be reduced to a single $\sum \prod$; in terms of trees, by stacking corollas on top of corollas; and in terms of polyboxes. We paid particular attention to lenses into a composite $p \rightarrow q \triangleleft r$.

This allowed us to explain how to think of dynamical systems $Sy^S \rightarrow p \triangleleft q$ with composite interfaces as multi-step machines: each

state produces a p -position i , and then for every $p[i]$ -direction produces a q -position j , and finally for every $q[j]$ -direction returns an updated state in S .

Finally, we discussed some facts of the composition product. For example, we showed that $- \triangleleft q$ has a left adjoint $\left[\begin{smallmatrix} q \\ - \end{smallmatrix} \right]$ and that $q \triangleleft -$ has a left multi-adjoint $- \frown q$. We also explained that $p \triangleleft q$ preserves all with limits in the variable p and all connected limits in the variable q . We also explained the duoidal interaction between $\otimes$ and $\triangleleft$, i.e. the natural lens $\triangleleft \otimes \triangleleft \rightarrow \otimes \triangleleft \otimes$, and how $\triangleleft$ interacts with cartesian lenses.

Polynomial composition is one of the best known aspects of polynomial functors. Again, see [GK12] for more on this. We learned of the left coclosure (see Proposition 6.56) from Josh Meyers, though it may have already been known in the containers community.

6.5 Exercise solutions

Solution to Exercise 6.7.

1. We have that $y^2 \triangleleft q \cong q \times q \cong y^6 + 2y^3 + 1$.
2. We have that $y^1 \triangleleft q \cong q \cong y^3 + 1$.
3. Combining the previous parts, we have that $(y^2 + y^1) \triangleleft q \cong q \times q + q \cong y^6 + 3y^3 + 2$.
4. Since $p[1] \cong 2$ and $q(1) \cong 2$, there are $2^2 = 4$ functions $p[1] \rightarrow q(1)$.
5. When $\bar{j}_1: p[1] \rightarrow q(1)$ is one of the two possible bijections, we have that

$$\sum_{a \in p[1]} q[\bar{j}_1 a] \cong q[1] + q[2] \cong 3 + 0 \cong 3.$$

When $\bar{j}_1: p[1] \rightarrow q(1)$ sends everything to $1 \in q(1)$, we have

$$\sum_{a \in p[1]} q[\bar{j}_1 a] \cong q[1] + q[1] \cong 3 + 3 \cong 6.$$

Finally, when $\bar{j}_1: p[1] \rightarrow q(1)$ sends everything to $2 \in q(1)$, we have

$$\sum_{a \in p[1]} q[\bar{j}_1 a] \cong q[2] + q[2] \cong 0 + 0 \cong 0.$$

6. Since $p[2] \cong 1$ and $q(1) \cong 2$, there are $2^1 = 2$ functions $p[2] \rightarrow q(1)$.
7. When $\bar{j}_2: p[2] \rightarrow q(1)$ maps to $1 \in q(1)$, we have that $\sum_{a \in p[2]} q[\bar{j}_2 a] \cong q[1] \cong 3$, and when $\bar{j}_2: p[2] \rightarrow q(1)$ maps to $2 \in q(1)$, we have that $\sum_{a \in p[2]} q[\bar{j}_2 a] \cong q[2] \cong 0$.
8. From the previous parts, we have that

$$\sum_{i \in p(1)} \sum_{\bar{j}: p[i] \rightarrow q(1)} y^{\sum_{a \in p[i]} q[\bar{j}_i a]} \cong (2y^3 + y^6 + y^0) + (y^3 + y^0) \cong y^6 + 3y^3 + 2,$$

which agrees with what $p \triangleleft q$ should be.

Solution to Exercise 6.8.

1. Given representable polynomials $p := y^A$ and $q := y^B$, we have that $p \triangleleft q \cong (y^B)^A \cong y^{AB}$, which is also representable.
2. Given linear polynomials $p := Ay$ and $q := By$, we have that $p \triangleleft q \cong A(By) \cong AB y$, which is also linear.
3. Given constant polynomials $p := A$ and $q := B$, we have that $p \triangleleft q \cong A$, which is also constant (see also Exercise 6.27).

Solution to Exercise 6.9.

Given $A \in \mathbf{Set}$ and $q \in \mathbf{Poly}$, we have

$$\begin{aligned}
 y^A \triangleleft q &\cong \sum_{\bar{j}: A \rightarrow q(1)} y^{\sum_{a \in A} q[\bar{j}a]} & (6.6) \\
 &\cong \sum_{\varphi: Ay \rightarrow q} y^{\sum_{a \in A} q[\varphi_1 a]} \\
 &\cong [Ay, q], & (4.79)
 \end{aligned}$$

because a lens $\varphi: Ay \rightarrow q$ has an on-positions function $A \rightarrow q(1)$ and uniquely determined on-directions functions.

Solution to Exercise 6.12.

We wish to show that (6.13) could replace (6.11) in Definition 6.10. We claim that (6.11) and (6.13) are in fact the same function; that is, that the following square commutes:

$$\begin{array}{ccc}
 p(q(X)) & \xrightarrow{f_{q(X)}} & p'(q(X)) \\
 p(g_X) \downarrow & & \downarrow p'(g_X) \\
 p(q'(X)) & \xrightarrow{f_{q'(X)}} & p'(q'(X))
 \end{array}$$

Indeed it does, by the naturality of f .

Solution to Exercise 6.17.

1. The instructions associated with a polynomial $p \triangleleft p \triangleleft p$ are:
 1. choose a p -position i ;
 2. for each $p[i]$ -direction a :
 - 2.1. choose a p -position i' ;
 - 2.2. for each $p[i']$ -direction a' :
 - 2.2.1. choose a p -position i'' ;
 - 2.2.2. for each $p[i'']$ -direction a'' :
 - 2.2.2.1. choose a future.
2. To choose an element of $p \triangleleft p \triangleleft 1$:
 1. choose a p -position i ;
 2. for each $p[i]$ -direction a :
 - 2.1. choose a p -position i' ;
 - 2.2. for each $p[i']$ -direction a' :
 - 2.2.1. done.

Solution to Exercise 6.18.

We have lenses $f: p \rightarrow p'$ and $g: q \rightarrow q'$.

1. By Proposition 3.44, the $q(X)$ -component of f is a function $f_{q(X)}: p(q(X)) \rightarrow p'(q(X))$ that sends every (i, h) with $i \in p(1)$ and $h: p[i] \rightarrow q(X)$ to $(f_1 i, f_i^\# \circ h)$. We can think of the function $h: p[i] \rightarrow q(X)$ equivalently as a function $\bar{j}: p[i] \rightarrow q(1)$ and, for each $a \in p[i]$, a function $h_a: q[\bar{j}a] \rightarrow X$. So $f_{q(X)}: (p \triangleleft q)(X) \rightarrow (p' \triangleleft q')(X)$ sends

$$(i, \bar{j}, (h_a)_{a \in p[i]}) \mapsto \left(f_1 i, f_i^\# \circ \bar{j}, \left(h_{f_i^\# a'} \right)_{a' \in p'[f_1 i]} \right).$$

2. By Proposition 3.44, the X -component of g is a function $g_X: q(X) \rightarrow q'(X)$ that sends every (j, k) with $j \in q(1)$ and $k: q[j] \rightarrow X$ to $(g_1 j, g_j^\# \circ k)$ in $q'(X)$. Then by Proposition 2.10, applying p' to this X -component yields a function $p'(q(X)) \rightarrow p'(q'(X))$ that sends every $(i', \bar{j}', (h'_{a'})_{a' \in p'[\bar{j}']})$ with $i' \in p'(1)$ as well as $\bar{j}': p'[\bar{j}'] \rightarrow q(1)$ and $h'_{a'}: q[\bar{j}'a'] \rightarrow X$ to

$$\left(i', \bar{j}' \circ g_1, \left(g_{\bar{j}'a'}^\# \circ h'_{a'} \right)_{a' \in p'[\bar{j}']} \right).$$

3. By Definition 6.10, the horizontal composite of f and g is the natural transformation $f \triangleleft g: p \triangleleft p' \rightarrow q \triangleleft q'$ whose X -component is the composite of the answers to #1 and #2, sending

$$\begin{aligned} (i, \bar{j}, (h_a)_{a \in p[i]}) &\mapsto \left(f_1 i, f_i^\# \circ \bar{j}, \left(h_{f_i^\# a'} \right)_{a' \in p'[f_1 i]} \right) \\ &\mapsto \left(f_1 i, f_i^\# \circ \bar{j} \circ g_1, \left(g_{\bar{j}f_i^\# a'}^\# \circ h_{f_i^\# a'} \right)_{a' \in p'[f_1 i]} \right). \end{aligned}$$

4. We use Corollary 3.47 to translate the answer to #3 into a lens $f \triangleleft g: p \triangleleft q \rightarrow p' \triangleleft q'$, as follows. Its on-positions function is the 1-component $(f \triangleleft g)_1$, which sends every $(i, \bar{j})$ with $i \in p(1)$ and $\bar{j}: p[i] \rightarrow q(1)$ to

$$(f_1 i, f_i^\# \circ \bar{j} \circ g_1).$$

Then for each such $(i, \bar{j})$, if we apply the $(p \triangleleft q)[(i, \bar{j})]$ -component of $f \triangleleft g$ to the element $(i, \bar{j}, (\iota_a)_{a \in p[i]})$, where $\iota_a: q[\bar{j}a] \rightarrow (p \triangleleft q)[(i, \bar{j})] \cong \sum_{a \in p[i]} q[\bar{j}a]$ is the canonical inclusion, then take the last coordinate of the result, we obtain for each $a' \in p'[f_1 i]$ the function

$$q'[g_1 \bar{j} f_i^\# a'] \xrightarrow{g_{\bar{j}f_i^\# a'}^\#} q[\bar{j} f_i^\# a'] \xrightarrow{\iota_{f_i^\# a'}} \sum_{a \in p[i]} q[\bar{j}a] \cong (p \triangleleft q)[(i, \bar{j})].$$

These can equivalently be thought of as a single function from

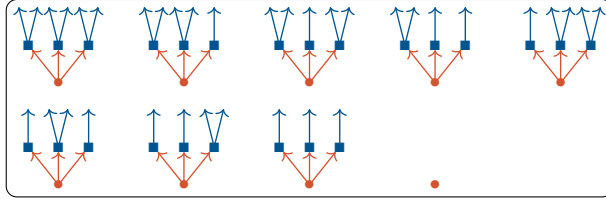
$$\sum_{a' \in p'[f_1 i]} q'[g_1 \bar{j} f_i^\# a'] \cong (p' \triangleleft q')[(f \triangleleft g)_1(i, \bar{j})]$$

which Corollary 3.47 tells us is the on-directions function of $f \triangleleft g$ at $(i, \bar{j})$, that sends every (a', b') with $a' \in p'[f_i i]$ and $b' \in q'[g_1 \bar{j} f_i^\# a']$ to

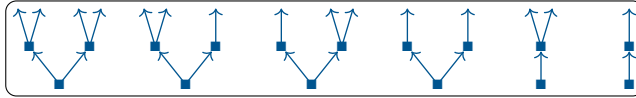
$$\left(f_i^\# a', g_{\bar{j} f_i^\# a'}^\# b' \right).$$

Solution to Exercise 6.25.

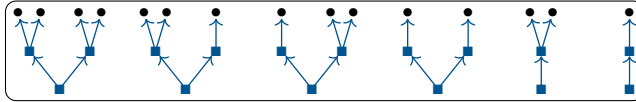
1. Here is a picture of $q \triangleleft p$, where each tree is obtained by taking a q -corolla and grafting p -corollas to every leaf:



2. Here is a picture of $p \triangleleft p$:



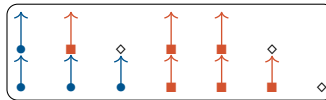
3. To obtain a picture of $p \triangleleft p \triangleleft 1$, we take our picture of $p \triangleleft p$ and graft the single, leafless 1-corolla onto every (height-2) leaf:



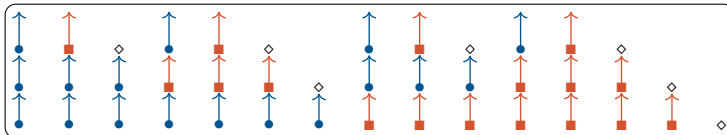
Before we draw the remaining composites, here is a picture of r itself, with different colors and shapes to distinguish the different positions:



4. Here is a picture of $r \triangleleft r$:

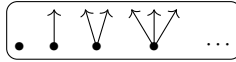


5. Here is a picture of $r \triangleleft r \triangleleft r$:

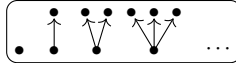


Solution to Exercise 6.27.

1. We pick the list polynomial, $p := 1 + y + y^2 + y^3 + \dots$, drawn as follows:



Then here is a picture of $p \triangleleft 1$:



Below, X is a set and p is a polynomial.

2. A constant functor composed with any functor is still the same constant functor, so $X \triangleleft p \cong X$. We can also verify this using (6.5):

$$X \triangleleft p \cong \sum_{i \in X} \prod_{a \in \emptyset} \sum_{j \in p(1)} \prod_{b \in p[j]} y \cong \sum_{i \in X} 1 \cong X.$$

3. When viewed as functors, it is easy to see that $p \triangleleft X \cong p(X)$. We can also verify this using (6.5):

$$p \triangleleft X \cong \sum_{i \in p(1)} \prod_{a \in p[i]} \sum_{j \in X} \prod_{b \in \emptyset} y \cong \sum_{i \in p(1)} \prod_{a \in p[i]} \sum_{j \in X} 1 \cong \sum_{i \in p(1)} \prod_{a \in p[i]} X \cong \sum_{i \in p(1)} X^{p[i]} \cong p(X).$$

Solution to Exercise 6.28.

Every component of the identity natural transformation on the constant functor X is the identity function $\text{id}_X: X \rightarrow X$ on X as a set. So by Definition 6.10, any component of the composition product $\varphi \triangleleft X$ viewed as a natural transformation is given by the composite function

$$p(X) \xrightarrow{\varphi_X} q(X) \xrightarrow{q(\text{id}_X)} q(X).$$

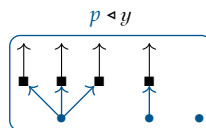
By functoriality, $q(\text{id}_X)$ is itself an identity function, so every component of $\varphi \triangleleft X$ is the X -component of φ . Therefore $\varphi \triangleleft X$ as a function can be identified with the X -component of φ .

Solution to Exercise 6.29.

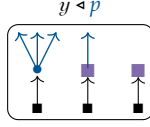
1. The polynomial y is the identity functor on **Set**.
2. Composing any functor with the identity functor yields the original functor, so $p \triangleleft y \cong p \cong y \triangleleft p$.
3. Before we draw $y \triangleleft p$ and $p \triangleleft y$, here are pictures of p and y individually as corolla forests:



Now here is a picture of $p \triangleleft y$, obtained by grafting the one-leaf y -corolla to all the leaves of each p -corolla in turn:



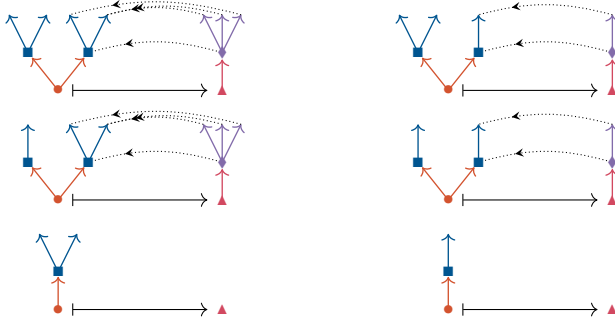
This is just p with every direction extended up one level, so it is still a picture of p . And here is a picture of $y \triangleleft p$, obtained by grafting each p -corolla to the single leaf of y :



This is just p with every position propped up one level, so it is also still a picture of p .

Solution to Exercise 6.31.

We draw $\psi \circ \varphi: q \triangleleft p \rightarrow q' \triangleleft p'$ according to Example 6.30:

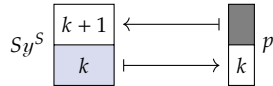


Solution to Exercise 6.32.

It is *not* necessarily the case that $\varphi \circ (p \triangleleft \psi) = \psi \circ (\varphi \triangleleft r)$. We can let $p := y$ and $q := 2$ so that φ is a lens $2 \rightarrow y \triangleleft 2 \cong 2$ (see Exercise 6.29) and ψ is a lens $2 \rightarrow 2 \triangleleft r \cong 2$ (see Exercise 6.27). Then by following the instructions for interpreting a composition product of lenses from either Exercise 6.18 or Example 6.30, we can verify that $p \triangleleft \psi = y \triangleleft \psi$ is a lens $2 \cong y \triangleleft 2 \rightarrow y \triangleleft 2 \triangleleft r \cong 2$ equivalent to the lens ψ , while $\varphi \triangleleft r$ is a lens $2 \cong 2 \triangleleft r \rightarrow y \triangleleft 2 \triangleleft r \cong 2$ equivalent to the lens φ . If, say, we let $\varphi: 2 \rightarrow 2$ be the function sending everything to $1 \in 2$ and $\psi: 2 \rightarrow 2$ be the function sending everything to $2 \in 2$, then in this case $\varphi \circ (p \triangleleft \psi) = \varphi \circ \psi \neq \psi \circ \varphi = \psi \circ (\varphi \triangleleft r)$.

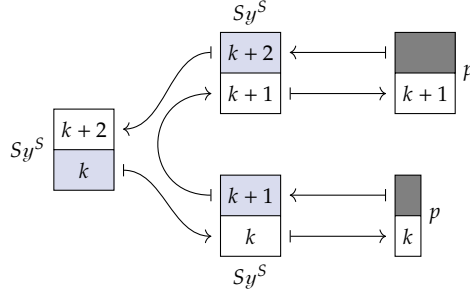
Solution to Exercise 6.44.

- Here is the polybox picture for φ (recall that we shade the upper box of a linear polynomial gray, as there is only one option for what to fill it with):



A single run through the system returns the current state $k \in \mathbb{N}$, then increases that state by 1.

2. Here is the polybox picture for $\delta \circ (\varphi \triangleleft \varphi): Sy^S \rightarrow p \triangleleft p$:



The new system has $p \triangleleft p \cong \mathbb{R}y \triangleleft \mathbb{R}y \cong \mathbb{R}^2y$ as its interface. Indeed, we see that it returns two numbers at once: the current state k (what the first run through φ would return) as well as the increased state $k+1$ (what the second run through φ would return). We update the current state from k to $k+1$ in one run, and from $k+1$ to $k+2$ in the next—thus increasing the current state by 2 overall.

Solution to Exercise 6.45.

We give two reasonable (and equivalent) ways to interpret the transition lens $\delta: Sy^S \rightarrow Sy^S \triangleleft Sy^S$.

One way is to first evaluate its interface as $Sy^S \triangleleft Sy^S \cong S(Sy^S)^S \cong (S \times S^S)y^{S \times S}$. Then we see that if the current state is $s_0 \in S$, the system returns a position consisting of that current state s_0 along with a function $S \rightarrow S$, namely the identity function $s_1 \mapsto s_1$. The system then takes as its direction a pair $(s_1, s_2) \in S \times S$, discarding s_1 and setting its new state to be s_2 .

Alternatively, we can draw from Example 6.40 to interpret δ as a dynamical system that behaves as follows. Each run through the system is a 2-step process: first, the current state $s_0 \in S$ is returned, and a new state $s_1 \in S$ is received. Then this new state s_1 is immediately returned, and an even newer state $s_2 \in S$ is received. Then the current state is updated to the newer state s_2 .

Solution to Exercise 6.51.

To prove Proposition 6.46, it suffices to verify (6.49) and (6.50), as (6.47) and (6.48) follow directly when $A := 2$.

Given polynomials $(q_a)_{a \in A}$, recall that the position-set of the sum $\sum_{a \in A} q_a$ is $\sum_{a \in A} q_a(1)$, while the direction-set at each position (a, j) with $a \in A$ and $j \in q_a(1)$ is $q_a[j]$. So by (6.5), we have that

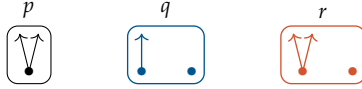
$$\begin{aligned}
 \left(\sum_{a \in A} q_a \right) \triangleleft r &\cong \sum_{\substack{a \in A, \\ j \in q_a(1)}} \prod_{b \in q_a[j]} \sum_{k \in r(1)} \prod_{c \in r[k]} y \\
 &\cong \sum_{a \in A} \sum_{j \in q_a(1)} \prod_{b \in q_a[j]} \sum_{k \in r(1)} \prod_{c \in r[k]} y \\
 &\cong \sum_{a \in A} (q_a \triangleleft r).
 \end{aligned}$$

We can also recall that the position-set of the product $\prod_{a \in A} q_a$ is $\prod_{a \in A} q_a(1)$, while the direction-set at each position $\bar{j}: (a \in A) \rightarrow q_a(1)$ is $\sum_{a \in A} q_a[\bar{j}a]$. So by (6.5), we have that

$$\begin{aligned}
 \left(\prod_{a \in A} q_a \right) \triangleleft r &\cong \sum_{\bar{j} \in \prod_{a \in A} q_a(1)} \prod_{\substack{a \in A, \\ b \in q_a[\bar{j}a]}} \sum_{k \in r(1)} \prod_{c \in r[\bar{j}]} y \\
 &\cong \prod_{a \in A} \sum_{j \in q_a(1)} \prod_{b \in q_a[j]} \sum_{k \in r(1)} \prod_{c \in r[k]} y \\
 &\cong \prod_{a \in A} (q_a \triangleleft r).
 \end{aligned} \tag{1.32}$$

Solution to Exercise 6.53.

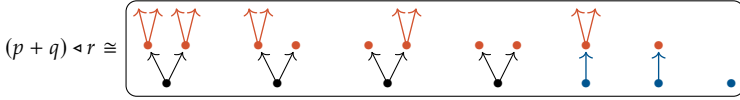
Let $p := y^2$, $q := y + 1$, and $r := y^2 + 1$, as shown here:



Then $p + q \cong y^2 + y + 1$ can be drawn as follows, consisting of every p -corolla and every q -corolla:

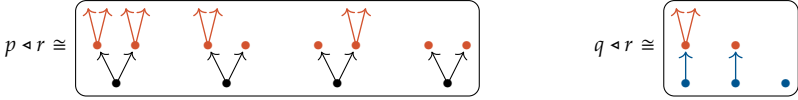


We can therefore draw $(p + q) \triangleleft r$ by grafting r -corollas to leaves of $p + q$ in every way, as follows:

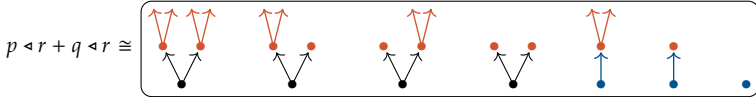


So each tree in $(p + q) \triangleleft r$ is obtained by taking either a p -corolla or a q -corolla, then attaching an r -corolla to each leaf.

Alternatively, we can compute $p \triangleleft r$ and $q \triangleleft r$ separately, grafting r -corollas to leaves of p in every way, then to leaves of q in every way:



Their coproduct then consists of all the trees from $p \triangleleft r$ and all the trees from $q \triangleleft r$:



So each tree in $p \triangleleft r + q \triangleleft r$ is either a p -corolla with an r -corolla attached to each leaf, or a q -corolla with an r -corolla attached to each leaf.

But it does not matter whether we graft r -corollas onto leaves first, or if we pool together corollas from p and q first: the processes are equivalent. Hence the isomorphism $(p + q) \triangleleft r \cong p \triangleleft r + q \triangleleft r$ holds.

Solution to Exercise 6.54.

The left distributivity of $\triangleleft$ over products from (6.50) implies that $(Ap) \triangleleft q \cong (A \triangleleft q)(p \triangleleft q)$, while Exercise 6.27 #3 implies that $A \triangleleft q \cong A$. So $(Ap) \triangleleft q \cong A(p \triangleleft q)$.

Solution to Exercise 6.55.

1. Let $p := y + 1$, $q := 1$, and $r := 0$. Then $p \triangleleft (qr) \cong (y + 1) \triangleleft 0 \cong 1$, while $(p \triangleleft q)(p \triangleleft r) \cong ((y + 1) \triangleleft 1)((y + 1) \triangleleft 0) \cong 2 \times 1 \cong 2$.
2. Again let $p := y + 1$, $q := 1$, and $r := 0$. Then $p \triangleleft (q + r) \cong (y + 1) \triangleleft 1 \cong 2$, while $(p \triangleleft q) + (p \triangleleft r) \cong ((y + 1) \triangleleft 1) + ((y + 1) \triangleleft 0) \cong 2 + 1 \cong 3$.

Solution to Exercise 6.59.

We prove Proposition 6.56 by observing that

$$\mathbf{Poly}(p, r \triangleleft q) \cong \prod_{i \in p(1)} r(q(p[i])) \quad (3.7)$$

$$\cong \mathbf{Poly}\left(\sum_{i \in p(1)} y^{q(p[i])}, r\right) \quad (3.7)$$

$$\cong \mathbf{Poly}\left(\left[\begin{smallmatrix} q \\ p \end{smallmatrix}\right], r\right). \quad (6.58)$$

Solution to Exercise 6.61.

1. Given a polynomial q and a lens $\varphi: p \rightarrow p'$, the functor $\left[\begin{smallmatrix} q \\ \varphi \end{smallmatrix}\right]$ should send φ to a lens $\left[\begin{smallmatrix} q \\ p \end{smallmatrix}\right] \rightarrow \left[\begin{smallmatrix} q \\ p' \end{smallmatrix}\right]$; by (6.58), this is a lens

$$\left[\begin{smallmatrix} q \\ \varphi \end{smallmatrix}\right]: \sum_{i \in p(1)} y^{q(p[i])} \rightarrow \sum_{i' \in p'(1)} y^{q(p'[i'])}.$$

We give $\left[\begin{smallmatrix} q \\ \varphi \end{smallmatrix}\right]$ the same on-positions function $p(1) \rightarrow p'(1)$ that φ has. Then viewing q as a functor, we define the on-directions function $\left[\begin{smallmatrix} q \\ \varphi \end{smallmatrix}\right]_i^\sharp: q(p'[\varphi_1(i)]) \rightarrow q(p[i])$ for each $i \in p(1)$ to be the function obtained by applying q to the corresponding on-directions function $\varphi_i^\sharp: p'[\varphi_1(i)] \rightarrow p[i]$. Functoriality follows trivially on positions and by the functoriality of q itself on directions.

2. Given a polynomial p and a lens $\psi: q' \rightarrow q$, the functor $\left[\begin{smallmatrix} \psi \\ p \end{smallmatrix}\right]$ should send ψ to a lens $\left[\begin{smallmatrix} \psi \\ p \end{smallmatrix}\right] \rightarrow \left[\begin{smallmatrix} \psi \\ p' \end{smallmatrix}\right]$; by (6.58), this is a lens

$$\left[\begin{smallmatrix} \psi \\ p \end{smallmatrix}\right]: \sum_{i \in p(1)} y^{q(p[i])} \rightarrow \sum_{i \in p'(1)} y^{q'(p'[i])}.$$

We let the on-positions function of $\left[\begin{smallmatrix} \psi \\ p \end{smallmatrix}\right]$ be the identity on $p(1)$. Then viewing ψ as a natural transformation, we define the on-directions function $\left[\begin{smallmatrix} \psi \\ p \end{smallmatrix}\right]_i^\sharp: q'(p[i]) \rightarrow$

$q(p[i])$ for each $i \in p(1)$ to be the $p[i]$ -component of ψ . Functoriality follows trivially on positions and because natural transformations compose componentwise on directions.

Solution to Exercise 6.62.

Given $q \in \mathbf{Poly}$, taking the left Kan extension along q gives the left adjoint to composition, which is exactly what we need for the left coclosure. More explicitly, for $p, r \in \mathbf{Poly}$, the left Kan extension $\text{Lan}_q p$ satisfies the natural isomorphism

$$\mathbf{Poly}(\text{Lan}_q p, r) \cong \mathbf{Set}^{\mathbf{Set}}(\text{Lan}_q p, r) \cong \mathbf{Set}^{\mathbf{Set}}(p, r \triangleleft q) \cong \mathbf{Poly}(p, r \triangleleft q),$$

and this same natural isomorphism with $\begin{bmatrix} q \\ p \end{bmatrix}$ in place of $\text{Lan}_q p$ gives the adjunction (6.57).

Solution to Exercise 6.63.

1. By (6.58),

$$\begin{bmatrix} B \\ A \end{bmatrix} = \sum_{i \in A} y^B \cong Ay^B,$$

so by (6.57),

$$\mathbf{Poly}(Ay^B, p) \cong \mathbf{Poly}(A, p \triangleleft B).$$

But A and $p \triangleleft B \cong p(B)$ are both constants, so a lens $A \rightarrow p \triangleleft B$ is just a function $A \rightarrow p(B)$ on positions. Hence (6.64) follows.

2. We prove (6.65) in two parts: that

$$\mathbf{Poly}(Ay \triangleleft p, q) \cong \mathbf{Poly}(p, y^A \triangleleft q) \quad (6.89)$$

and that

$$\mathbf{Poly}(p \triangleleft y^B, q) \cong \mathbf{Poly}(p, q \triangleleft By). \quad (6.90)$$

We have that $Ay \triangleleft p \cong Ap$ and $y^A \triangleleft q \cong q^A$, so (6.89) follows from (5.9). Meanwhile, for (6.90), we have by (6.58) that

$$\begin{bmatrix} By \\ p \end{bmatrix} = \sum_{i \in p(1)} y^{Bp[i]} \cong p \triangleleft y^B,$$

so (6.65) follows from (6.57). Then combining (6.89) and (6.90) yields

$$\mathbf{Poly}(Ay \triangleleft p \triangleleft y^B, q) \cong \mathbf{Poly}(p \triangleleft y^B, y^A \triangleleft q) \cong \mathbf{Poly}(p, y^A \triangleleft q \triangleleft By).$$

Solution to Exercise 6.69.

1. Setting $\mathcal{J}$ in (6.68) to be the empty category, the limit of the functor from $\mathcal{J}$ is just the terminal object. It follows that $1 \triangleleft p \cong 1$. In other words, as $\triangleleft$ preserves limits on the left and terminal objects are limits, $\triangleleft$ preserves terminal objects on the left.

Then a set X can be written as a sum $\sum_{x \in X} 1$, so by (6.49),

$$X \triangleleft p \cong \left(\sum_{x \in X} 1 \right) \triangleleft p \cong \sum_{x \in X} (1 \triangleleft p) \cong \sum_{x \in X} 1 \cong X.$$

2. Setting $\mathcal{G}$ in (6.68) to be the discrete category on the set A , the limit of a functor out of $\mathcal{G}$ is just an A -fold product, so (6.50) follows. In other words, since $\triangleleft$ preserves limits on the left, and since products are limits, $\triangleleft$ preserves products on the left.

Solution to Exercise 6.76.

1. Note that $\mathbf{Poly}(p, q \triangleleft 1) \cong \mathbf{Set}(p(1), q(1))$. A lens $p \rightarrow q \triangleleft r$ is an element of

$$\prod_{i \in p(1)} \sum_{j \in q(1)} \prod_{b \in q[j]} \sum_{k \in r(1)} \prod_{c \in r[k]} p[i] \cong \sum_{f: p(1) \rightarrow q(1)} \prod_{i \in p(1)} \prod_{b \in q[j]} \sum_{k \in r(1)} \prod_{c \in r[k]} p[i].$$

Fixing the function $f: p(1) \rightarrow q(1)$ as implicit in p , we have

$$\begin{aligned} \mathbf{Poly}/(q \triangleleft 1)(p, q \triangleleft r) &\cong \prod_{i \in p(1)} \prod_{b \in q[j]} \sum_{k \in r(1)} \prod_{c \in r[k]} p[i] \\ &\cong \mathbf{Poly}\left(p \overset{f}{\frown} q, r\right). \end{aligned}$$

2. This follows from the first isomorphism above.

Solution to Exercise 6.78.

1. Given lenses $f: p \rightarrow q \triangleleft 1$ and $g: p' \rightarrow p$, the functor $- \overset{f}{\frown} q$ should send g to a lens $p' \overset{g \circ f}{\frown} q \rightarrow p \overset{f}{\frown} q$; by (6.74), this is a lens

$$g \overset{f}{\frown} q: \sum_{i' \in p'(1)} q[f_1 g_1 i'] y^{p'[i']} \rightarrow \sum_{i \in p(1)} q[f_1 i] y^{p[i]}.$$

We give $g \overset{f}{\frown} q$ an on-positions function that is the on-positions function $p'(1) \rightarrow p(1)$ of g on the first coordinate $i' \in p'(1)$ and the identity on $q[f_1 g_1 i']$ on the second. Then we let the on-directions function at every position with first coordinate $i' \in p'(1)$ be the on-directions function $g_{i'}^\#: p[g_1 i'] \rightarrow p'[i']$. Functoriality follows trivially on both positions and directions.

2. Given a polynomial p and lenses $f: p \rightarrow q \triangleleft 1$ and $h: q \rightarrow q'$, the functor $p \overset{f}{\frown} -$ should send h to a lens $p \overset{f \circ (h \triangleleft 1)}{\frown} q' \rightarrow p \overset{f}{\frown} q$; by (6.74), this is a lens

$$p \overset{f}{\frown} h: \sum_{i \in p(1)} q'[h_1 f_1 i] y^{p[i]} \rightarrow \sum_{i \in p(1)} q[f_1 i] y^{p[i]}.$$

We let the on-positions function of $p \overset{f}{\frown} h$ be the identity on the first coordinate $i \in p(1)$ and the on-directions function $h_{f_1(i)}^\#: q'[h_1 f_1 i] \rightarrow q[f_1 i]$ on the second. Then we let the on-directions function at every position with first coordinate $i \in p(1)$ be the identity on $p[i]$. Functoriality follows trivially on both positions and directions.

Solution to Exercise 6.80.

1. It suffices to show for a connected category $\mathcal{G}$ and a functor $X: \mathcal{G} \rightarrow \mathbf{Set}$, we have

$$p \left(\lim_{j \in \mathcal{G}} X_j \right) \cong \lim_{j \in \mathcal{G}} p(X_j).$$

But we can identify **Set** with the full subcategory of constant functors in **Poly** and instead view X as a functor into **Poly**. Then by Exercise 6.27 #3, the left-hand side of the isomorphism we seek is isomorphic to $p \triangleleft (\lim_{j \in G} X_j)$, while the right-hand side is isomorphic to $\lim_{j \in G} (p \triangleleft X_j)$. These are isomorphic by Theorem 6.79.

2. As 1 is terminal in **Poly**, the product qr can also be written as the pullback $q \times_1 r$. While products are not connected limits, pullbacks are, so by Theorem 6.79, they are preserved by precomposition with p . Hence the desired isomorphism follows.
3. We will show that (6.81) holds for $p := y + 1, q := 1$, and $r := 0$. We have $p \triangleleft q = p \triangleleft 1 \cong (y + 1) \triangleleft 1 \cong 2$ and $p \triangleleft r \cong (y + 1) \triangleleft 0 \cong 1$, so $(p \triangleleft q) \times_{(p \triangleleft 1)} (p \triangleleft r) \cong 2 \times_2 1$. We saw in Example 5.39 that the position-set of a pullback in **Poly** is just the pullback of the position-sets in **Set**, while the direction-sets are given by a pushout of direction-sets in **Set**. As our polynomials have empty direction-sets, their pullback must have an empty direction-set as well, so this pullback is just a pullback of sets: $(p \triangleleft q) \times_{(p \triangleleft 1)} (p \triangleleft r) \cong 2 \times_2 1 \cong 1$. And indeed we have $p \triangleleft (qr) \cong (y + 1) \triangleleft 0 \cong 1$ as well.

Solution to Exercise 6.83.

1. The isomorphism always holds: we have that $(Ay) \otimes (By) \cong AB_y \cong (Ay) \triangleleft (By)$.
 2. The isomorphism always holds: we have that $y^A \otimes y^B \cong y^{AB} \cong y^A \triangleleft y^B$.
 3. The isomorphism does not always hold: while $A \otimes B \cong AB$, we have that $A \triangleleft B \cong A$. There is, however, always a canonical projection $AB \rightarrow B$; but there is not always a canonical lens $B \rightarrow AB$ (for example, take $A = 0 \neq B$).
 4. The isomorphism always holds: we have that $By \otimes p \cong \sum_{i \in p(1)} By \otimes y^{p[i]} \cong \sum_{i \in p(1)} By^{p[i]} \cong Bp \cong By \triangleleft p$.
 5. The isomorphism does not always hold: if, say, $p = B$, then $y^A \otimes B \cong B$, while $y^A \triangleleft B \cong B^A$. If $A = B = 0$, then $B^A \cong 1$, so there is not always a canonical lens from right to left, either. There is, however, always a canonical lens from left to right: $y^A \otimes p \cong \sum_{i \in p(1)} y^A p[i]$ while $y^A \triangleleft p \cong \sum_{\tilde{i}: A \rightarrow p(1)} y^{\sum_{a \in A} p[\tilde{i}(a)]}$. So there is a lens from left to right whose on-positions function sends $i \in p(1)$ to the constant function $A \rightarrow p(1)$ that always evaluates to i ; and whose on-directions function at i is the identity on $Ap[i]$.
 6. The isomorphism does not always hold: if, say, $p = y^A$, then $y^A \otimes By \cong By^A$, while $y^A \triangleleft By \cong (By)^A \cong B^A y^A$. If $A = B = 0$, then $By^A \cong 0$ while $B^A y^A \cong 0^0 y^0 \cong 1$, so there is not always a canonical lens from right to left, either. There is, however, always a canonical lens from left to right: $p \otimes By \cong Bp$ while $p \triangleleft By \cong \sum_{i \in p(1)} \sum_{\tilde{b}: p[i] \rightarrow B} y^{\sum_{a \in p[i]} 1} \cong \sum_{i \in p(1)} B^{p[i]} y^{p[i]}$. So there is a lens from left to right whose on-positions function sends $(b, i) \in Bp(1)$ to $(i, c_b) \in \sum_{i \in p(1)} B^{p[i]}$, where $c_b: p[i] \rightarrow B$ is the constant function that always evaluates to b ; and whose on-directions function at (b, i) is the identity on $p[i]$.
 7. The isomorphism always holds: we have that $p \otimes y^A \cong \sum_{i \in p(1)} y^A p[i] \cong p \triangleleft y^A$.
- Every on-directions function of every lens we found in this exercise is an isomorphism, so every lens we found in this exercise is cartesian.

Solution to Exercise 6.88.

Here $\varphi: p \rightarrow p'$ and $\psi: q \rightarrow q'$ are lenses.

1. If φ is an isomorphism and ψ is vertical, then $\psi \triangleleft 1: q \triangleleft 1 \rightarrow q' \triangleleft 1$ is an isomorphism, so $\varphi \triangleleft \psi \triangleleft 1: p \triangleleft q \triangleleft 1 \rightarrow p' \triangleleft q' \triangleleft 1$ is an isomorphism as well. Thus $\varphi \triangleleft \psi$ is vertical.
2. If φ is the unique lens $y \rightarrow 1$ and $q = 0$, then φ is vertical, but since $y \triangleleft 0 \cong 0$ and $1 \triangleleft 0 \cong 1$, the lens $\varphi \triangleleft 0: 0 \rightarrow 1$ is not.

7. Polynomial comonoids and retrofunctors

One of the most surprising aspects of **Poly** is its comonoid objects. In 2018, Daniel Ahman and Tarmo Uustalu characterized comonoids in $(\mathbf{Poly}, y, \triangleleft)$ as something surprisingly familiar: they are in fact small categories. For us, this story will emerge naturally as we expand our understanding of the state system of a dependent dynamical system.

7.1 State systems, categorically

Since defining dependent dynamical systems in Definition 4.18, we have evolved our understanding of their state systems over the course of the last few chapters. Let us take this moment to review what we know about these state systems so far.

Our original definition of a state system was as a monomial Sy^S for some set S . But in Example 4.43, we noted that this formulation requires us to discuss the positions and directions of a state system at the level of sets rather than in the language of **Poly**. Instead, let's take an arbitrary polynomial $s \in \mathbf{Poly}$ and attempt to characterize what it means for s to be a state system using only the categorical machinery of **Poly**. We will continue to refer to the positions of s as *states*, but we will shift from thinking of the directions of s as states to thinking of them as transitions from one state to another.

7.1.1 The do-nothing section

In Example 4.43, we saw that every state system s is equipped with a *do-nothing section*: a lens $\epsilon: s \rightarrow y$ that picks out a direction at each state that we would like to interpret as “doing nothing” and remaining at that state.

We drew ϵ in polyboxes in Example 4.45, but that was when we let ourselves assume that the position-set of a state system was equal to each of its direction-sets. Now all we know is that for each state $s \in \mathfrak{s}(1)$, the do-nothing section chooses an $\mathfrak{s}[s]$ -direction to signify staying at the same state; it does not make sense to say that this direction is literally equal to s . So we need a different name for the $\mathfrak{s}[s]$ -direction that ϵ identifies: call it id_s , because it behaves like a sort of identity operation on the state s .

So the revised polyboxes for the do-nothing section $\epsilon: \mathfrak{s} \rightarrow y$ are as follows:

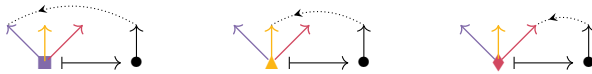
$$\begin{array}{|c|} \hline \text{id}_s \\ \hline s \\ \hline \end{array} \quad \epsilon \quad (7.1)$$

Exercise 7.2. Say I have a polynomial $\mathfrak{s} \in \mathbf{Poly}$, and I tell you that there is a lens $\epsilon: \mathfrak{s} \rightarrow y$. What can you say about the polynomial $\mathfrak{s}$?

Example 7.3 (The do-nothing section in tree pictures). We have seen the do-nothing section drawn in polyboxes, but let us see what it looks like in our tree pictures. We take $\mathfrak{s} := \{\blacksquare, \blacktriangle, \blacklozenge\}y^{\{\blacksquare, \blacktriangle, \blacklozenge\}}$, drawn as follows, with the directions associated with $\blacksquare$, $\blacktriangle$, and $\blacklozenge$ from left to right:



Then the do-nothing section $\epsilon: \mathfrak{s} \rightarrow y$ can be drawn as follows:

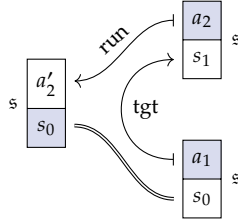


It picks out one direction at each position, namely the one with the same color.

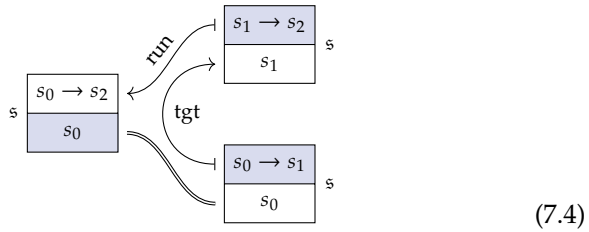
There is not much else we can say about the do-nothing section on its own, so let us revisit the other lens that every state system is equipped with before considering the relationship between the two.

7.1.2 The transition lens

We saw in Example 6.43 that $\mathfrak{s}$ also comes equipped with a *transition lens*: a lens $\delta: \mathfrak{s} \rightarrow \mathfrak{s} \triangleleft \mathfrak{s}$, which we can draw as



The arrow labeled *tgt* is the *target function*: given a state s_0 and a direction a_1 at that state, $\text{tgt}(s_0, a_1)$ tells us the new state s_1 that following a_1 from s_0 will lead to. We know that when s_0 is fixed, the target function on the second component a_1 should be an isomorphism $\mathfrak{s}[s_0] \rightarrow \mathfrak{s}(1)$; that is, there is exactly one direction at s_0 that leads to each state of $\mathfrak{s}$. But this property is a little tricky to state in the language of **Poly**; in fact, we will not attempt to do so just yet. Instead, we will use it to make a notational choice: given $s, t \in \mathfrak{s}$, we will let $s \rightarrow t$ denote the unique direction at s that leads to t , so that $\text{tgt}(s, s \rightarrow t) = t$. So we can redraw our transition lens as



In addition to the fact that $\text{tgt}(s_0, s_0 \rightarrow s_1) = s_1$ as intended, this picture tells us two more properties of δ .

The first is that the bottom arrow is the identity on $\mathfrak{s}(1)$. This is something we would like to be able to express categorically in the language of **Poly**. We will see that this property falls out naturally when we express how the transition lens plays nicely with the do-nothing section in Section 7.1.3.

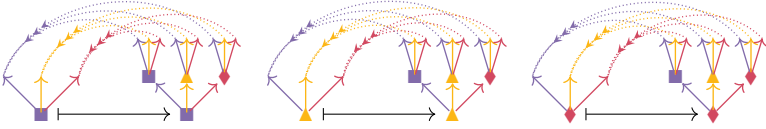
The second is that the *run* arrow, which runs the transition $s_0 \rightarrow s_1$ and the transition $s_1 \rightarrow s_2$ together into a transition starting at s_0 , should

have the same target as the second transition it follows: in this case, s_2 . Equationally, writing the left and right-hand sides only in terms of the contents of the blue boxes, we have that

$$\text{tgt}(s_0, \text{run}(s_0, s_0 \rightarrow s_1, s_1 \rightarrow s_2)) = s_2 = \text{tgt}(\text{tgt}(s_0, s_0 \rightarrow s_1), s_1 \rightarrow s_2). \quad (7.5)$$

We will see that this property arises naturally when we generalize the transition lens to more than two steps in Section 7.1.4.

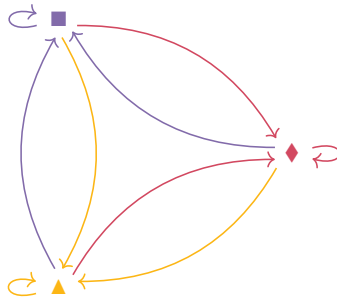
Example 7.6 (The transition lens in tree pictures). Continuing from Example 7.3, we draw the transition lens $\delta: \mathfrak{s} \rightarrow \mathfrak{s} \triangleleft \mathfrak{s}$ of $\mathfrak{s} := 3y^3 \cong \{\blacksquare, \blacktriangle, \blacklozenge\}y^{\{\blacksquare, \blacktriangle, \blacklozenge\}}$ (where directions are labeled with their targets) in tree pictures as well, recalling that the trees of $\mathfrak{s} \triangleleft \mathfrak{s}$ are obtained by taking an $\mathfrak{s}$ -corolla and grafting more $\mathfrak{s}$ -corollas to each of its leaves:



On positions, the target function of δ tells us which root of $\mathfrak{s}$ to graft onto each leaf of $\mathfrak{s}$. Then on directions, the run function of δ tells us how to collapse the height-2 leaves of the trees we obtain in $\mathfrak{s} \triangleleft \mathfrak{s}$ down to the original height-1 leaves of the corollas of $\mathfrak{s}$.

We can draw what the target function is doing more compactly by taking the corollas of $\mathfrak{s}$ and “bending the arrows” so that they

point to their targets, like so:

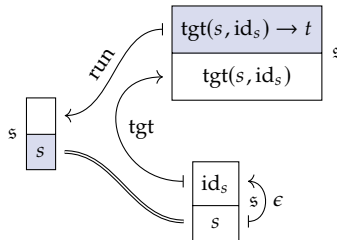


So the target function of δ turns our corolla picture of s into a complete graph on its roots. Then the run function takes any two arrows that form a path in the graph and collapses them down to a single arrow that starts (and, according to (7.5), ends) at the same vertex as the two-arrow path.

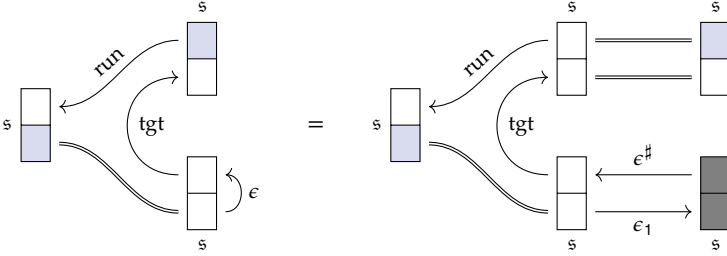
7.1.3 The do-nothing section coheres with the transition lens

For each state $s \in s(1)$, the do-nothing section $\epsilon: s \rightarrow y$ picks out the $s[s]$ -direction id_s that “does nothing” and keeps the system in the same state s . But it is the transition lens $\delta: s \rightarrow s \triangleleft s$ that actually sets our state system in motion, specifying the target of each direction and how two directions run together. Either of these directions could be our do-nothing direction id_s , so let us determine what should happen when we set each one in turn to id_s .

We can draw in polyboxes what happens when we set id_s , as specified by ϵ , to be the first direction that δ runs together like this:



Reading this picture from left to right, we see that it depicts the polyboxes of the composite lens $\delta \circ (\epsilon \triangleleft s) : s \rightarrow y \triangleleft s \cong s$ (recall that we sometimes denote the identity lens on s also by s). To make this interpretation more transparent, we could be a little more verbose with our polybox picture if we wanted to (omitting the contents of the boxes for clarity):

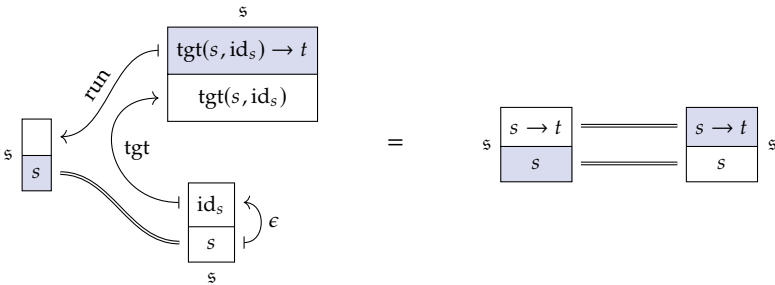


Now what should $\text{tgt}(\text{id}_s)$ be, and what should go in the direction box on the left?

If following the direction id_s from the state s is the same as doing nothing, then its target state should be the same state s that it emerged from. Moreover, running together id_s with any other direction $s \rightarrow t$ from s should be no different from the direction $s \rightarrow t$ on its own. So

$$\text{tgt}(s, \text{id}_s) = s \quad \text{and} \quad \text{run}(s, \text{id}_s, s \rightarrow t) = s \rightarrow t.$$

In fact, id_s should just be the direction $s \rightarrow s$. Pictorially, we have the equation



Or, if you prefer, we might say that $\delta \circ (\epsilon \triangleleft s) = \text{id}_s$, or that the following diagram commutes:

$$\begin{array}{ccc}
 y \triangleleft \mathfrak{s} & \xlongequal{\quad} & \mathfrak{s} \\
 \nwarrow \epsilon \triangleleft \mathfrak{s} & & \downarrow \delta \\
 & & \mathfrak{s} \triangleleft \mathfrak{s}
 \end{array}$$

This commutative diagram captures one way in which ϵ and δ always relate—and it is written entirely in the language of **Poly**, without having to talk about individual sets!

What about setting the second direction that δ runs together to what is specified by ϵ , rather than the first? To answer this, we should look at the composite lens $\delta \circ (\delta \triangleleft \epsilon)$: $s \mapsto s \triangleleft y \cong s$ instead. But the do-nothing direction should still do nothing, so here is what the polybox picture should look like:

The diagram illustrates the reduction of a program with a loop to a program without a loop. On the left, a state s transitions to a state containing $s \rightarrow t$ and s . This state then runs a loop id_t (labeled run) and targets a state $s \rightarrow t$ and s (labeled tgt). The right side shows the reduced program where the loop is unrolled, resulting in a direct transition from s to a state containing $s \rightarrow t$ and s .

The lens depicted on the right-hand side of the equation is again the identity lens on $\mathfrak{s}$.

If we match up the two white boxes on the right-hand side of the equation with the corresponding white boxes on the left, we can actually read two equations from this polybox picture. Matching up positions in the codomain tells us that the bottom arrow of δ on the left must send s to itself: it is the identity function on $s(1)$. Indeed, this is exactly what we wanted to say about that arrow in Section 7.1.2.

Meanwhile, matching up directions in the domain tells us that

$$\text{run}(s, s \rightarrow t, \text{id}_t) = s \rightarrow t,$$

as we would expect: id_t is just be the direction $t \rightarrow t$.

More concisely, we can express both these facts in **Poly** via the equation $\delta \circ (\mathfrak{s} \triangleleft \epsilon) = \text{id}_{\mathfrak{s}}$. The corresponding commutative diagram is as follows:

$$\begin{array}{ccc}
 \mathfrak{s} & \xlongequal{\quad} & \mathfrak{s} \triangleleft y \\
 \delta \downarrow & \nearrow \mathfrak{s} \triangleleft \epsilon & \\
 \mathfrak{s} \triangleleft \mathfrak{s} & &
 \end{array}$$

We can combine this with our previous commutative diagram to say that the relationship between the do-nothing section $\epsilon: \mathfrak{s} \rightarrow y$ and the transition lens $\delta: \mathfrak{s} \rightarrow \mathfrak{s} \triangleleft \mathfrak{s}$ of a state system $\mathfrak{s}$ is captured in **Poly** by the following commutative diagram:

$$\begin{array}{ccccc}
 y \triangleleft \mathfrak{s} & \xlongequal{\quad} & \mathfrak{s} & \xlongequal{\quad} & \mathfrak{s} \triangleleft y \\
 \epsilon \triangleleft \mathfrak{s} \swarrow & & \delta \downarrow & & \mathfrak{s} \triangleleft \epsilon \searrow \\
 & & \mathfrak{s} \triangleleft \mathfrak{s} & &
 \end{array} \tag{7.7}$$

7.1.4 The transition lens is coassociative

Shortly after Example 6.43, we noted that while the transition lens $\delta: \mathfrak{s} \rightarrow \mathfrak{s} \triangleleft \mathfrak{s}$ gives us a canonical way to model two steps of a dynamical system with state system $\mathfrak{s}$, we have a choice of how to model three steps through the same system: we could obtain a lens $\mathfrak{s} \rightarrow \mathfrak{s}^{\triangleleft 3}$ that runs three directions together by taking either one of the composite lenses $\delta \circ (\delta \triangleleft \mathfrak{s})$ or $\delta \circ (\mathfrak{s} \triangleleft \delta)$. That presents a problem for us: which one should we choose?

Happily, it turns out this choice is a false one. If we write out the two composite lenses in polyboxes, with $\delta \circ (\delta \triangleleft \mathfrak{s})$ on the left and $\delta \circ (\mathfrak{s} \triangleleft \delta)$ on the right, we find that they are equal:

$$\begin{array}{c}
 \text{Left side (} \delta \circ (\delta \triangleleft \mathfrak{s}) \text{):} \\
 \begin{array}{c}
 \text{Polybox 1: } s_0 \rightarrow s_3, s_0 \\
 \text{Polybox 2: } s_2 \rightarrow s_3, s_2 \\
 \text{Polybox 3: } s_0 \rightarrow s_2, s_0 \\
 \text{Polybox 4: } s_1 \rightarrow s_2, s_1 \\
 \text{Polybox 5: } s_0 \rightarrow s_1, s_0
 \end{array}
 \end{array}
 =
 \begin{array}{c}
 \text{Right side (} \delta \circ (\mathfrak{s} \triangleleft \delta) \text{):} \\
 \begin{array}{c}
 \text{Polybox 1: } s_0 \rightarrow s_3, s_0 \\
 \text{Polybox 2: } s_1 \rightarrow s_3, s_1 \\
 \text{Polybox 3: } s_0 \rightarrow s_1, s_0 \\
 \text{Polybox 4: } s_2 \rightarrow s_3, s_2 \\
 \text{Polybox 5: } s_1 \rightarrow s_2, s_1 \\
 \text{Polybox 6: } s_0 \rightarrow s_1, s_0
 \end{array}
 \end{array}
 \tag{7.8}$$

Remember: the way to read these polyboxes is to start at the lower blue square on the left and follow the path counter clockwise around the diagram; and if you reach a box with no arrows leading out of it, go up to the blue box above it and continue to follow the arrows from there.

There is a lot going on here, so let us break it down—we will focus on the run functions first. On the left-hand side, we run together $s_0 \rightarrow s_1$ and $s_1 \rightarrow s_2$ to obtain $s_0 \rightarrow s_2$, before running that together with $s_2 \rightarrow s_3$ to obtain $s_0 \rightarrow s_3$, as we see in the upper left box. Meanwhile, on the right, we run together $s_1 \rightarrow s_2$ and $s_2 \rightarrow s_3$ to obtain $s_1 \rightarrow s_3$, before running $s_0 \rightarrow s_1$ together with our newly obtained $s_1 \rightarrow s_3$ to again obtain $s_0 \rightarrow s_3$ in the upper left box. We could write this all out equationally, but all this is saying is that “running together” the directions of a state system is an associative operation. When running together three directions, it does not matter whether we run the first two together or the last two together to start. Not only is this guaranteed by the way in which we constructed δ , it also makes intuitive sense.

Exercise 7.9. Using only the contents of the blue boxes and the target and run functions, write down the equation that we can read from (7.8) expressing the associativity of the “running together” operation.

This associative property is what we get by matching up the white direction boxes on each domain side, but there are three more white position boxes on each codomain side that we can match up as well. The fact that the lower two of these pairs coincide is a consequence of the fact that the bottom arrow of δ is the identity, which we already knew from Section 7.1.3; so we do not learn anything new there. On the other hand, the fact that both the upper position boxes in the codomain contain s_2 implies that

$$\begin{aligned} \text{tgt}(s_0, \text{run}(s_0, s_0 \rightarrow s_1, s_1 \rightarrow s_2)) &= \text{tgt}(s_0, s_0 \rightarrow s_2) \\ &= s_2 \\ &= \text{tgt}(s_1, s_1 \rightarrow s_2) \end{aligned}$$

$$= \text{tgt}(\text{tgt}(s_0, s_0 \rightarrow s_1), s_1 \rightarrow s_2),$$

which is exactly what we wanted in (7.5). In words, this says that when we run together $s_0 \rightarrow s_1$ and $s_1 \rightarrow s_2$, the new direction's target is the same as the direction of $s_1 \rightarrow s_2$, the latter of the two directions that we ran together. Again, this coincides with our intuition: if we follow two directions in order, we should end up at wherever the latter direction leads us.

Hence both the associativity of running directions together and the relationship between the target and run functions from (7.5) are captured by the equality of lenses $\delta \circ (\delta \triangleleft s) = \delta \circ (s \triangleleft \delta)$. Equivalently, the following diagram in **Poly** commutes:

$$\begin{array}{ccc}
 s & \xrightarrow{\delta} & s \triangleleft s \\
 \delta \downarrow & & \downarrow s \triangleleft \delta \\
 s \triangleleft s & \xrightarrow{\delta \triangleleft s} & s \triangleleft s \triangleleft s.
 \end{array} \tag{7.10}$$

Another way to say this is that δ is *coassociative*: while δ is only a lens $s \rightarrow s^{\triangleleft 2}$ as defined, the commutativity of (7.10) tells us that the two ways of getting a lens $s \rightarrow s^{\triangleleft 3}$ out of δ are actually the same. (This is dual to an *associative* operation, which is a binary operation that gives rise to two identical ternary operations.)

So δ induces a canonical lens $s \rightarrow s^{\triangleleft 3}$, which we will call $\delta^{(3)}$, as it has 3 copies of s in its codomain. Armed with this new lens, we can model three steps through a system $\varphi: s \rightarrow p$ with interface $p \in \mathbf{Poly}$ as the composite lens

$$s \xrightarrow{\delta^{(3)}} s^{\triangleleft 3} \xrightarrow{\varphi^{\triangleleft 3}} p^{\triangleleft 3}.$$

In fact, coassociativity guarantees that δ induces a canonical lens $\delta^{(n)}: s \rightarrow s^{\triangleleft n}$ for every integer $n \geq 2$, starting with $\delta^{(2)} := \delta$.¹ For concreteness, we could then define $\delta^{(n)}$ for $n > 2$ inductively by $\delta^{(n)} := \delta \circ (\delta^{(n-1)} \triangleleft s)$, or just as well by $\delta^{(n)} := \delta \circ (s \triangleleft \delta^{(n-1)})$ or even $\delta^{(n)} := \delta \circ (\delta^{(\ell)} \triangleleft \delta^{(m)})$ for some pair of integers $\ell, m > 1$ satisfying $\ell + m = n$.

¹Perhaps this notation seems a little unnatural, but it helps to think of the original $\delta: s \rightarrow s \triangleleft s$ as the $n = 2$ case of a generalized transition lens modeling n steps through the state system.

Regardless, the coassociativity of δ means that it doesn't matter how we build a lens $\mathfrak{s} \rightarrow \mathfrak{s}^{\triangleleft(n+1)}$ out of $\delta, \circ, \triangleleft$, and identity lenses: we will always end up with the same lens. We will state this in more generality in Proposition 7.20, but here is some practice with the $n = 4$ case for a taste of what's to come.

Exercise 7.11.

1. Say we know nothing about $\mathfrak{s}$ or δ apart from the fact that $\mathfrak{s} \in \mathbf{Poly}$ and that δ is a lens $\mathfrak{s} \rightarrow \mathfrak{s} \triangleleft \mathfrak{s}$. List all the ways to obtain a lens $\mathfrak{s} \rightarrow \mathfrak{s}^{\triangleleft 4}$ using only copies of $\delta, \text{id}_{\mathfrak{s}}, \triangleleft$, and $\circ$. (You may write $\mathfrak{s}$ for $\text{id}_{\mathfrak{s}}$.)
2. Now assume that (7.10) commutes. Show that all the lenses on your list are equal. (Hint: use the fact that $(f \circ g) \triangleleft (h \circ k) = (f \triangleleft h) \circ (g \triangleleft k)$ for lenses f, g, h, k .)

7.1.5 Running dynamical systems

Finally, we are ready to fulfill our promise from way back in Example 4.43 by using the language of **Poly** to describe stepping through a dynamical system n times for arbitrary $n \in \mathbb{N}$. Given a dynamical system $\varphi: \mathfrak{s} \rightarrow p$ with interface $p \in \mathbf{Poly}$, we can construct a new dynamical system that we call $\text{Run}_n(\varphi)$, with the same state system $\mathfrak{s}$ but a new interface $p^{\triangleleft n}$, by defining $\text{Run}_n(\varphi) := \delta^{(n)} \circ \varphi^{\triangleleft n}$. Visually, we define $\text{Run}_n(\varphi)$ so that the following diagram commutes:

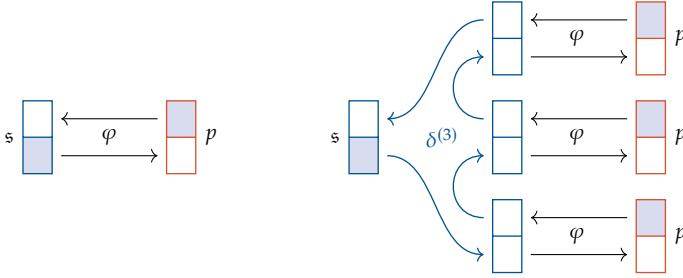
$$\begin{array}{ccccc} \mathfrak{s} & \xrightarrow{\delta^{(n)}} & \mathfrak{s}^{\triangleleft n} & \xrightarrow{\varphi^{\triangleleft n}} & p^{\triangleleft n} \\ & \searrow & & \nearrow & \\ & & \text{Run}_n(\varphi) & & \end{array}$$

One way to think of this is that $\text{Run}_n(\varphi)$ is a sped-up version of φ : one step through $\text{Run}_n(\varphi)$ is equivalent to n steps through φ . But this is just because a single interaction with the interface $p^{\triangleleft n}$ models a sequence of n interactions with the interface p , as detailed in Section 6.1.4 and Example 6.42. So $\text{Run}_n(\varphi)$ repackages n cycles through φ into a single step. Crucially, $\delta^{(n)}$ is what tells us how to sequence all n of these steps together on the state system side. We illustrated how δ does this for the $n = 2$ case in Example 6.43, and here is a polybox picture for the $n = 3$

case:

Given $\varphi: s \rightarrow p$,

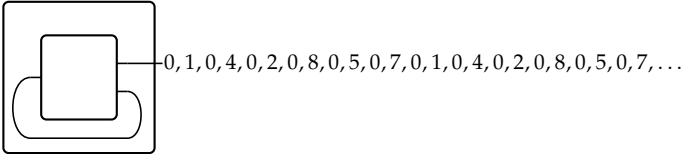
obtain $\text{Run}_3(\varphi)$:



Notice that we have only defined $\delta^{(n)}$, and thus $\text{Run}_n(\varphi)$, for integers $n \geq 2$. But $n = 0$ runs through n is doing nothing, modeled by the do-nothing section $\epsilon: s \rightarrow y$, while $n = 1$ run through φ is modeled by $\varphi: s \rightarrow s$ itself. So we want $\text{Run}_0(\varphi) = \epsilon$ and $\text{Run}_1(\varphi) = \varphi$; we can achieve this by setting $\delta^{(0)} := \epsilon$ and $\delta^{(1)} := \text{id}_s$. Here we should think of the do-nothing section $\delta^{(0)}$ as the transition lens modeling 0 steps through our state system, and the identity $\delta^{(1)}$ as the transition lens modeling a single step.

Exercise 7.12. Verify that when $\delta^{(0)} = \epsilon$ and $\delta^{(1)} = \text{id}_s$, if $\text{Run}_n(\varphi)$ is defined as $\delta^{(n)} \circ \varphi^{*n}$ for all $n \in \mathbb{N}$, then $\text{Run}_0(\varphi) = \epsilon$ and $\text{Run}_1(\varphi) = \text{id}_s$.

Example 7.13 (Returning every other position). In Exercise 4.65, we built a dynamical system $\varphi: Sy^S \rightarrow \mathbb{N}y$ that returns natural numbers—specifically digits, alternating between 0's and the base-10 digits of $1/7$ after the decimal point like so:



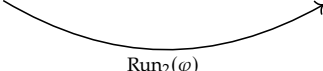
Say we only wanted the system to return the digits of $1/7$ after the

decimal point; we'd like to do away with all these 0's. In other words, we want a new system $Sy^S \rightarrow \mathbb{N}y$ that acts like φ , except that it only returns every other position that φ returns.

We could build such a system from scratch—or we can simply start from φ and apply Run_2 , yielding a system $\text{Run}_2(\varphi): Sy^S \rightarrow \mathbb{N}y \triangleleft \mathbb{N}y \cong \mathbb{N}^2y$ that returns the positions of φ two at a time:

$$(0, 1), (0, 4), (0, 2), (0, 8), (0, 5), (0, 7), \\ (0, 1), (0, 4), (0, 2), (0, 8), (0, 5), (0, 7), \dots$$

Then we just need to compose $\text{Run}_2(\varphi)$ with a lens $\pi_2: \mathbb{N}^2y \rightarrow \mathbb{N}y$ equal to the second coordinate projection on positions (and the identity on directions) to extract the positions we want. The new system $Sy^S \rightarrow \mathbb{N}y$ that skips over every position of φ is therefore the following composite:

$$Sy^S \xrightarrow{\delta^{(2)}} Sy^S \triangleleft Sy^S \xrightarrow{\varphi^{*2}} \mathbb{N}y \triangleleft \mathbb{N}y \cong \mathbb{N}^2y \xrightarrow{\pi_2} \mathbb{N}y.$$


We can apply this technique in general to skip (or otherwise act on) the positions of a dynamical system at regular intervals.

One drawback of the $\text{Run}_n(-)$ operation is that we need to keep track of a separate morphism $Sy^S \rightarrow p^{*n}$ for every $n \in \mathbb{N}$, as well as various ways to relate these morphisms for different values of n . Is there a way to package all this information into a single morphism that can model arbitrarily long runs through the system? We will answer this question in Chapter 8; but for now, let us investigate what is really going on with our state systems algebraically.

7.1.6 State systems as comonoids

It turns out that objects equipped with morphisms like those in Sections 7.1.1 and 7.1.2 that satisfy the commutative diagrams from Sections 7.1.3 and 7.1.4 are well-known to category theorists.

Definition 7.14 (Comonoid). In a monoidal category $(\mathbf{C}, y, \triangleleft)$, a *comonoid* $\mathcal{C} := (c, \epsilon, \delta)$ consists of

- an object $c \in \mathbf{C}$, called the *carrier*;
- a morphism $\epsilon: c \rightarrow y$ in $\mathbf{C}$, called the *eraser* (or the *counit*); and
- a morphism $\delta: c \rightarrow c \triangleleft c$ in $\mathbf{C}$, called the *duplicator* (or the *comultiplication*);

such that the following diagrams, collectively known as the *comonoid laws*, commute:

$$\begin{array}{ccccc}
 y \triangleleft c & \xlongequal{\quad} & c & \xlongequal{\quad} & c \triangleleft y \\
 \swarrow \epsilon \triangleleft c & & \downarrow \delta & & \searrow c \triangleleft \epsilon \\
 & & c \triangleleft c & &
 \end{array} \quad (7.15)$$

where the left triangle is known as the *left erasure* (or *counit*) *law* and the right triangle is known as the *right erasure* (or *counit*) *law*; and

$$\begin{array}{ccc}
 c & \xrightarrow{\delta} & c \triangleleft c \\
 \downarrow \delta & & \downarrow c \triangleleft \delta \\
 c \triangleleft c & \xrightarrow{\delta \triangleleft c} & c \triangleleft c \triangleleft c
 \end{array} \quad (7.16)$$

known as the *coassociative law*.

We may also say that the eraser and duplicator morphisms comprise a *comonoid structure* on the carrier, or we may identify a comonoid with its carrier if the eraser and duplicator can be inferred from context.

We refer to a comonoid $\mathcal{C}$ in $(\mathbf{Poly}, y, \triangleleft)$ as a *polynomial comonoid*.

Remark 7.17. The concept of a *comonoid* in a monoidal category is dual to that of a *monoid*, which may be more familiar. Monoids come with *unit* and *multiplication* morphisms that point the other way, so named because they generalize the unit and multiplication operations of a monoid in **Set**. (We will discuss monoids in **Set** further in Example 7.40.) Prepending ‘co-’ to each term yields the corresponding terms for comonoids.

The alternative names *eraser* for the counit and *duplicator* for the comultiplication are less standard, but we will favor them to avoid confusion between the counit of a *comonoid* and the counit of an *adjunction*—and so that their names match up with the Greek letters ϵ and δ that we will so often use to label them. The word “duplicator” comes from the fact that $\delta: c \rightarrow c \triangleleft c$ effectively turns one c into two, while the “eraser” $\epsilon: c \rightarrow y$ erases the c altogether, leaving only the monoidal unit y . Still, it can be helpful to think of comonoids as having a *coassociative* comultiplication along with a counit satisfying *left and right counit laws*.

Remark 7.18. Comonoids in a functor category with respect to the composition product are generally known as *comonads*. So it would be a little more precise and familiar to refer to our polynomial comonoids as *polynomial comonads*. But since we think of our polynomials more often in terms of positions and directions than as functors, we will favor the term *comonoid* over *comonad*.

Example 7.19 (State systems are polynomial comonoids). Nearly all our work on state systems up until now can be summarized thusly:

*every state system is a polynomial comonoid,
whose eraser is the do-nothing section
and whose duplicator is the transition lens.*

The comonoid structure on a state system s is what allows us to write canonical lenses $s \rightarrow s^{<n}$ for any $n \in \mathbb{N}$. We can then model n steps through a dynamical system $\varphi: s \rightarrow p$ with interface $p \in \mathbf{Poly}$ by composing this canonical lens with $\varphi^{<n}$ to obtain a “sped-up” dynamical system $\text{Run}_n(\varphi)$. This new system has the same state system s , but its interface is now $p^{<n}$.

The canonicity of $s \rightarrow s^{<n}$ is due to the following standard result about comonoids, which can be proved inductively.

Proposition 7.20 (Defining $\delta^{(n)}$). Given a comonoid (c, ϵ, δ) , let $\delta^{(n)}: c \rightarrow c^{<n}$ be given as follows. Let $\delta^{(0)} := \epsilon$ and inductively define $\delta^{(n+1)} := \delta \circ (\delta^{(n)} \triangleleft c)$ for all $n \in \mathbb{N}$. Then we have the follow-

ing:

- (a) $\delta^{(n)}$ is a morphism $c \rightarrow c^{\triangleleft n}$ for all $n \in \mathbb{N}$;
- (b) $\delta^{(1)} = c = \text{id}_c$;
- (c) $\delta^{(2)} = \delta$; and
- (d) $\delta^{(n)} = \delta \circ \left(\delta^{(k)} \triangleleft \delta^{(n-k)} \right)$ for all $k, n \in \mathbb{N}$ with $k \leq n$, so our choice of morphism $c \rightarrow c^{\triangleleft(n+1)}$ is canonical.

Proof. We leave parts (a), (b), and (c) for Exercise 7.21. Part (d) amounts to coassociativity. $\square$

We'll continue to use the notation introduced here throughout for general comonoids.

Exercise 7.21. Prove the first three parts of Proposition 7.20.

1. Prove part (a).
2. Prove part (b).
3. Prove part (c).

Example 7.22 (Not all polynomial comonoids are state systems). At this point, a natural question to ask is whether everything we know about a state system s is captured by the fact that state systems are polynomial comonoids. In other words, are state systems the only polynomial comonoids there are?

The answer turns out to be no. After all, there is one fact about state systems from Section 7.1.2 that we did not encode in **Poly**: for a fixed state $s \in s(1)$, the target function $s[s] \rightarrow s(1)$ sending directions at s to their target states is a bijection.

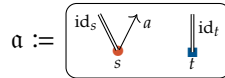
Nothing in our comonoid laws guarantees this bijectivity. An arbitrary polynomial comonoid might send different directions at s to the same target—given a second state t , there may be multiple ways to get from s to t . It might even send *no* directions at s to a target t , making it impossible to get from s to t . (We'll give an explicit example of a comonoid that is not a state system in Example 7.23.) State systems as we have defined them are just the polynomial comonoids that do not allow either of these variations,

for which the bijective property holds.

We consider this a feature, not a bug. After all, it is an abstraction to say that there is exactly one way to get from any one state in a system to another. It is perfectly plausible that the inner workings of a state system do not permit traveling between some states and differentiate ways of traveling between others. We won't formally introduce this idea into our theory of dependent dynamical systems, but we will often think of polynomial comonoids as a sort of generalized state system throughout the rest of the book.

Example 7.23 (A comonoid that is not a state system). The polynomial $y^2 + y$ is not a state system: one of its direction-sets has one fewer element than its position-set. But it can still be given a comonoid structure. We describe that structure here, but we will go a little quickly, because we'll soon discover a much more familiar way to think about comonoids.

Define $\alpha := \{s\}y^{\{id_s, a\}} + \{t\}y^{\{id_t\}} \cong y^2 + y$. Here is its tree picture:

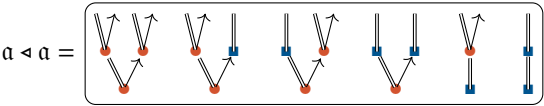


Notice that we have drawn one direction out of each position— id_s and id_t —with a double bar. We let these be the directions that the eraser $\epsilon: \alpha \rightarrow y$ picks out. The double bar is meant to evoke an equals sign from the root position to the eventual target position, which is appropriate, as these two positions should be equal for every direction that the eraser selects. We can draw the selections that ϵ makes like so:



Now we need a duplicator $\delta: \alpha \rightarrow \alpha \triangleleft \alpha$. Before we define it, let us draw out $\alpha \triangleleft \alpha$ to see what it looks like. Remember that we need

to graft corollas of α onto leaves of α in every possible way:



Each of these trees gives a way to match directions out of one position to positions they could lead to. On positions, δ will decide which matchings to pick by sending the red s to one of the four positions on the left and the blue t to one of the two positions on the right. We want the double-barred directions that the eraser picked out to have the same position on either end (in fact, the erasure laws guarantee this). So the only choice to be made is whether we want the other direction a at s to point to s or to t . Let us pick t for the time being, so that on positions, δ looks like this:



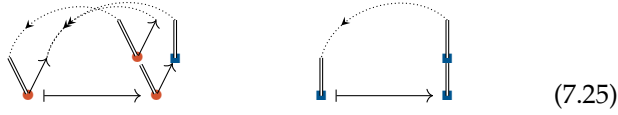
As in Example 7.6, we can interpret this as telling us how to “bend” the arrows of α so that they point to other positions:

The diagram shows a red circle (s) with two loops, connected by a horizontal arrow to a blue square (t) with two loops. This represents the bending of arrows of α so that they point to other positions.

(7.24)

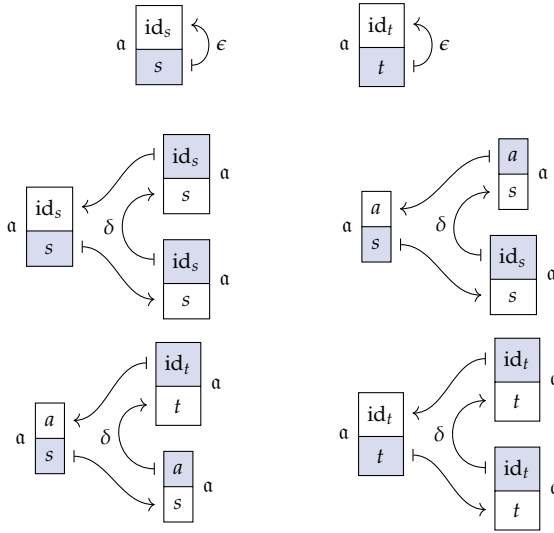
Meanwhile, on directions, δ should tell us how to run two directions together into one. Fortunately, there is not much for us to do here—we know that if one of the two directions δ runs together is one of the double-barred directions that the eraser picked out, then δ should ignore that “do-nothing” direction and yield the other direction (again, the erasure laws ensure this). Here is what that

looks like:



That is all we need to specify the triple $(\alpha, \epsilon, \delta)$.

Here are $\epsilon: \alpha \rightarrow y$ and $\delta: \alpha \rightarrow \alpha \triangleleft \alpha$ again, in terms of polyboxes.



Of course, we have yet to check that $(\alpha, \epsilon, \delta)$ really is a comonoid, i.e. that the diagrams in (7.15) and (7.16) commute. We leave that for Exercise 7.26.

Exercise 7.26. Verify that $(\alpha, \epsilon, \delta)$ as defined in Example 7.23 obeys the erasure laws in (7.15) and the coassociative law in (7.16).

Exercise 7.27. Show that if B is a set, then there exists a unique comonoid structure on the linear polynomial $B\gamma$.

Once you know that state systems are comonoids in **Poly**, but not the only ones, the natural question to ask is “what are all the other comonoids in **Poly**?” Or perhaps, as we led you through this case study of $\mathbb{s}$, you have already suspected the truth: a polynomial comonoid—what with its directions leading from one position to another, directions that can be run together associatively among which there are directions at every position that do nothing—is just another name for a category.

7.2 Polynomial comonoids are categories

What Ahman and Uustalu showed was that polynomial comonoids can be identified with categories. Every category in the usual sense is a comonoid in **Poly**, and every comonoid in **Poly** is a category. We find their revelation to be truly shocking, and it suggests some very different ways to think about categories. But let’s go over their result first.

Theorem 7.28 (Ahman-Uustalu). There is a one-to-one isomorphism-preserving correspondence between polynomial comonoids and (small) categories.

Our goal is to spell out this correspondence so that we can justly proclaim:

*Comonoids in **Poly** are precisely categories!*

7.2.1 Translating between polynomial comonoids and categories

First, we describe how to translate between the carrier $\mathfrak{c}$ of a comonoid $\mathcal{C} := (\mathfrak{c}, \delta, \epsilon)$ and the objects and morphisms of the corresponding category $\mathcal{C}$. The idea is pretty simple, and you may have already guessed it: positions are objects and directions are morphisms.

Positions as objects, directions as morphisms

More precisely, the positions of $\mathfrak{c}$ are the objects of $\mathcal{C}$:

$$\mathfrak{c}(1) = \text{Ob } \mathcal{C}. \tag{7.29}$$

Then for each such position or object i , the $\mathfrak{c}[i]$ -directions are the morphisms of $\mathcal{C}$ with domain i :

$$\mathfrak{c}[i] = \sum_{j \in \text{Ob } \mathcal{C}} \mathcal{C}(i, j). \quad (7.30)$$

The right-hand side above is a little clumsier than the left; this is because while we are used to thinking of hom-sets of categories such as $\mathcal{C}(i, j)$, consisting of all morphisms in $\mathcal{C}$ with a fixed domain and codomain, we aren't used to thinking about the collection of all morphisms in $\mathcal{C}$ with a fixed domain and an arbitrary codomain quite as often.² On the other hand, the carrier *only* encodes which morphisms have each object as its domain, i.e. which directions are at each position. Codomains will be encoded in the data of the comonoid elsewhere.

This is the key difference in perspective between the polynomial comonoid perspective of categories, in contrast to our usual hom-set perspective: the polynomial perspective is in a sense domain-centric, as highlighted by the following definition.

Definition 7.31 (Polynomial carrier). Let $\mathcal{C}$ be a category. For every object i in $\mathcal{C}$, denote the morphisms in $\mathcal{C}$ with domain i by $\mathcal{C}[i]$, so that³

$$\mathcal{C}[i] := \sum_{j \in \text{Ob } \mathcal{C}} \mathcal{C}(i, j).$$

Then the *polynomial carrier*, or simply *carrier*, of $\mathcal{C}$ is the polynomial

$$\sum_{i \in \text{Ob } \mathcal{C}} y^{\mathcal{C}[i]}.$$

So everything we have said so far about the correspondence from Theorem 7.28 can be summarized by saying that it preserves carriers: the carrier of the category $\mathcal{C}$ is the carrier $\mathfrak{c}$ of the comonoid $\mathcal{C}$, so that $\text{Ob } \mathcal{C} = \mathfrak{c}(1)$ and $\mathcal{C}[i] = \mathfrak{c}[i]$.

Remark 7.32. If we take the perspective that categories are equal if and only if their objects and morphisms are equal and obey the same laws,

²Except, perhaps, in the context of coslice categories.

³We may also write $f: i \rightarrow _$ to denote an arbitrary morphism $f \in \mathcal{C}[i]$, i.e. a morphism f in $\mathcal{C}$ with domain i and an unspecified codomain.

and similarly that polynomials are equal if and only if their position-sets and direction-sets are equal sets, then (7.29) and (7.30) really can be just strict equalities. This is why we are comfortable naming a “one-to-one correspondence” in Theorem 7.28 rather than just, say, some form of equivalence. Since the positions and directions of our polynomials always form *sets*, however, the categories we obtain under this correspondence are also necessarily *small*: their objects form a set, as do all of their morphisms. But we won’t worry too much about size issues beyond this.

Exercise 7.33. What is the carrier of each of the following categories (up to isomorphism)?

1. The category

$$\boxed{A \xrightarrow{f} B}$$

where we have drawn every morphism except for the identity morphisms.

2. The category

$$\boxed{B \xrightarrow{g} A \xleftarrow{h} C}$$

where we have drawn every morphism except for the identity morphisms.

3. The empty category.
4. A category with exactly 1 object and a morphism i , for which every morphism can be written uniquely as the n -fold composite of i for some $n \in \mathbb{N}$.
5. The category

$$\boxed{0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots}$$

where there is a unique morphism $m \rightarrow n$ if $m \leq n$ (and no other morphisms).

6. The category

$$\boxed{0 \leftarrow 1 \leftarrow 2 \leftarrow 3 \leftarrow \dots}$$

where there is a unique morphism $m \leftarrow n$ if $m \leq n$ (and no other morphisms).

But a category $\mathcal{C}$ is more than its carrier polynomial, just as a comonoid $\mathcal{C}$ is more than its carrier c . In particular, we have said nothing about the codomains of morphisms in $\mathcal{C}$, nor anything about identity morphisms, composition, or how the laws of a category are satisfied. Similarly, we have said nothing about the eraser ϵ or the duplicator δ of $\mathcal{C}$, nor anything about how the comonoid laws are satisfied. It turns out that all of these constituents and laws correspond to one another, as summarized by the following table. Here each item in the comonoid column—either a polynomial, a lens, or a lens equation—spans two rows, with the top row corresponding to positions and the bottom row corresponding to directions.

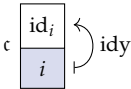
Comonoid	$\mathcal{C} = (c, \epsilon, \delta)$	Category	$\mathcal{C}$
carrier	$i \in c(1)$	objects	$i \in \text{Ob } \mathcal{C}$
	$f \in c[i]$	morphisms	$f: i \rightarrow _$
eraser	$\epsilon_1: c(1) \rightarrow 1$	—	—
	$\epsilon_i^\# : 1 \rightarrow c[i]$	identities	$\text{id}_i: i \rightarrow _$
duplicator	$\delta_1: c(1) \rightarrow (c \triangleleft c)(1)$	codomains*	$\text{cod}: \mathcal{C}[i] \rightarrow \text{Ob } \mathcal{C}$
	$\delta_i^\# : (c \triangleleft c)[\delta_1(i)] \rightarrow c[i]$	composition*	$\S$
right erasure law		* right identity law	
left erasure law		codomains of identities	$\text{cod id}_i = i$
		left identity law	
coassociative law		codomains of composites	$\text{cod}(f \S g) = \text{cod } g$
		associative law of composition	

Note that the on-positions function of ϵ , being a function into the terminal set, encodes no actual data. The asterisk $*$ indicates that the right erasure law on positions works together with the duplicator to ensure that codomains and composites are properly specified.

We have already covered the correspondence between the first two rows, so let us consider each of the following rows in turn. In some sense, we have already seen each piece of this correspondence in action for state systems in Section 7.1, so we'll go through it a little faster this time for the general case.

The eraser assigns identities

We know that the eraser $\epsilon: c \rightarrow y$ can be identified with a dependent function $(i \in c(1)) \rightarrow [i]$, sending each position $i \in (1)$ to a $[i]$ -direction. In terms of our category $\mathcal{C}$, the eraser sends each object $i \in \text{Ob } \mathcal{C}$ to a morphism $i \rightarrow _$. But this is exactly what we need to specify identity morphisms—a morphism out of each object. So the eraser of c specifies the identity morphisms of the corresponding category $\mathcal{C}$. We can interpret the polybox picture for ϵ like so:

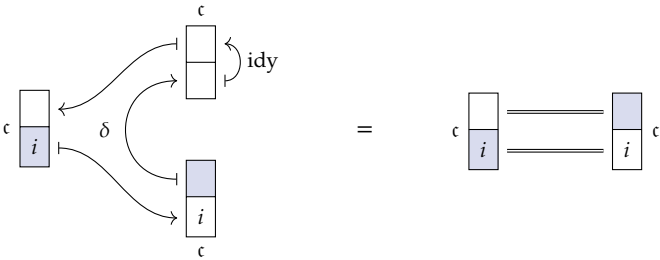


Here we have given the label id_y to the arrow sending objects to their identity morphisms.

Keep in mind that from the domain-centric polynomial perspective, we have not yet specified that the codomain of an identity morphism is equal to its domain; that comes later.

The right erasure law on positions: some bookkeeping

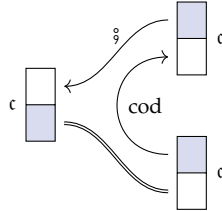
Keeping our label id_y for the arrow in ϵ , the right erasure law $\delta \circ (\epsilon \triangleleft c) = \text{id}_c$ from (7.15) can be drawn in polyboxes like so:



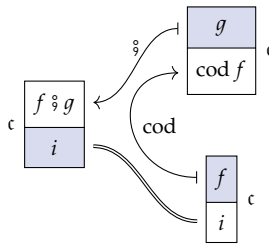
We have not filled all the boxes, but we have filled enough to interpret the right erasure law on positions: that the bottom arrow of the duplicator must be the identity function on $c(1)$. Equipped with this knowledge, we can focus on the other two arrows of δ .

The duplicator assigns codomains and composites

In fact, in the polybox picture for $\delta: c \rightarrow c \triangleleft c$, the middle arrow specifies codomains, and the top arrow specifies composition. We therefore label these arrows as follows:⁴



To check that this makes sense, we fill in the boxes:

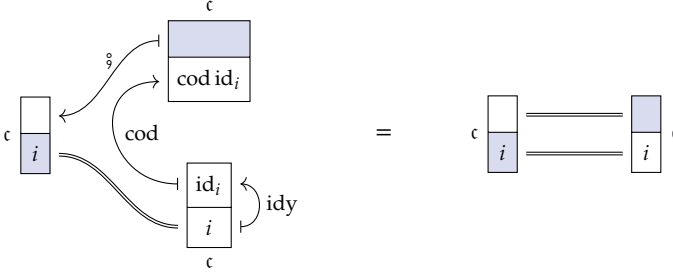


Remember: each position box contains an object of $\mathcal{C}$, while each direction box contains a morphism of $\mathcal{C}$ emanating from the object below. So δ takes an object $i \in \text{Ob } \mathcal{C}$ and a morphism $f: i \rightarrow _$ in $\mathcal{C}$ and assigns another object $\text{cod } f \in \text{Ob } \mathcal{C}$ to be the codomain of f . It then takes another morphism $g: \text{cod } f \rightarrow _$ in $\mathcal{C}$ and assigns a morphism $f \circ g: i \rightarrow _$ to be the composite of f and g . In this way, every morphism gets a codomain, and every pair of morphisms that can be composed (i.e. the codomain of one matches the domain of the other) is assigned a composite. As with the identity morphism, we don't know what the codomain of this composite morphism is yet; but we do know that the domain of $f \circ g$ matches the domain of f , as it should.

⁴Compare these labels to the names “target” and “run” that we gave to the arrows of a state system's transition lens.

The left erasure law on positions: codomains of identities

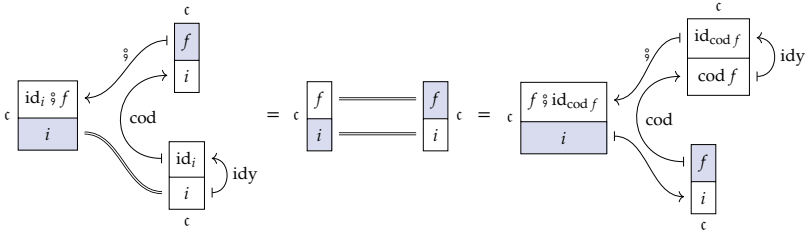
As with the right erasure law, we can partially fill in the polyboxes for the left erasure law $\delta \circ (\epsilon \triangleleft c) = \text{id}_c$ from (7.15) to read what it says on positions:



So the left erasure law on positions guarantees that $\text{cod id}_i = i$ for all $i \in \text{Ob } \mathcal{C}$. It makes sense that we would find this here: the eraser assigns identities, while the duplicator assigns codomains, so a statement about codomains of identities is a coherence condition between the eraser and the duplicator.

The erasure laws on directions are the identity laws

Let us finish filling in the polyboxes for the left and right erasure laws to see what they have to say on directions. In the picture below, the left equality depicts the left erasure law (to conserve space, we'll substitute i for cod id_i on the left, which we now know we can do), while the right equality depicts the right erasure law:



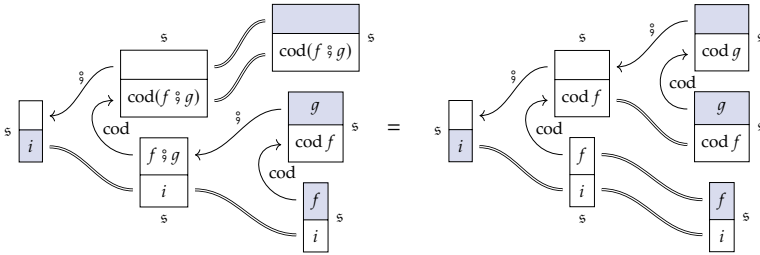
We find that on directions, the erasure laws state that for every object $i \in \text{Ob } \mathcal{C}$ and morphism $f: i \rightarrow _$ in $\mathcal{C}$,

$$\text{id}_i \circ f = f = f \circ \text{id}_{\text{cod } f}.$$

But these are precisely the identity laws of the category $\mathcal{C}$.

The coassociative law on positions: codomains of composites

It remains to consider the comonoid's coassociative law (7.16), $\delta \circ (\delta \triangleleft \mathfrak{s}) = \delta \circ (\mathfrak{s} \triangleleft \delta)$. To read what it says on positions, we draw the polyboxes and fill them in, stopping just short of the uppermost direction box of the codomain:



So on positions, the coassociative law states that given an object $i \in \text{Ob } \mathcal{C}$ and morphisms $f: i \rightarrow _$ and $g: \text{cod } f \rightarrow _$ in $\mathcal{C}$,

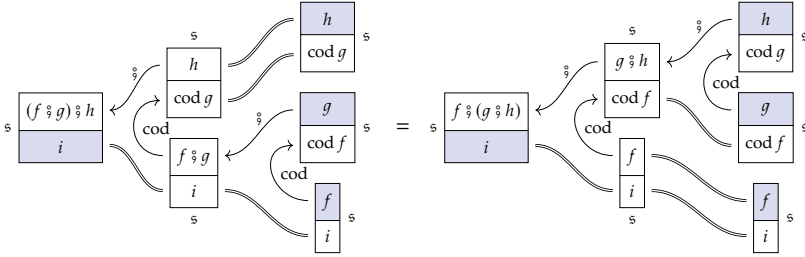
$$\text{cod}(f \circ g) = \text{cod } g.$$

Hence composites are assigned the proper codomains.

The coassociative law on directions is the associative law of composition

Finally, let us fill in the remaining polyboxes for the coassociative law (we'll substitute $\text{cod } g$ for $\text{cod}(f \circ g)$ on the left, which we now know we

can do):



Thus, on directions, the coassociative law states that given an object $i \in \text{Ob } \mathcal{C}$ and morphisms $f: i \rightarrow _$, $g: \text{cod } f \rightarrow _$, and $h: \text{cod } g \rightarrow _$ in $\mathcal{C}$,

$$(f \circ g) \circ h = f \circ (g \circ h).$$

But this is precisely the associative law of composition in a category.

We've seen that the data and equations of polynomial comonoids correspond exactly to the data and equations of categories. This proves Theorem 7.28.

Generalized duplicators as unbiased composition

Before we move onto examples, one more note about the theory: notice that both sides of our coassociative law are given by $\delta^{(3)}: c \rightarrow c^{\triangleleft 3}$, as defined in Proposition 7.20. On directions, $\delta^{(3)}$ tells us how to compose three morphisms $i \xrightarrow{f} _ \xrightarrow{g} _ \xrightarrow{h} _$ in $\mathcal{C}$ all at once to obtain $i \xrightarrow{f \circ g \circ h} _$ and (co)associativity ensures this is well-defined.

In general, $\delta^{(n)}: c \rightarrow c^{\triangleleft n}$ on directions tells us how to compose n morphisms in $\mathcal{C}$ for each $n \in \mathbb{N}$. After all, we have already seen that $\delta^{(2)} = \delta$ performs binary composition, that $\delta^{(1)} = \text{id}_c$ performs “unary” composition (the “unary composite” of a single morphism f is just f itself), and that $\delta^{(0)} = \epsilon$ performs “nullary” composition (the “nullary composite” at any object is just its identity). The directions of $c^{\triangleleft n}$ at positions in the image of $\delta^{(n)}$ are exactly the sequences of composable morphisms of length n , and $\delta^{(n)}$ sends each sequence to the single direction that is its composite.

7.2.2 Examples of categories as comonoids

Now that we know that polynomial comonoids are just categories, let's review some simple examples of categories and see how they may be interpreted as comonoids. As we go through these examples, pay attention to how the polynomial perspective causes us to view these familiar categories somewhat differently than usual.

Preorders

A *preorder* (or *thin category*) is a category in which every morphism $f: c \rightarrow d$ is the *only* morphism $c \rightarrow d$.⁵ Composition in preorders is easy to describe, because the composite of $c \rightarrow d$ and $d \rightarrow e$ is always just the unique arrow $c \rightarrow e$. As such, preorders are some of the simplest examples of categories to consider—we already saw several in Exercise 7.33—so let us interpret these as comonoids first.

Example 7.34. Let us revisit Example 7.23, where we first wrote down a comonoid that was not a state system. We defined $\alpha := \{s\}y^{\{\text{id}_s, a\}} + \{t\}y^{\{\text{id}_t\}} \cong y^2 + y$ and gave it a comonoid structure, with eraser $\epsilon: \alpha \rightarrow y$ specifying directions id_s and id_t and duplicator $\delta: \alpha \rightarrow \alpha \triangleleft \alpha$ pointing the direction a at t .

Looking at the picture we drew of the comonoid in (7.24), it should come as no surprise that the corresponding category $\mathcal{A}$ is the *walking arrow category*, which is a preorder with two objects and one morphism between them:

$$\mathcal{A} := \boxed{s \xrightarrow{a} t}$$

Here we omit the identity morphisms from our picture, but we know that they exist.

The category $\mathcal{A}$ has two objects, the α -positions s and t . It has two morphisms with domain s , the $\alpha[s]$ -directions id_s and a ; and one morphism with domain t , the $\alpha[t]$ -direction id_t . The mor-

⁵Sometimes these are also called *posets*, short for *partially ordered sets*, but strictly speaking the only isomorphisms in a poset are its identities, while a preorder allows objects to be isomorphic without being equal.

phisms id_s and id_t picked out by the erasure are the identity morphisms, and the duplicator assigns them codomains that are equal to their domains. The duplicator also assigns a the codomain t ; and as $\mathcal{A}$ is then a preorder, composites are determined automatically.

Exercise 7.35. Let (ϵ, δ) be the comonoid corresponding to the preorder depicted as follows (identity morphisms omitted):

$$B \xleftarrow{f} A \xrightarrow{g} C$$

1. What is the carrier ϵ ?
2. Characterize the eraser ϵ .
3. Characterize the duplicator δ .

Exercise 7.36. We showed in Exercise 7.27 that for any set B , the linear polynomial By has a unique comonoid structure. To what category does this comonoid correspond?

Exercise 7.37.

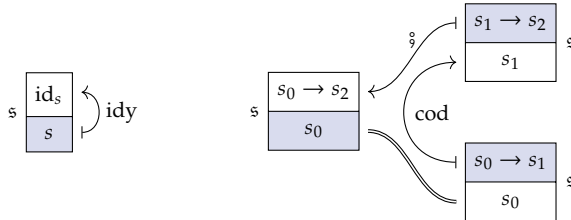
1. Find a comonoid structure for the polynomial $p := y^{n+1} + ny$ whose corresponding category is a preorder. (It is enough to fully describe the category that it corresponds to.)
2. Would you call your category “star-shaped”?

Example 7.38 (State systems as categories). We know that every state system $s \cong Sy^S$ with its do-nothing section $\epsilon: s \rightarrow y$ and its transition lens $\delta: s \rightarrow s \triangleleft s$ is a comonoid, so what category $\mathcal{S}$ does (s, ϵ, δ) correspond to?

Recall from Example 7.22 that state systems are exactly those comonoids whose codomain (i.e. “target”) functions $\text{cod}: s[s] \rightarrow s(1)$ for $s \in s(1)$ are bijections. That is, from every object $s \in \text{Ob } \mathcal{S} = s(1)$, there is exactly 1 morphism to every object $t \in \text{Ob } \mathcal{S}$. So not only is $\mathcal{S}$ a preorder, it is the *codiscrete preorder* on $s(1)$, where there

is always a morphism between every pair of objects.

Let us redraw the polyboxes for the do-nothing section of $\mathfrak{s}$ from (7.1) and the transition lens of $\mathfrak{s}$ from (7.4), this time with our new arrow labels, as a sanity check:

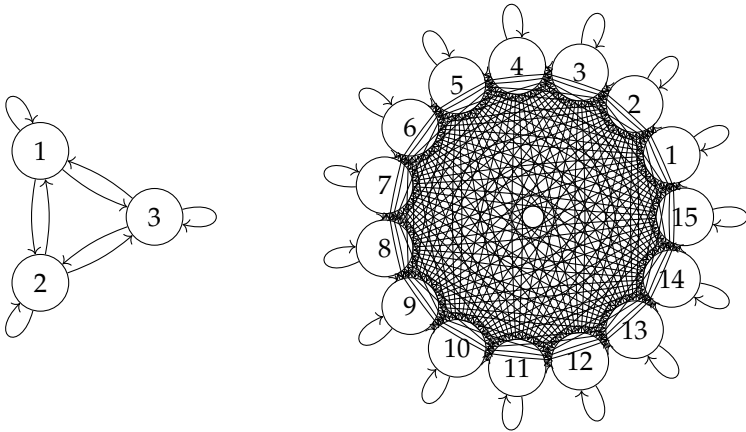


Indeed, we had already been writing the directions of $\mathfrak{s}$ as arrows $s \rightarrow t$, knowing that each was uniquely specified by its source s and its target t in $\mathfrak{s}(1)$. And in Section 7.1.3, we had already noted that id_s was just the arrow $s \rightarrow s$. So state systems have been categories with exactly one morphism between every pair of objects all along.

Other names for this category include the *indiscrete preorder* and the *codiscrete* or *indiscrete category*. These names highlight the fact that every object of this category is isomorphic to every other object: in fact, every arrow $s \rightarrow t$ is an isomorphism with inverse $t \rightarrow s$, for these compose as $\text{id}_s : s \rightarrow s$ in one direction and $\text{id}_t : t \rightarrow t$ in the other. Thus this category is also a *groupoid*, and it may be called the *codiscrete*, *indiscrete*, or *contractible groupoid*. . . but we will call it the *state category on S* , where S is the set of positions of $\mathfrak{s}$ or objects of $\mathcal{S}$.

Here are the state categories on 3 and on 15, with all maps (even

identities) drawn:



The picture on the left should look familiar: it is what we drew in Example 7.6 when took the corolla picture for $3y^3$ and bent the arrows to point at their targets according to its transition lens. Notice that the graphs we obtain in this way are always complete.

Exercise 7.39. Let S be a set. Is there any comonoid structure on Sy^S other than that of the state category?

Not only does Example 7.38 finally explain what our state systems really are (they're just special categories!), it illustrates two important features of our story. One is that on positions, the duplicator $\delta: c \rightarrow c \triangleleft c$ of a comonoid takes the corolla picture of c and "bends the arrows" so that they point to other roots, yielding the underlying graph of a category. Then δ on directions collapses two-arrow paths in the graph down to individual arrows, while the eraser $\epsilon: c \rightarrow y$ identifies empty paths with identity arrows.

Another important point is that we can view any category as a *generalized state system*: its objects as *states*, and its morphisms as *transitions* between states. The polynomial comonoid perspective is particularly suited for thinking about categories in this way: each object is a position

that we could be in, and each morphism out of that object is a direction that we might take. What is special about a comonoid is that each direction will always have another position at the end of it, making it reasonable to think of these directions as transitions between different states; and any sequence of transitions that we can follow is itself a transition.

Comparing these ideas, we see that they say the same thing: the first from the perspective of trees and graphs, the second from the perspective of dynamics. We might say that

*a comonoid structure on a corolla forest turns
roots into vertices and
leaves into composable arrows between vertices;*

or that

*a comonoid structure on a polynomial turns
positions into states and
directions into composable transitions between states.*

Monoids and monoid actions

Here we use *monoid* to refer to a monoid in the monoidal category $(\mathbf{Set}, 1, \times)$. We denote such a monoid by $(M, e, *)$, where M is the underlying set, $e \in M$ is the unit, and $*: M \times M \rightarrow M$ is the binary operation.

Example 7.40 (Monoids as representable comonoids). Recall that every monoid $(M, e, *)$ can be identified with a 1-object category $\mathcal{M}$ with a single hom-set M , a single identity morphism e , and composition given by $*$. Now we know that a 1-object category $\mathcal{M}$ is also a polynomial comonoid $(\mathfrak{m}, \epsilon, \delta)$ whose carrier has 1 position, with all of the morphisms of $\mathcal{M}$ becoming its directions. So the carrier of $\mathcal{M}$ is the representable polynomial y^M .

Then the eraser $\epsilon: y^M \rightarrow y$ picks out the identity morphism $e \in M$ on directions, while the duplicator $\delta: y^M \rightarrow y^M \triangleleft y^M \cong y^{M \times M}$ can be identified with the binary operation $*: M \times M \rightarrow M$. (We don't have to worry about codomains, since there is only one possi-

ble codomain to choose from.) In this way, every monoid $(M, e, *)$ in **Set** gives rise to a representable comonoid (y^M, ϵ, δ) in **Poly**. We can just as easily invert this construction, obtaining a monoid for every representable comonoid by taking the underlying set to be the carrier's set of directions, the unit to be the direction picked out by the erasure, and the binary operation to be the duplicator's on-directions function.

Exercise 7.41. Verify Example 7.40 by showing that $(M, e, *)$ satisfies the unitality and associativity requirements of a monoid in $(\mathbf{Set}, 1, \times)$ if and only if (y^M, ϵ, δ) satisfies the erasure and coassociativity requirements of a comonoid in $(\mathbf{Poly}, y, \triangleleft)$.

Example 7.42 (Cyclic lists). For any $n \in \mathbb{N}$, consider $\mathbb{Z}/n\mathbb{Z}$, the cyclic group of order n , viewed as a monoid or, equivalently, a 1-object category. Its carrier is $y^{\mathbb{Z}/n\mathbb{Z}}$.

As a polynomial functor, $y^{\mathbb{Z}/n\mathbb{Z}}$ sends each set X to the set of length- n tuples in X . But the comonoid structure lets us think of these tuples as *cyclic lists*: once we reach the last element, we can loop back around to the first element. Indeed, as a natural transformation, $\epsilon: y^{\mathbb{Z}/n\mathbb{Z}} \rightarrow y$ picks out the “current” element via its X -component $\epsilon \triangleleft X: y^{\mathbb{Z}/n\mathbb{Z}} \triangleleft X \rightarrow y \triangleleft X$, which is just a function $\epsilon_X: X^{\mathbb{Z}/n\mathbb{Z}} \rightarrow X$; and δ lets us move around the list.

We will see later that comonoids are closed under coproducts, so $\sum_{n \in \mathbb{N}} y^{\mathbb{Z}/n\mathbb{Z}}$ is also a comonoid.

Example 7.43 (Monoid actions). Suppose that $(M, e, *)$ is a monoid, S is a set, and $\alpha: S \times M \rightarrow S$ is a (*right*) monoid action. That is, for all $s \in S$ we have $\alpha(s, e) = s$ and $\alpha(s, m * n) = \alpha(\alpha(s, m), n)$ for

$m, n \in M$; equivalently, the diagrams

$$\begin{array}{ccc}
 S \times 1 & \xrightarrow{S \times e} & S \times M \\
 & \searrow & \downarrow \alpha \\
 & & S
 \end{array}
 \quad \text{and} \quad
 \begin{array}{ccc}
 S \times M \times M & \xrightarrow{S \times *} & S \times M \\
 \downarrow \alpha \times M & & \downarrow \alpha \\
 S \times M & \xrightarrow{\alpha} & S
 \end{array}$$

commute.

Then there is an associated category $\mathcal{M}\mathcal{A}$ with objects in S and morphisms $s \xrightarrow{m} \alpha(s, m)$ for each $s \in S$ and $m \in M$. This, in turn, corresponds to a comonoid $(S y^M, \epsilon, \delta)$, as we will see in the next exercise.

Exercise 7.44. With notation as in Example 7.43, characterize the comonoid structure on $S y^M$.

1. How can we define the erasure ϵ ?
2. How can we define the duplicator δ ?
3. Verify that the erasure laws hold.
4. Verify that the coassociative law holds.
5. Describe the corresponding category $\mathcal{M}\mathcal{A}$. In particular, what are the morphisms between any fixed pair of objects, what are the identity morphisms, and how do morphisms compose?
6. M always acts on itself by multiplication. Is the associated comonoid structure on $M y^M$ the same or different from the one coming from Example 7.38?

Example 7.45 (The category of B -streams). Fix a set B . The set $B^{\mathbb{N}}$ consists of countable sequences of elements in B , which we will call B -streams. We can write a B -stream $\bar{b} \in B^{\mathbb{N}}$ as

$$\bar{b} := (b_0 \rightarrow b_1 \rightarrow b_2 \rightarrow b_3 \rightarrow \cdots),$$

with $b_n \in B$ for each $n \in \mathbb{N}$.

Then there is a monoid action $\tau: B^{\mathbb{N}} \times \mathbb{N} \rightarrow B^{\mathbb{N}}$ for which

$$\tau(\bar{b}, n) := (b_n \rightarrow b_{n+1} \rightarrow b_{n+2} \rightarrow b_{n+3} \rightarrow \cdots).$$

Roughly speaking, $\mathbb{N}$ acts on B -streams by shifting them forward by a natural number of steps. We can check that this is a monoid action by observing that $\tau(\bar{b}, 0) = \bar{b}$ and that $\tau(\bar{b}, m + n) = \tau(\tau(\bar{b}, m), n)$.

So by Example 7.43, the corresponding comonoid is carried by $B^{\mathbb{N}} y^{\mathbb{N}}$. Each B -stream $\bar{b}$ is a position, and each $n \in \mathbb{N}$ is a direction at $\bar{b}$ that can be visualized as the sequence of n arrows starting from b_0 and ending at b_n . Then at the end of the direction n is a new B -stream: the rest of $\bar{b}$ starting at b_n . Indeed, this B -stream is exactly $\tau(\bar{b}, n)$, the codomain assigned to the direction n at $\bar{b}$.

Alternatively, if we shift from the domain-centric perspective to the usual hom-set perspective, this comonoid corresponds to a category whose objects are B -streams and whose morphisms $\bar{b} \rightarrow \bar{b}'$ consist of every way in which $\bar{b}'$ can be viewed as a contiguous substream of $\bar{b}$: that is, there is a morphism $n: \bar{b} \rightarrow \bar{b}'$ for each $n \in \mathbb{N}$ satisfying

$$(b_n \rightarrow b_{n+1} \rightarrow \cdots) = (b'_0 \rightarrow b'_1 \rightarrow \cdots).$$

The identity on $\bar{b}$ is given by $0: \bar{b} \rightarrow \bar{b}$; and the composite of two morphisms is the sum of the corresponding natural numbers, as a substream of a substream of $\bar{b}$ is just a substream of $\bar{b}$ shifted by the appropriate amount.

We will see this category again in Example 8.38.

Exercise 7.46. Let $\mathbb{R}/\mathbb{Z} \cong [0, 1)$ be the quotient of $\mathbb{R}$ by the $\mathbb{Z}$ -action sending $(r, n) \mapsto r + n$. More concretely, it is the set of real numbers between 0 and 1, including 0 but not 1.

1. Find a comonoid structure on $(\mathbb{R}/\mathbb{Z})y^{\mathbb{R}}$.
2. Is the corresponding category a groupoid?

The degree of an object

We could continue to list examples of polynomial comonoids, but of course any list of small categories is already a list of such comonoids. So instead, we conclude this section with some terminology that the polynomial perspective on a category affords.

Definition 7.47 (Degree, linear). Let $\mathcal{C}$ be a category and $c \in \text{Ob } \mathcal{C}$ an object. The *degree of c* , denoted $\deg(c)$, is the set of arrows in $\mathcal{C}$ that emanate from c .

If $\deg(c) \cong 1$, we say that c is *linear*. If $\deg(c) \cong n$ for $n \in \mathbb{N}$, we say c has *degree n* .

Exercise 7.48.

1. If every object in $\mathcal{C}$ is linear, what can we say about $\mathcal{C}$?
2. Is it possible for an object in $\mathcal{C}$ to have degree 0?
3. Find a category that has an object of degree $\mathbb{N}$.
4. Up to isomorphism, how many categories are there that have just one linear and one quadratic (degree 2) object?
5. Is the above the same as asking how many comonoid structures on $y^2 + y$ there are?

7.3 Morphisms of polynomial comonoids are retrofunctors

Now that we have characterized the comonoids of **Poly**, let us consider the morphisms between them. These turn out to correspond to a rather odd kind of map between categories known as a *retrofunctor*.⁶

⁶Many authors have referred to these as *cofunctors*, including ourselves in other work and in early versions of this book. However, the prefix *co* in category theory is very special—having to do with taking opposites—and we will see in Remark 7.59 that comonoid homomorphisms are not just opposite-functors. Thus to keep the prefix *co* more pristine, and in solidarity with other researchers, we have decided to use the term *retrofunctor*, which is an appropriate usage of the term defined by Bob Paré [Par23].

7.3.1 Introducing comonoid morphisms and retrofunctors

First, let us define morphisms of comonoids in the most general setting. If you've seen the definition of a monoid homomorphism (or even a group homomorphism), then this definition may look familiar.

Definition 7.49 (Comonoid morphism). Given a monoidal category $(\mathcal{C}, y, \triangleleft)$ with comonoids $\mathcal{C} := (c, \epsilon, \delta)$ and $\mathcal{C}' := (c', \epsilon', \delta')$, a *comonoid morphism* (or *morphism of comonoids*) $\mathcal{C} \rightarrow \mathcal{C}'$ is a morphism $F : c \rightarrow c'$ in $\mathcal{C}$ for which the following diagrams commute:

$$\begin{array}{ccc} c & \xrightarrow{F} & c' \\ \epsilon \downarrow & & \downarrow \epsilon' \\ y & \xlongequal{\quad} & y, \end{array} \quad (7.50)$$

called the *eraser preservation law* (we say F *preserves erasure*); and

$$\begin{array}{ccc} c & \xrightarrow{F} & c' \\ \delta \downarrow & & \downarrow \delta' \\ c \triangleleft c & \xrightarrow{F \triangleleft F} & c' \triangleleft c' \end{array} \quad (7.51)$$

called the *duplicator preservation law* (we say F *preserves duplication*). We may also say that F *preserves the comonoid structure*.

When the monoidal structure on $\mathcal{C}$ can be inferred, we let **Comon**($\mathcal{C}$) denote the subcategory of $\mathcal{C}$ whose objects are comonoids in $\mathcal{C}$ and whose morphisms are comonoid morphisms.

So when our monoidal category of interest is $(\mathbf{Poly}, y, \triangleleft)$, a morphism between polynomial comonoids is just a special kind of lens between their carriers that preserves erasure and duplication.

Exercise 7.52. There is something to be proved in Definition 7.49: that comonoids and comonoid morphisms really do form a category. Using the notation from Definition 7.49, verify the following:

1. The identity morphism on a comonoid is a comonoid morphism.

2. The composite of two comonoid morphisms is a comonoid morphism.

This will show that **Comon**($\mathcal{C}$) is indeed a subcategory of $\mathcal{C}$.

Notice that we were very careful in how we stated Theorem 7.28: while we asserted the existence of an isomorphism-preserving one-to-one correspondence between the objects of **Comon**(**Poly**) and **Cat**, we never claimed that these two categories are isomorphic or even equivalent. The strange truth of the matter is that they are not: polynomial comonoid morphisms correspond not to functors, but to different maps of categories called *retrofunctors*.

How exactly do these maps behave? If we specify Definition 7.49 to the case of $(\mathbf{Poly}, y, \triangleleft)$, we can write the eraser preservation law (7.50) in polyboxes as

$$\begin{array}{c} \mathfrak{c} \\ \boxed{} \\ \boxed{c} \end{array} \text{idy} = \begin{array}{c} \mathfrak{c} \quad \mathfrak{c}' \\ \boxed{} \quad \boxed{} \\ \boxed{c} \quad \boxed{} \end{array} \begin{array}{c} \xleftarrow{F} \\ \xrightarrow{F} \end{array} \begin{array}{c} \text{idy} \end{array} \quad (7.53)$$

and the duplicator preservation law (7.51) in polyboxes as

$$\begin{array}{c} \mathfrak{c} \\ \boxed{} \\ \boxed{c} \end{array} \begin{array}{c} \xleftarrow{\mathfrak{g}} \\ \xrightarrow{\text{cod}} \end{array} \begin{array}{c} \mathfrak{c} \\ \boxed{} \\ \boxed{} \end{array} \begin{array}{c} \xleftarrow{F} \\ \xrightarrow{F} \end{array} \begin{array}{c} \mathfrak{c}' \\ \boxed{h} \\ \boxed{g} \end{array} = \begin{array}{c} \mathfrak{c} \quad \mathfrak{c}' \\ \boxed{} \quad \boxed{} \\ \boxed{c} \quad \boxed{} \end{array} \begin{array}{c} \xleftarrow{F} \\ \xrightarrow{F} \end{array} \begin{array}{c} \mathfrak{c}' \\ \boxed{} \\ \boxed{} \end{array} \begin{array}{c} \xleftarrow{\mathfrak{g}} \\ \xrightarrow{\text{cod}} \end{array} \begin{array}{c} \mathfrak{c}' \\ \boxed{h} \\ \boxed{g} \end{array} \quad (7.54)$$

If we read off the equations from these polyboxes, interpreting polynomial comonoids as categories, we derive the following definition of a retrofunctor. (Here (7.56) is equivalent to (7.53), while (7.57) and (7.58) are together equivalent to (7.54).)

Definition 7.55 (Retrofunctor). Let $\mathcal{C}$ and $\mathcal{C}'$ be (small) categories. A *retrofunctor* $F: \mathcal{C} \rightarrow \mathcal{C}'$ consists of

- a function $F: \text{Ob } \mathcal{C} \rightarrow \text{Ob } \mathcal{C}'$ *forward on objects*⁷ and

- a function $F_c^\sharp: \mathcal{C}'[Fc] \rightarrow \mathcal{C}[c]$ backward on morphisms for each $c \in \text{Ob } \mathcal{C}$,

satisfying the following conditions, collectively known as the *retrofunctor laws*:

- i. F preserves identities:

$$F_c^\sharp \text{id}_{Fc} = \text{id}_c \quad (7.56)$$

for each $c \in \text{Ob } \mathcal{C}$;

- ii. F preserves codomains:

$$F \text{cod } F_c^\sharp g = \text{cod } g \quad (7.57)$$

for each $c \in \text{Ob } \mathcal{C}$ and $g \in \mathcal{C}'[Fc]$;

- iii. F preserves composites:⁸

$$F_c^\sharp g \circ F_{\text{cod } F_c^\sharp g}^\sharp h = F_c^\sharp (g \circ h) \quad (7.58)$$

for each $c \in \text{Ob } \mathcal{C}$, $g \in \mathcal{C}'[Fc]$, and $h \in \mathcal{C}'[\text{cod } g]$.

We let $\mathbf{Cat}^\sharp \cong \mathbf{Comon}(\mathbf{Poly})$ denote the category of (small) categories and retrofunctors.

Remark 7.59. We explain the term *retrofunctor* from Definition 7.55 here. Let $\mathcal{C}, \mathcal{C}'$ be categories, and consider them as monads in $\mathbf{Span}$. A functor between them consists of a function $F_{\text{Ob}}: \text{Ob}(\mathcal{C}) \rightarrow \text{Ob}(\mathcal{C}')$ and a 2-cell, as shown left

$$\begin{array}{ccc} \text{Ob}(\mathcal{C}) & \xrightarrow{\text{Mor}(\mathcal{C})} & \text{Ob}(\mathcal{C}) \\ F_{\text{Ob}} \downarrow & \Downarrow F_{\text{Mor}} & \downarrow F_{\text{Ob}} \\ \text{Ob}(\mathcal{C}') & \xrightarrow{\text{Mor}(\mathcal{C}')} & \text{Ob}(\mathcal{C}') \end{array}$$

$$\begin{array}{ccc} & \text{Mor}(\mathcal{C}) & \\ & \downarrow F_{\text{Mor}} & \\ \text{Ob}(\mathcal{C}) & \xrightarrow{\text{Mor}(\mathcal{C})} & \text{Ob}(\mathcal{C}) \\ & \uparrow \widetilde{F_{\text{Ob}}} \circ \text{Mor}(\mathcal{C}') \circ \widetilde{F_{\text{Ob}}} & \end{array}$$

satisfying the properties of a monad homomorphism.

⁷In keeping with standard functor notation, we omit the usual subscript 1 that we include for on-positions (in this case, on-objects) functions. We often omit parentheses when applying these functions as well.

⁸In particular, the codomains of either side of (7.58) are equal. This isn't actually guaranteed by the other laws, so it is worth noting on its own; see for example the proof of Proposition 7.61.

In [Par23, Definition 6.1], Paré defines *retromorphism of monads* in double categories like **Span**. We will not discuss the more general definition here, but in a framed bicategory (equipment), where we have companions $\check{f}$ and conjoiners $\widehat{f}$ of tight maps f , it is easy to check that monads lift along tight morphisms in the sense that the horizontal cell $(\widetilde{F_{\text{Ob}}} \circ \text{Mor}(C') \circ \widehat{F_{\text{Ob}}}) : \text{Ob}(C) \rightarrow \text{Ob}(C)$ is a monad if $\text{Mor}(C')$ is. Hence, a functor is equivalently given by a function $F_{\text{Ob}} : \text{Ob}(C) \rightarrow \text{Ob}(C')$ such that the 2-cell shown above right is a monad homomorphism.

A retromorphism of monads—in our case, a *retrofunctor*—is simply a monad map going the other way:⁹

$$\begin{array}{ccc}
 & \text{Mor}(C) & \\
 \text{Ob}(C) & \xrightarrow{\quad} & \text{Ob}(C) \\
 & \uparrow \parallel & \\
 & \text{Mor}(C) & \\
 & \xleftarrow{\quad} & \\
 & \widetilde{F_{\text{Ob}}} \circ \text{Mor}(C') \circ \widehat{F_{\text{Ob}}} &
 \end{array}$$

Henceforth we will identify the category $\mathbf{Cat}^\#$ with the isomorphic category **Comon(Poly)**, eliding the difference between comonoids in **Poly** and categories.

Since each retrofunctor includes a lens between its carrier polynomials, retrofunctors compose the way lenses do.

Exercise 7.60. Let $\mathcal{C}, \mathcal{D}, \mathcal{E}$ be categories, and let $F : \mathcal{C} \rightarrow \mathcal{D}$ and $G : \mathcal{D} \rightarrow \mathcal{E}$ be retrofunctors between them.

1. Characterize the behavior of the identity retrofunctor $\text{id}_{\mathcal{D}}$ on $\mathcal{D}$. Where does it send each object? Where does it send each morphism?
2. Characterize the behavior of the composite retrofunctor $F \circ G$. Where does it send each object and morphism?

On the surface, functors and retrofunctors have much in common: both send objects to objects and morphisms to morphisms in a way that

⁹As a hint to the connection in terms of the (yet-undefined) double category $\mathbf{Cat}^\#$, note that when the monads in question are left adjoints, their right adjoints will automatically be comonads, and a morphism between these comonads will be a retromorphism between the monads.

preserves domains and codomains as well as identities and composites. The main difference is that functors send morphisms *forward*, while retrofunctors send morphisms *backward*. As we work with retrofunctors, it will be helpful to remember the following:

A retrofunctor F goes forward on objects and backward on morphisms.

Codomains are objects, so F preserves them going forward.

Identities and composites are morphisms, so F preserves them going backward.

Before we explore just how different functors and retrofunctors can be, let us note a few more similarities that these two kinds of maps between categories share. For example, retrofunctors, like functors, preserve isomorphisms.

Proposition 7.61. Let $F: \mathcal{C} \rightarrow \mathcal{D}$ be a retrofunctor, $c \in \mathcal{C}$ be an object, and $g: Fc \rightarrow _$ be an isomorphism in $\mathcal{D}$. Then $F_c^\# g$ is also an isomorphism in $\mathcal{C}$.

Proof. Let $c' := \text{cod } F_c^\# g$, so that $Fc' = \text{cod } g$ by (7.57), and let $g^{-1}: Fc' \rightarrow Fc$ be the inverse of g . Then

$$\text{id}_c = F_c^\# \text{id}_{Fc} \quad (7.56)$$

$$\begin{aligned} &= F_c^\# (g \circ g^{-1}) \\ &= F_c^\# g \circ F_{c'}^\# (g^{-1}), \end{aligned} \quad (7.58)$$

so in particular $c = \text{cod id}_c = \text{cod } F_{c'}^\# (g^{-1})$, and

$$\text{id}_{c'} = F_{c'}^\# \text{id}_{Fc'} \quad (7.56)$$

$$\begin{aligned} &= F_{c'}^\# (g^{-1} \circ g) \\ &= F_{c'}^\# (g^{-1}) \circ F_{\text{cod } F_{c'}^\# (g^{-1})}^\# g \\ &= F_{c'}^\# (g^{-1}) \circ F_c^\# g. \end{aligned} \quad (7.58)$$

Hence $F_c^\# g$ and $F_{c'}^\# (g^{-1})$ are inverses, and the result follows. $\square$

Moreover, isomorphisms in **Cat** correspond to isomorphisms in **Cat**[#].

Exercise 7.62. We’ve justified the “isomorphism-preserving” part of Theorem 7.28 implicitly, but let’s make it explicit.

Recall that two categories $\mathcal{C}$ and $\mathcal{D}$ are isomorphic in **Cat** if there exist functors $F: \mathcal{C} \rightarrow \mathcal{D}$ and $G: \mathcal{D} \rightarrow \mathcal{C}$ that are mutually inverse, i.e. $F \circ G$ and $G \circ F$ are identity functors on $\mathcal{C}$ and $\mathcal{D}$. Similarly, $\mathcal{C}$ and $\mathcal{D}$ are isomorphic in **Cat**[#] if there exist retrofunctors $H: \mathcal{C} \rightarrow \mathcal{D}$ and $K: \mathcal{D} \rightarrow \mathcal{C}$ that are mutually inverse, i.e. $H \circ K$ and $K \circ H$ are identity retrofunctors on $\mathcal{C}$ and $\mathcal{D}$. Show that $\mathcal{C}$ and $\mathcal{D}$ are isomorphic in **Cat** if and only if they are isomorphic in **Cat**[#].

But while isomorphisms in **Cat**[#] are the same as those in **Cat**, the non-isomorphisms can be very different. We will see this in the examples to come.

7.3.2 Examples of retrofunctors

From a realm where functors reign supreme, the back-and-forth behavior of retrofunctors can seem foreign and counterintuitive. Whereas a functor $\mathcal{C} \rightarrow \mathcal{D}$ can be thought of as a *diagram*—a picture in the shape of $\mathcal{C}$, drawn with the objects and arrows of $\mathcal{D}$ —retrofunctors are much more like the dynamical systems of Chapter 4.¹⁰

That is, a retrofunctor $F: \mathcal{C} \rightarrow \mathcal{D}$ is a way of interacting with the states (objects) and transitions (morphisms) within $\mathcal{C}$ by way of $\mathcal{D}$. Imagine the retrofunctor as a box, with $\mathcal{C}$ on the inside and $\mathcal{D}$ on the outside. Some $c \in \mathcal{C}$ may be the current state inside the box, but all anyone outside the box can see is the object $Fc \in \mathcal{D}$ that the box chooses to display in lieu of c . Still, any transition g out of Fc can be selected from the outside; the box guarantees that whatever c is on the inside, there is a corresponding transition $F_c^\# g$ out of that c . As g is followed from Fc to $\text{cod } g$ on the outside, $F_c^\# g$ is followed from c to $\text{cod } F_c^\# g$ on the inside. But codomain preservation guarantees that the new state $\text{cod } g$ on the outside is equal to what the box would want to display in lieu of the new state $\text{cod } F_c^\# g$ on the inside, as $\text{cod } g = F \text{ cod } F_c^\# g$. Then the process repeats in a manner compatible with identities and composition.

¹⁰In fact, we will see in Chapter 8 that retrofunctors generalize our dynamical systems.

Here we give a variety of examples of retrofunctors to get a better handle on them. Often we will denote a category by its carrier when its comonoid structure can be inferred from context, and $\mathcal{C}$ will be a category throughout with carrier c .

Retrofunctors to preorders

Given a retrofunctor from $\mathcal{C}$ to a preorder $\mathcal{P}$, we can think of $\mathcal{P}$ as providing a simplified model or abstraction of the states and transitions possible in $\mathcal{C}$, picking canonical transitions in $\mathcal{C}$ along the way to exhibit the model. While the transitions in a general category may be more complex, all that a preorder tells you is whether you can get from one state to another or not. Let's see some examples.

Example 7.63 (Retrofunctors to discrete categories). The discrete category on a set S is the category with objects in S and only identity morphisms; its carrier is Sy . So a retrofunctor $F: \mathcal{C} \rightarrow Sy$ is completely determined by its behavior on objects: to preserve identities, it can only send the morphisms in Sy back to the identity morphisms in $\mathcal{C}$. We can identify F with a function $\text{Ob } \mathcal{C} \rightarrow S$, assigning each state in $\mathcal{C}$ a label in S without revealing anything about the transitions between them.

Exercise 7.64.

1. Show that y has a unique comonoid structure.
2. Show that y with its comonoid structure is terminal in $\mathbf{Cat}^\#$.
3. Explain why y is terminal using the language of states and transitions.

Example 7.65 (Retrofunctors to the walking arrow). Consider a retrofunctor $F: \mathcal{C} \rightarrow \mathcal{A}$, where $\mathcal{A}$ is the walking arrow category

$$\mathcal{A} := \boxed{s \xrightarrow{a} t}$$

from Example 7.34. On objects, F is a function $\text{Ob } \mathcal{C} \rightarrow \{s, t\}$, so

each object of $\mathcal{C}$ lies in either $\mathcal{C}_s := F^{-1}s$ or $\mathcal{C}_t := F^{-1}t$ (but not both). Then on morphisms, preservation of identities determines where $F^\sharp$ sends id_s and id_t , while preservation of codomains ensures that for each $c \in \mathcal{C}_s$, the morphism $F_c^\sharp a: a \rightarrow _$ that $F^\sharp$ sends a back to must satisfy

$$F \text{ cod } F_c^\sharp a = \text{cod } a = t$$

and thus $\text{cod } F_c^\sharp a \in \mathcal{C}_t$. In particular, for every object $c \in \mathcal{C}$ that F sends to s , there must be at least one morphism from c to an object that F sends to t , so that one of those morphisms can be $F_c^\sharp a$. As there are no nontrivial composites in $\mathcal{A}$, the retrofunctor F automatically preserves composites.

In summary, a retrofunctor $F: \mathcal{C} \rightarrow \mathcal{A}$ divides the objects of $\mathcal{C}$ between $\mathcal{C}_s$ and $\mathcal{C}_t$ and fixes a morphism from each object in $\mathcal{C}_s$ to some object in $\mathcal{C}_t$. We can think of F as separating the states of $\mathcal{C}$ into source states and target states, modeled by the s state and the t state in $\mathcal{A}$, respectively; then every source state is assigned a target state and a way of getting to that target state via a transition in $\mathcal{C}$.

Given a retrofunctor $F: \mathcal{C} \rightarrow \mathcal{D}$ and an object $d \in \mathcal{D}$, we will continue to use the notation $\mathcal{C}_d := F^{-1}d$ to denote the set of objects in $\mathcal{C}$ that F sends to d .

Exercise 7.66. Let $F: \mathcal{C} \rightarrow \mathcal{A}$ be a retrofunctor from $\mathcal{C}$ to the walking arrow category $\mathcal{A}$, as in Example 7.65. If $Fc = s$ for all $c \in \mathcal{C}$, what can we say about $\mathcal{C}$?

Exercise 7.67.

1. Recall the star-shaped category $y^{n+1} + ny$ from Exercise 7.37. Describe retrofunctors to it.
2. Describe retrofunctors to the preorder $(\mathbb{N}, \leq)$, viewed as a category: its objects are natural numbers, and there is a morphism $m \rightarrow n$ if and only if $m \leq n$.
3. Describe retrofunctors to the preorder $(\mathbb{N}, \geq)$: its objects are

natural numbers, and there is a morphism $n \rightarrow m$ if and only if $n \geq m$.

Example 7.68 (Retrofunctors to the walking commutative square). Consider a retrofunctor $F: \mathcal{C} \rightarrow \mathcal{CS}$, where $\mathcal{CS}$ is the *walking commutative square category*

$$\mathcal{CS} := \begin{array}{ccc} w & \xrightarrow{h} & y \\ f \downarrow & & \downarrow k \\ x & \xrightarrow{g} & z \\ f \circ g & = & h \circ k \end{array}$$

On objects, F is a function $\text{Ob } \mathcal{C} \rightarrow \{w, x, y, z\}$, so each object of $\mathcal{C}$ lies in exactly one of $\mathcal{C}_w, \mathcal{C}_x, \mathcal{C}_y$, and $\mathcal{C}_z$. Then on morphisms, out of every object $X \in \mathcal{C}_x$ there is a morphism $F_X^\# g: X \rightarrow _$ to an object in $\mathcal{C}_z$, and out of every object $Y \in \mathcal{C}_y$ there is a morphism $F_Y^\# k: Y \rightarrow _$ also to an object in $\mathcal{C}_z$. Finally, out of every object $W \in \mathcal{C}_w$ there is a morphism $F_W^\# f: W \rightarrow X_W$ to an object $X_W \in \mathcal{C}_x$ and a morphism $F_W^\# h: W \rightarrow Y_W$ to an object $Y_W \in \mathcal{C}_y$. As F preserves composites, these must all then satisfy

$$F_W^\# f \circ F_{X_W}^\# g = F^\#(f \circ g) = F^\#(h \circ k) = F_W^\# h \circ F_{Y_W}^\# k;$$

in particular, $F_{X_W}^\# g$ and $F_{Y_W}^\# k$ must share a common codomain $Z_W \in \mathcal{C}_z$, yielding the following commutative square in $\mathcal{C}$:

$$\begin{array}{ccc} W & \xrightarrow{F_W^\# h} & Y_W \\ F_W^\# f \downarrow & & \downarrow F_{Y_W}^\# k \\ X_W & \xrightarrow{F_{X_W}^\# g} & Z_W. \end{array}$$

Exercise 7.69. Let $\mathcal{A}$ denote the walking arrow category, as in Example 7.65, and let $\mathcal{CS}$ denote the walking commutative square category, as in Example 7.68.

1. List the retrofunctors $\mathcal{CS} \rightarrow \mathcal{A}$.
2. List the retrofunctors $\mathcal{A} \rightarrow \mathcal{CS}$.

Exercise 7.70.

1. What does a retrofunctor from y to a poset represent?
2. Consider the chain poset $[n] \cong \sum_{i=0}^n y^{i+1}$. How many retrofunctors are there from $[m] \rightarrow [n]$ for all $m \leq n$?

Retrofunctors to monoids

When a monoid $(M, e, *)$, viewed as a 1-object category y^M , is the codomain of a retrofunctor $\mathcal{C} \rightarrow y^M$, it plays the role of a joystick: an “input device” that “reports its . . . direction to the device it is controlling.”¹¹ Like a joystick, y^M stays in one “place”—a single state—but has a number of directions it can take that are reported back to $\mathcal{C}$, controlling the way it moves through its transitions. As we string together a sequence of directions in M , we chart a course through the transitions of $\mathcal{C}$. We make this analogy concrete in the following examples.

Example 7.71 (Arrow fields). Consider the monoid $(\mathbb{N}, 0, +)$ viewed as a category $y^{\mathbb{N}}$. Retrofunctors $\mathcal{C} \rightarrow y^{\mathbb{N}}$ have been called *admissible sections* [Agu97]. We prefer to call them *arrow fields* (on $\mathcal{C}$), for they turn out to resemble vector fields—but with arrows in $\mathcal{C}$ instead of vectors.¹² We’ll have more to say about these in Theorem 8.61, but our goal here is simply to unpack the definition.

To specify a retrofunctor $A: \mathcal{C} \rightarrow y^{\mathbb{N}}$, we first say what it does on objects, but this is already decided: there is only one object in $y^{\mathbb{N}}$, so every object of $\mathcal{C}$ is sent to it. This also means that codomains are automatically preserved. So as will be the case for all retrofunctors

¹¹Description from Wikipedia.

Every object has been assigned an emanating morphism drawn in solid red, but there need not be any rhyme or reason to our choice.

Exercise 7.73. How many arrow fields on the category $\boxed{\bullet \rightarrow \bullet}$ are there?

We will see later in Proposition 8.62 that the arrow fields on a category form a monoid, and that this operation $\mathbf{Cat}^\# \rightarrow \mathbf{Mon}^{\text{op}}$ is functorial and in fact an adjoint.

Exercise 7.74. Consider the monoid of integers $(\mathbb{Z}, 0, +)$ as a 1-object category $y^{\mathbb{Z}}$, and let $y^{\mathbb{N}}$ be the monoid of natural numbers $(\mathbb{N}, 0, +)$ viewed as a 1-object category as above.

1. Describe the data of a retrofunctor $\mathcal{C} \rightarrowtail y^{\mathbb{Z}}$.
2. What would you say is the canonical retrofunctor $y^{\mathbb{Z}} \rightarrowtail y^{\mathbb{N}}$?

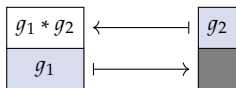
Exercise 7.75.

1. Suppose that M, N are monoids (each is a category with one object). Are retrofunctors between them related to monoid homomorphisms? If so, how?
2. Suppose $\mathcal{C}$ and $\mathcal{D}$ are categories and $F: \mathcal{C} \rightarrowtail \mathcal{D}$ is a retrofunctor. Does there necessarily exist a retrofunctor $\mathcal{C}^{\text{op}} \rightarrowtail \mathcal{D}^{\text{op}}$ that acts the same as F on objects?

Exercise 7.76 (Monoid actions). Recall from Example 7.43 that every monoid action $\alpha: S \times M \rightarrow S$, where S is a set and $(M, e, *)$ is a monoid, gives rise to a category carried by Sy^M . Show that the projection $Sy^M \rightarrow y^M$ is a retrofunctor.

¹²After all, a vector field is a section of a vector bundle. Our arrow fields will be sections of $\mathcal{C}$'s carrier, viewed as a bundle of directions over positions.

Example 7.77. Let $(G, e, *)$ be a group and (y^G, ϵ, δ) the corresponding comonoid. There is a retrofunctor $Gy^G \rightarrow y^G$ given by



To see this is a retrofunctor, we check that identities, codomains, and compositions are preserved. For any g_1 , the identity e is passed back to $g_1 * e = g_1$, and this is the identity on g_1 in Gy^G . Codomains are preserved because there is only one object in y^G . Composites are preserved because for any g_2, g_3 , we have $g_1 * (g_2 * g_3) = (g_1 * g_2) * g_3$.

Exercise 7.78. Does the idea of Example 7.77 work when G is merely a monoid, or does something go subtly wrong somehow?

Proposition 7.79. There is a fully faithful functor $\mathbf{Mon}^{\text{op}} \rightarrow \mathbf{Cat}^\sharp$, whose image consists of all categories whose carriers are representable.

Proof. Given a monoid $(M, e, *)$, we think of it as a category with one object; its carrier y^M is representable. A retrofunctor between such categories carries no data in its on-objects part, and codomains are automatically preserved. Retrofunctors $y^M \rightarrow y^N$ simply carry elements of N to elements of M , preserving identity and composition, exactly the description of monoid homomorphisms. $\square$

Proposition 7.80. There is an adjunction

$$\mathbf{Cat}^\sharp(\mathcal{C}, Ay) \cong \mathbf{Set}(\text{Ob } \mathcal{C}, A)$$

for $\mathcal{C} \in \mathbf{Cat}^\sharp$ and $A \in \mathbf{Set}$.

Proof. In the solution to Exercise 7.36, we saw that a category is discrete iff its carrier is a linear polynomial: this occurs when the only arrow

emanating from each object is its identity. Thus Ay corresponds to a discrete category. A retrofunctor from any category to a discrete category needs to say what happens on objects, but the rest of the data is determined because identities need to be sent back to identities. This is the content of the proposition. $\square$

Exercise 7.81 (Continuous arrow fields). Suppose we say that a *continuous arrow field* on $\mathcal{C}$ is a retrofunctor $\mathcal{C} \rightarrow y^{\mathbb{R}}$, viewing $y^{\mathbb{R}}$ as the monoid of real numbers with addition.

Describe continuous arrow fields in $\mathcal{C}$ using elementary terms, i.e. to someone who doesn't know what a retrofunctor is and isn't yet ready to learn.

Example 7.82 (Systems of ODEs). A system of ordinary differential equations (ODEs) in n variables, e.g.

$$\begin{aligned}\dot{x}_1 &= f_1(x_1, \dots, x_n) \\ \dot{x}_2 &= f_2(x_1, \dots, x_n) \\ &\vdots \\ \dot{x}_n &= f_n(x_1, \dots, x_n),\end{aligned}$$

can be understood as a vector field on $\mathbb{R}^n$. We are often interested in integrating this vector field to get flow lines, or integral curves. In other words, for each $x = (x_1, \dots, x_n) \in \mathbb{R}^n$, viewed as a point, and each $t \in \mathbb{R}$, viewed as a quantity of time, we can begin at x and move along the vector field for time t , arriving at a new point x^{+t} . These satisfy the equations

$$x^{+0} = x \quad \text{and} \quad x^{+t_1+t_2} = (x^{+t_1})^{+t_2}. \quad (7.83)$$

Let's call such things *differentiable dynamical systems* with time domain $(T, 0, +)$; above, we used $T := \mathbb{R}$, but any monoid will do.

Differential dynamical systems as defined above are retrofunctors $F: \mathbb{R}^n y^{\mathbb{R}^n} \rightarrow y^T$. In order to say this, we first need to say how

both $\mathcal{C} := \mathbb{R}^n y^{\mathbb{R}^n}$ and y^T are being considered as categories. The category $\mathcal{C}$ has objects $\mathbb{R}^n$, and for each object $x \in \mathbb{R}^n$ and outgoing arrow $v \in \mathbb{R}^n$, the codomain of v is $x + v$; in other words, v is a vector emanating from x . The identity is $v = 0$, and composition is given by addition. The category y^T is the monoid T considered as a category with one object, $\bullet$.

The retrofunctor assigns to every object $x \in \mathbb{R}^n$ the unique object $F(x) = \bullet$, and to each element $t \in T$ the morphism $F^\sharp(x, t) = x^{+t} - x \in \mathbb{R}^n$, which can be interpreted as a vector emanating from x . Its codomain is $\text{cod } F^\sharp(x, t) = x^{+t}$, and we will see that (7.83) ensures the retrofunctoriality properties.

The codomain law ii is vacuously true, since y^T only has one object. Law i follows because $F^\sharp(x, 0) = x^{+0} - x = 0$, and law iii follows as

$$\begin{aligned} F^\sharp(x^{+t_1}, t_2) + F^\sharp(x, t_1) &= \\ (x^{+t_1})^{+t_2} - x^{+t_1} + x^{+t_1} - x &= \\ x^{+t_1+t_2} - x &= \\ F^\sharp(x, t_1 + t_2). \end{aligned}$$

Retrofunctors from state categories

By now we should be very familiar with lenses from state categories, which are our original dynamical systems. A retrofunctor from a state category, then, is just a dynamical system that satisfies the retrofunctor laws. It turns out that retrofunctors from state categories are particularly noteworthy: just as a polynomial comonoid $\mathcal{C}$ can be identified with a category, a retrofunctor out of $\mathcal{C}$ can be identified with a number of equivalent categorical constructions on $\mathcal{C}$, perhaps the most familiar being a functor $\mathcal{C} \rightarrow \mathbf{Set}$. But these equivalences deserve their own subsection to examine in full; we'll defer them to Section 7.3.3. For now, let's look at some examples of retrofunctors out of state categories.

Example 7.84 (Retrofunctors $Sy^S \rightarrow \mathcal{C}$ are $\mathcal{C}$ -coalgebras). Recall from Example 6.66 that for a set S , lenses $Sy^S \rightarrow p$ correspond to

functions $S \rightarrow p(S)$ known as coalgebras for the functor p . As a retrofunctor $Sy^S \rightarrow \mathcal{C}$ is just a special kind of lens from Sy^S to $\mathcal{C}$, the carrier of $\mathcal{C}$, it should correspond to a special kind of coalgebra $S \rightarrow \mathfrak{c}(S)$ for the functor $\mathfrak{c}$.

Taking $A = B = S \in \mathbf{Set}$ in (6.64), we find that there is a natural isomorphism between dynamical systems $Sy^S \rightarrow p$ and functions $S \rightarrow p(S)$, also known as a *coalgebra for the functor p* or a *p -coalgebra*.¹³

Example 7.85 (Retrofunctors between state categories are very well-behaved lenses). Our familiar state category on S from Example 7.38 is the category with objects in S and exactly 1 morphism between every pair of objects; when we label each morphism with its codomain, its carrier is Sy^S , the identity of $s \in S$ is s , and (disregarding domains) $s \circ s' = s'$ for composable $s, s' \in S$.

Then a retrofunctor $Sy^S \rightarrow Ty^T$ between two state categories corresponds to what is known to functional programmers as a *very well-behaved lens*. We actually defined this way back in Example 3.43, where we called the on-objects (on-positions) function of such a retrofunctor $\text{get}: S \rightarrow T$, and the on-morphisms (on-directions) function $\text{put}: S \times T \rightarrow S$.¹⁴ Then the retrofunctor laws are as follows:

1. Preservation of identities (7.56) becomes

$$\text{put}(s, \text{get}(s)) = s,$$

for all $s \in S$, known as the *get-put law* (named in diagrammatic order: we apply get before we apply put).

2. Preservation of codomains (7.57) becomes

$$\text{get}(\text{put}(s, t)) = t,$$

for all $s \in S$ and $t \in T$, known as the *put-get law*.

¹³There are two versions of coalgebras we are interested in (and more that we are not) with distinct definitions: a *coalgebra for a functor*, which is the version used here, and a *coalgebra for a comonad*, which is a coalgebra for a functor with extra conditions that we will introduce later.

3. Preservation of composition becomes

$$\text{put}(\text{put}(s, t), t') = \text{put}(s, t')$$

for all $s \in S$ and $t, t' \in T$, known as the *put-put law*.

In fact, it turns out that these laws can be satisfied if and only if get is a product projection! For example, if the cardinalities $|S|$ and $|T|$ of S and T are finite and $|S|$ is not divisible by $|T|$, then there are no retrofunctors $Sy^S \rightarrow Ty^T$. A stringent condition, no? We'll explore it in Exercise 7.87 below.

Let's explore why retrofunctors between state categories are just product projections. A product projection $A \times B \rightarrow A$ always has a second factor B ; if every retrofunctor between state categories is a product projection, what is the second factor? It turns out to be

$$U := \{u: T \rightarrow S \mid \forall t, t' \in T, \text{get}(u(t)) = t \text{ and } \text{put}(u(t), t') = u(t')\}.$$

In other words, we will show that if (get, put) defines a retrofunctor $Sy^S \rightarrow Ty^T$, then there is a bijection $S \cong T \times U$ making $\text{get}: S \rightarrow T$ a product projection. We then prove the converse in Exercise 7.86.

Assume $(\text{get}, \text{put}): Sy^S \rightarrow Ty^T$ is a retrofunctor, so that it satisfies the enumerated laws. First, we define a function $\alpha: S \rightarrow T \times U$ as follows. Given $s \in S$, the function $u_s: T \rightarrow S$ defined by

$$u_s(t) = \text{put}(s, t)$$

lies in U : we check that it satisfies

$$\text{get}(u_s(t)) = \text{get}(\text{put}(s, t)) = t$$

by the put-get law for $t \in T$ and

$$\text{put}(u_s(t), t') = \text{put}(\text{put}(s, t), t') = \text{put}(s, t') = u_s(t')$$

by the put-put law for $t, t' \in T$. We can therefore define a function $\alpha: S \rightarrow T \times U$ by

$$\alpha(s) = (\text{get}(s), u_s).$$

In the other direction, we have a function $\beta: T \times U \rightarrow S$ given by

$$\beta(t, u) = u(t).$$

The two functions α and β are mutually inverse: $\alpha \circ \beta: S \rightarrow S$ is the identity because

$$\beta(\alpha(s)) = u_s(\text{get}(s)) = \text{put}(s, \text{get}(s)) = s$$

by the get-put law, while $\beta \circ \alpha: T \times U \rightarrow T \times U$ is the identity because

$$\alpha(\beta(t, u)) = (\text{get}(u(t)), u_{u(t)}) = (t, t' \mapsto \text{put}(u(t), t')) = (t, u),$$

as $\text{get}(u(t)) = t$ and $\text{put}(u(t), t') = u(t')$ by construction for $u \in U$. Thus $S \cong T \times U$, and the product projection $S \xrightarrow{\alpha} T \times U \rightarrow T$ sends $s \mapsto \text{get}(s)$, as desired.

We have therefore shown that for every retrofunctor $Sy^S \rightarrow Ty^T$, there exists a set U for which $S \cong T \times U$ and the on-positions function get is the product projection $S \cong T \times U \rightarrow T$. Notice that the on-directions function put can be uniquely recovered from the bijection $S \cong T \times U$ we constructed: it is determined by the functions $u_s: T \rightarrow S$ for $s \in S$, which in turn is determined by the second projection $T \times U \rightarrow U$.

More precisely, composing $\alpha: S \rightarrow T \times U$ with the projection to U yields a map $S \rightarrow U$ sending $s \mapsto u_s$; then $\text{put}(s, t)$ is given by $u_s(t)$. Of course, a priori U could just be a set—we may not know how to interpret its elements as a functions $T \rightarrow S$. This is where $\beta: T \times U \rightarrow S$ comes in: we know $\beta(t, u_s) = u_s(t)$. So $\text{put}(s, t)$ must be $\beta(t, u_s)$.

Exercise 7.86. Let S, T, U be sets for which we have a bijection $S \cong T \times U$. Show that there exists a unique retrofunctor $Sy^S \rightarrow Ty^T$ whose on-positions function $S \cong T \times U \rightarrow T$ is given by the product projection.

¹⁴More precisely, we are treating the on-morphism functions $T \rightarrow S$ for each $s \in S$ of a retrofunctor $Sy^S \rightarrow Ty^T$ as a single function $S \times T \rightarrow S$.

Exercise 7.87.

1. Suppose $|S| = 3$. How many retrofunctors are there $Sy^S \rightarrow Sy^S$?
2. Suppose $|S| = 4$ and $|T| = 2$. How many retrofunctors are there $Sy^S \rightarrow Ty^T$?

Example 7.88. We have a state category $c(1)y^{c(1)}$ on the set of objects of $\mathcal{C}$. Define a lens $c(1)y^{c(1)} \rightarrow c$ by

$$\begin{array}{ccc} \text{Ob } c & \boxed{\begin{array}{c} \text{cod } f \\ i \end{array}} & \begin{array}{c} \xleftarrow{\text{cod}} \\ \xrightarrow{\quad} \end{array} \begin{array}{c} \boxed{f} \\ i \end{array} \begin{array}{c} c[-] \\ \text{Ob } c \end{array} \\ c(1)y^{c(1)} & & c \end{array}$$

sending each object $i \in \mathcal{C}$ to itself on positions and, at i , sending each morphism $f: i \rightarrow _$ to its codomain $\text{cod } f$ on directions.

This lens is a retrofunctor $c(1)y^{c(1)} \rightarrow c$ because it sends identities back to identities, codomains forward to codomains, and preserves composition (trivially, since each morphism in $c(1)y^{c(1)}$ is determined by its domain and codomain).

Exercise 7.89. Fix an object $i \in c(1) = \text{Ob } c$. Then we have a state category $c[i]y^{c[i]}$ on the set $c[i] = \mathcal{C}[i]$ of morphisms out of i in $\mathcal{C}$. Define a lens $c[i]y^{c[i]} \rightarrow c$ by

$$\begin{array}{ccc} c[i] & \boxed{\begin{array}{c} f \circ g \\ f \end{array}} & \begin{array}{c} \xleftarrow{\circ} \\ \xrightarrow{\text{cod}} \end{array} \begin{array}{c} \boxed{g} \\ \text{cod } f \end{array} \begin{array}{c} c[-] \\ \text{Ob } c \end{array} \\ c[i]y^{c[i]} & & c \end{array}$$

sending each morphism $f: i \rightarrow _$ to its codomain on positions and, at f , sending each morphism $g: \text{cod } f \rightarrow _$ to the composite $f \circ g: i \rightarrow _$ on directions. Is this lens a retrofunctor $c[i]y^{c[i]} \rightarrow c$?

We'll revisit retrofunctors from state categories in Section 7.3.3.

Other retrofunctors

Example 7.90 (Objects aren't representable in $\mathbf{Cat}^\#$). In the world of categories and the usual functors between them, the terminal category $\mathcal{T} := \boxed{\bullet}$ with one object and one morphism *represents objects*, in the sense that functors $\mathcal{T} \rightarrow \mathcal{C}$ naturally correspond to objects in $\mathcal{C}$.

Unfortunately, the same cannot be said for retrofunctors: we'll see in Exercise 7.91 that there does not exist a fixed category $\mathcal{U}$ for which retrofunctors $\mathcal{U} \rightarrow \mathcal{C}$ are in bijection with objects in $\mathcal{C}$ for every category $\mathcal{C}$.

Retrofunctors $\mathcal{T} \rightarrow \mathcal{C}$ are somewhat strange beasts: because they must preserve codomains, they can be identified with objects $c \in \mathcal{C}$ for which the codomain of every emanating morphism $c \rightarrow c'$ is $c' = c$ itself.

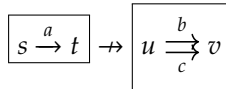
Exercise 7.91. We saw in Exercise 7.27 that $2y$ has a unique comonoid structure.

1. Show that for any category $\mathcal{U}$, retrofunctors $\mathcal{U} \rightarrow 2y$ are in bijection with the set $2^{\text{Ob } \mathcal{U}}$.
2. Use the case of $\mathcal{C} := 2y$ to show that if retrofunctors $\mathcal{U} \rightarrow \mathcal{C}$ are always in bijection with objects in $\mathcal{C}$, then $\mathcal{U}$ must have exactly one object.
3. Now use a different category $\mathcal{D}$ to show that if retrofunctors $\mathcal{U} \rightarrow \mathcal{D}$ are in bijection with objects in $\mathcal{D}$, then $\mathcal{U}$ must have more than one object. Conclude that objects are not representable in $\mathbf{Cat}^\#$ the way they are in $\mathbf{Cat}$.
4. Is there a fixed category $\mathcal{V}$ for which retrofunctors $\mathcal{E} \rightarrow \mathcal{V}$ are in bijection with objects in $\mathcal{E}$ for every category $\mathcal{E}$? If there is, find it; if there isn't, prove there isn't.

Example 7.92. Consider the category $\mathbb{R}y^{\mathbb{R}}$, where the codomain of r emanating from x is $x + r$, identities are 0, and composition is given by addition. What are retrofunctors into $\mathbb{R}y^{\mathbb{R}}$?

Let $\mathcal{C}$ be a category and $|\cdot|: \mathcal{C} \rightarrow \mathbb{R}y^{\mathbb{R}}$ a retrofunctor. It assigns to every object c both a real number $|c| \in \mathbb{R}$ and a choice of emanating morphism $|c|^{\sharp}(r): c \rightarrow c_r$ such that $|c| + r = |c_r|$. This assignment satisfies some laws. Namely we have $c_0 = c$ and, given reals $r, s \in \mathbb{R}$, we have $(c_r)_s = c_{r+s}$.

Exercise 7.93. How many retrofunctors



are there from the walking arrow category $\mathcal{A}$, drawn above on the left, to the walking parallel-arrows category $\mathcal{PA}$, drawn above on the right?

Exercise 7.94.

1. For any category $\mathcal{C}$ with carrier c , find a category with carrier cy .
2. Show that your construction is functorial; i.e. assign each retrofunctor $\mathcal{C} \rightarrow \mathcal{D}$ a retrofunctor $cy \rightarrow dy$ in a way that preserves identities and composites.
3. Is your functor a monad on $\mathbf{Cat}^{\sharp}$, a comonad on $\mathbf{Cat}^{\sharp}$, both, or neither?

Exercise 7.95. Suppose c, d, e are polynomials, each with a comonoid structure, and that $f: c \rightarrow d$ and $g: d \rightarrow e$ are lenses.

1. If f and $f \circ g$ are each retrofunctors, is g automatically a retrofunctor? If so, sketch a proof; if not, sketch a counterexample.
2. If g and $f \circ g$ are each retrofunctors, is f automatically a retrofunctor? If so, sketch a proof; if not, sketch a counterexample.

In the next chapter, we will delve deeper into the categorical structure and properties of $\mathbf{Cat}^{\sharp}$. We'll encounter many more categories and retrofunctors along the way. But first, we'll conclude this chapter

with several alternative characterizations of retrofunctors out of state categories.

7.3.3 Equivalent characterizations of retrofunctors from state categories

Fix a category $\mathcal{C}$ throughout with polynomial carrier $\mathfrak{c}$. How can we view the data of a retrofunctor from a state category $Sy^S \rightarrow \mathcal{C}$? This is actually a very natural categorical concept—we'll see some equivalent ways to express this data below, then state and prove even more equivalences next chapter when we have the machinery to do so.

As coalgebras

Recall from Example 6.66 that for a set S , lenses $Sy^S \rightarrow p$ correspond to functions $S \rightarrow p(S)$ known as coalgebras for the functor p . As a retrofunctor $Sy^S \rightarrow \mathcal{C}$ is just a special kind of lens from Sy^S to $\mathfrak{c}$, it should correspond to a special kind of coalgebra $S \rightarrow \mathfrak{c}(S)$ for the functor $\mathfrak{c}$. Indeed, whenever $\mathfrak{c}$ carries a comonoid $\mathcal{C}$ with respect to the composition product (i.e. a comonad), there is a special notion of a $\mathcal{C}$ -coalgebra (i.e. a coalgebra for the comonad $\mathcal{C}$), as follows.

Definition 7.96 (Coalgebra for a polynomial comonoid). Let $\mathcal{C} = (\mathfrak{c}, \epsilon, \delta)$ be a polynomial comonoid. A $\mathcal{C}$ -coalgebra (S, α) is

- a set S , called the *carrier*, equipped with
- a function $\alpha: S \rightarrow \mathfrak{c} \triangleleft S$,

such that the following diagrams, collectively known as the *coalgebra laws*, commute:

$$\begin{array}{ccc}
 S & \xrightarrow{\alpha} & \mathfrak{c} \triangleleft S \\
 \searrow & & \downarrow \epsilon \triangleleft S \\
 & & S
 \end{array}
 \qquad
 \begin{array}{ccc}
 S & \xrightarrow{\alpha} & \mathfrak{c} \triangleleft S \\
 \downarrow \alpha & & \downarrow \delta \triangleleft S \\
 \mathfrak{c} \triangleleft S & \xrightarrow{\mathfrak{c} \triangleleft \alpha} & \mathfrak{c} \triangleleft \mathfrak{c} \triangleleft S.
 \end{array}
 \tag{7.97}$$

A *morphism* of $\mathcal{C}$ -coalgebras $(S, \alpha) \rightarrow (T, \beta)$ is a function $h: S \rightarrow$

T such that the following diagram commutes:

$$\begin{array}{ccc} S & \xrightarrow{\alpha} & \mathfrak{c} \triangleleft S \\ h \downarrow & & \downarrow \mathfrak{c} \triangleleft h \\ T & \xrightarrow{\beta} & \mathfrak{c} \triangleleft T \end{array}$$

Proposition 7.98. Retrofunctors $Sy^S \rightarrow \mathcal{C}$ can be identified (up to isomorphism) with $\mathcal{C}$ -coalgebras carried by $\mathfrak{c}$.

Proof. Let $\mathfrak{c}$ be the carrier of $\mathcal{C}$. In Example 6.66, we showed that (6.64) gives a natural correspondence between lenses $\Phi: Sy^S \rightarrow \mathfrak{c}$ and functions $\varphi: S \rightarrow \mathfrak{c} \triangleleft S$. We can unravel this correspondence via the proof of Proposition 6.56 as follows. A lens $\Phi: Sy^S \rightarrow \mathfrak{c}$ can be drawn like so (we will adopt our former convention of identifying each morphism $s \rightarrow t$ from Sy^S with its codomain t):

$$\begin{array}{ccc} Sy^S & & \mathfrak{c} \\ \begin{array}{|c|} \hline t \\ \hline s \\ \hline \end{array} & \begin{array}{c} \xleftarrow{\Phi^\#} \\ \xrightarrow{\Phi_1} \end{array} & \begin{array}{|c|} \hline f \\ \hline i \\ \hline \end{array} \end{array}$$

Meanwhile, the corresponding function $\varphi: S \rightarrow \mathfrak{c} \triangleleft S$, equivalently a lens between constants, can be drawn thusly (recall that we fill a box with red dots when it is impossible to fill, i.e. when it can only be filled by an element of the empty set):

$$\begin{array}{ccc} S & & \mathfrak{c} \triangleleft S \\ \begin{array}{|c|} \hline \text{dots} \\ \hline s \\ \hline \end{array} & \begin{array}{c} \xrightarrow{\Phi^\#} \\ \xrightarrow{\Phi_1} \end{array} & \begin{array}{|c|} \hline t \\ \hline f \\ \hline i \\ \hline \end{array} \end{array}$$

Then it suffices to show that Φ satisfies the retrofunctor laws if and only if φ satisfies the coalgebra laws. We can verify this using polyboxes. From (7.53), the eraser preservation law for Φ would state the following

(remember that the arrow in the eraser for the state category Sy^S is just an equality):

$$Sy^S \begin{array}{|c|} \hline s \\ \hline s \\ \hline \end{array} \text{idy} = Sy^S \begin{array}{|c|} \hline \\ \hline s \\ \hline \end{array} \begin{array}{c} \xleftarrow{\Phi^\#} \\ \xrightarrow{\Phi_1} \end{array} \begin{array}{|c|} \hline \\ \hline \\ \hline \end{array} \text{idy}_c$$

Meanwhile, the commutative triangle on the left of (7.97) can be written as follows:

$$s \begin{array}{|c|} \hline \text{dots} \\ \hline s \\ \hline \end{array} = \begin{array}{|c|} \hline \text{dots} \\ \hline s \\ \hline \end{array} s = s \begin{array}{|c|} \hline \text{dots} \\ \hline s \\ \hline \end{array} \begin{array}{c} \xrightarrow{\Phi^\#} \\ \xrightarrow{\Phi_1} \end{array} \begin{array}{|c|} \hline \text{dots} \\ \hline \\ \hline \end{array} \begin{array}{c} s \\ \text{idy} \end{array}_c$$

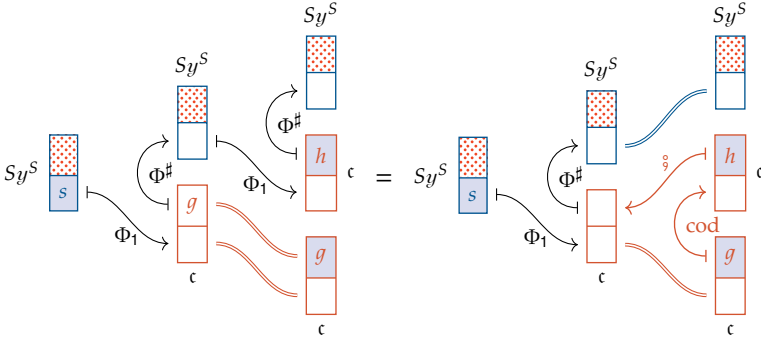
But these polybox equations are entirely equivalent.

Then from (7.54), the duplicator preservation law for Φ would state the following (remember that the three arrows in the duplicator for the state category Sy^S are all equalities)

$$Sy^S \begin{array}{|c|} \hline \\ \hline s \\ \hline \end{array} \begin{array}{c} \text{run} \\ \text{tgt} \end{array} \begin{array}{|c|} \hline \\ \hline s \\ \hline \end{array} \begin{array}{c} \xleftarrow{\Phi^\#} \\ \xrightarrow{\Phi_1} \end{array} \begin{array}{|c|} \hline c \\ \hline h \\ \hline \end{array} \begin{array}{c} \xleftarrow{\Phi^\#} \\ \xrightarrow{\Phi_1} \end{array} \begin{array}{|c|} \hline c \\ \hline g \\ \hline \end{array} = Sy^S \begin{array}{|c|} \hline \\ \hline s \\ \hline \end{array} \begin{array}{c} \xleftarrow{\Phi^\#} \\ \xrightarrow{\Phi_1} \end{array} \begin{array}{|c|} \hline c \\ \hline \\ \hline \end{array} \begin{array}{c} \xrightarrow{\circ} \\ \text{cod} \end{array} \begin{array}{|c|} \hline c \\ \hline h \\ \hline g \\ \hline \end{array}_c$$

Meanwhile, the commutative triangle on the right of (7.97) can be writ-

ten as follows:



But these polybox equations are equivalent as well. Hence the retrofunctor laws for Φ are equivalent to the coalgebra laws for φ . $\square$

So a retrofunctor from a state category to $\mathcal{C}$ bears the same data as a $\mathcal{C}$ -coalgebra. The equivalences don't stop there, however.

As discrete opfibrations

The concept of a $\mathcal{C}$ -coalgebra is in turn equivalent to a better known categorical construction on $\mathcal{C}$, which we introduce here.

Definition 7.99 (Discrete opfibration). Let $\mathcal{C}$ be a category. A pair $(\mathcal{S}, \pi)$, where $\mathcal{S}$ is a category and $\pi: \mathcal{S} \rightarrow \mathcal{C}$ is a functor, is called a *discrete opfibration over $\mathcal{C}$* if it satisfies the following condition:

- for every object $s \in \mathcal{S}$, object $c' \in \mathcal{C}$, and morphism $f: \pi(s) \rightarrow c'$ there exists a unique object $s' \in \mathcal{S}$ and morphism $\bar{f}: s \rightarrow s'$ such that $\pi(\bar{f}) = f$. Note that in this case $\pi(s') = c'$.

$$\begin{array}{ccc} s & \xrightarrow{\bar{f}} & s' \\ \pi \downarrow & & \downarrow \pi \\ \pi(s) & \xrightarrow{f} & - \end{array}$$

A *morphism* of discrete opfibrations $(\mathcal{S}, \pi) \rightarrow (\mathcal{S}', \pi')$ over $\mathcal{C}$ is a functor $F: \mathcal{S} \rightarrow \mathcal{S}'$ making the following triangle commute: We refer to $\bar{f}$ as the *lift* of f to s .

A morphism $(\mathcal{S}, \pi) \rightarrow (\mathcal{S}', \pi')$ between discrete opfibrations over $\mathcal{C}$ is a functor $F: \mathcal{S} \rightarrow \mathcal{S}'$ making the following triangle commute:

$$\begin{array}{ccc} \mathcal{S} & \xrightarrow{F} & \mathcal{S}' \\ \pi \searrow & & \swarrow \pi' \\ & \mathcal{C} & \end{array} \quad (7.100)$$

We denote the category of discrete opfibrations over $\mathcal{C}$ by **dopf**($\mathcal{C}$).

Exercise 7.101. Show that if $F: \mathcal{S} \rightarrow \mathcal{S}'$ is a functor making the triangle (7.100) commute, where both π and π' are discrete opfibrations, then F is also a discrete opfibration.

Exercise 7.102. Suppose $\pi: \mathcal{S} \rightarrow \mathcal{C}$ is a discrete opfibration and $i \in \mathcal{S}$ is an object. With notation as in Definition 7.99, show the following:

1. Show that the lift $\overline{\text{id}_{\pi(i)}} = \text{id}_i$ of the identity on $\pi(i)$ is the identity on i .
2. Show that for $f: \pi(i) \rightarrow c$ and $g: c \rightarrow c'$, we have $\overline{f \circ g} = \overline{f} \circ \overline{g}$.
3. Show how π could instead be interpreted as a retrofunctor.

As it turns out, not only do $\mathcal{C}$ -coalgebras carry the same data as discrete opfibrations over $\mathcal{C}$, they in fact comprise isomorphic categories.

Proposition 7.103. The category of $\mathcal{C}$ -coalgebras is isomorphic to the category **dopf**($\mathcal{C}$) of discrete opfibrations over $\mathcal{C}$.

As copresheaves

It is well-known in the category theory literature that the category of discrete opfibrations over $\mathcal{C}$ is equivalent to yet another familiar category: the category **Set** ^{$\mathcal{C}$} , whose objects are functors $\mathcal{C} \rightarrow \mathbf{Set}$. Such a functor is known as a *copresheaf* on $\mathcal{C}$ for short. These are relevant, e.g. in the theory of categorical databases [Spi12]. Here we review what is

needed to understand this equivalence. We begin by giving a standard construction on any copresheaf.

Definition 7.104 (Category of elements). Given a copresheaf on $\mathcal{C}$, i.e. a functor $I: \mathcal{C} \rightarrow \mathbf{Set}$, its *category of elements* $\int^{\mathcal{C}} I$ is defined to have objects

$$\text{Ob } \int^{\mathcal{C}} I := \{(c, x) \mid c \in \mathcal{C}, x \in Ic\}$$

and a morphism $f: (c, x) \rightarrow (c', x')$ for every morphism $f: c \rightarrow c'$ from $\mathcal{C}$ satisfying

$$(If)(x) = x'.$$

Identities and composites in $\int^{\mathcal{C}} I$ are inherited from $\mathcal{C}$; they obey the usual category laws by the functoriality of I .

The category is so named because its objects are the elements of the sets that the objects of $\mathcal{C}$ are sent to by I . Each morphism $f: c \rightarrow _$ in $\mathcal{C}$ then becomes as many morphisms in $\int^{\mathcal{C}} I$ as there are elements of Ic , tracking where If sends each such element.

The next exercise shows how this construction turns every copresheaf into a discrete opfibration.

Exercise 7.105. Let $I: \mathcal{C} \rightarrow \mathbf{Set}$ be a functor, and let $\int^{\mathcal{C}} I$ be as in Definition 7.104.

1. Show that there is a functor $\pi: \int^{\mathcal{C}} I \rightarrow \mathcal{C}$ sending objects $(c, x) \mapsto c$ and morphisms $f: (c, x) \rightarrow (c', x')$ to $f: c \rightarrow c'$.
2. Show that π is in fact a discrete opfibration.

In fact, the assignment of a discrete opfibration to every copresheaf given above is functorial, as the next exercise shows.

Exercise 7.106. Suppose that $I, J: \mathcal{C} \rightarrow \mathbf{Set}$ are functors and $\alpha: I \rightarrow J$ is a natural transformation.

1. Show that α induces a functor $(\int^{\mathcal{C}} I) \rightarrow (\int^{\mathcal{C}} J)$.
2. Show that it is a morphism of discrete opfibrations in the sense of Definition 7.99.

3. Have you now verified that there is a functor

$$\int^{\mathcal{C}} : \mathbf{Set}^{\mathcal{C}} \rightarrow \mathbf{dopf}(\mathcal{C})$$

or is there something left to do?

Exercise 7.107. Let G be a graph, and let $\mathcal{G}$ be the free category on it. Show that for any functor $S : \mathcal{G} \rightarrow \mathbf{Set}$, the category $\int^{\mathcal{G}} S$ of elements is again free on a graph.

Proposition 7.108. The category $\mathbf{Set}^{\mathcal{C}}$ of copresheaves on $\mathcal{C}$ is equivalent to the category of discrete opfibrations over $\mathcal{C}$.

Proof. By Exercise 7.106 we have a functor $\int^{\mathcal{C}} : \mathbf{Set}^{\mathcal{C}} \rightarrow \mathbf{dopf}(\mathcal{C})$. There is a functor going back: given a discrete opfibration $\pi : \mathcal{S} \rightarrow \mathcal{C}$, we define a functor $\partial\pi : \mathcal{C} \rightarrow \mathbf{Set}$ on objects by sending each $c \in \mathcal{C}$ to the set of objects in $\mathcal{S}$ that π maps to c ; that is,

$$(\partial\pi)(c) := \{s \in \mathcal{S} \mid \pi(s) = c\}.$$

Then on morphisms, for each $f : c \rightarrow _$ in $\mathcal{C}$ and $s \in (\partial\pi)(c)$ we have $\pi(s) = c$, so by Definition 7.99 there exists a unique morphism $\bar{f} : s \rightarrow _$ for which $\pi(\bar{f}) = f$. As $\pi(\text{cod } \bar{f}) = \text{cod } f$, we have $\text{cod } \bar{f} \in (\partial\pi)(\text{cod } f)$, so we can define

$$(\partial\pi)(f)(s) := \text{cod } \bar{f}.$$

On objects, the roundtrip $\mathbf{Set}^{\mathcal{C}} \rightarrow \mathbf{Set}^{\mathcal{C}}$ sends $I : \mathcal{C} \rightarrow \mathbf{Set}$ to the functor

$$\begin{aligned} c &\mapsto \{s \in \int^{\mathcal{C}} I \mid \pi(s) = c\} \\ &= \{(c, x) \mid x \in I(c)\} &= I(c). \end{aligned}$$

The roundtrip $\mathbf{dopf}(\mathcal{C}) \rightarrow \mathbf{dopf}(\mathcal{C})$ sends $\pi : \mathcal{S} \rightarrow \mathcal{C}$ to the discrete opfibration whose object set is $\{(c, s) \in \text{Ob}(\mathcal{C}) \times \text{Ob}(\mathcal{S}) \mid \pi(s) = c\}$ and this set is clearly in bijection with $\text{Ob}(\mathcal{S})$. Proceeding similarly, one defines an isomorphism of categories $\mathcal{S} \cong \int^{\mathcal{C}} \partial\pi$. $\square$

Proposition 7.109. Up to isomorphism, discrete opfibrations into $\mathcal{C}$ can be identified with dynamical systems on $\mathcal{C}$.

Take care to note that this association is only functorial on the groupoid of objects and isomorphisms.

Sketch of proof. Given a discrete opfibration $\pi: \mathcal{S} \rightarrow \mathcal{C}$, take $S := \text{Ob}(\mathcal{S})$ and define a lens $\varphi: Sy^S \rightarrow \mathfrak{c}$ by $\varphi_1 := \pi$ and with $\varphi^\sharp$ given by the lifting: $\varphi_s^\sharp g := \hat{g}$ as in Definition 7.99 for $g \in \mathfrak{c}[\pi s]$. We can check that this lens is a retrofunctor by verifying the conditions in Definition 7.55.

Conversely, given a retrofunctor $(\varphi_1, \varphi^\sharp): Sy^S \rightarrow \mathfrak{c}$, the function φ_1 induces a lens $Sy \rightarrow \mathfrak{c}$, and we can factor it as a vertical followed by a cartesian $Sy \rightarrow \mathfrak{s} \xrightarrow{\psi} \mathfrak{c}$. We can give $\mathfrak{s}$ the structure of a category such that ψ is a retrofunctor. $\square$

Given a dynamical system $Sy^S \rightarrow p$, we extend it to a retrofunctor $\varphi: Sy^S \rightarrow \mathcal{T}_p$. By Propositions 7.108 and 7.109, we can consider it as a discrete opfibration over $\mathcal{T}_p$. By Exercise 7.107 the category $\int \varphi$ is again free on a graph. It is this graph that we usually draw when depicting the dynamical system, e.g. in (4.24).

To summarize, we have four equivalent notions:

- (1) retrofunctors $F: Sy^S \rightarrow \mathcal{C}$;
- (2) $\mathcal{C}$ -coalgebras (S, α) , with $\alpha: S \rightarrow \mathfrak{c} \triangleleft S$;
- (3) discrete opfibrations $\pi: \mathcal{S} \rightarrow \mathcal{C}$, with $\text{Ob } \mathcal{S} = S$;
- (4) copresheaves $I: \mathcal{C} \rightarrow \mathbf{Set}$, with $\text{Ob } \int^{\mathcal{C}} I = S$.

Moreover, (2), (3), and (4) form equivalent categories. Translating between these notions yields different perspectives on familiar categorical concepts. In the next chapter, we will discover even more characterizations of the same data within **Poly** (see Proposition 8.68).

7.4 Summary and further reading

In this chapter we began by showing that for every set S , the thing that makes a dynamical system like $Sy^S \rightarrow p$ actually run is the fact that Sy^S has the structure of a comonoid. We then explained Ahman- and Uustalu's result that comonoids, i.e. polynomials p , equipped with

a pair of lenses $\epsilon: p \rightarrow y$ and $\delta: p \rightarrow p \triangleleft p$, are exactly categories [AU16]. We explained how ϵ picks out an identity for each object and how δ picks out a codomain for each morphism and a composite for each composable pair of morphisms. In particular we showed that the category corresponding to Sy^S is the *contractible groupoid on S* , i.e. the category with S -many objects and a unique morphism between any two.

We then discussed how comonoid morphisms $c \rightarrow d$ are not functors but *retrofunctors*: they map forwards on objects and backwards on morphisms. Retrofunctors were first defined by Marcelo Aguiar [Agu97], though his definition was opposite to ours; he referred to these as *co-functors*. Retrofunctors are the morphisms of a category we notate as $\mathbf{Cat}^\#$. We showed that retrofunctors between state categories $Sy^S \rightarrow Ty^T$ are what are known in the functional programming community as *very well-behaved lenses*. For more on lenses, see [nLa22].

7.5 Exercise solutions

Solution to Exercise 7.2.

Given a polynomial $s \in \mathbf{Poly}$ equipped with a lens $\epsilon: s \rightarrow y$, we know that ϵ picks out a direction at every position of s . So all we can say about s is that there is at least one direction at each of its positions. Equivalently, we could say that s can be written as the product of y and some other polynomial.

Solution to Exercise 7.9.

Following the arrows on either side of (7.8) all the way to the domain's direction box, we obtain an expression for each box's contents that we can then set equal to each other. The easiest way to actually write down these expressions is probably to start at the end with $s_0 \rightarrow s_3$ and follow the arrows backward, unpacking each term until only the contents of the blue boxes remain (namely $s_0, s_0 \rightarrow s_1, s_1 \rightarrow s_2$, and $s_2 \rightarrow s_3$). Here's what we get when we follow this process for the left-hand side of (7.8) (remember where to look for the three inputs to the run function):

$$\begin{aligned} s_0 \rightarrow s_3 &= \text{run}(s_0, s_0 \rightarrow s_2, s_2 \rightarrow s_3) \\ &= \text{run}(s_0, \text{run}(s_0, s_0 \rightarrow s_1, s_1 \rightarrow s_2), s_2 \rightarrow s_3) \\ &= \text{run}(s_0, \text{run}(s_0, s_0 \rightarrow s_1, s_1 \rightarrow s_2), s_2 \rightarrow s_3); \end{aligned}$$

and here is what we get for the right (also remember where to look for the two inputs to the target function):

$$\begin{aligned} s_0 \rightarrow s_3 &= \text{run}(s_0, s_0 \rightarrow s_1, s_1 \rightarrow s_3) \\ &= \text{run}(s_0, s_0 \rightarrow s_1, \text{run}(s_1, s_1 \rightarrow s_2, s_2 \rightarrow s_3)) \\ &= \text{run}(s_0, s_0 \rightarrow s_1, \text{run}(\text{tgt}(s_0, s_0 \rightarrow s_1), s_1 \rightarrow s_2, s_2 \rightarrow s_3)). \end{aligned}$$

Setting these equal yields our desired associativity equation:

$$\begin{aligned} & \text{run}(s_0, \text{run}(s_0, s_0 \rightarrow s_1, s_1 \rightarrow s_2), s_2 \rightarrow s_3) \\ &= \text{run}(s_0, s_0 \rightarrow s_1, \text{run}(\text{tgt}(s_0, s_0 \rightarrow s_1), s_1 \rightarrow s_2, s_2 \rightarrow s_3)). \end{aligned}$$

Solution to Exercise 7.11.

1. Given $s \in \mathbf{Poly}$ and a lens $\delta: s \rightarrow s \triangleleft s$, we want all the ways to obtain a lens $s \rightarrow s^{\triangleleft 4}$ using δ , id_s (i.e. s), $\triangleleft$, and $\circ$. Starting with s , the only way to get to $s^{\triangleleft 2}$ is with a single $\delta: s \rightarrow s^{\triangleleft 2}$. From there, we can get to $s^{\triangleleft 4}$ directly by composing with $\delta \triangleleft \delta$ to obtain $\delta \circ (\delta \triangleleft \delta)$. Alternatively, we can preserve either the first or the second s using the identity, then get to $s^{\triangleleft 3}$ from the other s in one of two ways: either $\delta \circ (\delta \triangleleft s)$ or $\delta \circ (s \triangleleft \delta)$. This gives us 4 more ways to write a lens $s \rightarrow s^{\triangleleft 4}$: either $\delta \circ (s \triangleleft (\delta \circ (\delta \triangleleft s)))$ or $\delta \circ (s \triangleleft (\delta \circ (s \triangleleft \delta)))$ if we chose to preserve the first s , and either $\delta \circ ((\delta \circ (\delta \triangleleft s)) \triangleleft s)$ or $\delta \circ ((\delta \circ (s \triangleleft \delta)) \triangleleft s)$ if we chose to preserve the second. Here's the full list, sorted roughly by how far to the left we try to apply each δ :

- (1) $\delta \circ ((\delta \circ (\delta \triangleleft s)) \triangleleft s)$
- (2) $\delta \circ ((\delta \circ (s \triangleleft \delta)) \triangleleft s)$
- (3) $\delta \circ (\delta \triangleleft \delta)$
- (4) $\delta \circ (s \triangleleft (\delta \circ (\delta \triangleleft s)))$
- (5) $\delta \circ (s \triangleleft (\delta \circ (s \triangleleft \delta)))$

This coincides with the 5 different ways to parenthesize a 4-term expression.

2. We wish to show that if (7.10) commutes, then all the lenses on our list are equal. The commutativity of (7.10) implies that $\delta \circ (\delta \triangleleft s) = \delta \circ (s \triangleleft \delta)$; so (1) and (2) from our list are equal, as are (4) and (5). Meanwhile, since $s = s \circ s$, we can rewrite (1) as

$$\delta \circ ((\delta \circ (\delta \triangleleft s)) \triangleleft (s \circ s)) = \delta \circ (\delta \triangleleft s) \circ (\delta \triangleleft s \triangleleft s),$$

where the associativity of $\triangleleft$ and $\circ$ allows us to drop some parentheses. Then the commutativity of (7.10) allows us to further rewrite this as

$$\begin{aligned} \delta \circ (s \triangleleft \delta) \circ (\delta \triangleleft s \triangleleft s) &= \delta \circ ((s \circ \delta) \triangleleft (\delta \circ (s \triangleleft s))) \\ &= \delta \circ (\delta \triangleleft \delta), \end{aligned}$$

so (1) and (3) are equal. Similarly, we can rewrite (5) as

$$\begin{aligned} \delta \circ ((s \circ s) \triangleleft (\delta \circ (s \triangleleft \delta))) &= \delta \circ (s \triangleleft \delta) \circ (s \triangleleft s \triangleleft \delta) \\ &= \delta \circ (\delta \triangleleft s) \circ (s \triangleleft s \triangleleft \delta) \\ &= \delta \circ ((\delta \circ (s \triangleleft s)) \triangleleft (s \circ \delta)) \\ &= \delta \circ (\delta \triangleleft \delta) \end{aligned}$$

so (3) and (5) are equal. Hence all the lenses on our list are equal.

Solution to Exercise 7.12.

We have $\text{Run}_n(\varphi) = \delta^{(n)} \circ \varphi^{\triangleleft n}$ for all $n \in \mathbb{N}$, as well as $\delta^{(0)} = \epsilon$ and $\delta^{(1)} = \text{id}_s$. Then $\text{Run}_0(\varphi) = \epsilon \circ \varphi^{\triangleleft 0} = \epsilon \circ \text{id}_y = \epsilon$ and $\text{Run}_1(\varphi) = \text{id}_s \circ \varphi^{\triangleleft 1} = \varphi^{\triangleleft 1} = \varphi$.

Solution to Exercise 7.21.

We complete the proof of Proposition 7.20, where we are given a comonoid (c, ϵ, δ) along with $\delta^{(0)} := \epsilon$ and $\delta^{(n+1)} := \delta \circ (\delta^{(n)} \triangleleft c)$ for all $n \in \mathbb{N}$.

1. We will show that $\delta^{(n)}$ is a morphism $c \rightarrow c^{\triangleleft n}$ for every $n \in \mathbb{N}$ by induction on n . We know $\delta^{(0)} = \epsilon$ is a morphism $c \rightarrow y = c^{\triangleleft 0}$, and for each $n \in \mathbb{N}$, if $\delta^{(n)}$ is a morphism $c \rightarrow c^{\triangleleft n}$, then the composite $\delta^{(n+1)} = \delta \circ (\delta^{(n)} \triangleleft c)$ is a morphism

$$c \xrightarrow{\delta} c \triangleleft c \xrightarrow{\delta^{(n)} \triangleleft c} c^{\triangleleft n} \triangleleft c \cong c^{\triangleleft (n+1)}$$

Hence the result follows by induction.

2. We have $\delta^{(1)} = \delta \circ (\delta^{(0)} \triangleleft c) = \delta \circ (\epsilon \triangleleft c) = \text{id}_c$ by the left erasure law from (7.15).
3. By the previous part, we have $\delta^{(2)} = \delta \circ (\delta^{(1)} \triangleleft c) = \delta \circ (c \triangleleft c) = \delta$.

Solution to Exercise 7.26.

We will eventually see that comonoids in **Poly** are categories; this gives us intuition for what is going on here. In fact the category corresponding to the comonoid in this exercise is the walking arrow, depicted in (7.24). The erasure (counit) laws are verified by looking at the picture of δ in (7.25) and noting that it sends the double (do-nothing) lines (the identities) back to the double lines.

Solution to Exercise 7.27.

Given a set B , we wish to give a comonoid structure on By and show that it is unique. There is only one way to define an eraser lens $\epsilon: By \rightarrow y$: the on-position function is the unique map $!: B \rightarrow 1$, while every on-directions function $1 \rightarrow 1$ must be the identity. Meanwhile $By \triangleleft By \cong B^2y$, so to specify a duplicator lens $\delta: By \rightarrow B^2y$, it suffices to specify an on-positions function $\delta_1: B \rightarrow B^2$, and every on-directions function will again be the identity.

All the comonoid laws should then hold trivially on directions, so it suffices to consider each law on positions. The erasure laws imply that the composite functions

$$B \xrightarrow{\delta_1} B \times B \xrightarrow{B \times !} B \times 1 \cong B,$$

where the second map is just the canonical left projection, and

$$B \xrightarrow{\delta_1} B \times B \xrightarrow{! \times B} 1 \times B \cong B,$$

where the second map is just the canonical right projection, are both the identity on B . The only function $\delta_1: B \rightarrow B \times B$ that satisfies this condition is the diagonal $a \mapsto (a, a)$. It is easy to verify that the diagonal is coassociative, so this does define a unique comonoid structure on By . (It turns out that this is equivalent to the well-known result that there is a unique comonoid structure on every set in **(Set, 1, ×)**.)

Solution to Exercise 7.33.

1. The category $\boxed{A \xrightarrow{f} B}$ has 2 morphisms out of A , namely id_A and f ; and 1 morphism out of B , namely id_B . So its carrier is $\{A\}y^{\{\text{id}_A, f\}} + \{B\}y^{\{\text{id}_B\}} \cong y^2 + y$.

2. The category $\boxed{B \xrightarrow{g} A \xleftarrow{h} C}$ has 1 morphism out of A , namely id_A ; 2 morphisms out of B , namely id_B and g ; and 2 morphisms out of C , namely id_C and h . So its carrier is $\{A\}y^{\{\text{id}_A\}} + \{B\}y^{\{\text{id}_B, g\}} + \{C\}y^{\{\text{id}_C, h\}} \cong 2y^2 + y$.
3. The empty category has no objects or morphisms, so its carrier is just 0.
4. The category in question has 1 object, and its set of morphisms is in bijection with $\mathbb{N}$, so its carrier is isomorphic to $y^{\mathbb{N}}$. (This category is the monoid $(\mathbb{N}, 0, +)$ viewed as a 1-object category; see Example 7.40 for the general case.)
5. The category in question has $\mathbb{N}$ as its set of objects, and for each $m \in \mathbb{N}$, the morphisms out of m are determined by their codomains: there is exactly 1 morphism $m \rightarrow n$ for every $n \in \mathbb{N}$ satisfying $m \leq n$, and no other morphisms out of m . So the carrier of the category is isomorphic to

$$\sum_{m \in \mathbb{N}} y^{\{n \in \mathbb{N} \mid m \leq n\}} \cong \mathbb{N}y^{\mathbb{N}},$$

as $\{n \in \mathbb{N} \mid m \leq n\} \cong \mathbb{N}$ under the bijection $n \mapsto n - m$. (This category is the poset $(\mathbb{N}, \leq)$.)

6. The category in question has $\mathbb{N}$ as its set of objects, and for each $n \in \mathbb{N}$, the morphisms out of n are again determined by their codomains: there is exactly 1 morphism $n \rightarrow m$ for every $m \in \mathbb{N}$ satisfying $m \leq n$, and no other morphisms out of n . So the carrier of the category is isomorphic to

$$\sum_{n \in \mathbb{N}} y^{\{m \in \mathbb{N} \mid m \leq n\}} \cong \sum_{n \in \mathbb{N}} y^{n+1} \cong y^1 + y^2 + y^3 + \dots$$

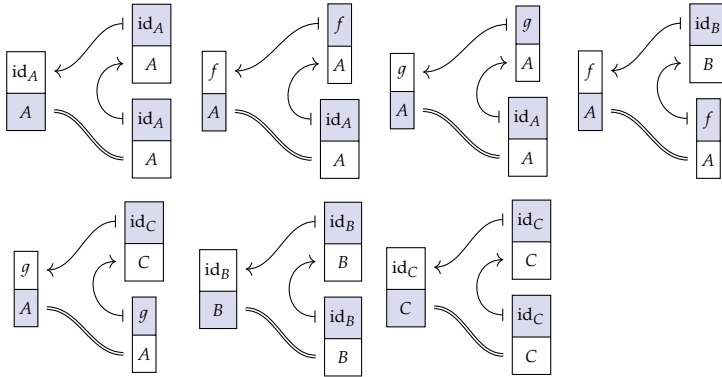
(This category is the poset $(\mathbb{N}, \geq)$.)

Solution to Exercise 7.35.

We are given a comonoid (ϵ, δ) corresponding to the preorder $\boxed{B \xleftarrow{f} A \xrightarrow{g} C}$.

1. There are three morphisms with domain A , namely id_A, f , and g ; the only other morphisms are the identity morphisms on B and C . So the carrier is $\epsilon = \{A\}y^{\{\text{id}_A, f, g\}} + \{B\}y^{\{\text{id}_B\}} + \{C\}y^{\{\text{id}_C\}}$.
2. It suffices to specify the eraser $\epsilon: \epsilon \rightarrow y$ on directions: as always, $\epsilon_i^\#: 1 \rightarrow \epsilon[i]$ picks out id_i for each $i \in \epsilon(1) = \{A, B, C\}$.
3. The duplicator $\delta: \epsilon \rightarrow \epsilon \triangleleft \epsilon$ tells us the codomain of each morphism, as well as how every pair of composable morphisms compose (which in the case of a preorder can be deduced automatically). So we can completely characterize the

behavior of δ using polyboxes as follows:



Solution to Exercise 7.36.

The linear polynomial By corresponds to a category whose objects form the set B and whose only morphisms are identities: in other words, it is the discrete category on B .

Solution to Exercise 7.37.

1. The polynomial $p := y^{n+1} + ny$ has $n + 1$ positions: 1 with $n + 1$ directions and the rest with 1 direction each. So any category carried by p has $n + 1$ objects, of which only 1 has any nonidentity morphisms coming out of it: in fact, it has n nonidentity morphisms coming out of it. But if the category is to be a preorder, each of these n nonidentity morphisms must have a distinct codomain. As there are exactly n other objects, this completely characterizes the category. Equivalently, it is the discrete category on n adjoined with a unique initial object, so that the only nonidentity morphisms are the morphisms out of that initial object to each of the other objects exactly once.
2. This category can be thought of as “star-shaped” if we picture the initial object in the center with morphisms leading out to the other n objects like spokes.

Solution to Exercise 7.39.

In the case of $S := 0$, the only comonoid structure on $Sy^S \cong 0$ is given by the empty category, the only category with no objects; and in the case of $S := 1$, the only comonoid structure on $Sy^S \cong y$ is given by the category with 1 object and no nonidentity morphisms, again the only such category. So in those cases, the comonoid structure on Sy^S is unique.

Now assume $|S| \geq 2$. The state category is always connected, but we can always find a comonoid structure on Sy^S given by a category that is not connected—and thus not isomorphic to the state category—as follows. Consider a category whose object set is S that has no morphisms between distinct objects, so that it is certainly not connected. Then to specify the category, it suffices to specify a monoid associated with each object that will give the morphisms whose domain and codomain are equal to that object.

But there is always a monoid structure on a given nonempty set S . If S is finite, we can take the cyclic group $\mathbb{Z}/|S|\mathbb{Z}$ of order $|S|$, so that the resulting category has carrier $Sy^{\mathbb{Z}/|S|\mathbb{Z}} \cong Sy^S$. On the other hand, if S is infinite, we can take the free monoid on S , which has cardinality $\sum_{n \in \mathbb{N}} |S|^n = |\mathbb{N}||S| = |S|$. So the resulting category will again have carrier Sy^S .

Solution to Exercise 7.41.

The fact that monoids $(M, e, *)$ in $(\mathbf{Set}, 1, \times)$ are just comonoids (y^M, ϵ, δ) in $(\mathbf{Poly}, y, \triangleleft)$, following the construction of Example 7.40, is a direct consequence of the fully faithful Yoneda embedding $\mathbf{Set}^{\text{op}} \rightarrow \mathbf{Poly}$ sending $A \mapsto y^A$ that maps $1 \mapsto y, A \times B \mapsto y^{A \times B} \cong y^A \triangleleft y^B$ naturally, and $M \mapsto y^M$.

We can also state the laws and the correspondences between them explicitly, keeping in mind that e and $*$ are just the on-directions functors of ϵ and δ . The monoid's unitality condition states that e is a 2-sided unit for $*$, or that

$$\begin{array}{ccccc} 1 \times M & \xlongequal{\quad} & M & \xlongequal{\quad} & M \times 1 \\ & \searrow e \times M & \uparrow * & \swarrow M \times e & \\ & & M & & \\ & & \downarrow & & \\ & & M \times M, & & \end{array}$$

commutes—equivalent to the comonoid's erasure laws, which state that

$$\begin{array}{ccccc} y \triangleleft y^M & \xlongequal{\quad} & y^M & \xlongequal{\quad} & y^M \triangleleft y \\ & \swarrow \epsilon \triangleleft y^M & \downarrow \delta & \searrow y^M \triangleleft \epsilon & \\ & & y^M \triangleleft y^M, & & \end{array}$$

commutes (trivial on positions, equivalent to the monoid's unitality condition on directions).

Similarly, the monoid's associativity condition states that $*$ is associative, or that

$$\begin{array}{ccc} M & \xleftarrow{\quad * \quad} & M \times M \\ \uparrow * & & \uparrow M \times * \\ M \times M & \xleftarrow{\quad * \times M \quad} & M \times M \times M, \end{array}$$

commutes—equivalent to the comonoid's coassociative law, which state that

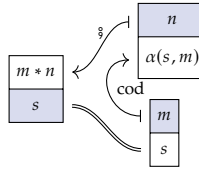
$$\begin{array}{ccc} y^M & \xrightarrow{\quad \delta \quad} & y^M \triangleleft y^M \\ \downarrow \delta & & \downarrow y^M \triangleleft \delta \\ y^M \triangleleft y^M & \xrightarrow{\quad \delta \triangleleft y^M \quad} & y^M \triangleleft y^M \triangleleft y^M, \end{array}$$

commutes (trivial on positions, equivalent to the monoid's associativity condition on directions).

Solution to Exercise 7.44.

Here $(M, e, *)$ is a monoid, S is a set, $\alpha: S \times M \rightarrow S$ is a monoid action, and $\mathcal{M}\mathcal{A}$ is the associated category, whose corresponding comonoid is (Sy^M, ϵ, δ) . We also know that for each $s \in S$ and $m \in M$, there is a morphism $s \xrightarrow{m} \alpha(s, m)$ in $\mathcal{M}\mathcal{A}$.

1. The erasure $\epsilon: Sy^M \rightarrow y$ picks out an element of $m \in M$ for every element $s \in S$ that will play the role of an identity, which in particular should also have s as its codomain. Since we want the codomain of the morphism m out of s to be $\alpha(s, m)$, we can take $m = e$ to guarantee that its codomain will be $\alpha(s, e) = s$. So we let ϵ be the lens whose on-directions function at each $s \in S$ is $\epsilon_s^\# : 1 \rightarrow M$ always maps to e .
2. The duplicator $\delta: Sy^M \rightarrow Sy^M \triangleleft Sy^M$ is determined by what we want the codomain of each morphism to be and how we want the morphisms to compose. We already know that we want the morphism $m \in M$ out of each $s \in S$ to have the codomain $\alpha(s, m)$. If we then have another morphism $n \in M$ out of $\alpha(s, m)$, its codomain will be $\alpha(\alpha(s, m), n) = \alpha(s, m * n)$, the same as the codomain of the morphism $m * n$ out of s . So it makes sense for the composite $s \xrightarrow{m} \alpha(s, m) \xrightarrow{n} \alpha(\alpha(s, m), n)$ to be the morphism $s \xrightarrow{m * n} \alpha(s, m * n)$. Thus, we can define δ in polyboxes as



3. We constructed δ above so that its bottom arrow is an identity function, so verifying the erasure laws amounts to checking that the direction $e \in M$ that ϵ picks out at each position $s \in S$ really does function as an identity morphism $s \xrightarrow{e} \alpha(s, e)$ under the codomain and composition operations specified by δ . We have already ensured that the codomain of e at s is $\alpha(s, e) = s$; meanwhile, given $m \in M$ we have that the composite of $s \xrightarrow{m} \alpha(s, m) \xrightarrow{e} \alpha(\alpha(s, m), e)$ is $m * e = m$ and that the composite of $s \xrightarrow{e} s \xrightarrow{m} \alpha(s, m)$ is $e * m = m$ by the monoid's own unit laws. So the erasure laws hold.
4. Verifying the coassociativity of δ amounts to checking that composition plays nicely with codomains and is associative. We already checked the former when defining δ , and the latter follows from the monoid's own associativity laws: given $m, n, p \in M$, we have $(m * n) * p = m * (n * p)$.
5. The associated category $M\mathcal{A}$ is a category whose objects are the elements of the set S being acted on, and whose morphisms $s \rightarrow t$ for each $s, t \in S$ are the elements of the monoid $m \in M$ that send s to t , i.e. $\alpha(s, m) = t$. The identity morphism on each object is just the unit $e \in M$, while morphisms compose via the multiplication $*$.
6. It is the same iff M is a group, i.e. if every $m \in M$ has an inverse. Indeed, the comonoid structure on My^M from Example 7.38 corresponds to a category in which every map is an isomorphism, so for each $m \in M$, the left-action $\alpha(m, -): M \rightarrow M$ would need to be a bijection, and this is the case iff M is a group.

Solution to Exercise 7.46.

1. Since $\mathbb{R}$ acts on $\mathbb{R}/\mathbb{Z}$ by addition modulo 1, (e.g. $\alpha(.7, 5.4) = .1$), we obtain a comonoid structure on $(\mathbb{R}/\mathbb{Z})y^{\mathbb{R}}$ by Example 7.43. For example, the erasure $(\mathbb{R}/\mathbb{Z})y^{\mathbb{R}} \rightarrow y$ sends everything to $0 \in \mathbb{R}$, because 0 is the identity in $\mathbb{R}$.
2. Yes it is a groupoid because $\mathbb{R}$ is a group: every element is invertible.

Solution to Exercise 7.48.

1. If every object in $\mathcal{C}$ is linear, then the only morphisms in $\mathcal{C}$ are the identity morphisms, so $\mathcal{C}$ must be a discrete category.
2. It is not possible for an object in $\mathcal{C}$ to have degree 0, as every object must have at least an identity morphism emanating from it.
3. Some possible examples of categories with objects of degree $\mathbb{N}$ are the monoid $(\mathbb{N}, 0, +)$ (see Exercise 7.33 #4), the poset $(\mathbb{N}, \leq)$ (see Exercise 7.33 #5), and the state category on $\mathbb{N}$ (see Example 7.38).
4. Up to isomorphism, there are 3 categories with just one linear and one quadratic object. They can be distinguished by the behavior of the single nonidentity morphism. Either its domain and its codomain are distinct, in which case we have the walking arrow category; or its domain and its codomain are the same, in which case it can be composed with itself to obtain either itself or the identity. So there are 3 possible categories in total.
5. Yes: since (isomorphic) categories correspond to (isomorphic) comonoids, there are as many categories with one linear and one quadratic object up to isomorphism as there are comonoid structures on $y^2 + y$.

Solution to Exercise 7.52.

As in Definition 7.49, we have a monoidal category $(\mathcal{C}, y, \triangleleft)$ with comonoids $\mathcal{C} := (\epsilon, \delta)$ and $\mathcal{C}' := (\epsilon', \delta')$ and a comonoid morphism $F: \mathcal{C} \rightarrow \mathcal{C}'$ (really a morphism $F: \epsilon \rightarrow \epsilon'$ in $\mathcal{C}$). Let's throw in another comonoid $\mathcal{C}'' := (\epsilon'', \delta'')$ and another comonoid morphism $G: \mathcal{C}' \rightarrow \mathcal{C}''$ (really a morphism $G: \epsilon' \rightarrow \epsilon''$ in $\mathcal{C}$).

1. To show that the identity morphism $\text{id}_{\epsilon}: \epsilon \rightarrow \epsilon$ is a comonoid morphism $\mathcal{C} \rightarrow \mathcal{C}$, we must check that it preserves erasure by showing that (7.50) commutes, then check that it preserves duplication by showing that (7.50) commutes:

$$\begin{array}{ccc} \epsilon & \xrightarrow{\text{id}_{\epsilon}} & \epsilon \\ \epsilon \downarrow & & \downarrow \epsilon \\ y & \xrightarrow{\quad} & y \end{array} \qquad \begin{array}{ccc} \epsilon & \xrightarrow{\text{id}_{\epsilon}} & \epsilon \\ \delta \downarrow & & \downarrow \delta \\ \epsilon \triangleleft \epsilon & \xrightarrow{\text{id}_{\epsilon} \triangleleft \text{id}_{\epsilon}} & \epsilon \triangleleft \epsilon. \end{array}$$

But they do commute, since id_{ϵ} is the identity on ϵ and $\text{id}_{\epsilon} \triangleleft \text{id}_{\epsilon}$ is the identity on $\epsilon \triangleleft \epsilon$.

2. To show that the composite $F \circ G: \epsilon \rightarrow \epsilon''$ of the two comonoid morphisms F and G is itself a comonoid morphism $\mathcal{C} \rightarrow \mathcal{C}''$, we check that it preserves erasure by showing that (7.50) commutes, then check that it preserves duplication by showing that (7.50) commutes:

$$\begin{array}{ccc} \epsilon & \xrightarrow{F \circ G} & \epsilon'' \\ \epsilon \downarrow & & \downarrow \epsilon'' \\ y & \xrightarrow{\quad} & y \end{array} \qquad \begin{array}{ccc} \epsilon & \xrightarrow{F \circ G} & \epsilon'' \\ \delta \downarrow & & \downarrow \delta'' \\ \epsilon \triangleleft \epsilon & \xrightarrow{(F \circ G) \triangleleft (F \circ G)} & \epsilon'' \triangleleft \epsilon''. \end{array}$$

But we can rewrite these squares like so, using the fact that $(F \circ G) \triangleleft (F \circ G) = (F \triangleleft F) \circ (G \triangleleft G)$ on the right:

$$\begin{array}{ccc} c & \xrightarrow{F} & c & \xrightarrow{G} & c'' \\ \epsilon \downarrow & & \epsilon' \downarrow & & \downarrow \epsilon'' \\ y & = & y & = & y \end{array} \qquad \begin{array}{ccccc} c & \xrightarrow{F} & c' & \xrightarrow{G} & c'' \\ \delta \downarrow & & \delta \downarrow & & \downarrow \delta' \\ c \triangleleft c & \xrightarrow{F \triangleleft F} & c' \triangleleft c' & \xrightarrow{G \triangleleft G} & c'' \triangleleft c'' \end{array}$$

Then the left square in each diagram commutes because F is a comonoid morphism, while the right square in each diagram commutes because G is a comonoid morphism. So both diagrams commute.

Solution to Exercise 7.60.

Here $F: \mathcal{C} \rightarrow \mathcal{D}$ and $G: \mathcal{D} \rightarrow \mathcal{E}$ are retrofunctors in $\mathbf{Cat}^\sharp$.

1. The identity retrofunctor $\text{id}_{\mathcal{D}}$ on $\mathcal{D}$ should correspond to the identity lens on the carrier of $\mathcal{D}$, which is the identity on both positions (objects) and directions (morphisms). So $\text{id}_{\mathcal{D}}$ sends each object $d \in \text{Ob } \mathcal{D}$ to itself, while $(\text{id}_{\mathcal{D}})^\sharp_d: \mathcal{D}[d] \rightarrow \mathcal{D}[d]$ sends each morphism out of d to itself as well.
2. The composite retrofunctor $F \circ G: \mathcal{C} \rightarrow \mathcal{E}$ should correspond to the composite of F as a lens between the carriers of $\mathcal{C}$ and $\mathcal{D}$ with G as a lens between the carriers of $\mathcal{D}$ and $\mathcal{E}$. So on objects, $F \circ G$ sends each $c \in \text{Ob } \mathcal{C}$ to $G(Fc) \in \text{Ob } \mathcal{E}$. Then given $c \in \text{Ob } \mathcal{C}$, the on-morphisms function $(F \circ G)^\sharp_c: \mathcal{E}[G(Fc)] \rightarrow \mathcal{C}[c]$ is the composite of on-directions functions

$$\mathcal{E}[G(Fc)] \xrightarrow{G^\sharp_{Fc}} \mathcal{D}[Fc] \xrightarrow{F^\sharp_c} \mathcal{C}[c],$$

sending each morphism h with domain $G(Fc)$ to $F^\sharp_c \left(G^\sharp_{Fc} h \right)$.

Solution to Exercise 7.62.

We want to show that categories $\mathcal{C}$ and $\mathcal{D}$ are isomorphic in $\mathbf{Cat}$ if and only if they are isomorphic in $\mathbf{Cat}^\sharp$. First, assume that $\mathcal{C}$ and $\mathcal{D}$ are isomorphic in $\mathbf{Cat}$, so that there exist mutually inverse functors $F: \mathcal{C} \rightarrow \mathcal{D}$ and $G: \mathcal{D} \rightarrow \mathcal{C}$. Then we can define a retrofunctor $H: \mathcal{C} \rightarrow \mathcal{D}$ such that for each $c \in \mathcal{C}$ we have $Hc := Fc \in \mathcal{D}$, and for each $g \in \mathcal{D}[Fc]$ we have $H^\sharp_c g := Gg \in \mathcal{C}[GFc] = \mathcal{C}[c]$. We can verify that H really is a retrofunctor: it preserves identities and composition because G does, and it preserves codomains because $H \text{ cod } H^\sharp_c g = F \text{ cod } Gg = FG \text{ cod } g = \text{cod } g$. Analogously, we can define a retrofunctor $K: \mathcal{D} \rightarrow \mathcal{C}$ with $Kd := Gd$ and $K^\sharp_d f := Ff$ for each $d \in \mathcal{D}$ and $f \in \mathcal{C}[Gd]$. Then $H \circ K$ is equal to $F \circ G$ both on objects and on morphisms, so it is the identity retrofunctor on $\mathcal{C}$; analogously, $K \circ H$ is the identity retrofunctor on $\mathcal{D}$. Thus $\mathcal{C}$ and $\mathcal{D}$ are isomorphic in $\mathbf{Cat}^\sharp$.

Conversely, assume $\mathcal{C}$ and $\mathcal{D}$ are isomorphic in $\mathbf{Cat}^\sharp$, so that there exist mutually inverse retrofunctors $H: \mathcal{C} \rightarrow \mathcal{D}$ and $K: \mathcal{D} \rightarrow \mathcal{C}$. Given objects c, c' and a morphism $f: c \rightarrow c'$ in $\mathcal{C}$, we have that $KHc = c$, so $K^\sharp_{Hc}$ is a function $\mathcal{C}[c] \rightarrow \mathcal{D}[Hc]$. In particular, $K^\sharp_{Hc} f$ is a morphism in $\mathcal{D}$ whose domain is Hc and whose codomain satisfies $K \text{ cod } K^\sharp_{Hc} f = \text{cod } f = c'$, and thus $\text{cod } K^\sharp_{Hc} f = Hc'$. Hence we can define a functor $F: \mathcal{C} \rightarrow \mathcal{D}$ such that for each $c \in \mathcal{C}$ we have $Fc := Hc \in \mathcal{D}$, and for each morphism

$f: c \rightarrow c'$ in $\mathcal{C}$ we have $Ff := K_{Hc}^\# f: Hc \rightarrow Hc'$. Functoriality follows from the fact that K preserves identities and composition. Analogously, we can define a functor $G: \mathcal{D} \rightarrow \mathcal{C}$ with $Gd := Kd$ for each $d \in \mathcal{D}$ and $Gg := H_{Kd}^\# g$ for each $g: d \rightarrow d'$ in $\mathcal{D}$. Then $F \circ G$ is equal to $H \circ K$ on objects; on morphisms, it sends each $f: c \rightarrow c'$ in $\mathcal{C}$ to $H_{KHc}^\# (K_{Hc}^\# f) = H_c^\# (K_{Hc}^\# f) = (K \circ H)_c^\# f = f$ itself. So $F \circ G$ is the identity retrofunctor on $\mathcal{C}$. Analogously, $G \circ F$ is the identity functor on $\mathcal{D}$. Thus $\mathcal{C}$ and $\mathcal{D}$ are isomorphic in **Cat**.

Solution to Exercise 7.64.

1. We actually already showed that y has a unique comonoid structure, corresponding to the category with 1 object and no nonidentity morphisms (which we will also denote by y), in Exercise 7.39, for the case of $S := 1$.
2. For any category $\mathcal{C}$, there is a unique retrofunctor $\mathcal{C} \rightarrow y$: it sends every object in $\mathcal{C}$ to the only object in y , and it sends the only morphism in y , an identity morphism, to each identity morphism in $\mathcal{C}$.
3. By Example 7.63, a retrofunctor from a category $\mathcal{C}$ to the discrete category on $S \in \mathbf{Set}$ is a way of assigning each state in $\mathcal{C}$ a label in S . In this case, y is the discrete category on 1, so there is only 1 label to choose from; hence there is always just 1 way to assign the labels.

Solution to Exercise 7.66.

Given a retrofunctor $F: \mathcal{C} \rightarrow \mathcal{A}$, where $\mathcal{A}$ is the walking arrow category as in Example 7.65, assume $Fc = s$ for all $c \in \mathcal{C}$. By Example 7.65, if $Fc = s$ for $c \in \mathcal{C}$, then there must be a morphism from c to an object in $\mathcal{C}$ that F sends to t for $a: s \rightarrow t$ in $\mathcal{A}$ to be sent back to via $F^\#$. But there are no objects in $\mathcal{C}$ that F sends to t . So the only way such a retrofunctor could be defined is if there are no objects in $\mathcal{C}$ that it sends to s , either: we conclude that $\mathcal{C}$ is the empty category.

Solution to Exercise 7.67.

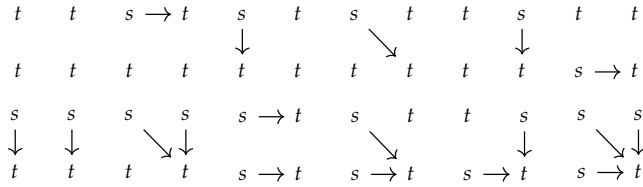
The star-shaped category has $n + 1$ objects, one of which is “central” in the sense that it maps uniquely to every object; the other objects have only identity maps.

1. A retrofunctor $\mathcal{C} \rightarrow y^{n+1} + ny$ comprises an assignment of a label to each object in $\mathcal{C}$: either it assigns “center” or it assigns an element of $\{1, 2, \dots, n\}$. If it assigns “center”, the object is equipped with n -many morphisms, with the i th one having as its codomain an object labeled i .
2. A retrofunctor $\mathcal{C} \rightarrow (\mathbb{N}, \leq)$ comprises an assignment of a natural number label to each object in $\mathcal{C}$, as well as a choice of morphism in $\mathcal{C}$ from each object labeled n to some object labeled $n + 1$.
3. A retrofunctor $\mathcal{C} \rightarrow (\mathbb{N}, \leq)$ comprises an assignment of a natural number label to each object in $\mathcal{C}$, as well as a choice of morphism in $\mathcal{C}$ from each object labeled $n + 1$ to some object labeled n .

Solution to Exercise 7.69.

1. Every object $c \in \mathcal{CS}$ in the commutative square must be labeled s or t . If c is labeled t , there are no further restrictions, since the only arrow emanating from

t is the identity. If c is labeled s , then we must choose an outgoing arrow from c to an object labeled t . So the possible retrofunctors are



2. Every object $a \in \mathcal{A}$ in the walking arrow must be labeled w, x, y , or z . If a is labeled z , there are no further restrictions. If it is labeled x or y , then a must have a map to an element labeled z ; in particular this implies that a must be the source object $s \in \mathcal{A}$. Finally, a cannot be labeled w because then a would need too many outgoing arrows. So the possible retrofunctors are

$$z \quad z \quad x \longrightarrow z \quad y \longrightarrow z$$

Solution to Exercise 7.70.

- Let $\mathcal{P}$ be a poset. A retrofunctor $y \rightarrow \mathcal{P}$ represents an maximal element $p \in \mathcal{P}$. Indeed, if there were some $p \leq p'$ with $p' \neq p$, then the retrofunctor would have to send the map $p \rightarrow p'$ to some morphism in y whose codomain is sent to p' ; this is impossible. But if p is maximal, then there is no obstruction to sending the unique object of y to p .
- Let $m = \boxed{\bullet^0 \rightarrow \dots \rightarrow \bullet^{m-2} \rightarrow \bullet^{m-1}}$. By the same reasoning as above, a map $[m] \rightarrow [n]$ must send the final object of $[m]$ to the final object of $[n]$. It can send the penultimate object $m-2$ to either the penultimate object $n-2$ or to the final object $n-1$ in $[n]$. Repeating in this way, we see that there are 2^m many retrofunctors. For example, the retrofunctors $[2] \rightarrow [2]$ are those labeled by $(2, 2, 2)$, $(1, 2, 2)$, $(1, 1, 2)$, and $(0, 1, 2)$.

Solution to Exercise 7.73.

We seek the number of arrow fields on the category $\boxed{\bullet \rightarrow \bullet}$. There are 2 choices of morphisms emanating from the object on the left, and 1 choice of morphism emanating from the object on the right, for a total of $2 \cdot 1 = 2$ arrow fields.

Solution to Exercise 7.74.

- A retrofunctor $\mathcal{C} \rightarrow y^{\mathbb{Z}}$ assigns to each object $c \in \mathcal{C}$ and integer $n \in \mathbb{Z}$ an emanating arrow $c.n \in \mathcal{C}$ to some other object, with the property that $c.0 = c$ and $c.n.n' = c.(n+n')$. But this is overkill. Indeed, it is enough to assign the $c.1$ arrow and to check that for every object c' there exists a unique object c with $\text{cod}(c.1) = c'$.
- We seek a canonical retrofunctor $y^{\mathbb{Z}} \rightarrow y^{\mathbb{N}}$. The canonical inclusion $i: \mathbb{N} \hookrightarrow \mathbb{Z}$ gives rise to a lens ι from $y^{\mathbb{Z}}$ to $y^{\mathbb{N}}$, whose sole on-directions function $\iota^\# : \mathbb{N} \hookrightarrow \mathbb{Z}$ coincides with i . We verify that ι is a retrofunctor: it preserves identities, as $\iota^\# 0 = 0$; it automatically preserves codomains; and it preserves composites, given by addition in either monoid, as $\iota^\#(m+n) = m+n = \iota^\#(m) + \iota^\#(n)$.

Solution to Exercise 7.75.

1. Retrofunctors $y^M \rightarrow y^N$ are the same as monoid homomorphisms $N \rightarrow M$. See Proposition 7.79.
2. No! This is the weirdest thing about retrofunctors. For example, there is a unique retrofunctor $y \rightarrow y^2 + y$ from the walking object to the walking arrow, so if we reverse the arrows, there is no longer a retrofunctor that acts the same on objects.

Solution to Exercise 7.76.

We are given a monoid $(M, e, *)$, a set S , and an M -action $\alpha: M \times S \rightarrow S$. The category Sy^M has e as the identity on each $s \in S$; the codomain of the map labeled m emanating from s is $\alpha(s, m)$, and composition is given by $*$. The projection $Sy^M \rightarrow y^M$ sends the identity back to the identity, trivially preserves codomains, and also preserves composition, so it is a retrofunctor.

Solution to Exercise 7.78.

This works!

Solution to Exercise 7.81.

A continuous arrow field on $\mathcal{C}$ assigns to each object $c \in \mathcal{C}$ and each real number $r \in \mathbb{R}$ a morphism $c.r$ emanating from c . These have the property that $c.0$ is the identity on c and that $(c.r).r' = c.(r + r')$. In other words, you can evolve c forward or backward in time by any $r \in \mathbb{R}$, and this works as expected.

Solution to Exercise 7.86.

Given a bijection of sets $S \cong T \times U$, we seek a unique retrofunctor $Sy^S \rightarrow Ty^T$ whose on-positions function get is given by the product projection $S \cong T \times U \rightarrow T$. Such a retrofunctor should also have an on-directions function $\text{put}: S \times T \rightarrow S$ such that the three laws from Example 7.85 are satisfied. With $S \cong T \times U$, such a function is uniquely determined by its components

$$S \times T \xrightarrow{\text{put}} S \xrightarrow{\text{get}} T \quad \text{and} \quad S \times T \xrightarrow{\text{put}} S \xrightarrow{\pi} U,$$

where $\pi: S \cong T \times U \rightarrow U$ is the other product projection. The left component is determined by the put-get law, which specifies the behavior of the composite $\text{put} \circ \text{get}$: it should send $(s, t) \mapsto t$. Identifying S with $T \times U$, the get-put law for $s = (t, u) \in T \times U$ (so $\text{get}(t, u) = t$) reads as

$$\text{put}((t, u), t) = (t, u),$$

while the put-put law reads as

$$\text{put}(\text{put}((t, u), t'), t'') = \text{put}((t, u), t'')$$

for $t', t'' \in T$. Applying get to both sides, we observe that the first coordinates (in T) of either side of each equation are automatically equal when the put-get law holds. So we are really only concerned with the second coordinates of either side (in U); applying π to both sides yields

$$\pi(\text{put}((t, u), t)) = u \quad \text{and} \quad \pi(\text{put}(\text{put}((t, u), t'), t'')) = \pi(\text{put}((t, u), t''))$$

Solution to Exercise 7.87.

1. There are 6 product projections $3 \rightarrow 3$, namely the three automorphisms, so the answer is 6.
2. There are 6 product projections $4 \rightarrow 2$, so the answer is 6.

Solution to Exercise 7.89.

Yes, it is a retrofunctor. One sees easily that it sends identities back to identities and composites back to composites. The codomain of $f \circ g$ is simply $f \circ g$ as an element of $\mathcal{C}[i]$, and it is sent forward to $\text{cod}(f \circ g)$, so the map preserves codomains.

Solution to Exercise 7.91.

1. Every object in $\mathcal{U}$ must be labeled with either 1 or 2, but there are no other requirements.
2. If retrofunctors $\mathcal{U} \rightarrow \mathcal{C}$ are always in bijection with objects in $\mathcal{C}$, then with $\mathcal{C} = 2y$, we have a bijection $2 \cong 2^{\text{Ob}(\mathcal{U})}$ by part 1, so $\mathcal{U}$ has one object.
3. Take $\mathcal{D}$ to be the walking arrow. Then if $\mathcal{U}$ has only one object, it cannot be sent to the source object of $\mathcal{D}$ because the emanating morphism would have nowhere to go. Hence the unique object of $\mathcal{U}$ must be sent to the target object of $\mathcal{D}$. But there is only one such retrofunctor, whereas there are two objects in $\mathcal{D}$. We conclude that objects are not representable in $\mathbf{Cat}^\#$ the way they are in $\mathbf{Cat}$.
4. No, there is no such $\mathcal{V}$. There is exactly one retrofunctor $0 \rightarrow \mathcal{V}$, but there are no objects of 0.

Solution to Exercise 7.93.

There are two: one of which sends $s \mapsto u$ and $t \mapsto v$ and the other of which sends both $s, t \mapsto v$.

Solution to Exercise 7.94.

1. Take the category $\mathcal{C}'$ that has the same objects as $\mathcal{C}$ and almost the same arrows, except that it has one more arrow i_c from each object $c \in \mathcal{C}'$ to itself. This new arrow is the identity. The composites of all the old arrows in $\mathcal{C}'$ are exactly as they are in $\mathcal{C}$. The old identity id_c^{old} is no longer an identity because $\text{id}_c^{\text{old}} \circ i = \text{id}_c^{\text{old}} \neq i$.
2. Given a retrofunctor $\varphi: \mathcal{C} \rightarrow \mathcal{D}$, we get a retrofunctor $\varphi y \rightarrow \mathcal{D} y$ that acts the same on objects and all the old arrows and sends the new identities back to the new identities. It preserves identities and composites going backward and codomains going forward, so it's a retrofunctor.
3. It is a monad: there is an obvious unit map $\mathcal{C} \rightarrow \varphi y$ and a multiplication map $\varphi y y \rightarrow \varphi y$ that sends the new identity back to the newest identity.

Solution to Exercise 7.95.

1. Counterexample: take $\mathcal{C} = 0$. Then the unique lens $f: \mathcal{C} \rightarrow \mathcal{D}$ is a retrofunctor and so is $f \circ g$ for any lens $g: \mathcal{D} \rightarrow \mathcal{E}$, but some lenses are not retrofunctors.
2. Counterexample: take $\mathcal{C} = y$. Then there is a unique retrofunctor $\mathcal{E}: \mathcal{C} \rightarrow y$. As long as f is copointed, meaning it sends identities backward to identities, then $f \circ g$ will be a retrofunctor. But not every copointed lens is a retrofunctor.

Solution to Exercise 7.101.

In general, a functor $F: \mathcal{C} \rightarrow \mathcal{D}$ is a discrete opfibration iff, for every object $c \in \mathcal{C}$, the induced map $F[c]: \mathcal{C}[c] \rightarrow \mathcal{D}[Fc]$ is a bijection.

In our case, if π and π' are discrete opfibrations then for any $s \in \mathcal{S}$ we have that both the second map and the composite in $\mathcal{S}[s] \xrightarrow{F[s]} \mathcal{S}'[Fs] \xrightarrow{\pi'[Fs]} \mathcal{C}[\pi s]$ are bijections, so the first map is too.

Solution to Exercise 7.102.

1. The identity map id_i satisfies $\pi(\text{id}_i) = \text{id}_{\pi(i)}$, so $\overline{\text{id}_{\pi(i)}} = \text{id}_i$ by the uniqueness-of-lift condition in Definition 7.99.
2. The morphism $\bar{f} \circ \bar{g}$ satisfies $\pi(\bar{f} \circ \bar{g}) = \pi(\bar{f}) \circ \pi(\bar{g}) = f \circ g$, so again this follows by uniqueness of lift.
3. To see that π is a retrofunctor, we need to understand and verify conditions about its action forward on objects and backward on morphisms. Its action forward on objects is that of π as a functor. Given an object $s \in \mathcal{S}$, the action of π backward on morphisms is the lift operation $\pi_s^\#(f) := \bar{f}$. This preserves identity by part 1, composition by part 2, and codomains because $\pi(\text{cod}(\pi_s^\#(f))) = \text{cod}(\pi(\pi_s^\#(f))) = \text{cod}(f)$.

Solution to Exercise 7.105.

1. The functor action on objects and morphisms is defined in the exercise statement, so it suffices to check that it preserves identities and composites. But this too is obvious: $\pi(\text{id}) = \text{id}$ and $\pi(f \circ g) = \pi(f) \circ \pi(g)$.
2. Given an object $(c, x) \in \int^{\mathcal{C}} I$ and $f: c \rightarrow c'$, we need to check that there is a unique object $s \in \int^{\mathcal{C}} I$ and morphism $\bar{f}: (c, x) \rightarrow s$ with $\pi(\bar{f}) = f$. Using $x' := I(f)(x)$ and $s := (c', x')$ and $\bar{f} := f$, we do indeed get $\pi(\bar{f}) = f$ and we find that it is the only possible choice.

Solution to Exercise 7.106.

1. Given a natural transformation $\alpha: I \rightarrow J$, we need a functor $\int^{\mathcal{C}} \alpha: \int^{\mathcal{C}} I \rightarrow \int^{\mathcal{C}} J$. On objects have it send $(c, x) \mapsto (c, \alpha_c x)$, where $x \in I(c)$ so $\alpha_c x \in J(c)$; on morphisms have it send $f \mapsto f$, a mapping which clearly preserves identities and composition.
2. To see that $\int^{\mathcal{C}} \alpha$ is a morphism between discrete opfibrations, one only needs to check that it commutes with the projections to $\mathcal{C}$, but this is obvious: the mapping $(c, x) \mapsto (c, \alpha_c x)$ and the mapping $f \mapsto f$ preserve the objects and morphisms of $\mathcal{C}$.
3. It is clear that $\int^{\mathcal{C}}$ preserves identities and composition in $\mathbf{Set}^{\mathcal{C}}$, so we have verified that this is functor.

Solution to Exercise 7.107.

Given a graph $G := (\text{src}, \text{tgt}: A \rightrightarrows V)$ and functor $S: \mathcal{G} \rightarrow \mathbf{Set}$, define a new graph H as the following diagram of sets:

$$\sum_{a \in A} S(\text{src}(a)) \xrightarrow[\text{(tgt, } S(a))]{\text{(src, id)}} \sum_{v \in V} S(v)$$

In other words, it is a graph for which a vertex is a pair (v, s) with $v \in V$ a G -vertex and $s \in S(v)$, and for which an arrow is a pair (a, s) with $a \in A$ a G -arrow, say $a: v \rightarrow v'$, and $s \in S(v)$ is an element over the source of that arrow. With this notation, the arrow (a, s) has as source vertex (v, s) and has as target vertex $(v', S(a)(s))$ where $S(a): S(v) \rightarrow S(v')$ is the function given by the functor $S: \mathcal{G} \rightarrow \mathbf{Set}$.

It remains to see that the free category on H is isomorphic to $\int^{\mathcal{G}} S$. We first note that it has the same set of objects, namely $\sum_{v \in V} S(v)$. A morphism in $\int^{\mathcal{G}} S$ can be identified with an object (v, s) together with a morphism $f: v \rightarrow v'$ in $\mathcal{G}$, but this is just a length- n sequence $(a_1, \dots, a_n)$ of arrows, with $v = \text{src}(a_1)$, $\text{tgt}(a_i) = \text{src}(a_{i+1})$ for $1 \leq i < n$, and $\text{tgt}(a_n) = v'$. This is the same data as a morphism in the free category on H .

8. Categorical properties of polynomial comonoids

While we defined the category $\mathbf{Cat}^\sharp$ of categories and retrofunctors in the last chapter, we have only begun to scratch the surface of the properties it satisfies. As the category of comonoids in $\mathbf{Poly}$, equipped with a canonical forgetful functor $U: \mathbf{Cat}^\sharp \rightarrow \mathbf{Poly}$ sending each polynomial to its carrier, $\mathbf{Cat}^\sharp$ inherits many of the categorical properties satisfied by $\mathbf{Poly}$. Underpinning this inheritance is the fact that U has a right adjoint, a cofree functor $\mathbf{Poly} \rightarrow \mathbf{Cat}^\sharp$. We will introduce this adjunction in the first section of this chapter. Then we will discuss how many of the categorical properties of $\mathbf{Poly}$, including much of what we covered in Chapter 5, play nicely with restriction to $\mathbf{Cat}^\sharp$. Finally, we will touch on other constructions we can make over the comonoids in $\mathbf{Poly}$, such as their comodules and coalgebras.

8.1 Cofree comonoids

Consider a dynamical system $\varphi: s \rightarrow p$ with state system s and interface p . In Section 7.1.5, we posed the question of whether there was a single morphism that could capture all the information encoded by the family of lenses $\text{Run}_n(\varphi): s \rightarrow p^{\triangleleft n}$ for all $n \in \mathbb{N}$, defined as the composite

$$s \xrightarrow{\delta^{(n)}} s^{\triangleleft n} \xrightarrow{\varphi^{\triangleleft n}} p^{\triangleleft n}$$

that models n runs through the system φ .

It turns out that there is: the key is that there is a natural way to interpret every polynomial p as a category $\mathcal{T}_p$, so that retrofunctors into $\mathcal{T}_p$ are exactly lenses into p . In other words, $\mathcal{T}_p$ will turn out to be the *cofree comonoid* (or *cofree category*) on p . Cofree comonoids in $\mathbf{Poly}$ are beautiful objects, both in their visualizable structure as a category and in the metaphors we can make about them. They allow us to replace

the interface of a dynamical system with a category and get access to a rich theory that exists there.

We'll go through the construction of the cofree comonoid and its implications in this section, featuring a purely formal proof of the fact the forgetful functor $U: \mathbf{Cat}^\# \rightarrow \mathbf{Poly}$ has a right adjoint $\mathcal{T}_-: \mathbf{Poly} \rightarrow \mathbf{Cat}^\#$, where for each $p \in \mathbf{Poly}$, the carrier $t_p := U \mathcal{T}_p$ of the category $\mathcal{T}_p$ is given by the limit of the following diagram:

$$\begin{array}{ccccccc}
 y & & p & & p^{\triangleleft 2} & & p^{\triangleleft 3} & & \dots \\
 \downarrow & & \downarrow & & \downarrow & & \downarrow & & \\
 1 & \xleftarrow{!} & p \triangleleft 1 & \xleftarrow{p \triangleleft !} & p^{\triangleleft 2} \triangleleft 1 & \xleftarrow{p^{\triangleleft 2} \triangleleft !} & p^{\triangleleft 3} \triangleleft 1 & \xleftarrow{\quad} & \dots
 \end{array} \tag{8.1}$$

Thus, we will show in Theorem 8.45 that U and $\mathcal{T}_-$ form a forgetful-cofree adjunction, making $\mathcal{T}_p$ the cofree comonoid on p . But first, let us concretely characterize the canonical comonoid structure on the limit of (8.1), before showing that it is indeed cofree.

8.1.1 The carrier of the cofree comonoid

Let t_p be the limit of the diagram (8.1) in **Poly**; it will turn out to be the carrier of the cofree comonoid on p (where p will be an arbitrary polynomial throughout). We could compute this limit directly, but we will be able to describe it more concretely in terms of what we call *trees on p* or *p -trees*: trees comprised of p -corollas. In doing so, we will formalize the tree pictures we have been using to describe polynomials all along.

Trees on polynomials

Definition 8.2 (Tree on a polynomial). Let $p \in \mathbf{Poly}$ be a polynomial. A *tree on p* , or a *p -tree*, is a rooted tree whose every vertex v is assigned a p -position i and a bijection from the children of v to $p[i]$. We denote the set of p -trees by $\mathbf{tree}_p$.

We can think of a p -tree as being “built” out of p -corollas according to these instructions:

To choose a p -tree in tree_p :

1. choose a p -corolla:
 - its root $i_0 \in p(1)$ will be the tree's root, and
 - its leaves in $p[i_0]$ will be the edges out of the root;
2. for each $p[i_0]$ -leaf a_1 :
 - 2.1. choose a p -corolla:
 - its root $i_1 \in p(1)$ will be the vertex adjoined to a_1 , and
 - its leaves in $p[i_1]$ will be the edges out of that vertex;
 - 2.2. for each $p[i_1]$ -leaf a_2 :
 - 2.2.1. choose a p -corolla:
 - its root $i_2 \in p(1)$ will be the vertex adjoined to a_2 , and
 - its leaves in $p[i_2]$ will be the edges out of that vertex;
 - 2.2.2. for each $p[i_2]$ -leaf a_3 :
 - ...

Of course, there may eventually be multiple copies of any one p -corolla as a subgraph in our p -tree, and these copies of the same p -corolla should be treated as distinct. So we should really think of the positions and directions of p involved each step as *labels* for the vertices and edges of a p -tree—although crucially, each $p[i]$ -direction must be used as a label exactly once among the edges emanating from a given vertex labeled with i .

Although these instructions continue forever, we could abbreviate them by writing them recursively:

To choose a p -tree in tree_p :

1. choose a p -corolla:
 - its root $i_0 \in p(1)$ will be (the label of) the tree's root, and
 - its leaves in $p[i_0]$ will be (the labels of) the edges out of the root;
2. for each $p[i_0]$ -leaf a_1 :
 - 2.1. choose a p -tree in tree_p :

- it will be the subtree whose root is adjoined to a_1 .

We would like to draw some examples of p -trees, but note that a p -tree can have infinite height—in fact, it always will unless every one of its branches terminates at a p -corolla with no leaves, i.e. a position with an empty direction set. This means that plenty of p -trees cannot be drawn, even when all the position- and direction-sets of p are finite; but we can instead consider finite-height portions of them that we will call *pretrees*.

Pretrees on polynomials

Before we define pretrees, let us give some examples.

Example 8.3 (A few example p -pretrees). Let $p := \{\blacklozenge, \blacksquare\}y^2 + \{\bullet\}y + \{\blacktriangle\}$. Here are four partially constructed p -trees:



Here only the third one—the single yellow triangle $\blacktriangle$ —would count as an element of tree_p . After all, in Definition 8.2, when we speak of a tree on p , we mean a tree for which every vertex is a position in p with all of its emanating directions filled by another position in p . Since three of the four trees shown in (8.4) have leaves emanating from the top that have not been filled by any p -corollas, these trees are not elements of tree_p .

However, each of these trees can be extended to an actual element of tree_p by continually filling in each open leaf with another p -corolla. These may continue forever, or they may terminate if every open leaf is filled with a direction-less yellow triangle.

Exercise 8.5. Let $q := y^2 + 3y^1$. Are there any finite q -trees? If

not, could there be any vertices of a given q -tree with finitely many descendents?

The trees in (8.4) can all be obtained by following just the first 3 levels of instructions for building a p -tree (in fact, exactly as many instructions as we initially wrote out). On the other hand, we know from Section 6.1.3 that such a tree represents a position of $p^{\triangleleft 3}$, whose directions are its height-3 leaves—and this is true for any $n \in \mathbb{N}$ in place of 3. So these trees are still important to our theory; but since they are not always complete p -trees, we will call them something else.

Definition 8.6. Given $p \in \mathbf{Poly}$, a *stage- n pretree* (or $p^{\triangleleft n}$ -pretree) is defined to be an element of $p^{\triangleleft n}(1)$. For each $i \in p^{\triangleleft n}(1)$ the *height- n leaves* of i is defined to be the set $p^{\triangleleft n}[i]$.

Remark 8.7. In Definition 8.6, we use *stage* instead of *height* to allow for the fact that a $p^{\triangleleft n}$ -pretree may not reach its maximum height n if all of its branches terminate early. For example, the yellow dot in (8.4) is a $p^{\triangleleft 1}$ -pretree, but it is also a $p^{\triangleleft 2}$ -pretree, a $p^{\triangleleft 50}$ -pretree, and indeed a p -tree.

Note that for any polynomial $p \in \mathbf{Poly}$ there is exactly one stage-0 pretree on p for any $p \in \mathbf{Poly}$, because $p^{\triangleleft 0}(1) = y(1) = 1$.

Example 8.8 (Trimming pretrees). Since $p^{\triangleleft 1}(1) \cong p(1)$ and $p^{\triangleleft 0}(1) \cong y(1) \cong 1$, the unique function $! : p(1) \rightarrow 1$ can be thought of as a function from $p^{\triangleleft 1}$ -positions to $p^{\triangleleft 0}$ -positions, or equivalently a function from stage-1 pretrees (i.e. corollas) to stage-0 pretrees on p . We can interpret this function as taking a corolla and “stripping away” its leaves along with the position-label on its root, leaving only a single unlabeled root: a stage-0 pretree.

This deceptively simple function has a surprising amount of utility when combined with other maps. For any $n \in \mathbb{N}$, we can take the composition product in $\mathbf{Poly}$ of the identity on $p^{\triangleleft n}$ and $!$, interpreted as a lens between constant polynomials, to obtain a lens $p^{\triangleleft n} \triangleleft ! : p^{\triangleleft n} \triangleleft p(1) \rightarrow p^{\triangleleft n} \triangleleft 1$, or equivalently a function $p^{\triangleleft n}(!) : p^{\triangleleft n+1}(1) \rightarrow p^{\triangleleft n}(1)$ from stage- $(n+1)$ pretrees to stage- n

pretrees on p .

We can deduce the behavior of this function on stage- $(n + 1)$ pretrees from what we know about how the composition product interacts with pretrees on p . The identity lens on $p^{\ast n}$ keeps the lower n levels of each $p^{\ast(n+1)}$ -pretree intact, while $!$ will “strip away” the $p^{\ast(n+1)}$ -pretree’s height- $(n+1)$ leaves, along with all the position-labels on its height- n vertices. Thus $p^{\ast n}(!)$ is the function sending every stage- $(n + 1)$ pretree on p to its stage- n pretree, effectively *trimming it down* a level.

We can go even further: composing several such functions yields a composite function

$$p^{\ast n}(1) \xleftarrow{p^{\ast n}(!)} p^{\ast(n+1)}(1) \leftarrow \dots \leftarrow p^{\ast(n+k-1)}(1) \xleftarrow{p^{\ast(n+k-1)}(!)} p^{\ast(n+k)}(1) \quad (8.9)$$

sending every stage- $(n + k)$ pretree on p to its stage- n pretree by trimming off its top k levels. We will see these functions again shortly.

Trees as a limit of pretrees

Before we go any further in the theory of p -trees, let us look at some more examples.

Example 8.10 (A few more actual p -trees). Keeping $p := \{\blacklozenge, \blacksquare\}y^2 + \{\bullet\}y + \{\blacktriangle\}$, here are some elements of tree_p that we could imagine (or even draw, at least in part):

- The binary tree whose every node is $\blacklozenge$.
- The binary tree whose every node of even height is $\blacklozenge$ and every node of odd height is $\blacksquare$.
- The binary tree whose root is $\blacklozenge$, but after which every left child is $\blacklozenge$ and every right child is $\blacksquare$.¹
- The tree where every node is $\blacklozenge$, except for the rightmost branch, which (apart from the $\blacklozenge$ root) is always $\bullet$.
- Any finite tree, where every branch terminates in a $\blacktriangle$ node.
- A completely random tree: for the root, randomly choose ei-

ther $\blacklozenge$, $\blacksquare$, $\bullet$, or $\blacktriangle$; and at every leaf, loop back to the beginning, i.e. randomly choose either $\blacklozenge$, $\blacksquare$, $\bullet$, or $\blacktriangle$; etc.

In fact, there are uncountably many trees in tree_p (even just tree_{2y} has cardinality $2^{\mathbb{N}}$), but only countably many can be uniquely characterized in a finite language like English (and of course only finitely many can be uniquely characterized in the time we have!). Thus most elements of tree_p cannot even be described.

Exercise 8.11. For each of the following polynomials p , characterize the set of trees tree_p .

5. $p := 1$.
6. $p := 2$.
7. $p := y$.
8. $p := y^2$.
9. $p := 2y$.
10. $p := y + 1$.
11. $p := By^A$ for some sets $A, B \in \mathbf{Set}$.

Exercise 8.12.

1. Say we were interested in n -ary trees: infinite (unless $n = 0$) rooted trees in which every vertex has n children. Is there a polynomial p for which tree_p is the set of n -ary trees?
2. Now say we wanted to assign each vertex of an n -ary tree a label from a set L . Is there a polynomial q for which tree_q is the set of L -labeled n -ary trees?

From here, a natural question to ask is how the set tree_p of p -trees is related to the set of $p^{<n}$ -pretrees $p^{<n}(1)$ for each $n \in \mathbb{N}$. A look back at Definition 8.6 gives a clue: every p -tree has a stage- n pretree obtained by removing all vertices of height greater than n , yielding a function $\text{tree}_p \rightarrow p^{<n}(1)$ that we will denote by $\pi^{(n)}$.

¹To formalize the notions of “left” and “right,” we could think of the direction-sets at $\blacklozenge$ and $\blacksquare$ as $2 \cong \{\text{left}, \text{right}\}$, so that out of every vertex there is an edge labeled left and an edge labeled right.

Moreover, since the stage- n pretree of a given p -tree agrees with the stage- n pretree of any stage- $(n + k)$ pretree of the p -tree, the functions $\pi^{(n)}$ should commute with our tree-trimming functions $p^{\triangleleft(n+k)}(1) \rightarrow p^{\triangleleft n}(1)$ from (8.9) in Example 8.8. In particular, the following diagram commutes for all $n \in \mathbb{N}$:

$$\begin{array}{ccc} \text{tree}_p & & \\ \pi^{(n)} \downarrow & \searrow \pi^{(n+1)} & \\ p^{\triangleleft n}(1) & \xleftarrow{p^{\triangleleft n}(!)} & p^{\triangleleft(n+1)}(1). \end{array}$$

All this says is that if we trim a p -tree down until only $n + 1$ levels are left via $\pi^{(n+1)}$, then trimmed off one more level via $p^{\triangleleft n}(!)$, it would be the same as if we had trimmed it down to n levels from the start via $\pi^{(n)}$. This is summarized by the following larger commutative diagram, which contains every function of the form Example 8.8:

$$\begin{array}{ccccccc} \text{tree}_p & & & & & & \\ \downarrow \pi^{(0)} & \searrow \pi^{(1)} & \searrow \pi^{(2)} & \searrow \pi^{(3)} & \searrow \cdots & & \\ 1 & \xleftarrow{!} & p(1) & \xleftarrow{p(!)} & p^{\triangleleft 2}(1) & \xleftarrow{p^{\triangleleft 2}(!)} & p^{\triangleleft 3}(1) \xleftarrow{\cdots} \end{array} \quad (8.13)$$

So tree_p with the functions $\pi^{(n)}$ forms a cone over the the bottom row—in fact, it is the universal cone. Intuitively, this is because a p -tree carries exactly the information that a compatible sequence of $p^{\triangleleft n}$ -pretrees does: no more, no less. But we can prove it formally as well.

Exercise 8.14. Prove that tree_p with the functions $\pi^{(n)}: \text{tree}_p \rightarrow p^{\triangleleft n}(1)$ is the limit of the diagram

$$1 \xleftarrow{!} p(1) \xleftarrow{p(!)} p^{\triangleleft 2}(1) \xleftarrow{p^{\triangleleft 2}(!)} p^{\triangleleft 3}(1) \xleftarrow{\cdots} \quad (8.15)$$

You may use the fact that (8.13) commutes.

If you know about coalgebras for functors, as mentioned in Example 6.66, then you might know the limit $t_p(1)$ of (8.15) by a different

name: it is the *terminal coalgebra for the functor p* , or the *terminal p -coalgebra*, because it is terminal in the category of p -coalgebras, as the following exercise shows.

Exercise 8.16. A p -coalgebra morphism between p -coalgebras $\varphi: S \rightarrow p(S)$ and $\psi: T \rightarrow p(T)$ (as in Example 6.66) is a function $f: S \rightarrow T$ such that the square

$$\begin{array}{ccc} S & \xrightarrow{\varphi} & p(S) \\ f \downarrow & & \downarrow p(f) \\ T & \xrightarrow{\psi} & p(T) \end{array}$$

commutes.

1. Choose what you think is a good function $\text{tree}_p \rightarrow p(\text{tree}_p)$.
2. Show that tree_p equipped with your function $\text{tree}_p \rightarrow p(\text{tree}_p)$ is the terminal object in the category of p -coalgebras and the morphisms between them.
3. Show that the function $\text{tree}_p \rightarrow p(\text{tree}_p)$ you chose is a bijection.

Positions of the cofree comonoid

Now recall from Example 5.36 that in **Poly**, the positions of a limit are the limit of the positions. Moreover, in (8.1), every vertical lens $p^{a^n} \rightarrow p^{a^n} \triangleleft 1$ is, in fact, a *vertical lens* in the sense of Definition 5.52: an isomorphism on positions. So (8.1) on positions collapses down to its bottom row, viewed as a diagram in **Set**. Yet this is precisely the diagram from (8.15), whose limit is tree_p . So tree_p is the position-set of the limit t_p of (8.1): we write $t_p(1) \cong \text{tree}_p$.

So, as we said above, $t_p(1)$ is the set carrying the terminal coalgebra of p , but we prefer to think of it as the set of p -trees, for it gives us a concrete way to realize the limit and its projections, as well as a natural interpretation of the directions of t_p at each position.

The directions of the cofree comonoid

Given that t_p is the limit of (8.1), Example 5.36 also tells us how to compute its directions: the directions of a limit are the colimit of the directions. But every polynomial in the bottom row of (8.1) has an empty direction-set, and there are no arrows between polynomials in the top row. So the directions of t_p are given by a coproduct of the directions of each $p^{\ast n}$.

More precisely, given a p -tree $T \in \text{tree}_p$ whose stage- n pretree for $n \in \mathbb{N}$ is $\pi^{(n)}T$, the direction-set $t_p[T]$ is given by the following coproduct:

$$t_p[T] := \sum_{n \in \mathbb{N}} p^{\ast n}[\pi^{(n)}T].$$

But by Definition 8.6, each $p^{\ast n}[\pi^{(n)}T]$ is the set of height- n leaves of the $p^{\ast n}$ -pretree $\pi^{(n)}T$, which in turn is the stage- n pretree of T . So its height- n leaves coincide with the height- n vertices of T . Therefore, we can identify $p^{\ast n}[\pi^{(n)}T]$ with the set of height- n vertices of T ; we denote this set by $\text{vtx}_n(T)$.

Since the coproduct above ranges over all $n \in \mathbb{N}$, it follows that $t_p[T]$ is the set of *all* vertices in T ; we denote this set by $\text{vtx}(T)$. The on-directions function at T of each projection $t_p \rightarrow p^{\ast n}$ from the limit must then be the canonical inclusion

$$p^{\ast n}[\pi^{(n)}T] \cong \text{vtx}_n(T) \hookrightarrow \text{vtx}(T) \cong t_p[T],$$

sending height- n leaves of $\pi^{(n)}T$ to height- n vertices of T .

Here is an alternative way to think about the directions of t_p and each $p^{\ast n}$ that will be helpful. A defining feature of a rooted tree is that its vertices are in bijection with its finite rooted paths: each vertex gives rise to a unique path to that vertex from the root, and every finite rooted path arises this way. So the directions of $p^{\ast n}$ at a given $p^{\ast n}$ -pretree correspond in turn to the rooted paths of that pretree leading to its height- n leaves; indeed, each such direction is comprised of a sequence of n directions of p , which together specify a length- n rooted path up the pretree. Then for $T \in \text{tree}_p$, the direction-set $t_p[T]$ consists of every finite rooted path of T .

Since only finite rooted paths correspond to vertices, all our paths will be assumed to be finite from here on out. This is our preferred way

to think about directions in t_p . When we wish to refer to what one might call an infinite (rooted) “path,” we will instead call it a (rooted) *ray*.

Exercise 8.17. For each of the following polynomials p , characterize the polynomial t_p . You may choose to think of the directions of t_p either as vertices or as rooted paths. (Note that these are the same polynomials from Exercise 8.11.)

1. $p := 1$.
2. $p := 2$.
3. $p := y$.
4. $p := y^2$.
5. $p := 2y$.
6. $p := y + 1$.
7. $p := By^A$ for some sets $A, B \in \mathbf{Set}$.

We summarize the results of this section in the following proposition, thus concretely characterizing the carrier t_p of the cofree comonoid on p in terms of p -trees. For reasons that will become clear shortly, we will denote each projection from the limit of (8.1) by $\epsilon_p^{(n)}: t_p \rightarrow p^{<n}$. We denote ϵ simply by ϵ .

Proposition 8.18. For $p \in \mathbf{Poly}$, let

$$t_p := \sum_{T \in \text{tree}_p} y^{\text{vtx}(T)}$$

be the polynomial whose positions are p -trees and whose directions at each p -tree are the rooted paths. Then t_p is the limit of the diagram (8.1), with projections $\epsilon_p^{(n)}: t_p \rightarrow p^{<n}$ for every $n \in \mathbb{N}$

making the following diagram commute:

$$\begin{array}{ccccccc}
 t_p & & & & & & \\
 \downarrow \epsilon_p & \searrow \epsilon_p^{(1)} & \searrow \epsilon_p^{(2)} & \searrow \epsilon_p^{(3)} & \searrow \dots & & \\
 y & p & p^{\triangleleft 2} & p^{\triangleleft 3} & \dots & & \\
 \downarrow & \downarrow & \downarrow & \downarrow & & & \\
 1 & \xleftarrow{!} p(1) & \xleftarrow{p(!)} p^{\triangleleft 2}(1) & \xleftarrow{p^{\triangleleft 2}(!)} p^{\triangleleft 3}(1) & \xleftarrow{\dots} \dots & &
 \end{array}
 \quad (8.19)$$

The lens $\epsilon_p^{(n)}: t_p \rightarrow p^{\triangleleft n}$ sends each p -tree $T \in \text{tree}_p$ to its stage- n pretree $\pi^{(n)}T$ on positions and each height- n leaf of $\pi^{(n)}T$ to the corresponding height- n rooted path of T on directions.

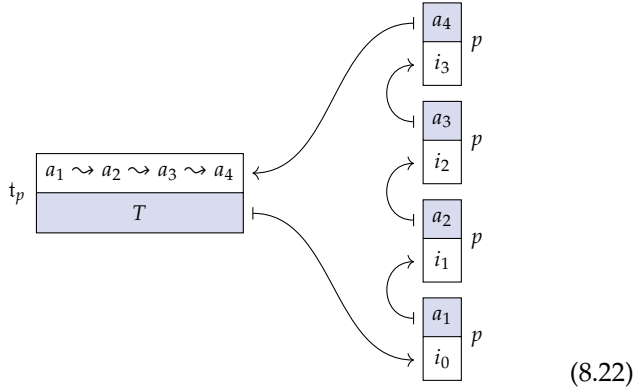
Example 8.20 (Drawing $\epsilon_p^{(n)}$ in polyboxes). Here is $\epsilon_p^{(n)}: t_p \rightarrow p^{\triangleleft n}$ drawn in polyboxes, where we continue to denote the stage- n pretree of a p -tree by $\pi^{(n)}T$:

$$\begin{array}{ccc}
 \text{vtx}(-) \begin{array}{|c|} \hline v \\ \hline \end{array} & \xleftarrow{\quad} & \begin{array}{|c|} \hline v \\ \hline \end{array} \text{vtx}_n(-) \\
 \text{tree}_p \begin{array}{|c|} \hline T \\ \hline \end{array} & \xrightarrow{\quad} & \begin{array}{|c|} \hline \pi^{(n)}T \\ \hline \end{array} p^{\triangleleft n}(1) \\
 t_p & & p^{\triangleleft n}
 \end{array}
 \quad (8.21)$$

On the right-hand side, v is a height- n leaf of $\pi^{(n)}T$; on the left, v is identified with the corresponding height- n vertex of T .

This isn't the only way we can write this lens in polyboxes, however; polyboxes have special notation for lenses to composites,

allowing us to write, say, the $n = 4$ case like so:

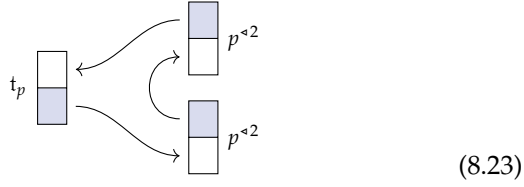


Here we are unpacking the construction of the first 4 levels of T , according to our nested instructions for building p -trees. The p -position i_0 is the label on the root of T , while the $p[i_0]$ -direction a_1 specifies one of the edges coming out of it—leading to a height-1 vertex of T labeled i_1 , and so on.

The contents of the position-boxes and the arrows going up on the codomain side carry all the data of the bottom 4 levels of T : namely the label i_0 of the root, the label i_1 of the vertex at the end of every direction a_1 out of the root, and so on until i_3 . All this specifies a unique $p^{\triangleleft 4}$ -position, a $p^{\triangleleft 4}$ -pretree, which is the same as the $p^{\triangleleft 4}$ -pretree T . Indeed, we can think of $\pi^{(4)}T$ as a shorthand for the protocol comprised of the 4 polyboxes on the right-hand side of (8.22) when their blue boxes are yet to be filled. So the position and direction depicted on the codomain side of (8.22) is equivalent to the position and direction depicted on the codomain side of (8.21). This generalizes to all values of n . The direction a_4 emanating from i_3 , together with all the data below it, corresponds to the rooted path we've denoted $a_1 \rightsquigarrow a_2 \rightsquigarrow a_3 \rightsquigarrow a_4$ in T .

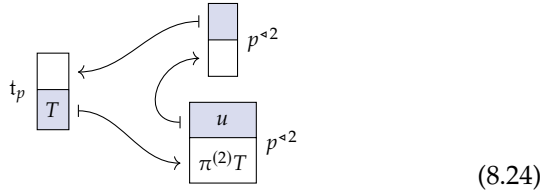
There are even more ways to express $\epsilon_p^{(4)}: t_p \rightarrow p^{\triangleleft 4}$ in polyboxes, however. After all, $p^{\triangleleft 4} \cong p^{\triangleleft 2} \triangleleft p^{\triangleleft 2}$. So we ought to be able to draw

$\epsilon_p^{(4)}$ as follows:



Let's think about how we should fill these boxes. We can still put a p -tree T in the lower left position box. From the commutativity of (8.19), we know the on-positions function $\pi^{(4)}$ of $\epsilon_p^{(4)}$ factors through the on-positions function $\pi^{(2)}$ of $\epsilon_p^{(2)}$, which tells us that $p^{<2>}$ -pretree that should go in the lower right position box should be $\pi^{(2)}T$: the $p^{<2>}$ -pretree of T .

Another way to think about this is that the polyboxes for the lower $p^{<2>}$ on the right side of (8.23) are equivalent to the polyboxes for the 2 lower copies of p on the right side of (8.22)—just like how the polyboxes for $p^{<n>}$ in (8.21) are equivalent to the polyboxes for all $n = 4$ copies of p in (8.22). There, the position could be represented with a single $p^{<n>}$ -pretree $\pi^{(n)}T$, and the direction $(a_1, \dots, a_n)$ is one of its length- n rooted paths. So here, too, we can package the 2 lower copies of p into a single pair of polyboxes for $p^{<2>}$, if we let u be the height-2 vertex of T at the end of the rooted path (a_1, a_2) :



But then the polyboxes for the 2 upper copies of p from (8.22) should also collapse down to a single pair of polyboxes for $p^{<2>}$, with a $p^{<2>}$ -pretree as its position and a height-2 leaf of that pretree as its direction. Indeed, once we have followed the directions (a_1, a_2) up to the height-2 vertex u , the subtree of T rooted at u is itself a p -tree: it has a label i_2 on its root, out of which we can follow the direction a_3 to reach a height-1 vertex labeled i_3 , then follow the

direction a_4 to reach a height-2 vertex that we call w . Of course, the vertex labeled i_3 is actually a height-3 vertex of the whole tree T ; likewise w corresponds to a height-4 vertex of T . However, from the perspective of the upper copy of $p^{\leq 2}$ in (8.24), we are starting over from u and moving up the subtree of T rooted at u —or, more precisely, the $p^{\leq 2}$ -pretree of the subtree rooted at u . So the polyboxes end up looking like this:

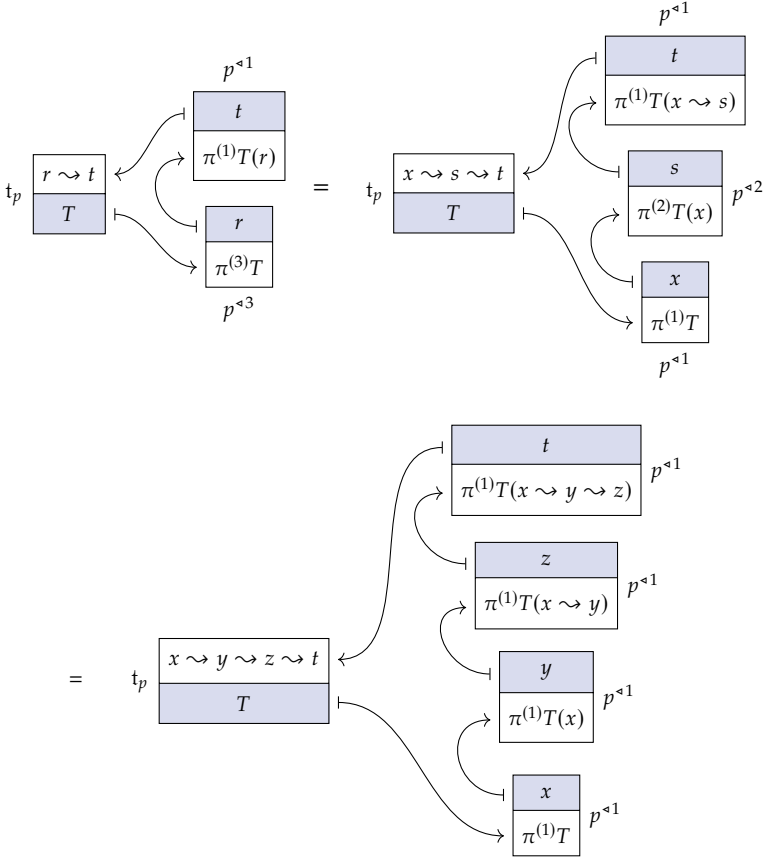
$$\begin{array}{c}
 \begin{array}{|c|} \hline u \rightsquigarrow w \\ \hline T \\ \hline \end{array} \quad \begin{array}{c} \swarrow \\ \searrow \end{array} \quad \begin{array}{|c|} \hline w \\ \hline \pi^{(2)}T(u) \\ \hline \end{array} \quad \begin{array}{c} \swarrow \\ \searrow \end{array} \quad \begin{array}{|c|} \hline u \\ \hline \pi^{(2)}T \\ \hline \end{array} \\
 t_p \qquad \qquad \qquad p^{\leq 2} \qquad \qquad \qquad p^{\leq 2}
 \end{array} \tag{8.25}$$

Here $T(u)$ denotes the p -tree equal to the subtree of T rooted at its vertex u , and w is one of $T(u)$'s height-2 vertices. When viewed as the subtree of T rooted at its the height-2 vertex u , the p -tree $T(u)$ has its height-2 vertex w identified with a height-4 vertex of T , a descendent of u that we denote by $u \rightsquigarrow w$. Alternatively, u corresponds to the rooted path (a_1, a_2) of T , while w corresponds to the rooted path (a_3, a_4) of $T(u)$, so $u \rightsquigarrow w$ corresponds to the concatenated rooted path (a_1, a_2, a_3, a_4) of T . As concatenation is associative, $\rightsquigarrow$ is associative as well.

Both the notation $T(u)$, for what we will call the p -subtree of the p -tree T rooted at $u \in \text{vtx}(T)$, and the notation $u \rightsquigarrow w$, for the vertex in $\text{vtx}(T)$ that $w \in \text{vtx}(T(u))$ coincides with when $T(u)$ is identified with the subtree of p rooted at u , will turn out to be temporary—we will soon justify why we can express these concepts in much more familiar terms.² As a sneak preview, they will be crucial to defining the categorical structure of the cofree comonoid on p .

Here are three more ways to depict $\epsilon_p^{(4)}$, viewing its codomain

in turn as $p^{\triangleleft 3} \triangleleft p^{\triangleleft 1}$, as $p^{\triangleleft 1} \triangleleft p^{\triangleleft 2} \triangleleft p^{\triangleleft 1}$, or as $p^{\triangleleft 1} \triangleleft p^{\triangleleft 1} \triangleleft p^{\triangleleft 1} \triangleleft p^{\triangleleft 1}$:



Here $r = x \rightsquigarrow s$ and $s = y \rightsquigarrow z$. Notice that the last depiction is just another way to write (8.22), with each $p^{\triangleleft 1}$ -pretree in place of the p -position that labels its root and height-1 vertices in place of their corresponding directions, which are just length-1 rooted paths.

Finally, all this could be generalized to other values of n . Here are different ways to draw polyboxes for $\epsilon_p^{(n)} : t_p \rightarrow p^{\triangleleft n}$, viewing its codomain as $p^{\triangleleft \ell} \triangleleft p^{\triangleleft m}$ with $n = \ell + m$ or $p^{\triangleleft i} \triangleleft p^{\triangleleft j} \triangleleft p^{\triangleleft k}$ with

$$n = i + j + k:$$

The diagram illustrates the decomposition of a p -tree T into three subtrees based on the equation $n = i + j + k$. On the left, a tree T with root $u \leadsto w$ is shown. It branches into two subtrees: one with root w (labeled $p^{\leq m}$) and another with root u (labeled $p^{\leq \ell}$). On the right, the same tree T is shown with root $x \leadsto s \leadsto t$. It branches into three subtrees: one with root t (labeled $p^{\leq k}$), one with root s (labeled $p^{\leq j}$), and one with root x (labeled $p^{\leq i}$). The two sides are separated by an equals sign, indicating that the tree structure is preserved under this decomposition.

8.1.2 Cofree comonoids as categories

We have now characterized the carrier t_p of the comonoid $\mathcal{T}_p$ that will turn out to be cofree on p ; but we have yet to describe the comonoid structure of $\mathcal{T}_p$ that t_p carries. This structure will allow us to interpret $\mathcal{T}_p$ as a category, whose objects will be p -trees and whose morphisms will be the vertices of each p -tree. We could go ahead and describe this category right now, but as category theorists, let us show that the eraser and duplicator of $\mathcal{T}_p$, as well as the comonoid laws that they satisfy, arise naturally from our construction of t_p as the limit of (8.1).

The eraser of the cofree comonoid

The eraser for a comonoid $\mathcal{T}_p$ carried by t_p should be a lens of the form $t_p \rightarrow y$. Conveniently, since y appears in (8.1), its limit t_p is already equipped with a canonical lens $\epsilon_p: t_p \rightarrow y$, as seen in (8.19). This lens will turn out to be the eraser of the cofree comonoid on p .

The eraser picks out a direction at each position of the carrier to be the identity morphism on that object. As each t_p -position is a tree $T \in \text{tree}_p$, and each $t_p[T]$ -direction is a vertex of T , the eraser ϵ_p should

²As a reminder, due to the definition of a p -tree T , a p -subtree of T is still itself a p -tree, whereas the $p^{\leq n}$ -pretree of T is not a p -tree unless the height of T is strictly less than n .

pick out a single vertex of every p -tree. Even without looking at (8.19), you may already have your suspicions as to which vertex this will turn out to be, as there is, after all, only one sensible way to choose a canonical vertex that every p -tree is guaranteed to have: choose its root. Indeed, Proposition 8.18 tells us that $\epsilon_p: t_p \rightarrow y$ sends each p -tree $T \in \text{tree}_p$ to its stage-0 pretree on positions, stripping away everything except for the single unlabeled root of T ; then on-directions , ϵ_p picks out the unique height-0 vertex of T in $\text{vtx}_0(T) \subseteq \text{vtx}(T)$, which is just its root.

So in the category $\mathcal{T}_p$, where morphisms out of objects are vertices of trees, the identity morphism on a tree is its root—in fact, we will henceforth denote the root of a p -tree by id_T (so that $\text{vtx}_0(T) = \{\text{id}_T\}$). Equivalently, we can identify id_T with the unique length-0 rooted path of T : the *empty rooted path*, which starts and ends at the root. The rest of the categorical structure of $\mathcal{T}_p$ will be determined by its duplicator.

The duplicator of the cofree comonoid

The duplicator for a comonoid $\mathcal{T}_p$ carried by t_p should be a lens $\delta_p: t_p \rightarrow t_p \triangleleft t_p$. But before we can specify such a lens, we need to figure out what kind of polynomial $t_p \triangleleft t_p$ is.

Here is where our work from Sections 6.3.2 and 6.3.3 comes in handy: since the diagram in (8.1) is connected, its limit t_p is a connected limit, and we showed in Theorem 6.79 that $\triangleleft$ preserves connected limits. Therefore $t_p \triangleleft t_p$ is itself the limit of some diagram, and a morphism to $t_p \triangleleft t_p$ is just a cone over that diagram.

Which diagram is it? First, we can use the fact that $\triangleleft$ preserves connected limits on the left to expand $t_p \triangleleft t_p$ as the limit of the following diagram, where we have applied the functor $-\triangleleft t_p$ to the diagram from (8.1):

$$\begin{array}{ccccccc}
 y \triangleleft t_p & & p \triangleleft t_p & & p^{\triangleleft 2} \triangleleft t_p & & p^{\triangleleft 3} \triangleleft t_p & & \cdots \\
 \downarrow & & \downarrow & & \downarrow & & \downarrow & & \\
 1 \triangleleft t_p & \longleftarrow & p \triangleleft 1 \triangleleft t_p & \longleftarrow & p^{\triangleleft 2} \triangleleft 1 \triangleleft t_p & \longleftarrow & p^{\triangleleft 3} \triangleleft 1 \triangleleft t_p & \longleftarrow & \cdots
 \end{array}$$

But we have $p^{\triangleleft n} \triangleleft 1 \triangleleft t_p \cong p^{\triangleleft n} \triangleleft 1$, so we can simplify this diagram like

so:

$$\begin{array}{ccccccc}
 y \triangleleft t_p & & p \triangleleft t_p & & p^{\triangleleft 2} \triangleleft t_p & & p^{\triangleleft 3} \triangleleft t_p & & \dots \\
 \downarrow & & \downarrow & & \downarrow & & \downarrow & & \\
 1 \longleftarrow & p \triangleleft 1 \longleftarrow & p^{\triangleleft 2} \triangleleft 1 \longleftarrow & p^{\triangleleft 3} \triangleleft 1 \longleftarrow & \dots
 \end{array} \quad (8.26)$$

(This works because according to Proposition 6.67, $\triangleleft$ actually preserves *all* limits on the left—including the terminal object 1.) So $t_p \triangleleft t_p$ is the limit of (8.26).

Then we use the fact that $\triangleleft$ preserves connected limits on the right to expand each $p^{\triangleleft \ell} \triangleleft t_p$ in (8.26) as the limit of the following:

$$\begin{array}{ccccccc}
 p^{\triangleleft \ell} \triangleleft y & & p^{\triangleleft \ell} \triangleleft p & & p^{\triangleleft \ell} \triangleleft p^{\triangleleft 2} & & p^{\triangleleft \ell} \triangleleft p^{\triangleleft 3} & & \dots \\
 \downarrow & & \downarrow & & \downarrow & & \downarrow & & \\
 p^{\triangleleft \ell} \triangleleft 1 \longleftarrow & p^{\triangleleft \ell} \triangleleft p \triangleleft 1 \longleftarrow & p^{\triangleleft \ell} \triangleleft p^{\triangleleft 2} \triangleleft 1 \longleftarrow & p^{\triangleleft \ell} \triangleleft p^{\triangleleft 3} \triangleleft 1 \longleftarrow & \dots
 \end{array} \quad (8.27)$$

So the limit of (8.26) is the limit of a larger diagram, where we have “plugged in” a copy of (8.27) in place of each $p^{\triangleleft \ell} \triangleleft t_p$ that appears.

It’s worth being a little careful, though, when we draw this diagram: each $p^{\triangleleft \ell} \triangleleft t_p$ in (8.26) appears with a lens to $p^{\triangleleft \ell} \triangleleft 1$, and we need to work out what arrow should go in its place. The lens in question is given by applying $p^{\triangleleft \ell} \triangleleft -$ to the unique lens $t_p \rightarrow 1$. But 1 actually shows up in the lower left corner of (8.1), so the unique lens $t_p \rightarrow 1$ is just the projection from the limit of (8.1) to that 1. Then once we apply the connected limit-preserving functor $p^{\triangleleft \ell} \triangleleft -$ to (8.1), yielding (8.27), we obtain the lens $p^{\triangleleft \ell} \triangleleft t_p \rightarrow p^{\triangleleft \ell} \triangleleft 1$ we desire as the projection from the limit of (8.27) to the $p^{\triangleleft \ell} \triangleleft 1$ in the lower left corner. So if we want to replace each $p^{\triangleleft \ell} \triangleleft t_p$ in (8.26) with a copy of (8.27), without changing the limit of the whole diagram, we should replace the arrow $p^{\triangleleft \ell} \triangleleft t_p \rightarrow p^{\triangleleft \ell} \triangleleft 1$ with an identity arrow from the $p^{\triangleleft \ell} \triangleleft 1$ in the lower left corner of (8.27) to the $p^{\triangleleft \ell} \triangleleft 1$ in (8.26). Of course, we can then collapse these identity arrows down without changing the limit, so the diagram we are left

with should look like this:

$$\begin{array}{ccccccc}
 p^{\triangleleft 3} & & p^{\triangleleft 3} \triangleleft p & & p^{\triangleleft 3} \triangleleft p^{\triangleleft 2} & & \dots \\
 \searrow & & \searrow & & \searrow & & \dots \\
 p^{\triangleleft 3} \triangleleft 1 & & p^{\triangleleft 3} \triangleleft p \triangleleft 1 & & p^{\triangleleft 3} \triangleleft p^{\triangleleft 2} \triangleleft 1 & & \dots \\
 \downarrow & & \downarrow & & \downarrow & & \dots \\
 p^{\triangleleft 2} & & p^{\triangleleft 2} \triangleleft p & & p^{\triangleleft 2} \triangleleft p^{\triangleleft 2} & & \dots \\
 \searrow & & \searrow & & \searrow & & \dots \\
 p^{\triangleleft 2} \triangleleft 1 & & p^{\triangleleft 2} \triangleleft p \triangleleft 1 & & p^{\triangleleft 2} \triangleleft p^{\triangleleft 2} \triangleleft 1 & & \dots \\
 \downarrow & & \downarrow & & \downarrow & & \dots \\
 p & & p \triangleleft p & & p \triangleleft p^{\triangleleft 2} & & \dots \\
 \searrow & & \searrow & & \searrow & & \dots \\
 p \triangleleft 1 & & p \triangleleft p \triangleleft 1 & & p \triangleleft p^{\triangleleft 2} \triangleleft 1 & & \dots \\
 \downarrow & & \downarrow & & \downarrow & & \dots \\
 y & & p & & p^{\triangleleft 2} & & \dots \\
 \searrow & & \searrow & & \searrow & & \dots \\
 1 & \longleftarrow & p \triangleleft 1 & \longleftarrow & p^{\triangleleft 2} \triangleleft 1 & \longleftarrow & \dots
 \end{array} \tag{8.28}$$

Here the bottom row of (8.26) is still the bottom row of (8.28), but in the place of each $p^{\triangleleft \ell} \triangleleft t_p$ in the top row, we have grafted in a copy of (8.27). Yet the limit of the diagram is preserved: the limit of (8.28) is still $t_p \triangleleft t_p$. In summary, for purely formal reasons, Proposition 6.67 and Theorem 6.79 ensure that since the limit of (8.1) is t_p , the limit of (8.28) is $t_p \triangleleft t_p$.

While (8.28) has become rather unwieldy to draw, it is easy to characterize: it is a diagram with a copy each of

$$p^{\triangleleft \ell} \triangleleft p^{\triangleleft m} \quad \text{and} \quad p^{\triangleleft \ell} \triangleleft p^{\triangleleft m} \triangleleft 1$$

for every $\ell, m \in \mathbb{N}$, with arrows

$$p^{\triangleleft \ell} \triangleleft p^{\triangleleft m} \rightarrow p^{\triangleleft \ell} \triangleleft p^{\triangleleft m} \triangleleft 1 \tag{8.29}$$

between them; along with arrows

$$p^{\triangleleft \ell} \triangleleft p^{\triangleleft (m+1)} \triangleleft 1 \rightarrow p^{\triangleleft \ell} \triangleleft p^{\triangleleft m} \triangleleft 1, \tag{8.30}$$

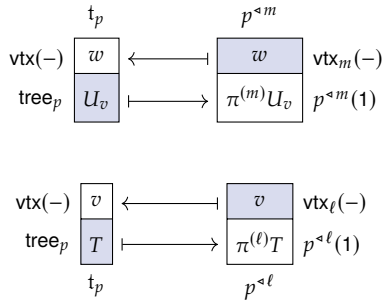
drawn vertically in (8.28); and

$$p^{\triangleleft \ell+1} \triangleleft p^{\triangleleft 0} \triangleleft 1 \cong p^{\triangleleft \ell+1} \triangleleft 1 \rightarrow p^{\triangleleft \ell} \triangleleft 1 \cong p^{\triangleleft \ell} \triangleleft p^{\triangleleft 0} \triangleleft 1, \tag{8.31}$$

drawn horizontally in (8.28) along the bottom row. Of course, we could write each $p^{\triangleleft \ell} \triangleleft p^{\triangleleft m}$ as $p^{\triangleleft (\ell+m)}$, but the notation $p^{\triangleleft \ell} \triangleleft p^{\triangleleft m}$ helps us distinguish it as the object appearing ℓ rows above the bottom and m columns to the right in (8.28), in contrast with any other $p^{\triangleleft \ell'} \triangleleft p^{\triangleleft m'}$ with $(\ell, m) \neq (\ell', m')$, even if $\ell + m = \ell' + m'$ guarantees that there are isomorphisms between these objects.

Nevertheless, it is these isomorphisms that induce a canonical lens $t_p \rightarrow t_p \triangleleft t_p$, by giving a map from the diagram (8.1) to the diagram (8.28) as follows. For all $\ell, m, n \in \mathbb{N}$ with $n = \ell + m$, we have a canonical isomorphism $p^{\triangleleft n} \xrightarrow{\cong} p^{\triangleleft \ell} \triangleleft p^{\triangleleft m}$ sending the $p^{\triangleleft n}$ that appears in (8.1) to the $p^{\triangleleft \ell} \triangleleft p^{\triangleleft m}$ that appears in (8.28), and similarly sending $p^{\triangleleft n} \triangleleft 1$ in (8.1) to $p^{\triangleleft \ell} \triangleleft p^{\triangleleft m} \triangleleft 1$ in (8.28). Then all the arrows that appear in (8.28)—i.e. all the arrows in (8.29), (8.30), and (8.31)—can be identified with arrows that appear in (8.1), so everything commutes. We therefore induce a lens from the limit t_p of (8.1) to the limit $t_p \triangleleft t_p$ of (8.28), which we call $\delta_p: t_p \rightarrow t_p \triangleleft t_p$. This turns out to be the duplicator of $\mathcal{T}_p$.

How does δ_p behave in terms of p -trees? First, we characterize $t_p \triangleleft t_p$ and its projections to each $p^{\triangleleft \ell} \triangleleft p^{\triangleleft m}$ in (8.28). Concretely, we know that a $t_p \triangleleft t_p$ -position is just a p -tree $T \in \text{tree}_p$ along with a p -tree $U(v) \in \text{tree}_p$ associated with every vertex $v \in \text{vtx}(T)$. Then a direction at that position is a choice of vertex $v \in \text{vtx}(T)$ and another vertex $w \in \text{vtx}(U(v))$ of the p -tree $U(v)$ associated with v . Each projection from $t_p \triangleleft t_p$ to $p^{\triangleleft \ell} \triangleleft p^{\triangleleft m}$ in (8.28) is then the composition product of the projections $\epsilon_p^{(\ell)}: t_p \rightarrow p^{\triangleleft \ell}$ and $\epsilon_p^{(m)}: t_p \rightarrow p^{\triangleleft m}$, which by Example 8.20 we can draw in polyboxes like so:



On positions, $\epsilon_p^{(\ell)} \triangleleft \epsilon_p^{(m)}$ sends the p -tree T to its stage- ℓ pretree $\pi^{(\ell)}T$, and it sends each p -tree U_v associated to a height- ℓ vertex $v \in \text{vtx}_\ell(T)$

to its stage- m pretree $\pi^{(m)}U_v$, to be $p^{\triangleleft m}$ -pretree associated with the height- ℓ leaf v of $\pi^{(m)}T$. Equivalently, this specifies a $p^{\triangleleft(\ell+m)}$ -pretree on the right: its bottom ℓ levels coincide with the bottom ℓ levels of T , and its top m levels coincide with the bottom m levels of the U_v 's for $v \in \text{vtx}_\ell(T)$. Then on directions, $\epsilon_p^{(\ell)} \triangleleft \epsilon_p^{(m)}$ is the canonical inclusion of vertices $\text{vtx}_\ell(T) \hookrightarrow \text{vtx}(T)$ sending $v \mapsto v$, followed by the canonical inclusion of vertices $\text{vtx}_m(U_v) \hookrightarrow \text{vtx}(U_v)$. These lenses comprise the universal cone over (8.28).

Meanwhile, the other cone we formed over (8.28) is comprised of lenses of the form $\epsilon_p^{(\ell+m)}: t_p \rightarrow p^{\triangleleft(\ell+m)} \cong p^{\triangleleft \ell} \triangleleft p^{\triangleleft m}$, each of which should factor through $t_p \triangleleft t_p$, as depicted in the following commutative diagram:

$$\begin{array}{ccc}
 t_p & \xrightarrow{\delta_p} & t_p \triangleleft t_p \\
 \searrow \epsilon_p^{(\ell+m)} & & \downarrow \epsilon_p^{(\ell)} \triangleleft \epsilon_p^{(m)} \\
 & & p^{\triangleleft \ell} \triangleleft p^{\triangleleft m}.
 \end{array} \tag{8.32}$$

Indeed, by the universal property of $t_p \triangleleft t_p$, our δ_p is the unique lens for which (8.32) commutes for all $\ell, m \in \mathbb{N}$. We will use the equation given by the commutativity of (8.32) repeatedly in what follows, whenever we work with δ_p .

Expressing (8.32) as an equation of polyboxes, using our usual labels for the arrows of the duplicator δ_p on the left and our depiction of the projection $\epsilon_p^{(\ell+m)}: t_p \rightarrow p^{\triangleleft \ell} \triangleleft p^{\triangleleft m}$ from Example 8.20 on the right, we have

$$\begin{array}{c}
 \begin{array}{c}
 t_p \\
 \begin{array}{|c|} \hline w \\ \hline \text{cod } v \\ \hline \end{array} \\
 \begin{array}{|c|} \hline v \\ \hline T \\ \hline \end{array} \\
 t_p
 \end{array}
 \begin{array}{c}
 \xrightarrow{\delta_p} \\
 \begin{array}{|c|} \hline w \\ \hline \pi^{(m)}(\text{cod } v) \\ \hline \end{array} \\
 \begin{array}{|c|} \hline v \\ \hline \pi^{(\ell)}T \\ \hline \end{array} \\
 p^{\triangleleft \ell}
 \end{array}
 \end{array}
 =
 \begin{array}{c}
 \begin{array}{|c|} \hline v \rightsquigarrow w \\ \hline T \\ \hline \end{array} \\
 \begin{array}{|c|} \hline w \\ \hline \pi^{(m)}T(v) \\ \hline \end{array} \\
 \begin{array}{|c|} \hline v \\ \hline \pi^{(\ell)}T \\ \hline \end{array} \\
 p^{\triangleleft \ell}
 \end{array}
 \end{array}$$

Recall from Example 8.20 that $T(v)$ denotes the p -tree equal to the subtree of T rooted at $v \in \text{vtx}(T)$, while $v \rightsquigarrow w \in \text{vtx}(T)$ for $w \in \text{vtx}(T(v))$ is the descendent of v that coincides with w when $T(v)$ is identified with the subtree of T rooted at v . Equivalently, we can identify $v \in \text{vtx}(T)$ with the rooted path in T that ends at the vertex v and $w \in \text{vtx}(T(v))$ with the rooted path of $T(v)$ that ends at the vertex w , so $v \rightsquigarrow w$ becomes the rooted path in T obtained by concatenating v and w . Then for this equality to hold over all $\ell, m \in \mathbb{N}$, we want $\text{cod } v := T(v)$ and $v \circ w := v \rightsquigarrow w$; in fact, $\text{cod } v$ and $v \circ w$ will henceforth be our preferred notation for $T(v)$ and $v \rightsquigarrow w$.

Verifying the comonoid laws

Putting together our constructions of the carrier, the eraser, and the duplicator of the $\mathcal{T}_p$ yields the following result.

Proposition 8.33. As defined above, $(t_p, \epsilon_p, \delta_p)$ is a polynomial comonoid corresponding to a category $\mathcal{T}_p$ characterized as follows.

- An object in $\mathcal{T}_p$ is a p -tree in $T \in \text{tree}_p$.
- A morphism emanating from T is a *rooted path* in T ; its codomain is the p -subtree rooted at the end of the path.
- The identity morphism on T is its empty rooted path.
- Composition is given by concatenating rooted paths: given a rooted path v in T and a rooted path w in $\text{cod } v$, the p -subtree rooted at the end of v , we identify w with the corresponding path in T that starts where v ends, then concatenate the two paths in T to obtain the composite morphism $v \circ w$, another rooted path in T .

Proof. We have already shown how to construct the given carrier, eraser, and duplicator purely diagrammatically, and we have given them concrete interpretations in terms of p -trees, their vertices, and their p -subtrees. So it remains to verify that the category laws hold for $\mathcal{T}_p$, or equivalently that the comonoid laws hold for $(t_p, \epsilon_p, \delta_p)$. Again, our argument will be purely formal, although it is not too hard to see that our concrete characterization above in terms of p -trees satisfies the laws for a category.

First, we verify the left erasure law: that

$$\begin{array}{ccc}
 y \triangleleft t_p & \xlongequal{\quad} & t_p \\
 \nwarrow \epsilon_p \triangleleft t_p & & \downarrow \delta_p \\
 & & t_p \triangleleft t_p
 \end{array}$$

commutes. We know the lens $\epsilon_p \triangleleft t_p : t_p \triangleleft t_p \rightarrow y \triangleleft t_p$ is characterized by its components $\epsilon_p \triangleleft \epsilon_p^{(n)} : t_p \triangleleft t_p \rightarrow p^{\triangleleft 0} \triangleleft p^{\triangleleft n}$, a projection out of the limit $t_p \triangleleft t_p$, for all $n \in \mathbb{N}$. Then by our construction of δ_p , each composite $\delta_p \circ (\epsilon_p \triangleleft \epsilon_p^{(n)}) : t_p \rightarrow p^{\triangleleft 0} \triangleleft p^{\triangleleft n}$ is the component of δ_p equal to $\epsilon_p^{(n)} = \epsilon_p^{(0+n)} : t_p \rightarrow p^{\triangleleft 0} \triangleleft p^{\triangleleft n} \cong p^{\triangleleft n}$ (this is just the commutativity of (8.32) in the case of $(\ell, m) = (0, n)$). Together, these characterize the composite lens $\delta_p \circ (\epsilon_p \triangleleft t_p) : t_p \rightarrow y \triangleleft t_p \cong t_p$ as the lens whose components are the projections $\epsilon_p^{(n)} : t_p \rightarrow p^{\triangleleft n}$ from the limit. It follows from the universal property of t_p that $\delta_p \circ (\epsilon_p \triangleleft t_p)$ can only be the identity lens on t_p . Hence the left erasure law holds.

The right erasure law, that

$$\begin{array}{ccc}
 t_p & \xlongequal{\quad} & t_p \triangleleft y \\
 \delta_p \downarrow & \nearrow t_p \triangleleft \epsilon_p & \\
 t_p \triangleleft t_p & &
 \end{array}$$

commutes, follows similarly: the composite $\delta_p \circ (t_p \triangleleft \epsilon_p)$ is characterized by its components over $n \in \mathbb{N}$ of the form $\delta_p \circ (\epsilon_p^{(n)} \triangleleft \epsilon_p) : t_p \rightarrow p^{\triangleleft n} \triangleleft p^{\triangleleft 0}$, which we know by (8.32) is equal to $\epsilon_p^{(n+0)} = \epsilon_p^{(n)}$, the projection out of the limit t_p . It follows from the universal property of this limit that $\delta_p \circ (t_p \triangleleft \epsilon_p)$ must be the identity lens on t_p .

Finally, we check the coassociative law: that

$$\begin{array}{ccc}
 t_p & \xrightarrow{\delta_p} & t_p \triangleleft t_p \\
 \delta_p \downarrow & & \downarrow t_p \triangleleft \delta_p \\
 t_p \triangleleft t_p & \xrightarrow{\delta_p \triangleleft t_p} & t_p \triangleleft t_p \triangleleft t_p,
 \end{array}$$

commutes. Because $\triangleleft$ preserves connected limits, we can write $t_p \triangleleft t_p \triangleleft t_p$ as a limit the way we did with $t_p \triangleleft t_p$: it is the limit of diagram consisting

of arrows

$$p^{\triangleleft i} \triangleleft p^{\triangleleft j} \triangleleft p^{\triangleleft k} \rightarrow p^{\triangleleft i} \triangleleft p^{\triangleleft j} \triangleleft p^{\triangleleft k} \triangleleft 1$$

for each $i, j, k \in \mathbb{N}$, with additional arrows between the position-sets. So a lens to $t_p \triangleleft t_p \triangleleft t_p$, such as those in the square above, is uniquely determined by its components to $p^{\triangleleft i} \triangleleft p^{\triangleleft j} \triangleleft p^{\triangleleft k}$ for all $i, j, k \in \mathbb{N}$, obtained by composing it with the projections $\epsilon_p^{(i)} \triangleleft \epsilon_p^{(j)} \triangleleft \epsilon_p^{(k)}$ out of the limit. Then by (8.32),

$$\begin{aligned} \delta_p \circ (t_p \triangleleft \delta_p) \circ (\epsilon_p^{(i)} \triangleleft \epsilon_p^{(j)} \triangleleft \epsilon_p^{(k)}) &= \delta_p \circ (\epsilon_p^{(i)} \triangleleft (\delta_p \circ (\epsilon_p^{(j)} \triangleleft \epsilon_p^{(k)}))) \\ &= \delta_p \circ (\epsilon_p^{(i)} \triangleleft \epsilon_p^{(j+k)}) \end{aligned} \quad (8.32)$$

$$= \epsilon_p^{(i+j+k)} \quad (8.32)$$

$$= \delta_p \circ (\epsilon_p^{(i+j)} \triangleleft \epsilon_p^{(k)}) \quad (8.32)$$

$$= \delta_p \circ \left((\delta_p \circ (\epsilon_p^{(i)} \triangleleft \epsilon_p^{(j)})) \triangleleft \epsilon_p^{(k)} \right) \quad (8.32)$$

$$= \delta_p \circ (\delta_p \triangleleft t_p) \circ (\epsilon_p^{(i)} \triangleleft \epsilon_p^{(j)} \triangleleft \epsilon_p^{(k)}).$$

for all $i, j, k \in \mathbb{N}$. So by the universal property of $t_p \triangleleft t_p \triangleleft t_p$, coassociativity holds. $\square$

We call the category $\mathcal{T}_p$ corresponding to the comonoid $(t_p, \epsilon_p, \delta_p)$ the *category of p -trees*, the *category of trees on p* , or the *p -tree category*.

Example 8.34 (The category of p -trees: states and transitions for (co)free). Let's step back and think about how p -tree categories relate to the original polynomial p from the perspective of the states and transitions of dynamical systems. Before we build any trees or categories out of it, a polynomial p is just a family of sets of directions indexed over a set of positions. The directions emerge from positions, but don't point to anywhere in particular. If we want to interpret the positions of p as states and the directions of p as composable transitions, first we would need to point the directions to other positions by assigning them codomains. There are many ways to do this, but we would like to do so "freely," without having to make any choices along the way that would bias

us one way or another, so as to give a canonical way to interpret p as a category.

So to avoid making choices, rather than fixing a codomain for each direction, we allow every possible direction to point to every possible position once. But a single direction of any one position cannot point to multiple positions, which is when we need to make several copies of each position: at least one copy for every possible combination of codomains that can be assigned to its directions. This is how we get from p to $p^{\triangleleft 2}$: the $p^{\triangleleft 2}$ -pretrees represent all the ways we could assign codomains to the directions in p . Essentially, we have freely refined our positions into more specific states to account for everywhere their directions could lead. Each $p^{\triangleleft 2}$ -pretree still remembers which $p^{\triangleleft 1}$ -pretree it grew from, giving us our canonical trimming operation $p^{\triangleleft 2}(1) \rightarrow p^{\triangleleft 1}(1)$.

Yet even that is not enough: sure, we've assigned codomains to the directions of p in every possible way, but now that we're building a category, we want our directions to be composable transitions. So each pair of composable directions of p —each length-2 rooted path of the $p^{\triangleleft 2}$ -pretrees—is now a possible transition as well, another morphism in our category. To keep everything canonical, we still want to avoid making any actual choices; we can't simply say that two directions of p compose to a third direction of p that we already have. Each pair of composable directions must be an entirely new morphism—and every new morphism needs a codomain.

You can tell where this is going. To avoid actually choosing codomains for the new morphisms, we need to refine our $p^{\triangleleft 2}$ -pretrees by making copies of them to account for every possibility, building $p^{\triangleleft 3}$ -pretrees as a result. Then their length-3 rooted paths are new morphisms, too, and they need new codomains, and so forth, ad infinitum. Indeed, this process cannot terminate in a finite number of steps, but that's okay—we can take the limit, yielding the ultimate free refinement of positions into states and free composites of directions as transitions.

This is where (8.1) comes from; it captures the infinite process of turning sequences of directions into morphisms by giving them

codomains in every possible way, then dealing with the longer sequences of directions that emerge as a result. Note that we also need to account for the fact that every object needs an identity; to avoid making a choice, we don't set it to be any direction or composite of multiple directions of p , reserving it instead for the empty sequence of directions that every pretree has. Then the limit t_p of (8.1) is all we need: the p -trees are states representing every possible way that sequences of directions emerging from a p -position can lead to other p -positions, and the rooted paths of each p -tree are transitions accounting for every finite sequence of composable directions from the corresponding state.

Since composites were freely generated, directions and thus entire rooted paths compose by concatenation—making the empty rooted path the correct identity. And the codomain is whatever the direction at the end of a rooted path has been freely assigned to point to—not just the p -position there but the whole p -subtree, an entire state representing all the sequences of directions one could take and the positions to which they lead, starting from the end of the path just followed. This gives the comonoid structure on t_p , completing the cofree construction of the category of p -trees $\mathcal{T}_p$.

Soon we will prove that $\mathcal{T}_p$ really is the cofree category on p , but first we give some examples to make all this concrete.

Examples of p -tree categories

In what follows, we will freely switch between the morphisms-as-vertices and the morphisms-as-rooted-paths perspectives of p -tree categories given in Proposition 8.33 whenever convenient.

Example 8.35 (The category of 1-trees). Let's start by taking $p := 1$. In Exercise 8.17 #1, we showed that there is a unique 1-tree: a tree with only 1 vertex, its root. So $t_1 \cong y$ is the carrier of the category of 1-trees $\mathcal{T}_1$.

Up to isomorphism, there is only one category carried by y : the terminal category with 1 object and no morphisms aside from the

1 identity. When we think of this as the category of 1-trees, we can characterize it as follows.

- The 1 object is the tree with 1 vertex: call the tree $\bullet$, because that's what it looks like.
- The 1 morphism $\bullet \rightarrow _$ is the 1 vertex of $\bullet$, its root. After all, there are no directions of 1 to freely compose, so the only morphism generated is the identity. Since the 1-subtree of $\bullet$ rooted as its root is just the entire 1-tree $\bullet$, the codomain of this morphism is still $\bullet$, which makes sense—it's the only object we have.
- The identity morphism $\text{id}_\bullet: \bullet \rightarrow \bullet$ is the root of $\bullet$. This is the same morphism we just mentioned. Equivalently, it is the empty rooted path.
- The composite morphism $\text{id}_\bullet \circ \text{id}_\bullet = \text{id}_\bullet \circ \text{id}_{\text{cod}(\text{id}_\bullet)}: \bullet \rightarrow _$ is obtained by concatenating the empty rooted path $\text{id}_\bullet$ of $\bullet$ with the empty rooted path $\text{id}_{\text{cod}(\text{id}_\bullet)}$ of $\text{cod}(\text{id}_\bullet)$, which is again just the empty rooted path of $\bullet$. Hence $\text{id}_\bullet \circ \text{id}_{\text{cod} \text{id}_\bullet} = \text{id}_\bullet$, as expected—it's the only morphism we have.

This is certainly too trivial an example to say much about, but it demonstrates how we can interpret the category carried by y as the category of p -trees for $p := 1$. Moreover, it helps us see concretely why taking the identity to be the root gives it the right codomain and compositional behavior.

Exercise 8.36 (The category of trees on a constant). Let B be a set, viewed as a constant polynomial.

1. What is the polynomial t_B ?
2. Characterize the B -tree category $\mathcal{T}_B$.

Example 8.37 (The category of y -trees). Now consider $p := y$. In Exercise 8.17 #3, we showed that there is a unique y -tree: a single ray extending from the root in which every vertex has exactly 1 child, so that there is exactly 1 height- n vertex—and 1 length- n rooted path—for every $n \in \mathbb{N}$. Then $t_y \cong y^{\mathbb{N}}$ is the carrier of the

category of y -trees $\mathcal{T}_y$. In fact, we can identify the set of rooted paths of this y -tree with $\mathbb{N}$, so that $n \in \mathbb{N}$ is the y -tree's unique length- n rooted path.

We know from Example 7.40 that comonoids with representable carriers correspond to 1-object categories, which can be identified with monoids. In particular, a comonoid structure on $y^{\mathbb{N}}$ corresponds to a monoid structure on $\mathbb{N}$. There is more than one monoid structure on $\mathbb{N}$, though, so which one corresponds to the category $\mathcal{T}_y$?

We can characterize $\mathcal{T}_y$ in terms of y -trees as follows.

- The 1 object is the unique y -tree, a single ray: call it $\uparrow$. Here is a picture of this ray, to help you visualize its vertices, rooted paths, and subtrees:



- The morphisms are the rooted paths of $\uparrow$; they comprise the set $\mathbb{N}$, where each $n \in \mathbb{N}$ is the unique rooted path of $\uparrow$ of length n . Each rooted path represents the free composite of n copies of the sole direction of y . Since $\uparrow$ is an infinite ray, the y -subtree of $\uparrow$ rooted at any of its vertices is still just a copy of $\uparrow$. So the codomain of each morphism is still $\uparrow$, which makes sense—it's the only object we have.
- The identity morphism $\text{id}_{\uparrow}: \uparrow \rightarrow \uparrow$ is the empty rooted path, which has length 0; so $\text{id}_{\uparrow} = 0$. In the corresponding monoid structure on $\mathbb{N}$, the element $0 \in \mathbb{N}$ must then be the unit.
- The composite morphism $m \circ n: \uparrow \rightarrow \uparrow$ for $m, n \in \mathbb{N}$ is obtained by concatenating m , i.e. the length- m rooted path of $\uparrow$, with n , i.e. the length- n rooted path of $\uparrow$, translating the latter path so that it begins where the former path ends. The result is then

the rooted path of $\uparrow$ of length $m + n$; so $m \circ n = m + n$. In the corresponding monoid structure on $\mathbb{N}$, the binary operation must then be given by addition.

Hence $\mathcal{T}_y$ is the monoid $(\mathbb{N}, 0, +)$ viewed as a 1-object category.

Example 8.38 (By-trees are B-streams). Let B be a set, and consider $p := By$. Generalizing Exercise 8.17 #5 for B instead of 2, or applying Exercise 8.17 #7 for the case of $A := 1$, we can deduce that a By -tree consists of a single ray for which every vertex is given a label from B . So the vertices of a By -tree are in bijection with $\mathbb{N}$, and By -trees are in bijection with functions $\mathbb{N} \rightarrow B$ assigning each vertex a label. Hence $t_{By} \cong B^{\mathbb{N}}y^{\mathbb{N}}$ is the carrier of the category of By -trees $\mathcal{T}_{By}$. As in Example 8.37, we identify the set of rooted paths of a given By -tree with $\mathbb{N}$, so that $n \in \mathbb{N}$ is the By -tree's unique length- n rooted path.

In fact, we have already seen the category $\mathcal{T}_{By}$ once before: it is the category of B -streams from Example 7.45.

- Recall that a B -stream is an element of $B^{\mathbb{N}}$ interpreted as a countable sequence of elements $b_n \in B$ for $n \in \mathbb{N}$, written like so:

$$\bar{b} := (b_0 \rightarrow b_1 \rightarrow b_2 \rightarrow b_3 \rightarrow \cdots).$$

This is just a By -tree lying on its side! Each arrow is a copy of the sole direction at each position of By , pointing to one of the positions in B for that direction to lead to next.

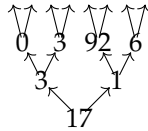
- Given a B -stream $\bar{b} \in B^{\mathbb{N}}$ above, recall that a morphism out of $\bar{b}$ is a natural number $n \in \mathbb{N}$, and its codomain is the substream of B starting at b_n :

$$\text{cod}(\bar{b} \xrightarrow{n} _) = (b_n \rightarrow b_{n+1} \rightarrow b_{n+2} \rightarrow b_{n+3} \rightarrow \cdots).$$

This coincides with how we view morphisms as rooted paths and codomains as subtrees rooted at the end of those paths in the category of By -trees.

- Whether we view $\bar{b}$ as a B -stream or a By -tree, its identity morphism $\text{id}_{\bar{b}}: \bar{b} \rightarrow \bar{b}$ corresponds to $0 \in \mathbb{N}$; from the latter perspective, it is the length-0 path from the root to itself.
- Whether we view $\bar{b}$ as a B -stream or a By -tree, composition is given by addition; from the latter perspective, concatenating a length- m rooted path with a length- n path yields a length- $(m + n)$ rooted path.

Example 8.39 (The category of $\mathbb{N}$ -labeled binary trees). Consider $p := \mathbb{N}y^2$. By Exercise 8.17 #7, or Exercise 8.12 #2 in the case of $L := \mathbb{N}$ and $n := 2$, an $\mathbb{N}y^2$ -tree is an infinite binary tree with vertices labeled by elements of $\mathbb{N}$. Here's how such an $\mathbb{N}y^2$ -tree might start:



We can characterize the category $\mathcal{T}_{\mathbb{N}y^2}$ as follows.

- The objects are $\mathbb{N}$ -labeled binary trees from the set $\mathbb{N}^{\text{List}(2)}$.
- A morphism out of an $\mathbb{N}$ -labeled binary tree is a binary sequence: a finite list of directions in 2, whose elements could be interpreted as “left” and “right,” thus uniquely specifying a rooted path in a binary tree. They comprise the set $\text{List}(2)$. The codomain of each rooted path is the $\mathbb{N}$ -labeled binary subtree rooted at the end of the path.
- The identity morphism on a given $\mathbb{N}$ -labeled binary tree is its empty rooted path.
- The composite of two binary sequences is obtained by concatenation.

Exercise 8.40 (The category of B -labeled A -ary trees). Characterize the By^A -tree category $\mathcal{T}_{By^A}$.

Exercise 8.41. Characterize the $(y + 1)$ -tree category $\mathcal{T}_{y+1}$.

Exercise 8.42. Let $p := \{a, b, c, \dots, z, \sqcup\}y + \{\bullet\}$.

1. Describe the objects of the cofree category $\mathcal{T}_p$, and draw one.
2. For a given such object, describe the set of emanating morphisms.
3. Describe how to take the codomain of a morphism.

Exercise 8.43. Let $p := \{\blacklozenge, \blacksquare\}y^2 + \{\bullet\}y + \{\blacktriangle\}$ as in Example 8.10.

1. Choose an object $t \in \text{tree}_p$, i.e. a tree in p , and draw a finite approximation of it (say four levels).
2. What is the identity morphism at t ?
3. Choose a nonidentity morphism f emanating from t and draw it.
4. What is the codomain of f ? Draw a finite approximation of it.
5. Choose a morphism emanating from the codomain of f and draw it.
6. What is the composite of your two morphisms? Draw it on t .

Exercise 8.44. Let $\mathbb{Q}_{\geq 0} := \{q \in \mathbb{Q} \mid q \geq 0\}$ and consider the monoid $y^{\mathbb{Q}_{\geq 0}}$ of nonnegative rational numbers under addition. Is it true that given $p \in \mathbf{Poly}$, any retrofunctor $\varphi: \mathcal{T}_p \rightarrow y^{\mathbb{Q}_{\geq 0}}$ factors through y as

$$\mathcal{T}_p \rightarrow y \rightarrow y^{\mathbb{Q}_{\geq 0}}?$$

8.1.3 Exhibiting the forgetful-cofree adjunction

We are now ready to give a diagrammatic proof of the main result of this section: as promised, the category $\mathcal{T}_p$ we constructed is the cofree comonoid on p .

Theorem 8.45 (Cofree comonoid). The forgetful functor $U: \mathbf{Cat}^\# \rightarrow \mathbf{Poly}$ has a right adjoint $\mathcal{T}_-: \mathbf{Poly} \rightarrow \mathbf{Cat}^\#$, giving rise to an adjunction

$$\mathbf{Cat}^\# \begin{array}{c} \xrightarrow{U} \\ \rightleftarrows \\ \xleftarrow{\mathcal{T}_-} \end{array} \mathbf{Poly} ,$$

such that for each $p \in \mathbf{Poly}$, the carrier $t_p := U\mathcal{T}_p$ of the category $\mathcal{T}_p$ is given by the limit of the diagram (8.1), repeated here:

$$\begin{array}{ccccccc} y & & p & & p^{\triangleleft 2} & & p^{\triangleleft 3} & & \dots \\ \downarrow & & \downarrow & & \downarrow & & \downarrow & & \\ 1 & \xleftarrow{!} & p \triangleleft 1 & \xleftarrow{p^{\triangleleft 1}!} & p^{\triangleleft 2} \triangleleft 1 & \xleftarrow{p^{\triangleleft 2} \triangleleft 1!} & p^{\triangleleft 3} \triangleleft 1 & \xleftarrow{\dots} & \dots \end{array}$$

That is, for any category $\mathcal{C} \in \mathbf{Cat}^\#$ with carrier $\mathfrak{c} := U\mathcal{C}$, there is a natural isomorphism

$$\mathbf{Poly}(\mathfrak{c}, p) \cong \mathbf{Cat}^\#(\mathcal{C}, \mathcal{T}_p).$$

Proof. To show that $\mathcal{T}_-$ is the right adjoint of U , it is enough to show that for every lens $\varphi: \mathfrak{c} \rightarrow p$, there exists a unique retrofunctor $F: \mathcal{C} \rightarrow \mathcal{T}_p$ for which

$$\begin{array}{ccc} \mathfrak{c} & & \\ \downarrow F & \searrow \varphi & \\ t_p & \xrightarrow{\epsilon_p^{(1)}} & p \end{array} \quad (8.46)$$

commutes. Here the projection $\epsilon_p^{(1)}: U\mathcal{T}_p \cong t_p \rightarrow p$ serves as the counit of the adjunction, and we identify the retrofunctor F with its underlying lens $UF: \mathfrak{c} \rightarrow t_p$.

First, we construct F from φ as follows. If we let ϵ and δ be the eraser

and duplicator of $\mathcal{C}$, the diagram

$$\begin{array}{ccc}
 c & \xrightarrow{\delta} & c \triangleleft c \\
 \epsilon \downarrow & \searrow \varphi & \downarrow \epsilon \triangleleft \varphi \\
 y & & p \\
 \downarrow & & \downarrow \\
 1 & \longleftarrow & p \triangleleft 1
 \end{array} \quad (8.47)$$

commutes: the pentagon in the lower left commutes trivially, while the triangle in the upper right commutes by the left erasure law of $\mathcal{C}$, as

$$\begin{aligned}
 \delta \circ (\epsilon \triangleleft \varphi) &= \delta \circ (\epsilon \triangleleft c) \circ \varphi \\
 &= \text{id}_c \circ \varphi && \text{(Left erasure law)} \\
 &= \varphi.
 \end{aligned}$$

Then by induction, the larger diagram

$$\begin{array}{ccccccc}
 c & \xrightarrow{\delta} & c \triangleleft c & \xrightarrow{\delta \triangleleft c} & c^{\triangleleft 3} & \xrightarrow{\delta \triangleleft c^{\triangleleft 2}} & c^{\triangleleft 4} \longrightarrow \dots \\
 \epsilon \downarrow & \searrow \varphi & \downarrow \epsilon \triangleleft \varphi & & \downarrow \epsilon \triangleleft \varphi^{\triangleleft 2} & & \downarrow \epsilon \triangleleft \varphi^{\triangleleft 3} \\
 y & & p & & p^{\triangleleft 2} & & p^{\triangleleft 3} \quad \dots \\
 \downarrow & & \downarrow & & \downarrow & & \downarrow \\
 1 & \longleftarrow & p \triangleleft 1 & \longleftarrow & p^{\triangleleft 2} \triangleleft 1 & \longleftarrow & p^{\triangleleft 3} \triangleleft 1 \longleftarrow \dots
 \end{array} \quad (8.48)$$

commutes as well: its leftmost rectangle is (8.47), and taking the composition product of each rectangle in (8.48) with the commutative rectangle

$$\begin{array}{ccc}
 c & \xlongequal{\quad} & c \\
 \varphi \downarrow & & \downarrow \varphi \\
 p & & p \\
 \parallel & & \parallel \\
 p & \xlongequal{\quad} & p
 \end{array}$$

yields the rectangle to its right. As t_p is the limit of the bottom two rows of (8.48), it follows that there is an induced lens $F: c \rightarrow t_p$ that, when composed with each projection $\epsilon_p^{(n)}: t_p \rightarrow p^{\triangleleft n}$, yields the lens

depicted in (8.48) from $\mathfrak{c}$ to $p^{\triangleleft n}$. This lens is the composite of the lens $\mathfrak{c} \rightarrow \mathfrak{c}^{\triangleleft(n+1)}$ in the top row, which by Proposition 7.20 is the canonical lens $\delta^{(n+1)}$ associated with the comonoid $\mathcal{C}$, composed with the lens $\epsilon \triangleleft \varphi^{\triangleleft n}: \mathfrak{c}^{\triangleleft(n+1)} \rightarrow p^{\triangleleft n}$.

Next, we prove uniqueness: that a retrofunctor $\mathcal{C} \rightarrow \mathcal{T}_p$ with underlying lens $f: \mathfrak{c} \rightarrow \mathfrak{t}_p$ is completely determined by the value of $f \circ \epsilon_p^{(1)}$. It suffices to show that we can recover the n^{th} component of g from its first component. **

□

8.1.4 The many (inter)faces of the cofree comonoid

The forgetful-cofree adjunction of Theorem 8.45 tells us that given a category $\mathcal{C} \in \mathbf{Cat}^\sharp$ with carrier $\mathfrak{c} := U\mathcal{C}$ and a polynomial $p \in \mathbf{Poly}$, there is a natural isomorphism

$$\mathbf{Poly}(\mathfrak{c}, p) \cong \mathbf{Cat}^\sharp(\mathcal{C}, \mathcal{T}_p).$$

So every lens $\varphi: \mathfrak{c} \rightarrow p$ has a corresponding retrofunctor $F: \mathcal{C} \rightarrow \mathcal{T}_p$ that we call its *mate*.

We can view φ as a dynamical system with an interface p and a generalized state system $\mathfrak{c}$, carrying an arbitrary category $\mathcal{C}$ of states and transitions. Then the lens $\text{Run}_n(\varphi): \mathfrak{c} \rightarrow p^{\triangleleft n}$ for $n \in \mathbb{N}$, defined as the composite

$$\mathfrak{c} \xrightarrow{\delta^{(n)}} \mathfrak{c}^{\triangleleft n} \xrightarrow{\varphi^{\triangleleft n}} p^{\triangleleft n},$$

models n runs through the system φ .

Meanwhile, as the limit of the diagram (8.1) with the row of polynomials of the form $p^{\triangleleft n}$ across the top, the carrier $\mathfrak{t}_p$ of the cofree comonoid on p comes equipped with a lens $\epsilon_p^{(n)}: \mathfrak{t}_p \rightarrow p^{\triangleleft n}$ for each $n \in \mathbb{N}$. Since the retrofunctor $F: \mathcal{C} \rightarrow \mathcal{T}_p$ has an underlying lens between carriers $f := UF: \mathfrak{c} \rightarrow \mathfrak{t}_p$, we can obtain another lens $\mathfrak{c} \rightarrow p^{\triangleleft n}$ as the composite

$$\mathfrak{c} \xrightarrow{f} \mathfrak{t}_p \xrightarrow{\epsilon_p^{(n)}} p^{\triangleleft n}.$$

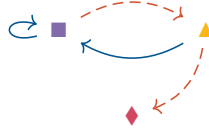
It follows from our forgetful-cofree adjunction that these two composites are equal.

Proposition 8.49. With the definitions above, the following diagram commutes:

$$\begin{array}{ccc}
 c & \xrightarrow{f} & t_p \\
 \delta^{(n)} \downarrow & & \downarrow \epsilon_p^{(n)} \\
 c^{\triangleleft n} & \xrightarrow{\varphi^{\triangleleft n}} & p^{\triangleleft n}.
 \end{array}$$

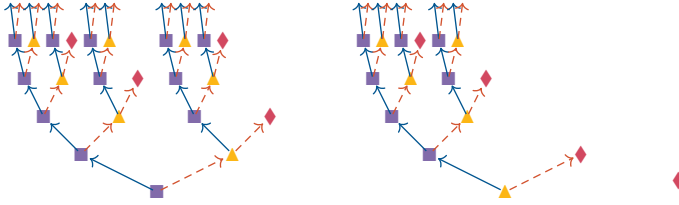
Now we have a better sense of what we mean when we say that $F: \mathcal{C} \rightarrow \mathcal{T}_p$ captures all the information that the lenses $\text{Run}_n(\varphi): c \rightarrow p^{\triangleleft n}$ encode: the category $\mathcal{T}_p$ is carried by a polynomial t_p equipped with lenses $\epsilon_p^{(n)}: t_p \rightarrow p^{\triangleleft n}$, each of which exposes a part of the category as the n -fold interface $p^{\triangleleft n}$. All together, t_p acts as a giant interface that captures the n -fold behavior of $p^{\triangleleft n}$ for every $n \in \mathbb{N}$. But to see all this explicitly, let's consider some examples.

Example 8.50. Let $S := \{\blacksquare, \blacktriangle, \blacklozenge\}$ and $p := y^2 + 1$, and consider the dynamical system $\varphi: Sy^S \rightarrow p$ modeling the halting deterministic state automaton from Exercise 4.23, depicted here again for your convenience:



Under the forgetful-cofree adjunction, the lens φ coincides with a retrofunctor $F: Sy^S \rightarrow \mathcal{T}_p$ from the state category on S to the category of p -trees. The retrofunctor sends each state in $S = \{\blacksquare, \blacktriangle, \blacklozenge\}$ to a p -tree; these p -trees are drawn below (the first two are infinite). Then the vertices of each p -tree are sent back to a morphism in the state category; the shape and color of each vertex indicates the

codomain of the morphism to which that vertex is sent.



The p -trees above, with elements of S as vertices, thus encode all the ways to navigate the automaton. In particular, the *maximal rooted paths* of these trees (i.e. those that terminate at a leaf) trace out all the ways in which the automaton can halt, and therefore all the words that the automaton accepts. Notice, too, that every p -subtree of any one of these p -trees is another one of these three p -trees.

In general, given a lens $\varphi: Sy^S \rightarrow y^A + 1$ modeling a halting deterministic state automaton and an initial state $s_0 \in S$, the $(y^A + 1)$ -tree to which the corresponding retrofunctor $F: Sy^S \rightarrow \mathcal{T}_{y^A + 1}$ sends s_0 encodes the set of words accepted by the automaton with that initial state in its maximal rooted paths.

Example 8.51 (Language recognized by deterministic state automata).

Recall from Proposition 4.17 that a deterministic state automaton with a set of states S and a set of symbols A can be identified with a lens $y \rightarrow Sy^S$, indicating the initial state $s_0 \in S$, and a lens $\varphi: Sy^S \rightarrow 2y^A$, indicating the subset of accept states $F := \varphi_1^{-1}(2) \subseteq S$ and the update function $u: S \times A \rightarrow S$ via $u(s, a) = \varphi_s^\#(a)$.

Under the forgetful-cofree adjunction, the lens φ coincides with a retrofunctor $F: Sy^S \rightarrow \mathcal{T}_{2y^A}$. By Exercise 8.40, $\mathcal{T}_{2y^A}$ is the category of 2-labeled A -ary trees. So $F(s_0)$ is an element of $\text{tree}_{2y^A} \cong 2^{\text{List}(A)}$: an A -ary tree where each rooted path corresponds to a list of elements of A and bears one of the elements of 2, indicating whether the course through the automaton corresponding to that rooted

path ends at an accept state or not. Equivalently, an element of $2^{\text{List}(A)}$ is a subset of $\text{List}(A)$; in the case of $F(s_0)$, it is the subset containing exactly the words in $\text{List}(A)$ accepted by the automaton. So on objects, F sends each possible start state $s_0 \in S$ of the automaton to the subset of words that the automaton would then accept! Backward on morphisms, $F_{s_0}^\#$ sends every possible word to the state the automaton would reach if it followed that word starting from s_0 .

Example 8.52 (Direction sequences to position sequences). We interpret our dynamical systems as converting sequences of directions to sequences of positions. The forgetful-cofree adjunction allows us to express this conversion formally in the language of **Poly**. For convenience, we'll focus on the example of an (A, B) -Moore machine $\varphi: Sy^S \rightarrow By^A$, although of course we can generalize this beyond monomial interfaces.

The lens φ corresponds to a retrofunctor $F: Sy^S \rightleftarrows \mathcal{T}_{By^A}$. By Exercise 8.40, $\mathcal{T}_{By^A}$ is the category of B -labeled A -ary trees; in particular, its carrier is $t_{By^A} \cong B^{\text{List}(A)} y^{\text{List}(A)}$.

Then for every initial state $s_0 \in S$, the B -labeled A -ary tree $F(s_0)$ can be interpreted as a decision tree of all the possible sequences of directions that the system may receive. The label in B corresponding to the vertex (or rooted path) specified by each sequence in $\text{List}(A)$ tells us the final position that the system returns when that sequence is fed in as directions. Put another way, $F(s_0) \in B^{\text{List}(A)}$ can be interpreted as a function $\text{List}(A) \rightarrow B$. So if the direction sequence is $(a_1, \dots, a_n) \in A^n \subseteq \text{List}(A)$, then the corresponding position sequence $(b_0, \dots, b_n) \in B^{n+1}$ is given (non-recursively!) by

$$b_i := F(s_0)(a_1, \dots, a_i).$$

Finally, $F_{s_0}^\#(a_1, \dots, a_i) \in S$ then corresponds to the system's state after that sequence of directions is given.

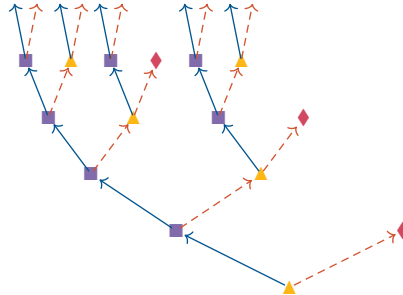
Example 8.53. In Example 8.50 we had a dynamical system with $S := \{\blacksquare, \blacktriangle, \blacklozenge\}$ and $p := y^2 + 1$, and $\varphi: Sy^S \rightarrow p$ from Exercise 4.23, depicted here again for your convenience:



(8.54)

Under the forgetful-cofree adjunction (Theorem 8.45), the lens φ coincides with a retrofunctor $F: Sy^S \rightarrow \mathcal{T}_p$ from the state category on S to the category of p -trees. We can now see this as a copresheaf on the category $\mathcal{T}_p$ itself.

The cofree category $\mathcal{T}_t$ is actually the free category on a graph, as we will see in Proposition 8.57, and so the schema is easy. There is one table for each tree (object in $\mathcal{T}_p$), e.g. we have a table associated to this tree:



The table has two columns, say left and right, corresponding to the two arrows emanating from the root node. The left column refers back to the same table, and the right column refers to another table (the one corresponding to the yellow dot).

Again, there are infinitely many tables in this schema. Only three of them have data in them; the rest are empty. We know in advance that this instance has three rows in total, since $|S| = 3$.

8.1.5 Morphisms between cofree comonoids

Given a lens $\varphi: p \rightarrow q$, the cofree functor gives us a comonoid morphism $\mathcal{T}_\varphi: \mathcal{T}_p \rightarrow \mathcal{T}_q$ as follows.

An object $t \in \text{tree}_p$ is a tree; the tree $u := \mathcal{T}_\varphi(t) \in \text{tree}_q$ is constructed recursively as follows. If the root of t is $i \in p(1)$ then the root of u is $j := \varphi_1(i)$. To each branch $b \in q[j]$, we need to assign a new tree, and we use the one situated at $\varphi_i^\sharp(b)$.

Exercise 8.55. Let $p := \{\blacksquare\}y^2 + \{\blacklozenge\}$ and $q := \{\blacksquare, \blacktriangle\}y + \{\blacklozenge, \bullet\}$.

1. Choose a lens $\varphi: p \rightarrow q$, and write it out.
2. Choose a tree $T \in \text{tree}_p$ with at least height 3.
3. What is $\mathcal{T}_\varphi(T)$?

Exercise 8.56. Let p be a polynomial.

1. Show there is an induced retrofunctor $\mathcal{T}_p \rightarrow \mathcal{T}_{p^{*n}}$ for all $n \in \mathbb{N}$.
2. When $n \geq 1$, is the induced retrofunctor an isomorphism?

8.1.6 Some categorical properties of cofree comonoids

Proposition 8.57. For every polynomial p , the cofree category $\mathcal{T}_p$ is free on a graph. That is, there is a graph G_p whose associated free category in the usual sense (the category of vertices and paths in G_p) is isomorphic to $\mathcal{T}_p$.

Proof. For vertices, we let V_p denote the set of p -trees,

$$V_p := \text{tree}_p(1).$$

For arrows we use the counit lens $\pi: \text{tree}_p \rightarrow p$ from Theorem 8.45 to define

$$A_p := \sum_{t \in \text{tree}_p(1)} p[\pi_1(t)].$$

In other words A_p is the set $\{d \in p[\pi_1(t)] \mid t \in \text{tree}_p\}$ of directions in p that emanate from the root corolla of each p -tree. The source of (t, d)

is t and the target is $\text{cod}(\pi_t^\sharp(d))$. It is clear that every morphism in $\mathcal{T}_t$ is the composite of a finite sequence of such morphisms, completing the proof. $\square$

Corollary 8.58. Let p be a polynomial and $\mathcal{T}_p$ the cofree comonoid. Every morphism in $\mathcal{C}_p$ is both monic and epic.

Proof. The free category on a graph always has this property, so the result follows from Proposition 8.57. $\square$

Proposition 8.59. The additive monoid $y^{\mathbb{N}}$ of natural numbers has a $\times$ -monoid structure in $\mathbf{Cat}^\sharp$.

Proof. The right adjoint $p \mapsto \mathcal{T}_p$ preserves products, so $y^{\text{List}(n)} \cong \mathcal{T}_{y^n}$ is the n -fold product of $y^{\mathbb{N}}$ in $\mathbf{Cat}^\sharp$. We thus want to find retrofunctors $e: y \rightarrow y^{\mathbb{N}}$ and $m: y^{\text{List}(2)} \rightarrow y^{\mathbb{N}}$ that satisfy the axioms of a monoid.

The unique lens $y \rightarrow y^{\mathbb{N}}$ is a retrofunctor (it is the mate of the identity $y \rightarrow y$). We take m to be the mate of the lens $y^{\text{List}(2)} \rightarrow y$ given by the list $[1, 2]$. One can check by hand that these definitions make $(y^{\mathbb{N}}, e, m)$ a monoid in $(\mathbf{Cat}^\sharp, y, \times)$. $\square$

Recall from Example 7.71 that an arrow field of a category $\mathcal{C}$ is a retrofunctor $\mathcal{C} \rightarrow y^{\mathbb{N}}$.

Corollary 8.60. For any category $\mathcal{C}$, the set $\mathbf{Cat}^\sharp(\mathcal{C}, y^{\mathbb{N}})$ of arrow fields has the structure of a monoid. Moreover, this construction is functorial: that is, there is a functor

$$\mathbf{Cat}^\sharp(-, y^{\mathbb{N}}): \mathbf{Cat}^\sharp \rightarrow \mathbf{Mon}^{\text{op}}.$$

Proof. We saw that $y^{\mathbb{N}}$ is a monoid object in Proposition 8.59. $\square$

A retrofunctor $\mathcal{C} \rightarrow y^{\mathbb{N}}$ is a policy in $\mathcal{C}$: it assigns an outgoing morphism to each object of $\mathcal{C}$. Any two such trajectories can be multiplied: we simply do one and then the other; this is the monoid operation. The policy assigning the identity to each object is the unit of the monoid.

We use the notation $\mathcal{C} \mapsto \vec{\mathcal{C}}$ for the monoid of arrow fields.

Theorem 8.61. The arrow fields functor

$$\mathbf{Cat}^\# \rightarrow \mathbf{Mon}^{\text{op}}$$

is right adjoint to the inclusion $\mathbf{Mon}^{\text{op}} \rightarrow \mathbf{Cat}^\#$ from Proposition 7.79.

Proof. Let $\mathcal{C}$ be a category and $(M, e, *)$ a monoid. A retrofunctor $F: \mathcal{C} \rightarrow y^M$ has no data on objects; it is just a way to assign to each $c \in \mathcal{C}$ and $m \in M$ a morphism $F_c^\#(m): c \rightarrow c'$ for some $c' := \text{cod}(F_c^\#(m))$. This assignment must send identities to identities and composites to composites: given $m' \in M$ we have $F_c^\#(m \circ m') = F_c^\#(m) \circ F_{c'}^\#(m')$. This is exactly the data of a monoid morphism $M \rightarrow \tilde{\mathcal{C}}$: it assigns to each $m \in M$ an arrow field $\mathcal{C}$, preserving unit and multiplication. $\square$

Proposition 8.62. There is a commutative square of left adjoints

$$\begin{array}{ccc} \mathbf{Mon}^{\text{op}} & \xrightarrow{U} & \mathbf{Set}^{\text{op}} \\ y^- \downarrow & & \downarrow y^- \\ \mathbf{Cat}^\# & \xrightarrow{U} & \mathbf{Poly} \end{array}$$

where the functors denoted U are forgetful functors.

Proof. Using the fully faithful functor $y^-: \mathbf{Mon}^{\text{op}} \hookrightarrow \mathbf{Cat}^\#$ from Proposition 7.79, it is easy to check that the above diagram commutes.

The free-forgetful adjunction $\mathbf{Set} \hookrightarrow \mathbf{Mon}$ gives an opposite adjunction $\mathbf{Set}^{\text{op}} \hookrightarrow \mathbf{Mon}^{\text{op}}$, where U is now left adjoint. We saw that $y^-: \mathbf{Set}^{\text{op}} \rightarrow \mathbf{Poly}$ is a left adjoint in Proposition 5.12, that $U: \mathbf{Cat}^\# \rightarrow \mathbf{Poly}$ is a left adjoint in Theorem 8.45, and that $y^-: \mathbf{Mon} \rightarrow \mathbf{Cat}^\#$ is a left adjoint in Theorem 8.61. $\square$

8.2 More categorical properties of $\mathbf{Cat}^\#$

Many of the properties of **Poly** we covered in Chapters 3 and 5 have analogues in $\mathbf{Cat}^\#$; we review these here.

8.2.1 Other special comonoids and adjunctions

We begin by highlighting a few other adjunctions involving $\mathbf{Cat}^\sharp$, as well as the special comonoids in $\mathbf{Cat}^\sharp$ they provide.

Proposition 8.63. The functor y^- from Proposition 8.62 factors through an isomorphism of categories

$$\mathbf{Cat}_{\text{rep}}^\sharp \cong \mathbf{Mon}^{\text{op}},$$

where $\mathbf{Mon}$ is the category of monoids and monoid homomorphisms and $\mathbf{Cat}_{\text{rep}}^\sharp$ is the full subcategory of $\mathbf{Cat}^\sharp$ consisting of categories with representable carriers y^M for some $M \in \mathbf{Set}$.

Proof. Let $\mathcal{C}$ be a category. It has only one object iff its carrier $\mathfrak{c}$ has only one position, i.e. $\mathfrak{c} \cong y^M$ for some $M \in \mathbf{Set}$, namely where M is the set of morphisms in $\mathcal{C}$. It remains to show that retrofunctors between monoids are dual—opposite—to morphisms between monoids.

A retrofunctor $f: y^M \rightarrow y^N$ involves a single function $f^\sharp: N \rightarrow M$ that must satisfy a law coming from unitality and one coming from composition, as in Definition 7.49. The result can now be checked by hand, or seen formally as follows. Each object in the two diagrams of (7.49) is representable by Exercise 6.8. The Yoneda embedding $\mathbf{Set}^{\text{op}} \rightarrow \mathbf{Poly}$ is fully faithful, so these two diagrams are equivalent to the unit and composition diagrams for monoid homomorphisms. $\square$

Exercise 8.64. Let $\mathbf{Cat}_{\text{lin}}^\sharp$ be the full subcategory of $\mathbf{Cat}^\sharp$ consisting of categories with linear carriers Sy for some $S \in \mathbf{Set}$. Show that there is an isomorphism of categories

$$\mathbf{Cat}_{\text{lin}}^\sharp \cong \mathbf{Set}.$$

Proposition 8.65 (Discrete categories). The inclusion $\mathbf{Cat}_{\text{lin}}^\sharp \rightarrow \mathbf{Cat}^\sharp$

has a left adjoint sending each $(c, \epsilon, \delta) \in \mathbf{Cat}^\sharp$ to the unique comonoid carried by $(c \triangleleft 1)y$ in $\mathbf{Cat}_{\text{lin}}^\sharp$.

Proof. We need to show that for any comonoid (c, ϵ, δ) and set A , we have a natural isomorphism

$$\mathbf{Cat}^\sharp(c, Ay). \cong \mathbf{Cat}^\sharp((c \triangleleft 1)y, Ay).$$

But every morphism in Ay is an identity, so the result follows from the fact that every retrofunctor must pass identities back to identities. $\square$

8.2.2 Vertical-cartesian factorization of retrofunctors

A retrofunctor is called *cartesian* if the underlying lens $f: c \rightarrow d$ is cartesian (i.e. for each position $i \in c(1)$, the map $f_i^\sharp: d[f_1(i)] \rightarrow c[i]$ is an isomorphism).

Proposition 8.66. Every retrofunctor $f: C \rightarrow D$ factors as a vertical morphism followed by a cartesian morphism

$$C \xrightarrow{\text{vert}} C' \xrightarrow{\text{cart}} D.$$

Proof. A retrofunctor $C \rightarrow D$ is a lens $c \rightarrow d$ satisfying some properties, and any lens $f: c \rightarrow d$ can be factored as a vertical morphism followed by a cartesian morphism

$$c \xrightarrow{g} c' \xrightarrow{h} d.$$

For simplicity, assume $g_1: c(1) \rightarrow c'(1)$ is identity (rather than merely isomorphism) on positions and similarly that for each $i \in c$ the map $h_i^\sharp: c'[i] \rightarrow d[h_1(i)]$ is identity (rather than merely isomorphism) on directions.

It suffices to show that the intermediate object c' can be endowed with the structure of a category such that g and h are retrofunctors. Given an object $i \in c'(1)$, assign its identity to be the identity on $h_1(i) = f(i)$; then both g and h preserve identities because f does. Given an emanating morphism $m \in c'[i] = d[f(i)]$, assign its codomain to be $\text{cod}(m) := \text{cod}(f_i^\sharp(m))$, and given an emanating morphism $m' \in c'[\text{cod}(m)]$, assign the composite $m \circ m'$ in c' to be $m \circ m'$ in d . In Exercise 8.67 we will

check that with these definitions, $\mathcal{C}'$ is a category and both g and h are retrofunctors. $\square$

Exercise 8.67. We will complete the proof of Proposition 8.66, using the same notation.

1. Show that composition is associative and unital in $\mathcal{C}'$.
2. Show that g preserves codomains.
3. Show that g preserves compositions.
4. Show that h preserves codomains.
5. Show that h preserves compositions.

Proposition 8.68. The wide³ subcategory of cartesian retrofunctors in $\mathbf{Cat}^\sharp$ is isomorphic to the wide subcategory of discrete opfibrations in $\mathbf{Cat}$.

Proof. Suppose that $\mathcal{C}$ and $\mathcal{D}$ are categories. Both a functor and a retrofunctor between them involve a map on objects, say $f: \text{Ob } \mathcal{C} \rightarrow \text{Ob } \mathcal{D}$. For any object $c \in \text{Ob } \mathcal{C}$, a functor gives a function, say $f_\sharp: \mathcal{C}[c] \rightarrow \mathcal{D}[f(c)]$ whereas a retrofunctor gives a function $f^\sharp: \mathcal{D}[f(c)] \rightarrow \mathcal{C}[c]$. The retrofunctor is cartesian iff $f^\sharp$ is an iso, and the functor is a discrete opfibration iff $f_\sharp$ is an iso. We thus transform our functor into a retrofunctor (or vice versa) by taking the inverse function on morphisms. It is easy to check that this inverse appropriately preserves identities, codomains, and compositions. $\square$

Proposition 8.69. The wide subcategory of vertical maps in $\mathbf{Cat}^\sharp$ is isomorphic to the opposite of the wide subcategory bijective-on-objects maps in $\mathbf{Cat}$:

$$\mathbf{Cat}_{\text{vert}}^\sharp \cong (\mathbf{Cat}_{\text{boo}})^{\text{op}}.$$

³A subcategory $\mathbf{D}$ of a category $\mathbf{C}$ is *wide* if every object of $\mathbf{C}$ is in $\mathbf{D}$.

Proof. Let $\mathcal{C}$ and $\mathcal{D}$ be categories. Given a vertical retrofunctor $F: \mathcal{C} \rightarrow \mathcal{D}$, we have a bijection $F_1: \text{Ob } \mathcal{C} \rightarrow \text{Ob } \mathcal{D}$; let G_1 be its inverse. We define a functor $G: \mathcal{D} \rightarrow \mathcal{C}$ on objects by G_1 and, for any $f: d \rightarrow d'$ in $\mathcal{D}$ we define $G(f) := F_{G_1(d)}^\#$. It has the correct codomain: $\text{cod}(G(f)) = G_1(F_1(\text{cod}(G(f)))) = G_1(\text{cod } f)$. And it sends identities and compositions to identities and compositions by the laws of retrofunctors.

The construction of a vertical retrofunctor from a bijective-on-objects functor is analogous, and it is easy to check that the two constructions are inverses. $\square$

Exercise 8.70. Let S be a set and consider the state category $\mathcal{S} := (S y^S, \epsilon, \delta)$. Use Proposition 8.69 to show that categories $\mathcal{C}$ equipped with a vertical retrofunctor $\mathcal{S} \rightarrow \mathcal{C}$ can be identified with categories whose set of objects has been identified with S .

Exercise 8.71. Consider the categories $\mathcal{C} := \boxed{\bullet \rightrightarrows \bullet}$ and $\mathcal{D} := \boxed{\bullet \rightarrow \bullet}$. There is a unique bijective-on-objects (boo) functor $F: \mathcal{C} \rightarrow \mathcal{D}$ and two boo functors $G_1, G_2: \mathcal{D} \rightarrow \mathcal{C}$. These have corresponding retrofunctors going the other way.

1. Write down the morphism $\mathfrak{d} \rightarrow \mathfrak{c}$ of carriers corresponding to F .
2. Write down the morphism $\mathfrak{c} \rightarrow \mathfrak{d}$ of carriers corresponding to either G_1 or G_2 .

We record the following proposition here.

Proposition 8.72. If $\varphi: p \rightarrow q$ is a cartesian lens, then $\mathcal{T}_\varphi: \mathcal{T}_p \rightarrow \mathcal{T}_q$ is a cartesian retrofunctor: that is, for each $t \in \text{tree}_p$, the on-morphisms function

$$(\mathcal{T}_\varphi)_t^\# : \mathcal{T}_q[\mathcal{T}_\varphi t] \xrightarrow{\cong} \mathcal{T}_p[t]$$

is a bijection.

Proof. Given $\varphi: p \rightarrow q$ cartesian, each tree $T \in \mathbf{tree}_p$ is sent to a tree in $\mathbf{tree}_q$ with the same branching profile. A morphism emanating from it is just a finite rooted path, and the set of these is completely determined by the branching profile. Thus we have the desired bijection. $\square$

8.2.3 Limits and colimits of comonoids

We saw in Theorems 5.34 and 5.44 that $\mathbf{Poly}$ has all small limits and colimits. It turns out that $\mathbf{Cat}^\sharp$ has all small limits and colimits as well. We start by discussing colimits in $\mathbf{Cat}^\sharp$, as they are somewhat easier to get a handle on.

Colimits in $\mathbf{Cat}^\sharp$

It is a consequence of the forgetful-cofree adjunction from Theorem 8.45 that $\mathbf{Cat}^\sharp$ inherits all the colimits from $\mathbf{Poly}$. As these results are somewhat technical, relying on a property of functors known as *comonadicity*, we defer their proofs to references.

Proposition 8.73 (Porst). The forgetful functor $\mathbf{Cat}^\sharp \rightarrow \mathbf{Poly}$ is comonadic.

Proof. The fact that a forgetful functor $\mathbf{Cat}^\sharp \cong \mathbf{Comon}(\mathbf{Poly}) \rightarrow \mathbf{Poly}$ is comonadic if it has a right adjoint follows from Beck's monadicity theorem via a straightforward generalization of an argument given by Paré in [Par69, pp. 138-9], as pointed out by Porst in [Por19, Fact 3.1]. So the result follows from Theorem 8.45. $\square$

Corollary 8.74. The category $\mathbf{Cat}^\sharp$ has all small colimits. They are created by the forgetful functor $\mathbf{Cat}^\sharp \rightarrow \mathbf{Poly}$.

Proof. A comonadic functor creates all colimits that exist in its codomain (see [nLa18]), and by Theorem 5.44, the category $\mathbf{Poly}$ has all small colimits. $\square$

Example 8.75 (Coproducts in $\mathbf{Cat}^\sharp$). Probably the most familiar kind of colimit in $\mathbf{Cat}^\sharp$ is the coproduct, as Corollary 8.74 tells us that it agrees with the usual coproduct from $\mathbf{Cat}$. Here's why.

For concreteness, let I be a set and $(C_i)_{i \in I}$ be categories with carriers $(c_i)_{i \in I}$. Then the coproduct of $(C_i)_{i \in I}$ in $\mathbf{Cat}$ is the category $\sum_{i \in I} C_i$, whose objects are given by the disjoint union of the objects in each summand, so that

$$\mathrm{Ob} \left(\sum_{i \in I} C_i \right) \cong \sum_{i \in I} \mathrm{Ob} C_i = \sum_{i \in I} c_i(1) \cong \left(\sum_{i \in I} c_i \right)(1),$$

and whose morphisms out of each $c \in \mathrm{Ob} C_j \subseteq \mathrm{Ob} \sum_{i \in I} C_i$ are just the morphisms out of c in the summand C_j , so

$$\left(\sum_{i \in I} C_i \right)[c] \cong C_j[c] = c_j[c] \cong \left(\sum_{i \in I} c_i \right)[c].$$

Hence $\sum_{i \in I} C_i$ is carried by the coproduct of polynomials $\sum_{i \in I} c_i$.

It remains to show that $\sum_{i \in I} C_i$ is also the coproduct of $(C_i)_{i \in I}$ in $\mathbf{Cat}^\sharp$. We already know from Corollary 8.74 that it has the right carrier: the coproduct of carriers $\sum_{i \in I} c_i$. It also has the right morphisms, for any object (i, x) with $x \in c_i(1)$, the set of emanating morphisms is—and should be— $c_i[x]$.

Exercise 8.76.

1. Show that 0 has a unique comonoid structure.
2. Explain why 0 with its comonoid structure is initial in $\mathbf{Cat}^\sharp$ in two ways: by explicitly showing it has the required universal property, or by invoking Corollary 8.74.

Exercise 8.77. Given a comonoid $(c, \epsilon, \delta) \in \mathbf{Cat}^\sharp$, show that there is an induced comonoid structure on the polynomial $2c$.

Limits in $\mathbf{Cat}^\sharp$

As in the case of colimits, there is a rather technical result showing that the forgetful-cofree adjunction implies the existence of all small limits of comonoids, which we summarize here.

Corollary 8.78. The category $\mathbf{Cat}^\sharp$ has all small limits.

Proof. By Theorem 8.45, the forgetful functor $U: \mathbf{Cat}^\sharp \rightarrow \mathbf{Poly}$ has a right adjoint, and by Theorem 5.34, $\mathbf{Poly}$ itself has all small limits. Furthermore equalizers in $\mathbf{Poly}$ are connected limits, so by Theorem 6.79, they are preserved by $\triangleleft$ on either side. Then the result follows from [Por19, Fact 3.4]. $\square$

8.2.4 Parallel product comonoids

The usual product of categories is not the categorical product in $\mathbf{Cat}^\sharp$. It is, however, a *monoidal* product on $\mathbf{Cat}^\sharp$, coinciding with the parallel product $\otimes$ on $\mathbf{Poly}$.

Proposition 8.79. The parallel product $(y, \otimes)$ on $\mathbf{Poly}$ extends to a monoidal structure $(y, \otimes)$ on $\mathbf{Cat}^\sharp$, such that the forgetful functor $U: \mathbf{Cat}^\sharp \rightarrow \mathbf{Poly}$ is strong monoidal with respect to $\otimes$. The parallel product of two categories is their product in $\mathbf{Cat}$.

Proof. Let $\mathcal{C}, \mathcal{D} \in \mathbf{Cat}^\sharp$ be categories corresponding to comonoids $(c, \epsilon_c, \delta_c)$ and $(d, \epsilon_d, \delta_d)$.

The carrier of $\mathcal{C} \otimes \mathcal{D}$ is defined to be $c \otimes d$. A position in it is a pair (c, d) of objects, one from $\mathcal{C}$ and one from $\mathcal{D}$; a direction there is a pair (f, g) of a morphism emanating from c and one emanating from d .

We define $\epsilon_{\mathcal{C} \otimes \mathcal{D}}: c \otimes d \rightarrow y$ as

$$c \otimes d \xrightarrow{\epsilon_c \otimes \epsilon_d} y \otimes y \cong y.$$

This says that the identity at (c, d) is the pair of identities.

We define $\delta_{\mathcal{C} \otimes \mathcal{D}}: (c \otimes d) \rightarrow (c \triangleleft d) \triangleleft (c \otimes d)$ using the duoidal property:

$$c \otimes d \xrightarrow{\delta_c \otimes \delta_d} (c \triangleleft c) \otimes (d \triangleleft d) \rightarrow (c \otimes d) \triangleleft (c \otimes d).$$

One can check that this says that codomains and composition are defined coordinate-wise, and that $(\epsilon \otimes \mathfrak{d}, \epsilon_{\epsilon \otimes \mathfrak{d}}, \delta_{\epsilon \otimes \mathfrak{d}})$ forms a comonoid. One can also check that this is functorial in $\mathcal{C}, \mathcal{D} \in \mathbf{Cat}^\sharp$. See Exercise 8.80. $\square$

Exercise 8.80. We complete the proof of Proposition 8.79.

1. Show that $(\epsilon \otimes \mathfrak{d}, \epsilon_{\epsilon \otimes \mathfrak{d}}, \delta_{\epsilon \otimes \mathfrak{d}})$, as described in Proposition 8.79, forms a comonoid.
2. Check that the construction $(\mathcal{C}, \mathcal{D}) \mapsto \mathcal{C} \otimes \mathcal{D}$ is functorial in $\mathcal{C}, \mathcal{D} \in \mathbf{Cat}^\sharp$.

The cofree construction works nicely with this monoidal product.

Proposition 8.81. The cofree functor $p \mapsto \mathcal{T}_p$ is lax monoidal; in particular there is a lens $y \rightarrow \mathfrak{t}_y$, and for any $p, q \in \mathbf{Poly}$ there is a natural lens

$$\mathfrak{t}_p \otimes \mathfrak{t}_q \rightarrow \mathfrak{t}_{p \otimes q},$$

satisfying the usual conditions.

Proof. By Proposition 8.79, the left adjoint $U: \mathbf{Cat}^\sharp \rightarrow \mathbf{Poly}$ is strong monoidal. A consequence of Kelly's doctrinal adjunction theorem [Kel74] says that the right adjoint of an oplax monoidal functor is lax monoidal. $\square$

Exercise 8.82.

1. What polynomial is $\mathfrak{t}_y$?
2. What is the lens $y \rightarrow \mathfrak{t}_y$ from Proposition 8.81?
3. Explain in words how to think about the lens $\mathfrak{t}_p \otimes \mathfrak{t}_q \rightarrow \mathfrak{t}_{p \otimes q}$ from Proposition 8.81, for arbitrary $p, q \in \mathbf{Poly}$.

8.3 Comodules over polynomial comonoids

Just as we can define a category of comonoids within any monoidal category, we can further define a notion of comodules over such comonoids. And much like how polynomial comonoids are categories, these comodules can also be described in categorical terms we are already familiar

with. There is much more to say about comodules over polynomial comonoids than we have room for here—we will merely give a glimpse of the theory and applications.

8.3.1 Left and right comodules

When the monoidal category is not symmetric, left comodules and right comodules may differ significantly, so we define them separately (but notice that the diagrams for one are analogous to the diagrams for the other).

Definition 8.83 (Left comodule). In a monoidal category $(\mathbf{C}, y, \triangleleft)$, let $\mathcal{C} = (c, \epsilon, \delta)$ be a comonoid. A *left $\mathcal{C}$ -comodule* is

- an object $m \in \mathbf{C}$, called the *carrier*, equipped with
 - a morphism $c \triangleleft m \xleftarrow{\lambda} m$ called the *left coaction*,
- such that the following diagrams, collectively known as the *left comodule laws*, commute:

$$\begin{array}{ccc}
 c \triangleleft m & \xleftarrow{\lambda} & m \\
 \epsilon \triangleleft m \downarrow & & \searrow \\
 m & &
 \end{array}
 \qquad
 \begin{array}{ccc}
 c \triangleleft m & \xleftarrow{\lambda} & m \\
 \delta \triangleleft m \downarrow & & \downarrow \lambda \\
 c \triangleleft c \triangleleft m & \xleftarrow{\epsilon \triangleleft \lambda} & c \triangleleft m
 \end{array}
 \tag{8.84}$$

When referring to a left $\mathcal{C}$ -comodule, we may omit its coaction if it can be inferred from context (or simply unspecified), identifying the comodule with its carrier.

A *morphism* of left $\mathcal{C}$ -comodules m and n is a morphism $\alpha: m \rightarrow n$ such that the following diagram commutes:

$$\begin{array}{ccc}
 c \triangleleft m & \xleftarrow{\quad} & m \\
 c \triangleleft \alpha \downarrow & & \downarrow \alpha \\
 c \triangleleft n & \xleftarrow{\quad} & n
 \end{array}$$

Here the top and bottom morphisms are the left coactions of m and n .

Exercise 8.85. Show that the category of $\mathcal{C}$ -coalgebras from Definition 7.96 is exactly the category of *constant* left $\mathcal{C}$ -comodules—i.e. the full subcategory of the category of left $\mathcal{C}$ -comodules spanned by those left $\mathcal{C}$ -comodules whose carriers are constant polynomials.

Definition 8.86 (Right comodule). In a monoidal category $(\mathbf{C}, y, \triangleleft)$, let $\mathcal{D} = (\mathfrak{d}, \epsilon, \delta)$ be a comonoid. A *right $\mathcal{D}$ -comodule* is

- an object $m \in \mathbf{C}$, called the *carrier*, equipped with
- a morphism $m \xrightarrow{\rho} m \triangleleft \mathfrak{d}$ called the *right coaction*,

such that the following diagrams, collectively known as the *right comodule laws*, commute:

$$\begin{array}{ccc}
 m & \xrightarrow{\rho} & m \triangleleft \mathfrak{d} \\
 \searrow & & \downarrow m \triangleleft \epsilon \\
 & & m
 \end{array}
 \qquad
 \begin{array}{ccc}
 m & \xrightarrow{\rho} & m \triangleleft \mathfrak{d} \\
 \rho \downarrow & & \downarrow m \triangleleft \delta \\
 m \triangleleft \mathfrak{d} & \xrightarrow{\rho \triangleleft \mathfrak{d}} & m \triangleleft \mathfrak{d} \triangleleft \mathfrak{d}
 \end{array}
 \quad (8.87)$$

When referring to a right $\mathcal{D}$ -comodule, we may omit its coaction if it can be inferred from context (or unspecified), identifying the comodule with its carrier.

A *morphism* of right $\mathcal{D}$ -comodules m and n is a morphism $\alpha: m \rightarrow n$ such that the following diagram commutes:

$$\begin{array}{ccc}
 m & \longrightarrow & m \triangleleft \mathfrak{d} \\
 \alpha \downarrow & & \downarrow \alpha \triangleleft \mathfrak{d} \\
 n & \longrightarrow & n \triangleleft \mathfrak{d}
 \end{array}$$

Here the top and bottom morphisms are the right coactions of m and n .

Exercise 8.88.

1. Draw the equations of (8.84) using polyboxes.
2. Draw the equations of (8.87) using polyboxes.

Exercise 8.89. Recall from Exercise 7.64 that y has a unique category structure.

1. Show that the category of left y -comodules is isomorphic to **Poly**.
2. Show that the category of right y -comodules is isomorphic to **Poly**. If it is similar, just say “similar”; if not, explain.

We can characterize left comodules as something far more familiar.

Proposition 8.90. Let $\mathcal{C}$ be a category. The category of left $\mathcal{C}$ -comodules is equivalent to the category of functors $\mathcal{C} \rightarrow \mathbf{Poly}$.

Sketch of proof. Let $\mathfrak{c}$ be the carrier of the category $\mathcal{C}$. Given a left $\mathcal{C}$ -comodule $m \rightarrow \mathfrak{c} \triangleleft m$, we saw in Exercise 8.88 that there is an induced function $|-|: m(1) \rightarrow \mathfrak{c}(1)$, so to each object $i \in \text{Ob}(\mathcal{C})$ we can associate a set $P_i^m := \{a \in m(1) \mid |a| = i\}$. We also have for each $a \in P_i \subseteq m(1)$ a set $m[a]$, and so we can consider the polynomial

$$p_i^m := \sum_{a \in P_i^m} y^{m[a]}.$$

The polynomial $p_i^m \in \mathbf{Poly}$ is easily seen to be functorial in the comodule m . Moreover, given a morphism $f: i \rightarrow i'$ in $\mathcal{C}$, we have for each $a \in p_i^m(1)$ an element $a.f \in m(1)$, with $|a.f| = i'$, i.e. $a.f \in P_{i'}^m = p_{i'}^m(1)$. Similarly for each $x \in m[a.f] = p_{i'}^m[a.f]$ we have $\lambda_a^\sharp(f, x) \in m[a] = p_i^m[a]$, and thus we have a lens $p_i^m \rightarrow p_{i'}^m$. In this way we obtain a functor $p_-^m: \mathcal{C} \rightarrow \mathbf{Poly}$, and this functor is itself functorial in m .

We leave the proof that this construction has an inverse to the reader. $\square$

Similarly, a right comodule is not an altogether foreign concept.

Proposition 8.91. Let $\mathcal{D}$ be a category. A right $\mathcal{D}$ -comodule m can be identified (up to isomorphism) with a functor $\mathcal{D} \rightarrow \mathbf{Set}^{m(1)}$.

Sketch of proof. Let $\mathfrak{d}$ be the carrier of $\mathcal{D}$. Given a right $\mathcal{D}$ -comodule $m \rightarrow m \triangleleft \mathfrak{d}$ we need a functor $F: \mathcal{D} \times m(1) \rightarrow \mathbf{Set}$. We again rely heavily

on Exercise 8.88. Given $j \in \mathcal{D}$ and $a \in m(1)$, define

$$F(j, d) := \{x \in m[a] \mid |a.f| = j\}.$$

Given $g: j \rightarrow j'$ we get $\rho_a^\sharp(x, g) \in m[a]$ with $|a.\rho_a^\sharp(x, g)| = j'$, and thus g induces a function $F(g, d): F(j, d) \rightarrow F(j', d)$. Again by laws from Exercise 8.88 these are functorial in g , completing the proof sketch. $\square$

Proposition 8.92. Let $\mathcal{C} = (\mathfrak{c}, \epsilon, \delta)$ be a comonoid in **Poly**. For any set G , the polynomial $y^G \triangleleft \mathfrak{c}$ has a natural right $\mathcal{C}$ -comodule structure.

Proof. We use the lens $(y^G \triangleleft \delta): (y^G \triangleleft \mathfrak{c}) \rightarrow (y^G \triangleleft \mathfrak{c} \triangleleft \mathfrak{c})$. It satisfies the unitality and associativity laws because $\mathfrak{c}$ does. $\square$

We can think of elements of G as “generators”. Then if $i': G \rightarrow \mathfrak{c} \triangleleft 1$ assigns to every generator an object of a category $\mathcal{C}$, then we should be able to find the free $\mathcal{C}$ -set that i' generates.

Proposition 8.93. Functions $i': G \rightarrow \mathfrak{c} \triangleleft 1$ are in bijection with positions $i \in y^G \triangleleft \mathfrak{c} \triangleleft 1$. Let $m := i^*(y^G \triangleleft \mathfrak{c})$ and let ρ_i be the induced right $\mathcal{C}$ -comodule structure. Then ρ_i corresponds to the free $\mathcal{C}$ -set generated by i' .

Proof. The polynomial m has the following form:

$$m \cong y^{\sum_{g \in G} \mathfrak{c}[i'(g)]}.$$

In particular ρ_i is a representable right $\mathcal{C}$ -comodule, and we can identify it with a $\mathcal{C}$ -set by Theorem 8.100. The elements of this $\mathcal{C}$ -set are pairs (g, f) , where $g \in G$ is a generator and $f: i'(g) \rightarrow \text{cod}(f)$ is a morphism in $\mathcal{C}$ emanating from $i'(g)$. It is easy to see that the comodule structure induced by Proposition 8.92 is indeed the free one. $\square$

Proposition 8.94. Let $\mathfrak{c}$ be a comonoid. Given a set I and right $\mathfrak{c}$ -comodules $(m_i)_{i \in I}$, their coproduct $\sum_{i \in I} m_i$ has a natural right $\mathfrak{c}$ -

comodule structure. Conversely, each representable summand in the carrier of a right $\mathfrak{c}$ -comodule is itself a right $\mathfrak{c}$ -comodule.

Sketch of proof. This follows from Proposition 6.46, that $- \triangleleft \mathfrak{c}$ commutes with coproducts: $(\sum_{i \in I} m_i) \triangleleft \mathfrak{c} \cong \sum_{i \in I} (m_i \triangleleft \mathfrak{c})$. $\square$

Proposition 8.95. If $m \in \mathbf{Poly}$ is equipped with both a right $\mathcal{C}$ -comodule and a right $\mathcal{D}$ -comodule structure, we can naturally equip m with a $(\mathcal{C} \times \mathcal{D})$ -comodule structure.

Proof. It suffices by Proposition 8.94 to assume that $m = y^M$ is representable. But a right $\mathcal{C}$ -comodule with carrier y^M can be identified with a retrofunctor $My^M \rightarrow \mathcal{C}$.

Thus if y^M is both a right- $\mathcal{C}$ comodule and a right- $\mathcal{D}$ comodule, then we have comonoid morphisms $\mathcal{C} \leftarrow My^M \rightarrow \mathcal{D}$. This induces a unique comonoid morphism $My^M \rightarrow (\mathcal{C} \times \mathcal{D})$ to the product, and we identify it with a right- $(\mathcal{C} \times \mathcal{D})$ comodule on y^M . $\square$

8.3.2 Bicomodules

We take special note of any object that is both a left comodule and a right comodule in compatible ways.

Definition 8.96 (Bicomodule). In a monoidal category $(\mathbf{C}, y, \triangleleft)$, let $\mathcal{C}$ and $\mathcal{D}$ be comonoids with carriers $\mathfrak{c}$ and $\mathfrak{d}$. A $(\mathcal{C}, \mathcal{D})$ -bicomodule is

1. an object $m \in \mathbf{C}$ that is both
2. a left $\mathcal{C}$ -comodule, with left coaction $\mathfrak{c} \triangleleft m \xleftarrow{\lambda} m$, and
3. a right $\mathcal{D}$ -comodule, with right coaction $m \xrightarrow{\rho} m \triangleleft \mathfrak{d}$,

such that the following diagram, known as the *coherence law*, commutes:

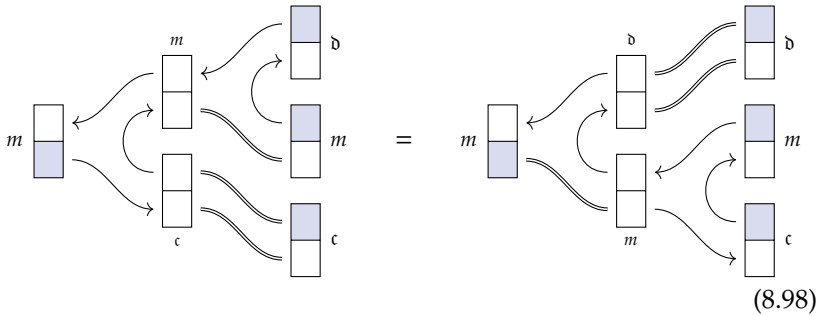
$$\begin{array}{ccc}
 & m & \\
 \lambda \swarrow & & \searrow \rho \\
 \mathfrak{c} \triangleleft m & & m \triangleleft \mathfrak{d} \\
 \searrow \mathfrak{c} \triangleleft \rho & & \swarrow \lambda \triangleleft \mathfrak{d} \\
 & \mathfrak{c} \triangleleft m \triangleleft \mathfrak{d} &
 \end{array} \tag{8.97}$$

We may denote such a bicomodule by $\mathcal{C} \xleftarrow{m} \triangleleft \mathcal{D}$ when its left and

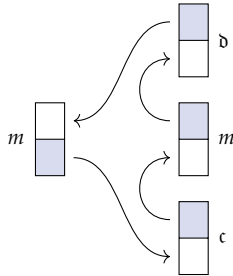
right coactions have yet to be specified or may be inferred from context—or even by $c \xleftarrow{m} d$ when the comonoid structures on c and d can be inferred from context as well.⁴

A *morphism* of $(\mathcal{C}, \mathcal{D})$ -bicomodules is one that is a morphism of left $\mathcal{C}$ -comodules and a morphism of right $\mathcal{D}$ -comodules.

We draw the commutativity of (8.97) using polyboxes.



This polybox equation implies that we can unambiguously write



for any bicomodule $c \xleftarrow{m} d$.

Exercise 8.99. Let $\mathcal{C} = (c, \epsilon, \delta)$ be a category. Recall from Exercise 7.64 that y has a unique category structure.

1. Show that a left $\mathcal{C}$ -module is the same thing as a $(\mathcal{C}, y)$ -bimodule.
2. Show that a right $\mathcal{C}$ -module is the same thing as a $(y, \mathcal{C})$ -

⁴The notation $c \xleftarrow{m} d$ has a mnemonic advantage, as each $\leftarrow$ goes in the correct direction: $c \leftarrow m \leftarrow d$ and $m \rightarrow m \rightarrow d$. But it also looks like an arrow going backward from d to c , which will turn out to have a semantic advantage as well.

bimodule.

3. Show that every polynomial $p \in \mathbf{Poly}$ has a unique (y, y) -bimodule structure.
4. Show there is an isomorphism of categories $\mathbf{Poly} \cong {}_y\mathbf{Mod}_y$.

8.3.3 More equivalences

We have seen that a retrofunctor from a state category to a category $\mathcal{C}$ carries the same data as a $\mathcal{C}$ -coalgebra, which is in turn equivalent to a discrete opfibration over $\mathcal{C}$ or a copresheaf on $\mathcal{C}$. We then showed that cartesian retrofunctors to $\mathcal{C}$ is an equivalent notion as well. We are now ready to show that several kinds of comodules are also equivalent to all of these concepts.

Theorem 8.100. Given a polynomial comonoid $\mathcal{C} = (\mathfrak{c}, \epsilon, \delta)$, the following comprise equivalent categories:

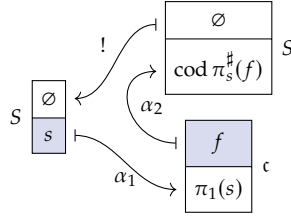
1. functors $\mathcal{C} \rightarrow \mathbf{Set}$;
2. discrete opfibrations over $\mathcal{C}$;
3. cartesian retrofunctors to $\mathcal{C}$;
4. $\mathcal{C}$ -coalgebras (sets with a coaction by $\mathcal{C}$);
5. constant left $\mathcal{C}$ -comodules;
6. $(\mathcal{C}, 0)$ -bicomodules;
7. linear left $\mathcal{C}$ -comodules; and
8. representable right $\mathcal{C}$ -comodules (opposite).

In fact, all but the first comprise isomorphic categories; and up to isomorphism, any one of these can be identified with a retrofunctor from a state category to $\mathcal{C}$.

Sketch of proof. $1 \simeq 2 \cong 3$: This was shown in Proposition 7.108.

$3 \cong 4$: Given a cartesian retrofunctor $(\pi_1, \pi^\sharp): \mathcal{S} \rightarrow \mathcal{C}$, let $S := \text{Ob}(\mathcal{S})$ and define a $\mathfrak{c}$ -coalgebra structure $\alpha: S \rightarrow \mathfrak{c} \triangleleft S$ on an object $s \in$

$\text{Ob}(\mathcal{S})$ and an emanating morphism $f: \pi_1(s) \rightarrow c'$ in $\mathcal{C}$ by



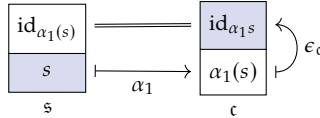
We check that this is indeed a coalgebra using properties of retrofunctors. For identities in $\mathcal{C}$, we have

$$\begin{aligned} \alpha_2(s, \text{id}_{\pi_1(s)}) &= \text{cod } \pi_s^\#(\text{id}_{\pi_1(s)}) \\ &= \text{cod id}_s = s \end{aligned}$$

and for compositions in $\mathcal{C}$ we have

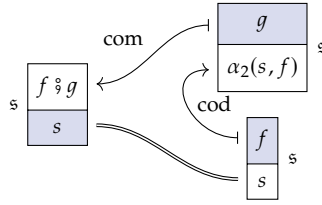
$$\begin{aligned} \alpha_2(s, f \circ g) &= \text{cod} \left(\pi_s^\#(f \circ g) \right) \\ &= \text{cod} \left(\pi_s^\#(f) \circ \pi_{\text{cod } \pi_s^\#(f)}^\# g \right) \\ &= \text{cod} \left(\pi_{\text{cod } \pi_s^\#(f)}^\# g \right) \\ &= \alpha_2(\alpha_2(s, f), g). \end{aligned}$$

Going backward, if we're given a coalgebra $\alpha: S \rightarrow c \triangleleft S$, we obtain a function $\alpha_1: S \rightarrow c \triangleleft 1$ and we define $s := \alpha_1^* c$ and the cartesian lens $\varphi := (\alpha_1, \text{id}): s \rightarrow c$ to be the base change from Proposition 5.75. We need to show s has a comonoid structure (s, ϵ, δ) and that φ is a retrofunctor. We simply define the eraser $\epsilon: s \rightarrow y$ using α_1 and the eraser on c :



We give the duplicator $\delta: s \rightarrow s \triangleleft s$ using α_2 for the codomain, and

using the composite $\circ$ from c :



We need only check that (s, ϵ, δ) is a comonoid, that $(\alpha_1, \text{id}): s \rightarrow c$ is a retrofunctor, that the roundtrips between cartesian retrofunctors and coalgebras are identities, and that these assignments are functorial.

4 $\cong$ 5: This is straightforward and was mentioned in Definition 7.96.

5 $\cong$ 6: A right 0-comodule is in particular a polynomial $m \in \mathbf{Poly}$ and a lens $\rho: m \rightarrow m \triangleleft 0$ such that $(m \triangleleft \epsilon) \circ \rho = \text{id}_m$. This implies ρ is monic, which itself implies by Proposition 5.18 that m must be constant since $m \triangleleft 0$ is constant. This makes $m \triangleleft \epsilon$ the identity, at which point ρ must also be the identity. Conversely, for any set M , the corresponding constant polynomial is easily seen to make the diagrams in (8.87) commute.

5 $\cong$ 7: By the adjunction in Theorem 5.4 and the fully faithful inclusion $\mathbf{Set} \rightarrow \mathbf{Poly}$ of sets as constant polynomials, Proposition 5.2, we have isomorphisms

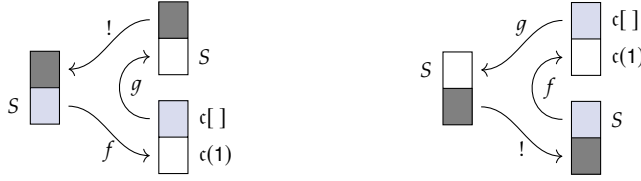
$$\mathbf{Poly}(Sy, c \triangleleft Sy) \cong \mathbf{Set}(S, c \triangleleft Sy \triangleleft 1) = \mathbf{Set}(S, c \triangleleft S) \cong \mathbf{Poly}(S, c \triangleleft S).$$

One checks easily that if $Sy \rightarrow c \triangleleft Sy$ corresponds to $S \rightarrow c \triangleleft S$ under the above isomorphism, then one is a left comodule if and only if the other is.

7 $\cong$ 8: By (6.65) we have a natural isomorphism

$$\mathbf{Poly}(Sy, c \triangleleft Sy) \cong \mathbf{Poly}(y^S, y^S \triangleleft c).$$

In pictures,



The last claim was proven in Proposition 7.109. $\square$

Let $\mathcal{C}$ be a category. Under the above correspondence, the terminal functor $\mathcal{C} \rightarrow \mathbf{Set}$ corresponds to the identity discrete opfibration $\mathcal{C} \rightarrow \mathcal{C}$, the identity retrofunctor $\mathcal{C} \rightarrow \mathcal{C}$, a certain left $\mathcal{C}$ comodule with carrier $\mathcal{C}(1)y$ which we call the *canonical left $\mathcal{C}$ -comodule*, a certain constant left $\mathcal{C}$ comodule with carrier $\mathcal{C}(1)$ which we call the *canonical $(\mathcal{C}, 0)$ -bicomodule*, and a certain representable right $\mathcal{C}$ -comodule with carrier $y^{\mathcal{C}(1)}$ which we call the *canonical right $\mathcal{C}$ -comodule*.

8.3.4 Bicomodules are parametric right adjoints

There is an equivalent characterization of bicomodules between a pair of polynomial comonoids due to Richard Garner. Thus we attribute the foundational theory of $\mathbf{Cat}^\#$ to Ahman-Uustalu-Garner.

Proposition 8.101 (Garner). Let $\mathcal{C}$ and $\mathcal{D}$ be categories. Then the following can be identified, up to isomorphism:

1. a $(\mathcal{C}, \mathcal{D})$ -bicomodule.
2. a parametric right adjoint $\mathbf{Set}^{\mathcal{D}} \rightarrow \mathbf{Set}^{\mathcal{C}}$.
3. a connected limit-preserving functor $\mathbf{Set}^{\mathcal{D}} \rightarrow \mathbf{Set}^{\mathcal{C}}$.

Parametric right adjoints model *data migrations* between categorical databases; see [SW15].

When a polynomial

$$m := \sum_{i \in m(1)} y^{m[i]}$$

is given the structure of a $(\mathcal{D}, \mathcal{C})$ -bicomodule, the symbols in that formula are given a hidden special meaning:

$$m(1) \in \mathbf{Set}^{\mathcal{D}} \quad \text{and} \quad m[i] \in \mathbf{Set}^{\mathcal{C}}.$$

Before we knew about bicomodule structures, what we called positions and directions were understood as each forming an ordinary set. In the presence of a bicomodule structure, the positions $m(1)$ have been organized into a $\mathcal{D}$ -set and the directions $m[i]$ have been organized into a $\mathcal{C}$ -set for each position i . We are listening for $\mathcal{C}$ -sets and positioning ourselves in a $\mathcal{D}$ -set.

Definition 8.102 (Prafunctor). Let $\mathcal{C}$ and $\mathcal{D}$ be categories. A *prafunctor* (also called a *parametric right adjoint functor*) $\mathbf{Set}^{\mathcal{C}} \rightarrow \mathbf{Set}^{\mathcal{D}}$ is one satisfying any of the conditions of Proposition 8.101.

8.3.5 Bicomodules in dynamics

We conclude with a peek at how bicomodules can model dynamical systems themselves.

Example 8.103 (Cellular automata). In Example 4.66 and Exercise 4.67 we briefly discussed cellular automata; here we will discuss another way that cellular automata show up, this time in terms of bicomodules.

Suppose that $\text{src}, \text{tgt}: A \rightrightarrows V$ is a graph, and consider the polynomial

$$g := \sum_{v \in V} y^{\text{src}^{-1}(v)}$$

so that positions are vertices and directions are emanating arrows. It carries a natural bicomodule structure

$$Vy \xleftarrow{g} \lhd Vy$$

where the right structure lens uses tgt ; see Exercise 8.104 for details. A bicomodule

$$Vy \xleftarrow{T} \lhd 0$$

can be identified with a functor $T: V \rightarrow \mathbf{Set}$, i.e. it assigns to each vertex $v \in V$ a set. Let's call $T(v)$ the color set at v ; for many cellular automata we will put $T(v) \cong 2$ for each v .

Then a cellular automata on g with color sets T is given by a map

$$\begin{array}{ccccc} Vy & \xleftarrow{g} & Vy & \xleftarrow{T} & 0 \\ & & \Downarrow \alpha & & \\ & & T & & \end{array}$$

Indeed, for every vertex $v \in V$ the map α gives a function

$$\prod_{\text{src}(a)=v} T(\text{tgt}(a)) \xrightarrow{\alpha_v} T(v),$$

which we call the *update* function. In other words, given the current color at the target of each arrow emanating from v , the function α_v returns a new color at v .

Note that if $V \in {}_{Vy}\mathbf{Mod}_0$ is the terminal object, then the composite $Vy \xleftarrow{g} Vy \xleftarrow{V} 0$ is again V .

Exercise 8.104. Let $\text{src}, \text{tgt}: A \rightrightarrows V$ and g and T be as in Example 8.103.

1. Give the structure lens $\lambda: g \rightarrow Vy \triangleleft g$
2. Give the structure lens $\rho: g \rightarrow g \triangleleft Vy$.
3. Give the set T and the structure lens $T \rightarrow Vy \triangleleft T$ corresponding to the functor $V \rightarrow \mathbf{Set}$ that assigns 2 to each vertex.

Example 8.105 (Running a cellular automaton). Let g be a graph on vertex set V , let T assign a color set to each $v \in V$, and let α be the update function for a cellular automaton. As in Example 8.103, this is all given by a diagram

$$\begin{array}{ccccc} Vy & \xleftarrow{g} & Vy & \xleftarrow{T} & 0 \\ & & \Downarrow \alpha & & \\ & & T & & \end{array}$$

To run the cellular automaton, one simply chooses a starting color in each vertex. We call this an initialization; it is given by a map of

bicomodules

$$\begin{array}{ccc}
 & V & \\
 \swarrow & \Downarrow_{\sigma} & \searrow \\
 Vy & & 0 \\
 \nwarrow & \Uparrow_T & \nearrow
 \end{array}
 \quad (8.106)$$

Note that g is a profunctor, i.e. for each $v \in V$ the summand $g_v = y^{\text{src}^{-1}(v)}$ is representable, so it preserves the terminal object. In other words $g \triangleleft_{Vy} V \cong V$.

Now to run the cellular automaton on that initialization for $k \in \mathbb{N}$ steps is given by the composite

$$\begin{array}{ccccccc}
 & & & & V & & \\
 & & & & \parallel & & \\
 Vy & \xleftarrow{g} & \cdots & \xleftarrow{g} & Vy & \xleftarrow{g} & Vy & \xleftarrow{V} & 0 \\
 \parallel & & & & \parallel & & \parallel & \Downarrow_{\sigma} & \parallel \\
 Vy & \xleftarrow{g} & \cdots & \xleftarrow{g} & Vy & \xleftarrow{g} & Vy & \xleftarrow{T} & 0 \\
 \parallel & & & & \parallel & & \parallel & \Downarrow_{\alpha} & \parallel \\
 Vy & \xleftarrow{g} & \cdots & \xleftarrow{g} & Vy & \xleftarrow{T} & & & 0 \\
 \parallel & & & & \parallel & & \parallel & \Downarrow_{\alpha \circ \cdots \circ \alpha} & \parallel \\
 Vy & \xleftarrow{\quad} & & & & & & & 0 \\
 & & & & T & & & &
 \end{array}$$

Exercise 8.107. Explain why (8.106) models an initialization, i.e. a way to choose a starting color in each vertex.

8.4 Summary and further reading

In this chapter we gave more of the theory of $\triangleleft$ -comonoids, sometimes called polynomial comonads. We began by defining an adjunction

$$\text{Cat}^{\#} \begin{array}{c} \xrightarrow{U} \\ \Rightarrow \\ \xleftarrow{\mathcal{T}_-} \end{array} \text{Poly}$$

where $\mathcal{T}_p$ is the cofree comonoid $\mathcal{T}_p$ associated to any polynomial p , and we gave intuition for it in terms of p -trees. A p -tree is a (possibly

infinite) tree for which each vertex is labeled by a position $i \in p(1)$ and its outgoing arrows are each labeled by an element of $p[i]$. The category corresponding to $\mathcal{T}_p$ has p -trees as its objects; the morphisms emanating from such a p -tree are the finite rooted paths up the tree, and the codomain of such a path is the tree rooted at its endpoint.

We then briefly discussed some other properties of $\mathbf{Cat}^\sharp$ including a formal proof of the fact that it has all limits and colimits. Colimits in $\mathbf{Cat}^\sharp$ are created by, i.e. fit nicely with, colimits in **Poly**, but limits are quite strange; we did not really discuss them here, but for example the product in $\mathbf{Cat}^\sharp$ of the walking arrow $\boxed{\bullet \rightarrow \bullet}$ with itself has infinitely many objects!

We then moved on to left-, right-, and bicomodules between polynomial comonoids. In particular, we showed that left $\mathcal{C}$ -comodules can be identified with functors $\mathcal{C} \rightarrow \mathbf{Poly}$, and that $(\mathcal{C}, \mathcal{D})$ -bicomodules correspond to parametric right adjoint functors (prafunctors) $\mathbf{Set}^{\mathcal{D}} \rightarrow \mathbf{Set}^{\mathcal{C}}$. This idea was due to Richard Garner [Gar19]. What we call prafunctors are sometimes called familial functors between (co-)presheaf categories; see [Web07], [GH18], and [Sha21] for more on this notion.

Here we have reached the end of our book, but we have only just scratched the surface of the theory of polynomial functors and their applications to interaction. We encourage the reader to use the knowledge gained from this text to read, watch, learn, and apply further work in this field, and perhaps make new discoveries of their own.

8.5 Exercise solutions

Solution to Exercise 8.5.

With $q := y^2 + 3y^1$, every vertex of a q -tree starting from the root has either 1 or 2 outgoing edges, from which it follows that the tree is infinite and that every vertex has infinitely many descendants.

Solution to Exercise 8.11.

1. To select a 1-tree, we must select a position from $1 \dots$ and then we are done, because there are no directions. So there is a unique 1-tree given by a root and no edges, implying that $\mathbf{tree}_1 \cong 1$.
2. To select a 2-tree, we must select a position from 2. Then we are done, because there are no directions. So every 2-tree is a root with no edges, and the root is labeled with one of the elements of 2. Hence $\mathbf{tree}_2 \cong 2$.

3. To select a y -tree, we must select a position from 1; then for the unique direction at that position, we must select a position from 1; and so forth. Since we only ever have 1 position to choose from, there is only 1 such y -tree we can build: an infinite ray extending from the root, where every vertex has exactly 1 child. Hence $\text{tree}_y \cong 1$.
4. To select a y^2 -tree, we must select a position from 1; then for each direction at that position, we must select a position from 1; and so forth. We only ever have 1 position to choose from, so there is only 1 such y^2 -tree: an infinite binary tree, where every vertex has exactly 2 children. Hence $\text{tree}_{y^2} \cong 1$.
5. To select a $2y$ -tree, we must select a position from 2; then for the unique direction at that position, we must select a position from 2; and so forth, one position out of 2 for each level of the tree. So every $2y$ -tree is an infinite ray, where every vertex is labeled with an element of 2 and has exactly 1 child. There is then a unique vertex of height n for each $n \in \mathbb{N}$, so there is a bijection between $2y$ -trees T and functions $f: \mathbb{N} \rightarrow 2$, where $f(n) \in 2$ is the label of the height- n vertex of T . Hence $\text{tree}_{2y} \cong 2^{\mathbb{N}}$.
6. To select a $(y+1)$ -tree, we choose either a position with 1 direction or a position with no directions: if the position has no directions, we stop, and if the position has 1 direction, we repeat our choice for the next level. So a choice of $(y+1)$ -tree is equivalent to a choice of when, if ever, to stop. That is, for each $n \in \mathbb{N}$, there is a unique $(y+1)$ -tree of height- n consisting of $n+1$ vertices along a single path, so that every vertex aside from the height- n leaf has exactly 1 child. Then there is exactly one more $(y+1)$ -tree: an infinite ray, obtained by always picking the position with 1 direction and never the one with no directions. Hence we could write $\text{tree}_{y+1} \cong \mathbb{N} \cup \{\infty\}$ (although this set is in bijection with $\mathbb{N}$).
7. To select a By^A -tree for fixed $A, B \in \mathbf{Set}$, we must select a position from B ; then for each direction in A at that position, we must select a position from B ; and so forth. This yields a tree in which every vertex has A -many children and a label from B . Such a tree has 1 root and $|A|$ times as many vertices in one level than the level below it, so its height- n vertices for each $n \in \mathbb{N}$ are in bijection with the set A^n : each n -tuple in A^n gives a length- n rooted path of directions in A to follow up the tree, uniquely specifying a height- n vertex. So the set of all vertices is given by $\sum_{n \in \mathbb{N}} A^n = \text{List}(A)$. Then specifying a By^A -tree amounts to assigning a label from B to every vertex via a function $\text{List}(A) \rightarrow B$. Hence $\text{tree}_{By^A} \cong B^{\text{List}(A)}$.

Solution to Exercise 8.12.

1. Yes: by Exercise 8.11 #11 in the case of $B = 1$, or by analogy with Exercise 8.11 #7 and #8, we have that tree_{y^n} is the set of n -ary trees.
2. Yes: by Exercise 8.11 #11, we have that tree_{Ly^n} is the set of L -labeled n -ary trees.

Solution to Exercise 8.14.

We defined tree_p in Definition 8.2 as a rooted tree T where every vertex v is labeled with an element of $p(1)$, together with a choice of bijection between the set of v 's children and the set $p[i]$. We need to translate between that description and the description as the set-theoretic limit

$$1 \leftarrow p(1) \leftarrow p(p(1)) \leftarrow \cdots$$

An element of this set consists of an element of 1, which is determined, an element $i \in p(1)$, an element $(i_1, i_2) \in \sum_{i \in p(1)} \prod_{d \in p[i]} p(1)$ that “agrees” with i , an element of $\sum_{i_1 \in p(1)} \prod_{d_1 \in p[i_1]} \sum_{i_2 \in p(1)} \prod_{d_2 \in p[i_2]} p(1)$ that agrees, etc. But all this agreement is easy to simplify: we need elements

$$\begin{aligned} i_1 &\in p(1), \\ i_2 &\in \prod_{d_1 \in p[i_1]} p(1), \\ i_3 &\in \prod_{d_1 \in p[i_1]} \prod_{d_2 \in p[i_2 d_1]} p(1), \\ i_4 &\in \prod_{d_1 \in p[i_1]} \prod_{d_2 \in p[i_2 d_1]} \prod_{d_3 \in p[i_3 d_1 d_2]} p(1) \\ &\vdots \end{aligned}$$

To go from the former description to the latter, take such a rooted tree T and let i_1 be the label on the root vertex. Since by assumption there is a bijection between the set children of that vertex and the set $p[i_1]$, take $i_2: p[i_1] \rightarrow p(1)$ to assign to each $d_1 \in p[i_1]$ the label on the associated child vertex. Repeat this indefinitely.

To go from the latter description to the former, simply invert the description in the previous paragraph.

Solution to Exercise 8.16.

1. To give a function $t: \mathbf{tree}_p \rightarrow p(\mathbf{tree}_p)$ we need to choose, for each tree $T \in \mathbf{tree}_p$, an element $i \in p(1)$ and a function $T.-: p[i] \rightarrow \mathbf{tree}_p$. Choose i to be the label on the root of T , and choose $T.d$ to be the tree obtained by following the branch labeled d for any $d \in p[i]$.
2. Given an arbitrary coalgebra (S, f) , where $S \in \mathbf{Set}$ and $f: S \rightarrow p \triangleleft S$, we need to show that there is a unique function $u: S \rightarrow \mathbf{tree}_p$ that commutes with f and t , i.e. with $t \circ u = p(f)$.

For each n , we have a function $f_n: S \rightarrow p^{<n} \triangleleft S$ by induction: for $n = 0$ use the identity function $S \rightarrow S$ and given $S \rightarrow p^{<n} \triangleleft S$ we compose with $p^{<n} \triangleleft f$ to get $S \rightarrow p^{<n} \triangleleft p \triangleleft S$. Let $f'_n: S \rightarrow p^{<n}(1)$ be given by $f'_n := f_n \circ (p^{<n} \triangleleft !)$. We know by Exercise 8.14 that $\mathbf{tree}_p$ is the limit of $p^{<n}(1)$, so we get a unique map $(f'_n)_{n \in \mathbb{N}}: S \rightarrow \mathbf{tree}_p$. It commutes with f and t by construction, and the fact that it is constructed using the universal property of limits implies that it is appropriately unique.

3. The function $\mathbf{tree}_p \rightarrow p \triangleleft \mathbf{tree}_p$ from the first part is invertible. Indeed, given $i \in p(1)$ and $T': p[i] \rightarrow \mathbf{tree}_p$, we construct an element of $\mathbf{tree}_p$ by taking its root to be labeled with i , its set of children to be $p[i]$, and for each $d \in p[i]$ taking the remaining tree to be $T'(d)$.

Solution to Exercise 8.17.

For the given values of p , we have already characterized the position-set $t_p(1) \cong \mathbf{tree}_p$ of t in Exercise 8.11, so it remains to characterize the directions at each position, i.e. the vertices of each p -tree.

1. We saw that $\text{tree}_1 \cong 1$ and that the unique 1-tree has a single vertex, its root. Equivalently, it has only 1 rooted path: the empty path from the root to itself. So $t_1 \cong 1y^1 \cong y$.
2. We saw that $\text{tree}_2 \cong 2$ and that each 2-tree has a single vertex, its root. Equivalently, its only rooted path is the empty path. So $t_2 \cong 2y^1 \cong 2y$.
3. We saw that $\text{tree}_y \cong 1$ and that the unique y -tree is a single ray extending from the root, where every vertex has exactly 1 child. So there is exactly 1 vertex of height- n for each $n \in \mathbb{N}$, yielding a bijection between the vertices of the y -tree to $\mathbb{N}$. Equivalently, the ray has exactly 1 rooted path of length n for each $n \in \mathbb{N}$. So $t_y \cong y^{\mathbb{N}}$.
4. We saw that $\text{tree}_{y^2} \cong 1$ and that the unique y^2 -tree is an infinite binary tree, where every vertex has exactly 2 children. So the vertices of height- n are in bijection with 2^n , yielding a bijection between the vertices of the y^2 -tree to $\sum_{n \in \mathbb{N}} 2^n \cong \text{List}(2)$. Equivalently, the rooted paths of an infinite binary tree are just finite binary sequences, which comprise the set $\text{List}(2)$. Hence $t_{y^2} \cong y^{\text{List}(2)}$.
5. We saw that $\text{tree}_{2y} \cong 2^{\mathbb{N}}$, and that every $2y$ -tree is an infinite ray, whose vertices (or rooted paths) are in bijection with $\mathbb{N}$. Hence $t_{2y} \cong 2^{\mathbb{N}}y^{\mathbb{N}}$.
6. We saw that $\text{tree}_{y+1} \cong \mathbb{N} \cup \{\infty\}$. Here the $(y+1)$ -tree corresponding to $n \in \mathbb{N}$ consists of $n+1$ vertices along a single path, so that every vertex aside from the height- n leaf has exactly 1 child; and the $(y+1)$ -tree corresponding to ∞ is an infinite ray. Thus the direction-set of t_{y+1} at $n \in \mathbb{N}$ is $n+1$, while its direction-set at ∞ is $\mathbb{N}$. Hence $t_{y+1} \cong \{\infty\}y^{\mathbb{N}} + \sum_{n \in \mathbb{N}} y^{n+1}$.
7. We saw that $\text{tree}_{By^A} \cong B^{\text{List}(A)}$, where the height- n vertices of a given By^A -tree are in bijection with the set A^n , so the set of all vertices of a By^A -tree is given by $\sum_{n \in \mathbb{N}} A^n = \text{List}(A)$. Equivalently, the rooted paths of the By^A -tree are just finite sequences in A , which comprise the set $\text{List}(A)$. Hence $t_{By^A} \cong B^{\text{List}(A)}y^{\text{List}(A)}$.

Solution to Exercise 8.36.

1. We compute t_B as follows. To select a B -tree, we must select a position from B . Then we are done, because there are no directions. So every B -tree consists of 1 vertex labeled with an element of B . Hence the set of B -trees $t_B(1) = \text{tree}_B$ is in bijection with B itself, yielding $t_B \cong By$. (We could have also applied Exercise 8.17 #7 in the case of $A = 0$, yielding $t_{By^0} \cong B^{\text{List}(0)}y^{\text{List}(0)} \cong By$, as $\text{List}(0) \cong 1 + 0^1 + 0^2 + \cdots \cong 1$ —the empty sequence is the only sequence with elements in 0 .)
2. We characterize the B -tree category $\mathcal{T}_B$ as follows. We saw in Exercise 7.27 and Exercise 7.36 that (up to isomorphism) there is a unique comonoid structure on By , corresponding to the discrete category on B . So $\mathcal{T}_B$ must be isomorphic to the discrete category on B . In the language of B -trees, the objects of $\mathcal{T}_B$ are trees with exactly 1 vertex, labeled with an element of B . Each tree has an identity morphism, its root; but there are no vertices aside from the root, so there are no morphisms aside from the identities. In particular, the only B -subtree of any one B -tree is the entire B -tree itself, so there are no morphisms from one B -tree to a different B -tree. So the category of B -trees can indeed be identified with the discrete category on B .

Solution to Exercise 8.40.

We characterize $\mathcal{T}_{By^A}$, following Example 8.39.

- The objects are By^A -trees. By Exercise 8.17 #7, or Exercise 8.12 #2 with $L := B$ and n replaced with an arbitrary set A , a By^A -tree is a (infinite, assuming $|A|, |B| > 0$) tree whose every vertex is labeled by an element of B and whose children are in bijection with A . We can call these B -labeled A -ary trees; they are in bijection with the set $B^{\text{List}(A)}$.
- A morphism out of a B -labeled A -ary tree is a finite list of directions in A , or a rooted path in an A -ary tree. They comprise the set $\text{List}(A)$. The codomain of each rooted path is the B -labeled A -ary subtree rooted at the end of the path.
- The identity morphism on a given B -labeled A -ary tree is its empty rooted path.
- The composite of two lists in A is obtained by concatenation.

Solution to Exercise 8.41.

We characterize $\mathcal{T}_{y+1}$ as follows.

- The objects are $(y+1)$ -trees. By Exercise 8.17 #6, a $(y+1)$ -tree is either a single length- n path for some $n \in \mathbb{N}$, which we denote by $[n]$, or a ray, which we denote by $[\infty]$. So the set of objects is $\text{tree}_{y+1} = \{[n] \mid n \in \mathbb{N} \cup \{\infty\}\} \cong \mathbb{N} \cup \{\infty\}$.
- A finite $(y+1)$ -tree $[n]$ for $n \in \mathbb{N}$ has $n+1$ rooted paths: 1 each of length 0 through n , inclusive. So a morphism out of $[n]$ can be identified with an element of the set $\{0, \dots, n\}$.

Meanwhile, the infinite $(y+1)$ -tree $[\infty]$, a ray, has exactly 1 rooted path of every length $\ell \in \mathbb{N}$. So a morphism out of $[\infty]$ can be identified with an element of the set $\mathbb{N}$. Every $(y+1)$ -subtree of the ray is still a ray, so the codomain of each of these morphisms is still $[\infty]$.

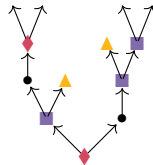
- The identity morphism on a given B -labeled A -ary tree is its empty rooted path.
- The composite of two lists in A is obtained by concatenation.

Solution to Exercise 8.42.

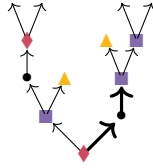
1. An object in $\mathcal{T}_p$ is a stream of letters $a, b, \dots, z$ and spaces $\sqcup$, that may go on forever or may end with a large period, $\bullet$. So an example is the infinite stream $aaaaa \dots$. Another example is $\text{hello_world}\bullet$.
2. The object $aaaaa \dots$ has $\mathbb{N}$ as its set of emanating morphisms. The object $\text{hello_world}\bullet$ has $\{0, \dots, 11\}$ as its set of emanating morphisms.
3. The codomain of any morphism out of $aaaaa \dots$ is again $aaaaa \dots$. The codomain of any morphism $0 \leq i \leq 11$ out of $\text{hello_world}\bullet$ is the string one obtains by removing the first i letters of $\text{hello_world}\bullet$.

Solution to Exercise 8.43.

1. Here are four levels of an object $t \in \text{tree}_p$ (you could have drawn others):



2. The identity morphism at t is the trivial path starting at the root node.
3. We indicate a morphism f using thick arrows:



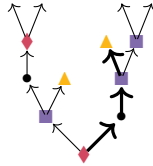
4. The codomain of f is the tree rooted at the target of the path, namely



5. Here is a morphism emanating from the codomain of f :



6. The composite morphism from the original tree is:



Solution to Exercise 8.44.

No: we give a counterexample below. Take $p := 2y$, and consider the object $x \in \mathcal{T}_p(1)$ given by the infinite sequence

$$x := (1, 2, 1, 1, 1, 1, 1, \dots).$$

We define a retrofunctor $\varphi: \mathcal{T}_p \rightarrow y^{\mathbb{Q}_{\geq 0}}$ as follows. The retrofunctor is uniquely defined on objects; on morphisms, for each $i \in \mathcal{T}_p(1)$ and $q \in \mathbb{Q}_{\geq 0}$, set

$$\varphi_i^\# q := \begin{cases} \text{id}_i & \text{if } i \neq x \text{ or } q = 0; \\ f & \text{if } i = x \text{ and } q > 0; \end{cases}$$

where f is the unique morphism from x to

$$x' := (2, 1, 1, 1, 1, 1, \dots).$$

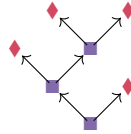
By construction, φ preserves identities, sending $0 \mapsto \text{id}_i$ at each $i \in \mathcal{T}_p(1)$; and since $y^{\mathbb{Q}_{\geq 0}}$ has only one object, φ preserves codomains vacuously; so to verify that φ is indeed a retrofunctor, it suffices to check that it preserves composites. That is, given $q, r \in \mathbb{Q}_{\geq 0}$, we must show that

$$\varphi_i^\# q \circ_{\text{cod } \varphi_i^\# q} \varphi_i^\# r = \varphi_i^\# (q + r)$$

holds. If $i \neq x$ or $q = 0$, then it holds because $\varphi_i^\# q = \text{id}_i$, so both sides equal $\varphi_i^\# r$. If, on the other hand, $i = x$ and $q > 0$, it holds because $\varphi_i^\# q = f$, making $\text{cod } \varphi_i^\# q = x' \neq x$, so both sides equal f . Therefore φ is a retrofunctor; it remains to show that φ does not factor through y . If it did, preservation of identities would mean that for all $i \in \mathcal{T}_p(1)$, the only morphism in the image of $\varphi_i^\#$ would be id_i . Yet $f \neq \text{id}_x$ is in the image of $\varphi_x^\#$, so φ cannot factor through y .

Solution to Exercise 8.55.

1. Let $\varphi: p \rightarrow q$ send $\blacksquare \mapsto \blacksquare$ and $\blacklozenge \mapsto \blacklozenge$ on positions; on directions, let it send the unique $q[\blacksquare]$ -direction to the left $p[\blacksquare]$ -direction.
2. We draw a tree $T \in \text{tree}_p$ below.



3. We draw $\mathcal{T}_\varphi(T)$ below.



Solution to Exercise 8.56.

1. By the adjunction from Theorem 8.45, retrofunctors $\mathcal{T}_p \rightarrow \mathcal{T}_{p^{*n}}$ are in bijection with lenses $t_p \rightarrow p^{*n}$, where $t_p = U\mathcal{T}_p$ is the carrier. From Proposition 7.20 we have a lens $\delta^{(n)}: \mathcal{T}_p \rightarrow \mathcal{T}_{p^{*n}}$, and we also have the counit of the cofree adjunction which we'll temporarily call $r: \mathcal{T}_p \rightarrow p$. Then the desired lens is $(\delta^{(n)} \circ r^{*n}): t_p \rightarrow p^{*n}$.
2. When $n = 1$ the induced retrofunctor is an isomorphism, but for $n \geq 2$ it is not. However, we should start by saying that the function on objects $\text{tree}_p \rightarrow \text{tree}_{p^{*n}}$ is a bijection. It sends a p -tree to the p^{*n} tree that simply compresses every height- n segment into a single vertex, labeled by that height- n segment. To go back, just decompress the segments. But this is not a bijection on maps, because every rooted path on $\text{tree}_{p^{*n}}$ would correspond to a rooted path on tree_p whose length is a multiple of n . When $n = 2$ and $p = y + 1$, some rooted paths in tree_p have odd lengths.

Solution to Exercise 8.64.

A category Sy with linear carrier is a discrete category on S , so we need to show that a retrofunctor between the discrete category on S and the discrete category on T is the same thing as a function $S \rightarrow T$. But this is clear: a retrofunctor $Sy \rightarrow Ty$ is a function $S \rightarrow T$ on objects and a function backwards on morphisms, and the latter is unique because each object in Sy has a unique outgoing morphism.

Solution to Exercise 8.67.

1. Given an object $i \in c'(1) = c(1)$, the set of emanating morphisms is $c'[i] := \mathfrak{d}[fi]$, and they compose according to the structure of $\mathfrak{d}$. Since morphisms in $\mathfrak{d}$ compose associatively and unitaly, so do morphisms in c' .
2. Given $i \in c(1)$ and $m \in c'[i] = \mathfrak{d}[fi]$, we have $g(\text{cod}(g_i^\sharp(m))) = \text{cod}(g_i^\sharp(m)) = \text{cod}(f_i^\sharp(m)) = \text{cod}(m)$ by definition.
3. For any $i \in c'(1)$, the lens $g_i^\sharp$ preserves compositions of morphisms in $c'[i] = \mathfrak{d}[fi]$ because it agrees with $f_i^\sharp$, which preserves compositions.
4. Given $i \in c'(1)$ and $m \in \mathfrak{d}[hi]$, we have $h(\text{cod}(h_i^\sharp(m))) = f(\text{cod}(f_i^\sharp(m))) = m$.
5. The map $h_i^\sharp$ was chosen to be the identity for each i , so it certainly preserves compositions.

Solution to Exercise 8.70.

The category corresponding to Sy^S is the contractible groupoid K_S on S . A vertical retrofunctor $K_S \rightarrow \mathcal{C}$ includes a bijection on objects, so we can assume that $\text{Ob}(\mathcal{C}) = S$. But the rest of the retrofunctor assigns to each map $s_1 \rightarrow s_2$ in $\mathcal{C}$ some choice of morphism $s_1 \rightarrow s_2$ in K_S , and there is exactly one. Thus the retrofunctor includes no additional data.

Solution to Exercise 8.71.

Let's take the two nonidentity arrows in $\mathcal{C}$ to be labeled s, t and the unique nonidentity arrow in $\mathcal{D}$ to be labeled f . Then the carrier of $\mathcal{C}$ is $\mathfrak{c} := y^{\{\text{id}_1, s, t\}} + y^{\{\text{id}_2\}}$ and that of $\mathcal{D}$ is $\mathfrak{d} := y^{\{\text{id}_1, f\}} + y^{\{\text{id}_2\}}$.

1. The boo functor $F: \mathcal{C} \rightarrow \mathcal{D}$ can be identified with a vertical retrofunctor $\mathfrak{d} \rightarrow \mathfrak{c}$ that sends back $\text{id}_1 \mapsto \text{id}_1$ and $s \mapsto f$ and $t \mapsto f$ and $\text{id}_2 \mapsto \text{id}_2$.
2. For $G_1: \mathcal{D} \rightarrow \mathcal{C}$, which we take to send $f \mapsto s$, the corresponding vertical retrofunctor $\mathfrak{c} \rightarrow \mathfrak{d}$ sends back $\text{id}_1 \mapsto \text{id}_1$ and $f \mapsto s$ and $\text{id}_2 \mapsto \text{id}_2$.

Solution to Exercise 8.76.

1. We actually already showed that 0 has a unique comonoid structure, corresponding to the empty category (which we will also denote by 0), in Exercise 7.39, for the case of $S := 0$.
2. We show that 0 has the universal property of the initial object in $\mathbf{Cat}^\sharp$ as follows. For any category $\mathcal{D}$, there is a unique retrofunctor $0 \rightarrow \mathcal{D}$: it vacuously sends each object in 0 to an object in $\mathcal{D}$, and since there are no objects in 0 , it does nothing to morphisms.
Alternatively, we know by Corollary 8.74 that $\mathbf{Cat}^\sharp$ has an initial object, and that the forgetful functor $\mathbf{Cat}^\sharp \rightarrow \mathbf{Poly}$ sends it to the initial object in $\mathbf{Poly}$, which is 0 (by Proposition 3.3 and the following discussion). So the initial object in $\mathbf{Cat}^\sharp$ must be the unique comonoid carried by 0 .

Solution to Exercise 8.77.

The coproduct of $\mathfrak{c}$ with itself is again a comonoid (as a category it is the disjoint union of $\mathfrak{c}$ with itself) and it has the right carrier polynomial, since the carrier functor $U: \mathbf{Cat}^\sharp \rightarrow \mathbf{Poly}$ is a left adjoint.

Solution to Exercise 8.80.

1. The highbrow way to check that $c \otimes d$ forms a comonoid is to say that

$$\otimes: (\mathbf{Poly}, y, \triangleleft) \times (\mathbf{Poly}, y, \triangleleft) \longrightarrow (\mathbf{Poly}, y, \triangleleft)$$

is a colax monoidal functor, so it sends comonoids to comonoids. A lowbrow way to check it is to see that the category corresponding to $c \otimes d$ is just the usual cartesian product $\mathcal{C} \times \mathcal{D}$ in **Cat**. This is a category, hence a comonoid in **Poly**.

2. Because $\otimes$ is symmetric, it suffices to show that for any retrofunctor $\varphi: \mathcal{C} \rightarrow \mathcal{C}'$, there is an induced retrofunctor $\varphi \otimes \mathcal{D}: \mathcal{C} \otimes \mathcal{D} \rightarrow \mathcal{C}' \otimes \mathcal{D}$.

Solution to Exercise 8.82.

1. The carrier of $\mathcal{T}_y$ is $t_y := y^{\mathbb{N}}$.
2. The lens $y \rightarrow t_y$ is the unique lens.
3. Given a p -tree T and a q -tree U , we get a $(p \otimes q)$ -tree by having its root be labeled by the pair of root labels for T and U , and for each child there—the set of which is the set of pairs consisting of a child in T and a child in U —recursively using the above formula to combine these two children.

Solution to Exercise 8.85.

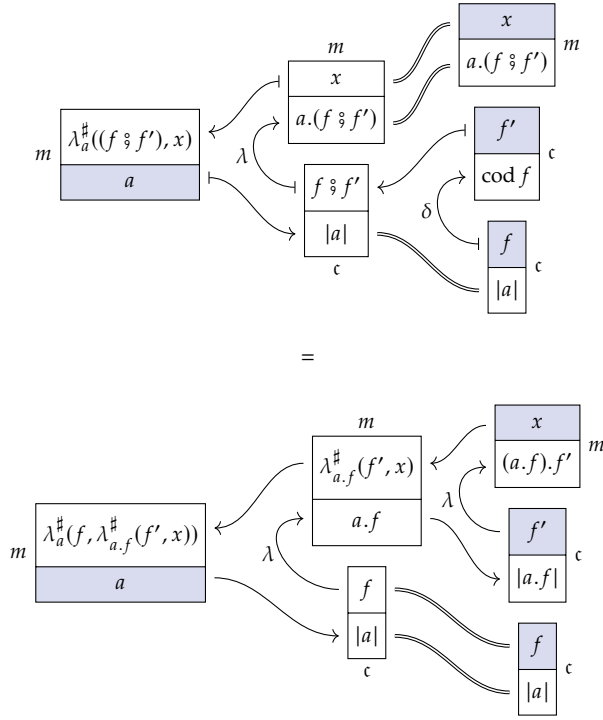
In Definition 8.83, if the carrier of the left $\mathcal{C}$ -comodule is always chosen to be a constant polynomial, i.e. a set, then we recover Definition 7.96. Hence constant left $\mathcal{C}$ -comodules are precisely left $\mathcal{C}$ -coalgebras; their notions of morphisms coincide as well, so the categories they form are isomorphic.

Solution to Exercise 8.88.

1. Here is the first equation in polyboxes:

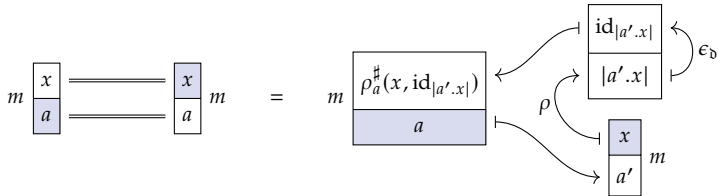
The equation says that for any $a \in m(1)$ we have $a = a.\text{id}_{|a|} := \lambda(a, \text{id}_{|a|})$ and that for any $x \in m[a]$ we have $x = x' := \lambda_a^{\#}(\text{id}_{|a|}, x)$.

Here is the second equation in polyboxes:



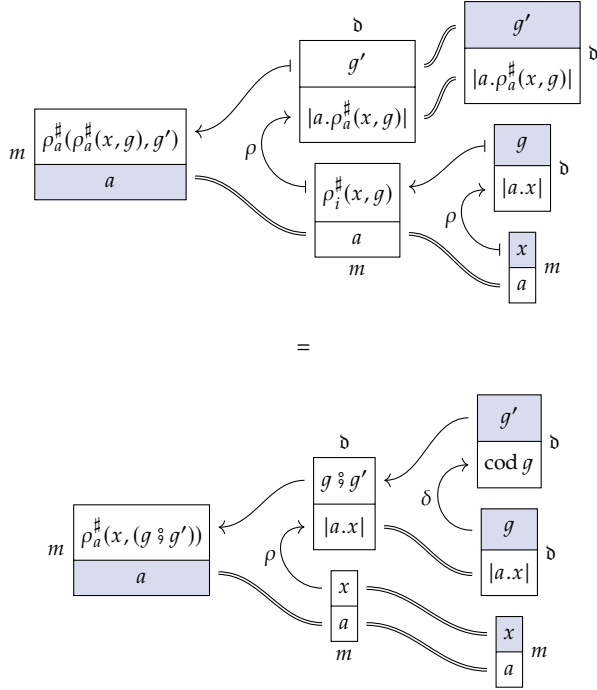
These say that $\text{cod}(f) = |a.f|$, that $a.(f \circ f') = (a.f).f'$, and that $\lambda_a^\#((f \circ f'), x) = \lambda_a^\#(f, \lambda_{a.f}^\#(f', x))$.

2. Here is the first equation in polyboxes:



The equation says that for any $a \in m(1)$ we have $a = a' := \rho_1(i)$ and that for any $f \in m[a]$ we have $x = \rho_a^\#(x, \text{id}_{|a'.x|})$.

Here is the second equation in polyboxes:



These equations say that $\text{cod}(g) = |a \cdot \rho_a^\#(x, g)|$ and that $\rho_a^\#(\rho_a^\#(x, g), g') = \rho_a^\#(x, (g \circ g'))$.

Solution to Exercise 8.89.

The comonoid structure on y has $\epsilon: y \rightarrow y$ and $\delta: y \rightarrow y \triangleleft y = y$ the unique lenses.

1. We first show that for any polynomial m , there is a unique y -comodule structure on m . By (8.84), a y -comodule structure is a lens $\lambda: m \rightarrow y \triangleleft m = m$ such that $\lambda \circ (! \triangleleft m) = \text{id}_m$, but $! \triangleleft m = \text{id}_m$, so $\lambda = \text{id}_m$ is forced. A morphism $(m, \text{id}) \rightarrow (n, \text{id}_n)$ of y -comodules is also easily seen to be just a lens $m \rightarrow n$.
2. Similar.

Solution to Exercise 8.99.

1. By Exercise 8.89, it suffices to check that the diagram (8.97) commutes vacuously when $\mathfrak{d} = y$.
2. By Exercise 8.89, it suffices to check that the diagram (8.97) commutes vacuously when $\mathfrak{c} = y$.
3. This follows from Exercise 8.89.
4. This follows from Exercise 8.89.

Solution to Exercise 8.104.

For each $v \in V$, let $A_v := \text{src}^{-1}(v)$, the set of arrows emanating from v .

1. We need a lens $\sum_{v \in V} y^{A_v} \rightarrow V \sum_{v \in V} y^{A_v}$, and we use the obvious “diagonal” lens, which on positions sends $v \mapsto (v, v)$ and on directions is the identity.
2. We need a lens $\sum_{v \in V} y^{A_v} \rightarrow \sum_{v \in V} \prod_{a \in A_v} \sum_{v' \in V} y$, which is the same as an element of

$$\prod_{v \in V} \sum_{v' \in V} \prod_{a \in A_{v'}} \sum_{v'' \in V} A_v$$

and we use $v \mapsto (v, a \mapsto \text{tgt}(a), a)$. Most of this is forced on us, and the only interesting part is the function $A_v \rightarrow V$, which we can take to be anything, the choice being exactly the choice of “target map” that defines our graph.

3. Let $T = 2V$ and take the map $2V \rightarrow V \times 2V$ to be $(i, v) \mapsto (v, i, v)$ for any $i \in 2$ and $v \in V$.

Solution to Exercise 8.107.

Recall that the $(Vy, 0)$ -bicomodule T assigns to each vertex $v \in V$ some set T_v of “colors”, whereas the $(Vy, 0)$ -bicomodule V assigns each vertex a singleton set. A map $V \rightarrow T$ between them chooses one color for each vertex, which we’re calling the “starting color”.

References

- [AAG05] Michael Abbott, Thorsten Altenkirch, and Neil Ghani. “Containers: Constructing strictly positive types”. In: *Theoretical Computer Science* 342.1 (2005). Applied Semantics: Selected Topics, pp. 3–27 (cit. on p. 46).
- [Abb+03] Michael Abbott, Thorsten Altenkirch, Neil Ghani, and Conor McBride. “Derivatives of containers”. In: *International Conference on Typed Lambda Calculi and Applications*. Springer, 2003, pp. 16–30 (cit. on p. 64).
- [Abb03] Michael Gordon Abbott. “Categories of Containers”. PhD thesis. University of Leicester, Aug. 2003 (cit. on p. 46).
- [Abo+16] Faris Abou-Saleh, James Cheney, Jeremy Gibbons, James McKinna, and Perdita Stevens. “Reflections on monadic lenses”. In: *A List of Successes That Can Change the World*. Springer, 2016, pp. 1–31 (cit. on p. 91).
- [Agu97] Marcelo Aguiar. *Internal Categories and Quantum Groups*. Cornell University, 1997 (cit. on pp. 341, 361).
- [ALS10] Thorsten Altenkirch, Paul Levy, and Sam Staton. “Higher-Order Containers”. In: *Springer Nature*, June 2010, pp. 11–20 (cit. on p. 218).
- [AU16] Danel Ahman and Tarmo Uustalu. “Directed Containers as Categories”. In: *EPTCS 207, 2016*, pp. 89–98 (2016). eprint: [arXiv:1604.01187](#) (cit. on pp. xviii, 361).
- [BL20] Michael Batanin and Florian De Leger. *Polynomial monads and delooping of mapping spaces*. 2020. eprint: [arXiv:1712.00904](#) (cit. on p. xvii).
- [Bor94] Francis Borceux. *Handbook of categorical algebra 1*. Vol. 50. Encyclopedia of Mathematics and Its Applications. Basic category theory. Cambridge University Press, 1994 (cit. on p. 22).

- [BPS19] Erwan Beurier, Dominique Pastor, and David I Spivak. “Memoryless systems generate the class of all discrete systems”. In: *International Journal of Mathematics and Mathematical Sciences* 2019 (2019) (cit. on p. 166).
- [BPV06] Aaron Bohannon, Benjamin C Pierce, and Jeffrey A Vaughan. “Relational lenses: A language for updatable views”. In: *Proceedings of the twenty-fifth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. ACM. 2006, pp. 338–347 (cit. on p. 91).
- [Cap22] Matteo Capucci. *Diegetic representation of feedback in open games*. 2022. eprint: [arXiv:2206.12338](https://arxiv.org/abs/2206.12338) (cit. on p. 219).
- [Che22] Eugenia Cheng. *The Joy of Abstraction: An Exploration of Math, Category Theory, and Life*. Cambridge University Press, 2022 (cit. on p. 22).
- [CJ95] Aurelio Carboni and Peter Johnstone. “Connected limits, familial representability and Artin glueing”. In: *Mathematical Structures in Computer Science* 5.4 (1995), pp. 441–459 (cit. on p. 45).
- [Con12] John Horton Conway. *Regular algebra and finite machines*. Courier Corporation, 2012 (cit. on p. 166).
- [Dwy80] W. G. Dwyer. “Twisted Homological Stability for General Linear Groups”. In: *Annals of Mathematics* 111.2 (1980), pp. 239–251. ISSN: 0003486X. URL: <http://www.jstor.org/stable/1971200> (cit. on p. 46).
- [EM54] Samuel Eilenberg and Saunders MacLane. “On the Groups $H(\Pi, n)$, II: Methods of Computation”. In: *Annals of Mathematics* 60.1 (1954), pp. 49–139. ISSN: 0003486X, 19398980. URL: <http://www.jstor.org/stable/1969702> (cit. on p. 46).
- [FS19] Brendan Fong and David I. Spivak. *Seven Sketches in Compositionality: An Invitation to Applied Category Theory*. Cambridge University Press, 2019 (cit. on p. 22).

- [Gar19] Richard Garner. *Polynomial comonads and comodules*. Homotopy Type Theory Electronic Seminar Talks. 2019. URL: <https://www.youtube.com/watch?v=tW6HYnqn6eI> (cit. on pp. xviii, 440).
- [GH03] Nicola Gambino and Martin Hyland. “Wellfounded trees and dependent polynomial functors”. In: *International Workshop on Types for Proofs and Programs*. Springer. 2003, pp. 210–225 (cit. on p. 45).
- [GH18] Richard Garner and Tom Hirschowitz. “Shapely monads and analytic functors”. In: *Journal of Logic and Computation* 28.1 (2018), pp. 33–83 (cit. on p. 440).
- [Gir88] Jean-Yves Girard. “Normal functors, power series and λ -calculus”. In: *Annals of Pure and Applied Logic* 37.2 (1988), pp. 129–177. ISSN: 0168-0072 (cit. on p. 45).
- [GJ12] Jeremy Gibbons and Michael Johnson. “Relating algebraic and coalgebraic descriptions of lenses”. In: *Electronic Communications of the EASST* 49 (2012) (cit. on p. 91).
- [GK12] Nicola Gambino and Joachim Kock. “Polynomial functors and polynomial monads”. In: *Mathematical Proceedings of the Cambridge Philosophical Society* 154.1 (Sept. 2012), pp. 153–192 (cit. on pp. 46, 273, 281).
- [Goo90] T Goodwillie. “The differential calculus of homotopy functors”. In: *Proceedings of the International Congress of Mathematicians*. Vol. 1. 1990, pp. 621–630 (cit. on p. 46).
- [Hed+16] Jules Hedges, Evguenia Shprits, Viktor Winschel, and Philipp Zahn. *Compositionality and string diagrams for game theory*. 2016. eprint: [arXiv:1604.06061](https://arxiv.org/abs/1604.06061) (cit. on p. 91).
- [Hed17] Jules Hedges. *Coherence for lenses and open games*. 2017. eprint: [arXiv:1704.02230](https://arxiv.org/abs/1704.02230) (cit. on p. 91).
- [Hed18a] Jules Hedges. *Limits of bimorphic lenses*. 2018. eprint: [arXiv:1808.05545](https://arxiv.org/abs/1808.05545) (cit. on pp. 71, 91).
- [Hed18b] Jules Hedges. “Morphisms of open games”. In: *Electronic Notes in Theoretical Computer Science* 341 (2018), pp. 151–177 (cit. on p. 91).

- [HS00] Peter Hancock and Anton Setzer. “Interactive programs in dependent type theory”. In: *Computer Science Logic: 14th International Workshop, CSL 2000 Annual Conference of the EACSL Proceedings 14*. Springer. 2000, pp. 317–331 (cit. on p. xvi).
- [Hyv09] Pierre Hyvernât. *Interaction systems and linear Logic, a different games semantics*. 2009. eprint: arXiv:0905.4062 (cit. on p. 165).
- [Jac17] Bart Jacobs. *Introduction to Coalgebra*. Vol. 59. Cambridge University Press, 2017 (cit. on pp. 45, 271).
- [Joy81] André Joyal. “Une théorie combinatoire des séries formelles”. In: *Advances in Mathematics* 42.1 (1981), pp. 1–82 (cit. on p. 46).
- [JRW12] Michael Johnson, Robert Rosebrugh, and Richard J Wood. “Lenses, fibrations and universal translations”. In: *Mathematical Structures in Computer Science* 22.1 (2012), pp. 25–42 (cit. on p. 91).
- [Kel74] G Max Kelly. “Doctrinal adjunction”. In: *Category Seminar*. Springer. 1974, pp. 257–280 (cit. on p. 426).
- [Koc] Joachim Kock. “Notes on Polynomial Functors”. <https://mat.uab.cat/~kock/cat/polynomial.pdf>. Accessed 2023/12/04 (cit. on p. 46).
- [Lei14] Tom Leinster. *Basic Category Theory*. Vol. 143. Cambridge University Press, 2014 (cit. on p. 22).
- [Lur09] Jacob Lurie. *(Infinity,2)-Categories and the Goodwillie Calculus I*. 2009. arXiv: 0905.0462 [math.CT]. URL: <https://arxiv.org/abs/0905.0462> (cit. on p. 46).
- [MA12] Ernest G Manes and Michael A Arbib. *Algebraic Approaches to Program Semantics*. Springer Science & Business Media, 2012 (cit. on p. 45).
- [Mac80] Ian G Macdonald. “Polynomial functors and wreath products”. In: *Journal of Pure and Applied Algebra* 18.2 (1980), pp. 173–204 (cit. on p. 46).

- [Mac98] Saunders Mac Lane. *Categories for the Working Mathematician*. 2nd ed. Graduate Texts in Mathematics 5. Springer-Verlag, 1998 (cit. on p. 22).
- [McB01] Conor McBride. “The derivative of a regular type is its type of one-hole contexts”. In: *Unpublished Manuscript* (2001), pp. 74–88 (cit. on p. 64).
- [McB09] Conor McBride. “Let’s see how things unfold: Reconciling the infinite with the intensional”. In: *International Conference on Algebra and Coalgebra in Computer Science*. Springer, 2009, pp. 113–126.
- [MM92] Saunders MacLane and Ieke Moerdijk. *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*. Springer, 1992 (cit. on p. 19).
- [MP00] Ieke Moerdijk and Erik Palmgren. “Wellfounded trees in categories”. In: *Annals of Pure and Applied Logic* 104.1-3 (2000), pp. 189–218 (cit. on p. 45).
- [Mye22] David Jaz Myers. *Categorical Systems Theory*. 2022 (cit. on p. xviii).
- [nLa18] Contributors To nLab. *Created limit — nLab*. 2018. URL: <https://ncatlab.org/nlab/show/created+limit> (cit. on p. 423).
- [nLa19] Contributors To nLab. *Connected limit — nLab*. 2019. URL: <https://ncatlab.org/nlab/show/connected+limit> (cit. on p. 276).
- [nLa22] Contributors To nLab. *lens (in computer science)*. <http://ncatlab.org/nlab/show/lens%20%28in%20computer%20science%29>. Revision 26. Aug. 2022 (cit. on p. 361).
- [NS22] Nelson Niu and David I. Spivak. *Collectives: Compositional protocols for contributions and returns*. 2022. arXiv: 2112 . 11518 (cit. on p. 88).
- [OCo11] Russell O’Connor. *Functor is to lens as applicative is to biplate: Introducing multiplate*. 2011. eprint: arXiv:1103.2841 (cit. on p. 91).
- [Par23] Robert Paré. *Retrocells*. 2023. eprint: arXiv:2306.06436 (cit. on pp. 331, 335).

- [Par69] Robert Paré. “Absolute coequalizers”. In: *Category Theory, Homology Theory and Their Applications I*. Ed. by Peter J. Hilton. Springer, 1969, pp. 132–145 (cit. on p. 423).
- [Pie91] Benjamin C. Pierce. *Basic Category Theory for Computer Scientists*. MIT Press, 1991 (cit. on p. 22).
- [Por19] Hans-E Porst. “Colimits of monoids”. In: *Theory and Applications of Categories* 34.17 (2019), pp. 456–467 (cit. on pp. 423, 425).
- [Rie17] Emily Riehl. *Category Theory in Context*. Courier Dover Publications, 2017 (cit. on pp. 11, 22).
- [Sch27] Issai Schur. “Über die rationalen Darstellungen der allgemeinen linearen Gruppe”. In: *Ges. Abhandlungen* 3 (1927), pp. 68–85 (cit. on p. 46).
- [SGF25] David I Spivak, Richard Garner, and Aaron David Fairbanks. “Functorial aggregation”. In: *Journal of Pure and Applied Algebra* (2025), p. 107883 (cit. on p. xvi).
- [Sha21] Brandon Shapiro. *Familial Monads as Higher Category Theories*. 2021. eprint: [arXiv:2111.14796](https://arxiv.org/abs/2111.14796) (cit. on p. 440).
- [Shu08] Michael Shulman. “Framed bicategories and monoidal fibrations”. In: *Theory and Applications of Categories* 20 (2008), Paper No. 18, 650–738 (cit. on pp. 209, 213).
- [Spi12] David I. Spivak. “Functorial data migration”. In: *Information and Computation* 217 (2012), pp. 31–51 (cit. on p. 357).
- [Spi19] David I. Spivak. *Generalized Lens Categories via functors $\mathbf{C}^{\text{op}} \rightarrow \mathbf{Cat}$* . 2019. eprint: [arXiv:1908.02202](https://arxiv.org/abs/1908.02202) (cit. on p. 91).
- [Spi22] David I. Spivak. *Polynomial functors and Shannon entropy*. 2022. eprint: [arXiv:2201.12878](https://arxiv.org/abs/2201.12878) (cit. on p. 219).
- [ST17] David I Spivak and Joshua Tan. “Nesting of dynamical systems and mode-dependent networks”. In: *Journal of Complex Networks* 5.3 (2017), pp. 389–408 (cit. on p. 166).
- [SW15] David I. Spivak and Ryan Wisnesky. “Relational Foundations for Functorial Data Migration”. In: *Proceedings of the 15th Symposium on Database Programming Languages*. DBPL. Pittsburgh, PA: ACM, 2015, pp. 21–28 (cit. on p. 436).

- [Web07] Mark Weber. “Familial 2-Functors and Parametric Right Adjoints”. In: *Theory and Applications of Categories* 18 (2007), Paper No. 22, 665–732 (cit. on p. 440).
- [Wik24] Contributors To Wikipedia. *Polynomial functor* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 03-April-2025]. 2024.
URL: [5Curl%7Bhttps://en.wikipedia.org/wiki/Polynomial_functor%7D](https://en.wikipedia.org/wiki/Polynomial_functor) (cit. on p. 46).

Index

- *-bifibration, 209
- 2-out-of-3, 204
- adjunction
 - between **Poly** and **Cat**[#], 408
 - between **Set** and **Cat**[#], 344
 - between **Set** and **Poly**, 186
 - between **Set**^{op} and **Poly**, 188
 - two-variable, 187
- affine polynomial, 34, 88
- Ahman-Uustalu theorem, 314
- applied category theory, xiii
- arrow field, 341, 417
 - monoid of arrow fields, 417
- arrow fields
 - as adjoint, 417
- associativity, 5, 264, 303, 304, 328, 362, 430
- bicomodule, 431
 - as parametric right adjoint, 436
 - dynamical systems and bicomodules, 437
 - morphism of, 432
- bundle, 69
 - vector, 343
- Cartesian closed category, 196
- cartesian product, *see* [product](#)
- category, *see also* [polynomial comonoid](#)
 - codiscrete, 325
 - cofree comonoids as, 393
 - connected, 272
 - copresheaf, 436
 - degree of an object, 331
 - discrete, 7, 22, 53, 290, 338, 345, 365, 368, 370
 - free, 360
 - of *p*-trees, *see* [cofree comonoid](#)
 - of bicomodules, 436
 - of coalgebras, 385
 - of elements, 200, 358, 359
 - of polynomial functors, 51–104
 - polynomial carrier of, 315
 - slice, 273
- Cat**[#], 335
 - categorical properties of, 377–440
 - isomorphisms in, 336
- cellular automata, 152, 154, 437
 - running, 438
- Chu space, 165
- clock, 190
 - tick, 250
- closed monoidal structure, 160, 194
- coalgebra, 354, 357, 377, 428, 433
 - as equivalent to other notions, 360
 - as retrofunctor from state category, 353
 - carrier of, 354
 - category of, 385
 - community, 46
 - dynamical systems as, 271
 - morphism of, 354
 - morphism of coalgebras, 385
 - of polynomial comonad, 346
 - terminal, 384, 385
- coassociativity, 302, 304, 305, 308–310, 317, 321, 322, 363, 366, 401
- cofree comonoid, 377–418
 - adjunction with **Poly**, 408
 - and Moore machines, 414
 - as category, 393
 - as free category on a graph, 416
 - carrier of, 378–393

- directions of, 386
- duplicator of, 394
- eraser of, 393
- examples of, 403–408
- interfaces for, 411
- morphisms between, 416
- positions of, 385
- preserves cartesian maps, 422
- colimit, 10, 11, 386
 - Poly** disagrees with **Set^{Set}**, 201
 - in **Cat[#]**, 423
 - in **Poly**, 71, 199
 - of functors, 18
 - of functors **Set** → **Set**, 19
- comodule, 426–439
 - bicomodule, *see* bicomodule
 - left, 427
 - morphism of, 427
 - right, 428
- comonad, *see* polynomial comonoid
- comonoid
 - cofree, *see* cofree comonoid
 - definition, 307
 - morphism of, 332
- comonoids, *see also* **Cat[#]**
- completely distributive category, 16, 20, 83
 - Poly** as, 17
 - Set** as, 17
 - Set^{Set}** as, 20
 - distributive law in, *see* distributive law
- composition product, 233–293
 - n*-fold, 239
 - as composition of functors, 234
 - as dependency, 239
 - cartesian lenses and, 279
 - coclosure, 268–271
 - connected limits and, 272, 275
 - formula for, 235
 - grafting corollas, 241
 - involving constant polynomials, 243
 - limits and, 268, 271
 - of special polynomials, 236
 - on lenses, 236–238, 240
 - on morphisms, *see* composition product, on lenses
 - positions and directions, 238
 - positions as strategies, 239
 - tree picture of, 244
 - unit of, 235
 - vertical lenses and, 280
- compositional game theory, 91
- connected limit, *see* limit, connected
- constant polynomial, 33, 34, 44, 45, 282, 382, 404, 435
 - as carrying bicomodule, 428
- control, 145, 148, 150, 341
- copresheaf, 433
 - and discrete opfibration, 358
 - as bicomodule, 436
 - as retrofunctor, 357
 - on cofree category, 415
- coproduct, xvii, 10, 16, 19, 22, 52, 82
 - composition and, 265
 - distributing over, 89
 - in **Poly**, 53
 - inclusion map into, 10
 - indexed, 188
 - of functions, 11
 - of indexed family, 52
 - of polynomial comonoids, 423
 - universal property of, 10, 54
- corolla, 38
 - representable as, 39
- corolla forest, 37–41, 45
 - corolla of, 38
 - for product of polynomials, 83
 - leaf of, 38
- database, 91, 358
 - data migration, 436
- Day convolution, 87–89
- decision
 - impossible, 36
 - multi-step, 254
- decision tree, 38, 250, 414
- dependent function, 8, 9, 12, 22, 132, 153
 - lens to *y* as, 68
- dependent lens, *see* lens

- dependent Moore machine, *see* dynamical system
- derivative, 65
 - $\dot{p}(1)$ as directions of p , 65
 - not well-behaved in **Poly**, 65
- deterministic state automaton, 112
 - halting, 116, 412
 - language of, 113, 117, 413
- diagram, 10
- dialectica category, xiii
- directed limit, 272
- directions, *see* polynomial functor, positions and directions
- Dirichlet product, *see* parallel product
- Dirichlet series, 86
- discrete opfibration, 433
 - as cartesian retrofunctor, 421
 - as copresheaf, 359
 - as retrofunctor, 356, 357
 - category of, 357
 - morphism of, 357
- disjunctive normal form, 17
- distributive law, 14, 17
 - coproducts over composition, 265, 267
 - for Day monoidal structures, 88
 - for parallel product, 88
 - products over composition, 265, 266
- duoidality, 278, 425
- duplication
 - multifold, 309
- duplicator, *see* polynomial comonoid, comultiplication of
- dynamical system, 114–122, 247
 - as bicomodule, 437
 - as retrofunctor, 345
 - composite interface, 256
 - composition product and, 247
 - constructing from product, 123
 - constructing new from old, 122–136
 - juxtaposing, 125
 - paddler, 139
 - repeater, 137
 - running, 305
 - speeding up, *see* dynamical system, composition product and
 - stepping through, 248
 - trees, 250
 - wrapper interface, *see* interface, wrapper
- dynamics, 123, 132, 139, 141, 150, 153, 164
- element
 - category of elements, 200
 - of a dependent product, 13
 - of a dependent sum, 13
 - of a nested dependent set, 13
 - of a product of sets, 8
 - of a sum of sets, 8
- elements
 - category of elements, 358
- environment, 164
 - universal, 164
- equality
 - use of, xvii
- equalizer, 272, 425
 - as connected limit, 272
- eraser, *see* polynomial comonoid, counit of
- event-based systems, 125
- exponentiable lens, *see* lens, cartesian
- exponential, 194–196, *see also* closed monoidal structure
- factorization system
 - epi-mono, 189, 194
 - vertical cartesian, 420
 - vertical-cartesian, 203–209
- familial functor, 46
- file reader, *see* Moore machine, file reader
- file searcher, *see* Moore machine, file searcher
- forgetful functor, *see also* functor, forgetful
- forgetful-cofree adjunction, 408
- functional programming
 - lenses in, 51, 91, 347

functor

- arrow fields, 418
- bijection on objects (boo), 422
- category of functors, xvii, 19
- coalgebra for, 353, 384
- cofree comonad, 416
- colimit of functors, 18
- comonadic, 423
- constant, 4, 20
- copresheaf, *see* copresheaf
- discrete opfibration, *see* discrete opfibration
- forgetful, 409, 418
- forgetful $\mathbf{Cat}^\# \rightarrow \mathbf{Poly}$, 377, 378, 409, 423–425
- identity, 4
- initial, 20
- limit of functors, 18
- limit-preserving, 395
- of sections, Γ , 71
- polynomial, *see* polynomial functor
- product of functors, 7
- representable, 3–7, 22
- set-valued, 3, 10, 18, 358, 359
- sum of functors, 7
- terminal, 20

future

- as placeholder for dependency, 239, 240, 242

Garner, Richard, 436

global sections, 188, 189, 201, 206, 221, 228

graph, 118, 152–154, 200, 375

- as dynamical system, 118
- category free on, 360
- cellular automaton on, 438
- complete, 299, 326
- free category on, 416, 417
- underlying a category, 326

indexed family, 277

indexed family of objects, 8

indexed set, 7–9, 401

indexing category

for (co)limit, 272

interaction, 136–160, 258, 305

- assembling, 159
- breaking bonds, 156
- cellular automata and, 153
- mode dependent, 166
- multi-step, 250
- picking up the chalk, 142–145, 147
- supplier change, 158
- wiring diagram, 147–154

interaction pattern, *see* interaction, 156, 164, 278

interdisciplinary, xiv

interface, 108, 115, 247, 250, 265, 271, 304, 305, 377, 411

changing, 233

closed, 142

composite, 256, 257

monomial, 106–114, 147, 148, 179, 414

multiple, 123

polynomial, 114

wrapper, 120, 129–142

isomorphism, xvii

- natural, 15–17, 21, 52, 54, 162, 183, 187–189, 195, 221, 223, 244, 265, 268, 270, 271, 273, 276, 277, 409, 411, 420, 435

of representable functors, 36

Joyal, André, 46

Kock, Joachim, 46

left Kan extension, *see* composition product, coclosure

lens, xiii, 51–82

- as interaction protocol, 55–57
- as natural transformation, 73–76
- between monomials, 71, 90
- bidirectionality of, 55, 57
- bimorphic, *see* lens, between monomials
- cartesian, 203, 216, 279
- composition of, 77

- corolla forest depiction, 57
- enumeration, 62
- epimorphism, 190
- exponentiable, 216
- from constant polynomial, 67
- from linear polynomial, 66
- from representable polynomial, 67
- identity, 76
- laws, 73
- monomorphism, 189
- polybox depiction, 59–61
- positions and directions, 54
- spreadsheet depiction, 59, 79
- to y as section, 69
- to composite, 252
- to constant polynomial, 67
- to linear polynomial, 66
- to representable polynomial, 67
- transition, 296, 302
- vertical, 203, 279
- very well-behaved, 72, *see also* retrofunctor, between state categories
- limit, 11
 - cofree comonoid as, 378
 - connected, 272, 395
 - in **Poly**, 196
 - of functors, 18
 - of functors **Set** $\rightarrow$ **Set**, 19
 - positions and directions, 197
 - preserved by $\triangleleft$, 395
- limits
 - computed pointwise, 199
- linear polynomial, 33, 34, 45, 88
 - comonoid structure on, 313
- list, 113, 171
 - cyclic, 328
- long division, 151
- monoid, 327–329
 - action of, 328
 - for parallel product, 88
 - multiplication of, 308
 - of natural numbers, 317
 - structure on any nonempty set, 326
 - unit of, 308
- monoidal closed structure, *see* closed monoidal structure
- monoidal structure, 82
 - cartesian (products), 82
 - cocartesian (sums), 82
 - corolla forest depiction, 86
 - from monoidal structure on **Set**, 88
 - parallel, *see* Parallel product
 - parallel product as pushout, 202
 - preservation of vertical and cartesian maps, 205
 - substitution, *see* composition product
- monomial, 34, 45, 88
 - principal, 189, 219
- Moore machine, 105–114, 124, 127
 - dependent, *see* dynamical system, dependent
 - deterministic state automaton, *see* deterministic state automaton
 - file reader, 111, 120
 - file searcher, 120
 - induced by function, 110
 - interface of, *see also* interface memoryless, 110
 - return, 105
 - states of, 105
 - tape of Turing machine, 111
 - transition diagram, 106
 - update, 105
- multivariate polynomial, *see* polynomial functor, multivariate
- natural numbers, $\mathbb{N}$, xvii
- natural transformation
 - and Yoneda embedding, 6
 - as copresheaf morphism, 358
 - between polynomials, *see* lens
 - between representables, 4
 - horizontal composition, 237, 238

- positions map as 1-component, 244
- vertical composition, 237
- parallel product, 85–90
 - and dynamical systems, 125–129
 - and interaction, 136
 - and the cofree construction, 426
 - as pushout, 202
 - closure for, 160
 - of polynomial comonoids, 425
 - of special polynomials, 87
 - section for, 142
- parametric right adjoint, 436
- polybox, 41–45, 252–265, 268, 273, 277, 297, 299, 303, 313, 320–322, 355, 356, 393, 432
 - direction box, 42
 - for constant polynomials, 44
 - for lens composition, 79–82
 - for linear polynomials, 44
 - for representable polynomials, 44
 - maps to composition product, 252
 - position box, 42
 - spreadsheet, 42
- polynomial comonad, *see* [polynomial comonoid](#)
- polynomial comonoid, 295–440
 - as category, 314–322, 331
 - coalgebra of, 346
 - colimit of comonoids, 423
 - colimits of, 423–424
 - comultiplication of, 308
 - coproduct of comonoids, 423
 - counit of, 308
 - examples, 323
 - linear, 419
 - morphism of, *see* [retrofunctor](#)
 - parallel product of, 425
 - positions and directions of, 314
 - preorders as, 323–324
 - representable, 327, 419
- polynomial functor, 3, 7
 - $p(1)$ as summands of, 31
 - action on functions, 32
 - action on sets, 31
 - affine, *see* [affine polynomial](#)
 - application as composition, 243
 - as sum of representables, 29
 - associated corolla forest, 37
 - cartesian closed structure, 194
 - category of polynomials, 51
 - colimits of polynomials, 199–203
 - composition of polynomials, 233
 - constant, *see* [constant polynomial](#)
 - copointed, 125
 - coproduct of polynomials, *see* [polynomial functors, sum of polynomials](#)
 - derivative of, *see* [derivative](#)
 - equalizer of polynomials, 199
 - exponentials of polynomials, 194
 - full subcategory of $\mathbf{Set}^{\mathbf{Set}}$, 51
 - limits of polynomials, 196–199
 - linear, *see* [linear polynomial](#)
 - monoidal structure on, *see* [monoidal structure](#)
 - monomial, *see* [monomial](#)
 - morphism of, *see* [lens](#)
 - multivariate, xvi
 - positions and directions, 32, 35–37
 - positions as $p(1)$, 244
 - product of polynomials, 7, *see* [polynomial functors, product of polynomials](#), 82, 83
 - pullback of polynomials, 198, 202, 205, 206, 225
 - pushout of polynomials, 202
 - representable, *see* [representable polynomial](#)
 - sum of polynomials, 7, 63, 82
 - summand of, 30
 - term usage by “coalgebra community”, 46
- positions, *see* [polynomial functor, positions and directions](#)
- principal monomial, *see* [monomial, principal](#)

- product, xvii, *see also* [polynomial functor](#), product of
 - [polynomials](#), 9, 10, 16, 22, 168
 - of functions, 11
 - projection map out of, 10
 - universal property of, 10
- pullback, *see also* [polynomial functor](#),
 - [pullback of polynomials](#), 272
 - as connected limit, 272
 - cartesian natural transformation
 - and, 206
 - wide, 276
- pure power, *see* [functor](#), [representable](#)
- representable polynomial, 33, 34, 45
- retrofunctor, 331
 - Aguilar's definition, 361
 - arrow field, 343
 - between state categories, 347
 - cartesian, 420
 - examples of, 337–353
 - from state category, 433
 - isomorphism, 336
 - similarity with functor, 335
 - to discrete categories, 338
 - to monoids, 341
 - to preorders, 338
 - vertical, 420, 421
- robot, 120–122, 128, 141
- rooted tree, *see* [tree](#), [rooted](#)
 - height of vertex, 37
 - rooted path in, 37
- section, 69–71, 71, 145, 147
 - do nothing, 296
- set
 - as constant polynomial, 33, 186
 - cardinality of, 9
 - category of sets, xvii
 - finite, xvii
 - indexing, 7
 - ordinal, xvii
 - product of sets, 7
 - sum of sets, 7
- state category, *see* [state system](#)
- state system, 108, 115, 247, 295, 302, 355
 - as polynomial comonoid, 309
 - comonoid as, 307
 - corresponding category of, 324
 - retrofunctor from, 346–350, 360
- streams, 406
 - category of, 329
 - cofree comonoid and, 406
- Systems of ODEs, 345
- time
 - composition product and, 233, 250
- topological space, 78
- tree
 - as limit of pretrees, 382
 - corolla and, 37
 - depiction of, 241
 - multi-level, 242
 - on a polynomial, 378
 - pretree, 380
 - rooted, 37
 - stages, 381
- type theoretic axiom of choice, 14
- wiring diagram, *see* [interaction](#), [wiring diagram](#)
- Yoneda embedding, *see* [Yoneda lemma](#)
- Yoneda lemma, 3, 6, 22, 52, 54, 62, 67, 73, 74, 163, 187, 188, 366, 419

